Welcome to
SB+ Solutions!

A Comprehensive Guide to Developing Quality Applications Quickly and Easily

[Chapter 1: Introduction](#)

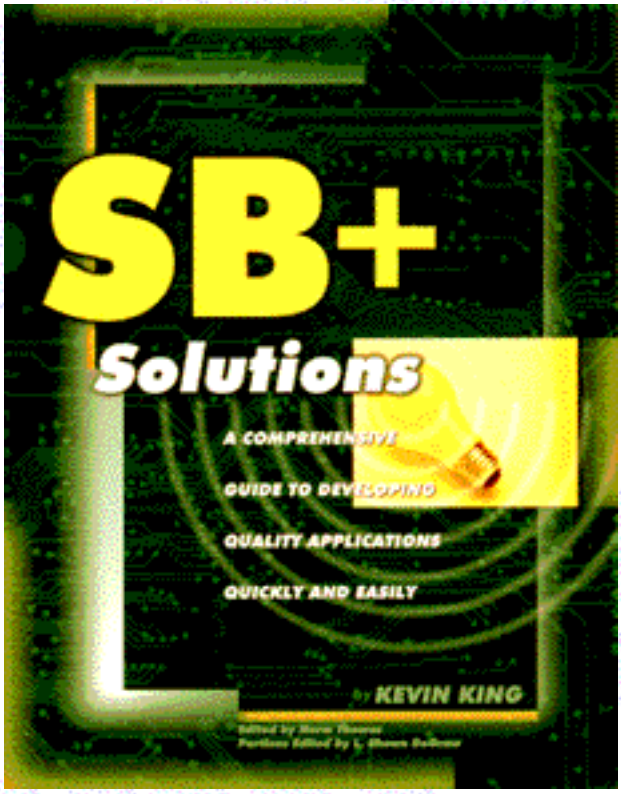
[Chapter 2: SB+ Fundamentals](#)

[Chapter 3: Entry Screen Solutions](#)

[Chapter 4: Query Screen Solutions](#)

[Chapter 5: Periodic Update Solutions](#)

[Chapter 6: Report Writing Solutions](#)



[Chapter 7: Creating Menus](#)

[Chapter 8: Reference Information](#)

[Chapter 9: Generating Code](#)

[Chapter 10: Epilogue](#)

[Appendix A: Development Standards](#)

[Appendix B: GUI Supplement](#)

[Appendix C: Glossary](#)

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher.

Published by Rocket Software
275 Grove Street Suite 3-410
Newton, MA 02466-2272, USA

Web: **http://www.rocketsoftware.com**

Original Text Authored by Kevin King, Precision Solutions, Inc.
Portions Edited by L. Shawn DeGraw, Norm Thomas, and others
Cover Design and Layout by Michelle DeMonnin

Licensed Materials - Property of Rocket Software

Copyright © Rocket Software, Inc. 1996-2010. All Rights Reserved.

Rocket Software, U2, SystemBuilder, UniData, UniVerse, U2 Web Development Environment, and wIntegrate are trademarks or registered trademarks of Rocket Software, Inc. and its subsidiaries.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries.

Windows and Microsoft Office Excel, Outlook, and Word are either registered trademarks or trademarks of Microsoft Corporation in the United States and/or other countries.

UNIX is a registered trademark in the United States and other countries licensed exclusively through X/Open Company Limited.

Other companies, products, or services mentioned may be covered by the trademarks, service marks, or product names as designated by the companies who own or market them.

Note

This product may contain encryption technology. Many countries prohibit or restrict the use, import, or export of encryption technologies, and current use, import, and export regulations should be followed when exporting this product.

Please be aware: Any images or indications reflecting ownership or branding of the product(s) documented herein may or may not reflect the current legal ownership of the intellectual property rights associated with such product(s). All right and title to the product(s) documented herein belong solely to Rocket Software, Inc. and its subsidiaries, notwithstanding any notices (including screen captures) or any other indications to the contrary.

Chapter 1: Introduction

Welcome to SB+!
Speaking the Language
System Administration
Security

Chapter 2: SB+ Fundamentals

The Keyword for SB+ Development: PLAN
Team Development
The SB+ Application Development Cycle
Portability Considerations
Using a Standard Process Library
Understanding the Size of the Application
Getting Started With SB+

Chapter 3: Entry Screen Solutions

Entry Screen Fundamentals	Adding F1 Help to the Screen Definition
Creating An Input Process	Conversions
So What's Really Going On Here?	Derived Values
Moving Prompts On A Screen	Special Processing for Single Valued Fields
Resequencing Prompts on a Screen Definition	Special Processing for Multivalued Fields
Painting Text on a Screen Definition	Process Tricks
Graphics	Structuring Processes
Resizing/Moving the Window	Using Split Dictionary/Data Files
Defining Function Keys/Action Bars	Using Logical Files
Finishing the Screen	Key Processing Techniques
Defaults	Subscreens
Validating an Entry	Linked Screens
Intuitive Help	Miscellaneous

Chapter 4: Query Screen Solutions

Query Screen Fundamentals
Don't Use a Shared Screen Definition
To Prompt or Not to Prompt?
Creating an Output Process
Creating a Non-Amendable Input Process
Cruising, Zooming, and other Stupid Pick Tricks
Special Processing for Query Screens
Miscellaneous

Chapter 5: Periodic Update Solutions

[Introduction To Periodic Updates](#)
[And Another Way to Look At It...](#)
[For BASIC Programmers Only](#)
[Executing a Periodic Update](#)
[Suppressing the "OK to Continue" Message](#)
[Common Types of Periodic Updates](#)
[In Summary](#)

Chapter 6: Report Writing Solutions

SB+ Reporting Options
Creating Query Reports
Creating SB+ ReportWriter Reports
Creating a Report for a Split Dictionary/Data File
Defining Stationery, Location, and Other Printer Specifics for a Report
How Many Process Slots?
In Summary

Chapter 7: Creating Menus

Creating SB+ Menus
The SB+ Main Menu and System Menu
User Menus
Constructing a Startup Menu
Constructing a Process Menu
The "More" Menu
Constructing Complex Intuitive Help Using Menus
Conditionally Quitting a Menu
In Summary

Chapter 8: Reference Information

[The Common Map](#)
[The Expression Language](#)
[Global Equates](#)
[The Paragraph Language](#)
[Using BASIC in an SB+ Application](#)
[Understanding Selection Criteria](#)
[Understanding Edit Keys](#)
[Understanding @RTN.FLAG](#)
[In Summary](#)

Chapter 9: Generating Code

[Why Generate BASIC Code?](#)
[Generating Code](#)
[In Summary](#)

Chapter 10: Epilogue

I'm So Glad We've Had This Time Together...

Appendix A: Development Standards

Saving Time and Money With Standards	Periodic Update Definition Standards
Standard Abbreviations	Process Definition Standards
System Standards	BASIC Standards
File Standards	Expression Standards
Field Definition Standards	Dialog Box Standards
Screen Definition Standards	Code Table Standards
Report Definition Standards	Security Standards
Menu Definition Standards	Common Map Standards
	Miscellaneous Standards

Appendix B: GUI Supplement

Introduction to GUI
The GUI Paradigm
GUI Objects Overview
Designing a Character-Mode Application for GUI
Using the GUI Form Painter
GUI and Reports
GUI Menus

Reference:

GUI Object Properties

Appendix C: Glossary

Access: The query language of the operating environment. In UniData, this is called UniQuery. On Microdata platforms, this is called English.

Action Bar: A horizontal list of options that appears at the top of the screen, typically when F10 is pressed in an entry screen.

Application: A collection of programs.

BASIC: Beginners All-Purpose Symbolic Instruction Code; The procedural language of Pick and Pick-like operating environments. With all the enhancements to BASIC in these environments, it's not so much for "beginners" anymore...

Break Modifier: A code which defines how a particular break field should be presented on a query report.

Columnar Report: A report where the information is presented in columns.

Conditional Field: A field on a report that evaluates a condition and then skips a certain number of lines.

Controlling Multivalue: On an entry screen, this is the multivalued field that "controls" input to dependent multivalues. If nothing is entered into the controlling multivalue, the dependents cannot be accessed.

Default: A value that is displayed when the field otherwise has no value.

Definition: A record which defines information about a process, screen, periodic update, or other SB+ resource.

Dependent Multivalue: A multivalue which can only be accessed if a value is entered in a controlling multivalue.

Drivers: Records created by SB+ for running a screen or report. These are stored in the XXXDRIVERS file and encapsulate much of the information required for running a screen or report.

Edit Key: A key which performs a task, such as up arrow or down arrow. SB+ stores the last edit key number pressed in @OTHER (18).

Equate: A symbolic name for an expression, typically defined specifically for a single paragraph. See "Global Equates".

Expression: A calculation, typically using SB+ functions, operators, and operands.

Externally Definition Function Keys: Function keys that are defined in XXXDEFN, not in the screen itself.

F1 Help: The help messages that are displayed when F1 is pressed on a prompt or menu.

F3 Intuitive Help: A process which is called when F3 is pressed on a prompt to help the user select a value.

Form Report: A report where the information can be shown literally anywhere on the page.

Function Keys: The "F" keys on the keyboard which are used to perform functions in the programs.

Global Equates: Symbolic names for expressions.

Input Area: The area on the screen where the user will enter a value into a prompt.

Input Process: An entry screen or subscreen.

Intuitive Help: The process to be called when F3 is pressed.

Key Field: Any field on a screen with a field position of "0" (zero).

Linking: The procedure whereby one process is tied to another via a "slot".

Local Variable: A variable which exists only during the execution of a paragraph. When the paragraph terminates, the local variables go away. Similarly, no process called from a paragraph has visibility of the local variables, except when a local variable is passed as a parameter to another process.

Mandatory Default: A default expression which is evaluated regardless of whether the field has a value.

Multivalue Industry: Formerly the "Pick industry".

Multivalued Field: A field that can accept multiple entries.

Multivalued Derived Value: A derived value field on a report which has "Suppress Repetition" set to "L" and "Max MV Lines" set to "0.M".

Non-amendable Field: A field that cannot be changed.

Output Process: A query screen.

P-Code: The pseudo-object code that SB+ generates for paragraphs and other processes.

Paragraphs: The SB+ procedural language.

Pick: An operating environment (OE) named after Richard Pick.

Process: The general building blocks of an SB+ application.

Program: A collection of processes which are invoked from a single menu selection.

Prompt: The description of a field as shown on a screen or report.

Read Step: The step in the SB+ input process driver where the main record is read from the file. This typically follows the entry of

the key field.

ReportWriter: The SB+ reporting facility.

Slot: A place where a process may be linked to another process.

Stacked Headings: Headings which appear on multiple rows.

Starting Process: The process which invokes a given program. All programs, regardless of their complexity, must resolve to a starting process.

Table: An SB+ resource which contains a list of codes and corresponding descriptions. Tables are typically used for validation and intuitive help.

Triad: In a **Selection Criteria**, the triad is one *field operator value* phrase.

UniData: Like UniVerse, only different. A multivalue operating environment, which has many Pick-like characteristics, plus a few bells and whistles of its own. Currently owned by Rocket Software.

UniVerse: Like UniData, only different. A multivalue operating environment, which has many Pick-like characteristics, plus a few bells and whistles of its own Currently owned by Rocket Software.

Validation: A code or series of codes used by SB+ to verify the accuracy of an entry.

Wildcard: A character that means "anything". In selection criteria, the character "[" is a wildcard meaning "anything prior". The character "]" is a wildcard meaning "anything following".

Window: The box that is shown around an input or output screen.

Window Co-Ords: Numbers which define the size and shape of the box shown around an input or output screen.

Window Size: The actual number of columns and rows in the box shown around an input or output screen.

Work Field: A field which updates the **common variable** @WORK, rather than @RECORD.

Welcome to SB+!

The SB+ development environment is unquestionably the most flexible and powerful development environment available today for Pick and Pick-like computer systems. With its easy-to-use interface and comprehensive suite of tools, SB+ has proven itself to be an industry leader in the area of rapid application development tools.

Historically, if a person wanted to create a product for the Pick system, a series of BASIC programs and subroutines would be designed, created, tested, documented, and then released to the customer(s). While this method of software development has been effective, it has been and continues to be a very costly endeavor. A significant amount of time must be dedicated in order to efficiently organize all of the components of the product, enforce user interface and development standards, code each part of the product, and test it all thoroughly. In short, this method of development requires the developer to create and organize everything.

Maintenance of BASIC applications can also be a costly endeavor. As years pass and programs mature, different styles and approaches can cause code to become cluttered and difficult to follow. When the time to understand a program as it currently exists eclipses the time required to actually change it, increased maintenance costs are inevitable.

With SB+, much of the product life cycle is unchanged. Design is still paramount to the success of the project. Standards are still effective in assuring quality throughout. However, with SB+'s rapid application development tools, coding, testing, and maintenance time can be dramatically reduced.

At its most fundamental level, SB+ is a comprehensive application development toolbox. Each tool is designed to promote a standard approach to application development, as well as provide a consistent user interface for the finished product. In short, rather than forcing developers to start an application 'from scratch', SB+ provides a wealth of tools and resources for getting the most done with the least amount of effort. As an added bonus, SB+ provides a structured framework for building applications, thus reducing the potential for unnecessary clutter as programs mature.

It is important to note, however, that using SB+ itself will not automatically reduce the expense of your project. At the outset, it may even increase your expense, as there is much to learn in order to use the product effectively. Over time, however, as you become familiar with the various techniques and methods for using the product effectively, a significant savings can be realized.

Therefore, the goal of this book is to provide you a set of techniques that you may draw on for many of the day-to-day challenges you will face in your SB+ development efforts. By demonstrating proven techniques for solving a variety of different problems, your training curve will appear less like a mountain and more like a speed bump.

Speaking the Language

One of the basic elements of understanding SB+ is an understanding of the terminology that is used. The following sections describe the most common new terms that are used in SB+ development.

System

A *system* is a collection of programs, processes, definitions, and other records which comprise a series of programs. In pre-SB+ terminology, this might be known as a *module*. For example, accounting software may have a general ledger system, an accounts payable system, an accounts receivable system, as well as a number of others.

There can be any number of systems in one SB+ account. Each system is autonomous in that the processes and programs in one system are not intrinsically visible outside of that system. Therefore, if the software has process named GL0021 in the GL system and a separate (perhaps different) process by the same name in the AP system, there is no conflict -- each process is unique. Note, however, that there cannot be multiple processes with the same name inside of a single system.

Files and dictionaries, however, supersede systems. If a file named GLTRANS is created in the GL system, the programs and processes in all other systems can see and use the data and field definitions (dictionaries) in that file.

When using SB+, you're always inside of one system or another. Though only one system is active at a time, your programming can jump from system to system without user intervention or even user knowledge.

Processes

In traditional BASIC programming, an application is constructed as a series of programs and subroutines. The structure of an SB+ application is similar, except that the building blocks are called *processes*.

From a BASIC point-of-view, processes are most similar to subroutines, in that they can be called from literally anywhere in the application and can themselves call other processes. Unlike subroutines however, there are a variety of different *types* of processes. For example, a "Report Writer process" can be used to generate a report while an "input process" forms the basic structure for an entry screen. With numerous types of processes to choose from, SB+ provides a wealth of options for building various types of applications.

With all these different types of processes, one might assume that SB+ forces the developer to use certain types of process for certain tasks. Fortunately, this is not the case. Any process can call any other type of process regardless of whether it makes sense or not. It is up to each developer to determine which relationships are beneficial to the application.

Programs

Unlike traditional programming methodologies, one doesn't build *programs* with SB+. Instead, processes are built and linked together to construct an application. Though this may seem a bit like wordplay, it does have significant implications.

Traditional programming methodologies typically define a strict boundary between programs. In short, if a person wants to run one program but is currently running another, they must exit the first program before the second can be started. With SB+, this is no longer necessary. Any process can call any other process, thereby eliminating the need to exit one program to start another. As

a result, an SB+ *program* is simply an option on a menu, and may include features that would traditionally be several different programs.

For example, let's assume you're looking at a customer screen. As you view the customer information, you wonder if this customer has any orders. By pressing a function key you can see a list of orders or the order detail itself. Once you're looking at the order detail, you may wonder where the order is being shipped. Another press of the function key shows you this information. Next, perhaps you wonder which carriers can ship to the state on the order. With just one more key press, the information you need is immediately available.

With this type of flexibility, there is no reason to force someone to exit one program to start another. By making all of your programs intrinsically related, menu selections become merely "windows" into the application as a whole.

Process Slots

Processes are called from within each other via *slots*. Simply stated, a slot is a place in a process where the name of another process can be entered.

Each type of process has a number of slots, and each slot corresponds to a particular step in the execution of the process. For example, the Process After Read in an input process is called immediately after a record has been read. Similarly, the Process After Update in an input process is called after the main record has been updated into the file. By strategically planting these process slots throughout the different process types, SB+ provides a consistent and predictable interface for linking processes together.

Most process slots can be easily identified in a particular type of process by the word "proc" or "process" in the prompt label. However, it is important to note that there are several process slots which are not immediately apparent, such as:

- Default Expression in a Field Definition
- Validation Code in a Field Definition
- Conversion in a Field Definition
- Derived Value in a Field Definition
- Any SB+ Expression
- Inside an SB+ Paragraph
- Derived Value in a Report Writer Definition

Starting Process

No matter how many processes are linked together to form an application, they must eventually resolve to one starting process. A program is started by invoking the starting process.

SB+ makes no distinction between a starting process and any other process. As a result, a process used to start an application from a menu may also be called as a supporting process when several programs are linked together. While incredibly flexible, this can get to be confusing, especially when there are thousands of processes and several different pathways to each program.

Ultimately, it is the responsibility of the developer to ensure that a proper starting process is identified for each program. Therefore, as you finish each program, record the starting process somewhere where you can find it later.

Definitions

Many types of processes in SB+ are not complete in and of themselves. Instead, the process requires a separate record called a *definition*. This definition record contains important information that the process needs in order to operate.

For example, an "input process" is a type of process used to execute an entry screen. However, the input process itself contains little information about how the screen should behave. Instead, the process references a *screen definition* which in turn references *field definitions*. These definitions, then, define the behavior of the screen.

While the definition contains enough information to run the screen, it is useless unless it is referenced by a particular type of process (in this case, an input process). Fortunately, the tools that are used to build definitions have the ability to create the corresponding process almost automatically, so developers are spared the task of defining the two separately.

This relationship is very important. Certain types of processes cannot function without corresponding definitions, and the definitions are useless unless referenced by their corresponding process. Therefore, it is important to recognize when you are creating a process and when you are creating a definition in SB+.

The following types of processes use definitions that are stored separately:

- Input Process -- Screen Definition
- Output Process -- Screen Definition
- Report Writer -- Process Report Writer Definition
- Menu Process -- Menu Definition
- Periodic Update Process -- Periodic Update Definition

Note that both input and output processes use screen definitions. As a result, it is possible to have two separate processes using the same screen definition, one for entry and one for query. While an interesting concept, this practice is generally not recommended.

Drivers

Whenever an input, output, or report process is called, SB+ gathers all of the information necessary to run the process into records called *drivers*. These drivers are then used in the execution of the process, rather than the actual information stored in the process's definition record.

In the case of an input process, for example, the information necessary to run the screen may exist in the screen definition as well as several different field definitions (i.e. file dictionaries). To read and re-read this information during the execution of the screen would be inefficient and would add significant processing overhead to the process. Therefore, SB+ gathers all of this information into the driver records first, then uses the drivers for running the screen.

In summary, the following types of definitions (and process types) use drivers:

- Screen Definitions (Input/Output Processes)
- Report Writer Definitions (Report Processes)

Expressions

One of the most powerful aspects of SB+ is its comprehensive **expression language**. This language serves as the basis for constructing derived values, conversions, default values, all other types of calculated values, and is also a fundamental part of the

SB+ paragraph language.

An expression can be as simple as a literal (such as the number 3 or the name "Kevin") or as complex as your imagination will allow. With a rich set of intrinsic functions and operators as well as full access to everything stored in memory and externally, the expression language in SB+ has many benefits over traditional BASIC expressions.

For example, consider the following expression:

```
(F("CUSTOMER",BRANCH:'*':CUST.ID)<10>)
```

In this example, we have a derived value which reads attribute 10 from a record from the CUSTOMER file, using a branch code and customer ID from a record stored in common. To do this in traditional BASIC would require something like the following:

```
BRANCH = RECORD<1>
CUST.ID = RECORD<2>
OPEN 'CUSTOMER' TO F.CUST THEN
  READV VALUE FROM F.CUST,BRANCH:'*':CUST.ID,10 ELSE
    VALUE = ''
  END
END
```

As you can see, the SB+ expression is more brief and easy to type!

Throughout this book, expressions will be shown as they apply to specific techniques. A more comprehensive reference to the expression language can be found in **Chapter 8**.

Function Keys and Action Bars

In SB+, additional features of entry and inquiry screens are accessed using function keys F1 through F10. Though each screen can have all 10 function keys defined, SB+ customarily reserves a few of the function keys for certain purposes, as follows:

- F1 -- Display the help message for a prompt.
- F2 -- Go to the "update step" of a screen and save the values as entered.
- F3 -- Invoke "Intuitive Help" for a prompt, which helps the user enter a valid response to the prompt.
- F4 -- Go to the "update step" of a screen and delete the record that is displayed.
- F10 -- Display the *action bar* for the screen, if one is defined.

Keys F5 through F9 are not reserved for any purpose, and SB+ has no restrictions on how these function keys are used. Also, if you choose not to use an action bar, F10 can be reassigned to whatever you like. The standard function keys, however, should be used as is.

In addition to the function keys, SB+ provides a facility called an *action bar* which is essentially a menu that appears on-demand at the top of the screen. This action bar, unlike the function keys, is 100% configurable in your own application.

There are several techniques that can be used when implementing function keys and action bars. On one hand, the action bar can be an extension of the function keys. Therefore, if you need more function keys than you have available, the action bar can be used to add the extra options. This is certainly functional, but has the side effect of "hiding" options on the action bar until someone presses F10.

Another option, which I personally prefer, is to have an action bar that appears the same on all screens. This action bar then provides standard functionality to all screens, including access to online documentation, a user toolbox, and a tool to run other programs in the application.

Like most things in SB+, though, this is all a matter of personal style and preference. You are certainly free to define your function keys and action bars in any way you please. Most importantly, be consistent throughout your application.

F1 Help and Intuitive Help

SB+ provides two completely different types of on-screen help within its tools, which can also be implemented extensively within your own application. These two types of help are called **F1 Help** and **Intuitive Help** (also called *F3 Intuitive Help*).

To illustrate the difference between the two, assume for a moment that a screen is running and the cursor is sitting on a prompt, waiting for a value to be entered. If I want SB+ to give me a brief description of what is supposed to be entered at this prompt, I can press the F1 key, which displays a message called F1 Help. If I continue to press F1, more detailed help messages are displayed.

In contrast, if I want SB+ to give me a list of valid options to choose from for a prompt, I can press F3 on the prompt to invoke Intuitive Help. In contrast to F1 Help, which displays only messages, Intuitive Help displays a list of valid entries for the prompt and allows a person to select one or more from the list.

Both F1 Help and F3 Intuitive Help are available throughout the SB+ tools and are extremely useful for learning the many valid options available there. More important, however, by building F1 and F3 help into your application, you can help someone to overcome the learning curve of your software.

SB+ System Administration

When SB+ is first installed, there are several system administration issues to be addressed, including:

Terminal Definition: What terminal hardware will you be using? SB+ supports many popular terminals, but the list is certainly not complete. If you're using anything that isn't commonly found in the industry, you'll probably have to define a terminal definition so SB+ can properly display information on the screen. Then again, even if you are using commonly found equipment, you may want to customize the terminal definition for a more comfortable color scheme or keyboard layout.

Printer Definition: SB+ versions 2.x and 3.x differ substantially in this area, and on certain platforms only certain people can change printer configuration. Typically, however, printer definition involves defining the stationery that your reports will print on, the printers that reports will print on, and routing information so each report can find the right printer.

Port Configuration: On systems which have fixed port assignments, you can configure SB+ to recognize the type of terminal that is being used on a particular port. This eliminates the need for SB+ to prompt for the terminal type when someone logs into the computer. If all of your terminals are the same, you can use the port configuration tool to define the default terminal for SB+.

Security Group Structure: SB+ defines security, or more precisely, restrictions, based on a tree structure. You determine the depth and complexity of this tree structure. As a general rule, don't make your security group structure too complicated, or it will be very difficult to administrate. You can read a little more about security in the following pages, or consult the SB+ System Administration manual for a complete review.

Security

Any application created with SB+ has a certain amount of security automatically built-in. Whether you use this security is entirely up to you, the needs of your application, and your users.

SB+ security allows you to define processes, menus, and options from menus that certain users can or cannot access. This puts you in control of who has access to what.

It's important to note that SB+ security does not restrict access to data. These types of restrictions must be defined with file permissions (Unix), or file access codes (Pick), and have nothing to do with SB+.

If a file has been restricted from all access at the Unix or OE level, you may be able to see a pointer to the file in the master dictionary or VOC file, but SB+ will not be able to open the file. Similarly, if the file is in a Unix directory which cannot be accessed by a particular user, the SB+ application will not be able to see the file for that user. Therefore, keep in mind that SB+ security is just one part of the whole security picture.

SB+ security is defined in terms of groups and users. A security group is a collection of users, and is used o collectively define all of the permissions and restrictions for users in that group. Security groups are arranged in an inverted tree structure, such as the following:

In this example, note that the main security group is called ROOT. This security group is the main group for the SB+ security structure, and must exist in every SB+ environment. Under the ROOT group we have three groups: SYSADM for system administrators, USERS for users, and TEST for a group of product testers. Under the USERS group we have two additional groups, FINANCE and ADMIN, where we can group accounting and administration personnel, respectively.

Users who are assigned to the ROOT group have access to everything. However, users should never be assigned to this group, as the ROOT record is overwritten each time SB+ is installed or upgraded. Instead, create a new group under ROOT which has access to everything, and you won't lose any security information when you upgrade.

Moving down the tree structure, security can get tighter and tighter. Following our example, if we place certain restrictions on the USERS group, we restrict not only this group, but also all of the groups that are under it. Furthermore, we can add more restrictions to the FINANCE group, and continue to make security tighter and tighter for any subordinate groups.

User security records, on the other hand, define a particular person, their preferences, and most importantly, the group that the user belongs to. Once a user has been defined, they can start SB+, log on, and get into your application. What they can do when they get there is defined in group security.

For more details on setting up users and groups with your version of SB+, consult the SB+ System Administration manual.

The Keyword for SB+ Development: PLAN

There's an old story about a manager of a programming shop who worked night and day trying to win a particular programming contract. After months of slaving over contract terms and proposals, he finally succeeded. As he announced the good news to his staff, one staff member posed a question. "That's great boss", he asked, "but just what exactly does the customer want?". Unfortunately, the manager had no idea. He had focused on procuring the contract for so long that he lost sight of the project itself. Not to admit his own shortcoming, however, he calmly responded as he left the room: "You guys start coding and I'll go see what the customer wants".

If there were a single word I could drill into your mind at the outset of starting development with SB+, it would be PLAN (**P**lease **L**ook **A**head **N**ow). Concise and complete planning is the best route to getting a quality product quickly when using SB+. In short, know your destination and course before you start the trip.

When writing traditional BASIC programs, it's easy to start programming with only a conceptual picture of what the application should do, and then mold the code as necessary as the picture becomes clearer. Because this type of programming is very unrestrictive, you are free to do just about anything to make the application behave as you want, regardless of how simple or barbaric the changes may be.

With SB+, however, it is paramount to have an absolutely complete picture of your intended result and the path to get there before a single process is created. This will ensure you are always making forward progress on your project, trodding on to that elusive goal of **completion**.

In many ways, SB+ development can resemble a maze -- there are many ways to move toward a given end, but only a few ways to actually arrive there. Occasionally you will encounter a situation where something appears to take you near your result, only to find some hidden surprise that mandates a complete rework of your code. After doing this a few times, your application begins to look something like a patchwork quilt, and you've wasted more time than you wanted getting somewhere you didn't want to be.

To avoid these pitfalls you should plan everything from start to completion, write it all down, and, most importantly, always, always, **always follow your plan!**

Team Development

If you're working with a variety of people for your project, it is important to define the roles of each person clearly. This will save confusion and questions as to whose job a particular task should be, thereby helping your development team maintain a solid esprit de corp in the heat of the development battle.

Each person on your development team should try to fill only one role at any given time. This is not to say that a person cannot fulfill more than one role, they simply should not try to do it all at the same time. By maintaining only one focus per person at a time, confusion can be minimized.

Consider the following roles as they apply to your development team:

- Designer -- A designer is responsible for defining the goal of a project. They are the "idea people" who provide the direction and definition of the product.
- Analyst -- An analyst is responsible for taking conceptual designs from a designer, converting the ideas into specification documents which describe how the result will be achieved.
- Developer -- The role of the developer is to create the actual product based on the specification documents received from the analysts.
- Tester -- A tester is responsible for evaluating the product that has been created by the developer in comparison to the goal defined by the designer. In the end, the software should match the conceptual design as envisioned by the designers, with no logic or performance errors.
- Technical Writer -- The technical writer is responsible for taking the product created by the developer, the specification documents created by the analyst, and the design documents created by the designer and using this information to create documentation to help a person use the product. This can include paper documentation, as well as extensive online help.

The goal of this illustration is not to say that every development project needs five separate people, but rather to show that there are separate and distinct tasks that must be accomplished for each project. Whether these tasks are done by one person or by a hundred people is completely up to you. Regardless of the size of the development team, however, each person should fill only one role at a given time.

The SB+ Application Development Cycle

Assume for a moment you are faced with the daunting task of writing a software system that will encompass dozens of entry screens, at least as many inquiry screens, and several dozen reports. Granted, you probably don't have to imagine much -- this is what most developers face every day!

Where do you start? How do you know what to start doing first?

The SB+ application development cycle described here is a series of steps you can follow to track the progress of your project. It provides a guideline to help you know where the project is going, where it's been, and most important, how and when your development team will arrive at your goal.

Step 1: Design the Application
Step 2: Define Standards
Step 3: Design Each Program
Step 4: Create Each Program
Step 5: Test
Step 6: Document

Step 1: Design the Application

When creating an application (using SB+ or not), the first step in the development cycle should always be to define the goal. This can be done by creating a Project Overview document that addresses the following three issues:

- What problem or need is this application intended to solve?

  The purpose of this question is to ensure you have a working knowledge of the problem before trying to come up with a solution. To quote an old phrase: "Never try to solve a problem you know nothing about".

- What programs will be created to solve this problem and what role does each program serve in this solution?

  During this task, try to envision what the application will do and how the different parts will interrelate to one another. For the purpose of breaking up the project into smaller pieces, categorize each program as an entry screen, inquiry screen, report, update, or miscellaneous utility.

- What files will be created or modified for this project, and what do the records look like in each file?

  This is generally the most time-consuming task, as it requires a complete evaluation of all of the files that will be created or modified for the project. The goal of this step is to have a list of files and a detailed list of attributes for each file that can be consulted throughout the project.

With these questions answered, the next step is to prioritize the programs that will be created and organize a task list in such a way that will maximize your efficiency in coding and testing. This is most easily done by evaluating the file or files that a particular program will access.

As an example, if a program updates file A which uses information from file B, then all of the programs for file B should be completed before the programs that update file A are developed. This ensures that the infrastructure for updating file B is completely in place before its information is needed in other applications.

Of course, to come up with a prioritized list of programs you must have at least some idea how each of the programs will work. You need not know *everything* about all of the programs, but the more you know at this stage about each program the easier it will be to prioritize your list. Also, look for commonalities between programs, and use this to come up with a list of standard processes that can be used throughout the software.

In the end, this document should show all of the programs to be created, describe all the files that will be used in the project, and prioritize the list of programs to be created in some logical order as described above. With this document completed, you have a tool that is invaluable for tracking your progress through the project, as well as for keeping your development team focused on your common goal.

Step 2: Define Standards

In any development project (whether using SB+ or not), effective standards are important. Before any formal design has been started on your project, naming standards and standard user interface conventions should be established.

Naming standards, simply stated, are guidelines for naming components of your software -- they define what your application will look like to the development team. To illustrate why this is important, assume that two of your development team members are asked to name a field that will hold the date a record was entered. One person may come up with "ENTRY.DT" as the name of the field, while another may prefer "DATE.ENT". With no documented guidelines, how does the development team know which one is appropriate?

In a traditional BASIC environment, naming standards were important for naming files, fields, programs, and subroutines. In an SB+ application, you must name not only these things, but processes, definitions, tables, and menus as well! Therefore it is important to have an effective standard for how to name all of these different components of your software.

Complimentary to naming standards, user interface standards define what your application will look like to the user(s). How will prompts be organized on a screen? What will a typical report look like? How will your function keys be defined? These, and many other issues, are important to maintain a consistent "look-and-feel" across the various programs in your project.

For a more detailed look at some established SB+ development standards, see Appendix A.

Step 3: Design Each Program

Once the Project Overview is complete, the next step in the development cycle is to create (or finish) detailed specifications for each program. While the purpose of this step is a full understanding of each program that will be created, invest a little extra effort looking for commonalities between programs. Anywhere that you can create a process or series of processes which can be reused throughout the application, valuable time can be saved during the coding and maintenance of the project.

Each specification should contain enough information for the reader to understand why the program is necessary. This is important for two reasons: First, if the designer is unable to define why a particular program is important, it probably isn't. Second, this information is useful for the developer to understand the relative importance of the program in relation to the overall application.

This section of the specification should be followed with specifics explaining how the program is to be implemented. For example, documents explaining screen applications (entry or inquiry) should show sufficient images and prompt detail to explain what the screens should look like. The same holds true for reports. On the other hand, updates and utilities need to be defined by what is selected and in what order, and how this information is used in updating other files and fields.

Step 4: Create Each Program

After all programs have been designed, the next step is to actually create them. In a typical BASIC programming environment, this step would be the most time consuming. However, using SB+ and the previous steps in the development cycle, this is far and away the least time consuming step!

When enough planning is done prior to writing even a single process, actually creating the programs is nothing more than filling in blanks. With each program fully defined, there is no rework, except in cases when the overall design changes. (When this happens, it's often best to re-evaluate the project from the beginning to minimize the amount of recoding that must be done.) Less rework means less patchwork and more productive code, which makes a leaner, more efficient, and more easily maintained application.

Step 5: Test

Like any development project, each program must be tested thoroughly to evaluate its performance compared to its design specification. Additionally, groups of programs must be tested together to evaluate the effectiveness of the information flow between programs. At the end of the project, there are two critical issues that must be addressed:

- Have all errors been removed from the software?
- Is the software solving the problem or stated need from the Project Overview document?

Of course, nobody wants to send problematic software out the door. Therefore, care must first be taken to ensure that all logic and design errors have been identified and corrected before this phase is complete. Second, the software must satisfy the stated need. Otherwise, the entire project has been for naught. Of course, following your Project Overview throughout the project virtually eliminates any possibility of veering too far from the design during the project.

Step 6: Document

The final step in the development process is often one of great concern to managers of software projects, i.e. how to document the software that has been created. "We know what it does", says one programmer, "so why do we have to write it all down?". The answer to this is really quite simple: If you don't tell your users how to use the software right, there's a sportin' chance they will figure out by themselves how to use it wrong.

Though there are no hard statistics to back this up, I would venture a guess that more bugs have been discovered through the lack of documentation than any other means.

Fortunately, SB+ provides a wealth of online documentation options available that require little to no programming effort. Using various combinations of techniques, the daunting task of documenting a large, complex project can be done with minimum time and effort.

One caveat, however: Documentation is a highly subjective topic. Two people using the same software may look at documentation from two completely different perspectives. Simply because there is a wealth of tools is no guarantee that your documentation will be perfect. It merely gives you more options to argue about!

Portability Considerations

SB+ has been marketed as a way to create software that is portable across all Pick platforms. There is certainly much truth to this. As long as your software is written in 100% SB+ with no BASIC code or use of system-specific TCL commands, it is highly likely that you will never encounter portability problems.

However, writing your application in 100% SB+ can severely limit your options when writing complex code. For example, BASIC is a very useful and powerful option when writing a routine to update many interrelated files. Also powerful is the ability to create temporary field definitions in the dictionary of a file to support varied or user-selectable sort and selection options for a report. This becomes complicated by the fact that there are subtle differences between dictionary formats, BASIC, and TCL command interfaces across different platforms.

Take for example the task of opening a dictionary of a file. Using UniData BASIC (known as UniBasic), a dictionary must be opened as follows:

```
OPEN 'DICT','CUSTOMER' TO F.DICT ELSE ...
```

With Advanced Pick, however, either of the following options are acceptable:

```
OPEN 'DICT','CUSTOMER' TO F.DICT ELSE...
OPEN 'DICT CUSTOMER' TO F.DICT ELSE...
```

Therefore, if the application is being written for both AP and UniData, the UniData OPEN syntax must be used to ensure portability without adapting the code to each particular target platform.

Unfortunately, all portability issues are not this straightforward. When writing code that will create dictionary records for a file, there is little common ground between Pick and UniData. Therefore, code must be written for each system and selectively executed depending on the target platform.

If at all possible, know your target platforms before starting development, exploit commonalities across platforms, and compensate for system-specific issues during development. Most important, **never be limited by your tools**. A healthy imagination can work wonders when it appears there are no options.

Using a Standard Process Library

During the design of your product, you may notice areas where similar or identical processing will be done in multiple areas of the software. When this happens, consider creating a standard process that can be used throughout the application, rather than coding and recoding the same feature each place it is needed. Similar to standard subroutines in a traditional BASIC programming project, standard processes can help you maximize your productivity by eliminating recoding of work done before.

As you notice these common areas of your software, build a library of standard processes that can be used in each case, and of course, document these processes well! With a good library in place you can save valuable time throughout the project, which translates directly into a significant savings for your development team.

See **Appendix A** for more information about constructing a standard process library.

Understanding the Size of the Application

If the size of an application was determined by the number of menu selections, there would be no appreciable difference between an SB+ application and a non-SB+ application. However, if the size of the application is based on the number of components required to make the software function, the SB+ application would unquestionably be the larger of the two.

SB+ applications, unlike traditional BASIC applications, are not simply programs and subroutines. Instead, applications are constructed from field definitions, screen definitions, processes, menus, BASIC code, error messages, control records, and several other different types of records. To make matters a little more complicated, these records are not stored in a central file, but are distributed across any number of files throughout each system.

This complicates the task of moving a program from one computer to another. With so many parts stored in so many places, it can be difficult to know for certain that everything is being properly moved from one machine to the other.

With SB+, the days of printing a listing of your programs is over. But then again, SB+ applications rarely require this type of review. With the comprehensive debugging tools inside SB+, you should rarely find the need to print a program in its entirety.

For your reference, the following table shows several common definitions created with SB+, and the actual number of physical records that are created for each.

Definition Type	***Number of Records***	***File Name***
Field Definition	2	Dictionary of a file
Screen Definition	2 3	Dictionary of a file xxxDRIVERS file
Query Report Definition	1	xxxDEFN file
ReportWriter Definition	2 2	Dictionary of a file xxxDRIVERS file
Process Definition	2	xxxPROCESS file
Menu Definition	1	xxxMENUS file
Code Table Definition	1	xxxDEFN file
Periodic Update Definition with *n* secondary updates	1+*n*	xxxDEFN file
Dialog Box Definition	1	xxxDEFN file

Getting Started With SB+

Before we get into details about the various wonderful and awe-inspiring things you can do with SB+, let's take a look at two fundamentals that simply cannot be ignored: 1) we must first install SB+ into an account, and 2) a system must be created.

Installing SB+ into an Account

When SB+ is installed on a machine, three new accounts are created. On a Pick machine, these accounts are named SB+, SB+.DEFN, and SBDEMO. The SB+ account and SB+.DEFN accounts hold the SB+ software. SBDEMO, in contrast, is a collection of development techniques created by the System Builder staff.

Never create your own software in any of these accounts. When you upgrade to a newer version of SB+, these accounts will be replaced and any software you have there will be lost.

WARNING: Care must also be taken when installing SB+ into an existing account. The SB+ installation procedure overwrites the logon proc that may exist in the account, and therefore has the potential of eliminating something (without warning) that may be important to the existing application.

Instead, create a new account on your system for your SB+ development, and use Q-pointers to reference existing files. This will promote some autonomy between existing applications and your SB+ projects, as well as ensure that you aren't faced with complications in your SB+ project due to pre-existing conditions.

Note: Instructions for loading the SB+ tape or diskettes is sent with each copy. Consult this document for specific information on loading the software on the SB+ tape or diskette onto your system.

After the SB+ accounts have been loaded, SB+ can be installed in a development account. To do so, create a Q-pointer in your MD named DM that appears as follows: (Note that this technique applies only to systems that support Q-pointers -- other systems may require a F-pointer with a fully qualified path name.)

```
001 Q
002 SB+.DEFN
003 DM
```

Once this Q-pointer has been filed, enter the following command at the TCL prompt:

```
RUN DM SB.INITIATE
```

This command will run a program in the DM file called SB.INITIATE. Once started, this program will:

- Catalog all of the subroutines and programs required for SB+ to run in the local account;
- Set up Q-pointers to the SB+ system files (in the SB+ and SB+.DEFN accounts);
- Overwrite the log on proc for the account (watch out for this one!), and;
- Prompt you for information necessary to create an SB+ system.

Creating an SB+ System

In SB+, a system is a collection of autonomous processes and definitions. Any account can have any number of systems.

SB+ must have at least one system in order to run in an account. Otherwise, it doesn't have a place to store the processes and definitions you develop.

At the end of installing SB+ on an account (using SB.INITIATE), SB+ asks for a system ID. This ID is simply a 2-4 character alphanumeric code that you use to identify your system. Warning: Choose a system ID carefully. It is difficult to change once development begins!

After you have entered a system ID, SB+ asks you to enter a description for the system. Unlike the system ID, the description is easy to change. Therefore, you may want enter something standard like "Development", knowing you can go back later and enter something more descriptive.

Once these two entries have been made, SB+ will create 7 files that comprise the core of the system being created. These files are:

- xxxCONTROL -- Control information
- xxxDEFN -- Definitions
- xxxDRIVERS -- Screen and report drivers
- xxxHELP -- Help messages
- xxxMENUS -- Menus
- xxxPROCESS -- Processes
- xxxPROGS -- BASIC subroutines and generated code

In each of the file names, note the xxx prefix. When each of these files is created, this prefix will be replaced by the system ID you entered. Therefore, if the system ID is "CUST", the system files will be named CUSTCONTROL, CUSTDEFN, etc. If the system ID is "PAY", the system files will be PAYCONTROL, PAYDEFN, etc.

Though SB+ does not prevent you from entering system IDs longer than 4 characters, you can see how long and unwieldy the system file names could be if you enter something longer!

At this point, simply note the names of these files and the fact that they comprise your SB+ system. For the purposes of development you will rarely need to know this information. When deploying your application on another machine, however, this information is very valuable.

Entry Screen Fundamentals

Screen Definition vs. Input Process

In SB+, an entry screen is called an input process. Oddly enough, however, your efforts will not be spent in the creation of this process. In fact, SB+ does most (and sometimes all) of the work in creating the actual input process.

Instead, your efforts will be dedicated to the creation of a **screen definition**. In this definition, SB+ stores important information used to run the screen, including the window size, prompts, graphics, function keys, action bars, and any special processing required.

It is very important to realize that the input process can do nothing without a screen definition. Likewise, the screen definition is useless without a process that references it. Both must be present for a screen to work.

Inserting, Amending, or Deleting

Input processes are used to perform three different types of updates. First, they can be used to insert records into a file. Secondly, they can be used to edit, or amend, existing records in a file. Lastly, input processes can delete records from a file.

For certain applications, you may want to restrict an input process from deleting records. Or, perhaps you want to create a screen that can be used to enter new records only. Using a variety of techniques and **options** these restrictions are easily applied.

An Input Process May or May Not Update Records

An input process may update information somewhere, but it's not required. Some programs use input processes to capture values that are never written.

In a typical application, most input processes will be used to enter and maintain records stored in files somewhere. However, remember that this is only one use of the input process. Anytime you need a screen where a user can enter something, an input process is the answer -- regardless of whether the information is saved or not.

Creating an Input Process

To create an input process, use this checklist:

- First, a file must be created. This is the file where the information will be read from and written to.
- Create field definitions for the file. This will define the fields that will be stored in each record.
- Create the screen definition, placing the fields you want where you want them.
- Run the screen definition. When a screen definition is run from inside the Screen Definitions tool, SB+ creates the input process for you, then runs the screen for your review.

Once these steps have been done, you have an input process that can be used to add, amend, or delete records from a file.

Fortunately, this is just the beginning. Though an input process may do something, that doesn't necessarily mean it's complete. By adding validation codes, defaults, help, and any variety of special processing, you can take a simple input process and make it something truly spectacular.

Creating a File
Creating Field Definitions
Creating a Screen Definition
Placing Fields on a Screen Definition/What About QuickBuild?
Running the Screen

So What's Really Going on Here?

An input process can be a very complicated thing, when you consider that this one process encompasses everything that SB+ needs to run a screen. However, on a fundamental level, what the input process does is really very simple. To get a clearer picture of this, let's take a look at the memory that SB+ uses, particularly in the area of variables that are common to all SB+ processes.

SB+ defines a number of **variables** that are common to all areas of the software. Six of these variables play critical roles in an input process, though many other variables may be used.

These six **common variables** are named as follows:

- MAINFILE
- F.FILE
- KEY
- RECORD
- ORIG.REC
- ACTION

When a screen is started, the name of the file to be updated is stored in the common variable MAINFILE. This file is then opened to a file variable called F.FILE.

When you enter a value into the key field on a screen, the value is stored in the common variable KEY. Following this, the record as stored under that key will be read and stored in the common variables RECORD and ORIG.REC.

Yes, there are two copies of the record in memory at any given time. SB+ loads RECORD, then copies it to ORIG.REC. While RECORD is changing constantly, ORIG.REC is never changed by SB+. Later, when a record is saved, the two can be compared to see if anything has changed. More importantly, by comparing the two variables one can see exactly what has changed.

Lastly, if the record is new, the ACTION variable will be set to 1. If the record is not new, ACTION will be set to 2. This way we know whether we're adding a new record or amending an existing one.

With this in mind, we can see that the input process basically prompts for a key, reads a record into common memory, allows for changes to common memory, and then writes a record based on what is stored in common memory. This sequence of events, then, forms the basis of every screen that you create in SB+.

Using /DEBUG

You can view these common variables using a useful debugging tool called /DEBUG. To use this, run your screen using None-Execute Now. After you have entered a value for the key field, enter /DEBUG at the next prompt. (Actually, you can use /DEBUG at any input prompt or menu.)

At the bottom of the screen, SB+ will ask you to enter the name of the common variable to view. At this prompt, you may enter any one of the common variable names, and SB+ will show you the contents of that variable at that time. For example, if you want to see what is in the current record, enter RECORD when prompted. To see the current value of the key, enter KEY when prompted.

It can be difficult to see a complete record with this tool, because all of the information shows up in one or two lines at the bottom of the screen. To overcome this, press the down arrow when DEBUG asks you to enter another variable name. This will cause the process to display a full screen of the information in the record, complete with line number references. This makes viewing the information much, much easier when you're dealing with big records.

Take some time to experiment with this process to get a good idea of when SB+ loads the different common variables -- this understanding will help you later when building complicated screens.

Also note that some common variables should not be viewed using /DEBUG. Certain variables contain embedded control characters that have a tendency to disturb the screen display or lock up your terminal, which can be irritating and a waste of valuable time.

When you're done using /DEBUG, simply press Escape to return back to your screen.

Using /EE

/DEBUG is not the only tool for viewing common variables. In fact, it's not even the best tool for that purpose. Instead, SB+ has a process named /EE (Evaluate Expression) which not only allows you to view common variables, it allows you to see the result of literally any expression! To use this process, run your screen and enter /EE at any prompt. (Actually, you can run /EE from any input prompt or menu.)

At the bottom of the screen, SB+ will ask you to enter an expression. At this prompt, enter any valid expression. Expressions can be as simple as a common variable, such as:

```
@RECORD
@KEY
@ORIG.REC
```

Or they may be a complex expression such as:

```
F("CUSTOMER",P("GET.CUSTOMER.ID,":@PARMS(2)<20,4>))
```

Note that like /DEBUG, it can be difficult to view a complete record with this tool, noting that all of the information shows up at the bottom of the screen in one or two lines. To overcome this, press the down arrow when EE asks you to enter another variable name. This will cause the process to display a full screen of the information in the record, complete with line number references. This trick is very handy when attempting to see what's in big records or complex common variables.

Also, as with /DEBUG, there are certain common variables you should not try to view. Among these are @TERM.DEFN, @PRINT.DEFN, and @USER.KEYS. If you try to view these using /EE, you may lock up your terminal or disturb the screen display.

When you're done using /EE, simply press Escape to return back to the screen.

Moving Prompts on a Screen

Once a prompt has been placed on the screen, it may be moved using one of three techniques. One technique moves a group of prompts at a time, while the other two move a single prompt at a time.

Moving a Group of Prompts

Moving a group of prompts at a time is done by inserting or deleting lines (rows) on the screen definition. When a line is inserted, all of the prompts on the cursor line and below are moved down, whereas deleting a line moves prompts below the cursor up.

The actual key presses used to insert and delete lines can be different for each terminal type. Therefore, to find out the actual key press for these functions, enter /KEYS at any menu or input prompt.

If there are several prompts on one line, they may be moved to the left by deleting characters before the prompts using the Delete Character key, or moved to the right by inserting characters using the Insert Character key. Again, use /KEYS to investigate the actual keypresses for these functions.

Moving One Prompt At A Time Using F8

To move one prompt at a time you may use the F8-Move function key. Using this feature you can move a prompt label and input area together, or move an input area without changing the position of the prompt label.

To move a prompt label and input area together, position the cursor on the first character of the prompt label and press F8-Move. Move the cursor where you want the label to start and press <cr>. Both the label and input area will be moved.

To move an input area independent of the prompt label, position the cursor on the first character of the input area for a prompt and press F8-Move. Move the cursor where you want the input area to start and press <cr>. The input area will be moved, but the prompt label will remain in its original location.

Moving One Prompt At A Time Using F5.

The F5-Field function key can also be used to move one prompt at a time. Move the cursor to the first character of the prompt label or the input area and press F5. In the fields labeled Prompt Col and Row and Input Col and Row, simply enter the coordinates where the field should be placed, pressing F2 to save the changed entries. This method allows you to move the prompt label, input area, or both to any location on the screen.

The Display Prompt (N/S/A) field on the F5-Field window can also be used to change the cursor position. Changing this value from "S"ide to "A"bove and back again will recalculate the default position for the field. In short, if you mess up the cursor position, use this to restore the field position to something sensible.

Moving One Prompt At A Time Using the Mouse

Using the SBClient terminal emulator, you can easily move one prompt on the screen by clicking on the label or input area of the prompt to be moved, and dragging the prompt to the new location. As with the F8-Move key, this feature can be used to move a

prompt label and input area either together or independently of one another.

Resequencing Prompts on a Screen Definition

SB+ makes a good effort to maintain the correct prompting sequence when you place new fields on a screen. However, if you have a tendency to arrange fields in multiple columns, or a mix of columns and rows, the prompting order that SB+ uses may not be what you want.

Fortunately, you have 100% control over the order of prompts on your screen, both at the time the prompts are first placed on the screen, and also after all of the prompts have been placed.

Setting the Prompt Sequence Prior To Adding Fields

On the F6-Params screen, there is a prompt labeled Field Sequence (C/R/A) which you can set to tell SB+ how you want new fields to be added to the prompt sequence. This flag applies only to new fields that are added after the flag has been set. In other words, changing this flag after fields have been added will not automatically resequence the prompts.

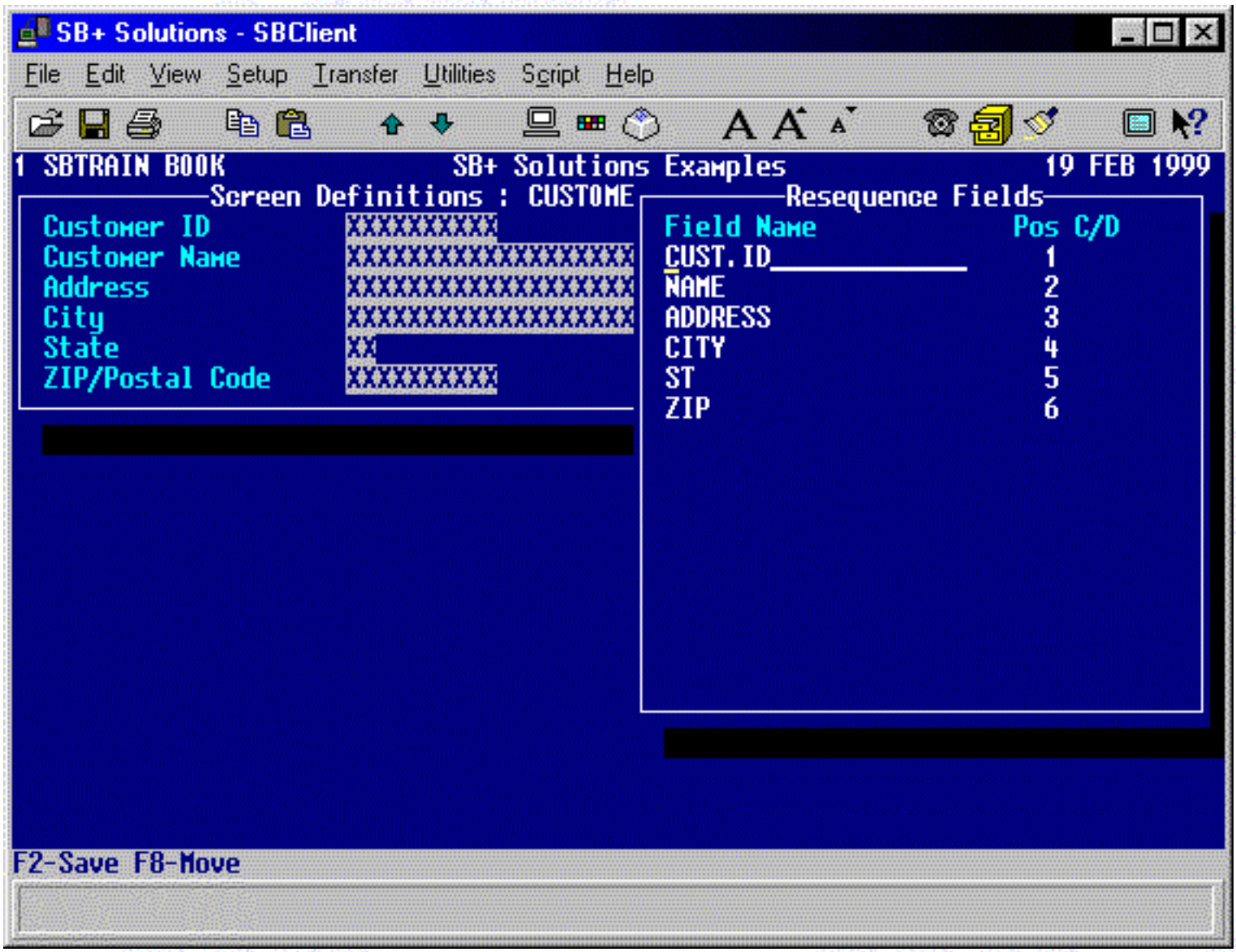
The valid values for this prompt are:

C	Column: Insert new fields into the prompt sequence in columnar order. With this setting, SB+ assumes that all of the fields on the left side of the screen will be prompted before the fields on the right, assuming a multi-column screen layout. In other words, the prompt order will be top to bottom, left to right.
R	Row: Insert new fields into the prompt sequence in row order. With this setting, SB+ assumes the fields are organized like a book, and will sequence the prompts from left to right, top to bottom.
A	Ask: As each new field is added, SB+ will ask you to enter the sequence number where this prompt should appear in the field sequence. This is useful for small screens with only a few prompts, but can be irritating for large screens with many prompts. Unfortunately, whenever you resequence the prompts on the screen (as described below), the Field Sequence (C/R/A) prompt will automatically be changed to this value. As a result, when prompts are adjusted or added after resequencing, SB+ will ask you for a prompt sequence number for each one. Fortunately, using the F6-Params screen, you can easily change the flag to something more sensible.

Changing Prompting Order

The prompting order of fields can be easily changed on a Screen Definition by pressing the F10-Action function key and selecting the Reseq option from the menu.

When this option is selected, a window is displayed which appears as follows:



To move a prompt in the sequence, place the cursor on the prompt to move and press F8-Move. When prompted for the position to move the field to, enter the sequence number as shown on the right that corresponds to the position where this prompt should be located. SB+ will then move the prompt in the list, resequencing all subsequent prompts.

When all of the changes have been made, press the F2-Accept function key to save the changes and return to the definition screen. The new prompt sequence will be used the next time the screen is executed.

Painting Text on a Screen Definition

When a screen is being painted, you may type characters directly onto the definition to be displayed as part of the window. These characters form an "overlay" to the screen image, and will be displayed in addition to the prompt labels when the screen is running.

Typically, general information that does not apply to any specific prompt is added to the screen definition this way. For example, a message may be typed near an address field to tell the user to enter the street address in the first line of the address, the apartment number in the second, and the city, state, and ZIP code in the third line. This, then, gives the user additional information that would otherwise only be available when the F1-Help key is pressed.

Some developers choose to enter their prompt labels right onto the screen definition this way, and then suppress the prompt labels that SB+ displays. Using this technique, the developer is not restricted to the prompt label as it is stored in the field definition, but can enter anything they please. On the negative side, however, the developer is not restricted to the prompt label as it is stored in the field definition, but can enter anything they please.

One thing that SB+ currently does not do on screens, though the feature has been promised for a future release, is support *stacked headings*. Simply stated, a stacked heading is a prompt label that occupies multiple rows, such as:

```
Distributor
Name
```

vs.

```
Distributor Name
```

Therefore, if you want to have a stacked heading on a screen, you have the following options:

- Type the entire label onto the screen definition in the format you want it, or;
- Type the upper part of the label, and use a prompt above on the field to display the lower part of the label.

Another way that this feature can be used is to place headings over multiple columns, such as in the following:

```
----Payment----
Number   Amount
xxxxx  xxxxx.xx
xxxxx  xxxxx.xx
```

The words Number and Amount can be standard prompts above the input area, but to display the Payment prompt above both of these fields, you must type it directly onto the screen definition.

The question then becomes whether to enforce standard prompt labels using the information in the field definition, or allow maximum flexibility by typing prompts directly onto the screen definition. This, like many things, should be considered a subject as you define your own development standards.

Graphics

To enhance the appearance of your screen, SB+ allows you to place graphic lines and boxes on the screen definition. These graphics help to separate and distinguish areas of the screen for special emphasis. As an example, if a screen has an order total field, but that field is difficult to spot immediately because it is buried among several other fields, a graphic box can be an effective enhancement to draw attention to it.

Of course, graphics can be overused, in which case they actually detract from the screen's appearance, rather than enhance it. Care must be taken, then, to use graphics sparingly and with a specific purpose in mind.

Placing Graphics On A Screen Definition
Special Rules for Graphics

Resizing/Moving the Window

When a screen definition is first created, there's a pretty good chance the window will not be exactly the right size or at exactly the right position. As prompts are added, the window may need to expand. As well, as prompts are moved or deleted from the screen, the window may need to shrink. Or perhaps you want to move the screen to the left, right, up, or down a bit.

Though there are no specific rules about window size, some people prefer to center each window and minimize the amount of unused screen space. Therefore, the window is only as large as it needs to be to accommodate its prompts -- no larger and definitely no smaller. Still others prefer to have uniformly sized windows placed at the same place for all applications. I prefer the first approach, as it makes the windows tight and to the point without a significant amount of unused space.

When a screen definition is first created, you can save time by creating the window as large as your screen will handle, which is generally 0,0,79,21. This way, there is literally no chance that you will need to expand the window while placing prompts on the screen. Once all of the prompts have been placed, you can then shrink the screen to the proper size.

On the other hand, consider what might happen if you estimate the size of your window. As prompts are added, you may find that the window needs to be expanded. After the window has been expanded and you add a few more prompts, you find that the window needs to be expanded again. Unless the screen starts as large as it can be and then is shrunk to proper size at the end, there's a good chance you will be wasting time resizing the screen over and over while placing prompts. Granted, resizing is quick and painless, but if you don't need to do it, you shouldn't.

There are two different methods that can be used to resize a screen. These methods are discussed in the following sections.

Resizing a Window using F6-Params

Press F6-Params in the Screen Definitions tool to invoke the F6-Params screen. If it looks familiar, good -- this is the same screen where the window coordinates were first defined. Change the window coordinates here and press F2 to update the display.

Usually, this isn't the best approach because you can't see the prompts on the screen while you're changing the window coordinates. As a result, it usually takes a couple of tries before getting the window size exactly the way you want it. Fortunately, there are better options...

Resizing a window using F10-Action, ResiZe

This method of resizing the window is actually the most flexible, as it allows you not only to paint the window size, but also have SB+ automatically size the window to the smallest possible coordinates, and/or move the window as a whole.

To use this technique, press F10-Action in the Screen Definitions tool to invoke the action bar. Once the action bar is displayed, press "Z" for "ResiZe". The following prompt will appear on the bottom of the screen:

Move cursor to new top left corner then <enter> or M move box or A auto size

To move the entire window without resizing, move the cursor to the position where you want the upper left corner to be placed and press "M".

To have SB+ automatically resize the window, press "A" at this prompt. If you want to have SB+ automatically resize the window and move the window at the same time, move the cursor to the position where you want the upper left corner to be placed, then press "A".

Alternatively, you may "draw" the box using the arrow keys and <cr>. To do this, move the cursor where you want the upper left corner and press <cr>. Then, move the cursor to where you want the lower right corner and press <cr>. Using this, SB+ will recalculate the window coordinates and display the window using the new size.

Important Note: Use caution when making a window smaller. If any prompts fall outside of the new window coordinates, you will not be able to move the cursor to the prompt. To correct this, simply make the window box large again, move the prompt where you want it, and then shrink the window to the size you want.

Defining Function Keys/Action Bars

Function keys, and optionally a well-defined action bar, can add significant functionality to each screen. More importantly, though, they provide a way to link several (otherwise unrelated) screens together seamlessly and with little effort.

In simple terms, defining a function key is nothing more than telling SB+ the name of a process to call when that key is pressed. However, knowing that the called process can do literally anything in your application, it becomes clear that this functionality is of great value to the application.

There are two places in a screen definition where function keys can be defined. The following sections describe the differences between the two.

Defining Standard Function Keys
Defining Named Function Keys
Defining the Default Function Key Set
So What's This G:U and G:DE?
Using An Action Bar
Highlighting An Action Bar Letter

Finishing the Screen

After the screen has been created and run once (to create the input or output process), the screen prototype is complete. You may have a functional screen, but the screen is far from being a complete user-ready product. To finish the screen, the following must be done:

For each prompt:

- **Defaults** may need to be added;
- **Validation codes** may need to be defined;
- **Intuitive help** should be defined everywhere possible;
- **Conversions** may need to be added for certain fields;
- **Derived values** may need to be added;
- **Special processing** may need to be added to fields to support features like **prompt skipping**, **conditional** **subscreens**, or other such features.

For the key field:

- Special techniques, such as **multi-part keys**, or **sequentially assigned keys**, may need to be implemented;

For the screen:

- Special processing may need to be added to ensure that the record **can be edited** by the screen;
- Special processing may need to be done to **call other screens** or to perform other important processing;

Of course, the term "special processing" is somewhat of a nebulous term, as it could mean nearly anything depending on what the application requires. The important thing to note from this list is the sequence of events. Rather than stumbling through the different things to be done in finishing your screen, you may follow this sequence of events to get nearly all of the work accomplished in an orderly and timely manner.

Defaults

A *default* is a value that a given prompt will assume when there is no value in the field. For example, if we have an entry screen with a state prompt, and the state will be "WA" most of the time, we can help the operator enter this value by setting the prompt default to the appropriate value. With this in place, the operator does not need to type "WA" <cr> on the prompt, but can simply press <cr> to accept the default.

Defaults are SB+ expressions, and therefore can be as simple as a literal (like "WA") or as complex as your imagination will allow. Also, like any SB+ expression prompt, the F3 intuitive help that is available to assist you in building your default expression is incredibly useful for learning the syntax of the various expression elements.

A default may be entered in one of two places for any given field on a screen. Most often, the default will be entered on the **field definition** in the Default Value prompt. This sets the default for the field for all screens where the field is referenced. This default may also be overridden at the field level on a screen definition, though the technique for doing this is unarguably not obvious.

When you are looking at a field on a screen using F5-Field, find the Process Before prompt. This prompt is typically used to call a process to do some pre-processing for the field. However, this prompt can be adapted to serve as a default override for the field by entering the following *inline default*:

```
D:(...)
```

In this example, the ellipses are shown for illustrative purposes only and should be replaced with the actual default expression to be used for the prompt. Therefore, to set a default of "WA" on a field, D:("WA") may be used. Or, to set a default on a field of 10 days past today, either D:(@DATE+10) or D:((@DATE+(5*2)), et al., could be used.

There are a few issues to keep in mind when overriding the default using this technique. First, when a default expression is entered this way SB+ must parse, evaluate, and then execute the expression in order to arrive at the default value. Normally the time it takes to do this is nominal (depending on the complexity of the expression). However, if this technique is used extensively on a screen, the sum of nominal speed losses can result in performance concerns when the drivers are being built. Secondly, because the Process Before on the prompt is used for the default override, there remains no open slot for linking any pre-prompt processing for the field, such as a process to skip the field. Last, except in cases where this technique is the standard, it is simply not obvious and can confuse someone who is trying to read your code.

Despite all the bad press, however, this technique is still a valuable asset to have available in certain circumstances. For example, this technique is perfect for setting a default of "NEW" on a key field when the real key is to be assigned at file time. In this case, a default of "NEW" on the field definition would be inappropriate, as it would then display the default when the key field is used on output processes (i.e. query screens). Therefore, to have a default that applies to the entry screen only, we need to use this default override technique.

Note that a field does not need to be an input field to be affected by a default. If the cursor passes through a display-only field with a default and the value of the field is null, the default will be calculated and updated into the record. Also note that an input field does not necessarily have to be affected by the default. For example, if the user moves the cursor to a field with a default and presses the down arrow instead of <cr>, the default will not be updated into the record (in version 2.x only -- this is not true for later versions.).

The following sections describe many of the ways that default values can be set for a field. This is by no means an exhaustive exposé of every possible option, but is rather intended to provide some fundamentals that you can use, adapt, and embellish in

your own application development.

Using the Defaults Up Front Option
Creating a Literal Default
Setting a Default From a Common Variable
Reading a Default From a Parameter Record
Setting Defaults Under Multiple Conditions
Calling a Process to Calculate a Default
Selecting From a List of Default Values
Using a Mandatory Default
Using Skip Defaults
Using a Mandatory Skip Default

Validating an Entry

No matter how attractive your screens may appear, if they do not effectively validate user's entries, the screen is essentially a beautiful way to trash your database. Robust and bulletproof applications require a significant programming investment in the area of validating input, if for no other reason than to insure that users doesn't hurt themselves. Fortunately, SB+ provides a wealth of validation options to choose from, all from a single prompt in the Field Definitions tool.

The process of constructing a validation for any single field requires answers to the following questions:

- Under what conditions is an entry considered valid for this field?
- What message or messages should be displayed and under which conditions does each message apply?

For data integrity, the first question is the most critical; you must know every valid value or domain of valid values for the prompt. Some types of entries are obvious, such as a State prompt on a USA address -- it is highly unlikely a new state will be added soon. (Incidentally, don't forget to include Puerto Rico as a state.) Other types of entries may not be so obvious. As an example, if we have a customer screen with Customer Type and Credit Limit prompts, the credit limit may be validated using different criteria depending on the type of customer we are entering. In this case, the number of valid values is not necessarily finite, so the validation must encompass all of the possible domains of values that could be entered under the variety of different circumstances.

Note that the validation code you place on a field applies only to that field when it appears on one or more entry screens. The validation does not apply during batch updates (called **periodic updates**), and does not apply if the field never appears on a screen or is never prompted. Therefore, there is no direct link between a validation code in SB+ and the referential integrity of your file system.

For an effective user interface, the second question must not be ignored. Should you display one generalized error message per prompt, or should each possible reason for failure have its own error message? What type of language will your error messages use? (Incidentally, avoiding words like
"moron" and "idiot" in your errors is highly recommended.) In short, what will you tell the user when they enter an invalid value?

Messages typically fall into one of two categories, which incidentally are the two types of validations that you can create:

- Warnings, where the value is acceptable but questionable, and;
- Errors, where the value is unacceptable.

Incidentally, both types of messages can appear as part of the same validation. When this happens, errors are usually validated first, with warnings validated later.

Certain Validations Happen Automatically

Depending on the type of a field, certain validations may happen without any effort on your part. For example, numeric fields will only accept numbers. Therefore, you'll never need to validate a numeric field for alphabetic or control characters. Monetary fields work the same as numeric fields in this respect.

Date fields also have a built-in validation. If a user enters a string that cannot be represented as a valid internal date, SB+ will reject the entry before the value is processed by the Validation on the field. Therefore, you as a developer are guaranteed that SB+ will never give you an invalid date when validating a date field.

Alpha fields, on the other hand, have no built-in validation, and can accept any values. Therefore, you'll want to be sure you have ample validation on these types of fields.

The Process After is NOT for Validation

In the prompting sequence for a given field, the validation code (from the field definition) and Process After (from the screen definition, F5-Field) both happen after a value has been entered. However, under no circumstances should the Process After be used to validate an entry.

After a value has been entered into a prompt, SB+ uses the validation code to check the entry. If the validation is successful, the entered value is updated into the record. After the record has been updated, the Process After will be executed. Therefore, even if the Process After rejects the entry, the record has already been changed, and therefore F2 can be pressed to save the record with the bad value.

Even **mandatory prompts** are fooled by this problem. When you press F2, SB+ validates that all mandatory fields have a value. It does nothing, however, to ensure the value is valid.

Therefore, never use a Process After to augment the validation for a field. All validation should be done from the Validation Code prompt on the field definition.

Overview of Validation Codes
Error/Warning Messages
Date Validations
Alphanumeric Validations
Numeric/Monetary Validations
Using Field Input Restrictions

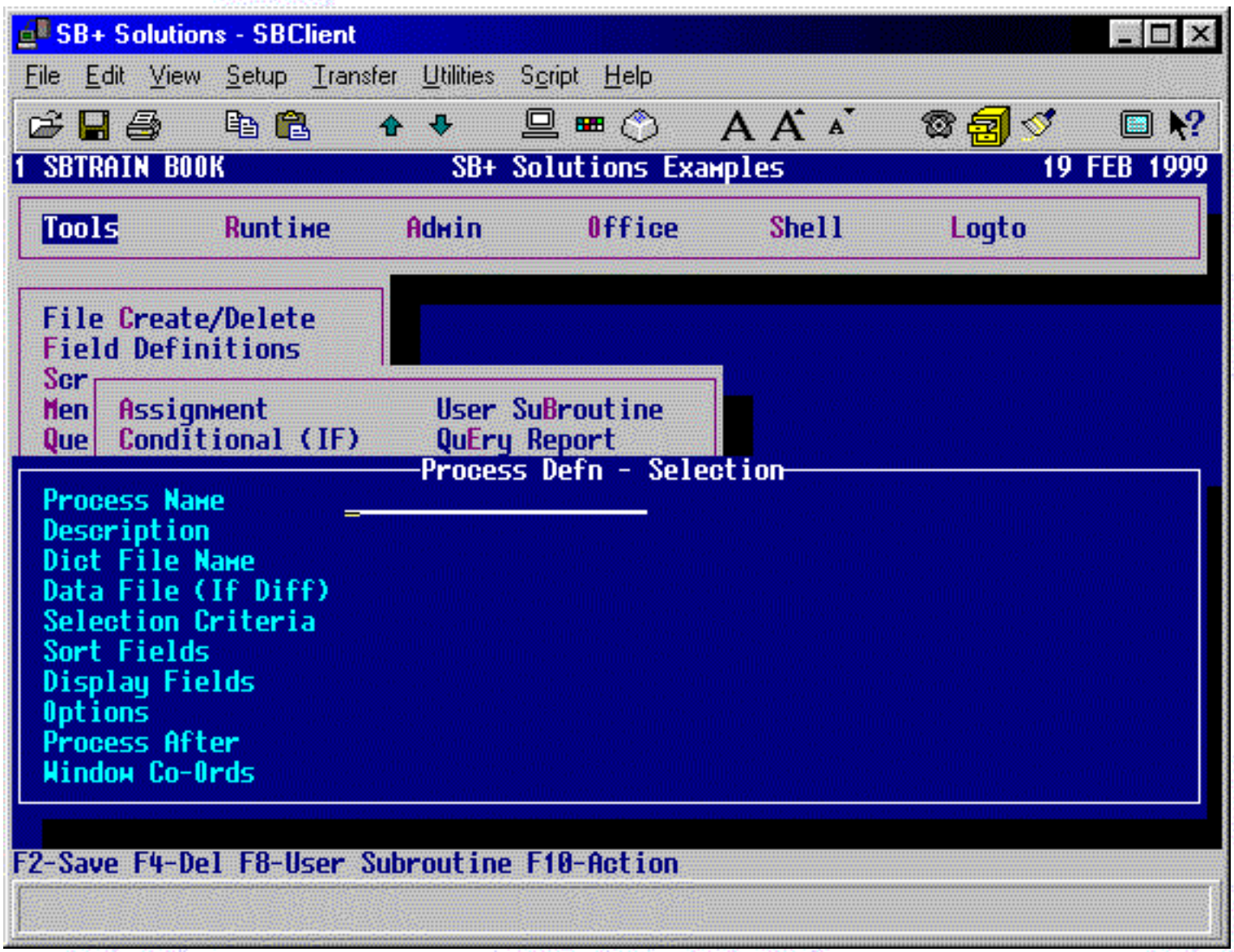
Intuitive Help

One of the finest features an SB+ application can have is an abundance of help. While F1-Help is useful for telling the user what to do, F3-Intuitive Help is often more useful for helping the user make a proper entry.

As a general rule, if a field has a **validation**, there should probably be some form of intuitive help available. This provides a safety net for the user who has entered an invalid value and is unsure about what the valid options are. True, **F1-Help** can be of some value, but there's nothing more user-friendly than simply selecting an option from a list of valid values.

Most intuitive help processes are Selection-type processes. The tool to enter these is called Process Definition - Selection, and is available by traversing through Tools, Process Definitions, and finally Selection in the menu tree. Or, for a more direct approach, simply enter /PD.S at any prompt or menu.

Process Definition - Selection appears as follows:



At the Process Name prompt, enter the name of a process to create or edit. If the process exists, it will be displayed for editing.

The Description prompt allows you to enter a description for the process. This description is displayed when someone presses F3 on the Process Name prompt, and also serves as the screen heading when using the "?" option to prompt for run time parameters.

At the Dict File Name prompt, enter the name of a file which contains fields that may be referenced later in the selection criteria, sort fields, or display fields.

If the records live in a file other than the file named in the Dict File Name, enter the appropriate file name in the Data File (if Diff) prompt. SB+ will then use the dictionaries from the dictionary file, but will select and read the records from the data file as named.

The **Selection Criteria** prompt is very important. This prompt tells SB+ which records are to be selected. There are a multitude of options available with this prompt, a number of which are described in the following sections.

Sort Fields contain a space-delimited list of fields to use for sorting the records to be displayed in the selection box. If you want to sort a field in descending order, append the literal "(D" to the field to be sorted in descending order. Note that the quotes as shown are for clarity and are not a part of the syntax. Additionally, the literal must be touching the field to be affected.

There are also a number of **Options** available for controlling the way a window looks, what is returned from the selection process, and the number of options that can be selected.

The Process After is a slot where you can link a process to modify the values that the user selected before those values are returned to the caller. This allows you to add your own sorting, duplicate check, or other special processing to the selected values.

Finally, the Window Co-Ords prompt allows you to enter specific coordinates for the selection window. (Note that you cannot change the location or size of the prompting window, if used.) This prompt is rarely used, because it allows the standard positioning to be overridden, therefore disrupting the standard look and feel of the application. Should you need to manually define your window coordinates, these window coordinates are entered in the same format as screen window coordinates:

xc,*yc*,*width*,*depth*

Where *xc* and *yc* are the column and row of the upper left corner, *width* is the number of characters that will be occupied by the display box, and *depth* is the number of rows that will be occupied by the display box. Note that the *width* and *depth* parameters do not define the number of columns or rows that are shown inside the box. Instead, they define the coordinates of the box.

From a user standpoint, the importance of intuitive help cannot be stressed enough. With solid, robust intuitive help throughout an application, users are rarely left wondering where to turn.

Two Intuitive Help Processes?
How Do Selection Processes Work?
Selecting From an SB+ Table
Selecting From a File
Selecting From Multiple Files
Selecting From an Existing Saved List
Selecting From a Multivalued List in Common
Selecting From a Multivalued List in an External Record
Multiple Intuitive Helps on a Single Prompt
Selecting a Single Record

Using SB+ Indices in Selection Processes

What Are All Those Options?

Adding F1 Help to the Screen Definition

One of the most powerful features of any software package is concise, effective help. This can be added to your application in the form of F1 help to display help messages, as well as **F3 intuitive help** to assist the user in making selections.

With SB+, you can define up to five levels of help messages to be displayed when the F1 key is pressed. The first four are specific to a single prompt on the screen, and are invoked when F1 is repetitively pressed. The fifth level of help, called "General Help" is one help level that applies to everywhere F1 can be pressed.

The best time to add F1 help is after the screen has been created, but before the extra processing logic (i.e. validation, processing before, processing after, etc.) has been added. This way, the screen allows cursor movement to each of the fields to be documented (while the screen is running) without the potential of interfering skip logic, defaults that prevent cursor advancement, or other such things that can slow down the process of documenting the screen. These things can always be added after the F1 help has been added.

F1 help for a prompt on a screen is stored in the field definition for that prompt. There are, however, two ways to enter this information. The following sections describe the differences between the two:

Option 1: Using the Field Definitions tool

The last prompt on the Field Definitions screen is called "Help Reminder (Y/N)". If you enter "Y" at this prompt, SB+ will prompt you for a help message to appear at the bottom of the screen. Once this message has been entered, the following prompt will appear:

Create Second Level Help (Y/N)

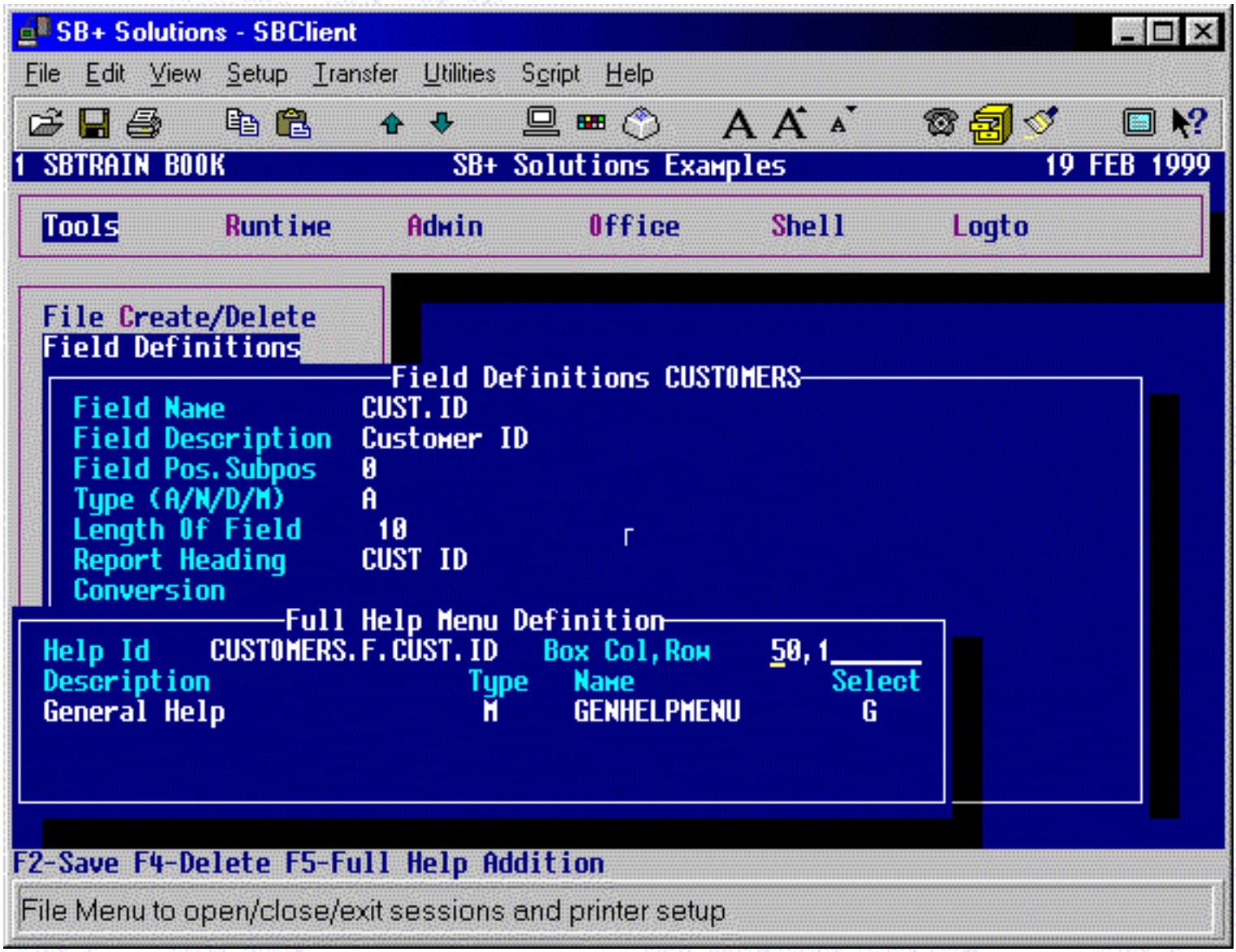
If you enter "Y" at this prompt, SB+ will ask you to draw the box that the second level help is to fit in. As in the Screen Definitions tool, the box is drawn by moving the cursor to the position where you want the upper left corner of the box followed by <cr>, followed by moving the cursor to the lower right corner of the box followed by <cr>. Once the box size has been defined you may then enter the text to appear inside of the box when the second level help
is to be displayed.

There is no limit to the amount of text that can appear in a second level help window. There is one rule of thumb to keep in mind, however. If the second level help is smaller than your window size, SB+ will display the help message and then return the cursor to the prompt so that the user can read the message and enter the value at the same time. If the message is longer than what will fit inside of one window, however, the cursor will remain inside of the window until the user presses <cr> or Escape, at which time the cursor will be returned to the prompt. With this option, the message is not displayed while the user is entering a value into the prompt.

After the second level help message has been entered, SB+ will clear the screen and prompt you to enter the third level help message. This is a full-screen help message, and can be used to describe the prompt in any level of detail you can imagine. If you don't want a third level help message, simply press Escape when the screen is displayed. Unlike the second level help, SB+ does not give you an option before prompting for the third level help, so this is the only way you can tell SB+ that this level of help is not necessary.

If the third level of help is necessary, and you press F2 to save the message(s) you have entered, SB+ will then prompt you for the

fourth level of help, which is by definition a help menu. The following screen will be displayed:



On this screen, define the options you want to be presented in a menu when the user gets to the fourth level help menu. This may include processes to help the user select the right option (similar to a F3 process), or processes to display any amount of additional help.

The name of the menu is assigned by SB+, and while you can change it, if you do so SB+ will not be able to find the menu. Therefore, simply press <cr> on this prompt.

The Box Row,Col prompt defines the upper left corner row and column where the menu is anchored. (The width and depth of the menu will be calculated based on the number of options and the width of the longest option on the menu.) There are really no restrictions to where the menu is anchored, though
your established standards should apply.

In the Description prompt, enter the descriptions of the different menu options. Like a normal SB+ menu, these descriptions will be what the user sees, so be as descriptive as possible.

In the Type prompt, enter "M" or "P" to invoke a menu or process, respectively. See **Chapter 7** for more information about the

other options available here, or simply press F3 for a list of available options.

In the Name prompt, enter the menu or process name to be invoked when this option is selected from the menu. Though the menu or process doesn't need to exist when the menu is created, be sure to have it completed before the software is released for production!

In the Select prompt, enter the single letter that should be highlighted on this menu option. Naturally, the select letter must be a letter in the description, and the same select letter cannot be used for more than one option.

Once the menu has been constructed, press F2 to save it. Once saved, the menu will become your fourth level of F1 help for that particular prompt.

Again, you may not need this much help for a single prompt. Yet, it's nice to have the option when you need it!

Option 2: The /OH1 Process

One of the problems with using the Field Definition tool to enter F1 Help is that you have little to no visibility to the screen when the help is being defined. For the first level help reminder, this poses little problem, but when drawing the box for the second level help, it can be useful to be able to see the screen so you can place the box properly. To help you accomplish this, SB+ has a process called /OH1 (Online Help - starting at the first level) which can be used to define help *while the screen is running*!

To use this process, run your screen normally. At each prompt, enter /OH1. SB+ will allow you to define the F1 Help for each prompt this way. However, note that the help text may not be available right away. Instead, once you've finished defining the help for a screen, exit that screen and restart it. At that time your help messages will appear normally.

Here's a tip: Define F1 help *before* defining validation for your fields. This way, you'll be able to freely move from field to field using /OH1 to add the F1 Help without being stopped to enter a valid value at each prompt.

Using Conversion Codes

A conversion code defines how a particular value is to be displayed. In contrast to a **derived value**, which is used to actually calculate a value, the conversion assumes a value has already been determined, and is used simply to format the value.

Conversions may be entered in one of several locations:

- In the Field Definitions tool, there is a prompt on the main screen called Conversion which allows you to define the conversion code for all places where this field is used.
- In the Screen Definitions tool, F5-Field, F6-Addit allows you to override the field definition conversion for a prompt.
- In the Report Definitions tool, F5-Field allows you to override the field definition conversion for a single field for a single report.

There are also two types of conversions:

- OE conversions are conversion codes which are supported by the operating environment. These include simple date, time, case, numeric, and other such conversion codes. These codes are entered without surrounding parentheses.
- Derived conversions are SB+ expressions which support a wider range of conversion possibilities, such as calling a process, totaling a list of multivalues, and a variety of other useful features. These types of conversions are entered with surrounding parentheses.

With the variety of OE conversion codes and the flexibility of the SB+ expression language when creating derived conversions, there is little to nothing that you cannot do with a conversion.

Standard OE Conversion Codes
Using Derived Conversions
Input Conversions
Reverse Conversions

Using Derived Values

A *derived value* is an expression or phrase that is used to calculate a value for a field. Unlike a **conversion**, which is used simply to format a value, a derived value expression is responsible for actually calculating the value, using other fields or **common variables** as needed. Also, these values are always calculated, never stored.

Derived values can be used in three places in an SB+ application:

- If a derived value is entered in the Field Definitions tool at the **Derived Value** prompt, the expression will apply to all places where that field is used. SB+ derived values must be surrounded by parentheses.
- If a derived value is entered in the Report Writer Definitions tool at the Derived Value prompt, the **expression** will apply only to that field on that report.
- A derived value may be entered in the **Global Equates** tool, which allows the SB+ expression to be referenced by a symbolic name.

Officially, SB+ supports two types of derived values: OE correlatives and SB+ expressions. By definition, OE correlatives are supported by the Pick or Pick-like system, similar to OE conversions. SB+ expressions, on the other hand, are much more powerful. (Off the record, don't bother using OE correlatives in an SB+ application; SB+ derived values are far and away more predictable, portable, and flexible.)

There are many uses for a derived value in a typical screen or report application. For example, when someone enters a customer ID you may want to read the customer record and display the customer's name beside the prompt. A derived value field can easily be created to do this. Or, if you want to display a sum of the prices shown on a sales order screen, a derived value field can be used here as well. In fact, only two things limit the capability of a derived value:

- Inadequate file structure, and
- Your imagination.

Actually, we know you have no shortage of imagination, so if your file structure is adequate you'll have no problems constructing a derived value to calculate literally anything.

Reading a Description From Elsewhere
Reading From Multiple Elsewhere
Creating a Table Derived Value
Calculating Extended Totals **Summing Extended Totals**
Calling a Process
Calling a Subroutine

Special Processing for Single Valued Fields

There are several little tricks you can do with single valued fields that don't really fall under the standard categories of **default**, **validation**, or **intuitive help**. Using various combinations of these features, a well-placed process, and a little magic, there are some incredibly useful things that SB+ will do for you to help bullet-proof your application. The following sections describe a few of these...

Limiting the Length of a Field
Skipping a Prompt
Creating an Automatic Recalculation Prompt

Special Processing for Multivalued Fields

Similar to single valued prompts, there are several little tricks that you can do with multivalued fields that don't really fall under the standard categories of **default**, **validation**, or **intuitive help**. Using various combinations of these features, a well-placed process, and a little magic, there are some incredibly useful things that SB+ will do for you to help you create a well-rounded application.

Incidentally, other than a minor change to the skipping logic, all of the **special processing for single valued fields** works for multivalued fields as well. Therefore, even though this book separates special processing for single valued fields from multivalued fields, the special processing for single valued fields works for both.

Skipping Multivalued Fields
Limiting the Number of Multivalues
Updating Hidden Multivalues
Updating Humongous Multivalue Sets
Interleaving Multivalue Prompts
Restricting Multivalue Prompts Using Multivalue Restrictions
Restricting Multivalue Prompts Using Field Input Restrictions
Calling a Process On Multivalue Delete
Displaying A Multivalue Page Indicator
Implementing Character-Based Check Boxes and Radio Buttons
Implementing a Word-Wrapped Comment Field

Process Tricks

With the flexibility of paragraphs and **BASIC**, there is little you cannot do with processes. There are a few things, however, you may need to do again and again. This section describes a few of those common, day-to-day processes.

One of the somewhat hidden strengths of SB+ lies in the way that processes are multidimensional. In other words, one process slot can call one process which can call several others, each of which can call several others, and so on. In the end, though the **Process After Read** on a screen definition may call only one process, several hundred other processes may be called as a result.

This has numerous benefits. First of all, it allows the developer to break up large, complex processes into smaller and more understandable fragments. With smaller processes, programming is easier and more focused. Also, the way a particular process is fragmented can actually help make the entire process easier to debug. Second, because all of the processes must eventually converge into a single process called from a single process slot, there is an implicit focus on the inputs to, and the outputs from, each process. Knowing this (and with good comments to illustrate what is going on), creating, debugging, and maintaining an application can be made significantly easier. Finally, because any process can call literally any other process from any number of different process slots, standard processes can be created to handle the most mundane and repetitive tasks, thereby enhancing the development environment as a whole.

Handling Conditions
Pre-Delete Verifications
Verifying a Record Can Be Edited
Updating Hidden Values in the Record
Updating Several Records Simultaneously

Structuring Processes

When creating an SB+ application, the content of the processes you create is, naturally, of paramount importance. However, one cannot overlook the importance of the structure of the processes.

It is theoretically possible to write all of the code for any given process slot in a single process, such as a paragraph or BASIC subroutine. For many techniques, this is certainly an appropriate thing to do. However, there are three process slots on a screen definition which can be uniquely complex:

- Process After Read
- Process After Screen Accept
- Process After Update

With these three process slots in particular, the structure of the process is very important. Proper structure contributes to code that is easy to read, trace, and debug. In contrast, improper or inadequate structure can make the process difficult to understand, trace, or debug. In the end, easier reading and debugging means less time wasted scanning through complex processes, which translates into some real financial savings!

The following sections describe one way to structure these particular processes. It is intended to demonstrate one particular technique, and as always should be adapted to your own particular project, standards, and style.

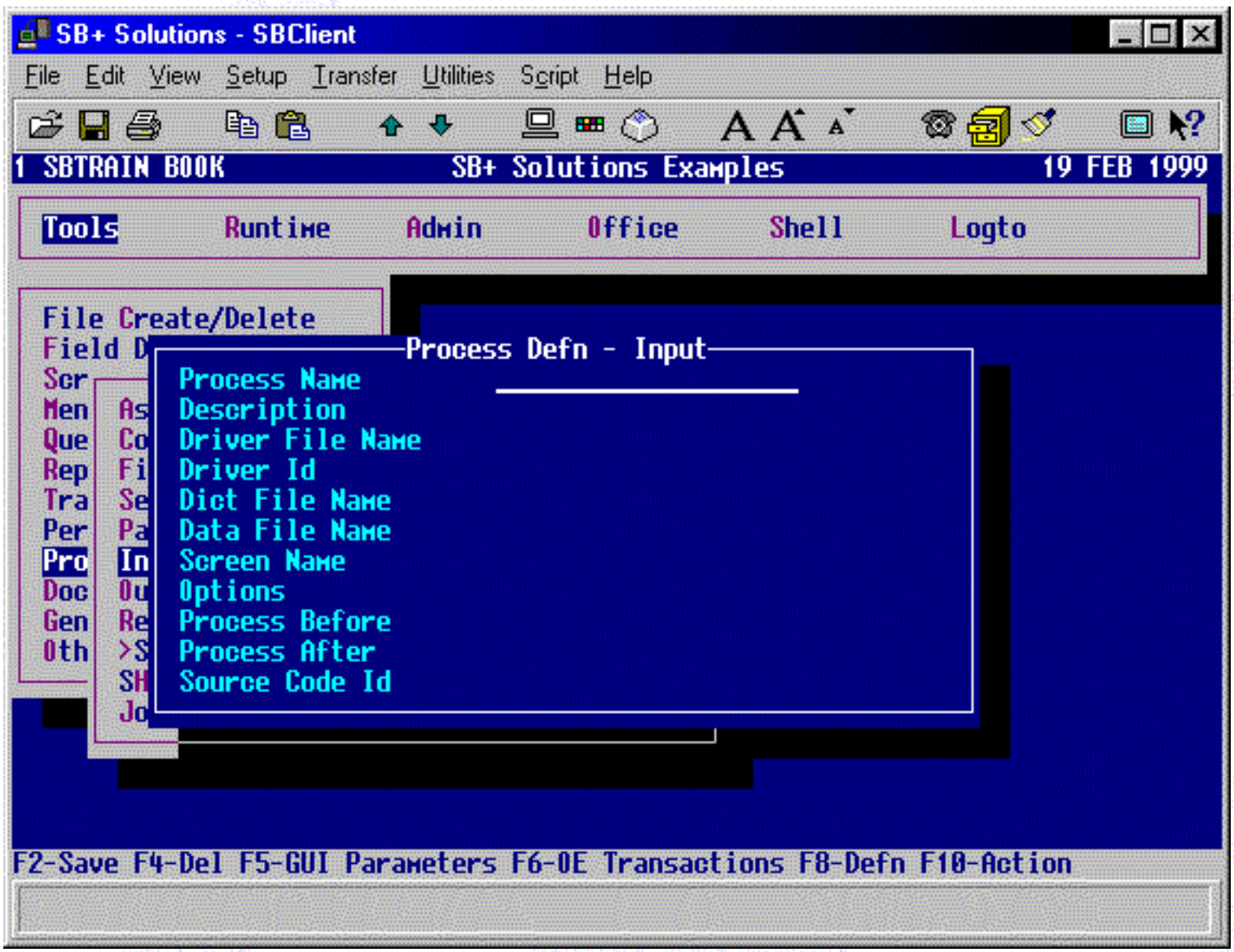
Structuring a Process After Read
Structuring a Process After Screen Accept
Structuring a Process After Update

Using Split Dictionary/Data Files

In a typical SB+ application, data entered into a screen will be stored in the file where the screen definitions and field definitions are stored. However, in certain applications, particularly those that are translated into several different languages, it can be necessary to read field definitions and screen definitions from one file, while updating the records entered into the screen into another file.

To do this, we need to tell SB+ that the data file is different from the dictionary file for a particular screen. Oddly enough, this is not a part of the screen definition, but instead is a part of the input process.

The Process Definition - Input tool is used to enter and edit the actual input process. Actually, this tool is rarely used to enter the original process -- when you press F2 in the Screen Definitions tool, select None-Execute Now, and then follow the prompts to run the screen, SB+ automatically builds this record for you. Though you rarely see it, you can still edit the process record. To see this tool, select Tools, Process Definition, and then Input, or enter /PD.I at any menu or entry prompt. This tool appears as follows:



When this tool is running, you can enter the name of the input process to create or edit.

Once a record is displayed in this tool, you may change any of the parameters that are displayed. For the purpose of this discussion, we'll change the field labeled Data File Name.

By changing the value in this field to the name of the file where the information is to be stored, the data will be written to this new file, while the screen definition and field definition will still be read from the dictionary file as named.

The value in this prompt may also be an **expression**. This allows the data file name to change dynamically, based on information in the **common variables** at the time the screen is invoked. For example, if we have a dictionary file called GL.BALANCE, and the data file is called GL.BALANCE.yy (where yy is a year, stored in, say, @PARMS(1)<12>), we can construct a data file name that appears as follows:

('GL.BALANCE.':@PARMS(1)<12>)

Note the enclosing parentheses. This tells SB+ that the file name is an expression instead of a constant. When the screen is first invoked, SB+ will evaluate the expression and will open the file based on the result of the expression.

Numerous other variations on this theme exist, but the fundamental thing to remember is that the data file name, if different from the dictionary, can be either an expression (enclosed in parentheses) or a literal. The dictionary name, however, must always be a literal name.

Using Logical Files

Usually, when you create an entry screen program using SB+, you **create the file**, **create the fields**, and then **create the screen definition**. It is assumed that the file you create will contain the information entered on the entry screen, though notably other files may also be updated.

But what if you're entering a type of transaction which doesn't need to be saved? What if a bunch of other files need to be updated using the information entered, but the record itself can be discarded? For situations like this, you can create a *logical file* in SB+.

Logical files are special types of files which contain no information of their own. Instead, the logical file contains references to any number of other files. When a screen is created on a logical file, SB+ loads no information from the logical file itself -- all of the information is read from the other files as defined. Also, when a record is saved on a screen in a logical file, nothing is written to the logical file itself -- instead, all of the other files as defined are updated. In short, logical files provide a convenient way for SB+ to automatically update multiple files without forcing you to write the code for each update.

Creating a Logical File
Creating a Screen in a Logical File
Resequencing Fields in a Logical File
Regenerating Fields for a Logical File
Considerations for Logical Files

Key Processing Techniques

In SB+, there are several different ways that record keys can be built and used. Keys can have one or many parts, can be hidden or seen, can be sequentially assigned by the system, or any combination of these techniques.

With simple screens, SB+ intrinsically "knows" which field is the key field, and therefore knows when to read the record. If the first field on a screen definition is defined as attribute zero, SB+ identifies this as the key and knows to read the record immediately after a value has been entered into the field.

However, as briefly explained above, this is only one of many ways that keys can be implemented in SB+. With a little programming and imagination, complex keys can easily be constructed for your records. The following sections describe some of the more common key processing techniques.

Implementing a Hidden Key
Implementing a Sequentially Assigned Key
Implementing a Multi-Part Key
Implementing a Virtual Key

Subscreens

Prior to version 3.x of SB+, there is a 36-prompt limit on each screen. While this limit has since been removed, having more than 36 prompts on a screen can be unwieldy. Instead of trying to fit all the prompts for an application on a single screen, it is often better to separate the prompts into functional groups on smaller screens (called *subscreens*), each called from one main screen. Any number of subscreens can be called from one main screen, and any one subscreen can, itself, call any number of other subscreens. In the end, a single application can be comprised of literally dozens of screens, though the main screen itself may be very small and compact.

There are several different ways that subscreens can be implemented, such as:

- A subscreen can be invoked by the press of a function key (called a manual subscreen);
- A subscreen can be invoked automatically before or after a field has been entered (called a popup subscreen);
- A subscreen can be invoked automatically before or after a field has been entered based on a condition (called a conditional popup subscreen).

Applications can use any mix of these techniques, depending on the functionality required.

Subscreen Fundamentals

Regardless of the technique used to implement a subscreen, there are five fundamental tasks that must be done in order to tell SB+ that a particular input process is a subscreen. These tasks are:

Create the screen definition

Creating a screen definition for a subscreen is not any different from creating a main screen, except that the subscreen will not have a key prompt. Instead, it will contain only the fields to be updated in the record or work variable.

Write Record = (null)

On the subscreen's F6-Params screen, there is a prompt called Write Record (Y/N). Looking at the prompt, it appears there are only two values that can be entered at this prompt -- "Y" and "N". However, there is actually a third possibility, which is the value we want to use for implementing a subscreen. Instead of entering a "Y" or "N" at this prompt, enter a backslash "\". This will set this prompt to null.

For subscreens, we don't want SB+ to open a file, read a record, or write a record. Instead, we'll simply use the current @RECORD, @KEY, and @WORK that was loaded on the main screen. The null entry at this prompt (as opposed to "N") tells SB + to behave this way.

Remove F4-Del on Subscreen

As a standard, the main record should be deleted only from the main screen. Therefore, the function key definition on the subscreen should be modified to remove the F4-Del key.

To do this, use the Screen Definitions tool to view the subscreen definition, then use the F7-Fnkeys key to display the defined keys for the subscreen. Once the keys are displayed, arrow down to the F4 key and enter a backslash "\" for the description and also for the process to be called on this key press.

Most people choose not to redefine this key on the subscreen, as it can be confusing for the user to have the F4 key mean something other than "Delete". However, SB+ doesn't specifically impose any restrictions on redefining this key. In the end, this becomes another issue to be dictated by the
project, standards, and style.

Set the Subscreen Option on the Subscreen Process

Once the screen definition for the subscreen is complete, you should use F2 to save the screen, and then use None-Execute Now to run the screen once. This will create a process record for the screen. Once the screen is running, press Escape -- you don't want to try entering information on a subscreen running out-of-context!

After the process record is created, we need to add an option to the process record to tell SB+ that this is a subscreen. This can be done using the following sequence:

- From the Add to Menu menu, select None-Execute Now.
- When asked Screen for Input/Output, enter I.
- When the process name is displayed, press the Insert Char key, and add the following to the beginning of the process name:
  /PD.I (and press the space bar once)
- press <cr>.

Once the Process Definition - Input screen is displayed and the proper process is loaded, arrow down to the Options field and add the following option:

S

This little option is of paramount importance. Normally, SB+ saves common memory to a holding area prior to calling a screen, and restores the original memory values when the called screen returns to the caller. On a subscreen, we don't want SB+ to do this, as we want the subscreen to share the same variables with the main screen. This option tells SB+ that the subscreen will share common memory with whatever calls it. Therefore, changes made by the subscreen will be reflected in the record that is being updated on the main screen.

Link the Screens Together

The last step in implementing a subscreen is to tie the main screen and the subscreen together. This can be done in a variety of different ways, explained in the following sections.

Implementing a Manual Subscreen
Implementing a Popup Subscreen
Implementing a Conditional Popup Subscreen

Linked Screens

In contrast to **subscreens**, where you must do something to connect the screens together, SB+ supports a feature called linked screens where SB+ does the linking for you.

With linked screens, once all of the prompts on one screen have been answered, a second screen appears automatically. After all of the prompts on the second screen have been answered, a third screen appears. This continues until all of the linked screens have been executed.

The user interface for linked screens can be very much different from a main screen/subscreen application. In particular, when implementing a subscreen the record can be saved from the main screen only. With linked screens, however, the record can be saved from any of the linked screens. Also, where function keys are used to move from screen to screen with a main screen/subscreen application, the Page Up and Page Down keys are used to move from screen to screen in a linked screen application. These differences aside, a linked screen can update fields in the record exactly as a main screen/subscreen application does.

To implement a linked screen application, simply name your screen definitions suffixed with a number. If the first screen is called ENTRY.1, the second is called ENTRY.2, and the third ENTRY.3, these three screens will be automatically linked together when ENTRY.1 is invoked. Therefore, after the last prompt on ENTRY.1 has been answered, ENTRY.2 will display. When the last prompt on ENTRY.2 has been answered, ENTRY.3 will display. All this happens without any additional programming on your part.

Unlike using manual or popup subscreens, linked screens are always processed sequentially, and the user doesn't have many options for moving from page to page. Because of this, many developers prefer connecting subscreens either through function keys or with conditional popups.

Miscellaneous

There are a number of topics which just don't fit anywhere else -- like these:

Input Process Options
Process Slot Overview
Input Screen Prompt Cycle
Understanding Input Process Drivers

Query Screen Fundamentals

Once information has been saved in files, it needs to be presented to the user in some fashion. Query screens are one way that information can be displayed.

A query screen is similar to an entry screen in many respects. Both are created using the Screen Definitions tool. Both take a key value, read a record, and display the contents of the record. However, unlike an entry screen, the query screen does not allow the record to be updated.

There are two different types of query screens that can be created with SB+:

- **Output processes** take a key value, read a record, and display fields from the record. All entry prompts on the screen (other than the key) become display-only prompts. The cursor moves to the key prompt and to the bottom of the screen only, not from prompt to prompt.
- **Non-amendable input processes** take a key value, read a record, display fields from the record, and then allow the cursor to be moved through the screen. None of the displayed values can be changed, but the cursor can be moved to each input prompt.

Because of the similarities between entry screens (input processes) and query screens, most of the techniques described in **Chapter 3** can be used in the construction of query screens. This chapter, then, will focus on new techniques that apply specifically to the implementation of query screens.

Don't Use a Shared Screen Definition

Because entry screens and query screens both require a screen definition, it is theoretically possible to create one screen definition that is called both as an entry screen and also as an query screen. While possible, this is NOT recommended.

Visually, entry screens should be structured in such a way as to maximize the productivity of the person entering the information. For example, if the information is being entered from a form, the input screen should be organized like the form to help the user enter the information quickly and efficiently.

In contrast, query screens should be structured in such a way as to maximize the productivity of the person who needs the information. For example, if the query screen is used in a customer service role, the information most requested by the customer should appear first or in a place that is visually distinct.

From a more technical perspective, the structure of a screen definition used for entry is somewhat different from a screen definition used for query. In all but the simplest input process, there will be processes called after the record has been read, before prompts, after prompts, and from function keys. These processes may serve a very valuable function for the entry screen, but are essentially baggage for the query. Also, if a screen definition is shared between an entry screen and a query screen and the screen is modified for some additional query functionality, the effectiveness of the entry screen may be compromised.

In summary, if you want to have a query screen that looks like the entry screen, copy the screen definition used for entry to a new name and create a new process to run the query. While it adds another screen definition to the application, it will inevitably save you headaches in the long run.

To Prompt or Not to Prompt?

When setting up a screen for query, there are basically two ways to handle the processing for the record key:

- The key can be entered at a prompt on the screen, or;
- The key can be extracted from **common memory**.

When the key is entered at a prompt on the screen, the query can be a self-contained entity. In contrast, when the key is extracted from common, the query is reliant upon some external pre-process in order to properly configure common memory. While this latter approach may seem like more programming (which it is), it also provides a few more bells and whistles that can be useful in linking query screens together.

To set up a screen to prompt for the key value, simply place a key field on the screen as an input field and make it the first prompt in the prompt sequence. SB+ will figure out what to do to prompt for the key and read the record. Multi-part keys can also be implemented using the same techniques as for entry screens.

To extract the key from common memory, the **hidden key technique for entry screens** as described in **Chapter 3** can be used. Essentially, by placing an expression in the Key Value (If Not Inp) slot on the F6-Params screen in the screen definition, SB+ will calculate the key instead of prompting for it. Of course, to use this, a key field cannot be present on the screen.

As an added bonus, if the expression is:

```
@VALUE
```

...and the contents of @VALUE is multivalued when the query process is invoked, SB+ will treat the expression as a list of keys and will extract each key one-by-one for display. This has important implications: If a calling screen has a multivalued list of keys to other records, simply by moving the list into @VALUE and using this technique on the called query, each of the records can be displayed sequentially.

For example, assume we have a customer query application. In the customer record, we have a multivalued list of sales order keys in attribute 31, as follows:

```
123]456]234
```

Each of these three keys points to a sales order that has been entered for this customer. If we want to display all of the orders for a customer in sequence, we can call the following paragraph from the customer query screen:

```
@VALUE = <POS(SO.XREF)> ;* Move the list to @VALUE
EXEC 'O*ORDER*INQ.A' ;* Call the query
```

Interestingly, if three keys are loaded into @VALUE this way, the user will view three records. If they want to exit the screen after the first or second record, they cannot press Escape to do so. SB+ supports a Quit key which can be used to abandon the remainder of a list, but this key isn't always defined in the terminal definition.

Creating an Output Process

Fundamentally, there are many similarities between an input process and an output process. Both process types display a screen, and therefore both require a screen definition. Because a screen definition is required, both process types require field definitions, function keys, and many other typical screen definition resources.

But there are also many contrasts between the two process types. The input process can create new records, amend existing records, and delete records. The output process, however, can only display the contents of existing records. Prompts on an input screen use **defaults**, **validations**, **intuitive help** processes, and processes called before and after the prompt. Prompts on an output screen, however, need only a field position and an optional conversion to display a value.

SB+ is smart enough to overlook most of the unnecessary information defined in a field definition on an output screen. Therefore, if a particular field with validation and intuitive help for an input screen is placed on an output screen, the validation and intuitive help will be ignored. Oddly enough, however, processes called before or after prompts will not be ignored.

To build a query screen using an output process, simply follow the instructions in **Chapter 3** for creating an entry screen. When the screen is complete, press F2 to save it, then select None-Execute Now from the menu. At this point, when SB+ asks you:

Screen for Input/Output (I/O)

Respond by entering "O" instead of "I". This will tell SB+ to create an output process for the screen definition. SB+ will then display the process name , which you can simply press <cr> to accept. Finally, SB+ will ask for a description of the process. As with an entry screen, this description will serve as the window title, and will be displayed in the center of the top line of the query window. Once this has been entered, the query screen will be executed and, more important, the output process will have been created.

Creating a Non-Amendable Input Process

Simply stated, a non-amendable input process is an input screen that doesn't allow anything to be changed. The cursor moves through the prompts just like a normal entry screen, but if the user attempts to change a value, SB+ ignores the keystrokes.

Unlike a standard output process, a non-amendable input process allows the cursor to be moved from prompt to prompt around the screen. With this flexibility, some very interesting functionality can be added to the screen that would otherwise be impossible. However, this flexibility comes with a price. Some users get confused regarding which screen is for entry and which screen is for query, particularly when the screens are reasonably similar.

To create a non-amendable input process, create a standard entry screen as outlined in **Chapter 3**. When the screen is complete, press F2 to save it, select None-Execute Now from the menu, and when SB+ asks:

Screen for Input/Output (I/O

Enter "I" to create a standard input process. SB+ will then display the process name, which you can accept by pressing <cr>. Finally, SB+ asks you to enter a description for the process. This description will then serve as the title for the window, shown centered on the top line. Once the description has been entered, the process will be executed. More important, however, the input process will have been created.

But we're not done yet. The steps to this point have merely created an input process. We now need to tell SB+ that this input process is for display only, or in SB+ parlance, non-amendable. To do this, we need to add the "O" option to the Process Definition - Input record for this screen. The easiest way to do this is as follows:

- Get to the menu where you select None-Execute Now. If you're editing the screen, press F2. If the screen is running, press Escape.
- Select None-Execute Now.
- When asked to create either an input or output process, press <cr> to accept the default (I).
- When the cursor is on the process name, press the Insert Char key.
- Type /PD.I <space> and press <cr>. (<space> means simply to press the space bar.)

This sequence of events will invoke the Process Definition - Input tool and will automatically load the record for the process we're working on. Once this is done:

- Enter /8 to move the cursor to the **Options** field.
- Add the letter "O" to the end of any value that may currently be shown in this prompt.
- Press F2 to save the record.

Once this is complete, the non-amendable input process is complete, and can be called exactly like any other input or output process -- through the menu, using the slash, or from another process elsewhere in the application.

Cruising, Zooming, and Other Stupid Pick Tricks

A number of years back, when Dick Pick introduced the Update Processor for Advanced Pick, he introduced a few new terms for the industry. These words, though I admit I still don't fully know what they mean, were "cruising", "zooming", and "double-clutching". Fortunately for the Pick industry, the words never caught on. Unfortunately, neither did the concepts.

The fundamental basis of Dick's idea, I believe, was to build applications that could move fluidly to other applications and back using simple key presses. With this structure, any application in a system could invoke any other application in the system which could invoke any other application in a system, etc. until eventually all applications were available from every other application. Think about it: No matter where you are in the software, every other program in the system is available simply by pressing a few keys. The potential is truly mind-boggling.

Oddly enough, while these concepts formed the basis of Dick's Update Processor, it was SB+ who provided the tools to truly master the technique. Using some simple process magic, it is incredibly easy to provide this functionality in your own software.

To see how this can be used, consider an application with an order query screen. For each order, we have a customer code, carrier code, several product codes, a payment type code, and a salesman code displayed on the query. If the user wants to see more information for the customer, they can press a function key and the customer query screen is displayed. If the user wants to see more information for the carrier, such as rate tables or a phone number, the information is just a key press away. If the user wants to see a commission summary for the salesman, another function key can be pressed. Following this example to the next level, once the customer query screen is displayed, the user can press a key to see the customer's account balance, shipping address, or payment terms. On the salesman query screen, other function keys may be assigned to see the commissions paid year-to-date, phone numbers, or even perhaps a work schedule for the next couple of months!

"Cruising" "Zooming" "Double-Clutching"?

Special Processing for Query Screens

For the most part, query screens can be created using the same techniques as entry screens. There are, however, a few specialized techniques that can be used to maximize the functionality of query screens:

- A list of keys can be passed into a query, and the user can page forward or backward through the list;
- Information from other files can be loaded into common memory for the query, and;
- Records can be "tagged" with a date/time/user stamp to denote the last time the record was viewed.

The following sections describe these techniques in more detail.

Paging Through a List of Keys
Loading Additional Information for the Query
Updating a Viewed Record

Miscellaneous

There are a number of topics which just don't fit anywhere else -- like these:

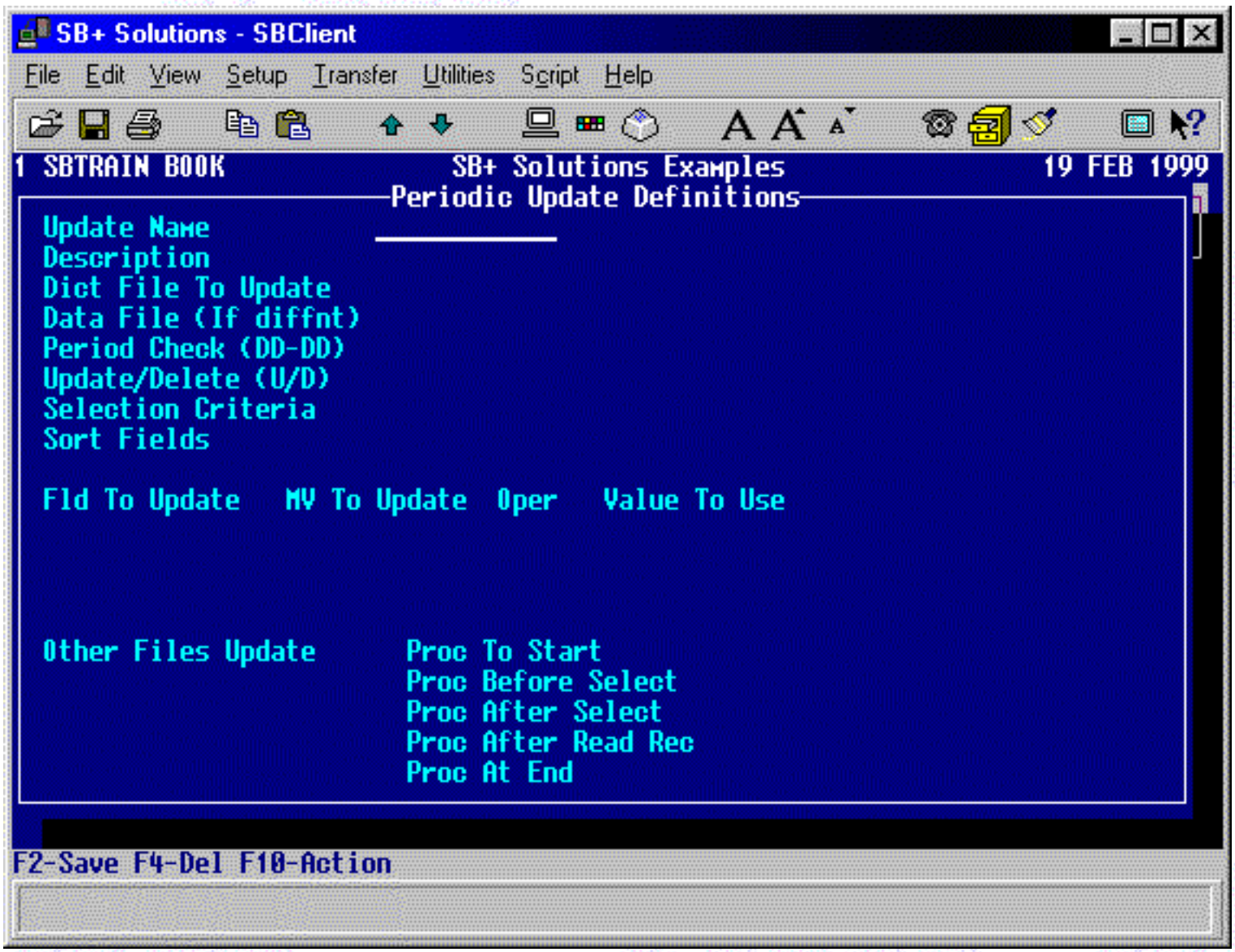
Output Process Options
Process Slot Overview
Understanding Output Process Drivers

Introduction to Periodic Updates

A *periodic update* is a type of process that selects a group of records and processes them one by one. As each record is processed, either the record itself may be changed or the record may be used to update other records.

Like many other types of SB+ processes, your efforts are not spent in the creation of the periodic update process itself. Instead, your efforts will be spent on the creation of a periodic update definition, and then SB+ can create the periodic update process to call the definition.

The Periodic Update Definitions tool may be accessed by selecting Tools, then Periodic Update Definitions from the main menu. Or, for a more direct approach, /PUD may be entered instead. Once the tool has been selected, it appears as follows:



Like all definitions, a periodic update must have a name. Typically, the name of the periodic update *definition* will match the periodic update *process* which calls it, so you can use your process naming standards for this type of definition. Next, the periodic update definition must have a description. This description will be displayed following the word "Performing" when the periodic update is executed, so try to create a description that makes sense in this context.

Next, the tool asks for the name of the dictionary to be updated. This is a required entry, and defines the name of the file where field definitions exist. If any field definitions are referenced in the **Selection Criteria**, Sort Fields, or Fld To Update, or Value to Use fields, they must exist in the dictionary named here.

If the file where the actual data is stored is different from the file where the field definitions are stored, the name of that data file must be entered into the Data File (If diff) prompt. This tells SB+ the name of the actual data file to be selected and read from. If the data and fields are in the same file, this may be left blank.

The Periodic Check prompt allows you to specify a minimum and maximum number of days that must elapse between runs. For example, if an update can be run only weekly, "7-7" may be entered. Or, if the update can be run only at the end of the month "28-31" may be entered. More commonly, "0-0" is entered here so that the update can be run anytime without this restriction. If necessary, other restrictions can be implemented prior to the calling of the periodic update process, or as a part of the Proc To Start. Finally, a dash "-" may be entered here instead of "0-0" which will cause the "OK To Continue" prompt to be suppressed.

The Update/Delete (U/D) prompt defines whether this periodic update will update records or delete them from the file. Enter "U" for an update process, "D" for a delete/purge process.

The Selection Criteria defines which records will be selected for this update. Like all Selection Criteria prompts in SB+, there are a variety of different options available here. See "**Understanding Selection Criteria**" in **Chapter 8** for an overview of the options available here.

If the list of records to be processed is to be selected in a certain order, the sort fields must be entered at the Sort Fields prompt. Any number of fields can be listed here, separated by spaces.

The next group of prompts (Fld to Update, MV to Update, Oper, and Value To Use) are used to define which fields in the main record are to be updated, and how each should be changed. This is a scrolling multivalued set of fields, so any number of fields can be specified here.

In the Fld to Update prompt, enter the name of a field to be updated. This field must exist in the dictionary specified earlier. If the field is multivalued, you must tell SB+ which value is to be updated by entering a value in the MV to Update prompt. The following rules apply to this prompt:

- If the MV to Update is zero or null, the entire field will be updated based on the operator and value to use.
- If the MV to Update is a quote-quote (i.e. ""), the value number calculated for the previous row will be updated. Of course, when you use this option, it's important to be sure that a previous row has calculated a multivalue position!
- If the MV to Update is numeric and non-zero, that specific value will be updated.
- If the value entered in the Oper prompt is one that will automatically calculate the value position, such as "IAR", or "IAL", etc., this prompt may be left blank.

There are a variety of operators available for each update in the Oper field, as follows:

+	Add the Value to Use to the field.
-	Subtract the Value to Use from the field.
*	Multiply the field by the Value to Use.
/	Divide the field by the Value to Use (**with** rounding).
| [vertical bar]	Divide the field by the Value to Use (**without** rounding).

=	Set the field equal to the Value to Use. (same as R)
I	Insert the Value To Use into the multivalued field at the position specified in the MV to Update. Watch out here -- this will insert values into a field whether or not the field is defined as multivalued!
D	Delete the value in the field as defined by the MV to Update. The Value To Use is ignored.
R	Set the field equal to the Value to Use. (same as =)
IAL	Locate the Value To Use in the field in ascending left order, and insert the Value To Use into the multivalued field at the proper position to maintain sorted order.
IAR	Locate the Value To Use in the field in ascending right order, and insert the Value To Use into the multivalued field at the proper position to maintain sorted order.
IDL	Locate the Value To Use in the field in descending left order, and insert the Value To Use into the multivalued field at the proper position to maintain sorted order.
IDR	Locate the Value To Use in the field in descending right order, and insert the Value To Use into the multivalued field at the proper position to maintain sorted order.
A	Aging: See the F1 Help for this prompt for more information about this feature.

The Value To Use is a standard SB+ expression that defines how the field will be changed, as explained earlier. Anything you can do in an SB+ expression can be entered here, including simple calculations, reading from various files, calling a process, or even some combination of these.

The Other Files Update prompt allows you to define other files that will be updated for each record in the selected list. For example, if the goal of the periodic update is posting payments, you may want to update the main record to denote that the payment is posted and also update other files (invoices, A/R balance, customer credit limit, etc.) accordingly.

For each row in this prompt you may enter one file name to be updated. You may not, however, define the actual updates to this file yet. This section is merely for describing what files are to be updated. How these are to be updated will be defined when you press F2.

The Proc To Start is a process slot that allows you to call a process before the periodic update starts. This process can then verify whether the update should be run and act accordingly. If the process sets @RTN.FLAG to "1", the update will be skipped. Normally, folks use this prompt to verify that the process should be run and then display a message to tell the user what's going on (with a cancel option).

The Proc Before Select is a process slot that allows you to do some processing prior to the selection of records for the update. This process slot is useful for clearing work variables or work files.

The Proc After Select is a process slot that allows you to do some processing after the selection of records, but before any of the records have been processed. Like the Process Before Select slot, this slot is useful for clearing work variables. However, because there will be an active select list when this process is executed, there should be no external TCL commands (i.e. CLEAR-FILE...) executed from this slot.

Process After Read is the most useful slot in a periodic update. Basically, this slot allows you to perform any processing imaginable for each record being processed. For each record selected, the **common variables** @KEY and @RECORD are set with the current key and record, respectively, and you can use this information in any capacity you need in order to either update the main record, update other records, or accumulate values into common memory.

Many people find that the Fld To Update and corresponding fields are too limited. Often a field must be updated under one or

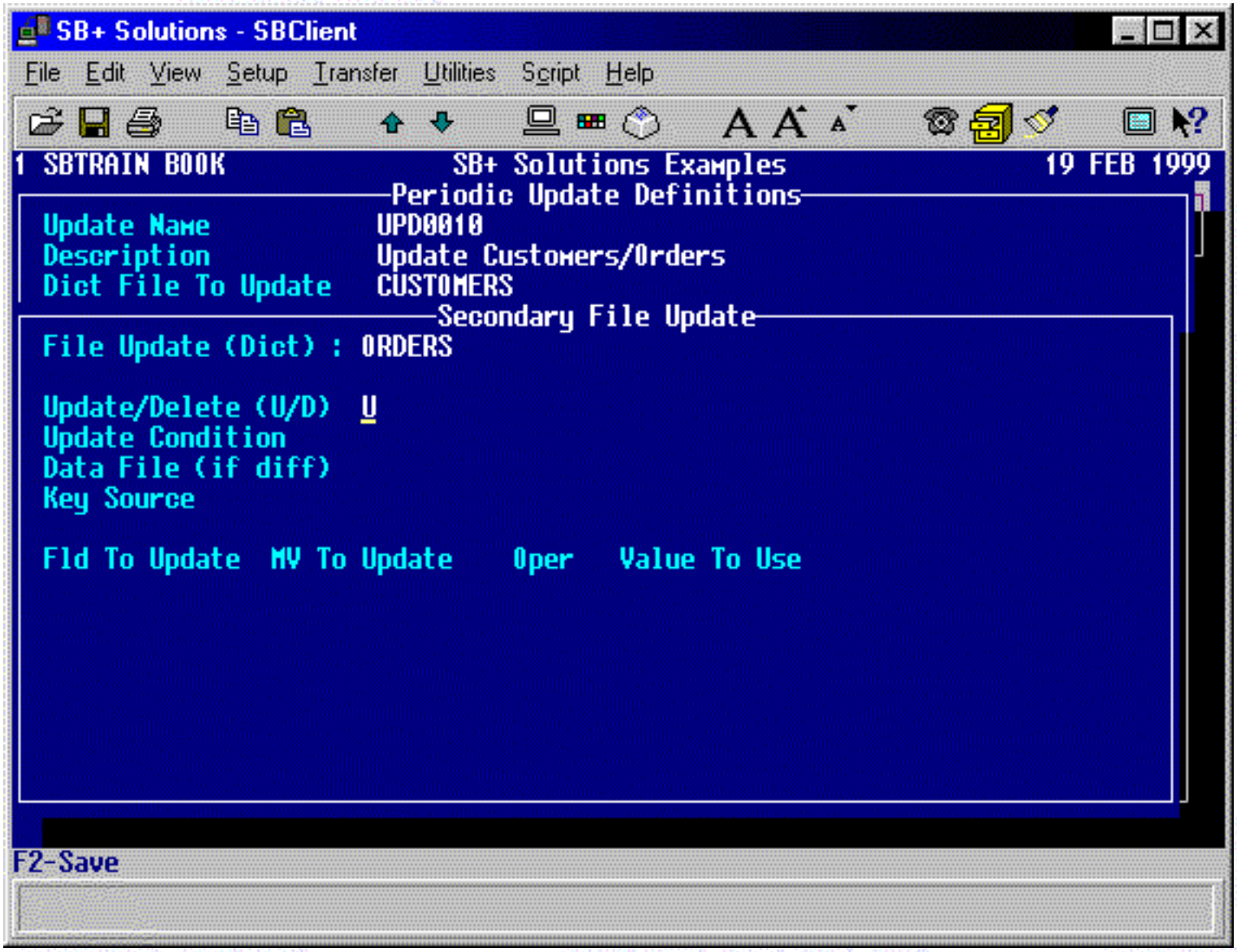
more conditions, and testing these conditions inside of the Value to Use can be frighteningly complicated. Therefore, instead of updating fields in the record using the Fld To Update and corresponding fields, the Process After Read can be used instead. This slot can then call a paragraph with properly structured conditions for updating the record.

Important Note: When the main record is updated via the Process After Read and the Fld To Update is blank, the periodic update will not write the record. The record must be written as part of the Process After Read.

Of course, this process slot can also be used for updating other files, either in place of or in conjunction with the Other Files Update prompt.

The Proc At End process slot allows you to call a process at the end of the periodic update. This process slot is typically used to display messages as to the status of the update, or reset various common memory areas that were changed as a result of the update.

When you press F2, if there are any files listed in the Other Files Update section, the following screen will display:



This screen allows you to define how the other file (called a "secondary file") will be updated.

The first prompt, "Update/Delete (U/D)", asks if a record in this file will be updated or deleted. This works like the same prompt

on the main screen.

The "Update Condition" prompt allows you to enter an expression which will evaluate whether the secondary record should be updated. This condition is evaluated after the secondary record is read, but prior to any changes being applied. If this expression results in a non-zero value, the update will occur. If the expression returns zero, the update will not occur. This is particularly valuable when the key expression is multivalued, as it allows each secondary record to be independently evaluated before being updated.

If the data file is not the same as the file name shown at the top of the box, you can enter the name of the actual file to be updated at the Data File (if diff) prompt. The name entered here can be a literal file name, or if necessary, an expression enclosed in parentheses.

The key source, however, is one of the most powerful prompts on this screen. At this prompt you may enter any valid SB+ expression. If the expression results in a multivalued list, each value in the list is considered a separate secondary key and each record will be updated separately. Of course, if the expression results in a single value, the single record will be updated.

The lower half of this screen defines the actual updates that will be done to each secondary record, and oddly enough, looks very similar to the Fld To Update section on the main screen. In fact, this section works exactly like the Fld To Update section on the main screen with a couple of exceptions:

- Fields can be referenced by both name and attribute number, not just by name;
- To copy a record from the main file to the secondary file, enter the name of the main file in the "Fld To Update" prompt and enter an expression to derive the key value in the "Value To Use".

Once all of the secondary updates have been defined for a file, press F2 to save. If there are other secondary file updates to be defined, they will be displayed sequentially until all have been defined. Once all of the updates have been defined, a menu will be displayed to let you know the update is complete.

And Another Way to Look At It...

Another way to look at the periodic update is to answer the following questions:

- What file contains the records to be processed?
- Which records are to be processed, and in what order?
- How should the main record be updated?
- How should other records (if any) should be updated?
- What other special processing needs to be done?

By answering these questions you can answer all of the prompts on the periodic update screen. Of course, the special processing needs a little more explanation, but for the most part, these questions define the scope of a periodic update.

For BASIC Programmers Only...

The concept of a periodic update is a common structure for BASIC programmers. Typically, this type of program is called either an "update" or a "batch update", and follows the following structure:

```
*
* Processing At Start
*
...
*
* Open File(s)
*
OPEN 'filename' TO F.FILE ELSE...
*
* Processing Prior to Selection
*
...
*
* Select file to be processed
*
CMD = 'SSELECT filename ...
EXECUTE CMD CAPTURING OUTPUT
*
* Process After Selection
*
...
*
* Main Processing
*
EOF = FALSE
LOOP
  READNEXT KEY ELSE EOF = TRUE
UNTIL EOF DO
  READU RECORD FROM F.FILE,KEY THEN
  *
  * Processing After Read
  *
  ...
  *
  * Update Main Record
  *
  ...
  *
  * Update Other Records
  *
  ...
  END
REPEAT
*
* Process At End
*
...
*
STOP
```

Generally speaking, this is a periodic update. You simply fill in the blanks using the Periodic Update Definitions tool.

Executing a Periodic Update

Like any SB+ process, a periodic update process can be invoked via a menu, the slash, or from another process. However, just because you create a periodic update definition doesn't mean you automatically get a periodic update process that calls it.

In the same way that input and output processes are created via the Screen Definitions tool, periodic update processes can be created from the Periodic Update Defns tool when you press F2-Save and then select "None-Execute Now" from the menu. SB+ prompts you for the name of the process (which should match the definition name), and a description. Once these have been entered, the process will be created.

Once the process is created, the periodic update will be started. The periodic update has been started when you see this prompt:

OK to Continue (Y/N)

If you respond by entering "Y", the periodic update will run, and a "Performing ..." message will be displayed for the duration of the run. If you enter "N", the periodic update will not be started.

Suppressing the "OK to Continue" Message

While periodic updates are the best way to create a main program to do batch updates, they are not always the main program. Periodic updates may also be used to perform a supporting role in an entry screen, report, or literally any other type of process. When the update is used in this role, the "OK to Continue (Y/N)" message is inappropriate. If you determine that an update is required at a certain point in the application, you don't want the user to have the option to run or not run the update. Can you just imagine the support calls?

Fortunately, this message can be suppressed a couple of ways. First, if a process is called from the Proc To Start slot, the message will not be displayed. This assumes that some other validation is being done at this slot (or perhaps a dialog box is being used to prompt the user), and therefore the prompt is unnecessary. Second, if the Periodic Check is entered as "-" (a dash) instead of "0-0", the prompt will be suppressed.

Common Types of Periodic Updates

There are 5 common types of periodic updates. Each type can perform a specific role in the application, either as a main program or as a supporting process. The following sections describe the different types.

Updating One File
Updating the Main File and Other Files
Updating Only Other Files
Updating Nothing
Break Updates

In Summary

Periodic updates are effective for a variety of different tasks, including:

- Updating records in one file
- Selecting records in one file and updating records in other files
- Summarizing information for use in reports or other tools
- Loading information into memory for use by other processes

With all of the flexibility, you should never be limited as to what you can accomplish for batch updating and miscellaneous loop-type processing.

SB+ Reporting Options

In SB+, there are two different types of reports that can be created. Depending on your needs, one type or the other (or a combination of the two) can be used in your application.

For simple columnar reports, SB+ has an interface to the query reporting language of your operating environment (usually known as Access, English, or UniQuery). For the purpose of this discussion, we'll refer to these as **Query reports**. These type of reports are very good for quickly viewing information that doesn't need much formatting to be presentable.

Secondly, SB+ has a built-in reporting tool called the **Report Writer**. This tool is an incredibly powerful and flexible feature of the development environment, allowing you to build columnar reports AND form-type reports (such as invoices or packslips), or pretty much anything else you can imagine.

In the next few sections, we'll explore the various ways that these tools can be used to help you get information out of your application.

Creating Query Reports

The Query Report Definition tool is used to define query reports to display or print your information. For those who know the query language, you'll find the intuitive help available on this tool to be a great asset in building complex reports. For those who have no experience in building these type of reports, you'll find yourself creating reports quickly and easily with this tool in no time.

Like many different types of SB+ processes, a query report has two complementary parts, one being the report *definition* and the other being the report *process*. In short, the process cannot do anything without a definition, and the definition is basically useless without a process which uses it. Therefore, your efforts in creating the report will be focused on the creation of the definition, and SB+ can optionally create the process when it is time to run the report the first time.

Creating a Query Report Definition
Creating a Query Report Process
Defining Break Fields on Query Reports
Defining Total Fields on Query Reports

Creating SB+ Report Writer Reports

The SB+ Report Writer tool is one of the most powerful tools in the SB+ toolbox, allowing you to do some incredible magic in the presentation of your information. Unlike the **query language**, which is limited to columnar reports with certain precedence orders for break fields and limited heading/footing formatting ability, the Report Writer tool allows you to create both columnar and form-type reports (such as invoices or packslips), complex headings and footings, and multiple break fields in any order that you prefer.

The Report Writer tool roughly resembles the Screen Definitions tool in many respects. First, like the Screen Definitions tool, the Report Writer tool is used to build report *definitions* instead of the actual report *processes*. Once a report definition has been created SB+ can actually create the report process for you, also similar to the Screen Definition tool. Lastly, many of the function keys used in the Screen Definition tool have comparable features in the Report Writer tool, which makes learning the tool much easier.

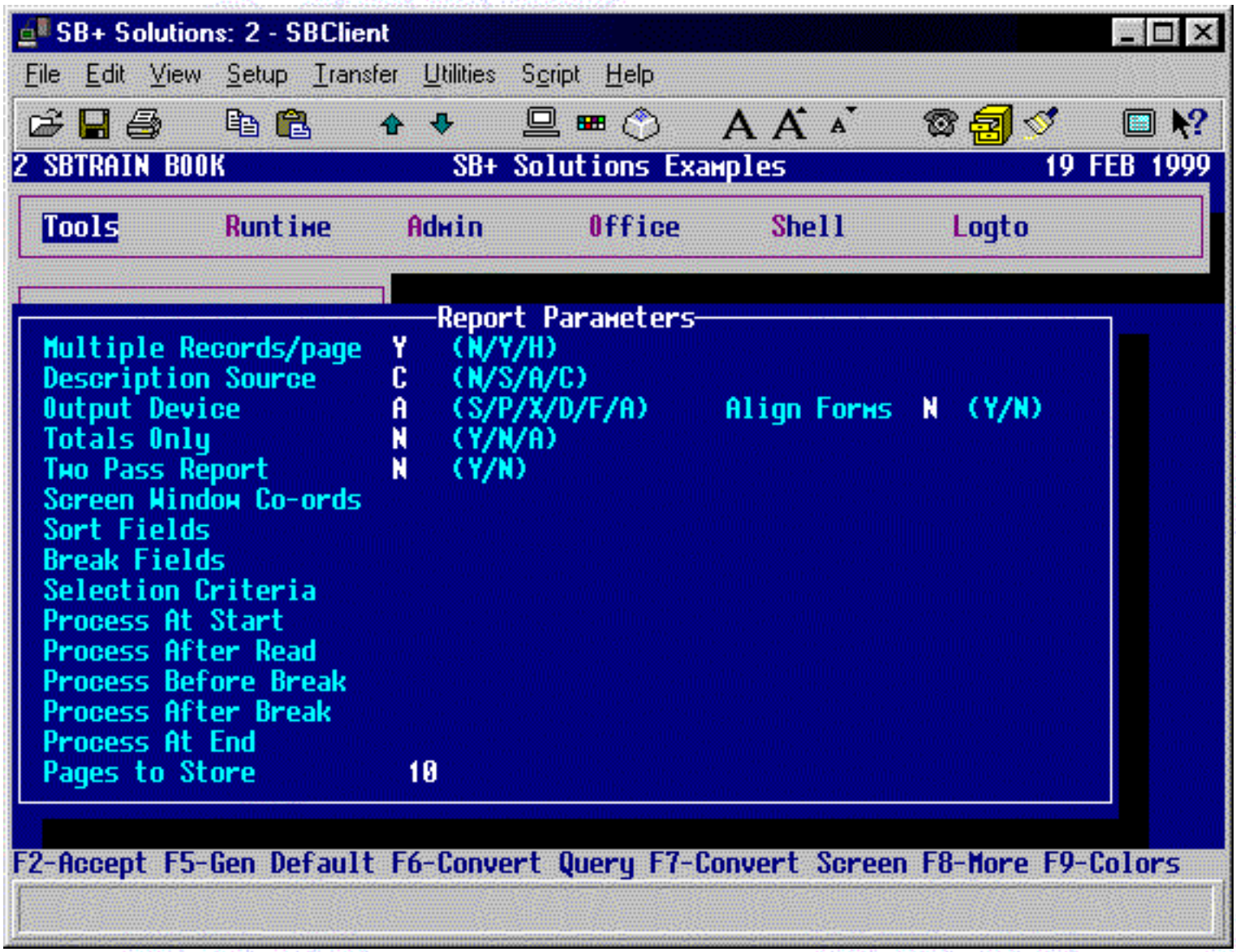
Starting the Report Definition Tool

To start the SB+ Report Definition tool using the menus, go to the main menu and select Tools, then Report Definitions. Or, for a more direct approach, you may start the tool by entering /RD at any menu or input prompt.

When the Report Definitions tool is started, a prompt appears asking you to enter the name of the file to create a report for. At this prompt, enter the name of any file that can be accessed from this account.

After the file name has been entered, you will be prompted to enter the name of a report. As with naming screen definitions, the name you enter here should be brief, descriptive, and should conform to your current naming standards.

Once the file name and report name have been entered, the following screen will be displayed where you can enter various parameters for the report:



When you are first creating a columnar report, there's no need to enter anything on this screen. Simply press F2 to accept what is displayed, and the report definitions screen will appear. On the other hand, if you are creating a form-type report (such as an invoice or packslip), enter "N" at the Multiple Records/Page prompt, then press F2 to begin defining the report.

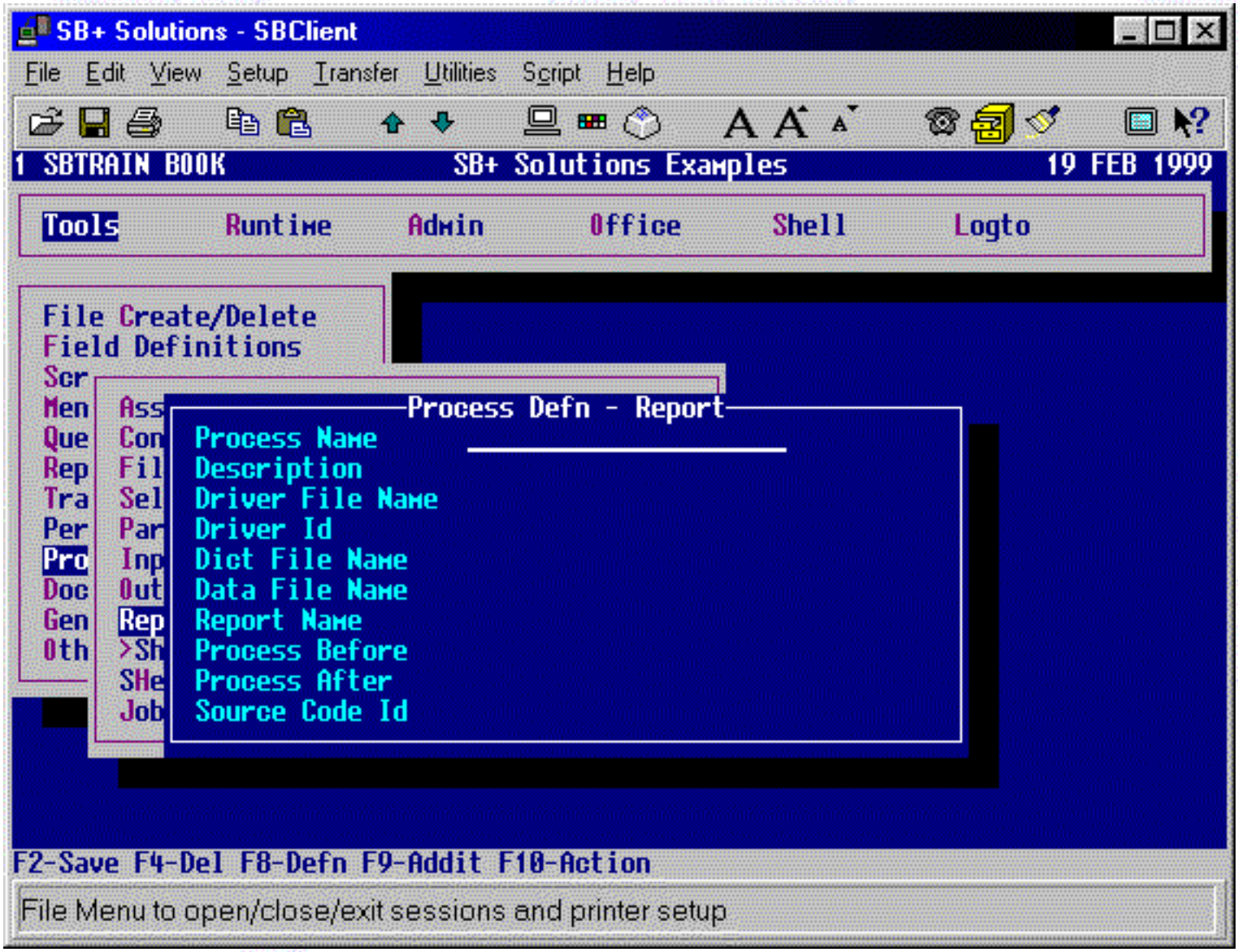
After you press F2, if you ever need to redisplay this screen for review or to make a change, simply press F6-Params while painting the report.

Columns or Forms?
Using Report Writer Derived Values
Creating Columnar Reports
Creating Form Reports
Report Writer Conversion Magic
Stripping Data From A Record Prior to Output
Building Information To Be Output
Creating a Report for Multiple Parallel Files
Running The Report From a Window
Selecting Records to Output
Calculating Keys On-The-Fly

Creating a Report with Multiple Selection Options

Creating a Report for a Split Dictionary/Data File

In certain applications, information for a report will be read from a file other than the dictionary file. To accomplish this, we need to tell SB+ that the data file (where the information will come from) is different from the dictionary file (where the report definition is stored). This is done with the Process Definition - Report tool, which appears as follows:



To invoke this tool, select Tools, the Process Definitions, then Report. Or, for a more direct approach, this tool can be started by entering /PD.R at any input prompt or menu.

Once the tool has been started, enter the name of a report process. (Remember: The report process is entered in the form "R*file*reportname".) If the report process exists, the details for the process will be displayed.

Notice the Dict File Name prompt. This prompt defines the file where the actual report definition exists. In contrast, notice the prompt labeled Data File Name (if diff). In this prompt, enter the name of the file where information should be read for the report.

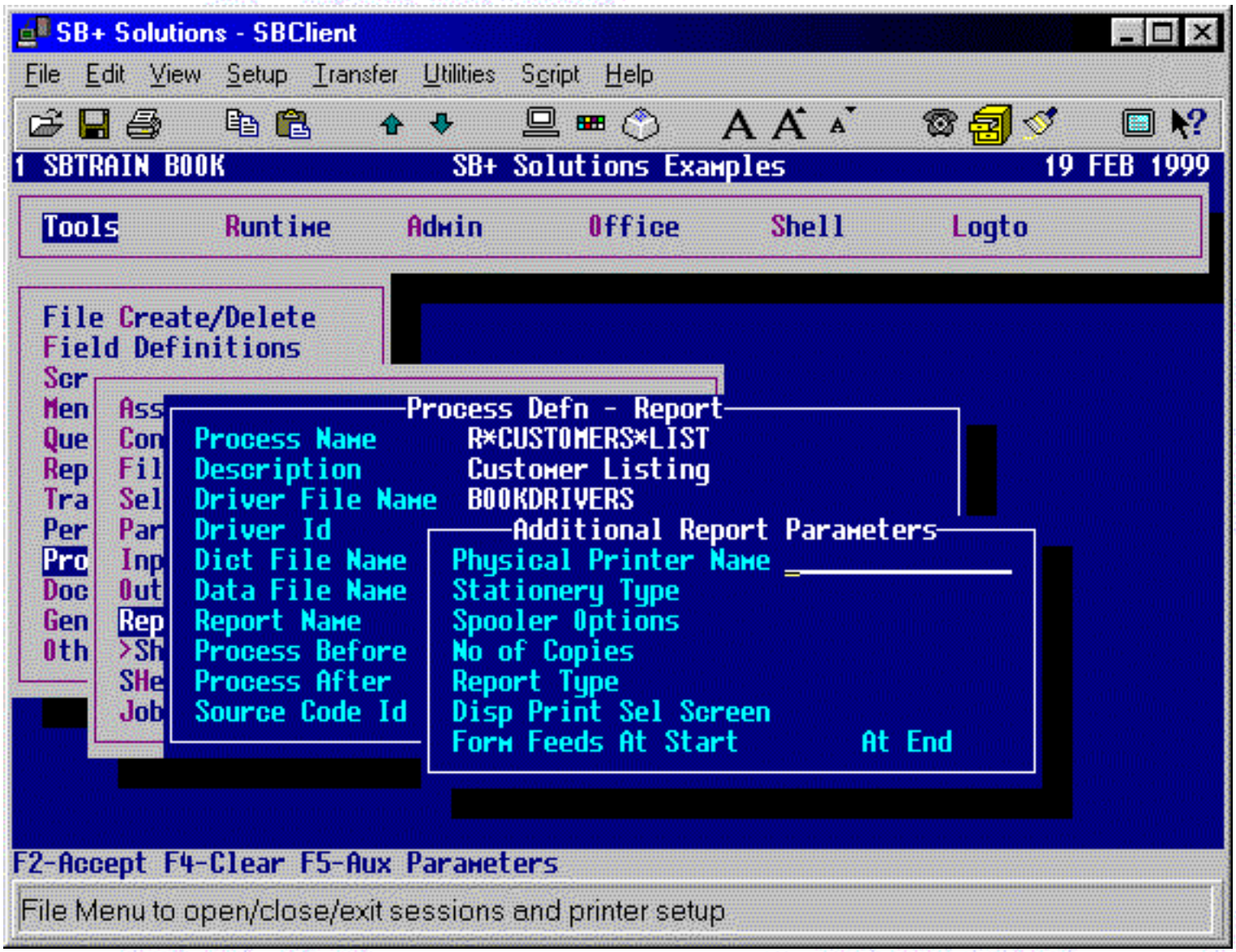
Once this change has been made, simply press F2 to save this record. The next time the report is executed, the data will be read

from the data file you've specified here.

Defining Stationery, Location, and Other Printer Specifics for a Report

After a report has been defined and tested, you should define the printer information for the report. In version 2.x this includes defining the stationery, location, and spooler options. In later versions this includes defining the stationery, printer, spooler options, and a number of other settings.

These settings are entered onto a popup screen in the Process Definition - Report tool. This tool appears in the previous section, and has a F9-Addit function key for the entry of the printer specific information, and may appear as follows:



(This screen shows what the printer information looks like in version 3.x and higher. If you are using an earlier version, this screen may appear slightly different.)

At the "Physical Printer Name", you may enter the name of a printer where this report should be output. If a the print selection screen (discussed later) is displayed, the printer name given here will be the default printer.

The "Stationery Type" defines the type of form that will be printed on. Essentially, stationery defines the width and depth of the printed page. SB+ uses this setting to know how much information will fit on a page. If a physical printer has been nominated, the printer must support the stationery as entered in this prompt.

At the "Spooler Options" prompt, enter codes to define how the report will be printed. Will it print immediately? Or perhaps the report will print to disk and be held there until released? Because these options vary slightly from system to system, press F3 on your own system to see the available options.

The "No of Copies" defines the number of copies to be printed when the report is printed to a printer. (This setting has no effect on reports printed to the screen.)

The "Report Type" prompt allows you to associate a specific report type with this report. Report types are used for dynamically assigning a printer to a report. For more information about report types, see the SB+ system administration manual.

"Disp Print Sel Screen" means "do you want to display the printer selection screen?". This prompt has a number of valid options, as follows:

(no value)	Use the default setting for the printer selection screen as configured for the user. The tool /USER.FLAGS can be used to set or change this value for the current user, or the user setup program /SEC.USER.SETUP can be used to change this flag for any user.
A	Always display the print selection screen for this report, regardless of the settings for this user.
N	Never display the print selection screen for this report, regardless of the settings for this user.

Finally, if you want to output blank pages at the top or bottom of the report, enter a number at the "Form Feeds At Start" or "At End" prompts. When the report is printed to a printer, SB+ will send form feeds (and hence output blank pages) at the top and bottom of the report based on these settings.

How Many Process Slots?

Just how many process slots are there in a report definition? Let's take a look at the more traditional process slots and how they can be used:

Process Before Report: Process Definition - Report (/PD.R)

This process slot appears in version 3.x and following and can be used to initialize variables, or to run some pre-report processing, such as custom security checks. It is executed only once, prior to the output of the report.

Process At Start: Report Definition (/RD, F6-Params)

This process slot is very useful for initializing **common variables**. If you have a Process After Read which needs to build summary information for the grand total section , this process slot can be used to clear the area of common where the summary information will be stored. Any process found in this slot will be executed once, prior to the report asking the user for the output destination (Screen/Print/Aux...)

Process After Read: Report Definition (/RD, F6-Params)

This process will be called once after each record has been read. This process slot can be used to 1) verify that a record can be processed , 2) build information to be shown on the report , 3) strip information from each record to limit the values shown on the report , or 4) build information to be output in later sections (i.e. break or grand total sections) of the report.

Process Before Break: Report Definition (/RD, F6-Params)

This process will be called prior to the output of the break section, each time any break is encountered. It is useful for clearing values prior to outputting the break, or for setting values to be shown in the break section.

Process After Break: Report Definition (/RD, F6-Params)

This process will be called after the break section has been processed, each time any break is encountered. It is useful for clearing values in common that may have been constructed in the Process Before Break or other preceding process slots..

Process At End: : Report Definition (/RD, F6-Params)

This process will be called after the grand total section has been output for the report, but while the spooler file (if output to the printer) is still open. This process is useful for chaining several reports together (end-to-end), with the output appearing in a single spooler file.

Process to Readnext ID: Report Definition (/RD, F6-Params, F8-More)

This process is responsible for setting @KEY. This is very useful for calculating key values, such as in the case of a linked list, where there is no active select list. The process must set @RTN.FLAG to 1 to stop the report, or the report will go on forever.

Process to Read Record: Report Definition (/RD, F6-Params, F8-More)

This process is responsible for reading a record into @RECORD using an incoming key value in @KEY. This is useful for retrieving a record when it could possibly exist in a variety of different places. This process will happen for each key that is calculated by the previous process or read from the active select list.

Process to Readnext ID and Record: Report Definition (/RD, F6-Params, F8-More)

This process is a combination of the previous two process slots. It is responsible for setting @KEY and @RECORD, and also must set @RTN.FLAG to 1 to stop the report. Like the previous two process slots, this process will be executed over and over and over until @RTN.FLAG is set to 1.

Though this list defines all of the process slots on a report definition, it does not define all of the places where a process can be called. In fact, for every field shown on the report, processes can be called to calculate a value (using the derived value prompt) and also convert the value (using the Conversion prompt). All told, it is theoretically possible for any report to call dozens of separate processes from all of these places. (Of course, one must weight functionality against obsessive-compulsive behavior!)

In Summary

This chapter has highlighted just a few of the many techniques that can be used to create sophisticated, elegant, and powerful reports. But this is just the beginning. The Report Writer can do so much more than is shown on these few pages. When combined with Periodic Updates for building summary information, paragraphs for processing information, and input screens for entering parameters, perhaps you can see that the Report Writer has a lot to offer.

When something has this much ability, it generally takes a significant amount of time to gain a comprehensive understanding. Yet, by mastering the techniques as shown in this chapter, you'll be well on your way to handling any reporting problem you're faced with.

Creating SB+ Menus

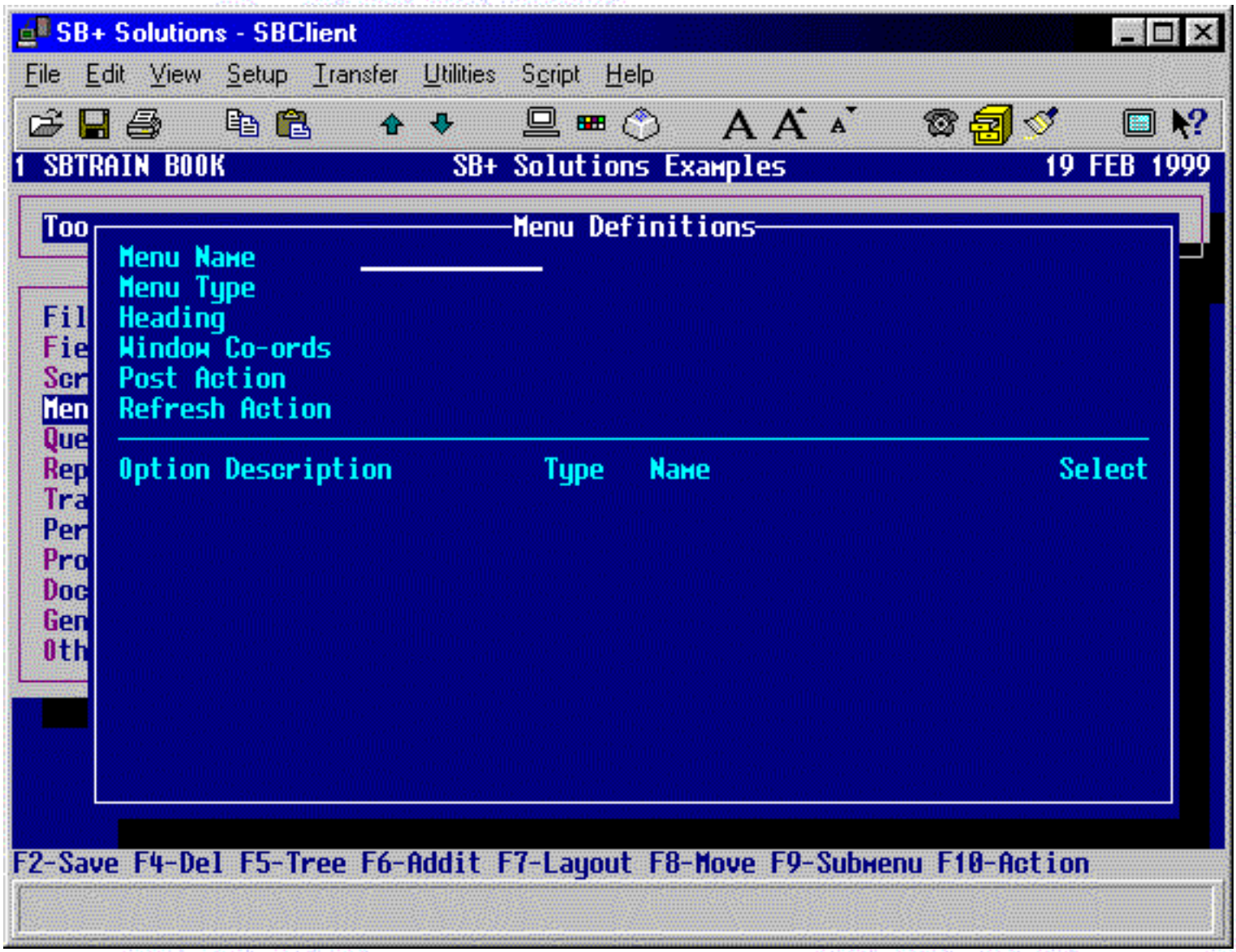
In SB+, there are two reasons for creating menus. First, and most obviously, menus are the best facility for starting your programs. True, users could use the /process ability of SB+ to start your programs, but chances are they will like your product better if they can simply point-and-click on menu options to start your programs. Traditionally, these are called *startup menus*.

Secondly, menus can be used to add functionality inside of your programs. Specifically, many developers find the list of available function keys to be restrictive, and therefore often use menus to expand the functionality of each function keypress. These types of menus are traditionally called *process menus*.

Regardless of what type of menu you're creating, the same tool is used. This tool is called Menu Definitions and is invoked by selecting Menu Definitions from the Tools menu. Of course, you can also invoke the tool process directly by entering /MD at any menu or input prompt.

There is nothing in SB+ that distinguishes a startup menu from a process menu. As far as SB+ is concerned, a menu is a menu is a menu. It can be useful, however, to know which menus are used for each purpose. Therefore, consider adopting naming standards that can be used for this type of identification. Just for point of reference, use process naming standards for process menus, and use a different standard for naming startup menus. Whatever you do, though, be consistent and predictable.

The Menu Definitions tool appears as follows:



Like any menu, you can give the menu any name you prefer. However, the name should adhere to any established naming standards for the project.

Menu Type can be any of the different menu types available. Press F3 on this prompt to see a list of valid menu types. Typically, menu type "1" is used for startup menus.

The Heading prompt is optional. Whatever value is entered at this prompt will become the heading for the menu, and will appear at the top of the menu window, similar to the way that "Menu Definitions" is shown (in the illustration above).

The Window Co-ords prompt is where the coordinates for the menu window should be displayed. These window coordinates are entered in the following format:

xc,*yc*,*width*,*depth*

Where *xc* is the column number (X-coordinate) for the top left corner of the box, *yc* is the row number (Y-coordinate) for the top left corner of the box, *width* defines the width of the box (in characters), and *depth* defines the number of rows in the box.

If you like, you can enter the coordinates for your window simply by entering these values into the prompt. For example, to create

a window with the top left corner at column 0, row 1, 79 characters wide, 20 rows deep, the following coordinates may be entered:

0,1,79,20

F3 may also be used to draw these coordinates, similar to the F3 on the comparable prompt in the Screen Definition tool.

Unlike the same prompt in the Screen Definitions tool, menus only require the first two coordinates to be entered. If a width and/or depth have not been entered when the menu is saved, SB+ will calculate the width and depth of the menu based on the number and length of the options on the menu.

The Post Action prompt defines how this menu will be terminated. It has the following options:

0	The user must escape to exit the menu.
1	Once an option has been selected and the corresponding process executed, the menu will go away.
2	This is a combination of 0 and 1. If the user selects an option and the process fully executes, the menu will go away. If the user selects an option but escapes out of the option (such as with a selection process), the menu will reappear to allow the user to select a different option.
3	This option is like option 1, except that the selected option number is returned in the **common variable** @MENU.OPT.
4	This option is like option 1, except that the process associated with the selected option will be executed after the menu has disappeared. This is considerably different from type 1, which executes the selected process and then exits the menu.

At the Refresh Action prompt, enter a value to define how the screen will be refreshed when the menu is finished. The following values may be entered:

0	Refresh the calling screen when the menu terminates.
1	Do not refresh the calling window when the menu terminates.
2	Refresh all underlying levels when the menu terminates. (Caveat: This can be ugly!)
3	Wipe the menu from the screen (print spaces over the menu image), but don't refresh any underlying levels. This works great as long as there is nothing under the menu.

The SB+ Main Menu and System Menu

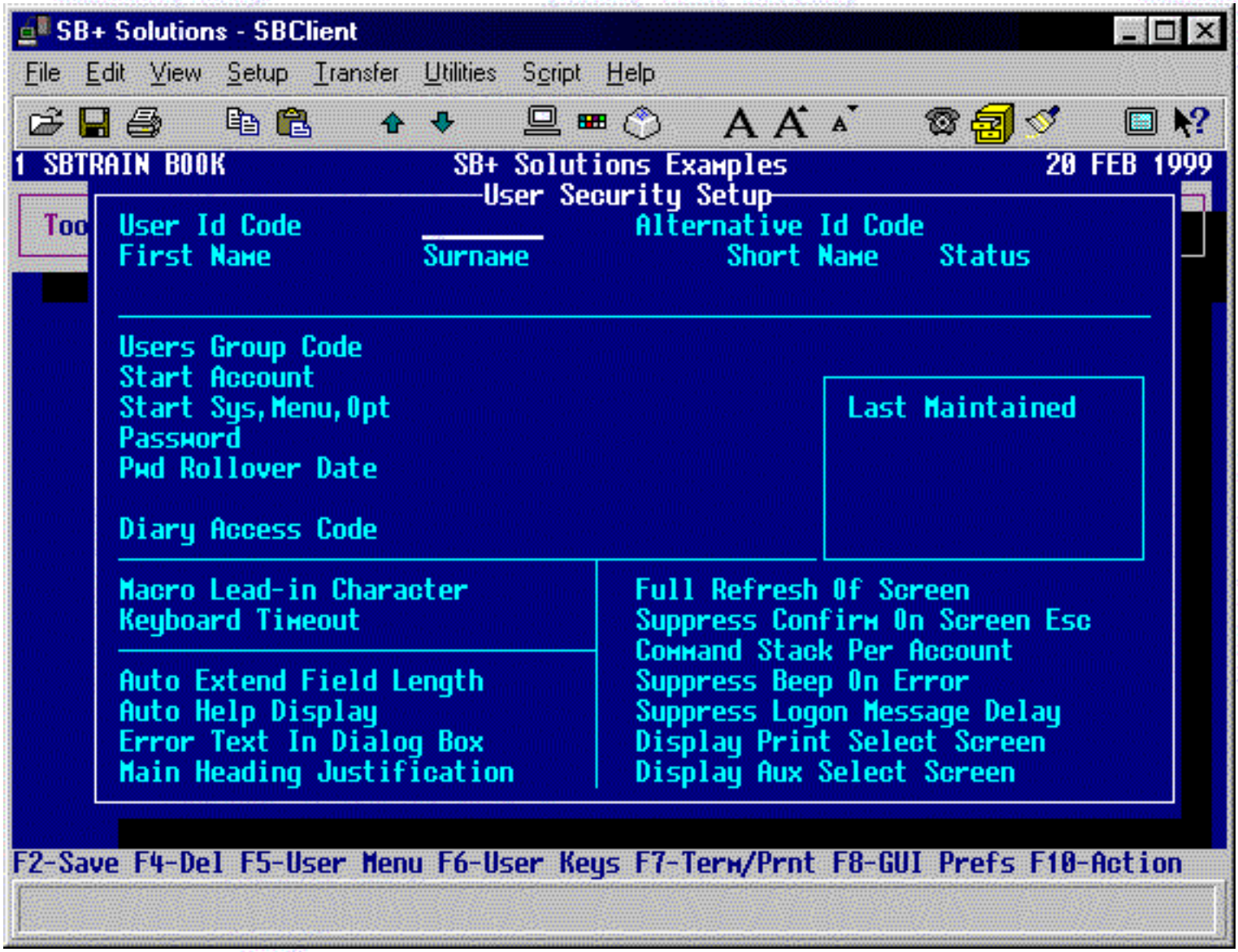
Every SB+ application starts with a menu called SBMENU. In fact, every SB+ system starts with this menu, which includes the Tools menu, the Runtime menu, and every other option you see when you first login to an SB+ system.

This menu, though one of great importance, is not protected from change. If you want your main menu to look a little different from the norm, simply use the Menu Definitions tool to change SBMENU.

Another menu of importance is called SBSYSMENU. This is the menu that is displayed when you have multiple SB+ systems running in one account. Normally, you don't need to change this menu, but if you want to change the description of the options that are displayed, simply use the Menu Definitions tool to call up the menu, edit it, and save your changes.

User Menus

SB+ users are defined using the User Security Setup program. This program can be started through the menus using Admin, SeCurity, then User Security Setup. Or, you can enter /SEC.USER.SETUP at any input prompt or menu to start the tool. Once started, the SB+ prompts you to enter your password. After the password has been entered, the tool appears as follows:



Note the prompt labeled "Start Sys,Menu,Opt". At this prompt, you can associate a system, menu, and menu option with each user. This allows you to create a specialized menu for each user, thereby limiting the programs that each user can see. This does not prevent a user from running a process with the "/", but if the user doesn't know the program exists, how would they know to use the "/"?

The starting system, menu, and option are entered with comma separators, and each are optional. Therefore, you could route a user to a specific system and menu using the following:

BOOK,MAINMENU

Or, if you want SB+ to select the option "E" (which, let's say, selects the "Entry Screen" option):

```
BOOK,MAINMENU,E
```

Constructing a Startup Menu

When building a menu for starting applications, the only thing to keep in mind is that you must setup the menu to invoke the starting process for each application. This seems like an obvious issue, no doubt, but you'd be surprised how many developers inadvertently create menus which call the wrong processes!

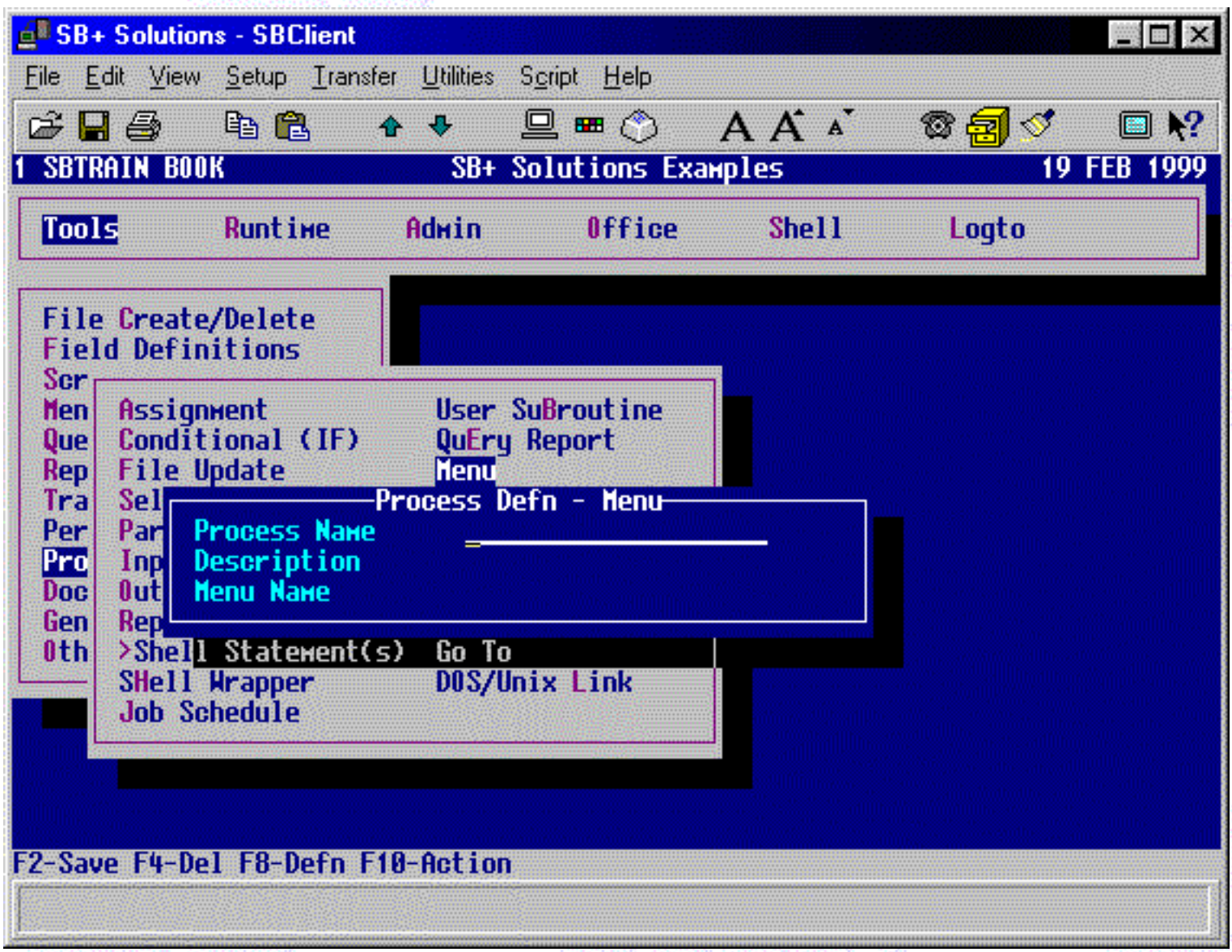
Most applications are too large to include all of the programs on a single menu. For this reason, several different cascading menus can be created for starting the individual programs. There are no strong rules for which programs belong together on which menus, though as a general practice reports are grouped either by functional group (such as order processing programs), or by program type (entry programs vs. report programs vs. inquiry programs). In the end, your application and the users of that application will probably have a big influence on how your startup menus appear.

Constructing a Process Menu

Process menus are built exactly the same as startup menus, with one exception: Process menus usually have a corresponding process, whereas startup menus do not. This is a convention, not necessarily a requirement, but it is useful for determining which menus are used for starting applications vs. which menus are used to perform a supporting role.

When a menu has a corresponding menu process, the menu can be invoked simply by calling the process name. Therefore, if the application needs to invoke a menu when the F9 key is pressed, simply list the menu process name as the process to be called when the function key is pressed. In short, as long as you create an actual menu process, menus can be called like any other processes.

The actual menu process is created using the Process Definition - Menu tool. This tool can be accessed by selecting Tools, Process Definitions, then Menu, or simply by entering /PD.M at any menu or input prompt. This tool appears as follows:



The purpose behind this tool is simply to create a process which points to a menu definition. This implies that the process name and the menu name can be different, but doing so is highly discouraged.

At the Process Name prompt, enter a standard process name. If you've previously created a menu, enter the name of the menu. If the menu has not yet been created, enter a name using whatever naming standards apply.

The Description defines the purpose for the menu. This field is displayed only when selecting a menu process with F3, or when using the /P? tool for searching for a process.

The Menu Name, as stated earlier, should be identical to the process name. Sure, it can be different, but doing so will inevitably complicate debugging of the application.

Once all this has been entered, press F2 to save the process record. After it's been saved, you can reference the process by name in your application. Incidentally, should you choose to not create a process record, you can invoke any menu from any prompt or menu by entering:

/M:*menuname*

...where *menuname* is the name of the menu to be invoked. Menus can also be called from any slot using:

M:*menuname*

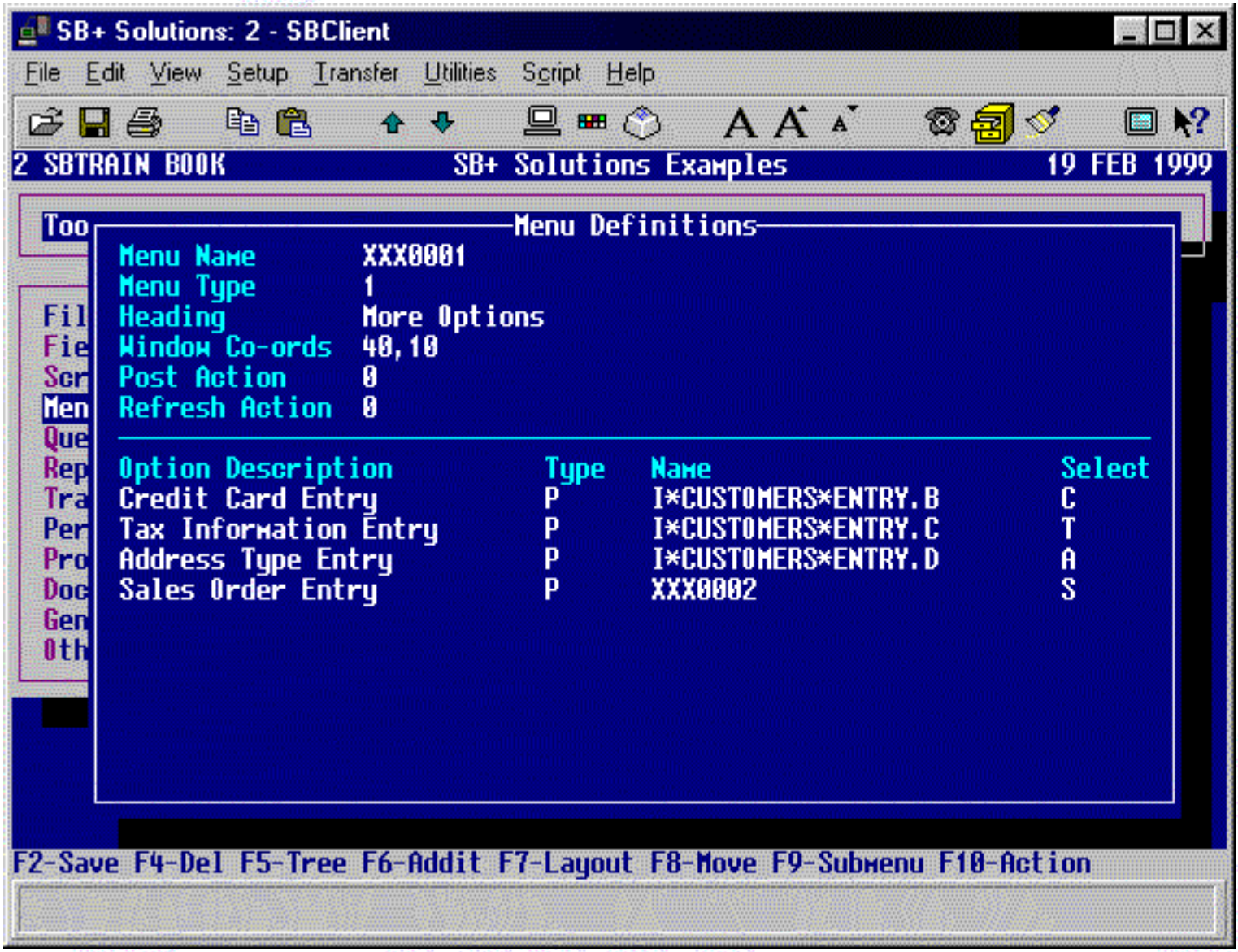
The "More" Menu

Generally speaking, on any given SB+ screen, you can define only 10 primary function keys. Of course, with toggled function keys and an action bar this number can be greater, but for the most part there are only 10 actual keys that can be programmed to perform some action in your application.

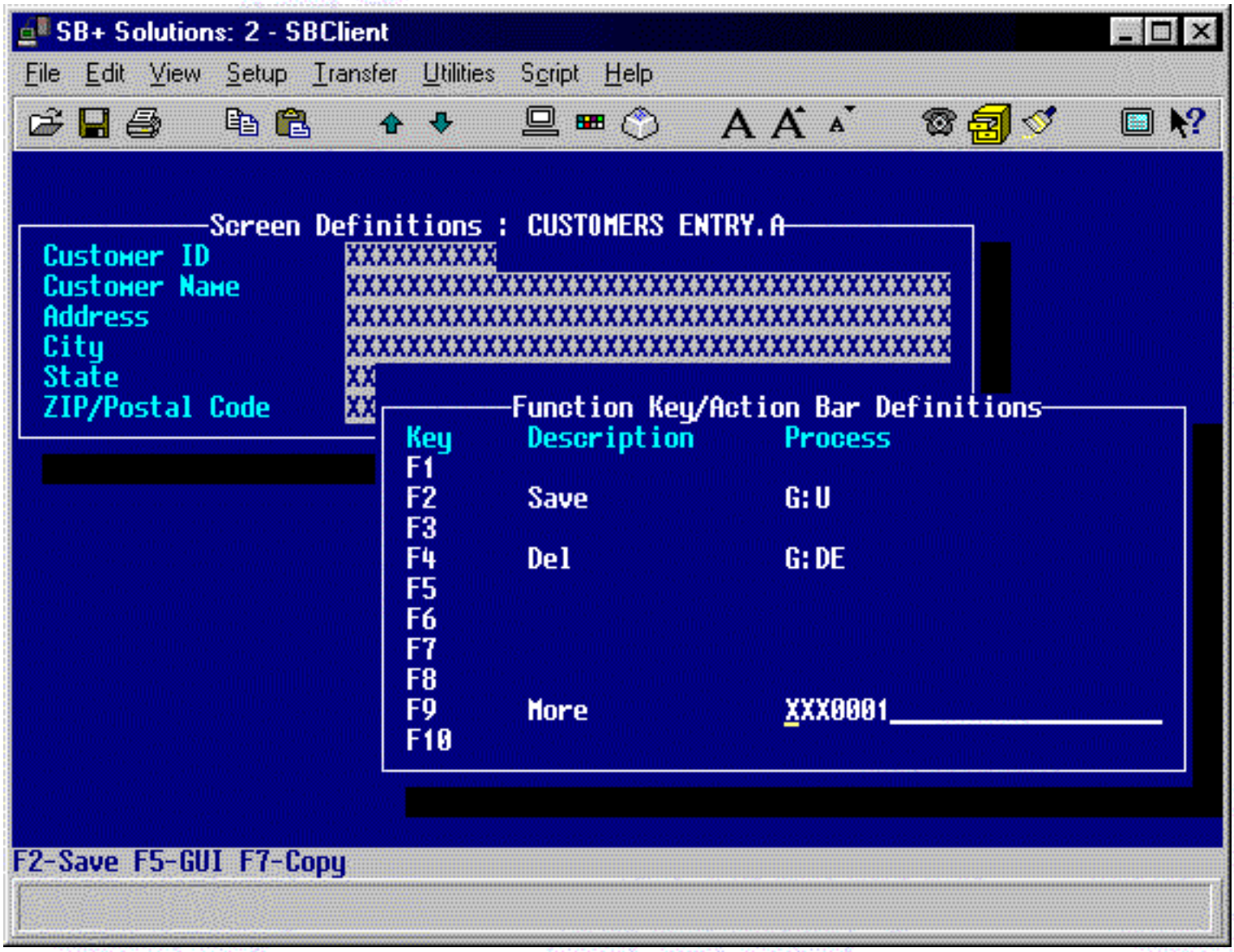
When you consider that F1 through F4 have a predefined function, and F10 has a commonly accepted function (i.e. invoking the action bar), this leaves only F5 through F9 available for general use on all applications. Sometimes this is enough. More times than not, however, five function keys is woefully inadequate.

One way to compensate for this shortage is by using a technique commonly called a "More" menu, which is simply a menu invoked by pressing F9. Because the menu can have any number of options (even several pages of options if necessary), this allows your F9 key to perform any number of different tasks, thereby expanding the list of function key options indefinitely.

At it's more basic level, the "More" menu is just a regular process menu. All guidelines for creating a process menu apply.



To be fair, all function keys could invoke menus. Though this would expand the number of options available even greater, it runs the risk of giving the application too many options, making it more difficult to use. Also, while the "More" menu could be invoked from any available function key, F9 is an effective standard.



Of course, in this example the menu isn't really necessary because there are a number of available function keys. Nonetheless, when you're running out of function keys before you're running out of options, the More menu is a great solution.

Constructing Complex Intuitive Help Using Menus

Just because SB+ provides one single process slot for intuitive help doesn't mean you can only have one intuitive help process for each prompt. Using menus, you can easily construct multiple selection options for each prompt in your application.

This is particularly important when indices are involved in your intuitive help process(es). Because SB+ doesn't allow you to use more than one index in a given selection, you must choose between one of the following options:

- Don't use indices (not a good choice), or
- Call a menu for intuitive help, with each option on the menu using a different index.

To illustrate this, let's assume we have an orders file which is indexed by salesman and customer number. When a user presses F3 on the order ID, we want them to be able to select by either salesman or customer (or both, if you really want a challenge). Therefore, when F3 is pressed, we can present a menu to the user with the following options:

Select Orders for a Salesman
Select Orders for a Customer

If the user selects the first option from the menu, we can call a selection process which prompts for the salesman ID, selects the order file using the index, displays some information for selection, finally returning the order IDs selected. Similarly, if the user selects the second option, we can call a selection process which prompts for the customer number, selects the order file using the customer index, displays some information for selection, finally returning the list of orders for the customer.

For the most part, F3 menus are little more than process menus. There are, however, a couple of guidelines to keep in mind:

- The Post Action setting on the menu must be either 1 or 2, so that the user is not forced to Escape from the menu to return the selected values back to the screen.
- All processes called from the menu must return a valid value in the **common variable** @VALUE, like any normal process called for intuitive help. A select list can also be returned if the intuitive help is being invoked on a key field.

With this simple technique, it's easy to provide any number of selection options for each prompt in your application.

Conditionally Quitting a Menu

Sometimes you need to create a menu that will stop under certain conditions which don't necessarily follow the conditions as defined by the Post Action setting. In these cases, we need to take a more proactive approach to telling SB+ when to stay in the menu, and when to get out.

(I'd like to take this opportunity to thank John Rich of Vancouver, Washington for coming up with this nifty little trick. We still don't know why it works -- but it does!)

Under normal circumstances, a menu with Post Action of "0" will continue to be displayed until the user Escapes from the menu. Therefore, we should be able to data stack an Escape key to get out, right? Unfortunately, for some reason this particular technique doesn't work -- at all -- so we need to find another approach.

If a process called from a menu sets @RTN.FLAG to "XX", the menu will go away. Problem is, however, this will also tell the caller to exit. So, while it does solve the problem of getting out of the menu based on some condition, it takes the user too far.

To correct this problem, we can use a paragraph to intercede between the main process and the menu, such as the following:

```
EXEC @PARAM
@RTN.FLAG = @RTN.FLAG[2,999]
```

This process is called RUN.MENU and is invoked using the following syntax (from a process slot):

```
RUN.MENU,menuname
```

As you can see, this process will call the menu as named in the parameter, and then strip off the first character in @RTN.FLAG before returning to its caller. Therefore, if a process called from the menu sets @RTN.FLAG to "XXU", this will exit the menu (stripping off the first "X"), the paragraph will strip off the second "X", and the caller will see the "U" and go to the update step. Or, to exit the menu and go to the key prompt, "XXIK" could be used.

In Summary

While menus are fantastic tools for starting programs, their usefulness doesn't stop there. Process menus can be a most valuable addition to your applications for expanding function keys, intuitive help, calculating defaults, and even for performing conditional validation. In short, whatever you currently do with process slots, menus can help you do more.

The Common Map

The SB+ common map is divided into three sections, commonly referred to as section 1 variables, section 2 variables, and section 3 variables. Though the names of the sections are clearly nondescript, understanding the characteristics of each section of the common map can be very beneficial in your SB+ development.

Section 1 variables are a collection of variables which SB+ saves when a new screen, report, or update is invoked, and restores when the process terminates. Using this technique, SB+ is able to maintain a very small collection of variables in this section without confusing the inputs and outputs of several non-related processes. On the other hand, when you want several non-related processes to share information, this makes your life more interesting.

Section 2 variables are a collection of variables whose values are constantly changing, or those variables which are not saved and restored by SB+. A couple of the more dynamic section 2 variables are the return flag (@RTN.FLAG) and refresh parameter (@REFRESH).

Section 3 variables are variables which are loaded when you first start SB+. These values rarely change, if ever, and include things like the current port number (@PORT), the current account name (@ACNT.NAME), and your user ID (@USER.ID).

The next couple of sections take a closer look at the individual common map configuration for the 2.x and later releases of SB+. Because of the similarity between the two versions, we'll look at the 2.x variables first, then discuss the changes between the 2.x and later common maps.

Version 2.x Common Map
Version 3.x and Higher Common Map

The Expression Language

The expression language forms the basis for many features of SB+. Expressions are the root of SB+ **conversions** and **derived values**, **field defaults**, **E: validations**, and also are a very important part of the **paragraph language**.

The following paragraphs take an inside look at constructing expressions, the different parts of the expression language, and the myriad of functions available in the language.

Expression Basics
Expression Operators
Expression Functions

Global Equates

When an expression is evaluated, SB+ checks to see if the expression references any Global Equates. Simply stated, global equates are symbolic names for expressions. Using these global equates, a developer can reference esoteric expressions symbolically, making the code easier to follow and understand.

For example, let's say we want to have a process which does different things depending on the name of the field which calls it. Which is more readable?

```
CASE @LINE<1,18> = 'NAME'
  EXEC 'PROCESS.NAME'
CASE @LINE<1,18> = 'ADDRESS'
  EXEC 'PROCESS.ADDRESS'
CASE @LINE<1,18> = 'CITY'
  EXEC 'PROCESS.CITY'
END CASE
```

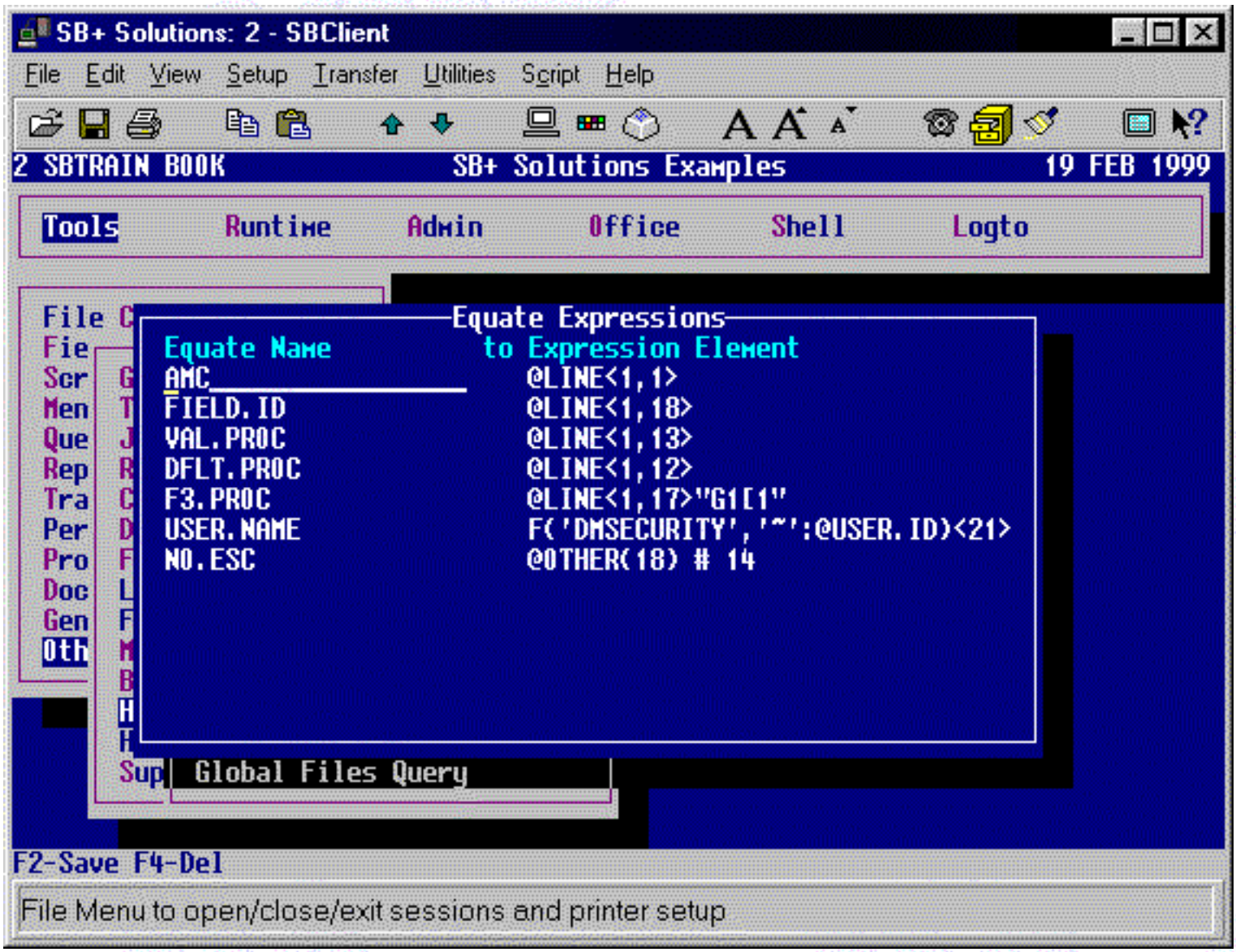
...or...

```
CASE FIELD.ID = 'NAME'
  EXEC 'PROCESS.NAME'
CASE FIELD.ID = 'ADDRESS'
  EXEC 'PROCESS.ADDRESS'
CASE FIELD.ID = 'CITY'
  EXEC 'PROCESS.CITY'
END CASE
```

Personally, I vote for the second approach: FIELD.ID is more memorable and descriptive than @LINE<1,18>.

Symbolic names like FIELD.ID are easily entered with the Global Equates Definition tool. This tool is invoked from the Tools menu, Other Tools/Utilities, House Keeping, Global Equates. Or, for the more direct approach, you can enter /EXP.EQU at any input prompt or menu.

The global equates screen appears as follows:



In the left column, enter a symbolic name for an expression. In the right column, simply enter the expression. Because global equates are truly "global" (rather than specific to a file), they cannot reference any fields by name. Rather, global equates can use any common variables to calculate a value.

The following are a few of my favorite global equates:

Equate Name	Expression	Description
AMC	@LINE<1,1>	The attribute number being updated by the current prompt. If this is negative, the field references an attribute in @WORK. If this number is positive, the field references an attribute in @RECORD. This is commonly used with /EE during debugging.
FIELD.ID	@LINE<1,18>	The name of the field at the current cursor position. This is commonly used with /EE during debugging and also in the context shown earlier.
VAL.PROC	@LINE<1,13>	The validation string from the current field. If the validation is an E: (expression) type, the value shown here will be translated into SB+'s P-code for expressions. This is commonly used with /EE during debugging.

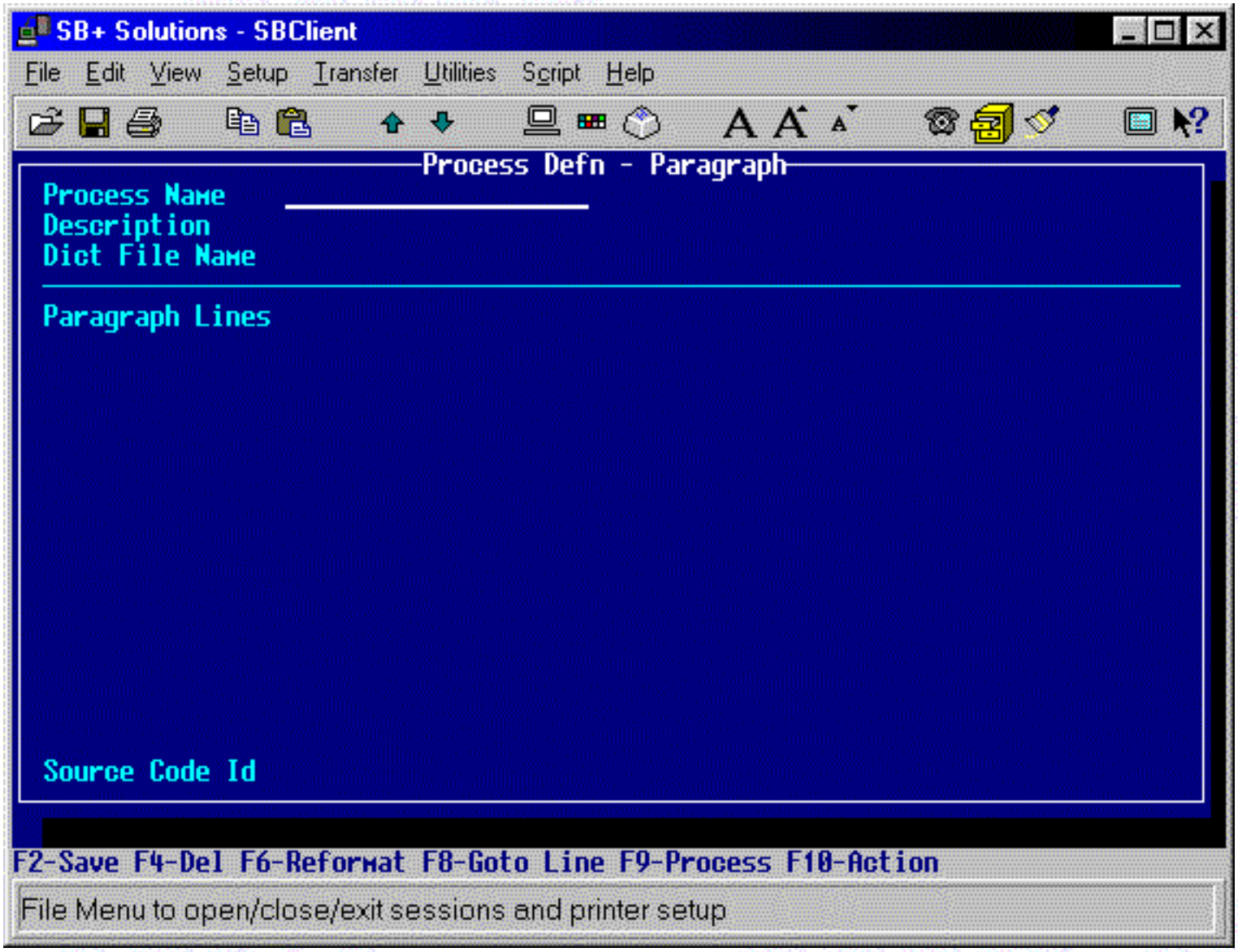
DFLT.PROC	@LINE<1,12>	The default expression from the current field. The value shown here will be translated into SB+'s P-code for expressions. This is commonly used with /EE during debugging.
F3.PROC	@LINE<1,17>"G1[1"	The name of the intuitive help process for the current field, if any. This is commonly used with /EE during debugging.
USER.NAME	F('DMSECURITY','~':@USER.ID)<21>	The name of the current user, read from DMSECURITY attribute 21. Note the tilde (~) prefix on the user ID when reading this file. This is commonly used in paragraphs or derived values, as in: LST.UPDT.BY = USER.NAME
UP.ARROW	@OTHER(18) = 3 OR @OTHER(18) = 25	Used in a Process Before process on a field. This is used to check if the up arrow was pressed when skipping fields. IF UP.ARROW THEN EXIT 1 ELSE EXIT S
NO.ESC	@OTHER(18) # 14	This is used to check if someone pressed escape to exit a selection process or input screen, such as: IF NO.ESC THEN EXEC 'someprocess' END

Like most system-level features of SB+, you should have standards for using global equates, i.e. how they will be used, and for what purpose(s). Though a relatively simple feature of SB+, global equates can make your development effort much more productive and enjoyable by eliminating the need to remember complicated syntax for commonly needed expressions.

The Paragraph Language

The SB+ paragraph language is, for all intents and purposes, a superset of the expression language and a subset of **BASIC**. Nearly all of the paragraph statements rely heavily on expressions for information, and the syntax of the language is remarkably BASIC-like. In short, expressions provide the information to be processed, while the BASIC-like paragraph statements tell the paragraph what to do with the information.

The Process Definition - Paragraph tool can be started from the menus by selecting Tools, Process Definitions, then Paragraph. Or, for the direct approach, simply enter /PD.P at any input prompt or menu. When the tool is started, the following screen is displayed:



Like all processes, you must first enter the name of the paragraph at the "Process Name" slot. Once the name has been entered, SB + will display the paragraph for you and allow you to edit it -- assuming, of course, that the paragraph exists. (While SB+ will allow you to enter process names in mixed case, always enter them in upper case.)

The "Description" prompt allows you to enter a long description for the paragraph. This description will be displayed everywhere

you can use F3 to get a process name, so it's important to be descriptive. Also, there's not much room to enter text here (and still be able to see it all), so you must also be brief.

The "Dict File Name" prompt is an optional entry and needs an entry only when the lines shown in the "Paragraph Lines" prompt reference a field name. SB+ can't find a field name without knowing what file to search, and this field defines that file. Therefore, if you're referencing the NAME field out of the CUSTOMERS file somewhere in the paragraph lines, be sure to reference the CUSTOMERS file here!

Following the "Paragraph Lines" label, any number of paragraph lines can be entered. There is very good F3 available on this prompt, so as you're learning the paragraph language, use the F3 as well as this text to see what options are available and how each is structured. Of course, while this F3 will help you construct syntactically correct paragraph statements, it won't help you determine what sequences of commands are appropriate for a given situation. Fortunately, that is exactly the function of this text.

While paragraphs can be defined with any number of lines, you'll find that your software will perform better if paragraph are kept reasonably small. For example, when running a paragraph with several hundred lines, the lower lines will execute progressively slower than the lines at the top. Therefore, strive to keep your paragraphs small -- perhaps under 50 lines -- and use EXEC freely to execute other processes as needed. (For point of reference, you can determine the number of lines in the paragraph by moving the cursor to the last line of the paragraph, then enter /EE and look at the common variable @CNT. You can also use the full screen editor mode (discussed next) and turn on line numbers using the action bar to see how big your paragraph is getting.)

While working in the Paragraph Lines field, you can press Ctrl-W once to turn the paragraph lines into a block of editable text. While in this *word-processing* mode, you can move the cursor freely throughout the text -- up, down, left, right -- without having to press <cr> on each line. If you press Ctrl-W again, the text of your paragraph will be ported into a full screen editor, where you can see more of your paragraph at a time. To exit the full screen editor or word-processing mode and update the paragraph, simply press F2. Then, to actually save the paragraph, press F2 again. Note that the standard paragraph F1 and F3 do not apply when in one of these alternate modes. If you're going to need intuitive help to create/edit the paragraph, you'll need to use the standard entry screen.

Note the F8-Goto key. This key allows you to jump to any specific line in the paragraph when in the normal paragraph entry. While this is a useful key, I find it better to use the word-processing or full screen modes, and then use the F7 key (in either mode) to move to a specific line. These modes also have a search key, F6, which can be used to find text inside of the paragraph lines, as well as a block key, F5, for moving, copying, or deleting lines. The normal paragraph entry screen doesn't have such features.

As the paragraph grows, formatting of the text can be problematic. For example, if an IF statement is added to the paragraph around 20 lines, it can be a bother to manually reformat the text. Fortunately, manual reformatting is unnecessary. Using the F6-Reformat key (F10-Action, reFormat on earlier versions), SB+ will automatically format the lines using standard indenting. When you press this key, SB+ will ask you for the number of spaces to indent each level (the default is 2), and after you've pressed <cr>, the paragraph will be formatted. Note that you can only reformat the paragraph when the cursor is in the Paragraph Lines field, and only when you're not using the word-processing for full screen modes.

The following section briefly defines the different statements in the paragraph language. Click on the link in the left column for more information about a specific command. (You'll find examples of most of these throughout the text of this book.)

*****	Comment Lines
n	Line Numbers
ASSIGN	Assign a Value to a Variable
CALL	Call a BASIC Subroutine

CASE	Evaluate Mutually Exclusive Conditions
CRT	Display a Message
DATA	Data Stack an Expression
DELETE	Delete a Record
DELETEI	Delete a Record and Update Indices
DISP	Display a Message
EQU	Create Paragraph Equate
ERROR	Display a Message
EXEC	Execute Another Process
EXIT	Exit the Paragraph and Set @RTN.FLAG
FOR/NEXT	Loop
GOTO	Jump to a Label
IF..THEN..ELSE	Evaluate a Condition
INPUT	Request User Input
LOCAL	Declare Local Variable(s)
OPEN	Open a File
PRINT	Print a Message
READ	Read a Record
READL	Read/Lock a Record With Exception
READNEXT	Read Key From Active Select List
READU	Read Record With Lock
READV	Read Attribute From File
RELEASE	Release a Record Lock
SLEEP	Pause the Paragraph
WHILE..DO..REPEAT	Loop
WRITE	Write a Record
WRITEI	Write a Record and Update Indices
WRITEV	Write an Attribute

Using BASIC in an SB+ Application

To SB+, a BASIC subroutine is just one more type of process. Like a **paragraph**, a BASIC subroutine can do literally anything. (The fact that SB+ itself is written in BASIC is evidence of this.) By following a few guidelines, you can create SB+ BASIC routines easily and quickly.

But why create BASIC? Doesn't SB+ do everything already? If you believe that, you've been talking to too many salesmen. Fact is, there are a few things that SB+ doesn't do well.

Please understand, it is not my purpose to discount SB+ with this section. Rather, by understanding a few of the shortcomings of SB+ perhaps you can minimize the amount of BASIC you write.

Some people feel that it's okay to write all of the **default**, **validation**, **intuitive help**, and special processing routines in BASIC and forego paragraphs and other types of SB+ processes. I disagree with this approach. When you have a toolbox as complete as SB+, use the tools at your disposal whenever you can. Only when you cannot find the perfect tool should you consider BASIC. Personally, I strive for 3% BASIC or less in any given application. This guideline helps me to look for ways to use SB+ effectively before going back to the old familiar ways of approaching problems, and has resulted in the dozens of techniques that comprise this text.

There is no question that for most things, BASIC is faster. But with this speed increase comes a price. As you move from system to system, BASIC versions differ slightly and there will be conversion work to be done. In contrast, when you use paragraphs as much as possible, SB+ handles the system differences, leaving little to no conversion work for you.

In my experience, BASIC is the most effective tool for handling the following issues:

- Let's say you have several related lists. The first list is in a sorted order (ascending left justified, per se) and the other lists are related to the sorted list. SB+ provides no facility for doing a LOCATE(...) with the 'AL', 'DL', 'AR', or 'DR' option returning an insertion position into a sorted list. Therefore, the only way to accomplish this is with a BASIC process.
- When there are several interrelated updates which could possible require a dozen or more records to be updated together, BASIC is a good tool to have available. When you find that you need more **local variables** in a paragraph and the paragraph simply cannot be broken into several smaller paragraphs, BASIC is a good next step.
- When you need to do something system-specific (such as implement transaction processing) BASIC must be used. SB+ does not support many of the system-specific features of many operating environments. Of course, it's important to realize that adding more system-specific code to your application gives you less options for rehosting to another platform in the future.
- When you need to add a custom tool to SB+, BASIC is always an effective implementation choice.

Creating the BASIC Subroutine
Creating the BASIC Process

Understanding Selection Criteria

In SB+, the prompt "Selection Criteria" appears in three different tools. These are:

- Process Definition - Selection (/PD.S)
- Report Definition (/RD)
- Periodic Update Definition (/PUD)

In each of these tools, the Selection Criteria prompt offers the same options. The prompt also appears in the Query Report Definition tool (/ARD or /QRD), but functions slightly differently in that tool. (Specifically, the option to run from a select list is handled by a separate prompt in the Query Report Definition tool.)

This section is intended to help you understand your options. (Note: These are alternate links to information in **Chapter 6**.)

Selecting One Record
Selecting a Group of Records Based on a Condition
Selecting All Records
Selecting All Records in a Saved List
Selecting Keys From a Multivalued List in Common Memory
QSELECTing A Multi-attribute List
Implementing Impossible Selection Criteria

Understanding Edit Keys

When the cursor is at the beginning of a line (in version 2.x) or anywhere on the line (in version 3.x), you may press an edit key to tell SB+ to do something. Edit keys are defined as those special keys which send special characters to the host when they are pressed. Some examples of edit keys include the arrow keys, the function keys, and any special keys like Page Up or Page Down.

Under certain circumstances you may want to know what key was the last edit key pressed. For example, let's say you have a prompt on a screen that, when the <cr> is pressed in the prior prompt, automatically displays the intuitive help window. On the other hand, if the user presses the down arrow to get to the prompt, the intuitive help process should not be automatically invoked. This type of processing can be easily accomplished in a Process Before Field.

The Process Before Field must check @OTHER(18), which is where SB+ stuffs a number which defines what the last edit key was. These numbers are as follows:

0	Carriage Return (<cr>)
1	Cursor Back
2	Cursor Forward
3	Cursor Up
4	Cursor Down
5	Start of Line
6	End of Line
7	Insert Toggle
8	Delete Character
9	Erase from Cursor to End of Line
10	Next Word
11	Previous Word
12	Extend Length/Search Again
13	Quit Process
14	Escape
15	User Macro Lead Character
16	Insert Line
17	Delete Line
18	Page Up
19	Page Down
20	Top of Text
21	Bottom of Text
22	Tab
23	Invoke Text Editor (with word wrapping)
24	Invoke Intuitive Help

25	Back Tab
26	Reserved
27	Reserved
28	Reserved
29	Mouse Lead-In Characters
30	Reserved
31	Function Key F1
32	Function Key F2
33	Function Key F3
34	Function Key F4
35	Function Key F5
36	Function Key F6
37	Function Key F7
38	Function Key F8
39	Function Key F9
40	Function Key F10

To check to see if the last keypress was an up arrow, the following paragraph line could be used:

IF @OTHER(18) = 3 THEN...

A generally accepted technique is to create a global equate called EDIT.KEY which is equated to @OTHER(18), which would make this paragraph fragment a bit easier to read:

IF EDIT.KEY = 3 THEN...

Or, using the **UP.ARROW** equate from this book:

IF UP.ARROW THEN...

In summary, it isn't often that you need to know the identity of the last edit key. However, when you do need it, it's nice to know it's available. Incidentally, when you need to know the specific key to press to invoke for a specific function, enter /KEYS at any input prompt or menu.

Understanding @RTN.FLAG

@RTN.FLAG is probably one of the least understood variables in the SB+ common map. This confusion is understandable, as this is the one variable whose values can mean different things depending on where the value is set.

@RTN.FLAG can have the following values:

0	This setting generally means that everything is okay. Particularly used with **validation** processes, this setting tells SB+ that the value as entered is acceptable and has passed the validation.
1	This setting could mean any number of things, depending on the process slot where it is set. • In a validation process, this setting tells SB+ that the value as entered has failed the validation. • In a Process Before Field, this value tells SB+ to **skip** back to the previous field. • In a Process After Field, this value tells SB+ to reprompt for the entry that was just validated successfully. (This isn't a very useful option.) • In a Process After Read, this value tells SB+ to **reject the read** (and go back to prompting for the key -- be sure to release the record lock!). • In the **Process After Screen Accept**, this value tells SB+ to reject the update attempt and stay in the record. • In a **periodic update** or **report**, this value tells SB+ to not update or print the current record.
B*n*	This option tells SB+ to go to the bottom prompt of the screen. If the optional *n* is provided, it tells SB+ to go to the bottom of a specific linked screen. The *n* may also be *+n* or *-n* to go to a specific linked screen relative to the current screen.
D	The 'D' option tells SB+ to go to the **delete step** in an entry screen, which is the same step as the update step (except that @ACTION = 3). The record will be deleted without confirmation.
DE	This option, like 'D', tells SB+ to go to the delete step in an entry screen. The "E" suffix tells SB+ to prompt for confirmation before deleting the record.
F*n*	This option tells SB+ to execute a process associated with function key number *n*. This is most commonly used to make an action bar option invoke something on a function key, such as pressing F2 or F4. This only applies on an input process, and is generally used when an action bar item should mimic a function key.
I1	In nearly all of the process slots on an input or output process (except the default and validation process slots prior to version 3.x), this setting tells SB+ to go back to the first input prompt on the screen. It is not used in any other types of definitions, such as selection processes, periodic updates, or reports.
IK	In nearly all of the process slots on an input or output process (except the default and validation process slots prior to version 3.x), this setting tells SB+ to go back to the key prompt. It is not used in any other types of definitions, such as selection processes, periodic updates, or reports
IR	In nearly all of the process slots on an input or output process (except the default and validation process slots prior to version 3.x), this setting tells SB+ to go back to the process after read and continue forward from that point. It is not used in any other types of definitions, such as selection processes, periodic updates, or reports
L*n*	Go to driver line *n*. In this syntax, the line number may be specific either as a regular number, or *+n* or *-n* for a line number relative to the current driver line being processed. Note: This is rarely something of value to the developer, as driver lines are very dynamic and can change from execution to execution if changes have been made to the screen definition.

P*xxx*	This option tells SB+ to execute a process named *xxx*. This is most commonly used to make an action bar option invoke a process after the action bar has gone away. In short, this option is like a promissory note. By setting @RTN.FLAG to this option, SB+ "promises" to execute the process when the input screen regains control. The most common use for this option is to print a screen. If someone selects an option from an action bar which executes the 'P' process, only the action bar will be printed. However, if @RTN.FLAG is set to 'PP', when the action bar returns control back to the input process SB+ will execute a print screen, thereby printing what the user wants to see. See "**Implementing an Action Bar "File" option**" for more information.
S*n*	This option tells SB+ to skip the current prompt. If the optional number *n* follows the 'S', it tells SB+ to skip that number of prompts (where the current prompt is considered #1). Note: For many applications, it is best to skip prompts one at a time, employing forward/backward skip logic as explained in **Chapter 3**.
T*n*	This option tells SB+ to go to the top of the screen. If the optional *n* is provided, it tells SB+ to go to a specific linked screen. The *n* may also be *+n* or *-n* to go to a specific linked screen relative to the current screen
U	In an input screen, this tells SB+ to go to the update step, which will execute the Process After Screen Accept, update the record, and then execute the Process After Update.
X	This option applies to the Process Before on an input, output, report, or periodic update process. It tells SB+ to exit the current process.
{*field*}	In this option, the braces are actually part of the syntax. It is used only with input processes and non-amendable output processes and this tells SB+ to move the cursor to a specific field on the screen. If this field has a Process Before Field, it will be executed before the cursor actually moves there.

In Summary

The common map, @RTN.FLAG settings, expressions, and other "behind-the-scenes" details may appear to be things that you rarely need. However, understanding these details can make a significant impact on your development effort. In the end, whatever you can do to save time will eventually save money, which can make or break your project.

Why Generate BASIC Code?

When an application is first created in SB+, it can be run interpretively. In other words, the subroutines which are SB+ interactively interpret the developer's programming as entered into input, output, report, paragraph, and other types of processes. Therefore, when the application is running, the application runs under SB+, SB+ runs under the operating environment, the operating environment runs under the operating system, and the operating system runs on the hardware.

The more levels of programs between your application and the hardware, the slower your application will perform. In many situations, this performance issue is really no concern -- SB+ can run efficiently enough to eliminate most performance concerns. However, there are certain situations when additional performance can be the difference between a good application and a truly great one.

Generating code is one way to improve an application's performance. When you generate code for a process, SB+ creates a BASIC subroutine to run in place of the actual process itself. Once a process has had code generated, SB+ ignores the process and calls the subroutine when the process is to be executed. This eliminates one level between the application and the hardware, with the potential of improving the performance of the application.

Notice I didn't say that it "will" improve the performance of the application. While it is true that generating code for processes makes them run closer to the hardware, generating code is not an alternative to bad programming practices. If you write a badly organized paragraph with a bunch of spaghetti code that doesn't run efficiently when running interpretively, generating code won't solve the problem. In fact, it may create new problems.

As of the 3.x release, the code that SB+ generates for a process may not perform exactly like the same code running interpretively. Therefore, something that works perfectly running interpretively could actually be broken by generating code!

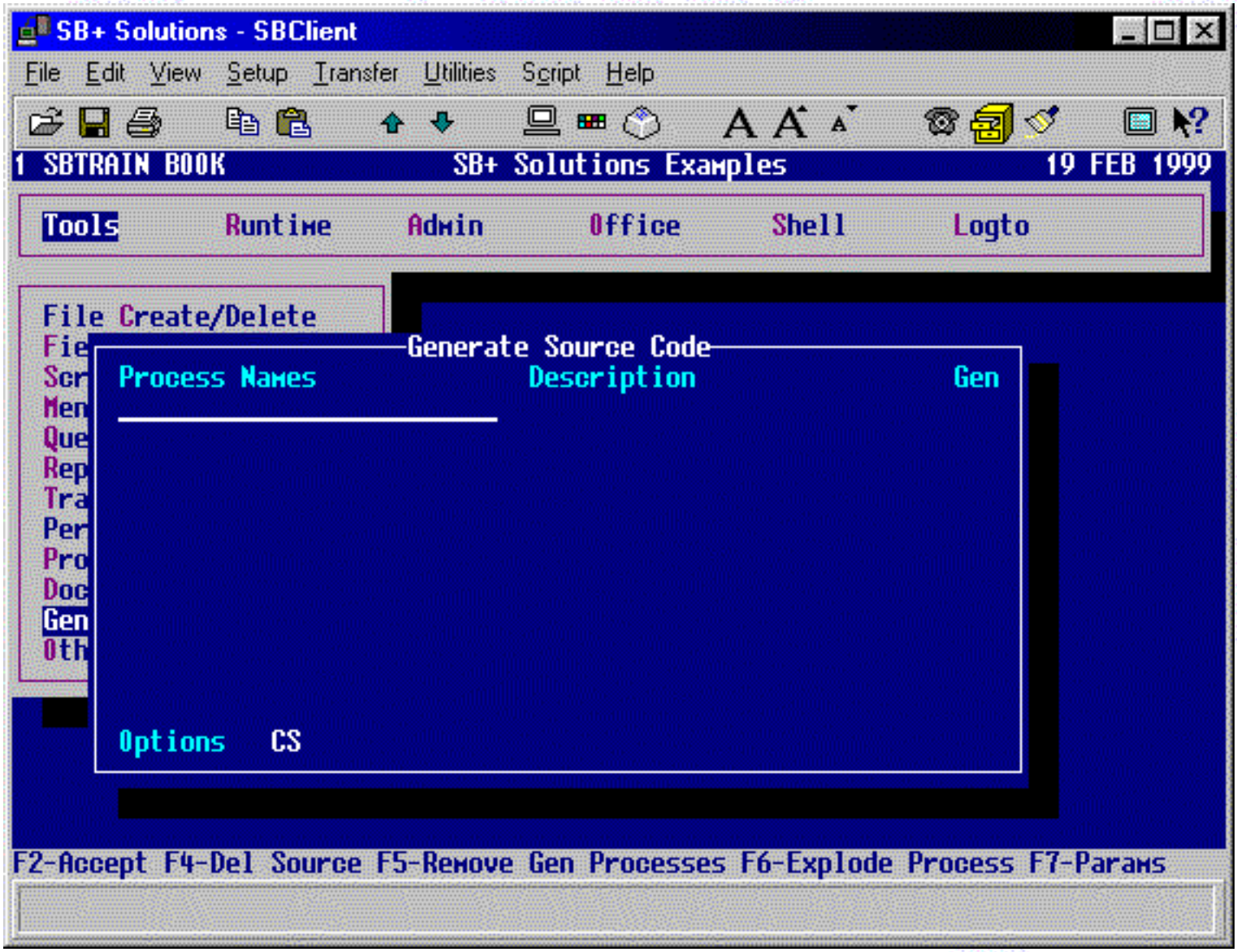
To be perfectly fair, with each release of SB+ the generated code inches closer and closer to working the same as the interpretive code. However, as of the current release the generated code feature has not yet arrived at a perfect match.

Nonetheless, if you have the time and the dedication to review the code that SB+ has generated, you can effectively generate code for the processes in your application and perhaps improve the speed and efficiency of your programming.

Generating Code

To generate code for processes in your application, use the Generate Code tool. This tool can be invoked by selecting Tools from the main menu, then Generate Code. Or, for a more direct approach, you may enter /GC at any input prompt or menu.

The tool appears as follows:



In the Process Names prompt, enter the name of the processes to generate code for. If code was previously generated for a particular process, a "G" will appear in the Gen column. If the generated code has been compiled and cataloged, a "C" will appear in the Code column. Any number of processes can be listed here, and F3 can be used to help load a bunch of processes.

In the options prompt, enter one or more of the following options:

- none -- If no options are entered, code will be generated for each process, but the code will not be compiled or cataloged.
- C -- Compile the code after being generated.

- S -- Suppress the symbol table when the code is generated. This will eliminate most debugging information that would otherwise be available when the program is running in the OE debugger.
- I -- Normally, SB+ will generate code for smaller processes that may be called from the process being generated. This has the distinct side-effect of eliminating the reference to the other process. Therefore, if the other process changes, SB+ knows nothing of the change, as it has encapsulated the old code inside of its caller. This option suppresses this behavior, causing SB+ to generate code for ONLY the process being generated.

In Summary...

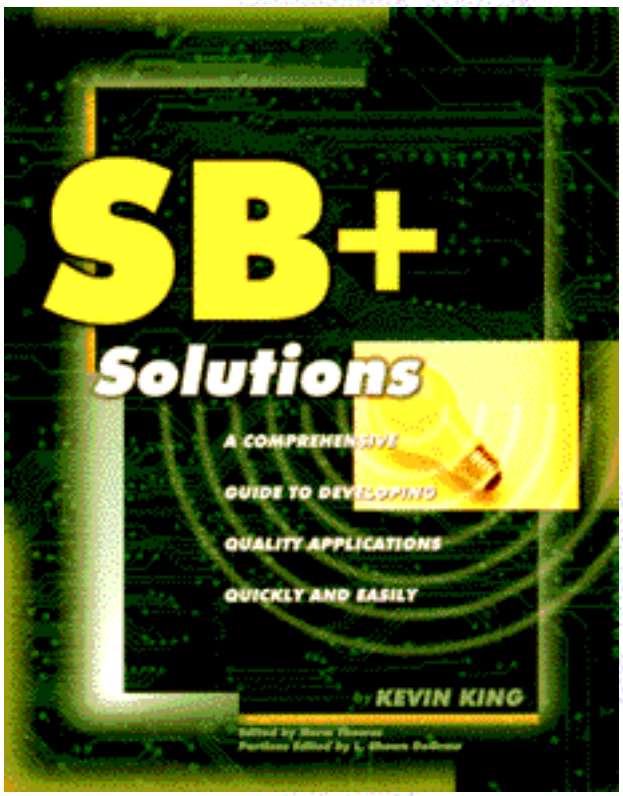
As stated earlier, generating code is not a substitute for good programming. Whether you anticipate running your application interpretively or through generated code, your programming technique, style, and structure will have the biggest impact on the speed and efficiency of the application. Generating code is an effective technique for improving system efficiency only when you've done all you can through structure, style, and technique.

I'm So Glad We've Had This Time Together...

Looking at the application development environments available today, SB+ is head-and-shoulders above the crowd. With it you can create an incredible product in a very short period of time with minimal effort. It's powerful, expandable, and flexible, but by no means is it a replacement for good programming technique and common sense.

There is so much you can do with this product it's impossible to squeeze everything into a few hundred pages. The original version of this book took over two years, this edition has taken several months, and still I've only included highlights of what you can do with SB+. In the end, it all comes down to something I've said only about 362 times in this book: What you get out of SB+ and how SB+ works for you isn't something you'll find in a book -- it's all a matter of your own personal tastes and style. Hopefully this text has provided some food for thought to help you on your way.

Good luck -- it's been fun!



Saving Time and Money with Standards

When starting development on a new application, there are dozens of unanswered questions. For example:

- What will an entry screen look like?
- Will all the query screens be tied together?
- How will reports be organized?
- What standard features will each program have?
- What will the action bar look like?
- How will function keys be used?

Of course this is just a small subset of a much larger list. As you consider all of the user interface issues and behind-the-scenes implementation variations, this list can grow to several pages.

How you answer these questions can either cost or save you significant time and money. To illustrate this, let's look at a couple of different scenarios of how people have dealt with standards:

- Person A works alone. He has designed the software and will be implementing it without any assistance. Rather than waste time creating and adopting standards (as he puts it), he jumps right into development, and works diligently to create his wonder-product.

  All is fine and good for a few weeks. One day, however, he realizes that the task he's faced with is something he's done prior. This gives him a couple of options. First, he can take the time to go back and figure out how he did it the first time. Or, he can simply re-create what he did before, though notably he may or may not do it the same way.

  If he takes the first option, he's investing time that could have been saved by formulating standards up-front. With standards, he could ensure consistency throughout the implementation while avoiding the questions of how to do something he's done before.

  If he takes the second option, there's no guarantee that he'll do the job the same way the second (and subsequent times). Depending on the complexity of the task, this could take him quite a bit of time that could have been avoided if standards were in place.
- Person B works as a part of a team with persons C and D. Because of the tight deadlines that the team has been facing, they have corporately decided to forego standards, as "there just isn't enough time".

  A few days into the project, one of the team members gets sick and the others are called in to pick up the project. As they look over the code, neither person can figure out what the author was thinking, and as a result they start rewriting the code.

  If this team had taken the time to formulate standards, they wouldn't be wasting time rewriting sections of code simply because they didn't understand what the author intended. It is possible that the author was following a significant implementation concept which could have saved the company millions of dollars over a period of years. However, because the team didn't follow a standard approach, the company is investing in rework, not new technology.

Avoiding standards because "it takes too much time", is the first step towards rework. Unfortunately, rework is a sinkhole -- you never save time or money doing a job over. To reinforce this idea, here are a few great adages:

- Anything you have time to do, you have time to do right.
- There's two ways to do a job: 1) Right, and 2) Again.
- People never plan to fail -- they merely fail to plan.

Always remember that whenever you decide to answer questions later, you'll inevitably be answering the same questions over and over. It's better to approach the problems head-on, formulate a plan, and then follow the plan.

The following sections are intended to provide some food-for-thought regarding standards for many of the issues you may face. It is, as standards are, one opinion. Whether you use these as-is or adapt them is not important. What is important is that you have standards.

Standard Abbreviations

When creating an application with SB+, you will create dozens (if not thousands) of records. Many of these records will be named by you. For example, when creating field definitions, you will decide the names of each of the fields in each file of your application. Similarly, when creating many types of processes, you must first decide on a proper name for the process.

Standard abbreviations are standardized shortcuts for commonly accepted terms. For example, if the commonly accepted standard for the term "payment" is "PMT", and "date" is "DT", a field called "payment date" can be given the name "PMT.DT". Without standard abbreviations, the field could be called "PAYMENT.DATE", "PMT.DATE", "PAYMENT.DT", or any number of other permutations, thus complicating the file layout and possibly causing confusion for other members of the development team.

Standard abbreviations are an effective tool for helping someone accurately "guess" the name of a particular record. Without this advantage, a person could waste significant time searching for a particular field, screen, or other resource.

Before any actual coding is started, work to create a list of standardized terms and create standard abbreviations for each of them. (Some abbreviations may actually match the original term, such as "GAS".) If you're new to SB+, consider creating an application for tracking your standard abbreviations as a "warm-up" application. This type of application provides not only a good training opportunity, but also gives your team a place to go when there are questions regarding your standard abbreviations.

System Standards

One Application/One System

All processes, files, definitions, programs, and other resources used to create a single application, regardless of the number of modules or programs, should all be kept in a single SB+ system. It is difficult to share standard processes when one application is broken into several systems (version 2.x).

Don't Create A System Called "DM"!

DM is the prefix used for the SB+ control files. Older versions of SB+ have allowed a new system called DM to be created, which caused great confusion. In other words, even if SB+ allows you to do it, don't.

File Standards

File Naming Standards

Files are to be named with brief but descriptive names. Standard abbreviations should be used whenever possible. For example, if the standard abbreviation for the word PAYMENT is PMT, the "payment" file would be named "PMT".

Creating Files Inside/Outside SB+

All files created for an application are to be created using the SB+ Create File tool. This will not only create the actual file, but will also update a list in SB+ of all of the files created for the application. Using CREATE-FILE at the TCL level, on the other hand, does not update this list.

Temporary files which are not intended to be a part of the finished product can be created using CREATE-FILE and deleted using DELETE-FILE.

Using Split Dictionary/Data Files

If an application is a candidate for translation to another language, two files should be created for each file used by the application. One file, a dictionary, will exist in the local account (where the processes and definitions are stored). The second file, a data file, will exist in a separate account where it can be accessed by several different language accounts. The local account will then use Q-pointers (or system equivalent) to access the shared file.

This structure is also useful for configuring an application for quick backup and restore. With split dictionary and data accounts, the dictionary account will rarely change. The data account, however, will constantly be changing. Therefore, rather than routinely backup a full file system including both programs and data, split dictionary/data accounts allow a person to backup only the data account, thereby saving time in the backup and restore procedure. Of course, master backups of all accounts on a system should be done regularly and kept available in the event of damage to the dictionary or data accounts.

Field Definition Standards

Field Names

Field Names should adhere to your defined list of standard abbreviations. For example, if the standard abbreviations for "order" and "date" are "ORD" and "DT", respectively, the "order date" field will be called "ORD.DT".

Key Field

Every file that is created will have a key field defined called xxxxx.ID, where xxxxx is the name of the file. For example, a file called PMT will have a key field defined as PMT.ID. Other fields may be used to define the key, but this field should always be present.

For files with multiple part keys, one field should be created for the key as a whole, several fields should be created to extract each part of the key (for reporting and selection purposes), and work fields should be created for entering each part of the key.

Common Fields

A field that is used in multiple locations will have the same name in all locations. For example, if a customer ID exists in both the Customer and Sales Order files, the field name will be the same in both places. The only exception to this guideline is when a multivalued list of IDs exists on a file, such as multiple stock numbers on an order record. In this situation, instead of naming the field STOCK.ID, the field should be named STOCK.XREF.

These rules will help developers to:

- Easily find fields based on common keywords, and
- Visually see file relationships based on the xxxxx.ID and xxxxx.XREF fields.

Field Descriptions/Report Headings

All field descriptions/report headings are to be entered in mixed case, with the first letter of each word capitalized, and the remaining characters in lower case. Common abbreviations (not necessarily standard abbreviations), such as "ID" for "identifier" are acceptable if used consistently throughout the application.

Spaces are to be used to separate words in a descriptive field.

Field Types

Date fields will always be entered as type "D" fields. Numeric fields that do not represent monetary amounts will be entered as type "N" fields. Numeric fields that represent monetary amounts (regardless of the currency) will be entered as type "M" fields. All other fields, including time fields, will be entered as type "A" fields.

Field Length

The following table illustrates standard field lengths for different types of fields. This table is only a guideline and should be adjusted to fit the needs of your own application.

Field Type	Length
Alphanumeric	n/a
Numeric	10
Date	9
Monetary	12

There is no standard length for alphanumeric fields, except when the field contains a time. In this situation, the standard length is determined based on the standard time conversion for your application. For other types of alphanumeric fields, care should be taken to ensure consistency in field lengths throughout the application. For example, if a field called NAME is used in several locations, inasmuch as possible the lengths for all occurrences should be the same.

If the length of a field must be shortened on a screen or report, the field length should be adjusted on the actual screen or report definition where it is used, not on the field definition.

Derived Values

Whenever a value must be calculated, such as a customer name based on the customer ID, a **derived value field** should be used. Derived value fields must always be expressed as SB+ expressions, instead of using operating environment syntax conventions. (Certain OE correlatives do not perform as expected in an SB+ application, and SB+ derived values are significantly easier to port to other environments.)

Default Code Standards

Strive to have a fully parameter-driven application. To this end, do not use literal defaults, such as "1" or "A". Instead, use either calculations to derive a default value, or read the defaults from a control record stored somewhere in the application.

Validation Standards

Avoid using the V: validation code, as it compares the last entry to a literal list of values, and therefore cannot be parameterized. Use a table validation instead. Remember that when your application gets large, there will be literally hundreds of drivers that will get cleared when you change a field definition. This can contribute to system degradation as the drivers are rebuilt during run time. By using tables to store validation codes, system degradation can be avoided, as drivers are not cleared when a table is changed.

If you use the E:(...) validation option, and the expression you're trying to evaluate contains more than two sets of parentheses (not counting the two shown above), or the expression contains more than two commas, use the C: validation instead. This will dramatically improve the readability of the application.

If a validation code requires more than one AND or OR, use the C: validation code instead. By creating a paragraph (or other

process) to do the validation you can have much more control over how the validation is executed.

If a validation requires more than one error message, use the C: validation option.

When using the F: validation option, be careful to allow for the situation where a null entry is valid for the field. Often, instead of:

F:file{error message}

...the better validation is...

"" OR F:file{error message}

Using this improved syntax, the field can be configured as mandatory separate from the validation. (NEVER count on validation codes for checking a mandatory entry!)

Avoid calling BASIC subroutines for validation unless the validation required simply cannot be done any other way. If a BASIC subroutine is required, create a BASIC process and call the process using the C: validation option, not the B: validation option.

Always refer to BASIC subroutines as processes. This way, you can quickly see how BASIC is used in your application simply by selecting and reporting on the BASIC processes in your process file. (You cannot look at the programs file because it is a mix of hand-written subroutines and generated code, and it is very difficult to distinguish between the two.)

Validation Error Messages

If at all possible, do not use the SB+ standard user error message item. It grows too large too quickly and will eventually cause system degradation when an error message is displayed. Instead, a more flexible approach is to create an error message file for your application and store each error message in a separate item. Then, to specify the error message at the end of the validation string, use:

{[Ufile,id<amc>]}

In this syntax, "file" is the error message file name, "id" is the name of the record key for a message, and "amc" is the attribute (usually 1) where the message is stored.

Intuitive Help

Intuitive Help on a field definition should call a process rather than use inline S:SELECT phrases. Long S:SELECT phrases are difficult to read and modify, and can often be replaced by equivalent and more understandable Selection processes.

Don't use the user select feature, where all of the selection phrases are stored in the XXXCONTROL SELECT record. There is no facility (other than the system editor) for editing this record, and it can grow very large very quickly. Instead, use Selection processes consistently throughout the application.

F1 Help Standards

F1 Help must exist for all field definitions that are used on a screen or in a selection expression. The first level of help, which is displayed when the F1 key is pressed the first time on a prompt, should provide a short message to tell the user what to enter at that particular field.

Second level F1 help (a help box) is displayed when the F1 key is pressed a second time, and is valuable and recommended. Care must be taken, however, to ensure that the help box does not overlay the screen is such a way as to make the screen difficult to read. Also remember that one field can be used in a variety of locations (screens, selections, etc.) and the help applies to all locations where the field is used. The text of secondary F1 help should briefly explain to the user a broader picture of how the field applies to the program or application at large.

Third level help, which is displayed when the F1 key is pressed a third time, is also valuable and recommended, but only for those prompts where first and second level help are inadequate. The text of third level help should explain everything possible regarding the field so the user will have no difficulty understanding why the field is requesting an entry.

Fourth level help, which is a menu that is displayed when the F1 key is pressed a fourth time, may be used as necessary. It should not, however, call the SB+ General Help menu, as this information is not generally valuable to the common user. The General Help menu should be disabled before the application is released for general customer use.

Screen Definition Standards

Input/Output Screens Sharing Definitions

No input screen definitions will be used for output processes. Likewise, no output screen definitions will be used in input processes. No screen definitions will be shared between an input process and output process, as the requirements of each type of program can be significantly different. Attempting to create a workable screen definition for both input and output can be a time consuming and frustrating waste of effort.

Naming Screen Definitions

The first entry screen created for any file on a system will be called ENTRY.A. **Subscreens** to ENTRY.A will be called ENTRY.B, ENTRY.C, ENTRY.D, etc.

For applications where there will be no direct entry of information to a file, but maintenance will be allowed, the first maintenance screen will be called MAINT.A. Subscreens to MAINT.A will be called MAINT.B, MAINT.C, etc.

The first inquiry screen created for any file on a system will be called INQ.A. Subscreens to INQ.A will be called INQ.B, INQ.C, INQ.D, etc.

For other types of entry or inquiry applications, the screen should be named with a common base name, with the ".A", ".B", etc. suffix. Note that the file name should not be used in the screen name. For example, if a customer file is named CUST, naming a screen CUST.A would result in an input process called I*CUST*CUST.A, which is redundant and not all that informative.

Any screen name that ends in ".A" is a primary screen. Any screen that ends in any other suffix is a subscreen. Primary screens and subscreens should use the same base name, such as "ENTRY" or "MAINT" as shown above.

Linked screens (screens named with numeric suffixes) are to be used only when necessary. Often, it is better to use subscreens invoked through function keys or process slots, as this provides a more flexible and predictable user interface for multiple screen programs.

Prompt Positioning

Single value prompts will always prompt to the left of the input area. Multivalued prompts will always prompt above the input area. Prompts to the left of the input area should always line up in the same column, and all input areas should align on a column. There should never be more than 2 spaces between the last character of the longest prompt and the first character of the input areas.

Comment fields that comprise an entire subscreen will not have a prompt displayed. The title of the subscreen will serve as the prompt.

Using QuickBuild

QuickBuild is not to be used for constructing any screens for the application. It's a great demonstration feature, but hopefully more

planning is involved when constructing real world applications.

Screen Layout

All screens will follow a standard layout, as follows:

- The top line of the screen (row 0) will be left blank, or will contain a standard title for the application in general.
- The second line of the screen (row 1) will be reserved for a standard action bar which will be displayed at all times during the input/output process.
- The screen window will be displayed starting at row 2, column 0, and will proceed to row 79 of column 21. This will reserve the final two lines of the display for help and error messages. If this default size is too large for the information to be displayed, the window should be reduced to an appropriate size, and centered on the display. Don't leave a significant amount of empty space on the screen, as it gives the illusion that something is missing.

Using the Accept Screen Prompt

Screen definitions will not use the "Accept/Change/Delete/File" prompt, as it is an SB+-ism that is inconsistent with the rest of the world. Instead, function keys F2 and F4 will be used to save and delete a record.

Changing a Field Length/Conversion

If a specific field on a screen requires a modification to the field length or conversion, the modification will be done on the screen definition, rather than on the field definition. Once field definitions have been created, the length and conversion should never be modified during the
construction of the application, unless a change to the field definition is intended to affect all locations where the field is used.

Creating Temporary Fields

Temporary fields, which are created when you enter a numeric field name or Wn field name in the F5-field window, are not to be used. Fields placed on a screen are therefore guaranteed to have a corresponding field definition, which aids in researching how fields are used in a given program and also in the application at large.

Overriding Intuitive Help

If a specific field on a screen requires an intuitive help process which is different from the intuitive help defined on the field definition, the appropriate intuitive help process will be defined on screen definition, and the field definition will remain unchanged. Once field definitions have intuitive help defined, you should never change them, as doing so can have negative repercussions on other programs in the application.

Function Key Standards

Function keys will be defined for all screen definitions used for input as follows:

F1-Help
F2-Save

F3-LookUp
F4-Del
F5-user defined
F6-user defined
F7-user defined
F8-user defined
F9-user defined
F10-Menu

If an application does not permit records to be deleted, F4 will be unassigned. It is not appropriate to reuse F4 for any function other than to delete a record.

If an application requires more functions than can be provided through F5-F9, F9 will be converted to F9-More, and will call a "More Menu" of additional selections. The "More Menu" is simply a menu that contains a list of all processes that this screen needs, minus those already accounted for with function keys.

For screen definitions used for output, the function keys will be defined as follows:

F1-Help
F2-Accept (or Done)
F3-LookUp
F4-do not use
F5-user defined
F6-user defined
F7-user defined
F8-user defined
F9-user defined
F10-Menu

Again, if the application requires more functions than can be accommodated by F5-F9, convert F9 to a "More Menu" as described earlier. F4 should not be defined for output screen definitions.

When a "More Menu" is used, the menu must fit completely inside of the calling window, with the lower right corner of the menu anchored on the lower right corner of the screen window. (There should be 1 space between the right border of the menu and right border of the window, though there are no lines between the bottom border of the menu and bottom border of the window.)

F10-Action will always call the process ACTION to invoke the action bar.

Using a Standard Action Bar

For every screen, an action bar will be displayed while the screen is running. This is accomplished by creating externally defined function key/action bar definitions and linking them to the individual screens on the F6-Params window in the Screen Definitions tool. Each function key/action bar defined will have Display Code = 2, which will display both the function key and action bar while the screen is running.

For input screens, the action bar will contain the following options:

File	This option will recall a "File Menu" containing options to start a new record (New), save the record being edited (Save), print the current screen (Print), and quit the application (Exit).

Toolbox	This option will recall a "User Menu". This user menu should contain tools that are useful to anyone using this screen, such as the calculator, calendar, or perhaps even the SB+ TCL process. Another useful tool to include here is a program which can be used to run any other program in the application.
Query	This option will recall a menu of standard inquiry programs. This will allow any user at any time to inquire on information in the application, as security allows. (Note: "Query" is used in place of "Inquiry" or "Enquiry" to avoid conflicts when the application is used both in and out of the USA.)
Office	This option will recall an "Office Automation Menu". This menu will contain the standard SB+ office automation tools for sending and receiving messages, the diary, and contact update.
Help	This option will recall a full screen help display which documents the purpose and functions of the screen being used

For output screens, the action bar will be the same, except that the "File Menu" will not contain a "Save" option.

Typing Text Onto a Screen Definition

Rather than typing prompt labels onto the screen definition, use the prompt labels from the field definitions for the fields on a screen. Non-prompt text must be typed into the screen definition, and should be used only when necessary.

Certain language translation tools require all text to be typed onto a screen definition, which is acceptable because it is used consistently throughout the application. Other language translation tools frown on typing text onto the screen definition. In other words, if you know you may be concerned about language translation at some time in the future, know the tool you intend to use to get you there before making a policy decision on this issue.

Line Numbered Prompts

All input prompts on an input screen definition are to be prefaced with a two-digit sequential number, (such as 01, 02, 03, etc.) followed by a space. No prompts on an output screen definition are to be numbered. If at all possible, prompts should not be manually numbered. Instead, use a utility to automatically number the prompts. This takes the prompt numbering drudgery away from the developer and ensures that the prompt numbering will always be accurate each time the screen is run.

Using Screen Graphics

Screen graphics (lines, boxes, etc.) are to be used very seldom, as they tend to cause a screen to appear overly complicated. Horizontal and vertical lines may be used to separate sections on a screen, but only when the different sections would otherwise be difficult to differentiate. Use boxes to accentuate important information, such as an order total, but use them sparingly.

Report Definition Standards

Using Query Reports

Query reports are not to be used. The user interface for query reports is different from SB+ Report Writer reports, and the query language is not as powerful. Rather than confuse the user with varying standards for viewing a report, use the SB+ Report Writer tool exclusively.

Naming Reports

Reports are to be given common sense names, using standard abbreviations whenever possible. Avoid using the name of the file in the report name. Common report definition names include "FORM" (such as an invoice or packslip), "LIST" (a list of information), and "INFO" (a collection of information).

Outputting a Report

Generally speaking, all reports will prompt for the output designation (Screen/Print/Aux/etc) at run time. This will allow the user to choose where the report will be output each time the report is used. The exception to this rule is any reports that must generate output, such as a report that a customer is being billed for. In this situation, the report must not prompt for the output designation, but must run to the printer only.

Standard Report Width

All columnar reports will be 132 columns in width.

Standard Heading

All standard columnar reports will have a standard heading, which includes the information as follows:

Report Name

Run By: user information	Date/Time: dd mmm yy hh:mm:ss
Selection: selection information	

The report title is derived from the process definition (which is read at run time) and will be centered on the first line. The box will begin on the next line and will be 4 or more lines long (depending on the maximum length of the selection statement). Inside the box the words "Run By:", "Selection:", and "Date/Time" will be typed onto the report template. The "user information" can include the user ID (from the **common variable** @USER.ID), the user name (from DMSECURITY or other source), or any other information you choose to display. The "selection information" will show a description of the **Selection Criteria** used for the report, and can be extracted from @OTHER(8).

The time field shown in this heading should not use the standard @TIME function, as doing so will cause the time will change from page to page. Instead, extract the starting time from @OTHER(8), and use @DATE for the date. For reports that will be running over several days, calculate the current date at the start of the report, store it in common memory somewhere, and retrieve the value from common memory for the report heading.

There will be a blank line following the heading before the column heading.

All form reports, such as checks and invoices, will be formatted according to the actual paper form the report is to print on, and are not required to have this heading.

Standard Footing

All columnar reports will have a standard footing, formatted as follows:

Company Name Page n Report Process Name

The company name will be derived from XXXCONTROL PARAMS. The page number will be derived from @RV.PAGE. The report process name will be the actual process name of the report, as found in @PROC.NAME<1>. The report process name is included to aid in process identification when users request changes to the report.

General Formatting Rules for Columnar Reports

Sort fields and break fields should appear clustered on the left side of the report. Fields being totaled should be clustered on the right side of the report. Other informational fields should appear near the center of the report.

Each column of information should have a column heading, and there should be one blank line between the column heading and the first row of information.

Using Derived Values

If a calculated field is required on a report, use a **derived value on the report** instead of creating a **derived value field** in the dictionary of the file. Derived values are used for all fields in the report heading and footing, and are also used exclusively for conditional fields.

When naming derived fields for a detail line in a report, preface the name of the derived field with "R." to denote that the variable exists only on the report. For conditional fields, preface the name of the field with "C.". Never create a field definition starting with "R."!

If the derived value expression contains more than 2 sets of parentheses or 2 commas, use the P(...) function to call a paragraph to calculate the derived value. This will greatly improve the readability of the report. If several processes need to be called this way, consider writing one process to called in the Process After Read to calculate all values to be output in the detail section. This one process can then call several other processes to calculate each field. By calling each of these processes from a central Process After Read, readability is further improved.

Using Break Fields

When defining a report with break fields, the report must output one blank line before each break and one blank line after each break. To do this, mentally separate the break section into several subsections. The first subsection in the report definition should be the most minor break, the second subsection should be the next level break, and so on until the last subsection in the break section is for the major break. The most minor break subsection should start and end with a blank line. All other subsections end with a blank line, but do not start with one. Conditional fields are then used to conditionally skip the break subsections that do not apply for a given break when the report is running.

Generically speaking, for a report with three breaks use the 322 rule: Three lines for the minor break (blank line, info line, blank line), 2 lines for the intermediate break (info line, blank line), and the same for the major break.

Typing Text Onto A Report Definition

Text may be typed onto a report definition for headings, footings, break lines and grand total lines. Caution should be taken, however, when typing text onto a column heading line. Instead, try to use the report headings from the field definitions when placing fields on a report. Again, like with screens, know your language translation tool before making a policy decision on this issue!

Using Graphics

Like screens, use graphics sparingly on columnar reports. Graphics should be used to draw attention to specific details like break totals or grand totals. Graphics on each detail line of a report is overkill and should be avoided.

Graphics can be used on form reports, but should be used only as needed. Avoid being clever with graphics, as it tends to distract from the purpose of the report.

Using Stationery/Printers

Printer and stationery setup should be complete prior to the definition of any reports. This will prevent last-minute changes to reports based on an unknown type of paper. Be certain to identify the desired stationery to the developer prior to having a report developed so the dimensions of the paper are known. Defining stationery or printers after reports are completed can result in unnecessary setbacks caused by reports that have been created based on unknown form sizes.

Templating

Before creating your first report, create a standard template for all columnar reports. This standard template includes the standard heading and footing with blank detail, break, and grand total sections. Once this has been defined, it can be copied to a new name for each report that is created. This will ensure a consistent format for all of the reports you create.

Menu Definition Standards

Naming Menus

Menus can be used for two purposes. The first, startup menus, are the most common. These are the menus which are used to start the programs in your application. The second, process menus, are those menus that play a supporting role in a program, such as a "More Menu" (see function keys).

Startup menus are to be named with a 2 or 3 letter prefix, followed by three digits, right justified and zero filled. For example, a menu used to start AP programs may be named AP001. A menu used to start GL programs may be named GL020.

Process names are named according to the process naming standard, which is a 2-4 letter prefix followed by 4 digits, right justified and zero filled. A process menu may therefore be named AP0304 or GL0221.

Menu Types

The first menu a user sees when they log into an application (i.e. the main menu) will be a type 2 (horizontal) menu. This menu should provide the following options to the user:

Menu	When this option is selected, the main menu for the application will be displayed. From here, the user will select one or several programs to be executed.
Toolbox	When this option is selected, a user menu will be displayed. This menu can be the same as, or a version of, the Toolbox as selected from the standard action bar.
Office	When this option is selected, the standard SB+ office automation menu will be displayed.
Help	When this option is selected, a menu of help topics for your application will be displayed. This menu will vary from application to application, and should be the place users go to get general information about the application as a whole.
Logto	When this option is selected, the standard SB+ LOGTO process will be invoked.

All subsequent menus in an application are type 1 (vertical) menus. When placing a type 1 menu under the type 2 menu, the upper left corner of the type 1 menu must align exactly under the first character of the selected option in the type 2 menu.

Menu Titles/Heading

Horizontal menus will not have a heading. Vertical menus must have a heading. The heading of the vertical menu must be the same as the title used to select the menu from its caller. For example, if a menu selection is displayed as "Menu" on the main menu, the title of the menu invoked will be "Menu". If a menu option called "Inquiry Programs" invokes a menu of inquiry programs, the menu invoked will have the a title of "Inquiry Programs". This standard helps assure users that they have made the right choice.

Standard Menu Positioning

Vertical menus called from the main menu must have the upper left corner of the box align exactly under the first letter of the

selection option on the main menu. The vertical menu must not overlay any part (including the window border) of the main menu.

Vertical menus that are called from other vertical menus must be 12 characters to the right and 2 lines down from the upper left corner of the calling menu. This promotes consistent cascading of the menus from left to right. The "Help" menu, which is the rightmost option on the startup menu, may invoke menus that cascade to the left, given that the upper right corner of the called menu is 12 characters and 2 lines away from the upper right corner of the calling menu.

Menu Option Descriptions

If a menu is used to invoke an input, output, report, or periodic update process, the option description on the menu must match the title of the program being invoked. This will provide a consistent and predictable interface for the user, thereby minimizing confusion and unnecessary support calls.

Using Function Keys on a Menu

As a general rule, function keys should not be used on menus. If necessary. however, function key F10 may be used to invoke a standard action bar for menus.

Using Paged Menus

If a menu is so large that all of the options cannot fit on the screen at the same time, break the menu into several submenus so that each menu can fit all options onto the screen at the same time. Creating menus with several pages is not recommended, favoring instead smaller menus or well structured type 3 (painted) menus.

Creating Menu Processes

If a menu is intended to be used as a startup menu, do not create a process for the menu. If the menu is intended to be a process menu, create a menu process named exactly the same as the menu definition. For clarity, the description of the menu process and the heading on the menu should match.

Periodic Update Definition Standards

Naming the Periodic Update

Periodic Update Definitions are to be named according to process naming conventions.

Creating a Periodic Update Process

For every periodic update definition created, a corresponding periodic update process must also be created with the same name.

Using the Period Check

On the "Period Check (nn-nn)" prompt of the Periodic Update Definitions tool, always use "-". Use external controls (Process Before, etc.) to better manage when the update should and should not be run.

Using Dialog Boxes

Any periodic update which is used to delete records from a file must have a dialog box warning called from the Process Before (or earlier) to display a clear warning to the user. This dialog box must have a "Cancel" button (shown first), and a "Proceed" or "OK" button to allow the user to run the update despite the warning.

Dialog boxes are recommended, though not required, on all periodic updates to warn the user of the changes that the periodic update will make.

Using the Process After Read

If the updates to apply to each record contain several conditions, or are intricately structured, use a Process After Read to update each record (or to update secondary records).

For periodic updates that perform a n:1 update, the Process After Read is the best option.

Ending Messages

Whenever possible, periodic updates should have an ending message which tells the user what happened, such as "xxx customers were processed, yyy were updated". This message is best shown in a dialog box, with a "Continue" or "OK" button.

Process Definition Standards

Naming Processes

Input, Output, and Report processes are to be named using the SB+ default naming convention for these types of processes (i.e. I*file*screen, O*file*screen, R*file*report, respectively). Other process types (paragraphs, BASIC, etc.) should be named using 2-4 letter prefix followed by a sequential number, right justified and zero filled to four digits. Note that this standard applies to routinely created processes -- Standard processes (discussed later) follow a different naming convention.

For example, a paragraph to validate an entry in the AP application may be named AP0124. Similarly, a process called after read on a GL screen may be named GL0431. Care should be taken to use only a few alphabetic prefixes per application to avoid confusion.

SB+ can automatically assign process names using this standard, which saves a significant amount of time and research into whether a particular process has been used. There are products available to do this.

Using Standard Processes

"Standard processes" are those processes that are used over and over in an application, each providing functionality that is not specific to any given program in the application. Some popular uses for standard processes include:

- Displaying an error message
- Rationalizing differences between two lists of IDs
- Getting a number from a custom accumulator
- Adding the port number to a "NEW" key before locking the record

This is a very small list, of course, as there can be anywhere from 20 to 100 (or more) standard processes for any given application.

Unlike the normal process naming convention, standard processes are typically given mnemonic names. This is done to make these special processes memorable so that they can be quickly used when needed, without forcing a developer to search through the process file (or a separate listing) for the right process. Nonetheless, a comprehensive list of standard processes should be maintained and available to the development team at all times.

Using the Process Description

Each process must have an accurate and complete description. In the case of Input, Output, and Selection Processes, the description must be a reasonable title for the window, as these titles will be shown to the user when the application is running.

Inline Processes

Inline processes (M:menu, U:update, P:(paragraph)) should not be used. By creating actual process records for each process in the application, someone following you can get a better picture of the all components for an application, and can also analyze the utilization of different process types throughout the application. Inline paragraphs are also very difficult to view, as there is often

more text scrolled off of the screen than can be viewed in the process slot.

Process IDs are Required

Though some of the tools will allow you to create a process without an ID, don't do it.

Process Types

Do not use Assignment, Conditional, Default, File Update, Go To, or Validation Processes. The functionality of each of these process types can be easily implemented using SB+ paragraphs, and the paragraph language is more flexible and versatile than these other process types. In cases where paragraphs are inadequate, such as performing a LOCATE(...) on a sorted list returning an insertion position, use BASIC processes. Even in this instance, try to generalize BASIC processes so your application has very few of them. Fewer BASIC subroutines translates to less difficulty when rehosting the application to a new operating environment.

Paragraph Standards

Paragraphs using **local variables** should declare all local variables on the first non-comment line(s) of the paragraph. Though local variable declarations can span multiple lines, all local variables should be declared together.

When naming local variables, use brief and descriptive names for each variable. Use extra caution to prevent unassigned variables when the paragraph is running. Also, be very careful not use a field name when naming local variables. (SB+ will allow you to define a local variable with the same name as a field name. While it may seem like a small mistake, it can be a very difficult mistake to find!)

Use comments liberally throughout the text of the paragraph to explain what the paragraph is doing, even if you think it is obvious.

If a paragraph cannot be done in 60 lines or less, break it into smaller paragraphs. Similarly, if a paragraph is reading and/or writing more than one record, attempt to fragment the paragraph into separate smaller paragraphs, one paragraph per file being updated.

Using **local equates** in a paragraph is a good practice as it can improve the readability of the code.

@OTHER.REC<n>, @WORK<n>, and @RECORD<n> may be abbreviated On, Wn, and <n>. Use care when using these shortcuts, however -- don't sacrifice readability for a few characters of text.

Selection Process Standards

Use care in entering a description for a selection process that prompts for input using the "?" feature. Whatever you enter for the process description will become the title for the prompting box.

Any fields that are prompted using the "?" feature must include the input conversion. Validation and default should also be included if defined. At a minimum, "?I" should be used for each prompt, and at best "?VDI". The "M" option, for a mandatory entry, may also be combined with these options as the application requires.

All selection processes that display a box in the lower left corner of the screen must include the "T" option to show the field descriptions above the information in the window. The "F" option may also be used (to force the window to be displayed if only one record is selected) if it is applied universally to all selection processes in the application.

BASIC Standards

Using Processes for BASIC Subroutines

For every **BASIC subroutine** created, create a **BASIC process**. This provides a way for you to quickly select all of your hand-written BASIC subroutines from the XXXPROGS file, without having to filter through all of the code that may have been generated by SB+.

BASIC Source Code Heading

Every BASIC subroutine should have a standard heading in the source code. This heading should tell who wrote the subroutine, the date it was written (or started), and a modification history explaining who modified the routine, when, and why. For example:

```
SUBROUTINE XXX.YYY0000
*
* Written By: (your name)
* Date: (current date)
*****************************************************************
* Modifications History:
* Date. ... Changed By..... Description of Change.....
*
*****************************************************************
*
* Include(s)
*
$INCLUDE DMSKELCODE COMMON
.
.
.
RETURN
```

Using Standard Conversions

If a BASIC subroutine is required to perform a date or monetary conversion using OCONV(...), the standard conversion should be retrieved from XXXCONTROL PARAMS. This will aid in converting the software to a different language or environment. Be aware that the standard conversion may have the "[I" suffix, which will have to be removed before the conversion is used in an OCONV(...).

Review Uses of BASIC

All uses of BASIC should be reviewed prior to creating the actual code to ensure that BASIC is actually necessary. The less BASIC written in an application, the easier the application will be to rehost.

Expression Standards

Intrinsics to Avoid

Avoid using nested IF(...) and CASE(...) intrinsics in expressions. If such constructs are required, use the P(...) intrinsic to call a paragraph to do the IF or CASE. This will make the application more readable.

Avoid using the B(...) intrinsic, favoring P(...) instead. When BASIC is called using B(...) it circumvents SB+ process level security. Besides, you should always have a process for any BASIC that has been written.

Using Expression Process Calls

If an expression is longer than 50 characters or has more than 4 sets of parentheses or 4 commas, consider calling a process using the P(...) intrinsic. This will make the expression more readable, particularly if the paragraph being called is well commented to explain what it is doing and why.

When using the P(...) intrinsic, take care to ensure that the process being called actually sets the **common variable** @VALUE to the proper value. Feel free to have one process call several other processes in setting this value, particularly if the structure helps the readability of the code.

Avoid Multiple F(file,item)<amc> References in Expressions

Use caution to avoid using multiple F(file,item)<amc> references in an expression, where the file and item ID reference the same item. Doing so will cause a READV of the same record for each occurrence, and as a result your application will be I/O bound. Instead, write a paragraph where the record can be read once, the result calculated, and a single value returned.

SB+ Accumulators

All SB+ accumulators are stored in a record called GENNO in the XXXCONTROL file. Because all accumulators are stored in the same record, there is great potential for lock contention, which can cause a serious bottleneck in your application. Also, because SB+ references accumulators by number, it is easy for a developer to inadvertently reference the wrong accumulator.

To remedy these problems, you may want to consider constructing your own application for storing and updating sequentially assigned numbers, referring to each accumulator by a descriptive (not numeric) key. However, should you decide against this approach, favoring SB+ accumulators instead, be sure to "claim" each accumulator when you use it. To do this, change the description for an accumulator from "ACCUMULATOR xxx" to a descriptive name to let others know you are using the accumulator for a specific purpose.

Using Global Equate(s)

When a particular expression will be used extensively throughout an application, consider creating global equates to improve the readability of your code. for example, if several locations in the application reference F("DMSECURITY","~":@USER.ID)<21> (the user name), you may use a global equate to give this the symbolic name USER.NAME. Once defined, this global equate can

be used everywhere the user name is needed, instead of the less readable F(...) as shown above.

The following table lists some useful global equates:

Equate Name	Expression
AMC	@LINE<1,1>
CONV	@LINE<1,5>
INP.CONV	@LINE<1,10>
DFLT.PROC	@LINE<1,12>
VAL.PROC	@LINE<1,13>
REQD	@LINE<1,14>
FIELD.ID	@LINE<1,18>
GROUP	@PASS.DEFN<14>
EDIT.KEY	@OTHER(18)
NO.ESC	@OTHER(18) # 14
UP.ARROW	@OTHER(18) = 3 OR @OTHER(18) = 25
USER.NAME	F('DMSECURITY','~':@USER.ID)<21>

With these global equates defined, developers can use /EE on any prompt on a screen to view the attribute number, conversion, validation code, or any of the other information as listed. This information, used with the /MP process, can help you rapidly and accurately debug your programs.

Dialog Box Standards

Naming Dialog Boxes

Dialog boxes are named using the process naming conventions.

Naming Dialog Box Buttons

Dialog boxes with two buttons for positive/negative responses should have the positive response button labeled "Proceed" and the negative response button labeled "Cancel". An alternate standard is to have the text of the dialog box pose a yes/no question, with the affirmative button labeled "Yes" and the negative button labeled "No".

Dialog boxes with one button will have the button labeled "Continue".

Take Care With @RTN.FLAG!

Care should be taken with dialog boxes so that the value returned by the dialog box (in the **common variable** @RTN.FLAG) isn't misinterpreted by the calling process.

Code Table Standards

Code Table Files

In a typical application, there will be code tables that will be updated solely by the application developer and other tables that will be updated by the users of the application. Tables that are updated by the developer should be stored in the XXXDEFN file. Tables that are to be updated by the user should be stored in the XXXTABLES file (a new file, where XXX is the current system ID).

By doing this, upgrades and revisions that are installed after the software has been in use will not overwrite or destroy any user tables that are important to the application.

Naming Code Tables

Code tables should be given mnemonic names according to their purpose, using standard abbreviations.

Security Standards

Use a Small Group Structure

When setting up SB+ group security, avoid creating complex security structures that are more than 4 or 5 levels deep. Anything more complicated creates an administration nightmare as accounts are added and removed from the system.

Don't Put Users in ROOT!

Don't put users in the ROOT group. Doing so can cause more problems than it is worth when you upgrade SB+ to a new version. Instead, create a group under ROOT called "SUPER" (merely a suggestion) with the same rights as ROOT. Assign your superusers to this new group. Any subordinate groups should then fall under this group.

When an SB+ upgrade doesn't proceed as planned, you can count that ROOT will be essentially destroyed. The less you have hanging off of ROOT, the easier it will be for you to recover.

Assigning User IDs

Be consistent in assigning user IDs. In other words, if your user IDs are 4 numeric characters, use this standard for all users.

Common Map Standards

Don't Touch FILES.OPENED or FILEVAR

Changing these values can cause your programs to behave very, very strangely.

Define @PARMS(1) for Application Control

The **common variable** @PARMS(...) is a 40 position array which can hold anything. As a standard, allocate @PARMS(1) to hold information that will be needed by all applications. For example, you may use this to hold additional security information, or simply to hold frequently accessed parameters.

Work Fields

When a work field is used, create a field definition in the current file to "claim" the work field. This provides some documentation to someone in the future showing which attributes of the @WORK common variable have been used.

Miscellaneous Standards

Modifying SB+

SB+ processes, drivers, definitions, and other resources should never be changed.

Creating Custom Tools

Though SB+ provides a wealth of tools and resources for application development, every now and again you may find that you need a new feature or an adaptation of an existing feature to perform some function required by your application. A few customizations might include:

- Standard Abbreviations Update (and Inquiry)
- Custom Accumulators
- Custom Error Message Update and Handler
- Automatic Input Process Line Numbering
- User-Defined Inquiry Screens (yes, it is possible!)
- Generalized Prompt Box
- Purge Definition
- Mailing Label Definition
- Enhanced Security

Though you should never change the standard SB+ tools, sometimes it can be very useful (and educational) to create a few new tools of your own, such as these shown here. SB+ provides a very strong infrastructure for doing things like this, given you have a comprehensive understanding of the common map and the way that processes work together.

Like SB+, the best custom tools you will create will be those that are parameter driven. Therefore, you should plan the information that your tool will need, then create an SB+ application to put the information in a file somewhere. Don't try to use the SB+ data files (such as *XXX*DEFN or *XXX*CONTROL) as there is plenty of information in those files already.

Once you have your parameters stored in a file, the next step is to create a "processor" which will take the parameters and do something intelligent with the information. Most processors will be written as BASIC subroutines, though paragraphs can certainly be a strong option as well. Plan your processor so that the ID of the parameter record to be processed is passed in the **common variable** @PARAM, noting that this is the standard way of passing information into processes. If multiple parameters are required, plan for all of them to come into your processor as a comma delimited list in @PARAM. The processor can then parse what it needs from the string.

To invoke your process, simply invoke your processor normally, passing the parameters required. For example, for a custom processor call PROMPT.BOX, a person could invoke a prompt box called TEST with this paragraph statement:

```
EXEC "PROMPT.BOX,TEST"
```

Or, to assign the next sequential number from a custom accumulator called ORDERS:

```
@KEY = P('GET.NEW.KEY,ORDERS')
```

Be sure to invest quality testing time on your custom tools. Only when the tool has been tested in a variety of environments should it be ready to be used in your application development. Remember, if your tool is flawed, everything you will create with your tool will also be flawed.

Introduction to GUI

Within the last several years, few things have generated more attention than tools for adding a graphical user interface (or *GUI*) to Multivalue applications. Look through the industry periodicals -- it seems as if all vendors in this industry either have, or are working on, some sort of GUI-enabling (aka *GUItization*) product for helping developers put a fresh face on character-based legacy applications.

With the worldwide domination of Microsoft Windows, we developers must realize that those who use the products we create will most likely be more comfortable with GUI applications than with their character-based alternatives. There are a number of reasons why this is true:

- GUI apps are *visual* -- Instead of having to enter a specific value in a prompt, the user can instead click a button with the mouse for the same end result. By showing all of the options in check boxes (or similar objects) on the screen, the user is supposedly more protected from the potential of entry error.
- GUI apps are *common* -- most computers in use today are of the PC genre, and as a result use some form of GUI operating environment like the Mac OS or Windows. This commonality helps to remove the stigma of complexity often associated with character-based applications.
- GUI apps are *marketable* -- If you're in the business of selling software, it's important to have a product with intrinsic appeal above and beyond the basic features. Though some argue that GUI introduces no significant benefit over character-based applications, one cannot discount that fact that GUI sells.

GUItization products typically fall into one of two categories:

- *Screen scrapers* are programs which "read" the prompts on a screen and convert them character for character to achieve the "GUI" look and feel. These programs typically achieve only nominal success in converting a legacy application to GUI, as the underlying logic of the program usually remains unchanged. Therefore, while the screen "looks" GUI, it doesn't "feel" GUI because the underlying logic has not been written for the GUI (event processing) paradigm.
- *APIs* (application programming interfaces) are common routines which perform the presentation and navigation for a GUI application. These types of products can achieve an excellent GUI look and feel. Unfortunately, however, many of these products have no significant infrastructure on which to build an application, and therefore a significant amount of work is required to create the GUI application. And when you're done, you have a GUI application only -- generally these tools provide no character-based alternative.

Then there's SB+. Unlike screen scrapers, SB+ doesn't attempt to "read" the prompts on the screen -- it takes control of them. Unlike traditonal APIs, SB+ starts with the most robust tool set in the industry, and therefore creating even a complex application
is easy. (In fact, using SB+'s *auto-GUI* feature, you'll find that the conversion from character to GUI is easier than you've ever imagined.) And if that's not enough, SB+ will run on both GUI and character-based devices *simultaneously* with no additional programming on your part!

Using the GUI forms painter, existing character screens (both for entry and query) can easily be GUItized and new screens quickly built. Menus and periodic updates require no conversion -- if you're running GUI when the menu or update is called, it will automatically be converted. Reports, on the other hand, run only in a hybrid GUI/character mode.

Getting Started With GUI

There are two ways to achieve a GUI application with SB+:

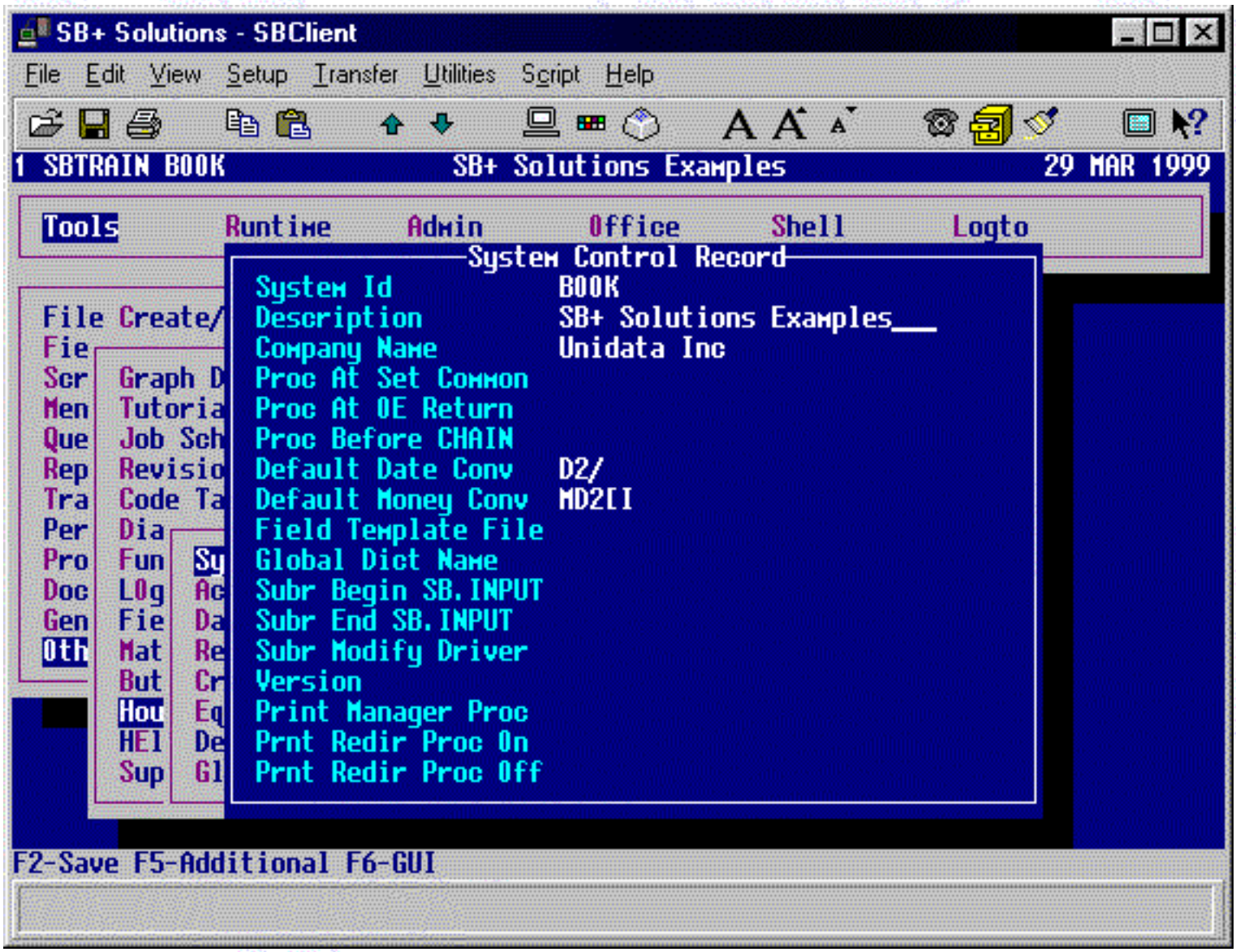
- Create a character-based application and then convert it screen-for-screen to GUI, or
- Create the application from scratch using the GUI tools.

For the ultimate in flexibility and portability, forget about creating the application from scratch using the GUI tools. By first creating a character-based application and then converting it to GUI, you end up with a product that works in both modes (and can be sold to two different markets!). You should also forget about creating the GUI first and then converting it to a character-based version later, unless of course you're one of those folks who love to do everything the hardest way possible.

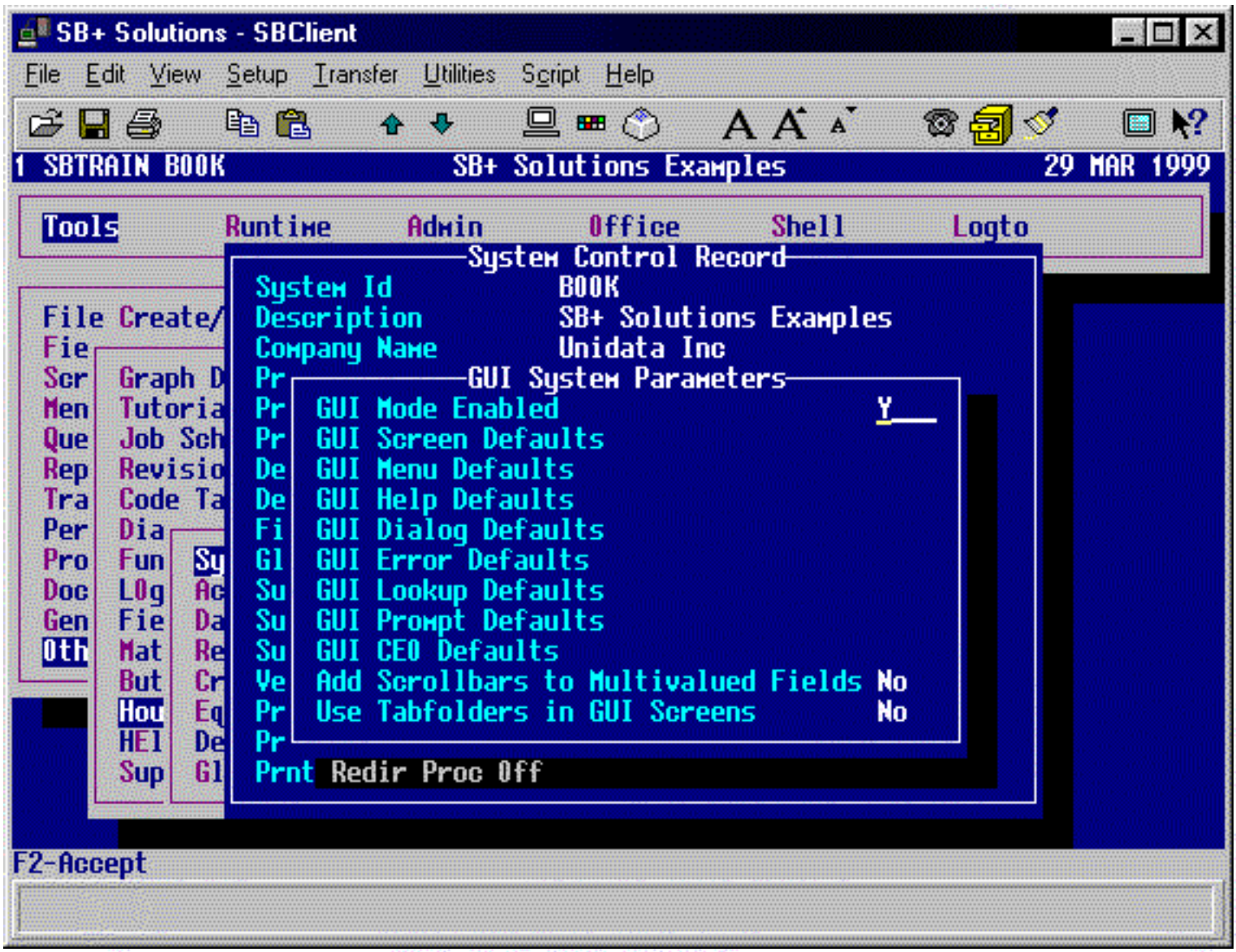
Generally speaking, you'll find that you can get more done in less time when developing your product in character mode. This mode requires less I/O between SBClient and the server, and therefore the tools appear faster and operate with less overhead. Once you have a working application, you can then go back and convert your screens to GUI mode.

"But", you say, "I have several hundred screens! I don't have time to have to go back and edit all of them!" Actually, you may not need much time, regardless of the number of screens in your application. In fact, unless you want your GUI screens to look significantly different than your character screens, SB+ will do most of the work for you automatically!

Once the character screens have been created and tested, you can enable GUI in the current system using the System Control Record Maintenance tool. This tool can be started either by selecting Tools, Other Tools/Utilities, House Keeping, then System Control Record or by entering /HK.CONTROL at any menu or input prompt. This tool appears as follows:



The System Control Record Maintenance tool is used for changing various parameters for the current **system**. Among these parameters are a collection of GUI settings which can be reviewed or changed by pressing the F6-GUI key. When this key is pressed, the following screen is displayed:



At this first prompt, enter "Y" to enable GUI mode for this system. Once this is enabled, character screens will automatically be converted to GUI at the time they are invoked (when using SBClient in GUI mode). The remainder of the prompts on this screen control the appearance of GUI screens, menus, etc., and are discussed in later sections of this chapter. To update the parameters and get into GUI mode, press F2 twice from this screen, exit SB+, login again, and select a GUI terminal emulation (discussed next).

(Important Note: If you want to change the default appearance of the GUI forms and objects, these changes should be made before any GUI development is started. See "**Controlling the GUI Look and Feel**" for more information.)

Using SBClient in GUI mode

For every terminal emulation supported by SBClient, there is a corresponding GUI terminal definition, where the name ends in ".GUI". This is how SB+ knows whether you want your application presented in GUI or character mode.

If you normally use the TU.WYSE50 terminal definition, use the TU.WYSE50.GUI terminal definition to enable a GUI presentation of the application. (The same is true for the TU.VT220, TU.ADDSVP, TU.ANSI, and TU.WYSE60 terminal definitions.) If the GUI flag is enabled for the system and you're using a GUI terminal definition, your character screens will automatically be converted as they are invoked. Note that this automatic conversion isn't infinitely sophisticated -- while it will

convert your menus and screens to look *more* like a Windows application, it will not change the basic format of the screen. Therefore, don't expect to see check boxes, drop-down lists, and other GUI items that could never appear on a character screen, because the conversion won't add these types of fields automatically.

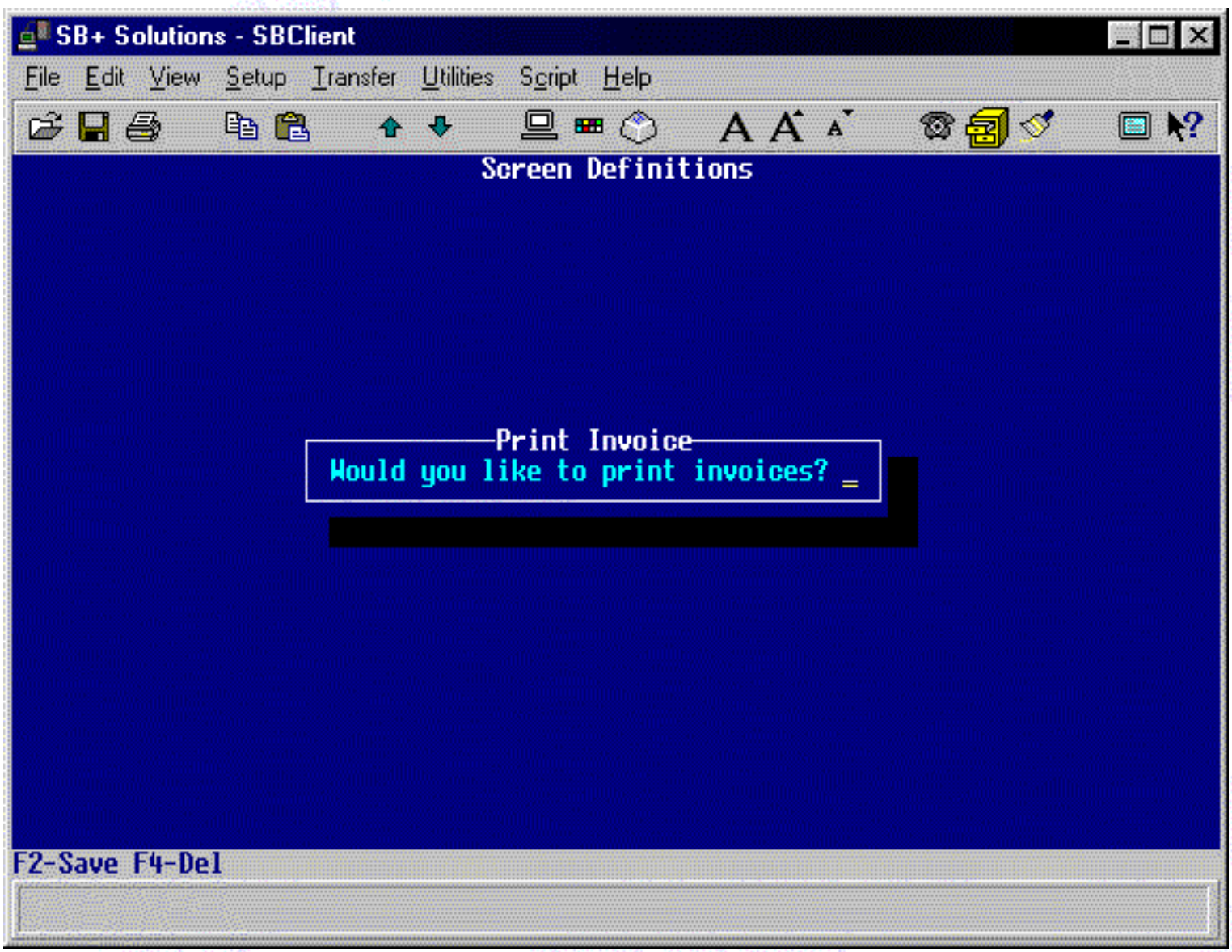
Having said this, you CAN do a little up-front preparation in your character mode application to control the automatic GUI conversion. See "**Designing a Character-Mode Application for GUI**" for more information about how this is possible.

The GUI Paradigm

In the character mode world, screens are comprised of input prompts at specific locations on a screen. While the same can be said for GUI screens, the GUI paradigm introduces a number of additional features (as well as additional complexity) that is not found in the character world.

On a character screen, every input prompt is *textual* -- that is, each prompt receives keyboard (text) input. As a result, if the screen needs to capture a yes/no setting, the user will enter the letter "Y" for Yes and the letter "N" for No (in English-speaking countries). In GUI, however, things are not necessarily so straight-forward. GUI prompts can be handled via a number of different *objects*, thereby giving you the developer more variety over the presentation of your prompts. As a result, a yes/no setting is not restricted to being simply a textual entry, as it is in character mode. Instead, such a prompt can be implemented using radio buttons, a check box, or perhaps even a drop-down combo box.

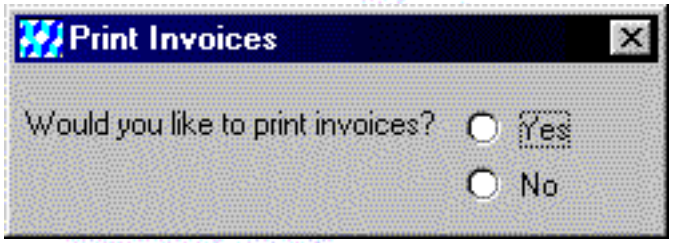
As a textual entry in character mode, the yes/no prompt might appear like this:



In GUI mode, however, the same prompt appears in a graphic window:



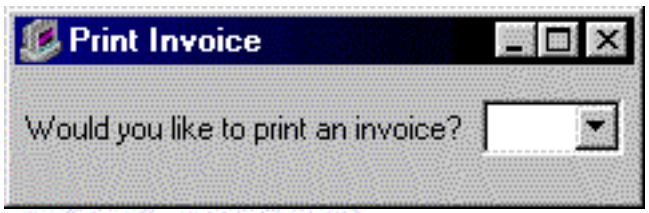
Or, the prompt can be implemented using Yes and No *radio buttons*, where only one of the buttons can be selected at a time.



Or, you may choose to use a *check box*, which gets its value from being either checked or unchecked.



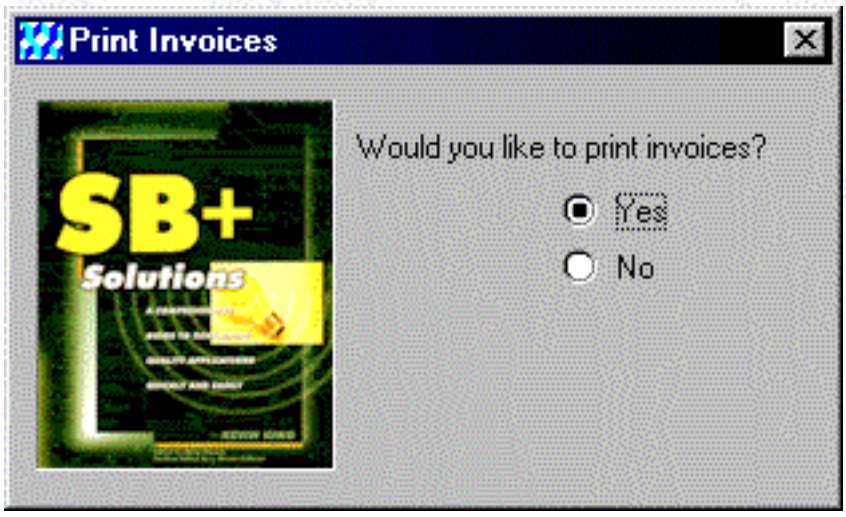
And finally, as a drop-down combo box, the prompt might appear like this:



(If we click on the arrow shown to the right of the prompt, a small window opens where we can select one of the valid values.)

There are a number of benefits to using these non-textual prompting objects. First, the prompts provide all of the information about the valid entries right on the screen. There's no need for the user to press F3 for intuitive help -- as one would typically do on a character screen -- to see the available options. Second, validation on these types of fields is significantly easier because the user doesn't actually type a value for the field. Instead, the value for the field is determined by the user's *action*, not their *entry*. Therefore, there's no need to worry about the right values being entered in the wrong case, or the wrong values being entered at all. With these GUI objects, incorrect values simply can't be entered. (Note that this does not, however, shield the user from making incorrect *selections*!)

In addition to the flexible prompting options provided by the GUI, you also have new options for for presentation of the screen itself. Rather than settle for a simple grey background with prompts, the GUI allows you to place images (pictures) in your windows, such as is shown following:



Or, if you're simply tired of the same-old same-old look, perhaps you'd like to try something a bit more drastic:



GUI gives you ultimate control over positioning, fonts, styles, and literally all aspects of the presentation of your screen. In other words, it gives you a whole new world of *options*. And when trying to put together that product that is *just so*, options are a developers best friend.

(In case you're thinking that these last examples are complex and take a lot of time, worry not -- both examples took only a matter of seconds to set up and paste into this page!)

GUI Objects Overview

Each GUI object has its own characteristics, abilities, and quirks, as follows:

(Click on the object name to see the properties for that object.)

Object	Looks like...	...and acts like...
Textclass		A textclass prompt is a character (textual) entry. Any number of characters can be entered into a text prompt and the values are stored in the record exactly as entered. There are no restrictions to the types of validation and intuitive help that can be used with a text prompt. Additionally, input and output conversions are used (as they would be on a character screen) when displaying and entering values into the text prompt. In short, these are the same as textual prompts on a character-mode screen; you can do anything you want with the **default**, **validation**, **intuitive help**, **conversion**, **input conversion**, etc. Textclass prompts can be either single or multivalued.
Toggle (aka "check box")		A toggle prompt is either selected or not selected. Prompts which use this object must be validated either with a table (using the E: validation code) or with a validation list (V: validation code). When **validating from a table**, the first option in the table represents the selected value and the second option in the table represents the unselected value. Therefore, if your table includes "Y" for Yes and "N" for No (in that order), checking the box will put a "Y" in the record, and removing the check mark will put "N" in the record in the appropriate attribute. All other options in the table are ignored. It's important to recognize that the table validation must be expressed using the E:TABLE('*name*')>" validation code in order to use the toggle object for a prompt. If the prompt calls a process for validation, and that process uses a table, you won't be able to use a GUI toggle on that prompt. When validating against a list of valid values with the V: validation code, the first option represents the selected value, and the second option represents the unselected value. All other options in the validation list are ignored. Intuitive help and **F1 Help** are suppressed on toggle prompts, so F1 and F3 have no function when the cursor is on a toggle object. Note that toggle prompts can only be used for single valued fields; there is no comparable feature for multivalued fields.

Radio Buttons		Radio buttons are a set of mutually exclusive options. Only one of the buttons in a set can be selected at a time. Prompts using the radio button object must be validated either with a table (E: validation code) or with a validation list (V: validation code). Though you can validate radio buttons using either a validation list or a table, it's best to use a table. SB+ can figure out the labels for each button when a table is used, and as the table changes, the GUI screen changes to match. When validating against a list of valid values (with the V: validation code), you must manually edit the labels when new options are added to the list. When a user selects one of the radio buttons, the corresponding table entry or valid value is inserted into the record at the appropriate position. Note that radio buttons can only be used for single valued fields; there is no comparable feature for multivalued fields.
Combo Box		A combo box, also known as a *drop down list*, is something like a textclass object, in that the values shown are textual. However, unlike a textclass object, the combo box has a list of valid values which are exposed when the user clicks on the arrow shown to the right of the text display area. An *editable combo box* takes this paradigm a bit further, and allows the user to add new items to the list. A combo box must be validated against a list of valid values (using the V: validation code), or against a file (using the F: validation code) or against a table (using the E: validation code). When validating against the list of valid values or against a file, you have little control over what is displayed when you click on the arrow to show the list of acceptable options. However, when validating against a table, the conversion as defined for the field controls what is displayed in the drop down list. If you are simply validating against the table and there is no conversion on the field, the drop down list will contain codes only. If, however, your conversion is something like: (@VALUE : ' ' : TABLE('*tablename*')) ...the code and description will be shown for the items in the drop down list.

Image		An image is simply a picture. This is useful for showing a picture of the product a customer is ordering, or perhaps a photograph of an employee in a H/R application. Images are display only objects. Therefore, validation, default, and the other input field prompts are not applicable to image prompts. Image prompts can be either fixed or variable. In the SB+ Form Painter, you can nominate the picture to be shown for a given field and that picture will be shown each time the form is displayed. This is useful for displaying a corporate logo or other fixed information on the screen. Variable images, however, are much more powerful. With a variable image, the image will change based on information in the record. For example, let's say you're working on an NT server where the E: drive is a shared drive holding photographs of employees (in the E:\pix directory for example). Each employee has a unique identification number which also -- incidentally -- happens to be the name of their photograph in this directory. For the sake of example, let's say your employee ID is 2126. Therefore, in your employee record you have E:\pix\2126.gif stored in attribute 7. By placing a field definition (which points to attribute 7) on a screen as an image prompt, when someone calls up your employee record, the corresponding image will be displayed. As of this writing, SB+ supports the display of graphic images using the following formats (and corresponding file extensions): BMP - Bitmap GIF - Graphic Interchange Format JPG - JPEG

Designing a Character-Mode Application for GUI

While a GUI can "just happen", as in using SB+'s **Auto-GUI** abilities, the end result is pretty limited. While it is true that the business rules and logic that are under the character mode screens remain intact under the GUI layer, Auto-GUI doesn't fully utilize the spectrum of possibilities that GUI provides. In order to use a GUI effectively -- that is, to make the interface an integral part of the solution -- you should consider the impact the GUI will have on your character applications during the initial design.

"But", you say, "I already have a character mode application! Do I need to start over to use the GUI effectively?"

Fortunately, no -- you do not have to start over to use the GUI effectively. However, before doing anything from this point forward, stop and consider what you're trying to accomplish. As stated in **Chapter 2**, if you don't **plan** ahead, how will you know when you've achieved the end result? Or stated differently, without a plan, how will you know what the end result is supposed to be? If all you're trying to do is put a pretty face on an old character-mode application, a GUI will certainly help you towards the end result, though you may not achieve as powerful of an application as possible. Or, to quote an old proverb, "You can put new lipstick on a old cow, but it's still an old cow".

The GUI introduces a number of additional features and abilities not present in character mode. Therefore, because the palette of options is greater, the potential for problems is greater. And if the equation isn't complex enough, consider for a moment that you have the ability with SB+ to create both a character mode and GUI application *simultaneously*!

Fortunately, there are a few things you can do during the design (or conversion) of your character-mode application which will help you achieve a truly functional graphical interface without having to rework the overall concept.

- Fight the urge to get something done quickly.

  I see this time and time again. For one reason or another -- typically overzealous marketing folks -- a GUI ASAP mandate is issued. Typically, this type of mandate occurs when your company's competitor -- who has a less functional product -- releases their GUI version first. For some reason there is a perception that GUI, simply by its nature, adds functionality to a product. Whether this is true or not is inconsequential; the fact that it does help sell software is of great consequence.

  (Despite the hype, getting to the market first in the software business is not an enviable position to hold. How many people rush out to buy version 1.0 of any product?)

  Instead of trying to be first, dedicate yourself to doing the job right. Plan ahead. Systematically implement your plan. DO NOT HACK UP A PERFECTLY GOOD APPLICATION TRYING TO MAKE IT PRETTY.
- Envision the character-mode application in GUI.

  When creating your character screens, try to paint a mental picture of how the screen might appear in GUI. How are prompts arranged? Does the order make sense on a GUI screen? Or can several prompts be grouped together to help the user make some mental connection between them?

  What about color? Contrast? Fonts? Should one area of the screen receive more attention or emphasis?

  On each prompt, would the user interface be improved by the use of one of the various types of **objects**, instead of simple text entry fields? If only two options are available for a given prompt, a check box is a perfect fit. If 3-10 options are available for a given prompt (and you have the screen real estate available) and the options are reasonably consistent, radio buttons may be a good choice. If there are numerous options available at a prompt, a combo box is probably the best option. For everything else, the text box is probably the best choice.

The goal here is *consistency*. As you're envisioning how you might implement the GUI, consider ways to promote consistency between screens. In other words, if a Yes/No prompt is implemented as a check box in one area of the software, implement all Yes/No prompts as check boxes throughout the application. If a list of type codes is displayed in a combo box in one area of the software, use a combo box for this prompt in all areas of the software, despite areas where radio buttons (or some other object) might work as well.

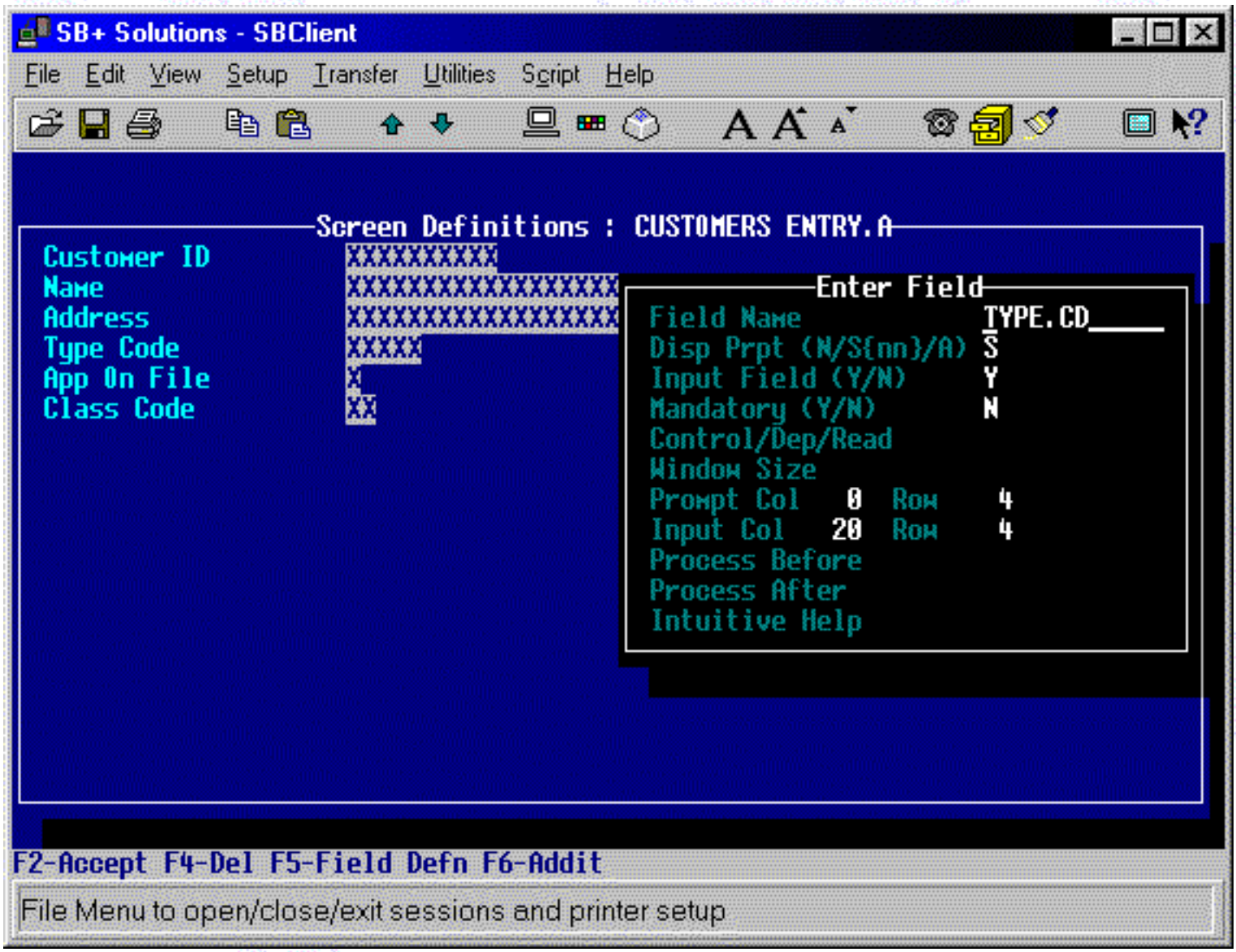
- Leave a little space where you might need it.

  Once you've determined where you need radio buttons or even perhaps an image, consider leaving an extra line or two between that prompt and the following one. This will allow SB+ to place the object without overwriting any subsequent prompts. Of course, don't make the character screen look stupid so it'll look great in GUI -- it's better to rearrange the prompts in GUI later if such a risk exists.)
- Consider the user.

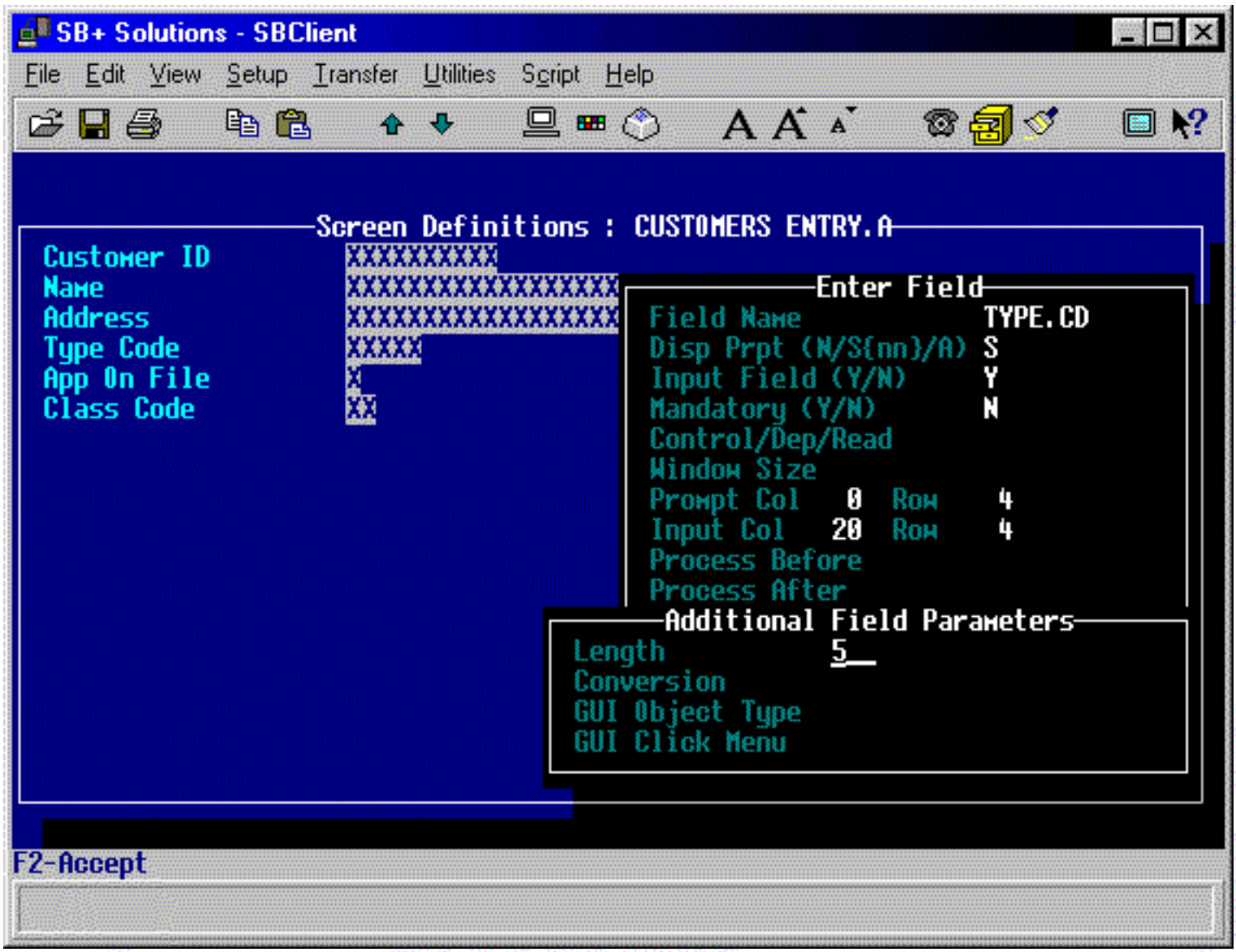
  Though typically ignored, this is the most important user interface aspect of all. Somewhere out there is a person who will spend a good portion of their life in front of your program(s). Consider your application as if you were that person. Can you use the GUI to make the application easier to learn and use? Or are you simply using the GUI as a channel to a more salable application? While there's nothing wrong with the latter, functionality should always come before form. Consider the operator who will live with your work before the executive who will purchase it, and you'll be well on the way to using the GUI effectively.

With these considerations in mind, you can do a lot of the up-front GUI preparation while working in character mode. Once you know what type of object is to be used for a specific prompt, you can define each prompt to use a specific GUI object. Then, when SB+ converts your screen to GUI, you only have to modify the field widths and prompt locations.

When adding a field to a screen using F5, the following window appears:



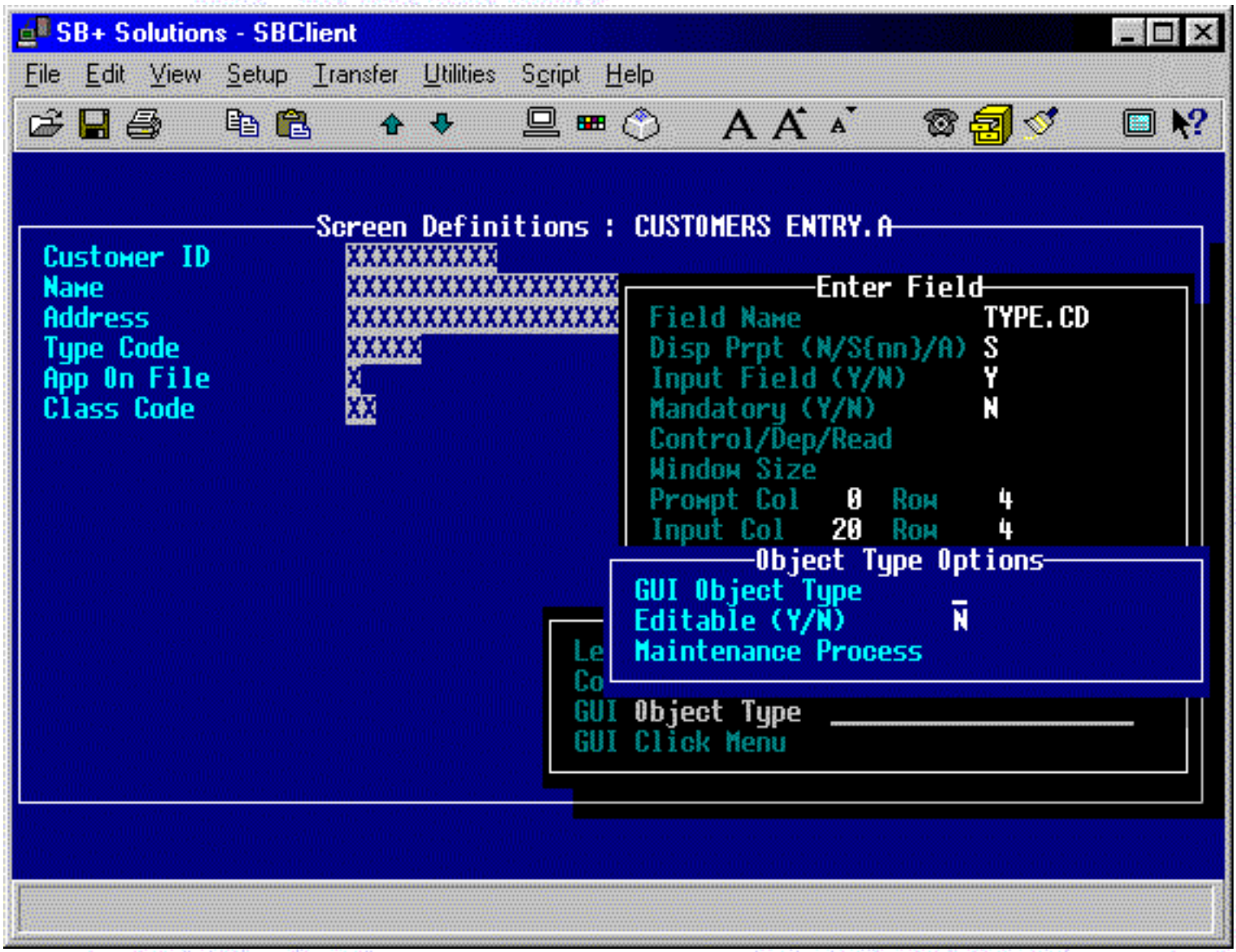
From this window, you can press F6 for additional parameters, at which point the following screen appears:



Note the prompt "GUI Object Type" on this screen. This is how you tell SB+ which object is to be used for this prompt. Valid options for this prompt include:

C	Combo Box (not editable)
CE	Combo Box (editable)
I	Image
R	Radio Buttons
T	Toggle (Check Box)
X	Text

If you move the cursor to this prompt and press F3, the following window appears:



At this first prompt, enter one of the above codes or press F3 to select the appropriate code from the list. When SB+ generates the default GUI screen, the object you selected will be used in place of the standard text box. (Note that the width will probably need to be adjusted, so don't expect perfection right away...)

If you select the combo box object, and you're using either the F:*file* or V:*file item* validation code on the prompt, the cursor moves to the Editable (Y/N) prompt. At this prompt, if you want the combo box option to be editable, enter "Y". Otherwise, if you want the options in the combo box to be fixed, enter "N". When a combo box is editable, the user is allowed to select any of the options in the combo box, as well as enter new ones. If the combo box is not editable, the user can only select from the options shown.

If you've made your combo box editable, the cursor will proceed to the Maintenance Process slot. At this slot, you may enter the name of a process that is used to update the file or list (as shown for validation) when the user enters a new value. For example, if we create a screen with a customer type code, and that type code is validated against a file, we can allow the user to add new records to our type code file by adding a paragraph to this slot which adds the record, as in:

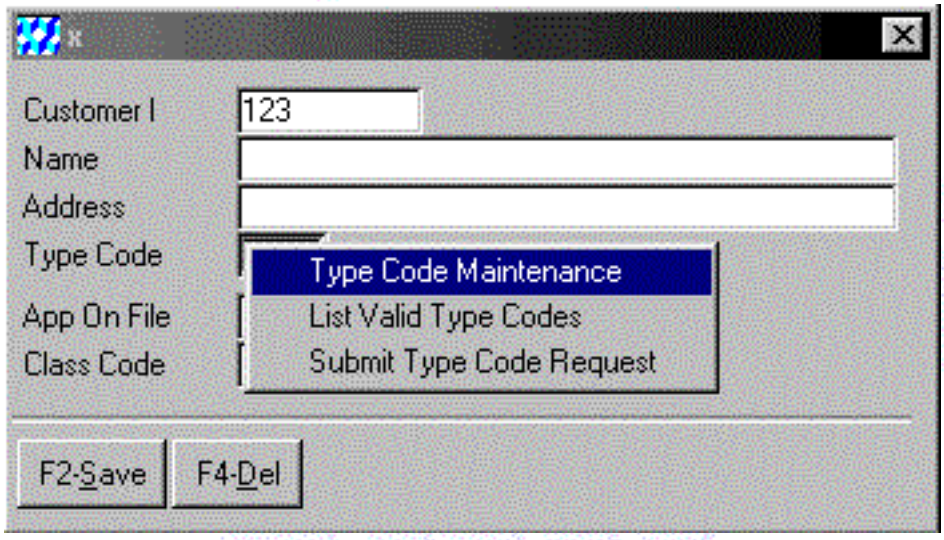
```
WRITE 'New Type Code' ON 'TYPE.CDS',@VALUE
```

In this example, we're assuming that the description of the type code is in the first attribute. We don't know anything else about the type code at this point, so this is all we need to write. We could, of course, call a screen from this slot, and have that screen

write the record to the TYPE.CDS file as well -- the ultimate complexity of this prompt is completely up to you and your application.

Note that these last two prompts apply only to combo box objects. You cannot make any other object editable (with the exception of a text box, which is editable by default).

Back on the additional parameters screen, we have one other GUI prompt available. At the "GUI Click Menu" prompt, enter the name of a menu that should appear when the user presses the right mouse mouse button (a.k.a. the "right-click") while positioning the mouse pointer over the field. Any menu name can be included here, allowing the user to move fluidly throughout the software without having to use the slash (as they would in character mode). To the user, this may appear something like the following (when right-clicking on the Type Code field):



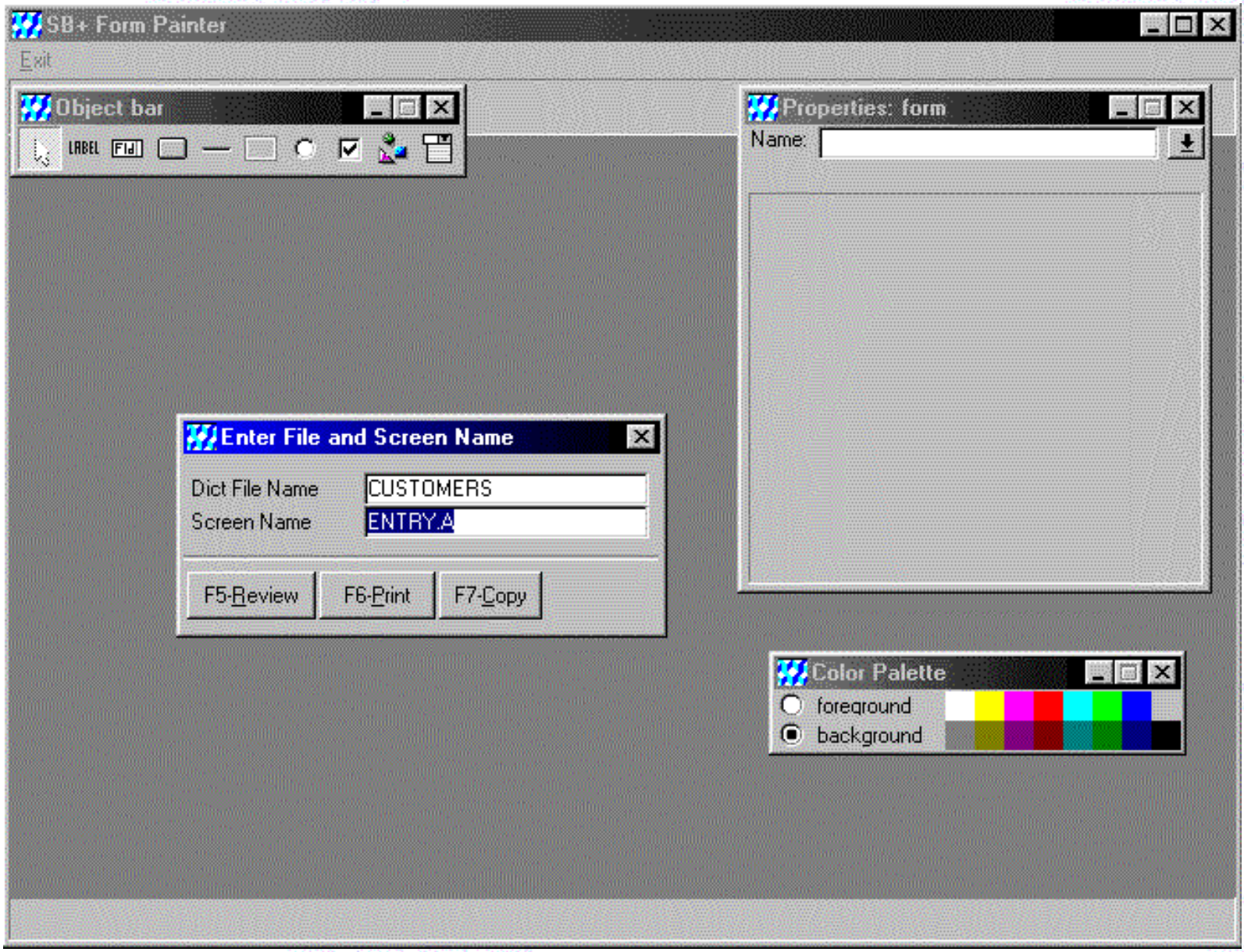
It's no secret that on a typical connection, the SB+ tools themselves are faster in character mode than in GUI. Therefore, if you're looking to accomplish the most in the least amount of time, do as much GUI preparation as you can while still in character mode. Then, once you've gone to GUI, all you'll need to do is tweak your prompt positions, widths, and add some really cool **images**...

Using the GUI Form Painter

Whether creating new GUI forms or making minor adjustments to a form generated by SB+, the SB+ form painter is the tool to use. When running in GUI mode, the form painter performs the same function as the Screen Definition tool, except that everything is GUI -- not character mode.

The GUI form painter can be started in one of two ways. First, you can start the GUI form painter from the Windows 95/98 Start button. Assuming you know the menu structure of your machine, you can click on Start | Programs, and then select the appropriate folder for SBClient. In the SBClient startup folder is an icon called "Form Painter". Once selected, the form painter will be started.

If you're in a GUI session, you can simply start the Screen Definition tool to start the form painter. This tool can be started by selecting Tools, Screen Definitions from the menu, or by entering /SD at any input prompt or menu. When the GUI form painter is started, the following screen is displayed:



(Note that the position of the individual windows will vary.)

Note the active window (with the blue title bar). In this window, enter the name of the file and screen to be created/edited. This is exactly the same as when starting the Screen Definitions tool in character mode. The file name entered here must exist. However, the screen need not previously

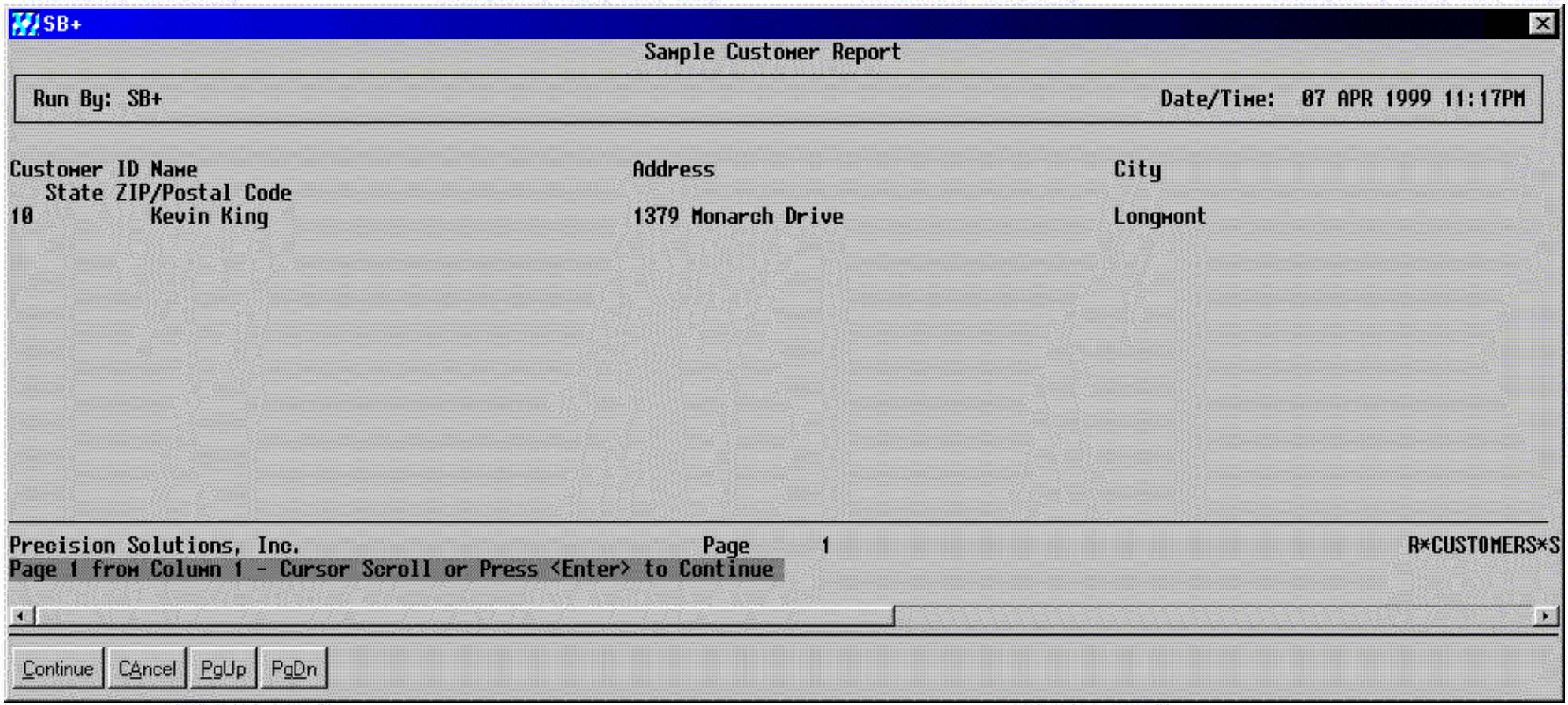
exist. (Note, though, that it is generally easier to convert a character screen to GUI rather than to create a GUI screen from scratch.)

The other windows are used for designing the screen. Near the top we have the object bar, which contains buttons allowing us to select any of the various SB+ form **objects** (radio buttons, toggles, combo boxes, images, etc.) On the right is the properties box. This box contains information about each of the individual objects. (It's blank right now because we haven't started editing any objects.) Finally, note the color palette below the properties window. This window is used for changing the foreground (text) and background colors of a given object on the form.

Converting a Character Screen to a GUI Form
Creating a New GUI Form
Moving/Resizing Objects on the GUI Form
Using the Color Palette
Running the GUI Form
Controlling the GUI Look and Feel

GUI and Reports

When running in GUI mode, it can be useful to be able to see reports just as you might when running in character mode. To support this functionality, SB + has a special "report GUI" mode which formats reports under the GUI presentation layer. Such a report might appear as follows:



The report is essentially the same as you might see in character mode, with the exception of the GUI control elements. Note the scroll bar for scrolling right and left, and the buttons for moving forward, exiting, or paging through the report.

Reports can also be designed in the GUI mode. However, there is absolutely no benefit to doing this. As of version 4.x, reports are always output in a textual form, and therefore receive no benefit from the GUI tools. In short, there is no GUI form painter for reports, and therefore there is no reason to design reports in this mode. (Actually, and I regret having to admit this, but I have found it literally impossible to create a report in the GUI mode under SB + 4.x. Every attempt I have made to use the GUI interface for designing a report has resulted in a need to Ctrl-Alt-Del to abort SBClient, and therefore my recommendation.)

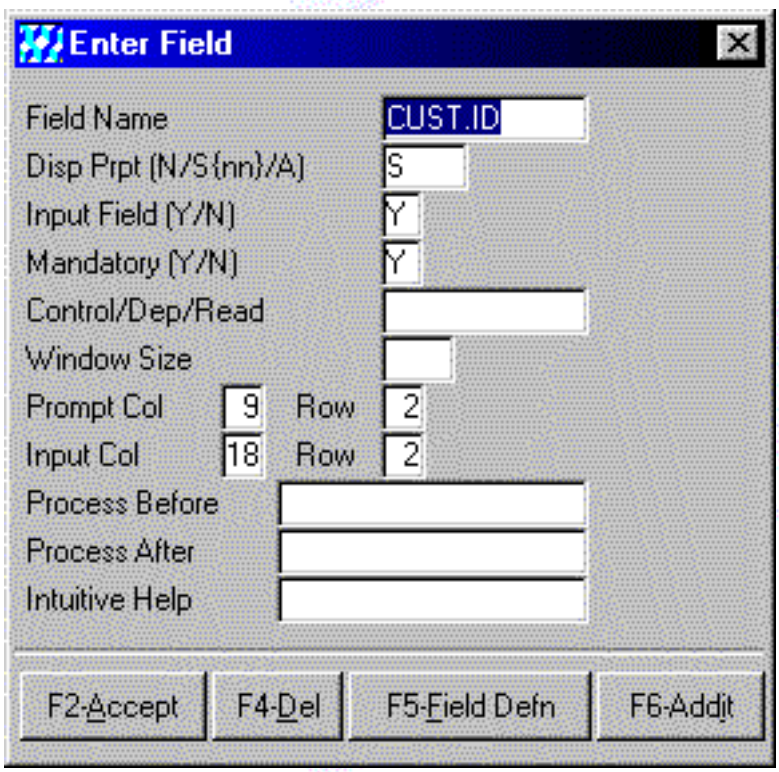
GUI Menus

There is nothing special to be done to have SB+ use a menu in GUI. All menus are automatically converted to a GUI format when using SBClient in GUI mode.

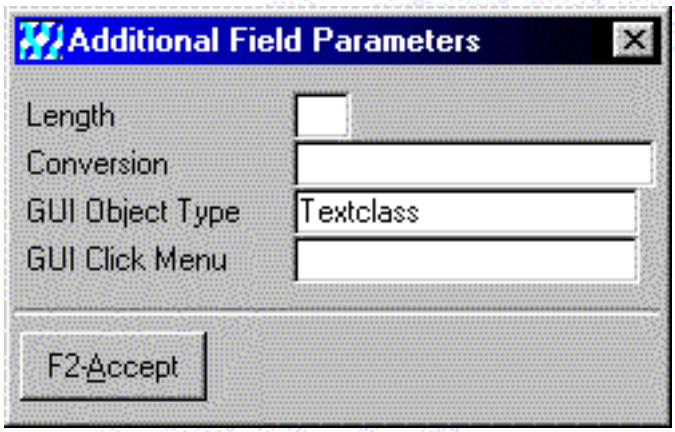
As in character mode, menus in GUI can be used either to start programs (**startup menus**) or to play a supporting role in them (**process menus**). Additionally, however, menus can be associated to nearly every object on a form, excluding rectangles and separator lines.

Any object on a form which has an underlying field definition can have a "GUI Click Menu" associated with it. This menu appears when the user clicks the right mouse button while hovering over the object, and provides additional functionality for the object. Each object can have its own GUI click menu, though notably one menu can be associated to a number of different objects.

To configure a GUI click menu for a given prompt, enter the GUI form painter and click on the field which is to have the menu. Then press F5. The following familiar window appears:

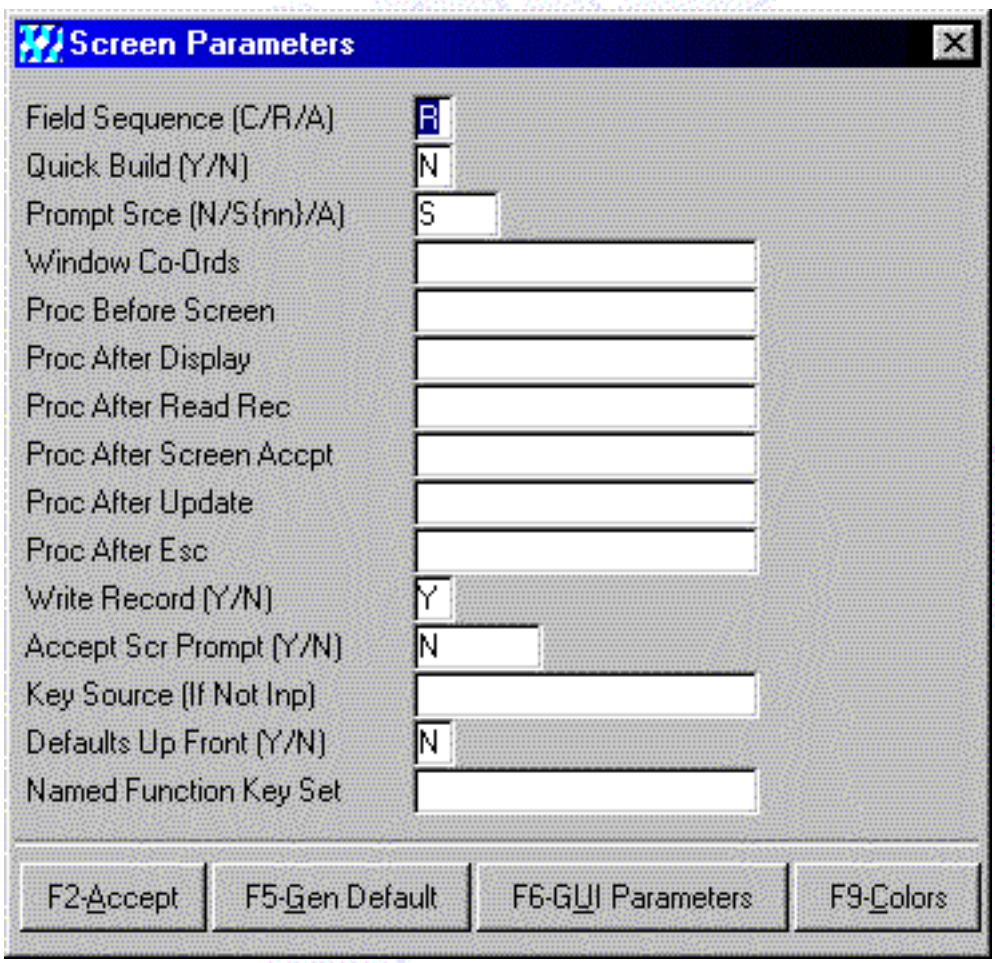


On this screen, either press F6 or click on the F6-Addit button. This will invoke the following screen:

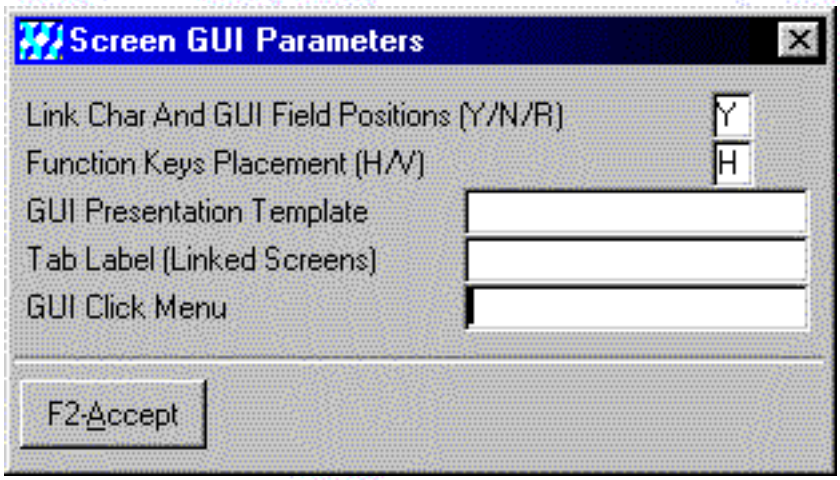


At the "GUI Click Menu" prompt, simply enter the name of a menu to be invoked when the user presses the right mouse button while hovering over the object. Once the menu name has been entered, press F2 a couple of times to return back to the GUI form painter, and the menu is installed!

You may also want to configure a GUI click menu to appear when someone clicks on an open area of the form. To define this, press F6 in your screen definition to invoke the F6-Params screen:



Next, either press F6, or click on the button marked F6-GUI Parameters. This will invoke the following screen:



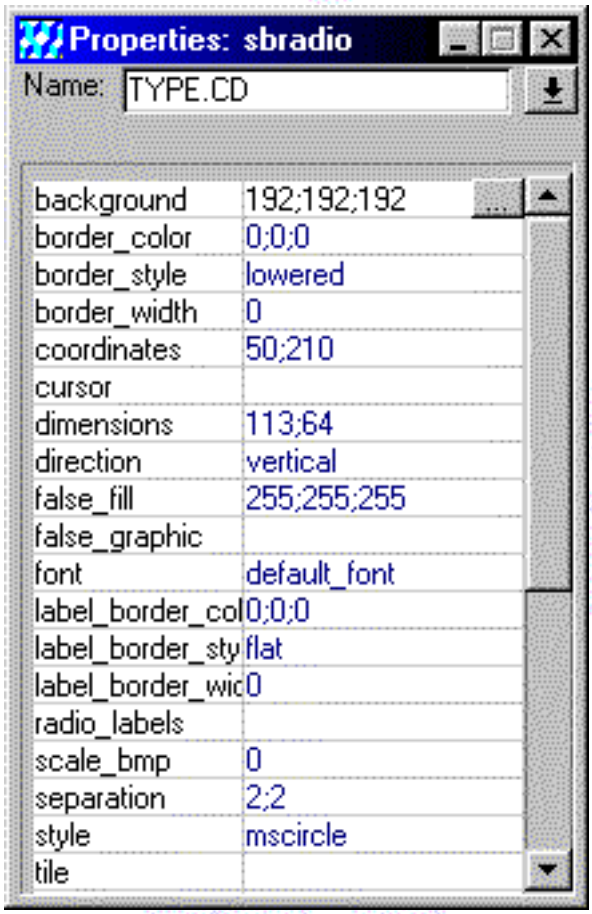
At the "GUI Click Menu" prompt, simply enter the name of a menu to be invoked when the user presses the right mouse button while hovering over an open area of the form. Once the menu name has been entered, press F2 a couple of times to return back to the GUI form painter, and the menu is installed!

GUI Object Properties

In the GUI, every object has *properties* which define the location and appearance of the object. Some properties control color, others control the font that is used. Some properties control the appearance of a given prompt, such as whether the prompt appears raised up from the form or lowered into the form. Regardless of the object, these properties are values you can change to modify the appearance of your GUI forms.

Different types of objects have different properties, though there are similarities between them. For example, toggles and textclass objects have significantly different properties -- yet, the "font" and "foreground" properties control the font and text color used for both of them. By understanding the different properties, you can exercise great control over the appearance of your GUI forms.

To change the properties for a given object, simply click on the object to be modified. The properties window will then show the properties for that object. (If the properties window is not displayed, click on the Window option in the main menu and make sure the Properties option has a check mark beside it. If Properties is checked, and you still can't see the Properties window, click on Options on the main menu and select Windows on Top. Then restart the form painter and the Properties window should be displayed.) This window appears as follows:



The following sections detail the different properties for each object type.

Form Properties
Label Properties
Textclass Properties

Command Button Properties
Separator Properties
Rectangle Properties
Radio Button Properties
Toggle Properties
Image Properties
Combo Box Properties

LOCAL - Declare Local Variable(s)

This statement declares local variables for the paragraph. This is how a developer creates new variables for the paragraph "out-of-the-air". For example, the following statement will create two local variables for a paragraph:

```
LOCAL STATEMENT.DATE,CUSTOMER.ID
```

Once these local variables have been declared, they can be used in the paragraph. Incidentally, all local variables are considered to be type-less fields. In other words, you can use a local variable to store an alphabetic, numeric, or alphanumeric value, or the variable may be used to hold a dynamic array, complete with several attributes, values, and subvalues.

When the paragraph exits, the local variables are cleared and henceforth cease to exist -- at least until the process is called again. When this happens, the variables are created new and any values the variables may have previously had will be lost.

Also, two paragraphs can use the same local variable names without conflict -- local variables have no scope outside of the current paragraph, unless they are passed as parameters from process to process. Therefore, unless you specifically move a local variable into a **common variable**, you will not be able to see the local variable using the /EE (Evaluate Expression) process.

Adding F1 Help to the Screen Definition

One of the most powerful features of any software package is concise, effective help. This can be added to your application in the form of F1 help to display help messages, as well as F3 **intuitive help** to assist the user in making selections.

With SB+, you can define up to five levels of help messages to be displayed when the F1 key is pressed. The first four are specific to a single prompt on the screen, and are invoked when F1 is repetitively pressed. The fifth level of help, called "General Help" is one help level that applies to everywhere F1 can be pressed.

The best time to add F1 help is after the screen has been created, but before the extra processing logic (i.e. validation, processing before, processing after, etc.) has been added. This way, the screen allows cursor movement to each of the fields to be documented (while the screen is running) without the potential of interfering skip logic, defaults that prevent cursor advancement, or other such things that can slow down the process of documenting the screen. These things can always be added after the F1 help has been added.

F1 help for a prompt on a screen is stored in the field definition for that prompt. There are, however, two ways to enter this information. The following sections describe the differences between the two:

Option 1: Using the Field Definitions tool

The last prompt on the **Field Definitions** screen is called "Help Reminder (Y/N)". If you enter "Y" at this prompt, SB+ will prompt you for a help message to appear at the bottom of the screen. Once this message has been entered, the following prompt will appear:

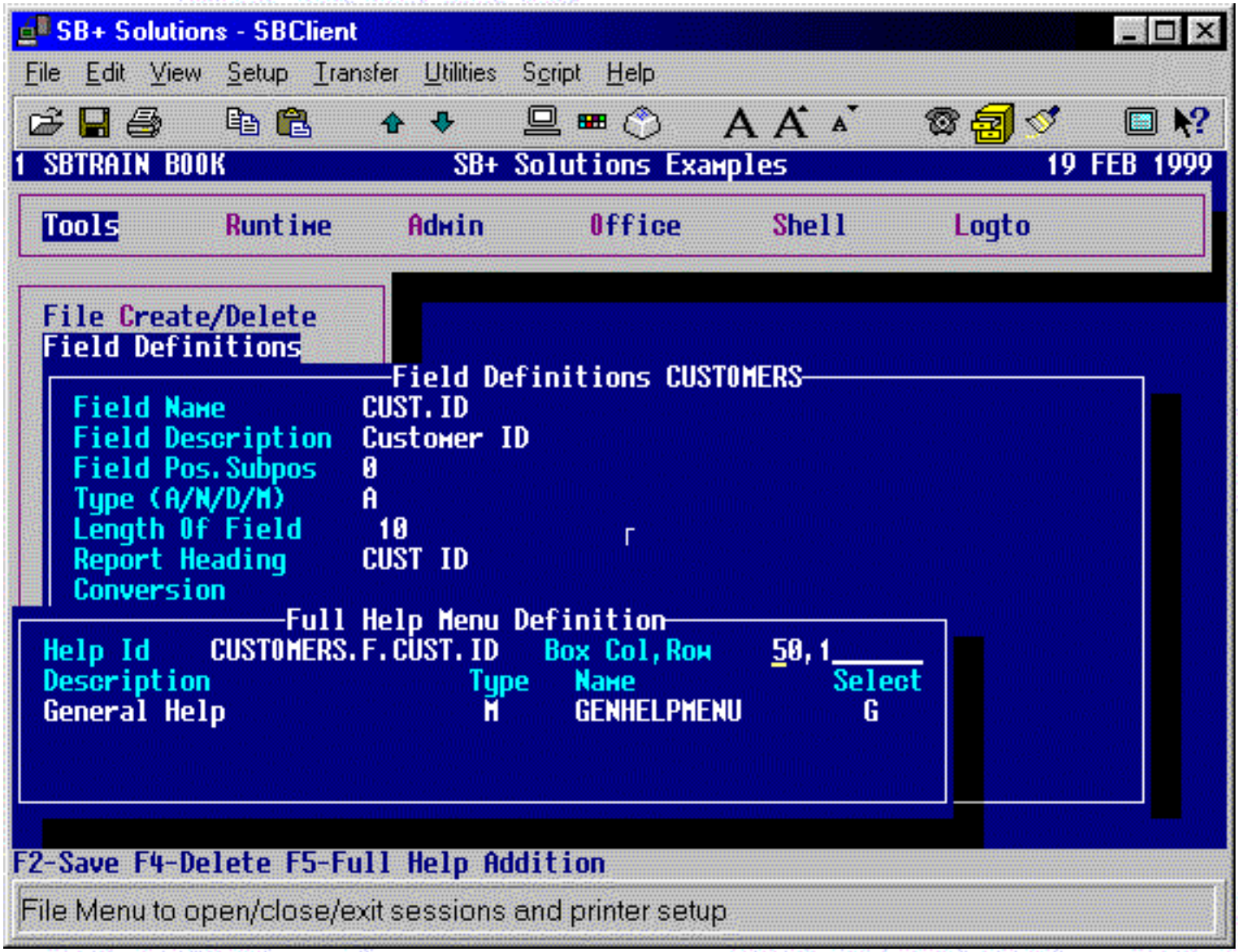
Create Second Level Help (Y/N)

If you enter "Y" at this prompt, SB+ will ask you to draw the box that the second level help is to fit in. As in the Screen Definitions tool, the box is drawn by moving the cursor to the position where you want the upper left corner of the box followed by <cr>, followed by moving the cursor to the lower right corner of the box followed by <cr>. Once the box size has been defined you may then enter the text to appear inside of the box when the second level help is to be displayed.

There is no limit to the amount of text that can appear in a second level help window. There is one rule of thumb to keep in mind, however. If the second level help is smaller than your window size, SB+ will display the help message and then return the cursor to the prompt so that the user can read the message and enter the value at the same time. If the message is longer than what will fit inside of one window, however, the cursor will remain inside of the window until the user presses <cr> or Escape, at which time the cursor will be returned to the prompt. With this option, the message is not displayed while the user is entering a value into the prompt.

After the second level help message has been entered, SB+ will clear the screen and prompt you to enter the third level help message. This is a full-screen help message, and can be used to describe the prompt in any level of detail you can imagine. If you don't want a third level help message, simply press Escape when the screen is displayed. Unlike the second level help, SB+ does not give you an option before prompting for the third level help, so this is the only way you can tell SB+ that this level of help is not necessary.

If the third level of help is necessary, and you press F2 to save the message(s) you have entered, SB+ will then prompt you for the fourth level of help, which is by definition a *help menu*. The following screen will be displayed:



On this screen, define the options you want to be presented in a menu when the user gets to the fourth level help menu. This may include processes to help the user select the right option (similar to a F3 process), or processes to display any amount of additional help.

The name of the menu is assigned by SB+, and while you can change it, if you do so SB+ will not be able to find the menu. Therefore, simply press <cr> on this prompt.

The Box Row,Col prompt defines the upper left corner row and column where the menu is anchored. (The width and depth of the menu will be calculated based on the number of options and the width of the longest option on the menu.) There are really no restrictions to where the menu is anchored, though
your established standards should apply.

In the Description prompt, enter the descriptions of the different menu options. Like a normal SB+ menu, these descriptions will be what the user sees, so be as descriptive as possible.

In the Type prompt, enter "M" or "P" to invoke a menu or process, respectively. See **Chapter 7** for more information about the other options available here, or simply press F3 for a list of available options.

In the Name prompt, enter the menu or process name to be invoked when this option is selected from the menu. Though the menu or process doesn't need to exist when the menu is created, be sure to have it completed before the software is released for production!

In the Select prompt, enter the single letter that should be highlighted on this menu option. Naturally, the select letter must be a letter in the description, and the same select letter cannot be used for more than one option.

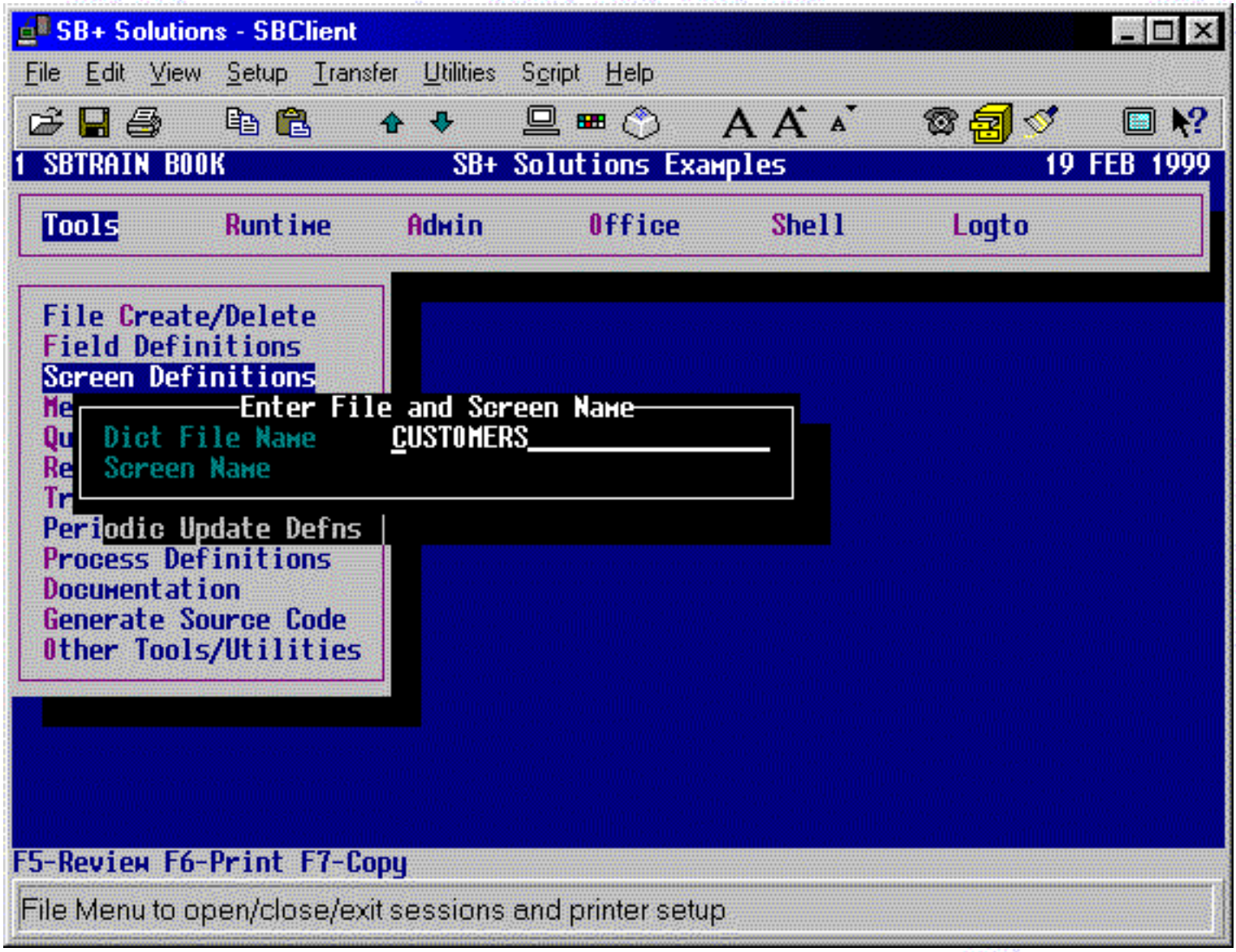
Once the menu has been constructed, press F2 to save it. Once saved, the menu will become your fourth level of F1 help for that particular prompt.

Again, you may not need this much help for a single prompt. Yet, it's nice to have the option when you need it!

Creating a Screen Definition

Screens are created using the Screen Definitions tool, which is started by selecting Screen Definitions from the SB+ Tools menu. The tool may also be called from any input prompt or menu simply by entering /SD.

When the Screen Definitions tool is started, the following screen is displayed:



At the "Dict File Name" prompt, enter the name of the file where records entered into this screen will be read from and written to. Note that the actual screen definition record(s) will be stored in the dictionary of the file you nominate at this prompt. Therefore, you cannot name a screen with the same name you've given a field definition. Also, though the prompt says "Dict File Name", you can simply enter the name of the file without the preceding "DICT".

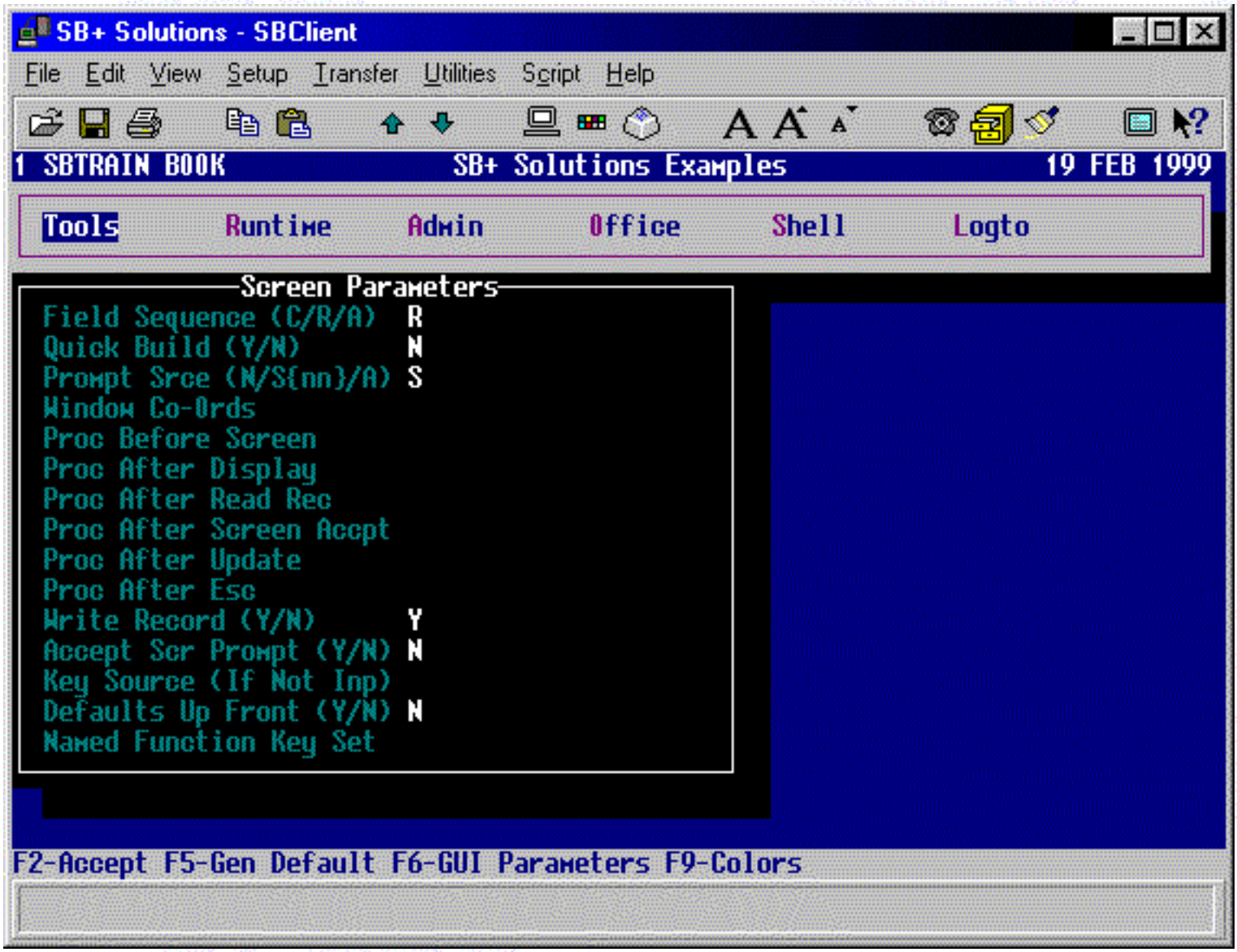
At the "Screen Name" prompt, enter the name of the screen to be created in that file.

The screen name should be short and easily remembered, and is best named according to established **standards**. For example, I recommend using ENTRY.A as the first screen created for a file, with **subscreens** named ENTRY.B, ENTRY.C, etc. For

maintenance applications, MAINT.A, MAINT.B, etc. can be used. Again, this is just my standard -- your standards may be different.

Under normal circumstances, screen names should not end with numbers, such as ENTRY.1 or ENTRY.2. When screens are named this way, SB+ automatically links the screens together so that after the last prompt on ENTRY.1 has been entered, ENTRY.2 is automatically invoked. This is undoubtedly a useful technique, but should be used only when linked screens are the desired result. In short, if you want control over when your screens appear and disappear, avoid numeric suffixes!

After the screen name has been entered, the "F6-Params" or "Params" screen is displayed, as follows:



This screen is used for entering general parameters about this screen definition, such as the window coordinates, processes to be called at certain times, and several other parameters. Notably, all these prompts can be a bit overwhelming when creating a new screen! Fortunately, there is really only one prompt here that you need to focus on at this stage. That prompt is Window Co-Ords, which is used to define the size and shape of your entry screen.

An entry screen is not required to have window coordinates. However, creating screens without window coordinates is not a recommended practice. Windowed screens give the application a more visually appealing user interface, and using windows avoids a particularly nasty SB+ bug in the Screen Definitions tool.

The window coordinates are entered in the following format:

xc,yc,width,depth

...where xc is the column number (X-coordinate) for the top left corner of the box, yc is the row number (Y-coordinate) for the top left corner of the box, width defines the width of the box (including the box characters themselves), and depth defines the number of rows in the box (including the top and bottom lines of the box).

If you like, you can enter the coordinates for your window simply by entering these values into the prompt. For example, to create a window with the top left corner at column 0, row 1, 79 characters wide, 20 rows deep, the following coordinates may be entered:

0,1,79,20

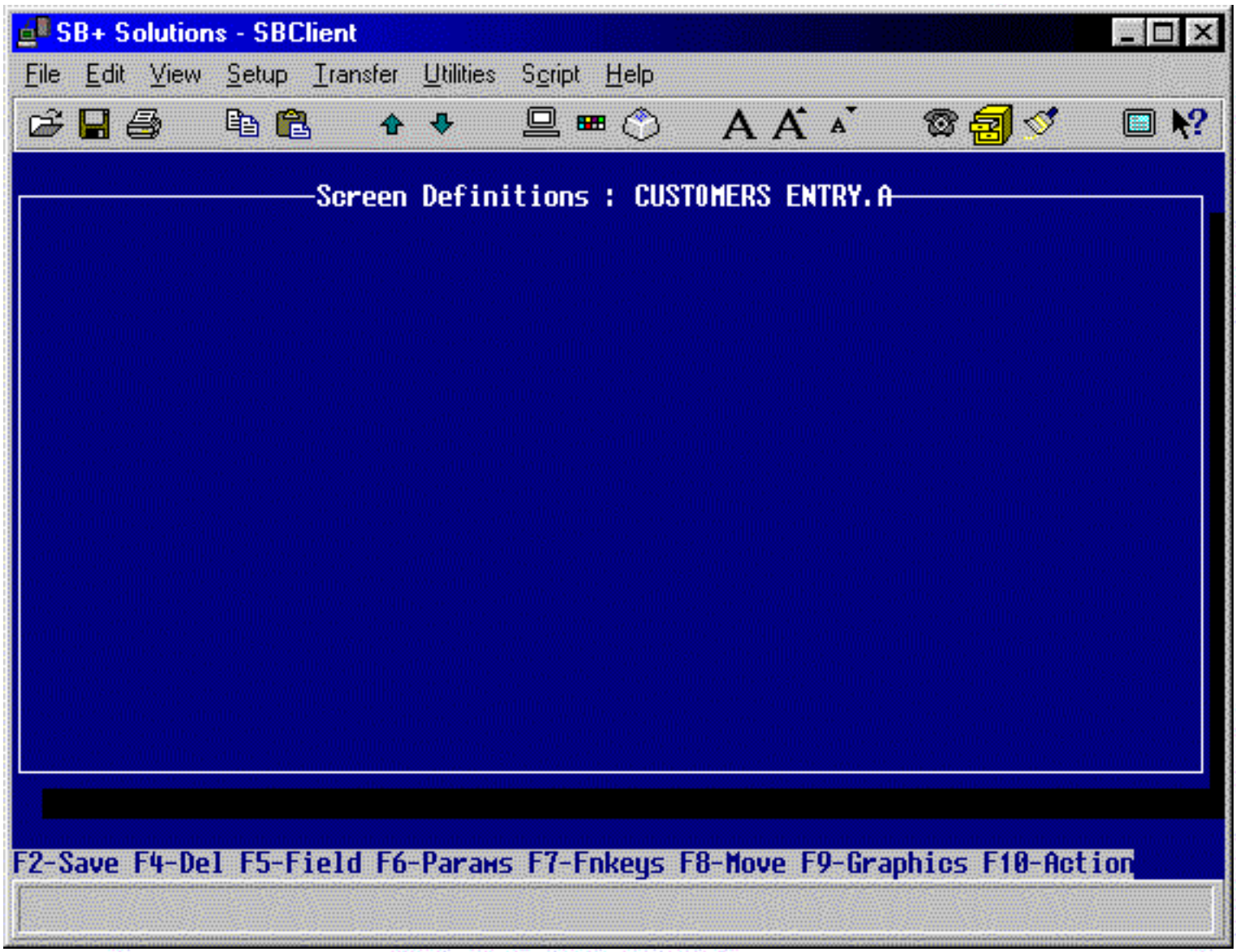
Depending on your standards, care should be taken to avoid placing a window starting in row zero. The action bar, if used, typically goes here. Also, watch out for the bottom of the window -- if it's too close to the bottom of the terminal screen it may be overwritten by help messages when F1 is pressed.

An easier approach is to let SB+ calculate the size of the window. To do this, move the cursor to the Window Co-Ords prompt and press the F3 key. Next, move the cursor to where you want your top left corner. When the cursor is properly placed, press <cr>. Next, move the cursor to the place for the bottom right corner and press <cr> again. Once you've pressed <cr> on the bottom right corner, the F6-Params screen will be redisplayed and SB+ will fill in the appropriate coordinates..

Again, this is the only prompt of concern when first creating a new screen definition. The remaining information on this screen will be discussed as needed in other parts of this chapter.

After the window coordinates have been entered, press the F2 key. This will save the information shown on this screen, draw your box, and allow you to add prompts to your screen. Whenever you ever need to get back to the F6-Params screen, simply press the F6 key when your screen layout is displayed in the Screen Definitions tool.

Based on the window coordinates stated earlier, the entry screen window should look something like this:



Using the arrow keys, notice that the cursor moves inside of the window, but is not allowed outside of the defined coordinates. Noting this, the next step in putting together the screen is to place fields inside of your window.

Important Note: When moving the cursor around the screen definition, the "/" feature of SB+ is disabled. If you need to call a process, press the F10-Action key and select the option labeled PrOc (Call Process).

Input Process Options

There are a variety of useful options available on the Process Definition - Input (/PD.I) screen to modify the behavior of your input process(es). Though many of the options are very specialized, it's good to be aware of the different features offered by this prompt.

There are six options available at the Options prompt on this screen. Each can be used independently, or combinations of codes may be used. These codes are as follows:

A	Restrict amendment: With this option set, the process will allow records to be added and deleted only. Keep in mind that this restricts ALL editing of existing records. If you want to **restrict editing** for only certain records, use a process linked to the **Process After Read** slot for the screen.
D	Restrict deletion: With this option set, the process will allow records to only be added or recalled for editing. Oddly enough, removing the definition for the F4 key on the screen has the same effect, and isn't nearly as annoying as the standard error message.
I	Restrict insertion: With this option set, the process will allow only existing records to be recalled for editing and/or deleted. Keep in mind that this universally restricts the process from creating new records. If this is too restrictive. use a process linked to the Process After Read to conditionally restrict the addition of new records.
O	Non-Amendable: With this option set, the cursor will move through the input prompts on a screen, but nothing can be changed. This is useful when creating an inquiry screen where the function keys perform differently on various prompts.
N	No refresh: Normally, SB+ repaints the underlying screen image when a screen is exited. With this option set, this refreshing is not done.
S	Subscreen: This process will share common memory with the calling process. See "**Subscreens**" in this chapter.

With this in mind, if you want to create a screen that is used only to edit existing records, the options of "ID" would be used. To create a screen which can only insert new records, "AD" could be used. Incidentally, options can appear in any order.

Finally, note that the options I, A, and D are mutually exclusive of the S option. When you consider that a subscreen does no actual updating of the record on the file, there is no purpose for these restrictions on this type of screen.

Creating a File

The first issue to address when a screen is being created is the question "where will the information be stored?". Normally, screens will read information from a file somewhere, and also will most likely write information somewhere.

SB+ has a tool for creating files quickly and easily. This tool replaces the file creation verb at the OE level (typically CREATE-FILE), thus sheltering developers from difficult syntax variations. As an added bonus, the tool keeps track of the names of the files that are created within the system for later reference. This tool, Create File, can be accessed through the menus as follows:

From the main menu of SB+, select Tools. (This menu provides access to all of the tools you will need when creating your SB+ application.) From the Tools menu, select File Create/Delete. Once selected, another menu will appear which allows you to create a new file or delete an existing file. Press <cr> to select Create a New File and a window will appear asking you to enter information about the file to be created, which appears as follows:



At the "File Name" prompt, enter the name of the file that you want to create. If there is no record in the MD (or VOC) of the current account under the name you enter, the name will be accepted. Note that though SB+ does not restrict this entry to upper

case, you should always create file names using upper case letters.

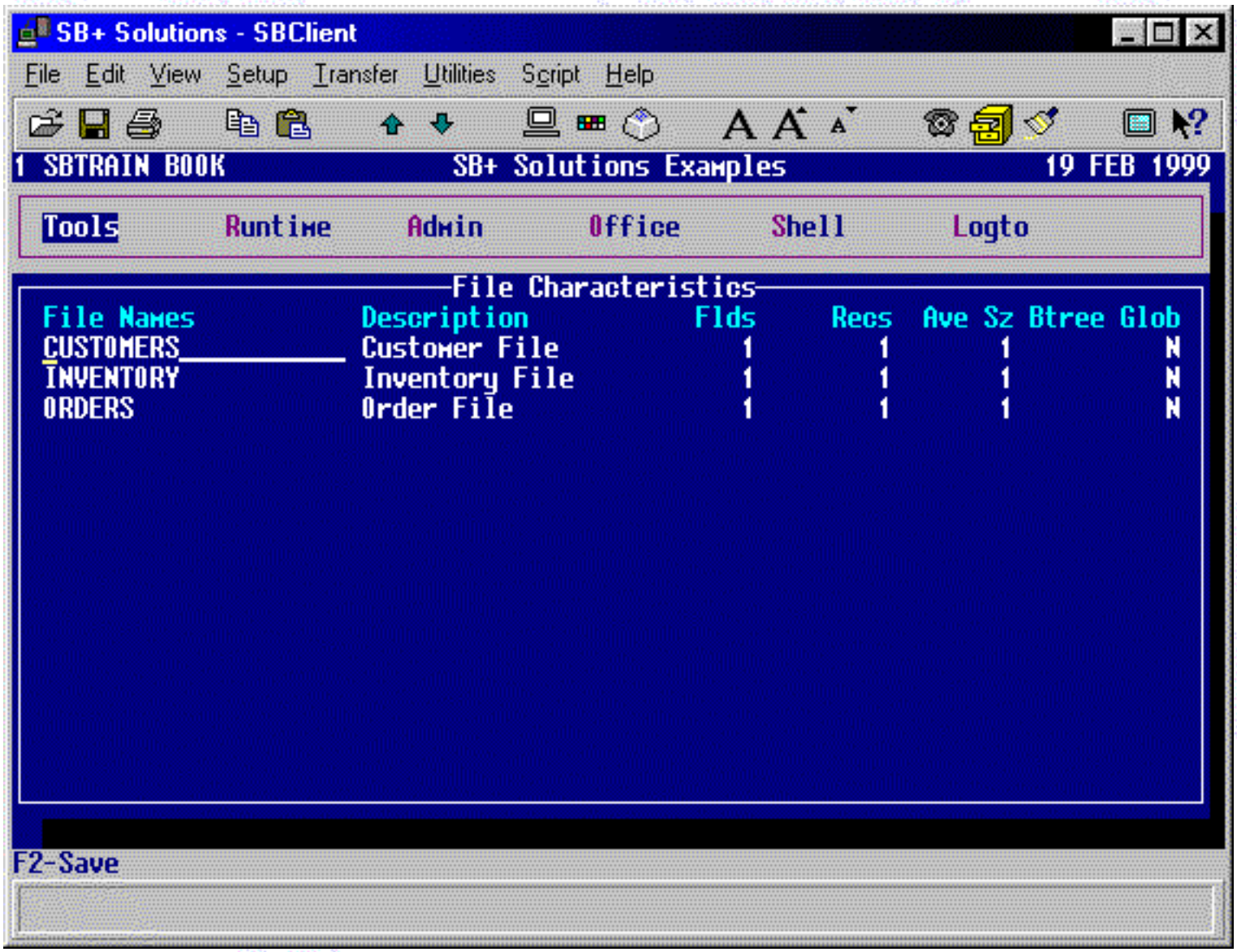
At the "Description" prompt, enter a description of the file. This should be entered in mixed case, and succinctly define the purpose of the file.

Notice that this tool does not prompt for modulo and separation, two elements required by the TCL CREATE-FILE command. Instead, SB+ prompts for the number of fields in each record, the number of records that will be stored in the file, and an average record length, then uses this information to calculate the modulo and separation for the file.

To create a file quickly, don't worry too much about these sizing parameters. Files can be resized at any time in the development cycle, so what you enter here is essentially inconsequential. Therefore, to save time and to save valuable disk space for development, simply enter "1" for each of the sizing prompts so that the file is created quickly using the least space possible.

Once values have been entered into these prompts, the last thing to do is press F2 to accept the values as entered. Once F2 has been pressed, the file will be created and the Create File screen will be redisplayed. At this time you may create other files, or press the Escape key to step back through the menus.

Notice the function keys at the bottom of the screen. When F5-File Characteristics is pressed, a screen is displayed showing all of the files created in the system, as follows:



This listing gives you a complete picture of all files created with the Create File tool. Note that files created at TCL with the CREATE-FILE command are not included in this list.

The F10-Action key is common to many SB+ tools. This is called an **action bar** and is a menu of additional features. For the Create File tool, the following action bar options are available:

QPointer - When this option is selected, SB+ will display a window where you can define a pointer to a file in another account. This consistent interface helps to eliminate the confusion caused by differing Q-Pointer formats on different operating environments.

Review - This option will display a list of the files created in the account. Unfortunately, it doesn't contain as much information as F5-File Characteristics.

Doc - When this option is selected, the screen becomes a word processor where you can define documentation for the file. This documentation can be used as a reference for developers working on the file, or can be included in documentation generated by SB +.

Help - This option will display a menu of help topics for the Create File tool. If you have any questions about this tool that are not

answered by what you've just read, consult the online help. Honestly, you won't find much information there, so feel free to try all of the different options for yourself to see
everything the File Create tool can do.

Creating Field Definitions

Once files have been created, the next step is to create field definitions. Field definitions (i.e. dictionaries) serve two basic functions in an SB+ application:

- Each field definition assigns an alphanumeric name to an attribute position in a record, and;
- Each field definition defines information about how a field should be displayed and/or prompted when it appears on a screen.

Field definitions are entered into the SB+ application using the Field Definitions tool. This tool may be accessed through the menus as follows:

From the main SB+ menu, select Tools, then Field Definitions. Or, for a more direct approach, the tool may be invoked by entering /FD at any menu or input prompt.

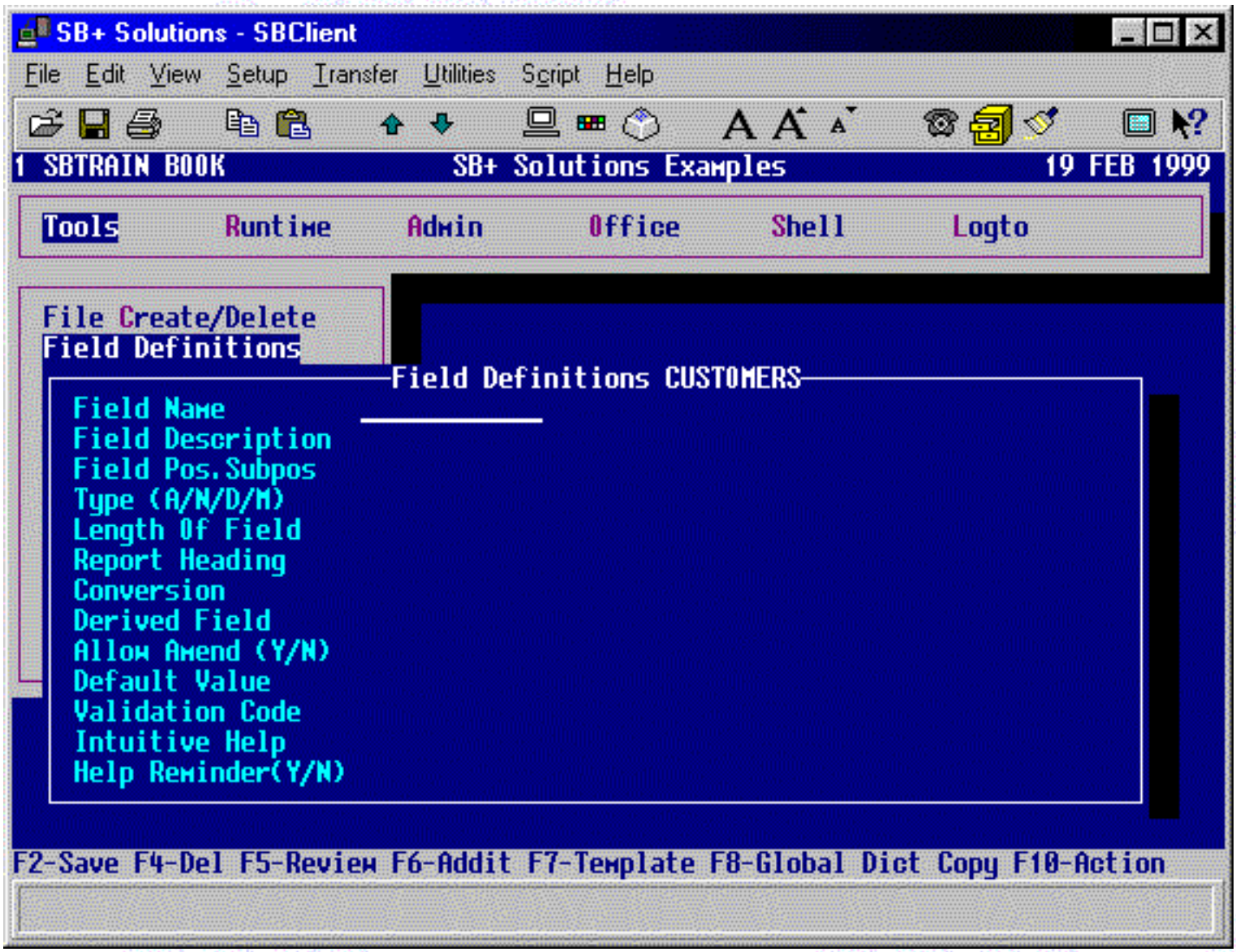
When the Field Definitions tool is started, it displays the following prompt on the bottom of the screen:

```
Dict File Name
```

At this prompt, enter the name of the file where the field definitions are to be stored. Incidentally, though the prompt says "Dict File Name", you need not enter the "DICT" in front of the file name -- simply enter the name of the file.

Field definitions may be entered for any file that is accessible by the current account, not only those files that were created using the SB+ Create File tool. However, only field definitions entered in SB+ can be edited by SB+ -- existing Pick dictionaries may not be edited using the Field Definitions tool.

After the file name has been entered, the Field Definitions screen appears as follows:



At this stage of development, the goal of creating field definitions is primarily to associate a name to each of the numbered attributes of a record in the file. This is called prototyping. Fill in all of the prompts up to and including Conversion, and ignore the rest of the prompts for now.

To create the field definitions, start by entering a field name for your key field at the Field Name prompt. Though SB+ doesn't restrict the entry to upper case, it is very important to enter your field names that way. As with everything else in your project, fields should be named according to an established list of standards so there is little to no confusion as to what a field name represents or what it is used for.

The Field Description is much more than simply a descriptive field; any description entered here will become the prompt text when this field is placed on a screen definition with the prompt beside the input area. Therefore, what you enter here can have real significance on the appearance of screens in your application.

Similarly, the Report Heading is not as innocuous as it may appear at first glance; any description entered into this prompt will become the prompt text when this field is placed on a screen definition with the prompt above the input area. Additionally, this description can appear in the heading when this field is placed on a report. (It can also show up in Selection processes under certain conditions.) Therefore, like the Field Description, take care in what you put into these fields -- whatever you enter will appear again and again as the fields are used in the software.

The Field Pos.Sub Pos prompt is used to enter the actual attribute number that this field name represents. If the field is a key field, the field position is zero. If the field is an attribute in the record, the field position is the actual attribute number. (Other options for this prompt will be explored later.)

This prompt is also used to tell SB+ whether a field is multivalued or not. If .M ("dot M") is entered following the field number, the field is considered multivalued and will be treated as such when placed on a screen definition.

The Type (A/N/D/M) prompt is used to define the type of information that will be stored in this field, and is one of the following single character codes:

A	Alphanumeric, no restrictions
N	Numeric entries only, non-monetary
D	Date entries only
M	Money: Numeric entries only

Though 'N' and 'M' are very similar, they have one subtle difference: Money fields have a default monetary conversion code, whereas numeric fields do not.

Notice that there is no field type for storing time, binary, byte, or other specialized types of fields. If such a field type is needed, generally the "A" type must suffice.

The Length of Field prompt, like the Field Description and Report Heading prompts, is somewhat misleading. One might think that this field is used to limit the length of a value that can be entered on a screen. Certainly this is one use for this prompt. However, more generally speaking, this prompt defines the number of spaces that a prompt will occupy when placed on a screen definition. If you find it difficult to define a length that is appropriate for your screens, don't worry too much about it -- the length of a prompt can be easily adjusted on a screen-by-screen basis.

One very useful feature of SB+ is built-in horizontal scrolling on every prompt on a screen. With this feature enabled, if you need to enter more characters than the space allowed for a prompt, the input area of the prompt will scroll horizontally ad infinitum, allowing you to enter as much text as you desire. Therefore, just because a field is defined in the field definition as, say, 10 characters, any amount of text can be entered. (Incidentally, when the feature is disabled there is a key you can use on a prompt-by-prompt basis for extending each field. Use /KEYS to determine the actual key to press for your terminal type.)

The last prompt of concern at this stage of development is the **Conversion** prompt. Here you may enter a standard Pick conversion code (normally found in the ICONV(...) or OCONV(...) functions in BASIC). This applies primarily to date and monetary fields, though any field in any file may have a conversion.

Once all of this information has been entered, press F2 to save the field definition. After the field definition has been saved, the cursor will return to the Field Name prompt on the screen to allow you to continue entering field definitions until all fields are defined. When you've defined (at least) one field definition for each attribute in a record, your prototyping is complete! Next step: Let's build a screen!

(Important Notes: Entering field names in lower case can cause later problems when using certain operating environments. Also, though we've ignored many of the prompts in the Field Definitions screen, each prompt has a specific purpose and function in the SB+ application. In the following sections, dozens of pages have been dedicated to explaining the myriad of options available with defaults, validation, intuitive help, and the other prompts that have been skipped here.)

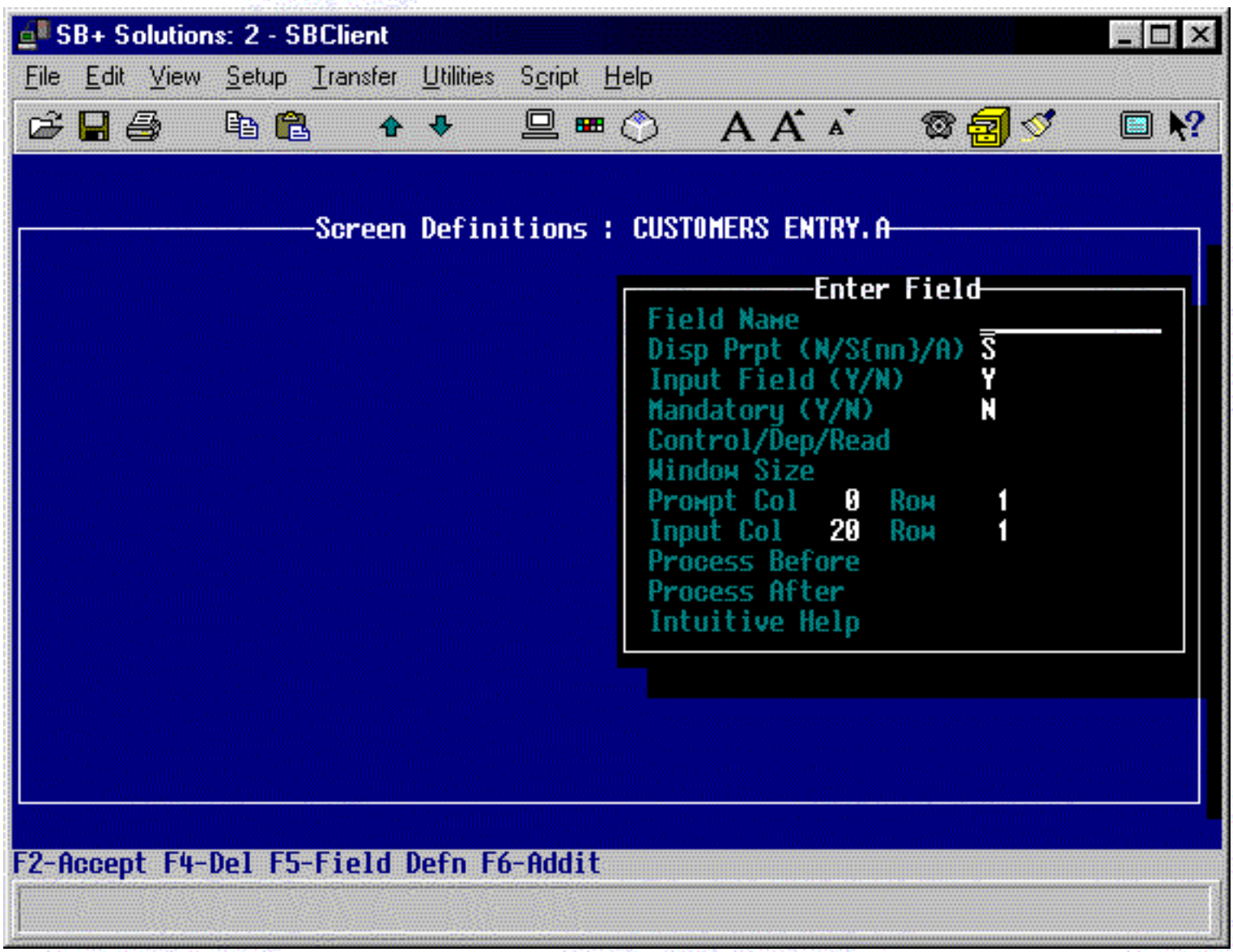
Placing Fields On A Screen Definition

In order for a screen to do anything, it must have some fields placed on it. Otherwise, the screen has no place where values may be input or displayed. Fields may be added to the screen anytime after saving your entries on the F6-Params screen.

Fortunately, placing fields on a screen is very easy, and there are several different ways it can be done. The following sections describe three different ways to place fields on a screen.

Placing Fields Individually

This first method of placing fields is the most flexible and most frequently used. To place each field individually, move the cursor to where you want a field and press the F5-Field key. Once this key has been pressed, a window displays on the right side of the screen as follows:



At the Field Name prompt, enter the name of a field (created using the **Field Definitions** tool) to be placed at this position. To select a field from a list of fields in the file, press F3. Once the list is displayed you may select one field name from the list.

Once the field name has been entered, several other values will be displayed in this screen for your review and editing. Most of the time you will simply press F2 to accept the field name as entered, which will place the field on the screen definition. If you want to change any of the values that are displayed, however, simply press the up and down arrows to cycle through the prompts, changing the values as needed.

The Display Prmpt (N/S/A) prompt is used to tell the screen definition where the prompt label should be placed in relation to the input area. The valid entries at this prompt are as follows:

N	None: No label will be placed on the screen, and the input area will be placed at the current cursor location. This doesn't mean you cannot have a label for the prompt, it simply means that you must type the label for the prompt onto the screen definition.
S	Side: The prompt label will be placed at the current cursor location and the input area will be placed to its right, starting 20 characters from the start of the label. Note that the prompt label will be taken from the Field Description on the field definition. This is the most common option when placing single valued fields or a single controlling multivalued field.
A	Above: The prompt label will be placed at the current cursor location and the input area will be placed in the row below the label. This is the most common option when placing groups of multivalued fields. Note that the prompt label will be taken from the Report Heading on the field definition.

It is interesting to note that this entry is primarily used to calculate the Prompt Col and Row and Input Col and Row shown later in this window. Regardless of what is entered into this prompt, the column and row values may be adjusted manually, thus overriding the positions as calculated by SB+.

At the Input Field (Y/N) prompt, enter "Y" if the user should be able to enter values into the prompt or "N" if this field is for display only. If you need a prompt that is for entry sometimes and for display only sometimes, enter "Y" here and see "**Skipping a Prompt**", later in this chapter.

The Mandatory (Y/N) prompt tells the screen definition that this field must have a value before the record can be saved. Entering "Y" means that an entry is required into this field, whereas "N" means an entry is optional. Note that when you denote a field as a mandatory entry, the field is checked for a value both when the cursor is on the field and also immediately before the record is updated.

For multivalued fields, the Control/Dep/Read prompt is used to define which multivalued field is the controlling field, and which are the dependents.

To illustrate how this is used, let's take a look at a stock window on an order entry application. In this example, let's assume we have the following prompts on the screen:

```
Stock Number Quantity Price
xxxxxxx      xxxx  xxxxx.xx
xxxxxxx      xxxx  xxxxx.xx
xxxxxxx      xxxx  xxxxx.xx
xxxxxxx      xxxx  xxxxx.xx
xxxxxxx      xxxx  xxxxx.xx
```

Let's also assume we will display up to 5 stock numbers and associated quantities and prices at a time. Regardless of this window size, we can enter any number of rows of information into this multivalued set.

The question becomes, how do you want this information to be prompted?

On one hand, we could prompt for all of the stock numbers first, then go back to the quantity and enter the quantity for each stock number, then go back and enter the price for each quantity of each stock number. In short, we enter one whole column of information before moving to the next column. Of course, there's a good chance your users would hate you severely with a screen like this.

Instead, it's more sensible to enter a stock number, then move across and enter the corresponding quantity and price for that stock number before moving on to the next row. In this case, the stock number becomes the *controlling* value, and the other columns are *dependents*.

Therefore, when placing the stock number prompt on our example, we would enter a "C" at the Control/Dep/Read prompt. For the quantity and price fields, a "D" would be entered at this prompt. As long as the prompt sequence is such that the controlling multivalue precedes any dependents, the screen will function as we want it to.

Any number of multivalued sets of information can be entered on a screen this way. Simply add a new controlling multivalued field followed by any number of dependent fields, check the prompt sequence, and you're on your way.

The last prompt on the F5-Field screen that is of concern at this time is Window Size. This prompt is only applicable to the controlling multivalued field, and is used to define how many values should be displayed in the multivalued set. Note that this value affects only the number of rows that are displayed. If you want to limit the number of entries allowed into a multivalued field, additional programming will be required.

Though not discussed here, the remainder of the prompts on this screen are very valuable, and are covered later as they apply to various techniques.

Selecting Several Fields to Place Individually

Rather than pressing F5, entering a field name, and then pressing F2, then repeating the entire process over and over again, you can first select the fields you want to add to the screen, then use F5 to place the fields in the order you have selected them.

To do this, before pressing F5-Field, press F3. Oddly enough, this key is not labeled on the function key line, but will display a selection box of all of the fields as defined for the file. Using the arrow keys, move through the list and press the space bar to select ("tag") or deselect ("untag") a field name. Important note: Be sure to tag field names in the order you want to place them on the screen!

Once you have tagged the fields to be placed on the screen, press F2. What you notice may surprise you: SB+ appears to do nothing! Fortunately, this is merely an illusion. While SB+ doesn't automatically place the fields on the screen, it does build a list of the fields you selected to be recalled one-by-one when you press F5-Field.

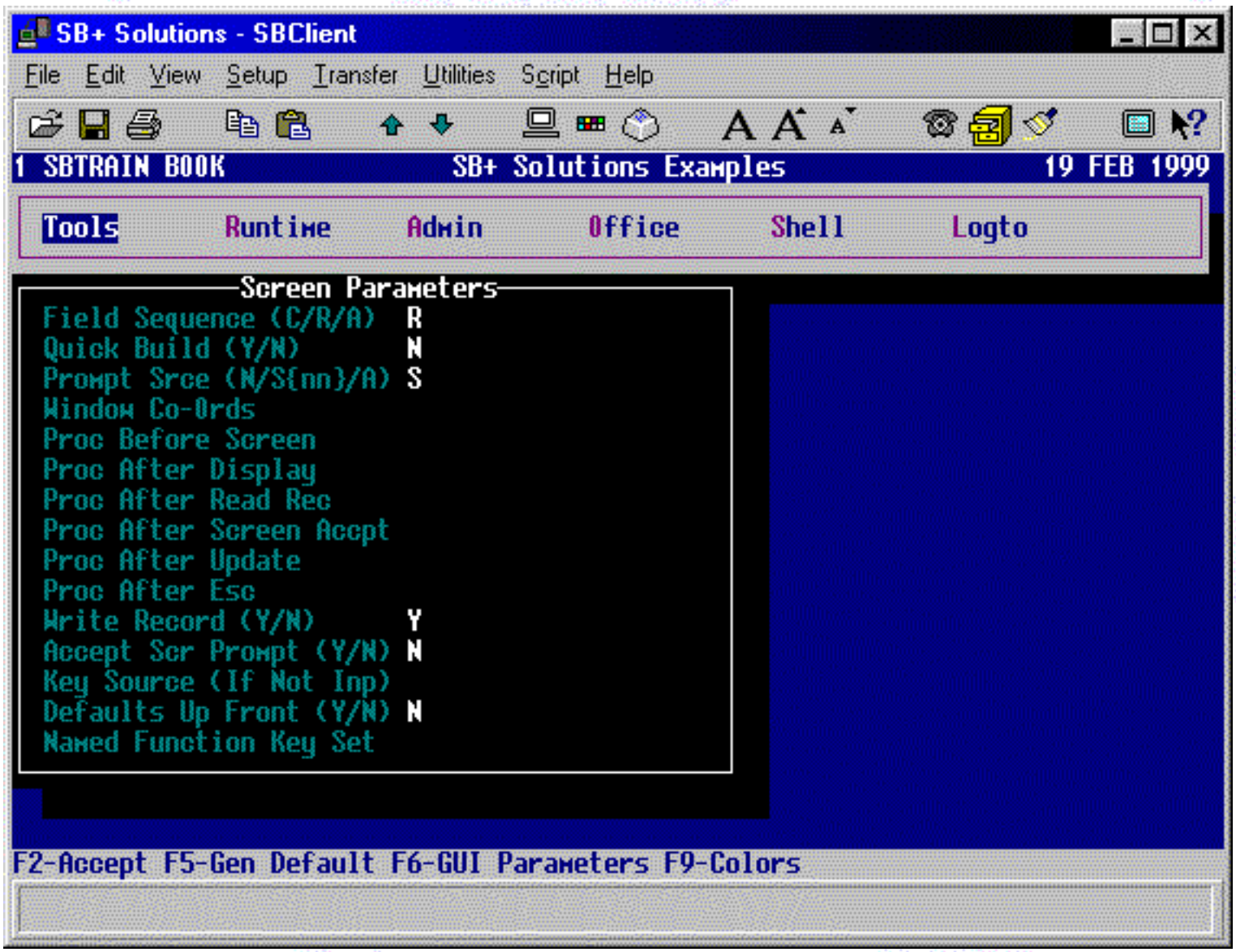
To see this in action, after selecting a series of fields with F3, move the cursor to where you want the first tagged field to be placed and press F5-Field. The first field you selected will be automatically loaded. Edit the displayed information as necessary and press F2 to finish placing the field. Repeat this again and again until all of the selected fields have been placed.

This technique is slightly faster than placing each field individually, simply because you don't have to type the field names into the F5-Field window each time. However, this is not the fastest way to set up a screen -- there is still one other way...

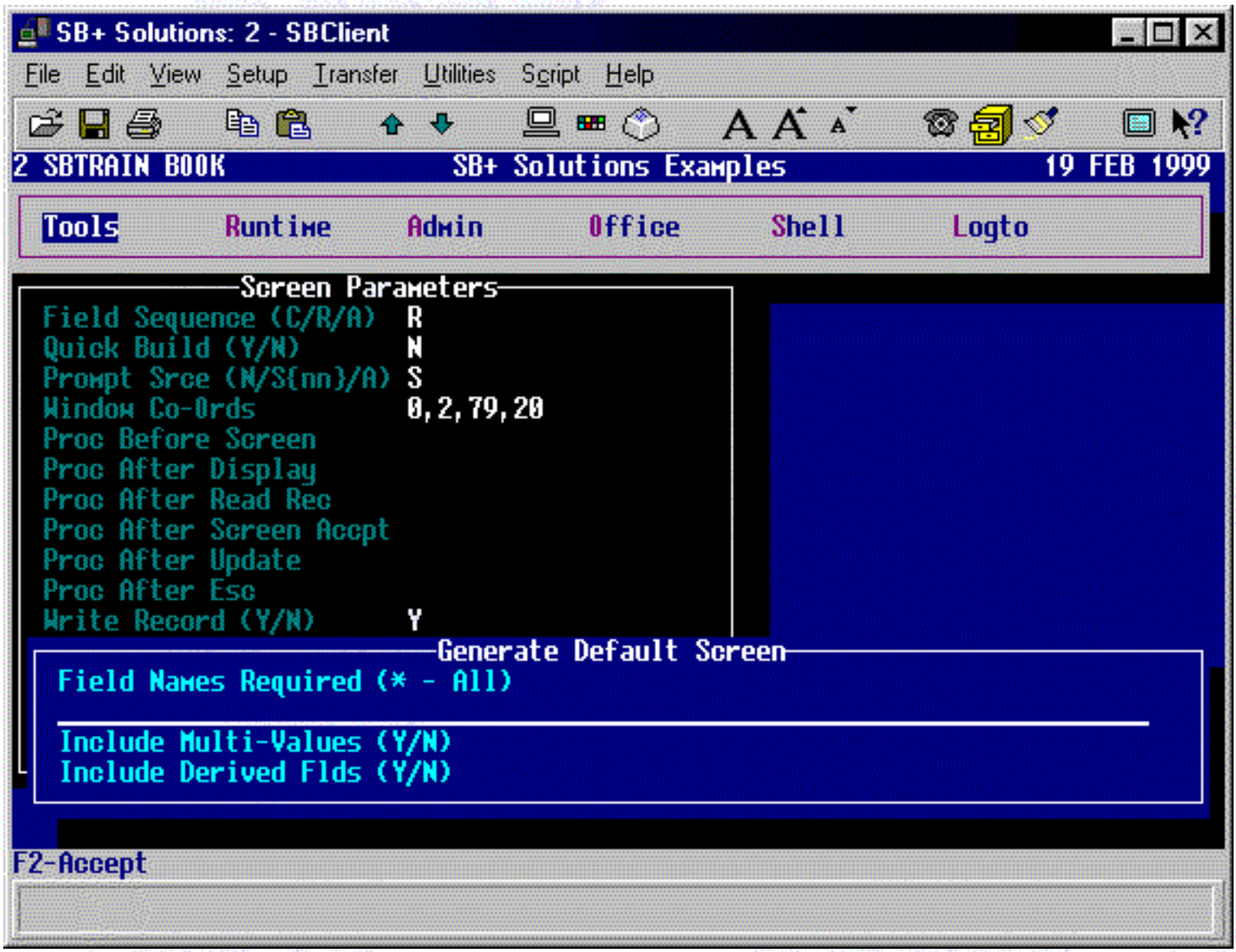
Placing All Prompts At The Same Time

This technique is by far the fastest way to build a screen definition, though it has two major drawbacks. First, this technique can be used only on a new screen, prior to any other fields being placed. Second, the prompts are placed on the screen in simple top-to-bottom fashion, which usually requires some shuffling to improve the overall appearance of the screen.

Oddly enough, though the other techniques for placing fields on a screen used the F5-Field key, this key is not used at all when placing all of the prompts on the screen all at once. Instead, we use the F6-Params screen, where the window coordinates were entered earlier.



On the F6-Params window is a function key called F5-Gen Default. When F5 is pressed, a window is displayed which appears as follows:



At the Field Names Required (* - All) prompt, simply enter the names of the fields you want displayed on your screen in the order you want them to appear. To include all of the fields in the file (sorted by attribute number), enter an asterisk.

If you press F3 on this field, a list of the fields for this file will be displayed and you may individually tag each of the fields you want to include on the screen. When F2 is pressed from this selection box, the names of the fields will be returned into this prompt in the order selected.

The Include Multivalues (Y/N) and Include Correlatives (Y/N) prompts are used when you enter an asterisk at the previous prompt. In short, if you want to include all fields except multivalued fields, enter "N" at the Include Multivalues (Y/N) prompt. Or, if you want to include all fields except derived values, enter "N" at the Include Correlatives (Y/N) prompt. Lastly, if you want to include all fields except multivalues and derived values, enter "N" at both of these prompts.

Once these prompts have been filled in, simply press F2 to generate the default entry screen.

What About Quick Build?

Quick Build is a nifty little feature of the screen definition that allows you to build a screen before you've created the dictionaries for a file. With Quick Build, you create the file and then proceed directly to setting up a screen. When you're ready to run the

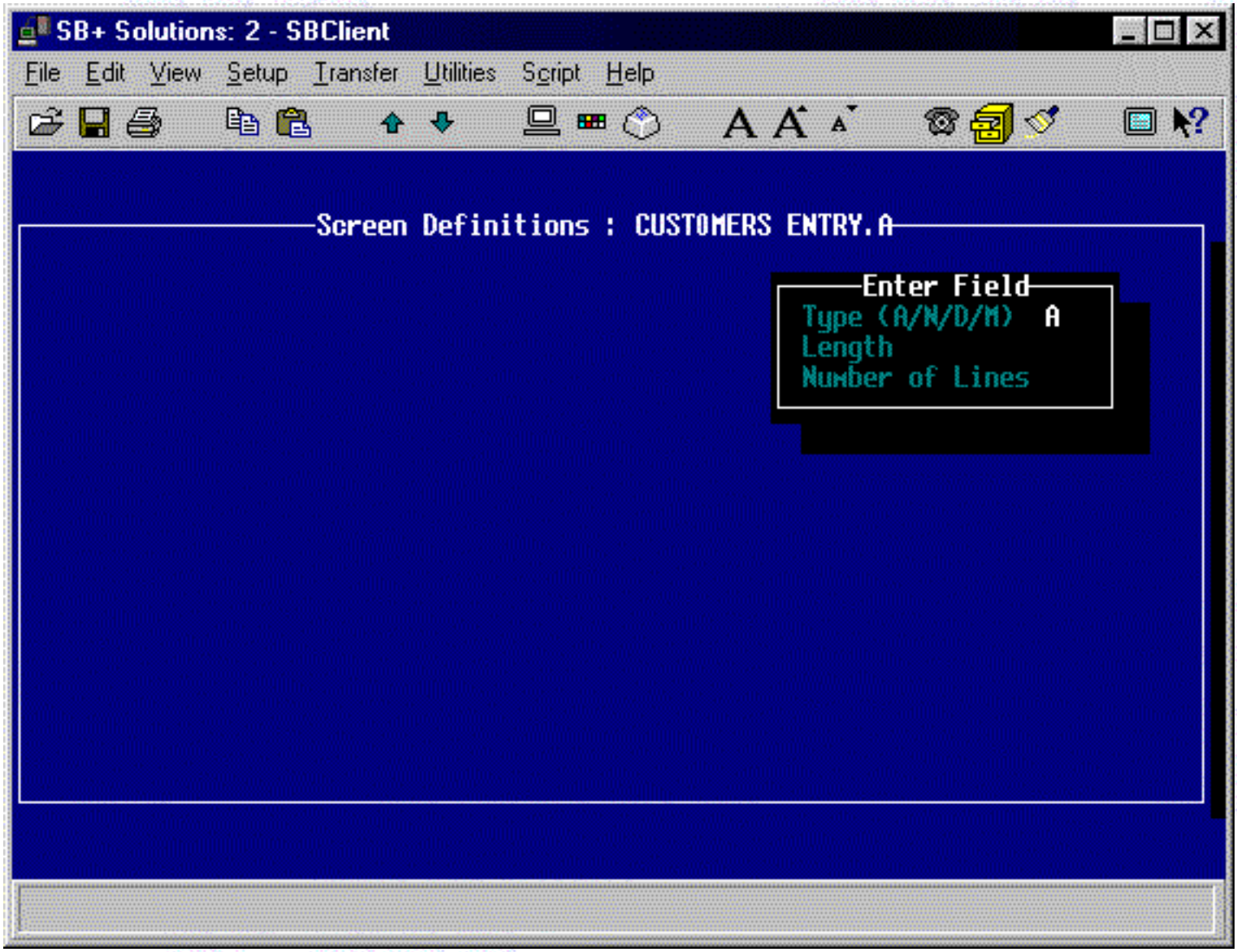
screen, SB+ will create the field definitions for the screen automatically.

Conceptually, it's a good idea: By having SB+ create the field definitions for a file you don't have to think much about the layout of the file being updated. But herein lies the rub: When you don't think about how your files are structured, there's a pretty good chance that the files aren't structured well, and the application as a whole suffers.

I hope you want the best application possible. To get this, you'll need to plan. QuickBuild is the antithesis of planning. Therefore, it's a great tool to impress your friends (or to convince them to buy SB+), but of limited value for creating screens with long-term viability.

In several years of working with SB+ I've occasionally played with Quick Build, but never actually used it for anything other than recreation. Nonetheless, it is fun to impress friends and family occasionally, so let's take a look at how Quick Build works.

To use Quick Build, set the Quick Build (Y/N) flag on the F6-Params screen to "Y". Once this flag is set, the F5-Field window will appear differently, as follows:



Remember, Quick Build will create the field definitions. Therefore, SB+ doesn't need to prompt for a field name or any other information related to the field name.

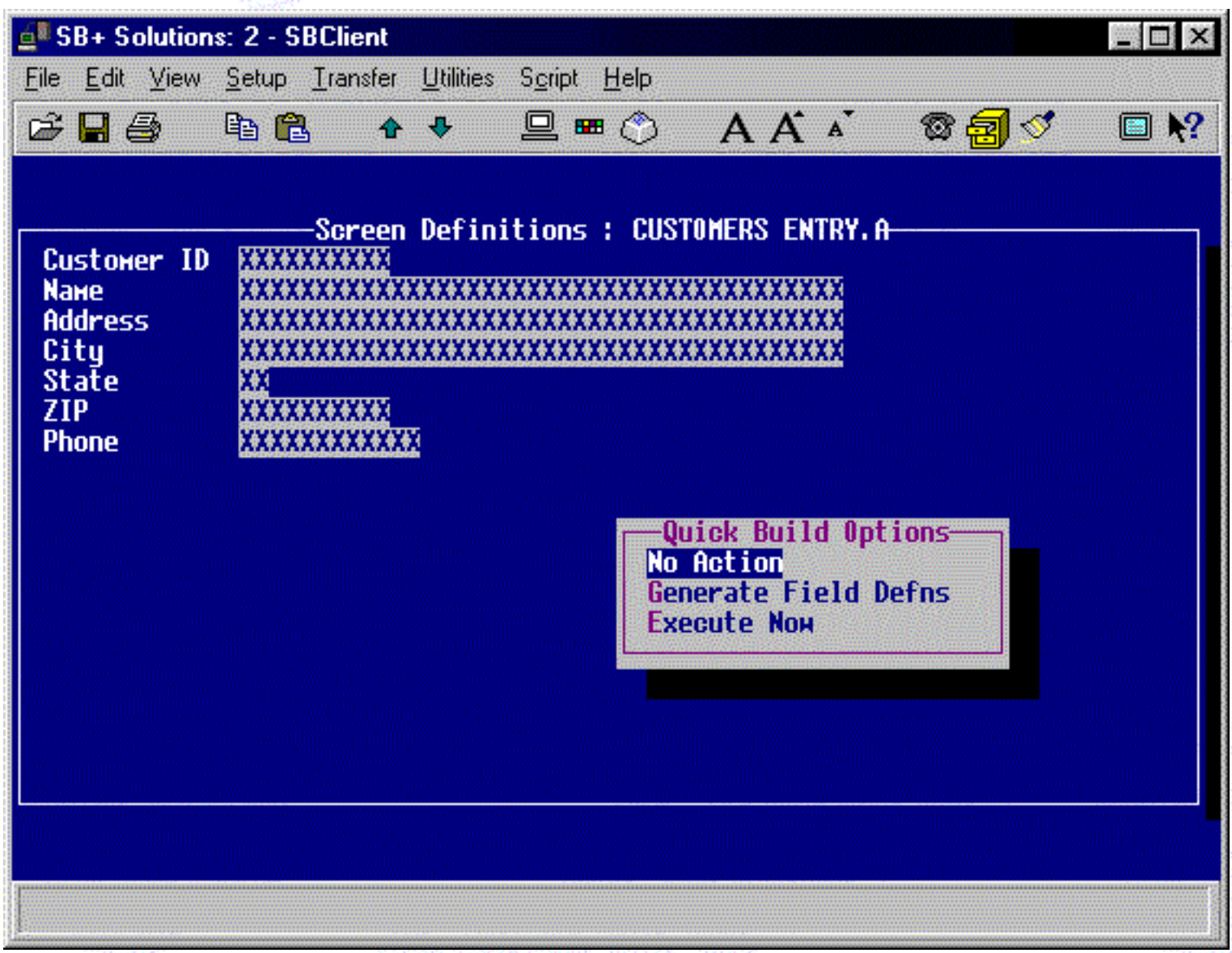
At the Type (A/N/D/M) prompt, enter one of the codes as listed in parentheses. Enter "A" for an alphanumeric field (any characters can be entered), "N" for a numeric field (numbers only), "D" for a date field, or "M" for a monetary field (also numbers only).

At the Length prompt, enter the number of characters that this prompt will occupy on the screen. This will be used later when setting up the default field length for the field definition that is created.

The Number of Lines prompt is used for building multivalued prompts. If "1" is entered here, the prompt is single valued. If a number greater than "1" is entered, the field will be considered multivalued with a window size equal to the number entered.

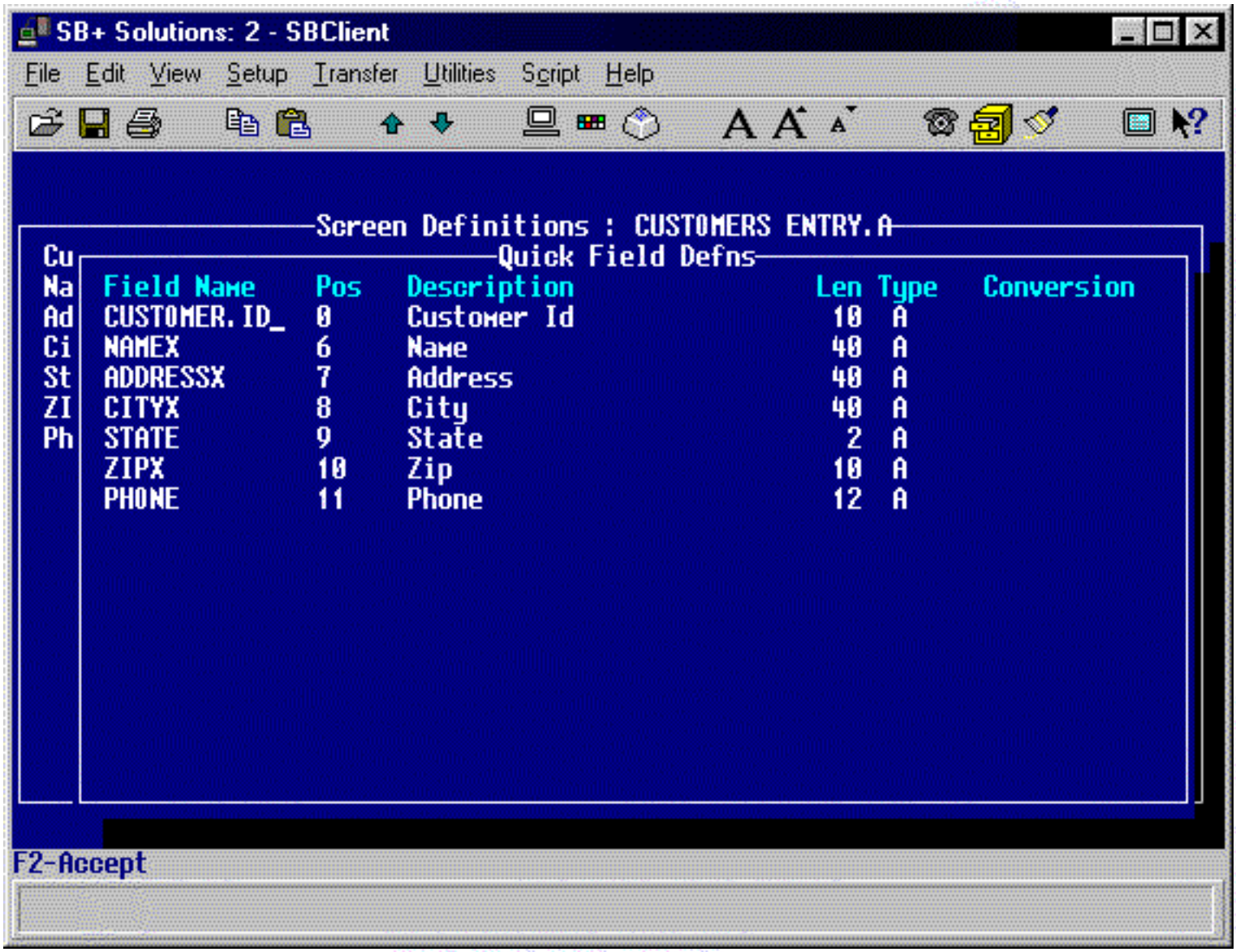
Note that without a field definition, you'll need to type the labels for each prompt onto the screen definition . This is normal for Quick Build. However, each prompt is more than just a description; SB+ will evaluate the text preceding each prompt and will create the actual field names from the label text.

Once all of the prompts have been placed on the screen with F5-Field, press F2 to save the screen. The following Quick Build Options are available:



If you're done with the screen, select No Action. This will take you back to the beginning prompts of the Screen Definitions tool, where you can create a new screen, or edit another screen in a file somewhere.

To create the actual field definitions, select Generate Field Defns. This option will display a window showing all of the fields that are on the screen, as shown following:



On this screen, review the information as displayed and edit as necessary. All values on this screen can be changed. When all of the field information is the way you want it, press F2-Accept to tell SB+ to create the fields.

Before creating the fields, SB+ will ask "Remove Text From Screen If Placed In Field Description (Y/N)". If you like having your labels stored on the screen definition, enter "N". If you want the prompt labels for this screen to be removed from the screen and moved to the individual field definitions, enter "Y". After this question has been answered, SB+ will create the field definitions, turn Quick Build off, and you can view and edit the screen as if Quick Build were never used. Consequently, this is the last thing you do when using Quick Build.

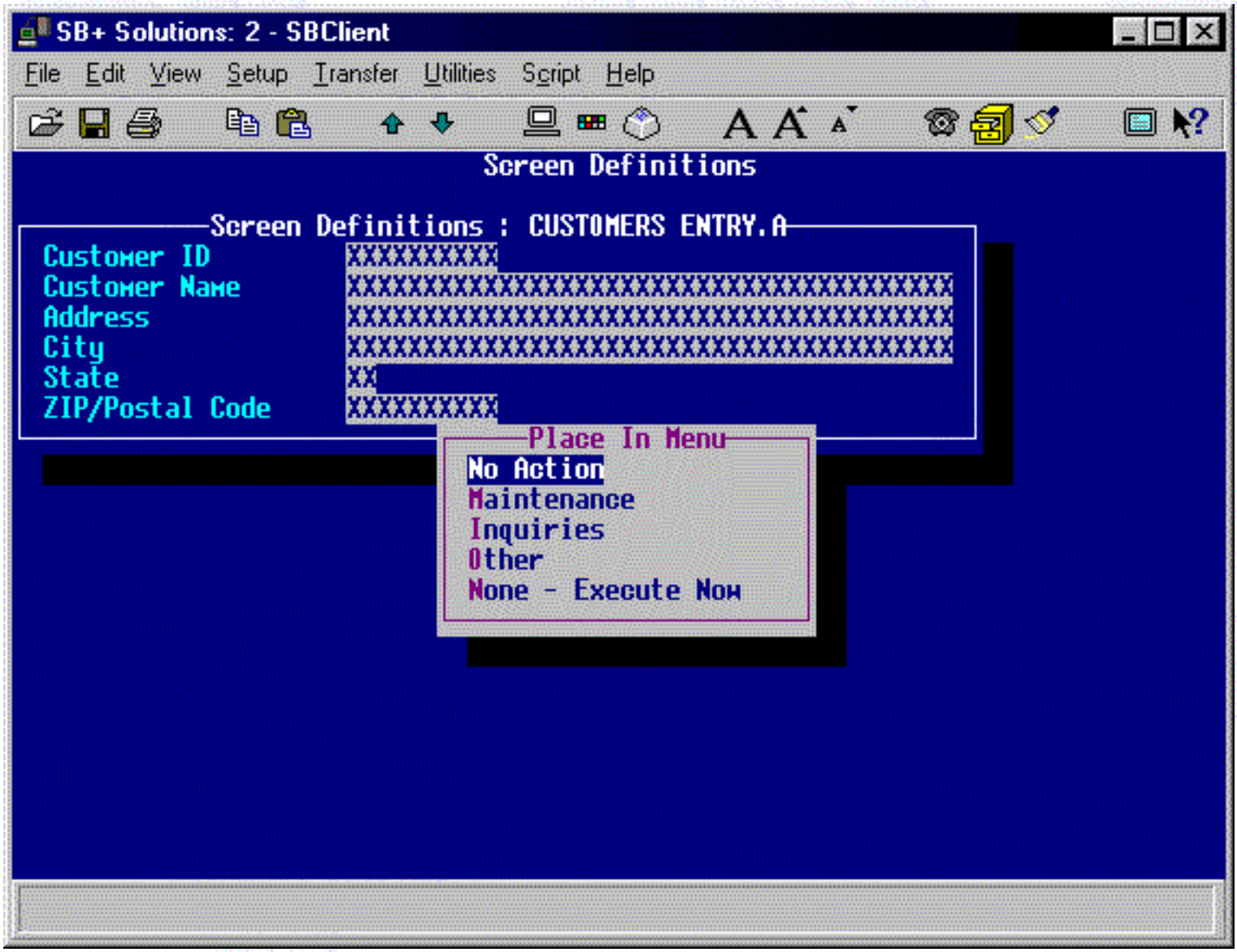
If you select Execute Now from the Quick Build Options menu, SB+ will invoke the screen. The first prompt shown on the screen (reading top to bottom) is assumed to be the key, and all subsequent prompts (reading like a book) update attributes 1, 2, etc. in the record. Without question, this will get a screen up and running very, very quickly. However, the weird thing about this is that

it allows you to add information to the file, edit information in the file, and delete records from the file, all without a dictionary. In other words, you can put information into records in a file, but you have no control over the attribute numbers where this information is stored. If you use this option first, then use Generate Field Defns (changing field positions as needed) you're on a one-way street to either conversionville or a meeting with Madam CLEAR-FILE.

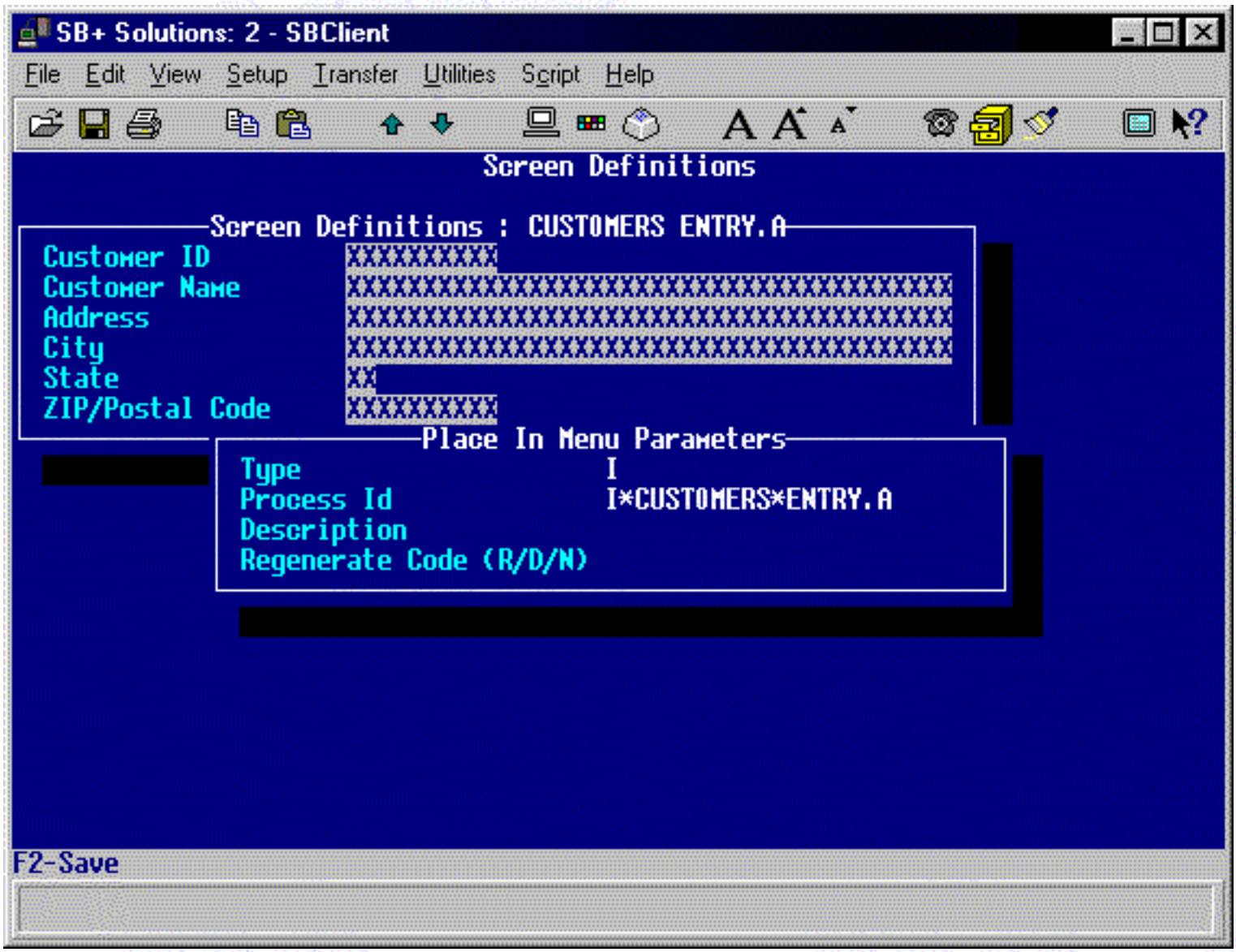
Again, Quick Build is a really nifty feature, and can be quite effective in impressing people with your application development prowess with SB+. For production programming, however, leave it in the toy box.

Running the Screen

After fields have been placed, the screen may be executed. This will give you an idea of how your screen will look to your user(s). To do this, press F2 to save the screen definition, which will display the following menu:



To run the screen, select None-Execute Now from this menu by moving the cursor to the option and pressing <cr>, or by pressing the letter "N". SB+ will then display the following screen:



At this first prompt, SB+ is asking whether this is a screen for Input (Entry) or Output (Inquiry). If this is an entry screen, enter "I". If this is an inquiry screen, enter the letter "O".

Next, a process name will be constructed and displayed. Remember, a screen definition must have an input process in order to do anything!

The process name that SB+ creates is formulated as follows:

*type*file*screen*

...where *type* is either "I" for Input or "O" for Output, *file* is the name of the file you entered when the Screen Definitions tool was first started, and *screen* is the name of the screen that you have created.

You may enter a name that is different from the default name that SB+ provides, but this is not recommended. SB+ will provide this default each time you run the screen using None-Execute Now, so using the default name ensures you don't have to remember something different each time you run the screen.

After the process name has been entered, you will be prompted to enter a description. The description you enter here will become

the title of your screen, shown centered along the top border of the window.

Finally, after the description has been entered, the screen will be started, and you can enter information into it. To save records, enter information and use the F2-Save key. To delete a record, enter the ID of the record to delete and press the F4-Del key to remove it from the file.

As a bonus, SB+ automatically adds **Intuitive Help** to the key field. Therefore, to see a list of all of the records on file, simply press the F3 key when the cursor is on the key field for the screen!

Placing Graphics on a Screen Definition

To draw a graphic on the screen definition, place the cursor at the position where the graphic is to start, and press the F9-Graphic function key. A prompt will be displayed asking what type of graphic you want, or whether you want to erase a graphic, as follows:

(H)oriz Line (V)ert Line (B)ox (C)ross (E)rase _

The following sections describe these options in more detail:

Drawing a Horizontal Line

To draw a horizontal line, place the cursor at the position where the line should start (drawing left to right), press the F9-Graphic function key, and enter "H" at the prompt. Next, move the cursor with the right arrow to the place where the line should stop and press <cr>.

Note that horizontal lines must be drawn from left to right. SB+ will not let you draw a line from right to left.

Drawing a Vertical Line

To draw a vertical line, place the cursor at the position where the line should start (drawing top to bottom), press the F9-Graphic function key, and enter "V". Next, move the cursor down to the place where the line should stop and press <cr>.

Note that SB+ does not allow lines to be drawn from bottom to top.

Drawing a Box

To draw a box, move the cursor to the position where the top left corner of the box should be placed, press F9-Graphic, and enter "B". Move the cursor with the right and down arrows to the position where the lower right corner should be placed and press <cr>.

Note that boxes must be drawn from the top left corner to the bottom right corner.

Drawing a Cross Character

The cross character is somewhat unique, as it is the only graphic that is only 1 character. This character is used to intersect horizontal and vertical lines on the screen definition.

To draw a cross character, move the cursor to the position where the character is to be placed, press F9-Graphic, and enter "C".

Erasing a Graphic

To erase a graphic, move the cursor to the place where the graphic was started. For lines, this is the leftmost or topmost position. For boxes, this is the top left corner. For the cross character, this is on the character itself. Once the cursor is in the right position,

press the F9-Graphic function key and enter "E". The graphic will then be removed from the screen definition.

Special Rules for Graphics

Moving a Graphic

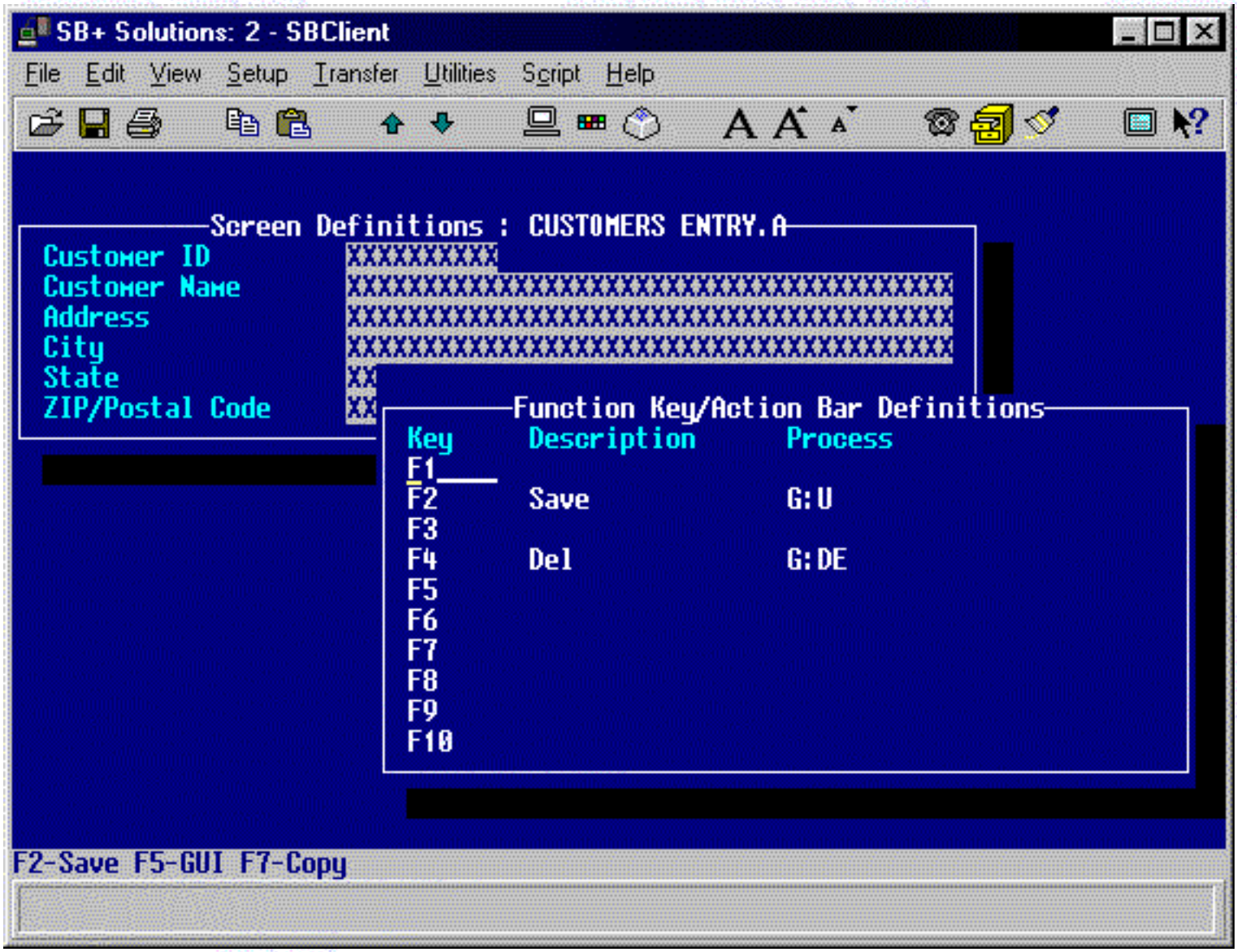
In the Screen Definition tool there is no way to move a graphic. Instead, you must erase the existing graphic and then redraw the graphic at the position where you want it.

Graphics And Prompts Should Not Share the Same Position

A graphic and field can both be placed at the same location on a screen. When the screen runs, the field name will be placed first, the graphic next, and then data will overlay the graphic. This can result in a very strange looking screen, and therefore this is not recommended.

Defining Standard Function Keys

In the Screen Definitions tool, standard function keys are defined by pressing the F7-Fnkeys function key, which appears as follows:



In the first column is the name of the function key -- you cannot change the values shown here.

The second column is the label that you want shown for the key. As you see in the example, F2 is assigned the word "Save". When the screen is running, this will appear as "F2-Save". There is a limited amount of space available for function keys (75 characters, if I remember correctly), so plan your labels carefully.

The third column is the process to be called when the key is pressed. As stated earlier, this process can be used to do just about anything, including linking this program to a completely different one.

To change one of these values, move the cursor using the up and down arrows until the cursor is on the function key to change, then press <cr>. Next, enter the label for the key. Keep in mind that all of the labels together must fit on one line at the bottom of

the screen, so each label must be short and to the point. Once a label has been entered, enter the name of the process to be called when that key is pressed. Once all of the labels and processes have been defined for all keys, press F2 to save this information.

Though only ten function keys are shown on this F7-Fnkeys screen at a time, there are actually three pages of keys that can be defined here. To access the other pages, use the up and down arrows, or Page Up and Page Down to scroll through the values in the screen.

The first page on this screen is where you define the "regular function keys", i.e. the keys as you want them to appear when your screen is first invoked. There are only ten keys here, of which four (F1-F4) are reserved for special purposes. Therefore, you have only six function keys available for your application, and in some cases, even fewer.

The second page on this screen is where you define the "toggled function keys". Here you set up a set of 10 alternate definitions for function keys F1 through F10. Using the standard SB+ process named TOGGLE, your screen will toggle between the regular function key definitions and toggled function key definitions. When this technique is used, the number of available function keys is reduced to five.

To use TOGGLE, simply type TOGGLE in the Process column for one of the available function keys on the first screen. Also, be sure you define one of the toggled function keys to also call TOGGLE -- otherwise, the user will be able to get to the toggled function keys, but won't be able to return to the regular function keys. Preferably, if F9 on the regular function keys calls TOGGLE, F9 on the toggled function keys should call it as well.

Commentary: While TOGGLE can be useful for expanding the number of available function key features, it can also confuse the user and cause them to press the wrong key simply because the wrong function key line was displayed. Therefore, instead of using toggle, I use the F9 key to call a menu (called a "**More Menu**") which displays additional options that wouldn't fit on the function key line.

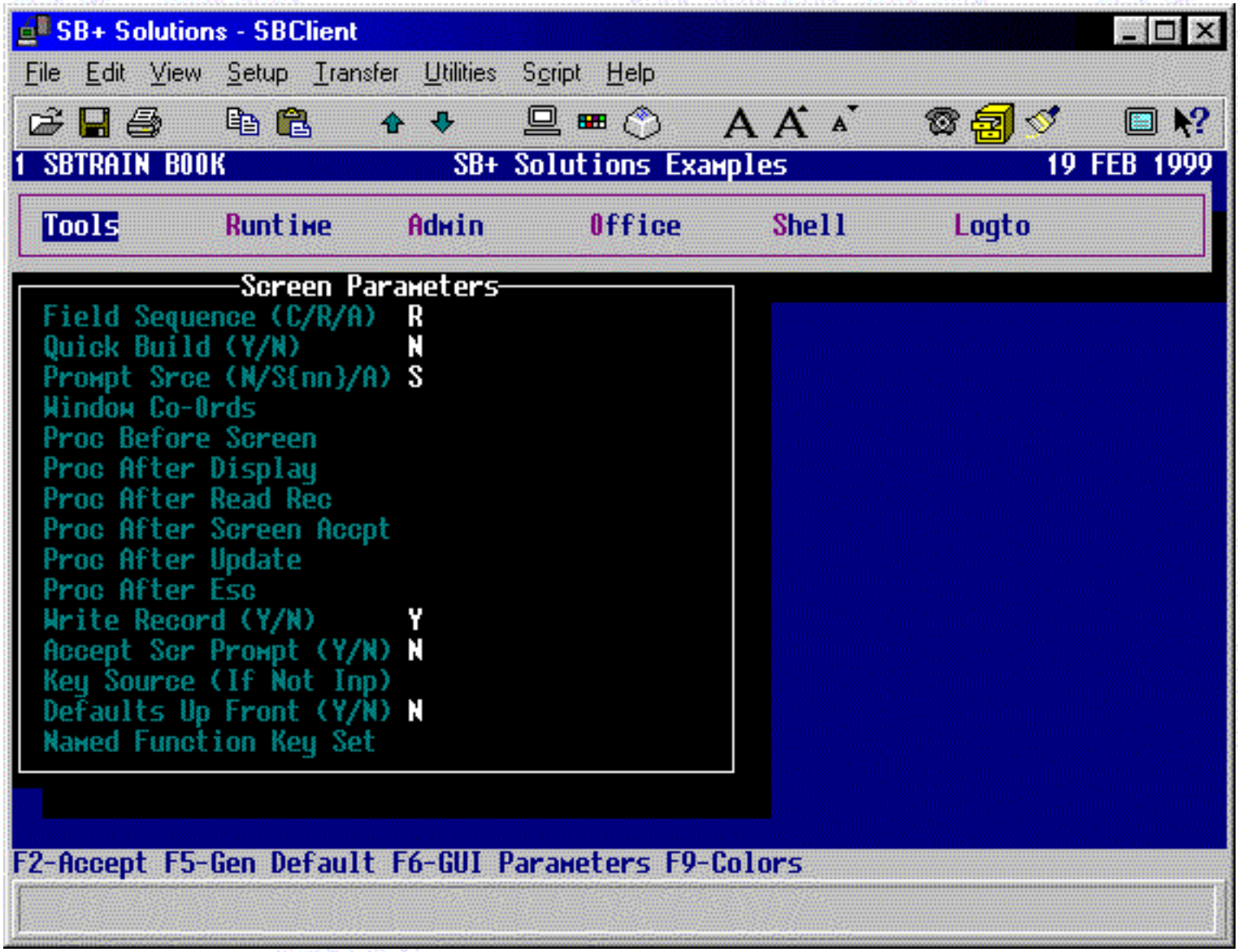
The third page of this screen is where you define the options to appear on the *action bar*. Each screen can have an action bar, which defines a series of menu options displayed at the top of the screen (outside of the window). When the action bar is invoked (typically by pressing the F10 function key) the user is allowed to select one of the action bar options.

To make this work, first define options for your action bar. Like function keys, action bar options are defined in terms of the description (or label) that will be displayed and the process that will be called. Unlike function keys, however, up to 15 action bar options may be specified. Once all of these have been defined, go back to the regular function keys and define F10 so that it calls a process named ACTION. When this is complete, press F10 when your screen is running and the action bar you've defined will be displayed.

Note: If an action bar is defined but F10 is not defined, SB+ automatically redefines this key to invoke the action bar. While this is a nice automatic feature, it can cause confusion months after the original development. If someone looks at a screen definition and finds F10 undefined, they may put something there without thinking that SB+ may be automatically redefining this key to invoke the action bar.

Defining Named Function Keys

On a screen definition, the F6-Params screen appears as follows:



Note the last prompt on this screen. Using this prompt, you can link an *externally defined function key* set to your screen definition. When you use this feature, SB+ ignores the function keys that are defined on the screen definition and instead uses the function key assignments on the externally defined set.

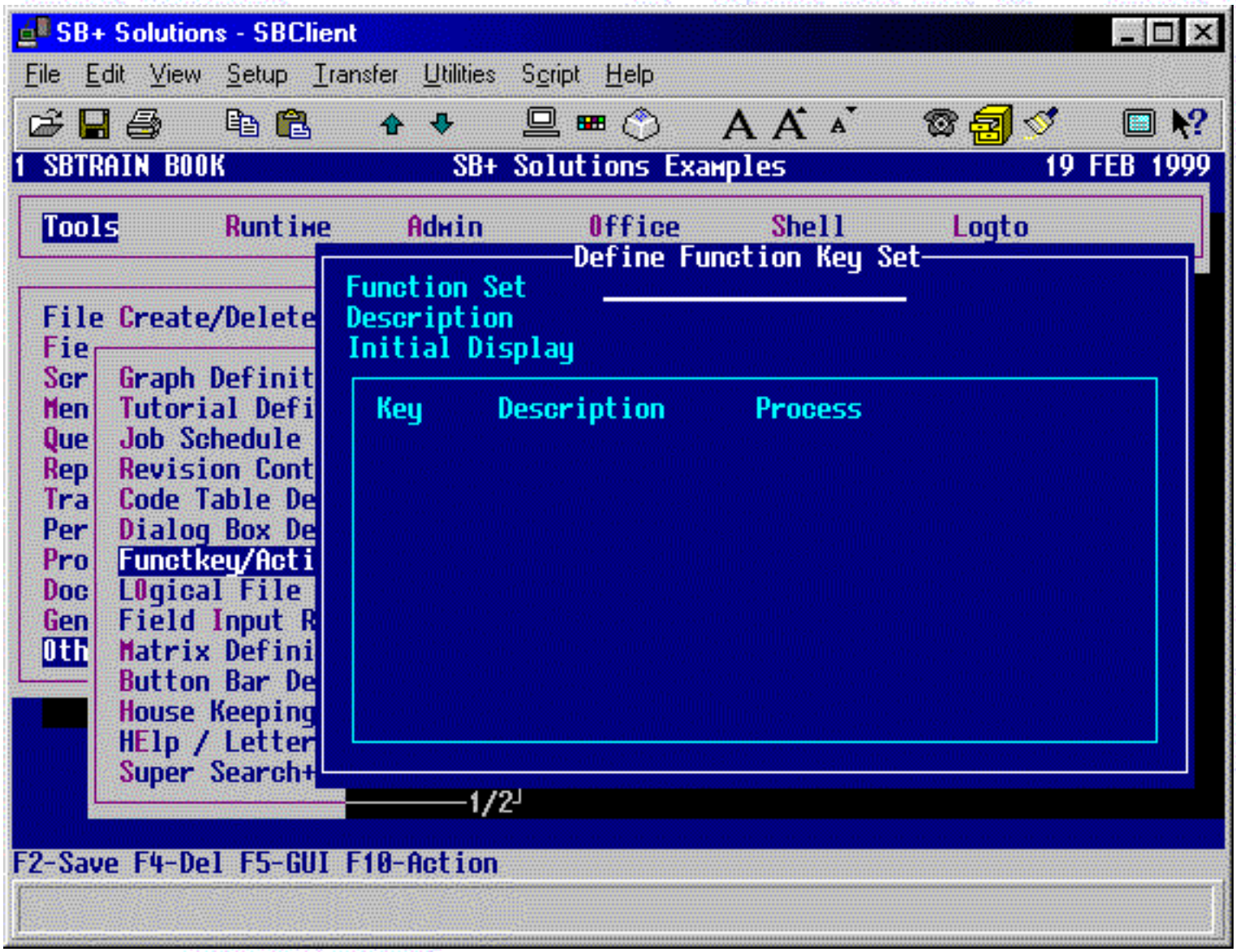
Named function key sets have several advantages over standard function key definitions. First, a named function key set may be accessed by any number of screen definitions, whereas a standard function key set applies only to a single screen. Second, the named function key set allows you to specify how the function key set is to be initially displayed. Several different display options are available, allowing you to display the standard set and action bar, toggled set and action bar, toggled set without the action bar, among others. This option makes named function key sets a preferred option in my own development, as it allows control over not only what the function keys do, but also how they appear.

You may define a named function key from inside the Screen Definitions tool as follows:

Press F6-Params. When this screen is displayed, use the up and/or down arrows to move to the last prompt on the screen. Once the cursor is on this prompt, enter /FNKEY.DEFN to invoke the tool that is used to create the named function key sets. (You can do this without moving the cursor to the prompt -- I simply prefer to do this so when I finish defining the set, the cursor is on the right prompt!)

If you're not in the Screen Definitions tool, select "Tools" from the main menu, then "Other Tools/Utilities", then "Funckey/Action Bar Defns" from that menu to start the tool.

The FNKEY.DEFN process will first ask for the name of the file where the definitions are to be stored. At this prompt, press <cr> to accept the default. The following screen will then be displayed:



At the first prompt, enter a name for the function key set. This name should be easily remembered and easily entered into the Named Function Key Set prompt on the screen definition.

In the Description field, enter a description for the function key set. This is merely a comment entry, but should be complete enough to adequately identify how this function key set will be used.

In the Initial Display prompt, enter a code to determine which sets of keys will be displayed when the screen is started. This may be one of the following values:

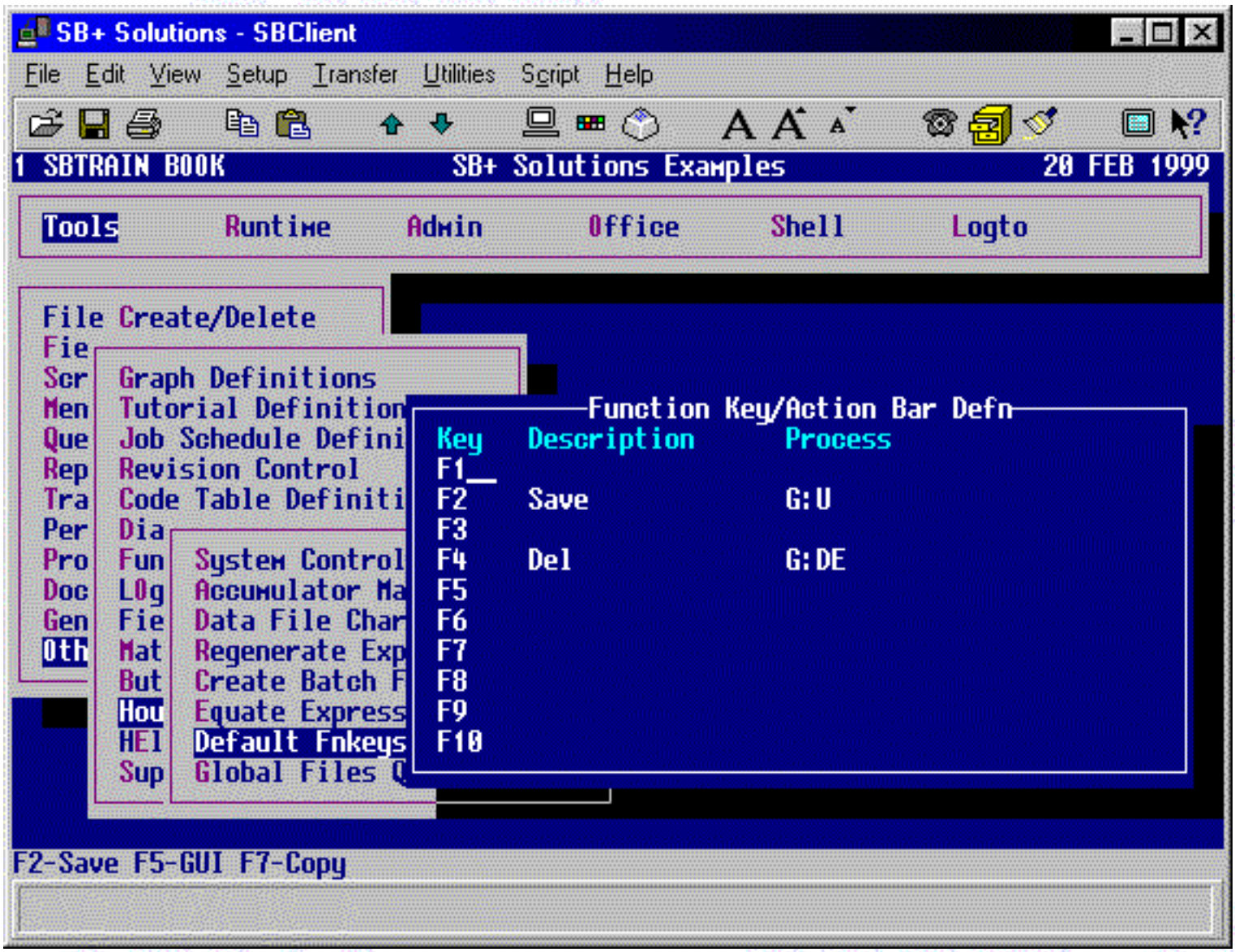
0	(Default) Display the first set of function keys only
1	Display the second set (also known as the "toggled set") only
2	Display the first set of function keys and also the action bar
3	Display the second set and also the action bar

The remainder of this screen works just like the standard function key definition part of the Screen Definitions tool. This screen is explained in detail in the **previous section**.

Defining the Default Function Key Set

When you define a new set of function keys (either **standard** or **named** sets), a few keys (like F2 and F4) are automatically loaded by SB+. Wouldn't it be nice if SB+ could automatically load more than just these keys? For instance, it sure would be nice if SB+ would automatically load a consistent action bar every time I create a new set of function keys. This way, I would never have to define the action bar -- SB+ would load the default action bar every time a new set of function keys is created.

Fortunately, the default function keys can easily be edited for a given **system**. From the main menu, select Tools, Other Tools/ Utilities, House Keeping, and finally Default Fnkeys/Action Bar. Or, for the direct approach, enter /SD.FNKEYS.DEF at any input prompt or menu. When this tool is started, the following screen is displayed:



This screen allows you to edit the function key/action bar template that is copied when new function key sets are created. Because this template is used only when new sets are created, changing this template will not affect any function key sets that have been previously defined.

So What's This G:U and G:DE?

After looking at the function key definition you may be wondering about the codes "G:U" and "G:DE" which are automatically assigned to function keys F2 and F4, respectively. They always seem to appear by default, even though they don't seem to have been entered anywhere. Why is this so? Most important, however, are they significant?

The answer is, of course, yes -- they are very significant, as these two little sequences of characters tell the screen definition that F2 is used to update (save) the record, and F4 is used to delete the record.

Any time you see a G: followed by a letter or sequence of letters, you are seeing an inline Goto process. Stated simply, the letter or letters following the G: tell the screen definition where you want to go. In the case of G:U, the "U" tells the screen to go to the "update step", which is where the record is written. The "DE" in G:DE, on the other hand, tells SB+ to ask the user if the record can be deleted, and then go to the "delete step" to remove the record after the user has confirmed the delete.

Incidentally, the delete step and the update step are the same step in the screen definition. The difference between the two is in how the **common variable** @ACTION is used. If the record is being deleted, @ACTION is changed to 3 as a signal to the update cycle that the record is being deleted. If the record is simply being saved, @ACTION maintains its original value which was set when the record was read.

Behind the veil, the G: prefix merely says to set the value of the common variable **@RTN.FLAG** to whatever follows the colon. SB+ is constantly checking this variable to see which prompt (or "step") to go to next. Therefore, G:U is equivalent to a paragraph or assignment process that assigns the literal value "U" to @RTN.FLAG. When SB+ sees this value in @RTN.FLAG, it jumps to the update step and saves the record.

There are times, however, when using the "G:" feature may not be enough. In certain cases, you may want to do some additional processing before telling SB+ to go somewhere. In this case, you may need to set the RTN.FLAG variable yourself. Consider this example:

On a customer entry screen, if the customer has an outstanding balance we do not want the user to delete the customer's record. To implement this, we simply need to prohibit the common variable RTN.FLAG from being set to "D" (delete without confirmation) or "DE" (delete with confirmation). Knowing that pressing a function key will call whatever process is listed, we cannot simply list G:DE as the process to call when F4 is pressed. Instead, we must call another process to do the verification, setting RTN. FLAG to the proper value if the verification succeeds. In this case, a process such as the following paragraph may be used:

```
IF BAL = 0 THEN
  @RTN.FLAG = 'DE'
END
```

In this case, your screen definition will not use G:DE to set the common variable RTN.FLAG. Instead, the screen definition will call the paragraph, which will set @RTN.FLAG only if the BAL field is zero.

G:U, however, is an exception to the exception. If F2 is the function key to save the record, F2 must be defined as G:U. If it is defined as anything other than F2, the cursor will "fall off" the bottom of the screen after the last prompt, rather than cycling back to the first prompt after the read step (i.e. normal cursor movement). If there is additional validation to be done prior to saving the record, it must be done in the Process After Screen Accept (defined in the F6-Params screen).

Lastly, G:U and G:DE are complete and valid processes, and as such can be called anywhere a normal process can be called. If

you want the screen to go to the update step after the last prompt on the screen, for example, you can simply put the G:U on the Process After slot of the last field.

A **complete reference** of valid values for the RTN.FLAG common variable may be found in **Chapter 8**.

Using An Action Bar

Action bars are defined using the third page of function key information on either a standard or named function key set. Of course, like anything in SB+, how you use this feature is entirely up to you.

There are basically two schools of thought when the matter of action bars is discussed. Perhaps by taking a look at the two most popular opinions on this matter you will have less difficulty deciding which one (if either) will be best for your own development.

On one side of the issue we have people who feel that the action bar is an extended function key line, and should be available to be used for whatever the application requires. Each screen, then, has its own action bar and there is little, if any, similarity between the action bar on one screen vs. the action bar on another screen. This is certainly the most flexible approach, as it puts no constraints on how the action bar can be used. On the minus side, however, this can result in applications that are more difficult to use, as there is nothing to prevent options from being "hidden" in the action bar instead of being readily available from a function key.

In contrast to this, the other camp is of the opinion that the action bar should be standard across all applications. This one standard action bar is then used to add consistent functionality to all screens, similar to a Microsoft Windows® top menu. Of course, this approach is not as flexible as the other, but results in an application with a consistent look-and-feel throughout all screens.

Is one way more right than the other? I suppose it depends on who you talk to. Personally, I prefer the second approach for the standard user interface, rather than the flexibility of the non-standard action bar. Without question, others will disagree. Ultimately, then, it becomes an issue that will be decided either by the people who will use your software or the people who sign your checks.

Using A Standard Action Bar

In an application with a standard action bar, there can be much debate over the functionality that the action bar should have. This section is intended to provide some "food for thought" to help you get started on the right track.

Regardless of the options you have on your standard action bar, it is paramount to set it up on the default function key set before creating any screens. With the standard action bar as part of the default function key/action bar set, the action bar options will be automatically applied to every new function key/action bar set.

Personally, I prefer to have the following options on a standard action bar:

File	Invoke a File Menu, ala Microsoft Windows
ToolBox	Display a standard toolbox of user options
Query	Invoke a menu of all of the query screens for the application
Office	Invoke the SB+ office automation tools, or equivalent
Help	Display a full page of help for the screen

Many of these options invoke menus. In this case, the upper-left corner of the menu box should appear right below the first character of the option.

The following sections describe how these options can be implemented.

Implementing an Action Bar "File" Option
Implementing an Action Bar "Toolbox" Option
Implementing an Action Bar "Query" Option
Implementing an Action Bar "Office" Option
Implementing an Action Bar "Help" Option

Highlighting an Action Bar Letter

When an **action bar** is defined, SB+ allows you to highlight a letter in the label of each action bar option. These highlighted letters then serve as shortcut keys to help users select each option more quickly.

Without highlighted letters, a user can press F10 to invoke the action bar (assuming F10 is either calling the ACTION process or is undefined), but must then use the arrow keys to move right or left before pressing <cr> to select an option. With the highlighted letters, however, once the action bar is displayed the user can simply press the highlighted letter of the option of choice to select it in a single keystroke.

To highlight a letter in an action bar label, simply type the ampersand (&) character in front of the character to be highlighted. As an example, if your action bar contains a "Help" option and the "H" is to be highlighted, the label for this option should be entered as "&Help". If the "U" in "Query" should be highlighted, the label should be entered as "Q&uery".

One thing to keep in mind: SB+ will allow two or more options to have the same highlighted letter. When this happens, you may select only the first option by pressing the highlighted letter. Though SB+ may highlight the second instance of the letter, it will ignore it when you're selecting from the action bar.

Skipping a Prompt

Depending on the complexity of your application, you may find times when it is appropriate to skip a field on the screen under certain conditions. This feature can be accomplished a variety of different ways, depending on the characteristics of the prompt being skipped. The following sections describe several popular skipping techniques.

Using a Non-Amendable Field
Using the Process Before Field
Skipping and Clearing an Input Prompt
Can Multiple Prompts be Skipped At Once?

Implementing a Conditional Popup Subscreen

By definition, a conditional popup subscreen is a standard popup subscreen, but is called based on a condition. This is similar to using the SUBSCREEN process discussed earlier, with the exception that this technique applies to popup, rather than manual, subscreens.

To see how this can be used, assume we're creating a customer entry screen that appears as follows:

```
Customer ID xxxxx
Name       xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Address    xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
City       xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
State      xx
ZIP Code   xxxxxxxxxx

Does the customer have a separate business address? x
```

Notice the question at the bottom of the screen. If the user enters a "Y" at this prompt, we'll want to display a separate screen for the entry of the business address.

To implement this, we must first create both the main screen (as illustrated above) and the subscreen. Once these screens are functional, the next step is to create a process to be called from the main screen to invoke the subscreen. Assuming the field name for the bottom screen question is called W.SEP.BUS.ADDR, we can create a paragraph as follows:

```
IF (W.SEP.BUS.ADDR = 'Y') THEN
  EXEC 'I*CUSTOMER*ENTRY.B'
END
```

Once created, we can link this paragraph to the Process After slot on the F5-Field screen for this prompt.

This is, of course, just one way to implement this type of subscreen. Literally anything could be done by a process called from the Process After slot on a field, not just simple prompting as is illustrated here.

Implementing a Multi-Part Key

With certain files, it may be necessary to have a key field that has several different components. As an example, if we're implementing an inventory file with stock balances per warehouse, we need a record for each inventory item for each warehouse. In this example, we need a key for the inventory file that appears as follows:

stocknumber*warehouse

Given this format, if the stock number is "210" and the warehouse is "4", the key to the inventory record would be:

210*4

When implementing an entry screen for this inventory file, we could certainly prompt the user to enter the entire key in one entry. The problem with this, however, is two-fold:

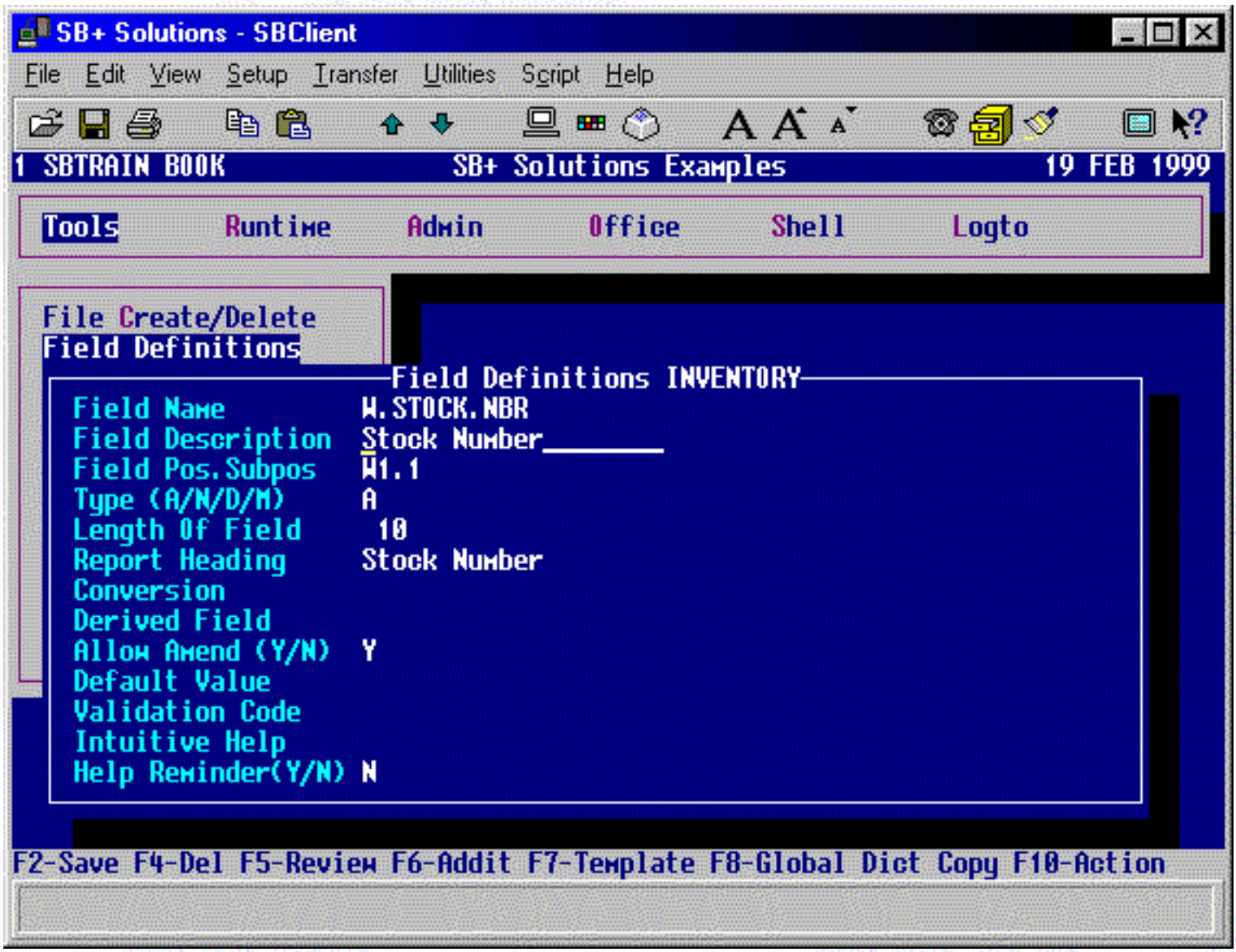
- The entry is prone to errors, and would require significant **validation** to ensure the user entered both parts and both are valid.
- It would be very difficult to provide **F3-Intuitive Help** for the prompt.

A much better approach is to prompt for the stock number and warehouse separately, and then use these values to build a key behind the scenes. With separate prompts, validation can be done quickly and easily, and intuitive help can be constructed for each prompt.

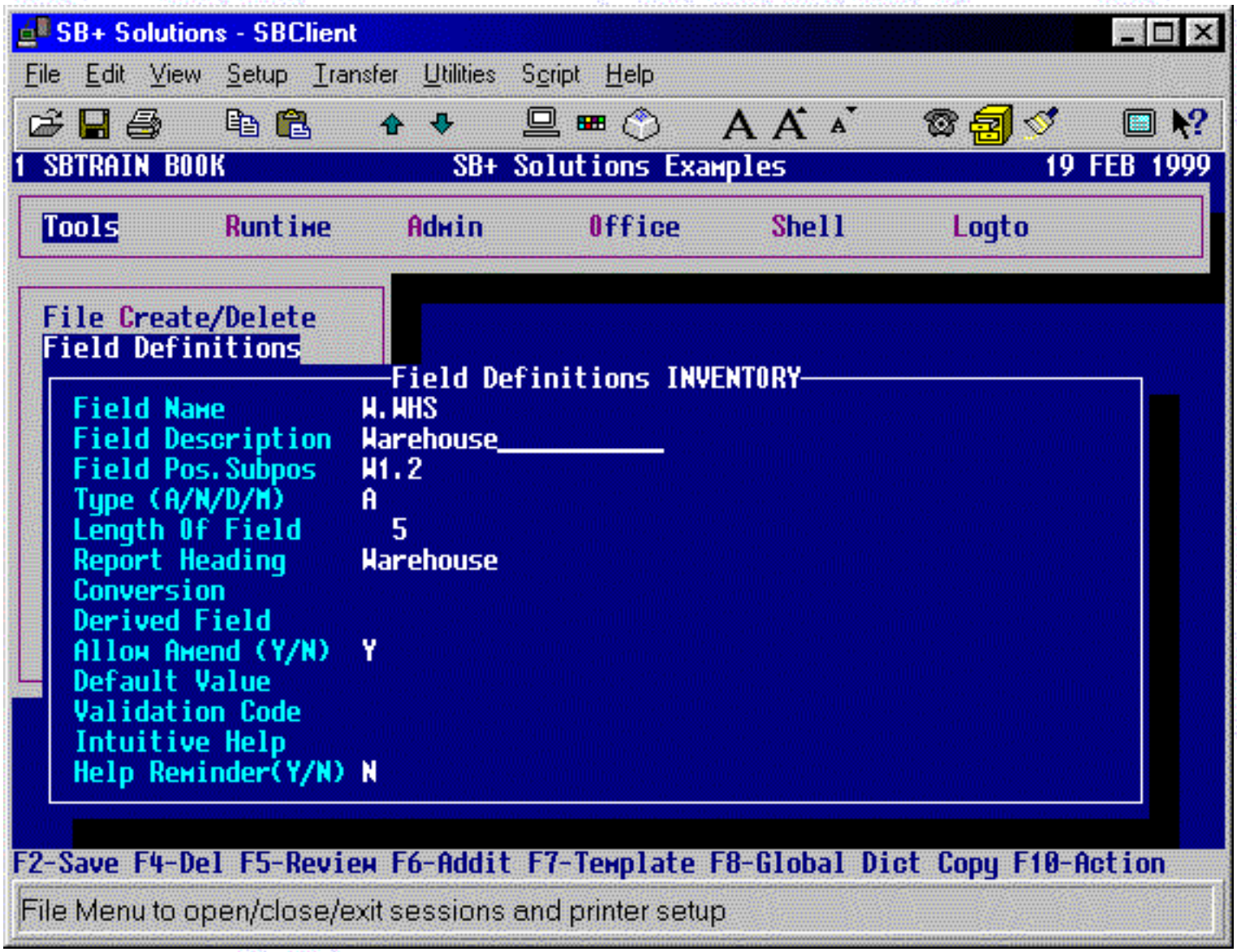
To implement this particular example, we need to accomplish the following tasks:

- Work fields need to be created for each part of the key field.
- The work fields need to be placed on the screen definition.
- A process needs to be called to concatenate the work fields together into a key value.

For this example, the work fields are called W.STOCK.NBR and W.WHS, and appear as follows:



And for the warehouse field...



Note the field position for both of these fields. The preceding "W" tells SB+ that this is a work field. More specifically, this tells SB+ that this field name references the **common variable** @WORK instead of @RECORD.

Field position "1.1" and "1.2" are very significant to this technique. The first part of the key field should be in @WORK attribute 1, value 1. The second part should be in @WORK attribute 1, value 2. For keys with more than two parts, this pattern should be followed for all parts of the key (in sequence). This standard then allows us to create one process (shown below) to build the key that can be used on all multi-part key screens.

To place the work fields on the screen, simply use the F5-Field key in the Screen Definitions tool to place each field where you want it. The prompting order of the fields is insignificant, though each of the work fields should be set up as a mandatory entry on the F5-Field screen.

Finally, to construct a key, two things must be done. First, we need to tell SB+ when to read the record. Normally, with a single part key, SB+ can figure this out. However, with a multi-part key, SB+ has no idea where the record should be read during the process of running the screen. Therefore, on the F5-Field screen for the last field used to construct the key field (W.WHS in this example), place the following on the Control/Dep/Read field:

R

This dinky little code tells SB+ to read a record following the entry of this field. Problem is, we've just entered information into work fields -- the key hasn't been set yet!

To correct this situation, we can add a Process After on the last field used to construct the key (W.WHS in this example) to build the key using the work fields. For this task, I've created a standard process called BUILD.KEY -- a paragraph that appears as follows:

```
@KEY = ''
WHILE @PARAM DO
  @KEY = @WORK<1,@PARAM> : IF(@KEY # '','*':@KEY,'')
  @PARAM = @PARAM - 1
REPEAT
```

When this process is called, a parameter must be specified to tell the process how many parts are to be used in the construction of the key. Knowing this, the process loops from back to front, constructing the key using the information in the various values of @WORK<1>.

With this in mind, for our example we need to call up the F5-Field screen, move the cursor to the Process After Field, and enter the following:

```
BUILD.KEY,2
```

Note that BUILD.KEY universally uses asterisks as delimiters between the various parts of the key field. If a different delimiter were required, you could adapt BUILD.KEY to use the following variation which parameterizes the delimiter:

```
LOCAL DELIM
*
@PARAM = FIELD(@PARAM,',',1)
DELIM = FIELD(@PARAM,',',2)
*
IF DELIM = '' THEN DELIM = '*'
*
@KEY = ''
WHILE @PARAM DO
  @KEY = @WORK<1,@PARAM> : IF(@KEY # '',DELIM:@KEY,'')
  @PARAM = @PARAM - 1
REPEAT
```

With this variation, two parameters are passed into the routine: 1) the number of key components, and 2) the delimiter to use. (If no delimiter is specified, the asterisk is used by default.) Using this process, if we wanted a percent-sign delimiter, the following could be entered as the Process After on the last field used to construct the key:

```
BUILD.KEY,2,%
```

Clearing a Multi-Part Key

On a typical screen with a single part key, SB+ takes the responsibility for clearing the key value when a record is saved or abandoned. With a multi-part key, however, SB+ clears the key, but doesn't clear the work fields used to construct the key. This causes the old key values to be displayed even after the record is saved or abandoned.

Different people view this different ways. To some, the fact that the work fields aren't cleared serves to provide default values for the next entry. Other people, however, view this as a problem, and want the values cleared. This section is for that latter group.

When a record is saved, regardless of the number of parts in the key, the cursor will always be returned to the first prompt on the screen, which is generally the first prompt of the multi-part key. When a record is abandoned, however, the cursor will be returned to the last prompt used for the multi-part key. If the user presses the Escape key on the last prompt used for the multi-part key, the cursor will be moved to the first prompt on the screen, regardless of the number of prompts used for the key.

This particular nuance of SB+ is important enough to say again: If the user presses the Escape key on the last prompt used for the key, the cursor will be moved to the first prompt on the screen, regardless of the number of prompts used for the key. Therefore, if there are four prompts used for the construction of the key and the user presses the Escape key on the fourth prompt, the cursor will be moved to the first prompt -- not the third.

With this in mind we can be confident that there are primarily two places in the screen definition where we need to clear the fields used for the multi-part key. On the first prompt used for the key we should call a process to clear all fields used for the key, and on the last prompt we should call a process to clear that field only. We could also add processes to clear each part of the key when the user presses the up arrow to move from field to field, which isn't a bad idea.

Following the example from the previous section, we can construct a paragraph to be called from the Process Before slot on the F5-Field screen for the W.STOCK.NBR field which appears as follows:

```
W.STOCK.NBR = ""
W.WHS       = ""
@REFRESH    = 2
```

Note the @REFRESH = 2. This tells SB+ to refresh all of the data values on the screen, which will clear both the W.STOCK.NBR and W.WHS fields on the display.

For the W.WHS field, we can construct a paragraph to be called from the Process Before slot on the F5-Field screen as follows:

```
W.WHS = ""
```

Partially Prompted Multi-Part Keys

When using multi-part keys, all of the key parts don't need to be prompted, in contrast to the technique described earlier. Instead, any of the following are possible:

- All key parts can be prompted,
- Some of the key parts can be prompted and others taken from common variables,
- All of the key parts can be taken from common variables.

To illustrate a screen where only some of the key parts are prompted, consider we're creating a screen to enter general ledger information. The information entered into this screen will be stored in a file called LEDGER, with a key that is constructed of four parts:

Company Code
Cost or Profit Center
Account Number
Sub-Account Number

The screen that will be used to enter information into this file will be accessed by several different companies. However, we don't want one company viewing or changing another company's information, so we cannot prompt for the company code. Instead, we'll assume the company code has been loaded by an earlier process into @PARMS(1)<4>. Knowing this, we can construct a screen with key prompts such as the following:

```
Center      xxx
Account     xxxxx
Sub-Account xx
```

In this example, the Cost/Profit center prompt should update @WORK<1,2> (Field Pos.Sub Pos = W1.2), Account should update @WORK<1,3> (Field Pos.Sub Pos = W1.3), and Sub-Account should update @WORK<1,4> (Field Pos.Sub Pos = W1.4). This will ensure we can use the BUILD.KEY process as shown a few pages back.

However, BUILD.KEY requires @WORK<1,1> to be set to the first part of the key, which is stored in @PARMS(1)<4>. We need to move @PARMS(1)<4> to @WORK<1,1> somewhere prior to the BUILD.KEY call. Fortunately, there is a wealth of locations where we can do this:

- Process Before on the input process definition (/PD.I)
- Process Before Screen on the screen definition (/SD, F6-Params)
- Process After Display on the screen definition (/SD, F6-Params)
- Process Before on any prompt used for the multi-part key on the screen definition (/SD, F5-Field)

In any of these process slots, we can call a paragraph to move the value from @PARMS(1)<4> to @WORK<1,1> with the following line:

```
@WORK<1,1> = @PARMS(1)<4>
```

Or, if we have a field definition called W.COMPANY which has a field position of W1.1, we can adapt this paragraph line as follows:

```
W.COMPANY = @PARMS(1)<4>
```

Taking this one step further, if we've defined a global equate for @PARMS(1)<4> called THIS.COMPANY, we can make this paragraph even more understandable:

```
W.COMPANY = THIS.COMPANY
```

Using this technique, you can create nearly infinitely complex key structures. However, note that overuse of multi-part keys leads to poor file design, and poor file design leads to a poorly functioning application. And nobody wants that!

Implementing a Sequentially Assigned Key

With certain applications, it can be useful to have the computer assign the record keys, rather than having the user assign each key value. Journals, customer IDs, order IDs, etc. can all utilize this feature. Using accumulators, an SB+ application can sequentially assign keys in a variety of different ways.

Using Accumulators
Sequentially Assigning a Key as a Default
Sequentially Assigning a Key Prior to Saving the Record With Record Lock
Sequentially Assigning a Key Prior to Saving the Record Without Record Lock

Verifying a Record Can Be Edited

In many applications, once a record has been posted it is no longer available to be edited. This functionality can easily be implemented with a process called from the Process After Read slot on a screen definition.

For example, if we have an adjustment entry screen, and each adjustment record has a field called POST.DT, we can reject editing of a record where this field is non-null with the following paragraph lines:

```
IF (POST.DT # '') THEN
  RELEASE 0,@KEY
  ERROR "This record is posted"
  EXIT 1
END
```

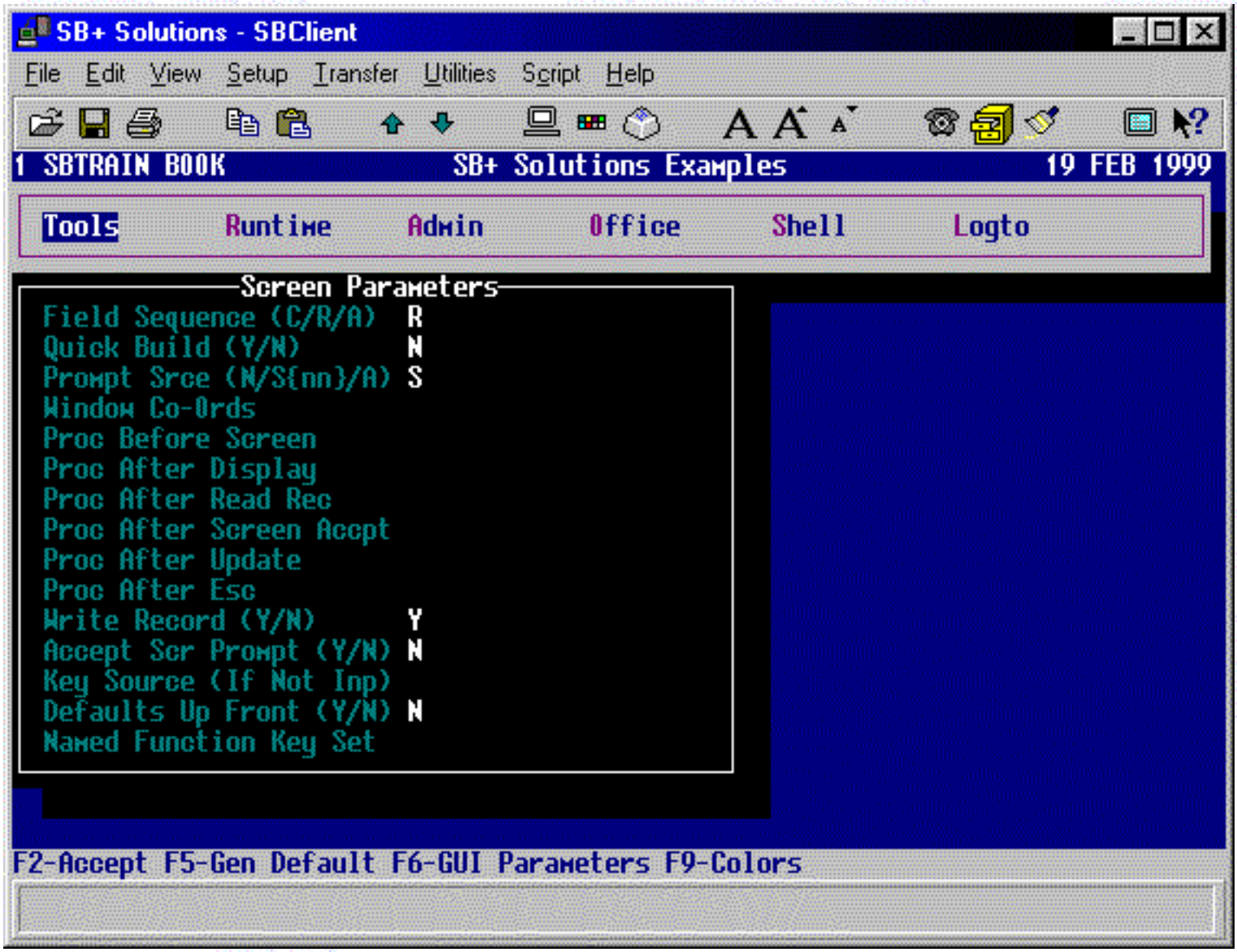
If this process (called in the Process After Read slot) sets @RTN.FLAG to 1, the cursor will return to the key field without displaying anything from the record. Note that SB+ does nothing to unlock the record. For this reason, you must **release the record lock** yourself prior to exiting the paragraph.

This is, of course, a very small example of how this technique can be used. With the flexibility of the paragraph language, the condition can be as simple or as complex as is required for the particular application. Regardless of the complexity, however, always remember these two must-do's:

- If the read is rejected, the process must release the record lock, and
- To reject the read, set @RTN.FLAG to 1 (as is being done using **EXIT** 1 above) before terminating the paragraph.

Using the "Defaults Up Front" Option

In the F6-Params screen on the Screen Definitions tool, there is a prompt labeled "Defaults Up Front (Y/N)" (second prompt from the bottom):



Normally, this is set to "N", which tells SB+ to calculate each field default when the cursor arrives at the field. Sometimes, however, it can be useful to have all of the defaults calculated immediately after the key has been entered. Setting this flag to "Y" will do this.

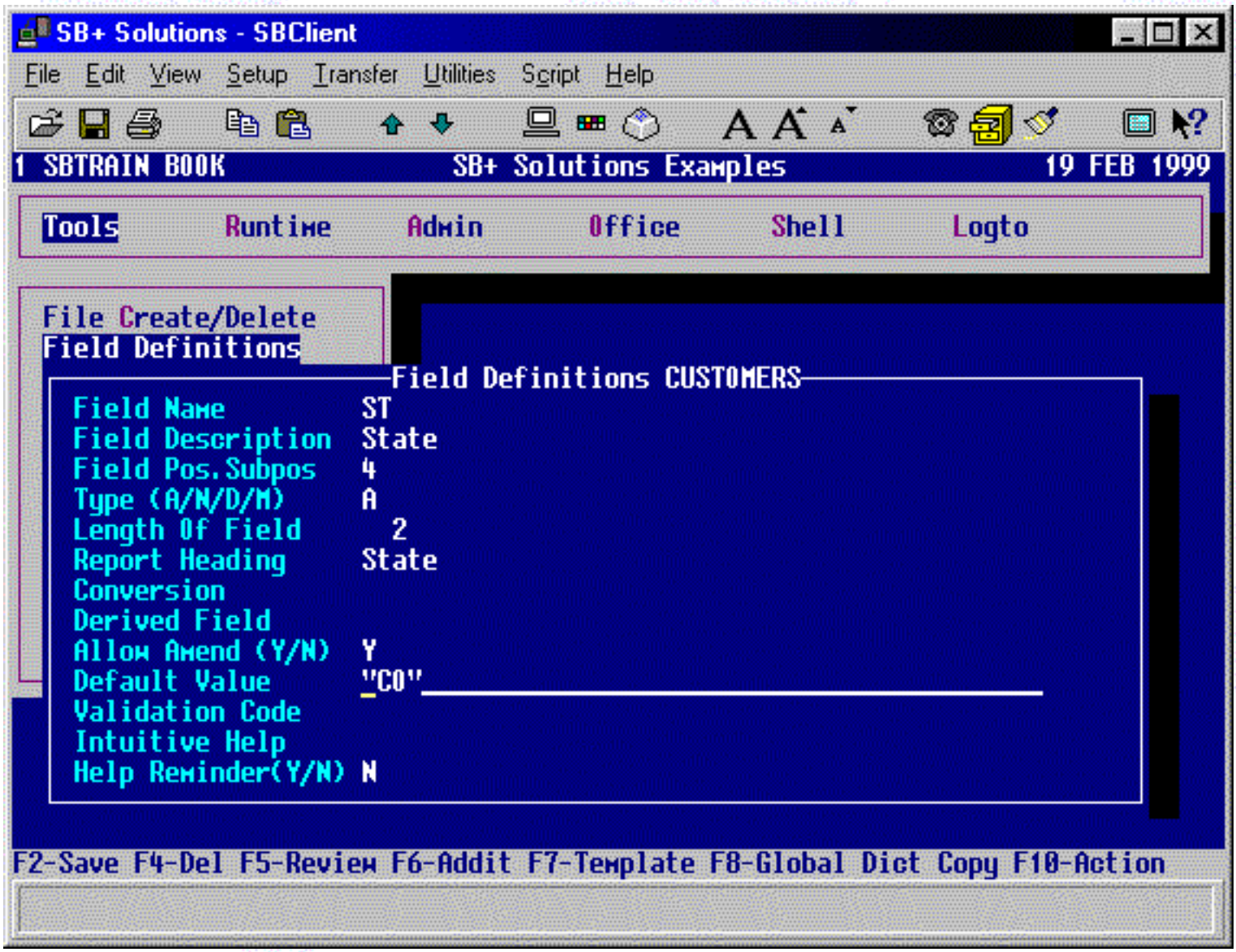
When this flag is set to "Y", SB+ will cycle through all of the fields on the screen definition immediately after the record has been read, processing and displaying the defaults for each field according to the following rules:

- If a field has a default and no value, the default will be processed.
- If a field has a default and also has a value, the default will not be processed.
- If the field has a mandatory default, the default will be processed, regardless of whether the field has a value.

This setting should be used only when the screen definition contains simple default expressions, such as literal defaults or simple calculations. If a screen has defaults that are based on values entered into the screen, calculating all defaults up front will probably not work as expected.

Creating a Literal Default

Literal defaults, such as "WA" or 75, are the easiest types of defaults to enter. If the default value is purely numeric, simply enter the number as the default expression in the Default Value prompt. If the default value is anything else, enter the value surrounded by quotes.



This type of default, though easy to create, is not very flexible. Therefore, you may want to consider loading your defaults from **common variables** or parameter records. The following sections describe these techniques.

Setting a Default from a Common Variable

Any SB+ **common variable** may be used for calculating a default for a field. For example, if you have a default for a field in the third attribute of @PARMS(2), the following is a perfectly suitable default expression for the field:

```
@PARMS(2)<3>
```

But what if there are several different defaults, depending on some condition? For example, let's say that if the record is new (@ACTION = 1), the default should be loaded from @PARMS(2) attribute 3. However, if the record is being amended (@ACTION = 2), the default should be loaded from @PARMS(2) attribute 6. To accomplish this, we can use the following **expression**:

```
IF(@ACTION=1,@PARMS(2)<2>,@PARMS(2)<6>)
```

Or, looking at this same expression a different way:

```
@PARMS(2)<IF(@ACTION=1,2,6)>
```

While any common variable can be used for setting a default, there are some variables which just don't make sense in this context. For example, you'd never want to use @F.FILE for calculating a default, as this would reference an open file buffer and cause your program (and SB+) to crash! In short, don't reference a common variable unless you're absolutely certain of its contents!

There is an incredible amount of flexibility in this technique, as long as your defaults are where you want them. In the next section we'll take a look at how these are set.

Reading a Default from a Parameter Record

Rather than setting a default to a fixed literal value, it may be more appropriate for the default to be read from a parameter record stored in another file in the account. This way, the parameter record can be changed (thereby minimizing impact on the software) if the default is to change.

As an example, let's assume we're creating a product to be used by several different companies on different machines. In this product we have a customer entry screen which has a prompt called Status. One company that uses this screen may want status codes of "A" for "Active" or "I" for "Inactive", while another company may want to use numeric codes such as "1" and "2". Depending on the number of different customers who use the screen, there could be any number of combinations that the software should support.

Hard-coding the status code default is certainly not a good approach in this situation, as this would require each copy of the software to be customized for each client. As more clients use the software, more modifications may be required, which ultimately translates directly to a need for more product support. Besides, the more customized your clients become, the less likely they will be able to accept any upgrades to your product. Without upgrades, there's little to keep your client from taking their future business elsewhere.

With a parameter record, the default can be changed easily without customizations for each customer. This results in one version of the software for everyone, though each copy can be modified through different parameter settings. Of course, your software will require an additional program or series of programs to allow the parameters to be changed, but the extra programs are a small price to pay for the flexibility this technique affords.

There are two techniques that can be used for setting defaults from information in a parameter record. One technique uses very little memory, but isn't very efficient. The other technique is very efficient, but it uses more memory. It's up to you to find the balance of the two that makes the most sense in your own development.

Using an On-Demand Parameter Record

An on-demand parameter record is a record that is stored in a file somewhere, but is not accessed until needed. This technique uses little memory, but isn't very efficient.

For the customer screen discussed earlier, let's assume we have a file called XXXCONTROL which holds the parameters for the software. (The name of this file is no accident; I typically put parameters in the control file for whatever SB+ system is being used. Therefore, for a system called CPS, the parameters would be stored in the CPSCONTROL file.)

For the sake of this discussion, let's assume we have a record in this control file called CONFIG which holds the parameters for our software. Let's also assume that attribute 9 of this record holds a status code default parameter.

Knowing this, the default expression for the status code prompt can be entered as:

```
F('XXXCONTROL','CONFIG')<9>
```

This is an SB+ **expression** which states "read a record from the XXXCONTROL file using 'CONFIG' as the key and return attribute 9 from that record. Therefore, each time the default is needed, the record will be read and the proper attribute extracted for the default.

Though this is very short and to-the-point, it is not very efficient. Because the expression will read the parameter record each time the default is needed, there is additional disk I/O which can make the application run slower. For low impact screens this may be an insignificant issue. However, for high impact, high profile screens where efficiency is paramount, a different approach should be considered. Such a technique is described next...

Preloading a Parameter Record

In the process definition record for the input process (which can be viewed or edited using /PD.I) there is a prompt labeled "Process Before" which can be used to load parameters into common memory where later default expressions, validations, and intuitive help processes can access the information. This technique is more efficient than the previous one (i.e. uses less disk I/O), but requires more memory.

SB+ has a 40-position dimensioned array in memory called @PARMS(...) that can be used to hold parameters while a screen is running. By loading parameters once and holding onto them, disk I/O is reduced and the software runs more efficiently.

Using the customer screen example, we can create a single line paragraph to load the status default, as follows:

```
@PARMS(3)<1> = F('XXXCONTROL','CONFIG')<9>
```

By linking this process to the Process Before as described above, our status default will be loaded at the start of the input process. As long as we don't change the contents of the first attribute of @PARMS(3), we can rest assured our default will be there when we need it.

Following this example, our default for the status field then becomes:

```
@PARMS(3)<1>
```

instead of the F(...) function used earlier.

One of the highlights of this technique is also one of the weaknesses of it: SB+ never touches this **common variable**. It is completely programmer-defined, so there is nothing in SB+ that will change, preserve, or restore the contents of this variable as you move from screen to screen in your application. Therefore, each application must be cognizant of the @PARMS(...) usage of any applications that may interface with it and work to avoid collisions where one application uses a particular @PARMS(...) element differently. (Or, the application will need to use the @USERDATA(...) variable, which has different rules than @PARMS. See **Chapter 8** for more information about common variables like @PARMS(...) and @USERDATA(...).)

For applications which use @PARMS(...) extensively, it is unrealistic to believe all uses of @PARMS(...) can be rationalized across applications that may be linked together. Frankly, any program in an application should be able to interface to any other program in the application without having to worry about clashes in the common map. Though SB+ does not support this functionality directly, there is a small process you can create which will help you do this.

This process is called NEWMEM and is a BASIC process. To create this process, create a BASIC subroutine called XXX.NEWMEM in your XXXPROGS file that reads as follows: (remember to replace the XXX with your own system ID):

```
SUBROUTINE II.NEWMEM
*
$INCLUDE DMSKELCODE COMMON
*
```

```
EQU PUSH TO 0,POP TO 1
DIM CURR.COMVARS(52),HOLD.PARMS(40)
*
PROCESS.ID = PARAM
HOLD.BC = BOX.CORDS
BOXCORDS = ''
CALL SB.STACK.SCREEN(PUSH,BOXCORDS,'')
CALL SB.PRESERVE.COM(PUSH,MAT CURR.COMVARS)
MAT HOLD.PARMS = MAT PARMS
*
CALL SB.PROCESS(PROCESS.ID)
*
BOXCORDS = HOLD.BC
CALL SB.STACK.SCREEN(POP,BOXCORDS,'')
CALL SB.PRESERVE.COM(POP,MAT CURR.COMVARS)
MAT PARMS = MAT HOLD.PARMS
*
RETURN
```

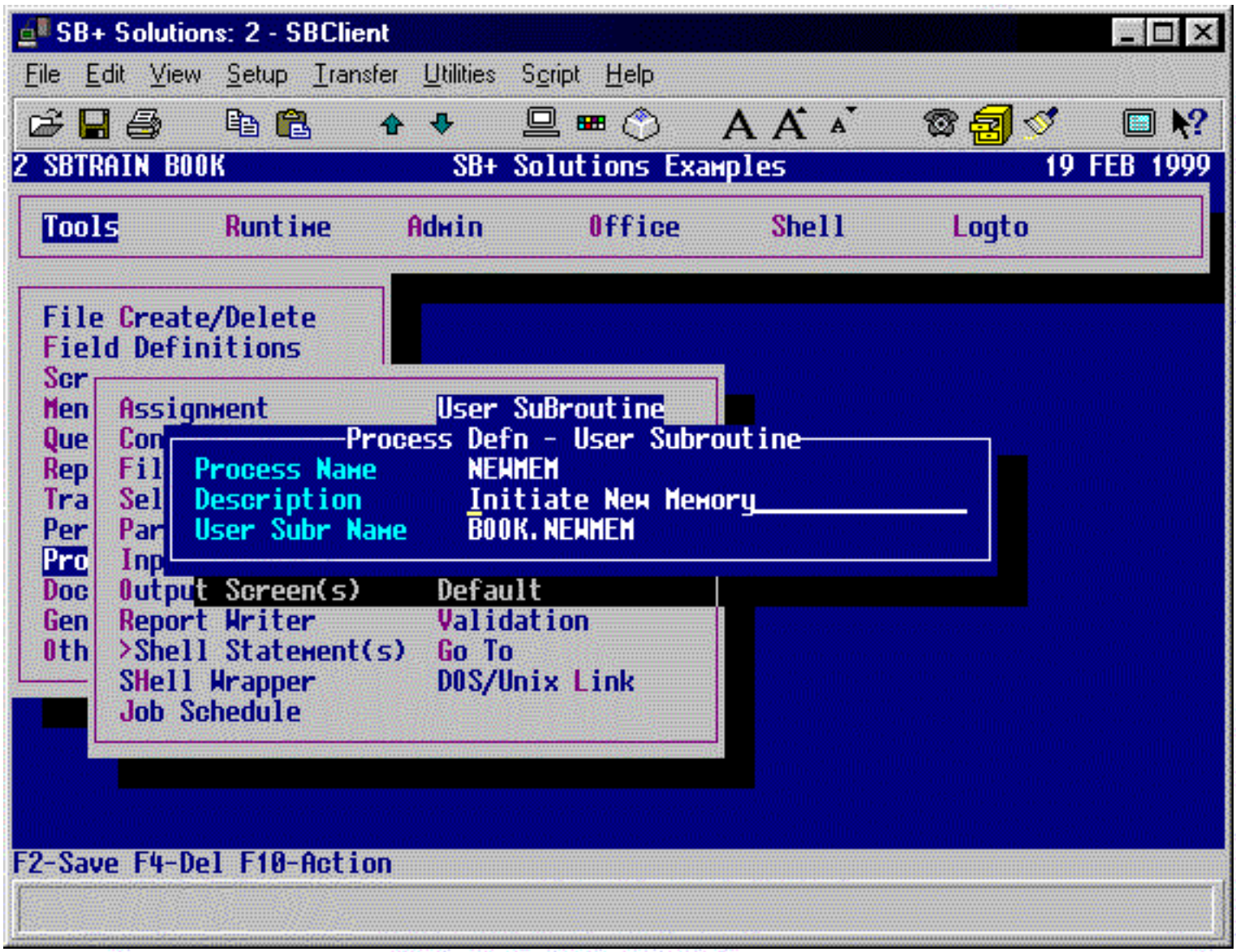
NEWMEM mimics the way SB+ handles common variables when one input process calls another input process. Specifically, both SB+ and NEWMEM preserve common variables when an input process is called and restore the original values when the called process terminates. Unlike typical SB+, however, NEWMEM also preserves and restores the values in @PARMS(...), thereby allowing each process to have its own autonomous parameter records loaded.

On lines 8 and 13 of this subroutine, note that the name of the process to be called is passed into this routine in the common variable @PARAM. Therefore, the syntax for calling NEWMEM requires a parameter listing the name of the actual process to be called, such as in these examples:

Function Key: NEWMEM,processname
Paragraph: EXEC 'NEWMEM,processname'
Expression: P('NEWMEM,processname')

In each of these examples, the process NEWMEM will be called. Because this is a **BASIC** process, the subroutine XXX.NEWMEM will be invoked. This subroutine will then preserve the current values of the common variables, call the process as named in processname, and then restore the original values for the common variables when the called process terminates. In the end, then, the parameters that have been set up for a given process will be preserved before another process is called and restored upon the return, ensuring autonomous common values for each process..

Once this subroutine has been created, compiled, and cataloged, create the BASIC process (using the /PD.B tool) as follows:



Note that this process associates the name NEWMEM with the subroutine DEMO.NEWMEM. After this BASIC process has been entered and saved, the NEWMEM process is fully installed and ready to be used in your application.

Setting Defaults Under Multiple Conditions

In certain cases there may be several conditions to check in order to calculate the default value for a field. One possible option to do this is with the CASE(...) function in the default expression.

The CASE(...) function allows you to test multiple conditions inside of an expression, returning a result based on which condition is true. To its credit, this is a very powerful function. Unfortunately, however, it has a complex and confusing syntax, and is therefore difficult to remember and use.

The syntax for this function appears as follows:

```
CASE(condition,value;condition,value;...)
```

In this sample, the ellipses signify that any number of conditions may be tested. The first condition that is true will end the tests, returning the value that is associated to the true condition.

Let's say we have a customer screen in our application with two fields called Customer Type and Credit Limit. The Customer Type field may be "C" for corporate customers, "P" for partnerships, or "S" for sole proprietorships. Depending on the customer type, the credit limit will vary. Corporate customers will have a credit limit of $100. Partnerships will have a credit limit of $200. Every other customer type will have a default of $10. To implement a conditional default using these rules, the following expression may be used:

```
CASE(TYPE='C',100;TYPE='P',200;1,10)
```

Notice the "1,10" near the end of the expression. If none of the previous conditions is true, the "1" tells SB+ to assume a true condition and return a value of "10" (i.e. $10) from the CASE(...). This is a convenient way to plan for the "all others" conditions that would otherwise fall through the cracks.

I've not met too many SB+ developers who actually use this construct, mostly because it's too difficult to remember and inflexible. Instead, when multiple conditions are being tested, most people prefer to call a process, such as a paragraph, to calculate the default value.

Calling a Process to Calculate a Default

When multiple conditions are needed to calculate a default, or when many factors must be considered in the calculation, it is best to create a process to do the calculation and return the result. This type of process will generally be a paragraph, but may also be a BASIC subroutine depending on the complexity required.

The goal of any process called from a default expression is to set the **common variable** @VALUE. Whatever is in this variable when the process terminates will be the default value for the field.

In the default expression, then, the following syntax is used to call the process:

P('*processname*,*p1*,*p2*,...')

In this example, *processname* is the name of a process to be called. The optional parameters *p1* and *p2* (and any others as represented by the ellipses) are optional and will be passed to the process in the common variable @PARAM.

Let's say we have a customer screen in our application with two fields called Customer Type and Credit Limit. The Customer Type field may be "C" for Corporate customers, "P" for Partnerships, or "S" for Sole Proprietorships. Depending on the customer type, the credit limit will vary. Corporate customers will have a credit limit of $100. Partnerships will have a credit limit of $200. Every other customer type will have a default of $10. The following paragraph may be called to calculate the credit limit default based on these rules:

```
CASE TYPE = 'C'
  @VALUE = 100
CASE TYPE = 'P'
  @VALUE = 200
CASE 1
  @VALUE = 10
END CASE
```

Of course, this small paragraph is just a sample to illustrate the technique. With the strength and flexibility of the paragraph language, there is virtually no limit to the complexity of default that you can calculate. Whatever you do, however, always remember these tips:

- The process that is called to calculate the default must return the default in the common variable @VALUE;
- The P(...) function must be used in the default expression;
- Someone else will be reading your code sometime in the future, so work to make your process as understandable as possible.

Selecting From a List of Default Values

In certain situations you may want to let the user select one of several possible default values. Similar to intuitive help, this allows the user the freedom to select one of several values from a list. Unlike intuitive help, however, the selected value may be edited before being updated into the record.

There are two options available when you want to have a list of defaults for the user to select from. These are:

- Select the default from a **menu**
- Select the default from a **selection process**

Each of these techniques has its benefits and side-effects, as described in the following sections.

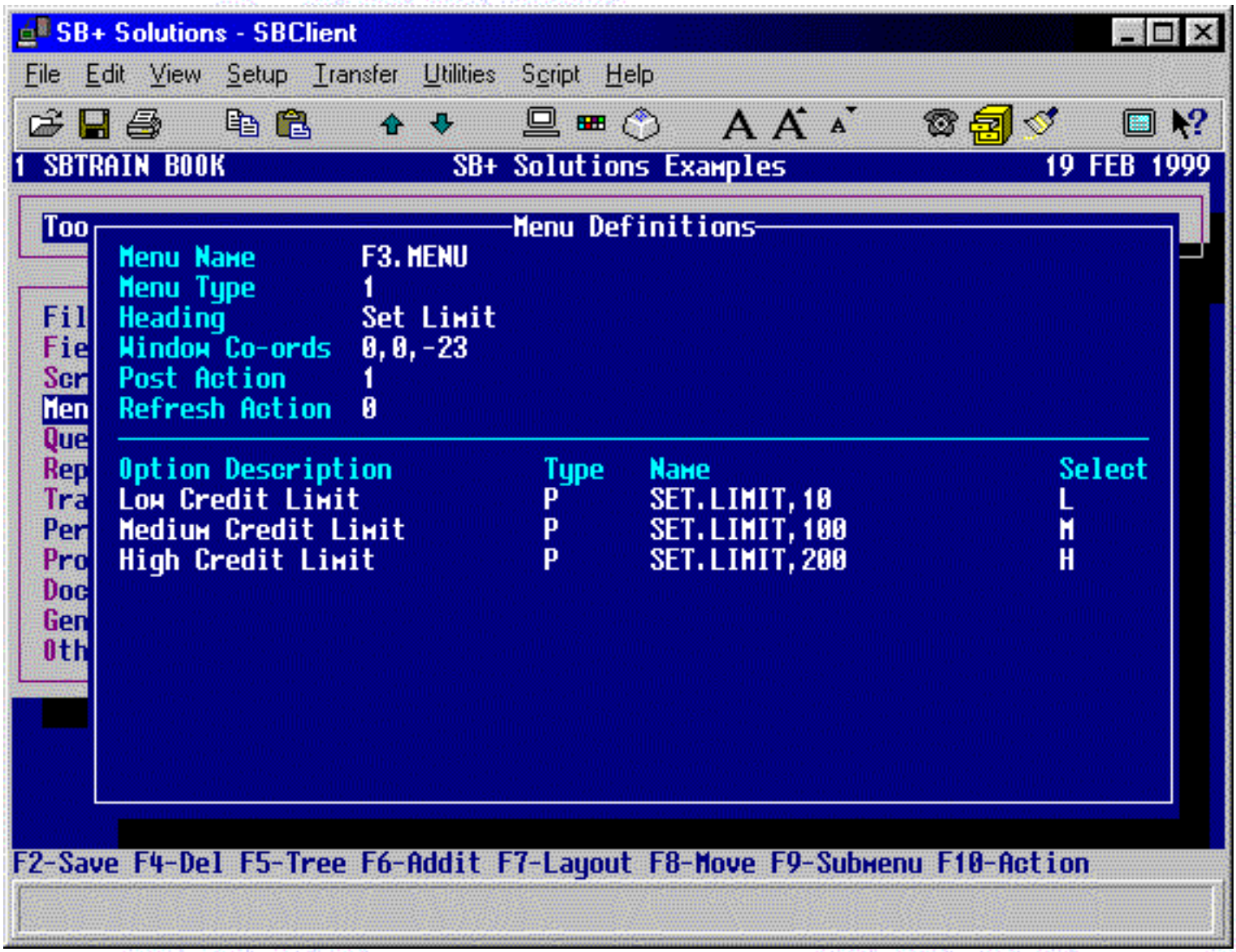
Selecting the Default From a Menu

A menu process can be called for the calculation of the default value, similar to the way a paragraph can be used. Menus are an attractive approach to setting the default this way, as they have highlighted letters for each option to help the user make a selection quickly. Also, because each option on the menu can call a process, the menu approach provides the ultimate flexibility for the calculation of the default value. However, because menus are a part of the software, it is difficult to parameterize the options that should appear on the menu, and changes often require the menu to be customized.

To create a menu to do this, three things must be done:

- Use /MD to create the **menu definition**
- Use /PD.M to create a **menu process** which references the menu definition
- Each option on the menu must set the **common variable** @VALUE to the desired default value.

The menu definition tool (described in detail in **Chapter 7**) appears as follows:



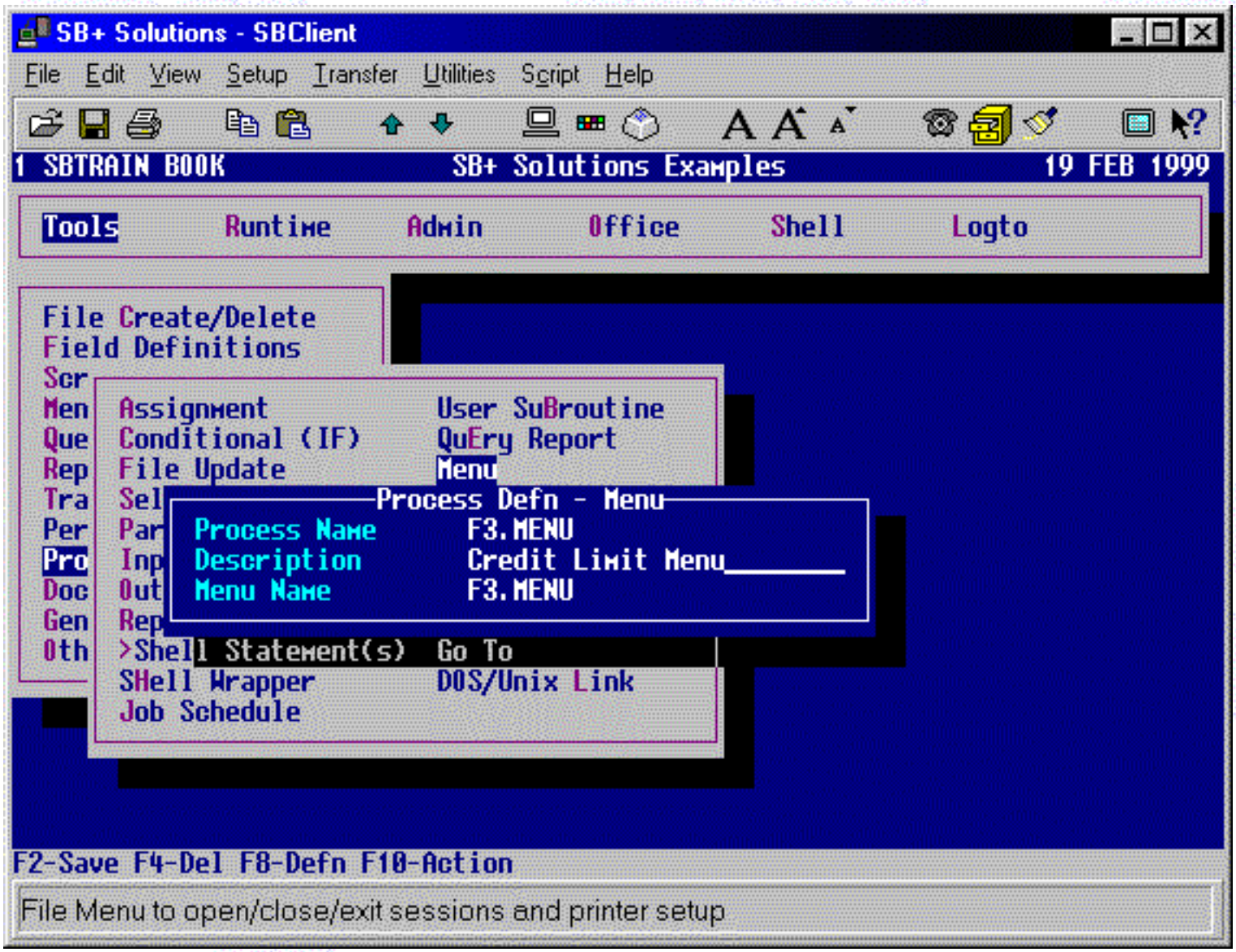
First, note that Post Action is "1". This tells SB+ to display the menu and then exit immediately after an option has been selected. Without this, the menu will continue to be displayed until the user presses the Esc key, which isn't intuitively obvious. With this prompt set to "1", the user will select an option, the menu will disappear, and the default as selected will be displayed on the prompt.

Second, note the process names under the Name field in the lower section of the screen. In these processes, anything can be done, as long as the result is returned in @VALUE. In this case, I've decided to pass the credit limit to a standardized process, which simply sets @VALUE equal to @PARAM, as follows:

```
@VALUE = @PARAM
```

Any process can be called from the Name prompt on this menu for each option. Therefore, any paragraph or BASIC process may be called to calculate a default value for one or more of the options on the menu. Remember, though, the underlying function of each process called here must be simply to set the common variable @VALUE, regardless of any extraneous processing that may be required to derive this value.

Once a menu definition has been created and saved, the next step is to create a process record to reference the menu definition. This is done using /PD.M, and appears as follows:

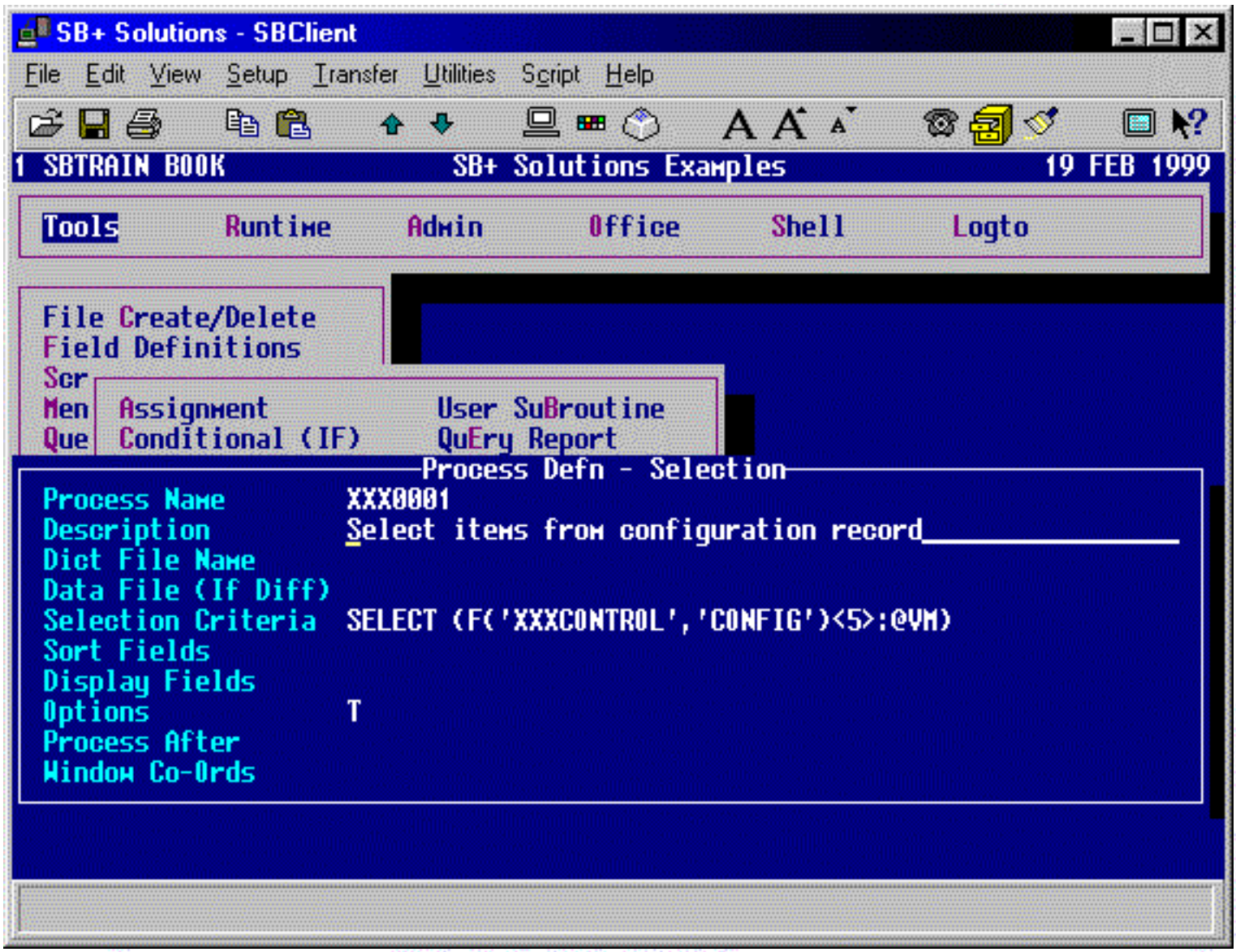


In this example, simply note that a process is created to reference the menu definition. The fact that the name of the menu definition and the name of the menu process are identical is merely a convention.

Selecting the Default from a Selection Process

A selection process can be called to set a default value, similar to the way a paragraph or menu can be used. The selection process is not as attractive as a menu (there are no highlighted letters) and it is more difficult to make the selection process call a process for the calculation of the default based on the option selected. However, selection processes can display a list of values from a parameter record much more easily than menus can.

Selection processes are entered using the Process Definition - Select (/PD.S) tool, which appears as follows:



In this example, the selection process will read a list of values from the XXXCONTROL file, CONFIG item, attribute 5, and will display the items in a selection box. Noting that the purpose of a selection process is to set the common variable @VALUE, whatever option is selected will automatically be returned in the proper common variable.

This particular selection criteria is very useful for a variety of techniques other than this one. In short, this syntax says to build a selection window based on the multivalued result of the expression as shown between the parentheses. An important part of this expression is the :@VM at the end of the expression. Without it the selection process will not work properly if the result of the expression is not multivalued. For more information about this type of selection statement, see the **discussion of selection criteria** in **Chapter 8**.

Using a Mandatory Default

Under normal circumstances, SB+ processes the default expression for a field only when the field has no value. Therefore, if a field has a value and the cursor moves to that field, the default expression will be ignored. Unfortunately, if the default expression calls a process, that process will also be ignored.

If you want the default expression to be evaluated whether the field has a value or not, you must use a *mandatory default*. To do this, simply add [M] to the end of the default expression. Therefore, if the default expression is the literal "100", the default would be stated as:

```
100[M]
```

Or if the default expression calls a process called LIMIT.MV:

```
P('LIMIT.MV')[M]
```

It is important to note that the "[M]" must be outside of any quotes or parentheses in the string, and must appear last in the default expression. Also note that this should be used with caution, as a mandatory default can change a value in a record even if the field is for display only!

Using Skip Defaults

One interesting feature of setting defaults in SB+ is called a *skip default*. This allows you to set a default for a field when the field is null AND skip to the next field on the screen so the user cannot override the default value.

To make a normal default expression a skip default, simply add [S] to the end of the default expression. Like a mandatory default (discussed in the previous section), the [S] must be outside of any quotes. However, unlike the mandatory default, the [S] may appear inside of a set of parentheses, such as when used with the IF(...) function in an expression.

For example, if we have a customer screen in our application with Status and Credit Limit fields. Valid status codes consist of two characters: the first determines whether the credit limit is fixed, and the second determines the actual credit limit. If the first character is "X", then the credit limit is fixed. Any other first letter code means that the credit limit may vary from the default. The second character of the status code, then, determines the default credit limit. If the second code is "A", the credit limit is $1000. If the second code is "B", the credit limit is $2000. If the second code is any other value, the credit limit is $3000.

With this in mind, if we enter a customer with the status code of "XB", that customer would have a fixed credit limit of $2000. Conversely, if we enter a customer with a status code of "AC", that customer will have a default credit limit of $3000, which can be edited by the user.

To implement this, a default **expression** such as the following could be used:

```
IF(STATUS[1,1]='X',P('CR.LIMIT')[S],P('CR.LIMIT'))
```

This expression states "If the first character of the status code is "X", call a process named CR.LIMIT to calculate the value for this field and do not allow entry. Otherwise, call the CR.LIMIT process to calculate the default for this field and allow it to be modified.".

Other than being difficult to read, perhaps you can see that the embedded [S] is not all that obvious in the syntax. Therefore, it's best to avoid skip defaults, favoring regular defaults instead. Skipping is best handled separately, as is explained in "**Skipping a Prompt**" elsewhere in this chapter.

Using a Mandatory Skip Default

Mandatory defaults and skip defaults may be combined in one default **expression**, forming a mandatory skip default. If the default must occur every time the cursor moves to the field but you never want the user to override the default value, this technique can be used.

The most common form of a mandatory skip default is where the entire expression is both mandatory and the field should be skipped unconditionally. This is accomplished simply by adding [S][M] to the end of the default expression. Similar to the [M] for a mandatory default, this must appear at the end of the default expression and cannot be embedded inside of any quotes or parentheses.

A mandatory skip default may also be constructed where the entire default expression is mandatory, but the skip applies only under certain conditions. Using the customer screen example from the previous section, we can make this skip default a mandatory skip default by changing:

```
IF(STATUS[1,1]='X',P('CR.LIMIT')[S],P('CR.LIMIT'))
```

to:

```
IF(STATUS[1,1]='X',P('CR.LIMIT')[S],P('CR.LIMIT'))[M]
```

Again, the only difference between the two is the [M] at the end. This makes the entire expression a mandatory default, but the skip applies only when the condition as tested by the IF(...) function is true.

Overview of Validation Codes

Most of the validation codes in SB+ are entered in the format x:info where x is a single letter code, followed by a colon followed by some additional information needed for the particular process code. This makes the validation codes visually distinct and completely different from every other type of code in the SB+ development environment.

Validation codes may be combined with either AND or OR, but not both. This allows limited combination of options, but is not flexible enough for all possible situations. When the validation becomes more complicated than one or two simple conditions, it is always best to call a process to do the validation, which can then do literally anything to validate the entry.

The following are the valid validation codes that SB+ supports:

B:*subrname{,p1,p2...}*	**BASIC**: Call the BASIC subroutine *subrname*. Optionally, parameters *p1*, *p2*, and others (represented by the ellipses) may be passed to the routine in the **common variable** @PARAM. (Braces are shown for clarity and are not a part of the syntax.)
C:*procname{,p1,p2}*	Call: Call the process named *procname*. Optionally, parameters *p1*, *p2*, and others (represented by the ellipses) may be passed to the routine in the common variable @PARAM. (Braces are shown for clarity and are not a part of the syntax.)
D:	Data File: Verify entry is a data file.
E:*exprn*	**Expression**: If the result of the expression *exprn* is non-zero, validation passes. If the expression returns zero, the validation fails. For example, to validate that an entry is greater than the literal "5", the following could be used: E:(@VALUE > 5) (I prefer to enclose the expression in parentheses for clarity. This is just a convention, not a syntax requirement.)
F:*file,dispfield,flag*	**File Read**: Use @VALUE as the key to read a record from *file*. If *flag* is "Y", the record must exist for the validation to pass. If *flag* is "A", the record must exist (like "Y"), but null is considered a valid entry. If *flag* is "N", the record must not exist for the validation to pass. If *dispfield* is entered, it names the field to be displayed on the bottom left corner of the screen from the record as read. This form is typically abbreviated F:file which assumes no display field and *flag* = "Y".
I:	Integer: Value must be an integer.
L:*length*	**Length**: Value must be exactly *length* characters long.
M:	Mandatory: A value, if even a space, must be entered. Note: This is not the same as a mandatory prompt when the field is placed on a screen. A mandatory prompt on a screen validates that the field has a value both when the cursor is on the field and also when F2 is pressed. This validation code will verify that an entry has been made only when the cursor moves through the field.
P:*pattern*	**Pattern Match**: The value must match *pattern*.
R:*fromval* TO *toval*	**Range**: The value must be between *fromval* and *toval* (inclusive)
S:	No Spaces: The entry must not contain spaces.
U:	**Unique**: Entries in a multivalued field must not be duplicated.

V:*n,m,o,...*	**Valid Values**: Value must be one of the entries in the comma delimited list. Any number of values may be listed following the colon.

Other, non-colon options include:

(exprn)	Same as E:.
"*literal*","*literal*",...	Same as V:

Frankly, because the non-colon validation options are merely syntax variations of other validation codes, you may want to standardize on using the colon-ized validation codes for all validations to minimize confusion.

For many of these codes, SB+ will automatically construct an error message to be displayed when the validation determines an entry is invalid. You will not see this message in the field definition, nor will you be able to change it. You can, however, enter your own message in braces following the list of validation codes. For example, if a customer ID is entered on an order screen, it should be validated to be sure that the customer has a record on file. If such a record does not exist, a message such as "Customer not on file" should be displayed to tell the user of the problem. The following validation code will do this:

F:CUSTOMER{Customer not on file}

But error messages are not limited to simple literal messages such as this. There are many, many options for the display of **error messages**...

Error/Warning Messages

When validating an entry, one of two things could happen. If you want to reject the entry altogether, you'll want to display an error to tell the user why the entry is unacceptable. In contrast, there may be times when you'll want to display a warning, allowing the user to accept the entry despite the message.

When anything but the C: validation code (Call Process) is used, SB+ can only reject the entry and display an error. It can never be used to display a warning. However, when the C: validation code is used, SB+ is not so limited. Processes called for validation can display errors and reject an entry just like any other type of validation code, and can also display warnings, allowing the users to continue past the messages.

As far as the message is concerned, there is nothing in a message that differentiates a warning from an error. Instead, the secret lies in the **common variable** @RTN.FLAG.

When a validation process is called and terminates with @RTN.FLAG = 0, the entry will be accepted. Any number of warning messages may be displayed by the process, but if @RTN.FLAG is zero when the process is done, the entry will be accepted. In contrast, if @RTN.FLAG is 1 when the process terminates, the entry will be rejected. Keep in mind that anything which sets @RTN.FLAG to a 1 will reject the entry, whether the entry should be rejected or not. Therefore, you as the developer must always be cognizant of the value of @RTN.FLAG when developing validation processes.

When reviewing the following sections, keep a close eye on the EXIT 1 statement in the example paragraphs. This statement will exit the paragraph at that point and set @RTN.FLAG to 1, which is the signal to SB+ to reject the entry. This undoubtedly violates the single entrance/single exit paradigm of structured programming, but is the generally accepted technique for the development of processes in SB+ .

The Single Message Dilemma
Errors vs. Warnings
Error Message Formats
Variable Error Messages

Date Validations

SB+ handles date fields very nicely. Literally anything can be entered into a date field, and as long as the entry is not too weird, SB + can generally figure out what you mean. Therefore, to enter the date equivalent of December 25, 2010, any of the following could be entered:

12/25
12.25
25 DEC
DEC 25

Or, if today is the 25th of December, you could simply enter the letter "T" to have SB+ enter the current date. (However, if the current date is Christmas one would hope you're celebrating the holiday, not reading a book about how to use SB+.)

In addition to handling all the different date formats, SB+ automatically validates a date field to ensure a valid date has been entered. Therefore, when a date field is placed on a screen, there is no way SB+ will accept anything but a valid date (or null). You cannot prevent this automatic validation on date fields, but then again, why would you want to?

Only after SB+ has determined the entry is a valid date, does it go through any additional validation as listed in the field definition. The entry that is validated is the internal representation of the date entry (a.k.a. ICONV(...) format), not the actual entry itself. Therefore, you can be assured that any date entry you validate will always be either a number or null.

The following sections describe a few common date validations. This is by no means a complete listing of all that can be done with date validations. Hopefully, though, this will stimulate your own thoughts and creativity on how date validations can be used in your own applications.

Validating a Date Greater Than Today
Validating a Date Range
Validating a Weekday Entry
Validating an End of Month Entry
Calculating Date Format

Alphanumeric Validations

By and large, most of the validation codes you will construct will be on alphanumeric (a.k.a. alpha) fields. Alpha fields can represent text, codes, lists, or combinations of all the above, and therefore are typically the most frequently used fields in any given application.

The following sections describe some typical validations for alpha fields. This is by no means a comprehensive list of everything that can be done with alpha fields, but rather is intended to stimulate ideas regarding how these types of validations can be used effectively in your own programming.

Validating the Length of an Entry
Validating Against a Pattern
Validating Against Another File
Validating Against a List
Validating Against a Table
Validating a Multivalued Entry is Unique

Numeric/Monetary Validations

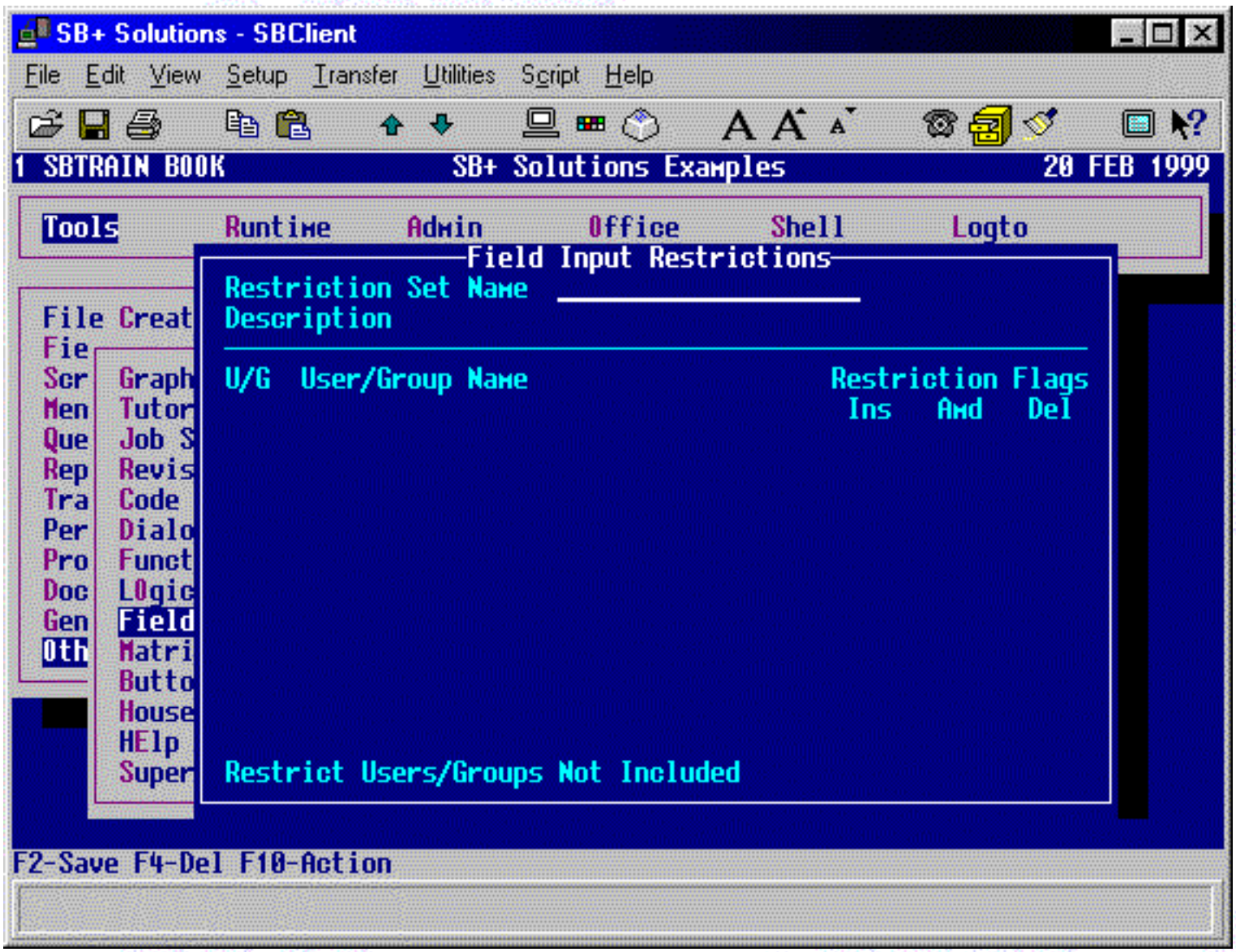
Like dates and alpha fields, numbers sometimes need to be validated. You may need to check that a particular number is greater than another number, verify the number falls within a range, or maybe even check to see that the number is prime. Whatever validation you need, SB+ probably has something that will fit very nicely.

Verifying X Greater Than Y
Validating Against a Range
Validating a Number is Prime

Using Field Input Restrictions

Field Input Restrictions are codes which define which users and groups can insert, amend, or delete values from a particular field. These restrictions are defined using the Field Input Restrict Defs tool. This tool can be started from the menus using Tools, Other Tools/Utilities, Field Input Restrict Defs. Or, for the direct approach, the tool may be started by entering /FIR.DEFN at any input prompt or menu.

When the tool is started, SB+ will ask you to enter the name of a file where the restrictions will be stored. Normally, this is the *XXX*DEFN file (where *XXX* is the current system ID), though you can put these definitions in any file you wish. Once the file name has been entered, the following screen will be displayed:



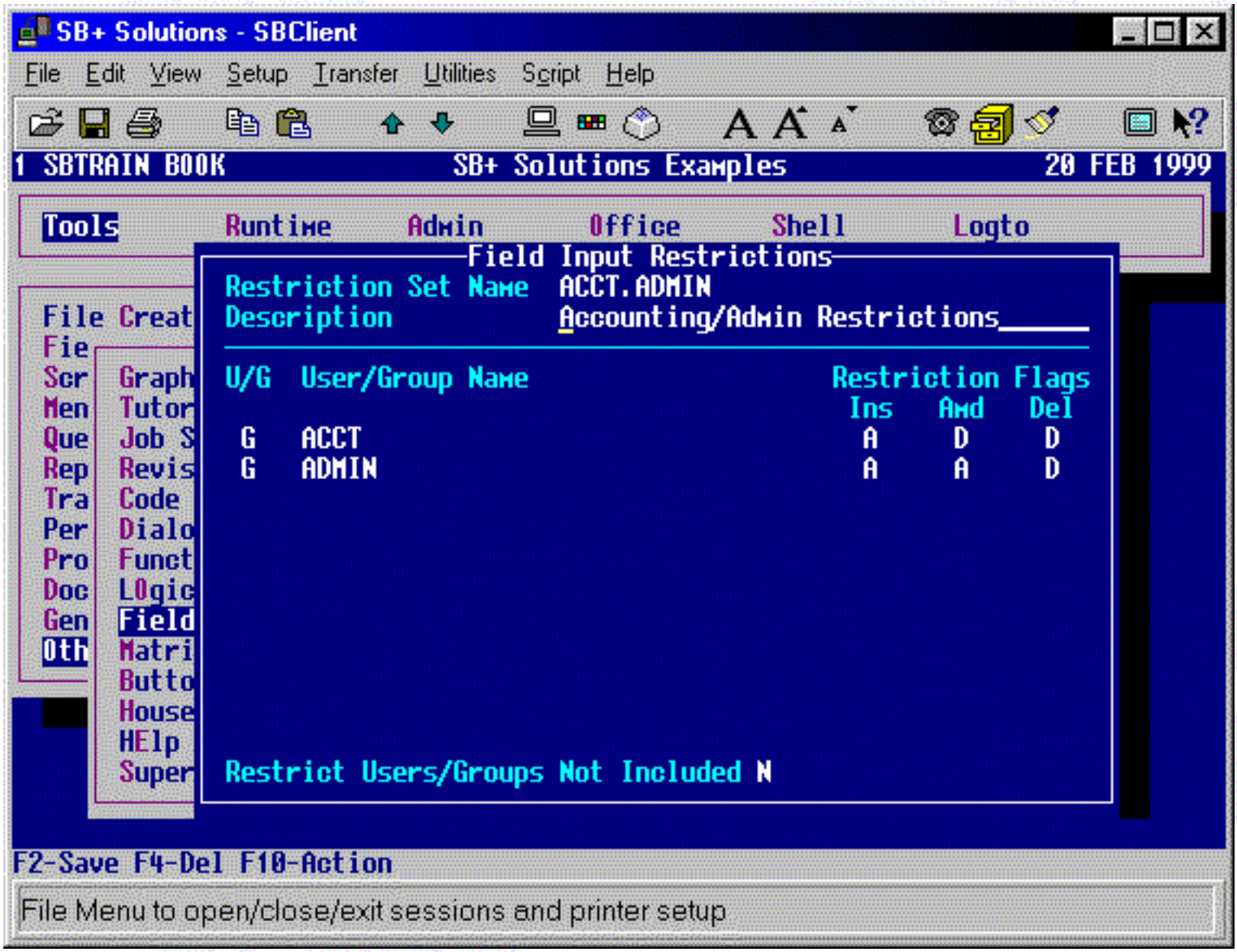
When defining field input restrictions, you must first decide on an appropriate name, such as "RESTRICT.ACCT" (for defining restrictions to accounting personnel). As with all SB+ tools, be sure to enter the restriction set name in upper case only, using whatever naming standards may apply.

Next, enter a description. This description will appear when someone uses F3 on the Restriction Set Name, so use a description

which can be used to help you find your restriction set months into the future.

At the "U/G" prompt, enter a "U" to define a user restriction, or a "G" to define a group restriction. Following this, enter the name of the appropriate user or group to be restricted in the "User/Group Name" field.

In the "Restriction Flags", enter either an "A" to allow or a "D" to disallow inserting new values (changing a value from null to a non-null value), amending existing values (changing an existing value to another value), or deleting values (changing an existing value to null). For example, the following definition allows accounting personnel to add new information, but they cannot amend or delete information. Also, this definition allows administration personnel to insert and amend, but they cannot delete information



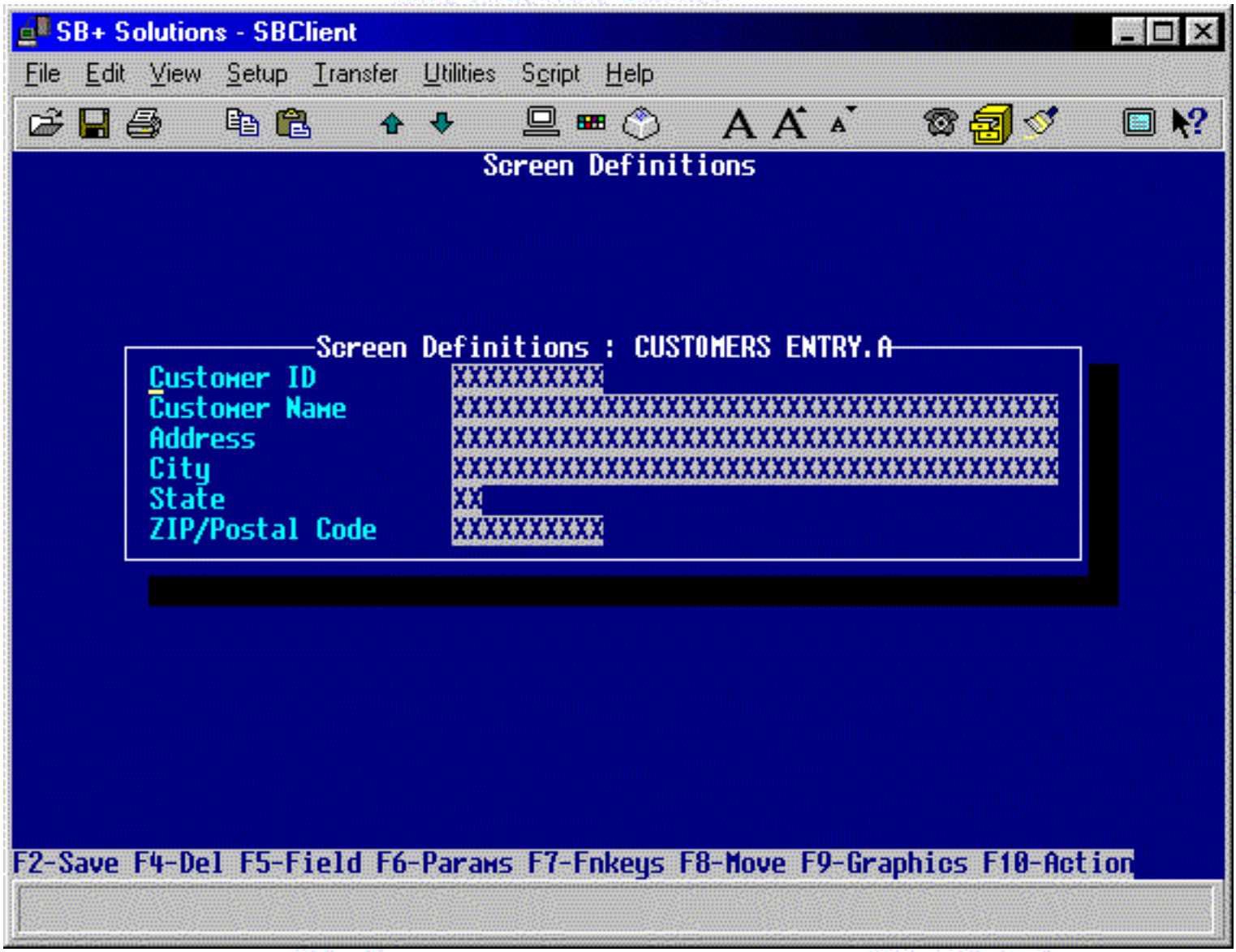
Note that multiple users and groups can be restricted simply by listing each user and/or group on a separate line, as is shown in the example.

At the "Restrict Users/Groups Not Included" prompt, enter a "Y" if all of the users and groups that are not listed are to be restricted. This has the effect of locking down anyone who isn't specifically included in the list. If you want to leave all other users and groups unrestricted, however, enter an "N" at this prompt.

Notice that the restriction set defines who can and cannot insert, amend, and delete values. It does not, however, define which

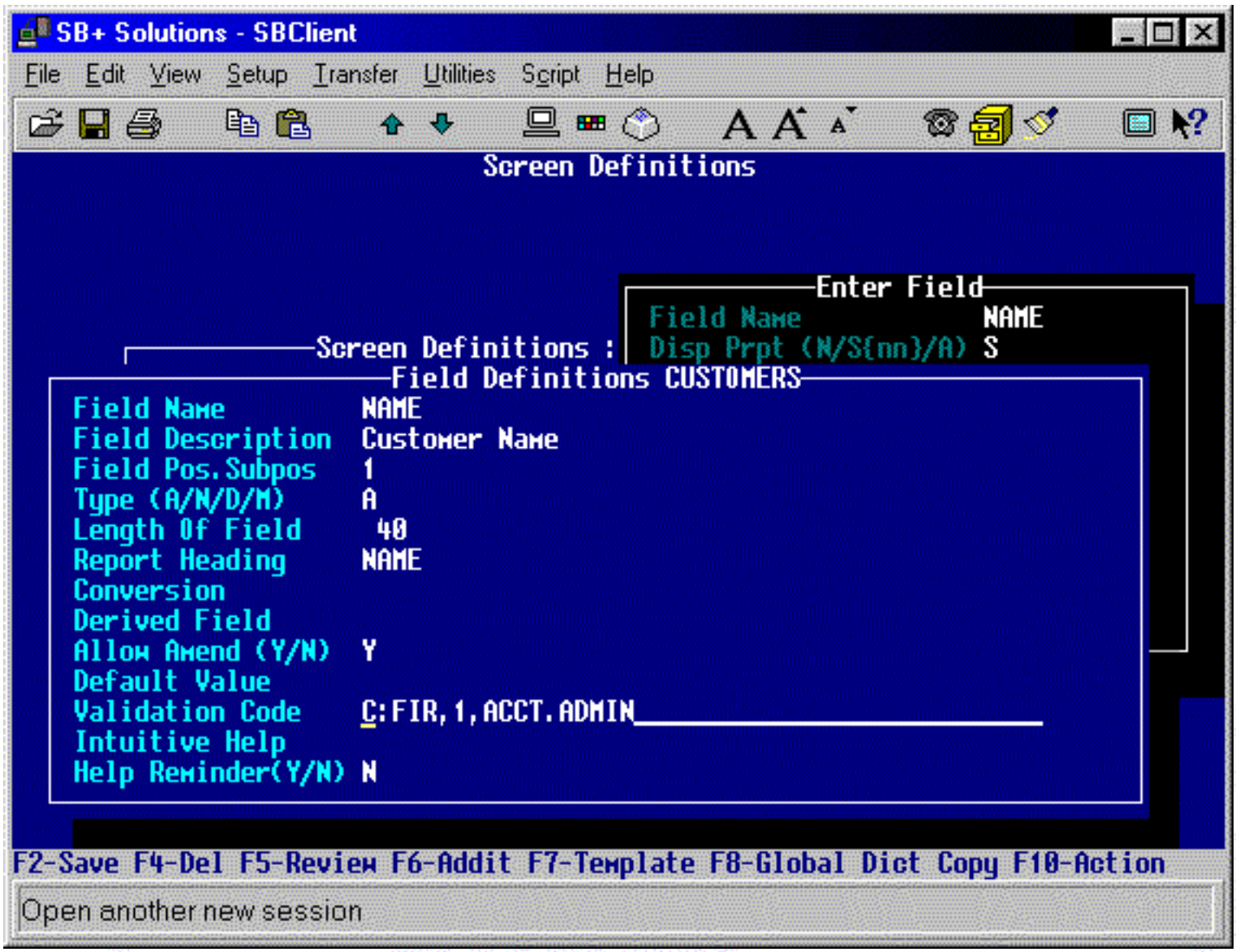
screens or fields are to be affected by the restrictions. To do this, we'll need to install a special validation process.

Let's say we have a customer entry screen which looks like this:



On this screen, we want to apply the above restrictions to the customer name, address, city, state, and zip fields. With these restrictions in place, accounting and administration personnel can add new values (on new records), administration personnel can amend these fields, but neither may delete (set to null) any of these values.

To do this, we need to edit each field individually. Move the cursor to the first character of the Customer Name field and press F5 twice to bring up the field definition. Change the validation as follows:



This validation calls a standard SB+ process called FIR (Field Input Restriction), passing two parameters. The first parameter, "1", defines that the FIR process is being used for validation. This process can also be used for **restricting deletions from a multivalued set** (when the parameter is "2"). The second parameter is the name of the field input restriction to use.

Once these changes are complete, press F2 to save this field and move to the next field. Add this validation to any field that is to be restricted. But what happens if the field already has a validation? No problem -- just add a the process call to the beginning of the existing validation. For example, if the existing validation is:

F:STATES,,Y{Must be a valid state}

This can be changed to:

C:FIR,1,ACCT.ADMIN AND F:STATES,,Y{Must be a valid state}

Note that no error message is associated to the FIR process -- if a person is restricted, either individually or because of the group they belong to, the FIR message will display an appropriate error message.

Or, if the existing validation is a process, such as:

```
READ @OTHER.REC FROM 'STATES',@VALUE
IF @RTN.FLAG THEN
  ERROR 'Must be a valid state'
  EXIT 1
END
```

The additional FIR validation can be added as follows:

```
EXEC 'FIR,1,ACCT.ADMIN'
IF @RTN.FLAG THEN
  EXIT 1
END
*
READ @OTHER.REC FROM 'STATES',@VALUE
IF @RTN.FLAG THEN
  ERROR 'Must be a valid state'
  EXIT 1
END
```

Remember, the FIR process must be called from every field that is to be restricted.

What the User Sees...

When a screen with field input restrictions is running, the user will be allowed to attempt to change the restricted value. When they press <cr> after the change, the FIR process verifies their eligibility to make the attempted change and displays an appropriate message if the change is not allowed. In short, while the program doesn't stop them from making the change, it does prevent the change from actually getting updated into the record. Interestingly enough, this is exactly how all validations function with SB+.

What Are All Those Options?

When building a selection process, one of the prompts on the screen is called Options. This prompt has a variety of different valid values, as displayed when you press F3 for intuitive help on the prompt. What isn't quite so clear is what the net effect of various combinations will do. The following explains these option codes in more detail:

A	Allow multiple selections, returning the selections in @VALUE, attribute mark delimited.
C	Allow multiple selections, returning the selections in @VALUE, separated by commas.
D	**Data stack** the selected option. Normally, the selected option is returned in the **common variable** @VALUE and the value is automatically entered into the prompt. For single valued fields, this works the same as no option, so there's no need to use it in this instance.
E	Ensign heading: At the end of the **Selection Criteria**, you can type the word HEADING followed by a heading. This options tells SB+ to look here for the heading for the box, rather than either making one up or using the process description.
F	Force selection box if only one value found: Normally, if SB+ only finds one value, it simply returns the value to the prompt without a selection box. With this option, a selection box will be displayed even if only one value will be shown.
G	Return first field. This is similar to the W option, however, if the field has an embedded space, the entire field (all words) will be returned. In contrast, the W option will return only the first space delimited word.
H	Hide key field: If used alone, this option will suppress the display of the key field in the selection box. If used with W, this will suppress the display of the first (space delimited) word. Considering you have complete control over the fields that are displayed in the Display Fields prompt, this is superfluous.
I	Show selection box immediately: In older versions of SB+, all items were selected prior to the selection box being displayed. In the later versions, this option will display the selection box as soon as a window full of information has been found, rather than waiting until all information has been found for all pages of the selection box. Incidentally, this is normally on by default.
J	Multiple selections joined together. This option allows multiple records to be selected, returning all of the selected values concatenated together. Incidentally, this appears to be the option used on the F3 for the Options prompt in the Process Definition - Selection tool.
K	Select a value and return as the key: With this option, when a selection is made the cursor will return to the key prompt on the screen and the selected value will be used as a new key.
L	Return line count of selected option: When this option is used, each line in the selection box is prefaced by a number. When a value is selected, its line number is returned instead of the key value for the selected item.
M	Multiple selections returned in @VALUE, separated by spaces: This option will allow multiple selections to be made in the selection box, returning the selected values in a space-delimited list.
N	No refresh: After all the selections have been made in the selection box, the box will not be erased when the underlying window is redrawn. In short, the box stays on the screen indefinitely.
P	Print error when no items are selected. Without this option, the process will return to its caller quietly, leaving users to wonder if the process did anything.
Q	Multiple selections returned in @VALUE surrounded with quotes: This option will allow multiple selections to be made in the selection box, returning the selected values in a single entry, each surrounded by double quotes (").
R	No window: This option tells the selection box to find all of the values and return them in the **common variable** @VALUE without displaying a window. This is most commonly used for reformatting a list of values, such as taking a saved select list and returning the values in a multivalued list (see the RMV combination of options, below).

S	Multiple selections returned in active select list: This option will allow multiple selections to be made in the selection box, returning an active select list of those selected. However, all of the selected values are not returned this way; the first value will be in the common variable @VALUE and the remainder of the selected values will be in the active select list. To put all of the values in the select list, the options of "DS" must be used.
T	Show report headings from field definitions as titles in selection box: When this option is used, the report headings for the display fields will be shown on the top line of the selection box to annotate the display. This is becoming a generally accepted option for all selection boxes, and is therefore recommended.
U	Use a user-supplied selection routine: If your software has specialized selection routines or custom indices, this option can be used, allowing you to supply your own selection routine. The name of the BASIC subroutine to use for selection can be entered using the function key F8 in the /PD.S tool. For more information about creating your own BASIC selection subroutine, look at the SB+ skeleton subroutines named SEL.USER.EG1 and SEL.USER.EG2 in the DMSKELCODE file.
V	Multiple selections directly into @RECORD: This option will allow multiple selections to be made in the selection box, and will update the current multivalue in the current record (in the common variable @RECORD) when all values have been selected. This should be used with caution, as it does not validate whether the entries should be inserted into the list, does not verify that entries are previously in the list, and overwrites existing values depending on the position of the cursor at the time F3 is pressed.
W	Return first word: If a line of text is being selected, but only the first space-delimited word is applicable to the prompt, this option returns only the first word when an option is selected. Looking at this from another angle, this option will return whatever is in the first column, instead of the actual key value used to display that column.
X	Explode multivalued fields. If a sort field is multivalued, explode each multivalue to a separate list entry using BY-EXP or BY-EXP-DSND (depending on whether the sort is ascending or descending) instead of BY or BY-DSND. Prior to version 3.x, this happens automatically.
Z	Suppress status messages. During the selection, SB+ displays a status message stating "Selecting Records..." or "Selecting Records From Index...". When this option is used, these messages are not displayed.

A few of these codes can be added together into combinations, which function differently from the codes used independently. A few of the more useful combinations include:

TDS	Display the report headings as the selection box title and return all selected options in an active select list. Remember, if 'S' is used alone, the first item selected is not in the list, but rather in the common variable @VALUE.
TMV	Display the report headings as the selection box title and return all selected options in a multivalued list in the common variable @VALUE.
RA	Return all selected values in an attribute mark-delimited list in the common variable @VALUE without displaying a selection box.
RMV	Return all selected values in a multivalued list in the common variable @VALUE without displaying a selection box.

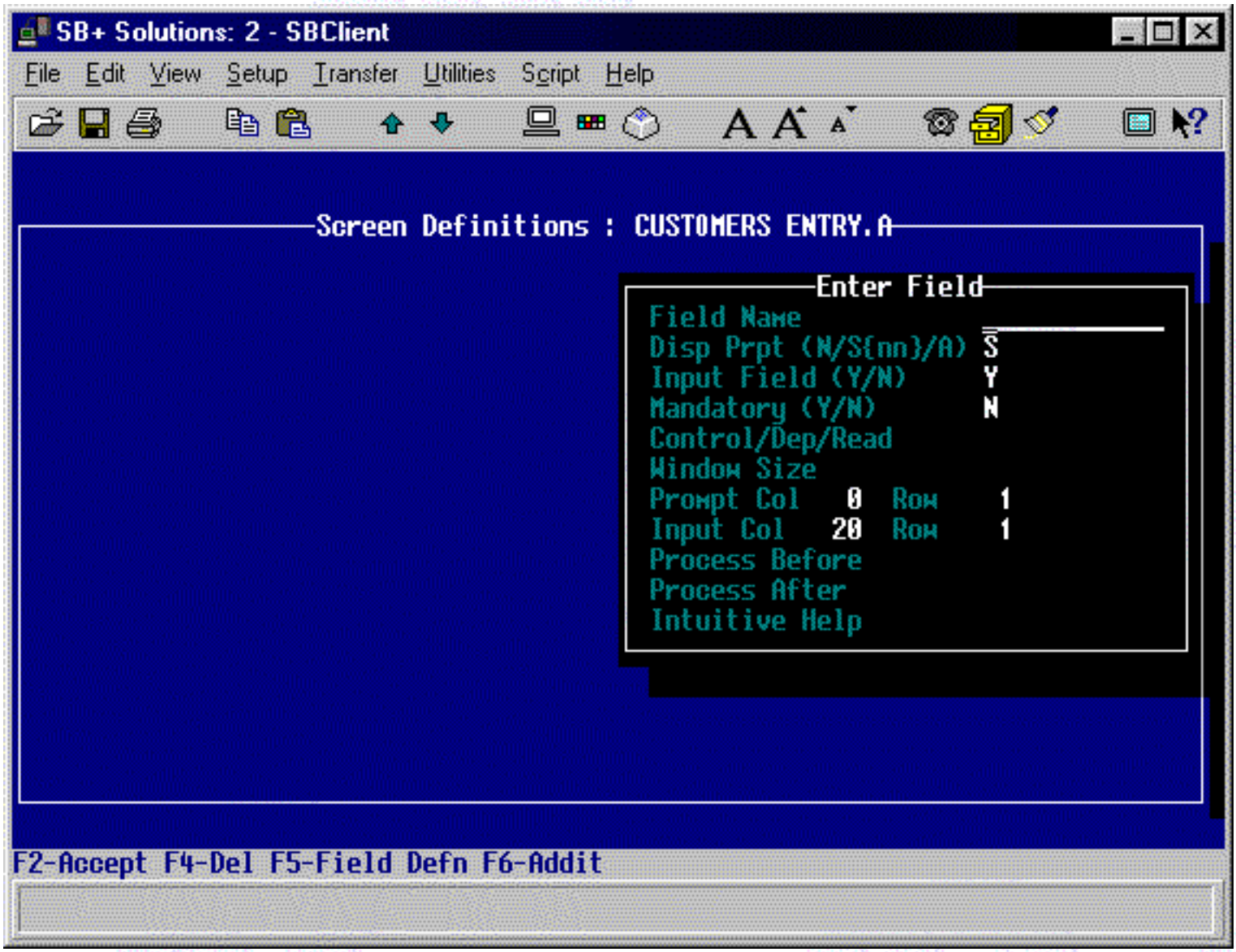
Note that it doesn't matter what order the options are in. Therefore, RMV, MVR, and VRM are all acceptable variations of the same options.

Two Intuitive Help Processes?

There are two different process slots in SB+ where intuitive help can be defined for a given prompt. The first is on the field definition; the second is on the screen definition.

If intuitive help is defined on the field definition, the process applies to every place the field is used. This includes all screens, as well as all selection processes where the field may be prompted.

At the bottom of the F5-Field screen in the Screen Definitions tool, however, another intuitive help slot exists as follows:



If a process is listed in this slot, it will override any intuitive help process on the field definition for this screen only. Two important issues to keep in mind: The two are mutually exclusive and the screen definition slot always overrides the field definition slot.

There are several reasons why the intuitive help for a prompt on a screen might be different from the intuitive help as defined for the field definition. For example, the prompt on a certain screen might need a selection process which supports multiple

selections, whereas the selection process on the field definition allows only a single selection. Or, the selection process defined for the field definition may be for a different program, and as such is not adequate for the current screen.

The real complexity comes when you have the same prompt appearing in multiple screens with different selection needs in each instance. In this case, there is little choice but to create several selection processes, linking them to the field on the F5-Field window on each screen, or create several field definitions for the same attribute.

How Do Selection Processes Work?

There are four main functions of a selection process. These are:

- Optionally prompt the user to enter selection parameters,
- Select records from some source in some order,
- Allow the user to select one or more of the displayed values, and
- Return those values to the program in some format.

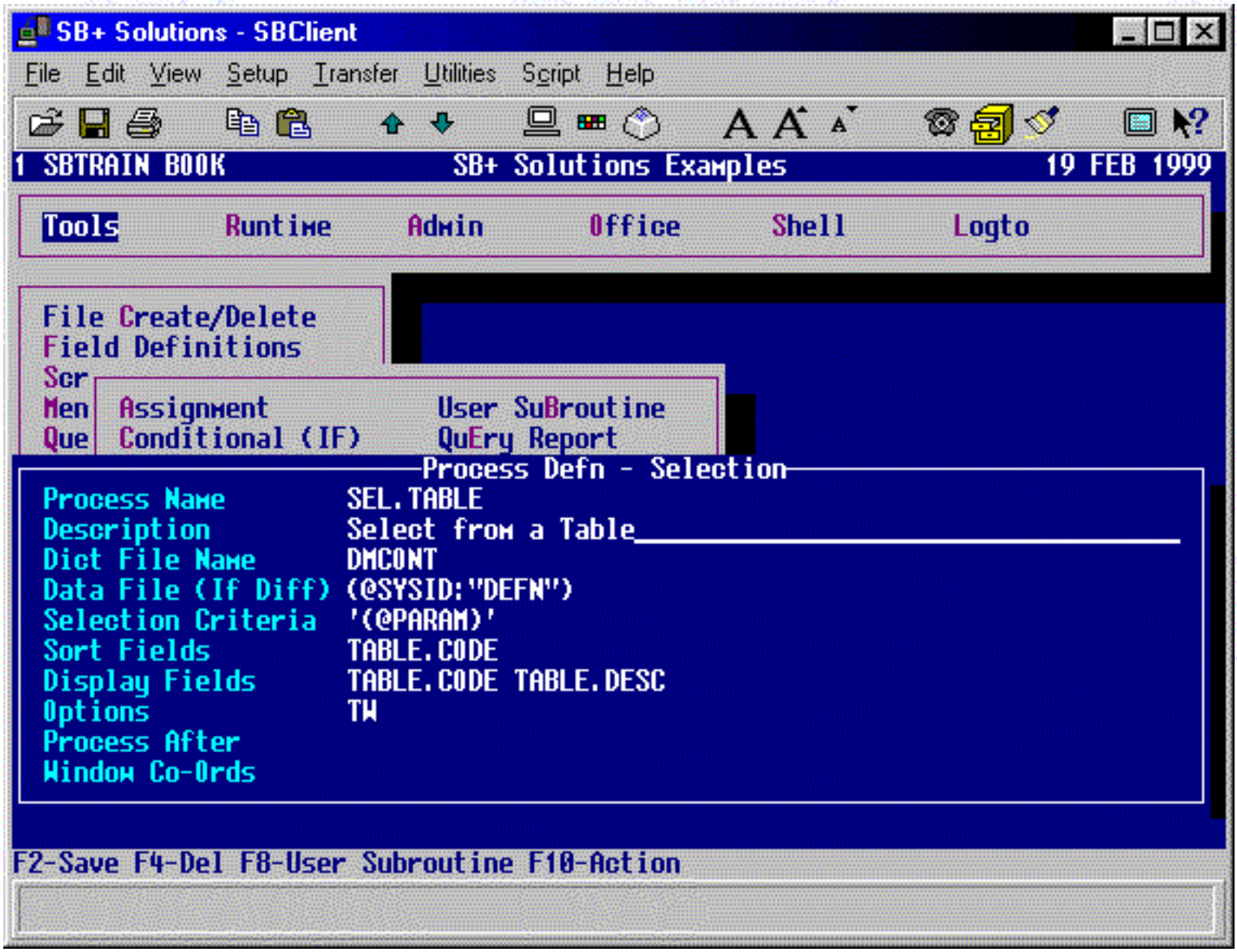
When a selection process is invoked, SB+ first looks at the **Selection Criteria** to see which records are to be selected. Using this information, SB+ will construct a standard OE SELECT command (or SSELECT if sort fields are expressed) and will execute it. From this, SB+ will receive a list of values (usually record keys) which it can use for displaying the values that the user will select from.

Next, records are read and columns of values are displayed, based on the fields as listed in the Display Fields prompt. This is what the user will see. At this point, the user can select one or several values from the selection box (depending on the options that are used). Once all selections have been made by the user, SB+ reformats the selected values according to whatever options are being used, and these value(s) are returned to the program.

SB+ retains the order that values are selected by the user. If you want to re-sort the values after the user has made a selection, you certainly can -- but SB+ won't do it automatically.

Selecting from an SB+ Table

When a prompt has a validation that is based on a table, it is very easy to build intuitive help to select values from the table. On one hand, using the F3 intuitive help on the Conversion prompt in the Field Definitions tool, SB+ will build the intuitive help process for you automatically. However, I often wonder why the intuitive help built automatically by SB+ is so long and drawn out. A much simpler alternative is a standard process that I call SEL.TABLE, which appears as follows:



One of the advantages to SEL.TABLE is that this one process can be used for selecting all tables in a given system. With this one generalized process, if the format of the selection process requires some modification (i.e. the addition or removal of column headings) the change can be made in one place and all table selections throughout the software will be instantly affected. Second, one cannot overlook the small overhead that SB+ requires to evaluate the S:SSELECT... phrase. In contrast, SEL.TABLE does not require this evaluation step, and therefore will perform faster and with less utilization of system resources.

To call SEL.TABLE, enter the following at the Intuitive Help prompt, either in the field definition or on the F5-Field window in the Screen Definitions tool:

SEL.TABLE,*tablename*

In this syntax, *tablename* is simply the name of the SB+ code table to be selected. SEL.TABLE will present the box, allow the user to make a single selection, and will return that selection to the caller.

But what if multiple selections are required? How can SEL.TABLE be used for both single and multiple selections?

Truth be told, one selection process cannot be used for a single selection one time and for multiple selections another time. Therefore, we'll need to create two different table selection processes -- one for single selections and one for multiple selections.

In planning the interface to these processes, however, there is some cause for debate. Do we create two autonomous processes for selecting from a table, or should we create one single process that interfaces to both selection types?

First, let's look at the problem from the autonomous processes perspective. We already have a process called SEL.TABLE for single selections. With a quick copy to SEL.TABLE.MV (to return a multivalued list of selected options), and a quick change of the options from "T" to "TMV", we have two separate and distinct processes, quick and simple.

The only downside to this approach is the fact that the user must know that both processes exist in the standard process library, instead of having to remember only one process name. This is a minor handicap, mind you, but still an issue that some people may dislike.

On the other side of the issue, we can easily create one process which calls either the single or multiple selection table selection processes. This would most likely be a paragraph similar to the following:

```
IF (FIELD(@PARAM,',',2) = 'M') THEN
  EXEC 'SEL.TABLE.MV,':@PARAM"G,1"
END ELSE
  EXEC 'SEL.TABLE.SINGLE,':@PARAM"G,1"
END
```

Note that the original SEL.TABLE process has been renamed to SEL.TABLE.SINGLE. This is done so that the main paragraph can still be called SEL.TABLE. With this paragraph and the two selection processes in place, you can enter the following in the Intuitive Help prompt for a single selection:

```
SEL.TABLE,tablename
```

...or for a multiple selection...

```
SEL.TABLE,tablename,M
```

.
Of course, this syntax variation is really not much easier to remember than the two selection process names, so on this issue alone the two techniques are comparable. However, there is one area where the two techniques differ: When the two autonomous processes are used, a table selection will always invoke only one process. Using the second approach, where a paragraph makes a decision and then calls one of a couple of separate selection processes, two processes are being executed for each table selection: First, SEL.TABLE is invoked, then it selectively invokes the actual selection process. This may seem inconsequential on the surface, but when you consider that SB+ checks security for every process that is executed and calls a subroutine (or several) to interpret each process, it becomes apparent that there is potential for this additional process call to rob valuable system resources, particularly if table selections are used extensively in your application.

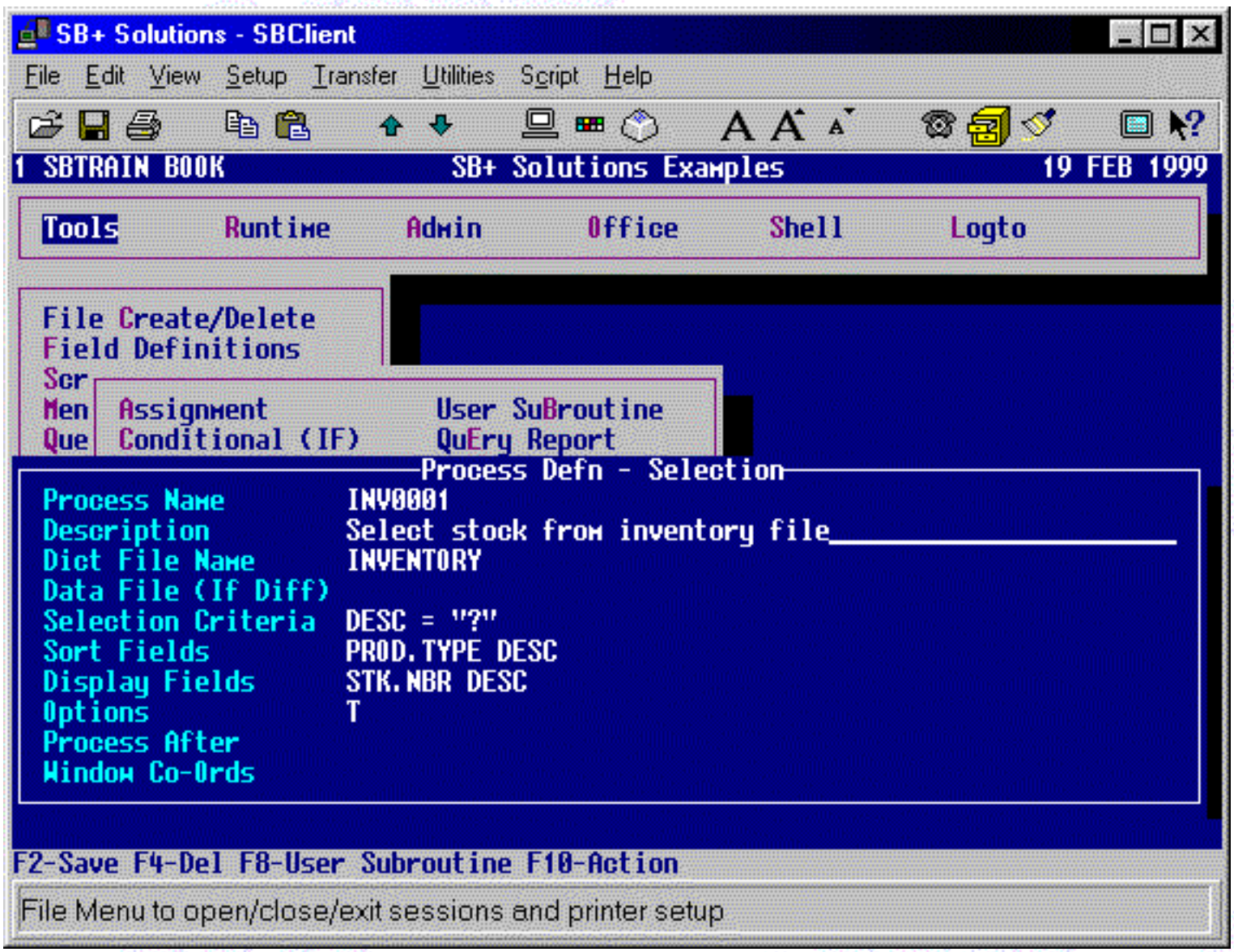
In the final analysis, whatever technique you use should be a balance between your development standards, system resources, and

personal style.

Selecting From a File

The most widely used application for selection processes is unarguably the selection of information from other files. For example, on a sales order screen, the customer ID prompt should have intuitive help to select a record from the CUSTOMERS file. The stock number on the same screen should have intuitive help to select records from an INVENTORY file. And of course, the sales order ID prompt itself should have intuitive help to assist the user in recalling orders that were previously entered into the ORDERS file.

Even though a screen may be for one file, such as ORDERS, any number of selection processes may be built that select from other files. For example, to select a stock number from the INVENTORY file, a selection process such as the following could be used:



In this selection process, note that the Dict File Name is INVENTORY. This prompt tells SB+ where to find the field names that may be referenced elsewhere in the definition. Also, if no data file name is entered below the dictionary name, this prompt tells the selection process the name of the file to be selected.

In the **Selection Criteria** prompt, this syntax tells SB+ to prompt for a description and then use the entered value to select records

from the INVENTORY file. The value entered in the prompting screen will replace the question mark in the syntax prior to the selection, allowing the selection to be different each time it is used.

Next, the Sort Fields prompt says to sort the selected records by product type, and then by description within product type. Any number of sort fields can be specified this way.

The Display Fields prompt is where you tell the selection process what you would like displayed when the selection box is drawn and presented to the user. Any number of fields may be specified here, though some fields may be truncated depending on the width of the display screen and the lengths of the fields to be shown.

Finally, the **Options** for this particular example state that we want titles shown above each of the fields in the selection box. This makes the selection box a bit more informative, and is quickly becoming the generally accepted standard option for selection boxes.

To implement this process as a part of your screen, simply enter the name of the process on the Intuitive Help slot either in the field definition or on the F5-Field screen in the Screen Definitions tool.

With selection criteria, sort options, display fields, and options, this simple selection process has everything you need (and maybe even more than what you need) to perform just about any simple selection on any file in your system.

Selecting From Multiple Files

It's very simple to create a selection process which will select a group of records out of a single file. What if, however, you need to select a group of records that live in several different files?

While this task is somewhat out of the reach of a simple selection process, it is certainly not impossible. Using a combination of different processes you can create complex intuitive help that can select information from any number of files.

First, let's examine the features of a selection process. A selection process:

- May prompt the user for input to be used in the selection;
- Selects records from a file;
- Displays those records in a box where one or more can be selected (depending on the **options** used), and;
- Returns the selected values to the caller.

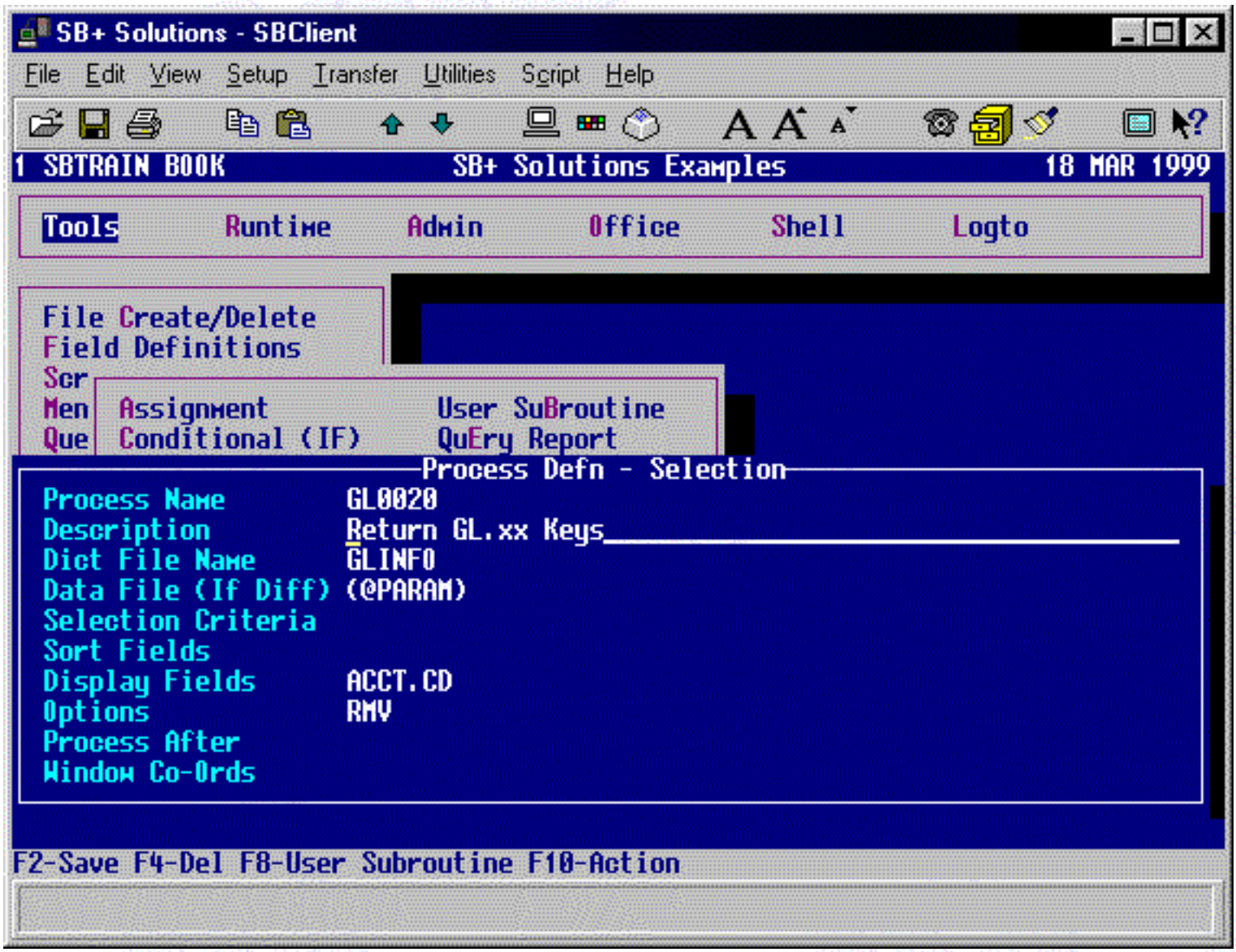
Just because a selection process *can* do all these things doesn't mean that the selection process *must* do all these things all the time. A selection process doesn't have to prompt for input -- an input process can be used instead. A selection process doesn't have to display any values -- it can instead quietly return a group of selected records.

For the purpose of this example, we'll assume that we do not need to prompt the user for any information. Instead, we simply want to select asset records from the GL.97, GL98, and GL.99 files. Once we've selected the records from these three files, we'll display information about each of the selected records. This is where the example will get interesting -- each record displayed in the selection box may come from a different file!

For our main intuitive help process, we'll use a paragraph. This paragraph will then call a variety of other processes to achieve the end result. This paragraph appears as follows:

```
@PARMS(2) = ""
@PARMS(2) = INS(@PARMS(2),1,-1,P('GL0020,GL.97')) ;* Load the asset records from GL.97 into P2
@PARMS(2) = INS(@PARMS(2),1,-1,P('GL0020,GL.98')) ;* Load the asset records from GL.98 into P2
@PARMS(2) = INS(@PARMS(2),1,-1,P('GL0020,GL.99')) ;* Load the asset records from GL.99 into P2
*
IF @PARMS(2) # "" THEN
  EXEC 'GL0021' ;* Allow the user to select an asset
END
```

Note that to make this work, we need two additional processes. The GL0020 process will select asset records from the file name as passed in the parameter (following the comma after the process name), and will return the selected records in @VALUE. This is accomplished using a selection process with the options of RMV, as follows:



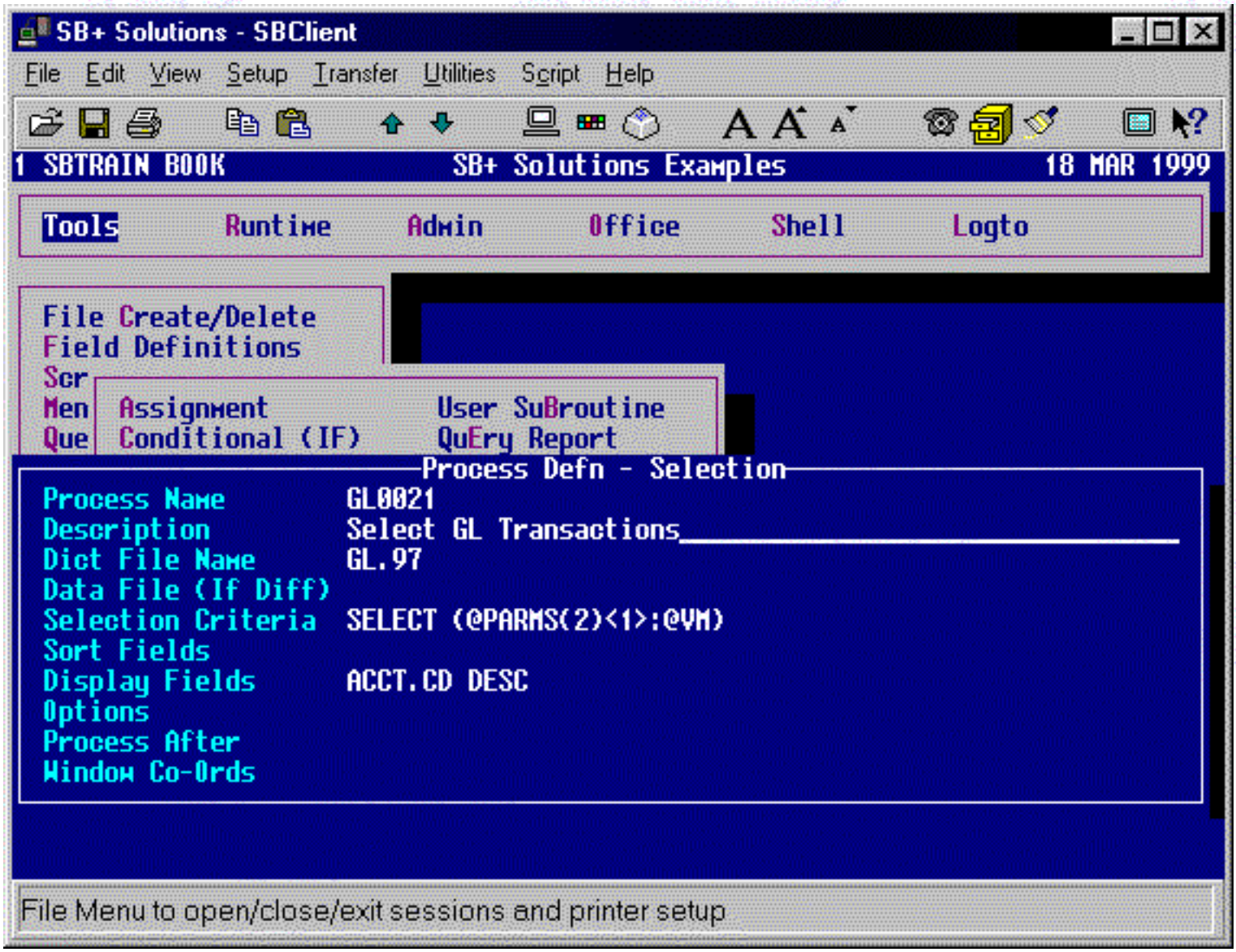
The options of RMV will return all of the selected records in the common variable @VALUE, which are then returned to the INS (...) function and updated into @PARMS(2). (In versions 3.x and higher we may also choose to use the Z option to suppress SB +'s status messages from being displayed.) Note that while the selection box is intended to quietly select and return a result without displaying anything for the user, we still list a field to display. This causes SB+ to do the selection without any additional prompting. If we were to forget this little detail, SB+ will display a prompting window (under the assumption that because we didn't list selection criteria, sort fields, or display fields, we must have inadvertently missed something). Listing a field here -- any field -- prevents this unwanted behavior.

Ok, so now that we have all of the selected record IDs from all three files in @PARMS(2)<1>, how can we get the selection box to display each records from the appropriate file? We need a selection box, of course, to display the values and allow us to select one, but how can we tell SB+ to read from any number of different files?

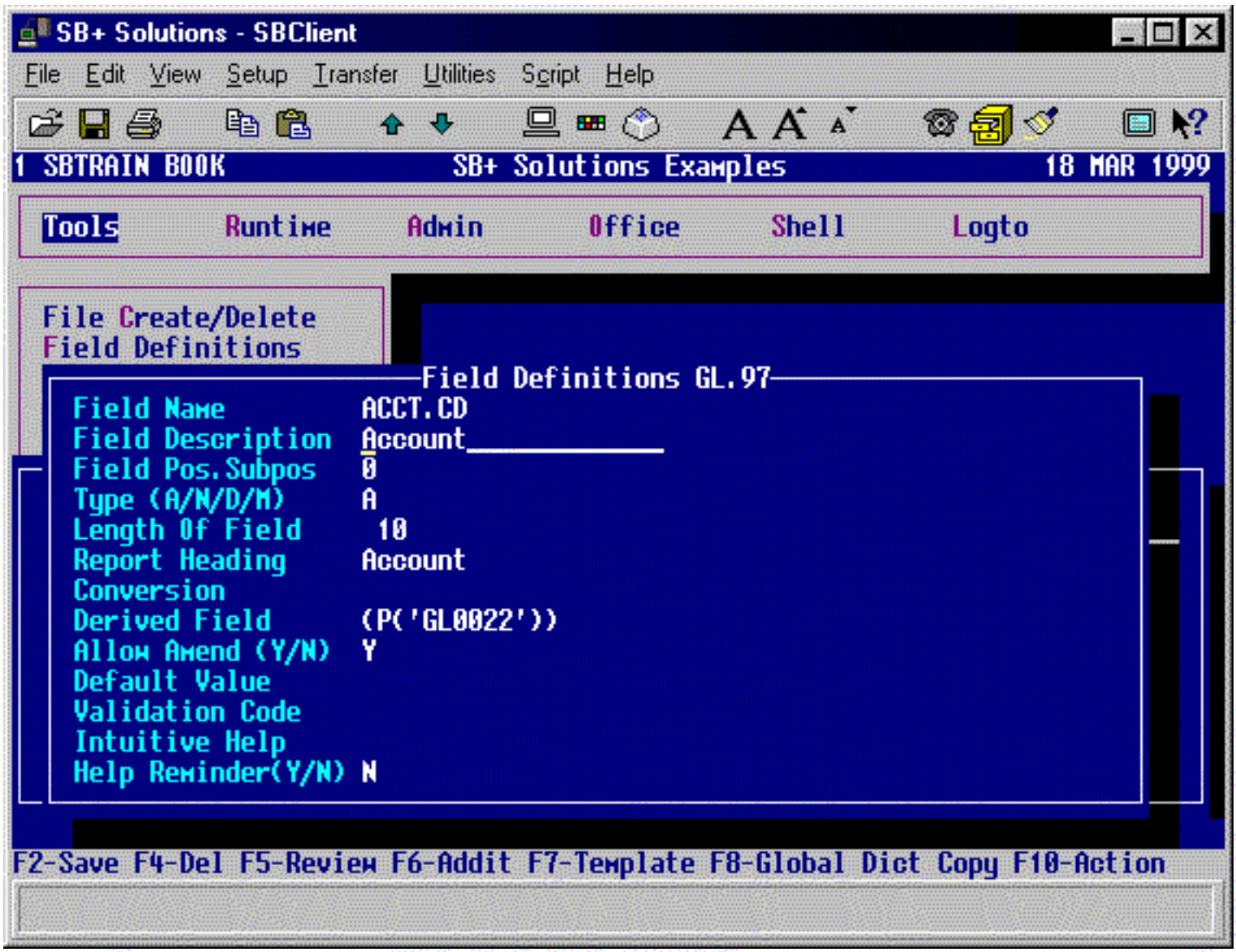
When a selection process is invoked, SB+ will first look to see if the Data File (if Diff) prompt has a value. If it does, SB+ will open the file as named. If there is no data file name listed, SB+ will open the dictionary file name, and expects to find all of the selected records in that file. However, all of our records aren't in that file!

If you look through the Process Definition - Selection screen, you'll find there is no slot available for plugging in your own read logic. Though no process is found in this tool, there is a process slot available, though it's not generally considered in this context. That process slot is the Derived Value on a field being displayed in the selection window. Before we get too far into this,

however, take a look at the selection process GL0021:



Pay special attention to the display fields. We're displaying the GL account number and description from the GL.97 file -- however, we're not telling the process about the other files. Or are we? Look closer... specifically, let's go into the Field Definitions tool and take a look a this field for the GL account number:



Note the process being called in the Derived Field prompt. This process, GL0022, is the key to making our display work as expected. This process is a paragraph, which appears as follows:

```
@RTN.FLAG = 0
READ @RECORD FROM 'GL.97',@KEY
IF @RTN.FLAG THEN
  @RTN.FLAG = 0
  READ @RECORD FROM 'GL.98',@KEY
  IF @RTN.FLAG THEN
    @RTN.FLAG = 0
    READ @RECORD FROM 'GL.99',@KEY
    IF @RTN.FLAG THEN
      @RECORD = ''
    END
  END
END
*
@VALUE = @KEY
```

The last line of this paragraph sets up the value to be displayed for this field -- everything else sets up the values to be displayed for all of the other fields. SB+ gives us @KEY for each record ID selected, and we can use this to locate the record and set

@RECORD. All subsequent fields (which extract values from @RECORD) will then use the record we loaded.

It should be noted that this process as written could be more efficient. As stated earlier, SB+ expects all of the records to be in either the data file or dictionary file (depending on the value in the Data File (if diff) field). As a result, this process will read a record from GL.97 for each selected ID. Immediately after, our process will again read GL.97, GL.98, and GL.99 until we find a record. Why bother reading GL.97 twice?

If we were confident that most of the records were to be loaded from the GL.97 file, we could implement a small variation on this theme which would be more efficient:

```
IF @RECORD = "" THEN
  @RTN.FLAG = 0
  READ @RECORD FROM 'GL.98',@KEY
  IF @RTN.FLAG THEN
    @RTN.FLAG = 0
    READ @RECORD FROM 'GL.99',@KEY
    IF @RTN.FLAG THEN
      @RECORD = ""
    END
  END
END
*
@VALUE = @KEY
```

In this process, we first check to see if the record is not found. If the record is not found, we then look in the GL.98 and GL.99 files to find it. Note that there's no reference to the GL.97 file. Instead, because GL.97 is our Data File (if diff), SB+ does its reading and our process looks for another record only if nothing is found in GL.97. To be most efficient, you'd want your Data File (if diff) to reference the file where the records will most likely be found. This, then, eliminates the need to look in the other files for a greater percentage of records.

Finally, note that this technique assumes you'll be using files with unique keys. If there is a record in GL.97 with the same ID as a record in GL.98, you'll run into problems with the above process, as it will not know which file is being referenced by each ID. When this happens, append the file name to each ID in @PARMS(2)<1> (in the form *file*id*), and modify the above process to read the record from the appropriate file.

Selecting From An Existing Saved List

In the selection process, if the value entered at the **Selection Criteria** prompt is a single word, SB+ assumes that the word is the name of a select list that has been created and stored via some other means. Therefore, if the selection criteria says XYZ, SB+ will look for a list called XYZ to get the values for the selection box. Of course, if the list does not exist, no box will be displayed.

In many circumstances, it is necessary to maintain some autonomy between lists being used by two or more different users. In other words, if I run a program, I don't want to use anyone else's list, and I certainly don't want them clobbering my list. To provide this functionality, SB+ provides a suffix of :PORT that can be appended to the end of the list name. If this is the last part of the list name, it will be replaced with the current port number before SB+ attempts to retrieve the contents of the list.

For example, if I want a list called WRK.2 on port 2, WRK.5 on port 5, and WRK.603 on port 603, I can enter the following code at the Selection Criteria prompt in the selection process:

```
WRK.:PORT
```

In version 2.x, when a select list is used, all sort fields are ignored -- the order of the incoming list is the order that the options will be displayed. In later versions, this is not the case -- SB+ uses the sort fields to reselect the file in the proper order.

(Incidentally, SB+ will do nothing to clean up after you when you're creating and using select lists. Take care to incorporate logic to delete old lists as appropriate in your application!)

Selecting From a Multivalued List in Common

Let's assume for a moment we're creating a sales order entry screen. On this screen we have an inventory ID prompt and a corresponding price for each inventory ID. In each inventory record there is a valid list of prices in attribute 15, which we will want to select when the user presses F3 on the Price field on the screen.

At the time the inventory ID is validated, we'll load the inventory record into @OTHER.REC in preparation for selecting a price or validating that the price as entered is valid for the product. Therefore, when F3 is pressed, we'll want to select the valid prices from the **common variable** @OTHER.REC, instead of reading the information from the file again. By entering the following at the Selection Criteria prompt, this effect can be achieved:

```
SELECT (O15:@VM)
```

This particular selection criteria is called a *multivalued expression* selection. It is identified by the word SELECT followed by some **expression** in parentheses, usually ending in :@VM. In this example, the expression is O15 (yes, the letter "O"), which is shorthand notation for @OTHER.REC<15>, with a value mark appended to the end. Considering this attribute contains a multivalued list of prices, this does exactly what we want to do, without the extra overhead of re-reading the inventory record. The trick, though, is to ensure that the inventory record is held in @OTHER.REC when the inventory ID is validated.

Note the trailing :@VM. This is very important to making this technique work. It does so by ensuring that the field will be multivalued, regardless of whether O15 has one or multiple values. If we forgot this suffix and O15 was a single value, SB+ would treat the single option as the name of a select list, not a valid list of options to be selected. Such functionality is rarely necessary, so it's important to remember the :@VM suffix every time this technique is used.

Finally, it's important to note that any expression can be entered between the parentheses. This can be a regular common variable, such as @VALUE, a file read using the F(...) function, a process call using the P(...) function, or some combination of any expression elements. Always remember, though: no matter what expression you use you should never forget the :@VM suffix!

Selecting From a Multivalued List in an External Record

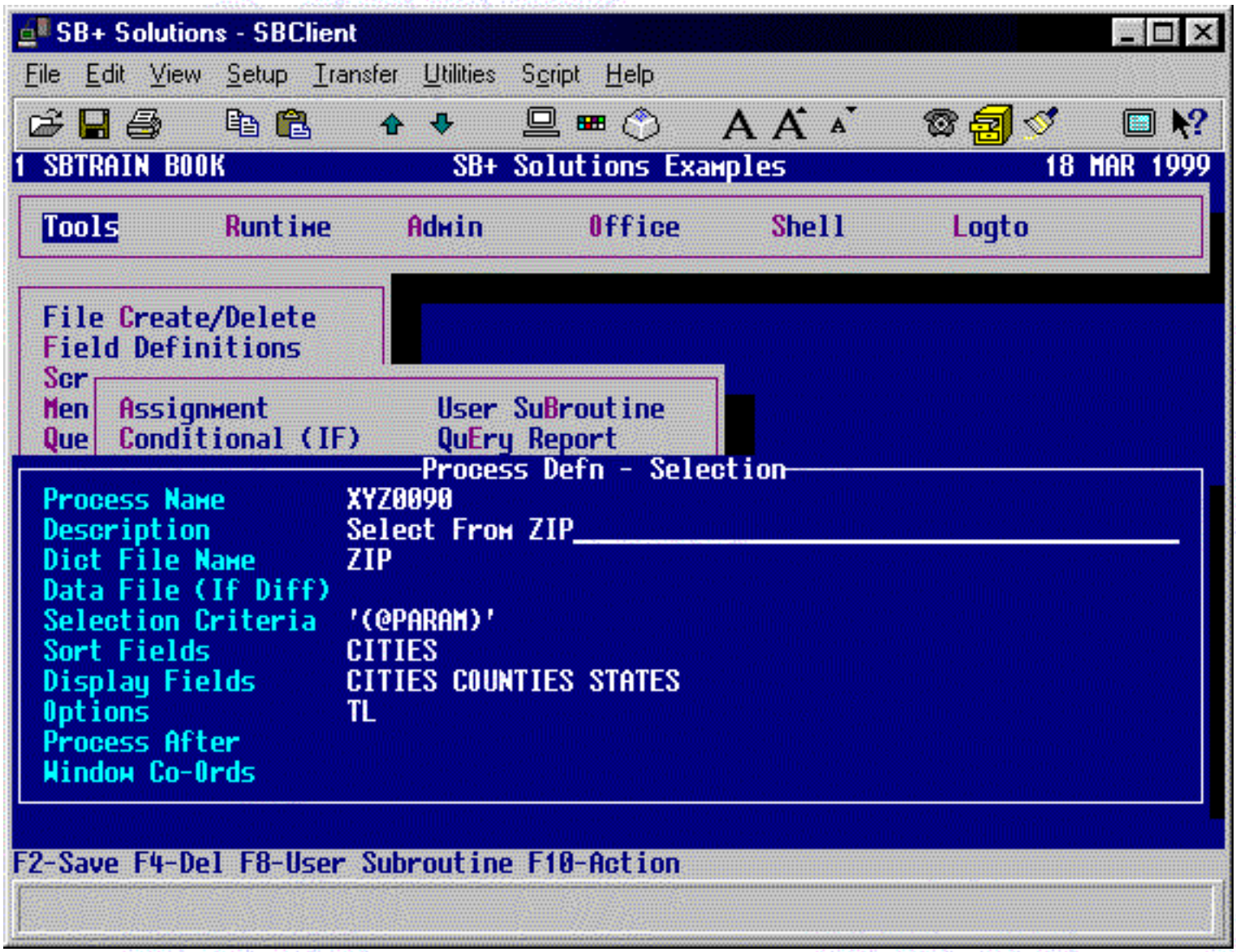
When you have a multivalued list in memory, it's easy to bring up the list for selection using the **previous technique**. However, what if you need to select one multivalue from a different record stored on the disk somewhere?

For example, let's say we have a ZIP/Postal Code file which has the following fields:

ZIP.ID	0	Key to the record in the format *country*ZIP/Postal*
CITIES	1.M	A multivalued list of cities in this country and ZIP/Postal area
COUNTIES	2.M	A corresponding list of counties in this area
STATES	3.M	A corresponding list of states in this area

Using this file, we can provide F3 on the City field on a screen, as long as a country and ZIP/Postal code is known. When F3 is pressed, we want to show the user a list of cities, counties, and states to choose from. If they select one of the options as shown, we can then load the appropriate city, county, and state into the next three fields (via **data stacking**).

In a selection process, if the primary sort field is multivalued, SB+ will do an *exploding sort* on the multivalues in the record. As a result, each multivalue will appear as a line in the selection process. (Incidentally, this is the same technique used by **SEL. TABLE**.) Therefore, all we need to do is tell SB+ that the options in the selection box should come from a single record (from our ZIP file, per se), and each multivalue should be a separate entry for selection. The following selection process will do just that:



Note the **Selection Criteria**. This selection criteria says that we're going to **select a single record**, and the name of that record will be passed into the process in the **common variable** @PARAM. Next, notice the sort fields. Because CITIES is a multivalued field, each city (and corresponding state and county) will be extracted and shown on a separate line in the selection process. Finally, note the 'L' **option**, which displays line numbers in front of each line, and also returns the line number of the selected option to the caller.

Normally, when you display a selection box and allow the user to make a choice, SB+ returns the key of the selected record(s) back to the caller. This works fine as long as you're selecting from a number of records, but in this case, all of the options came from the same record! Therefore, if we don't specify otherwise, SB+ will return the same value (the ZIP ID) no matter which option is selected. Of course, such a feature has little value in our (or any!) application.

Instead, the L option causes SB+ to return a number which tells the caller which value (row) was selected. This can then be interpreted by a calling paragraph such as this:

```
LOCAL NDX,ITEM.ZIP
*
ZIP.ID = CNTRY : '*' : ZIP
READ ITEM.ZIP FROM 'ZIP',ZIP.ID
@RTN.FLAG = 0
```

```
*
NDX = P("XYZ0090,":ZIP.ID)
IF @OTHER(18) # 14 THEN
  @VALUE = ''
  DATA ITEM.ZIP<1,NDX>,ITEM.ZIP<2,NDX>,ITEM.ZIP<3,NDX>
END
```

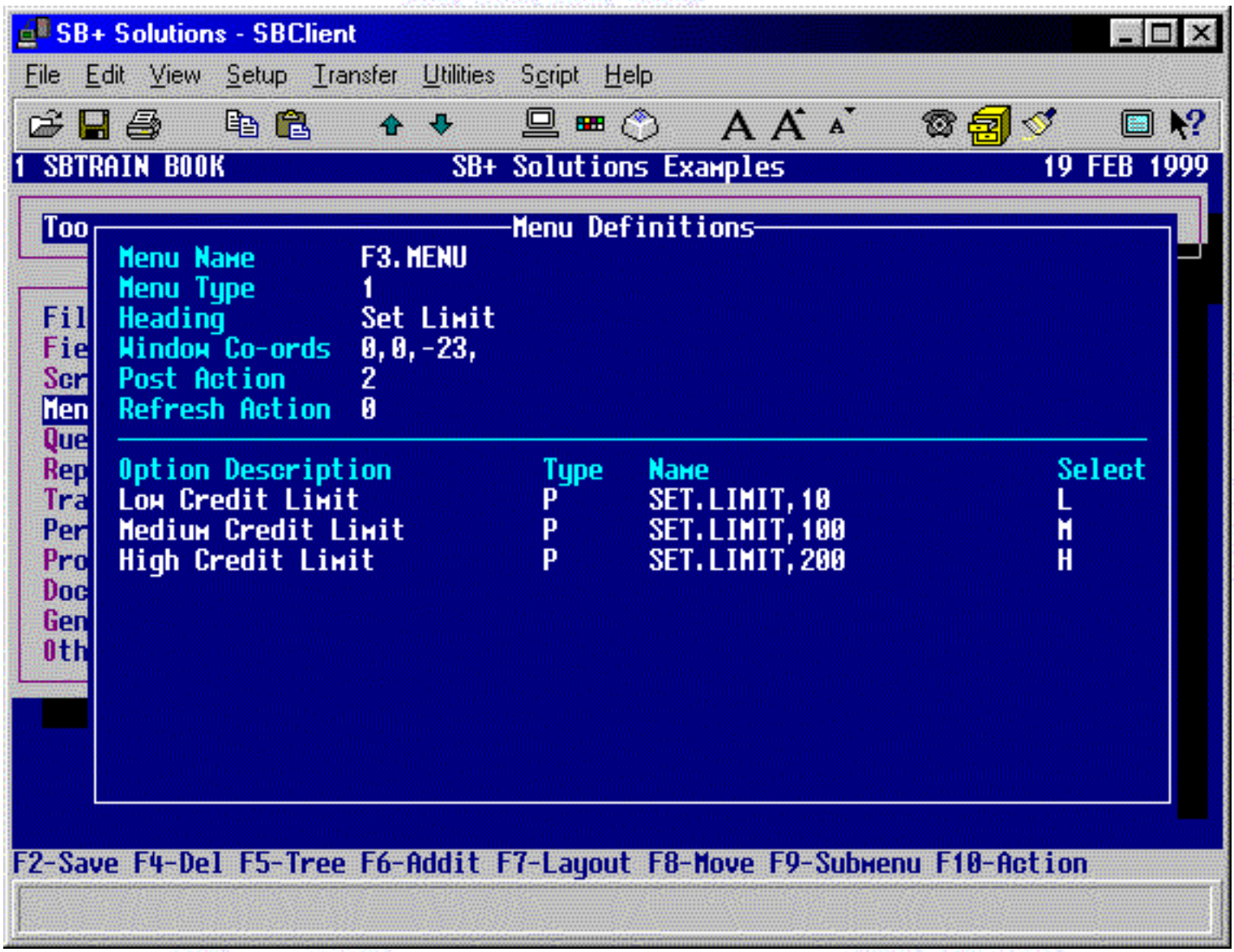
Note that this process reads the record, then calls the XYZ0090 (selection) process which will read the record again. This redundant reading of the record is unfortunately unavoidable, because we can't prevent the selection process from reading the record, and this process needs the record so we have values to stack. Next, note that @VALUE is cleared before the values are stacked. (We could actually clear @VALUE before or after the DATA statement -- it really doesn't matter, as long as @VALUE is cleared. Otherwise, SB+ will load the contents of @VALUE into the prompt before stacking the values, and as a result, everything will be put into the wrong prompt.)

This paragraph would be called for intuitive help on the CITY field on the screen, and it in turn would invoke the selection process as illustrated earlier.

Multiple Intuitive Helps on a Single Prompt

Even though SB+ provides only one slot for intuitive help for a prompt (either on the field definition or screen definition), multiple selection options can be supported. This is accomplished by calling a **menu as the selection process**, and then having the menu call several other selection processes depending on the option chosen.

Menu construction is discussed in more detail in **Chapter 7**, so we won't go into detail here regarding the basic menu. Nonetheless, the menu definition tool appears as follows:



Look at the field labeled Post Action. The "2" at this prompt tells SB+ to exit the menu after an option has been executed. Other than actually naming the processes in the second half of the screen, this is the next most important aspect of a menu that is used for intuitive help. With this option set improperly, SB+ may run the appropriate selection box, but the menu will not go away until you press Escape. Therefore, always remember to set this option to a "2" (or "1") when creating a menu for intuitive help.

Selecting a Single Record

This is one of those techniques with limited value, but is included here for the sake of being complete. There may come a time when you want to select one record from a file, and here's you how to do it.

In the **Selection Criteria** prompt in the Process Definition - Selection tool, if a value is enclosed are apostrophes, any literal or parenthetical expression between the apostrophes will be taken as a single key value.

Therefore, to select a record with a key of ABC, the following selection criteria could be used:

'ABC'

Or, to take the contents of the fourth attribute in @VALUE and use it to select a single record, the following selection criteria could be used:

'(@VALUE<4>)'

Note that any valid SB+ **expression** may be placed inside of the parentheses inside the apostrophes. With this flexibility, even a process call could be used with the following syntax:

'(P("GET.KEY"))'

Of course, when doing this you need to be constantly aware of matching quotes and apostrophes, taking extra care to ensure that quotes are not embedded inside of quotes, and apostrophes are not embedded inside of apostrophes. The following is an example of this problem:

'(P('GET.KEY'))'

Normally, the apostrophe is a valid delimiter inside of the P(...) function. However, because the entire expression is enclosed in apostrophes, the apostrophe is invalid in this context, and therefore this expression will not work properly. Normal quotes around GET.KEY, however, would work just fine.

Using SB+ Indices in Selection Processes

Normally when a selection process is invoked, it will search through the entire file looking for records that match the selection criteria. For small files, this is probably acceptable. For larger files, however, the time required to do the selection and the system resources used in the endeavor are probably excessive, and should be minimized if at all possible.

In this scenario, it's best to create indices for various fields in the file's records. These will then serve as indexed lists of record keys that can be used in place of selecting the entire file. By directing the selection to an indexed list of keys the selection is not only faster, it is also more efficient. In the end, fewer system resources are necessary for processing the indexed list vs. selecting the entire file.

In SB+, there are two (and sometimes three) things to do in order to setup each index for records in a file. These are:

- Define the indexed fields;
- Create the index file;
- If the file being indexed has information, regenerate the index.

Defining an SB+ Index
Creating an SB+ Index File
Regenerating an SB+ Index
Changing an SB+ Index
Deleting an Indexed Field
Deleting an Index File
Reading From an Index
Using SB+ Indices With Split DICT/DATA Files
Using Derived Indices

Standard OE Conversion Codes

The following charts show a list of OE conversion codes that are supported on most Pick and Pick-like platforms. This is by no means a comprehensive list of the conversion codes for any given machine, but rather is a generally accepted subset of common formatting options.

The input value in the chart shows what the incoming value is prior to the conversion. The output value illustrates what the net effect of the conversion code will be.

OE Date Conversions

Conversion Code	Input Value	Output Value	Comments
D	9999	17 MAY 1995	International date, year format unspecified
D2	10000	18 MAY 95	International date w/ 2 digit year
D4	10001	19 MAY 1995	International date w/ 4 digit year
D2/	9999	05/17/95 (USA) 17/05/95 (International)	2 digit year w/ slash separator
D2-	9999	05-17-95 (USA) 17-05-95 (International)	2 digit year w/ dash separator
D4/	9999	05/17/1995 (USA) 17/05/1995 (International)	4 digit year w/ slash separator
D4-	9999	05-17-95 (USA) 17-05-95 (International)	4 digit year w/ dash separator
DW	9999	3	Day of week: Sunday is zero
DWA	9999	WEDNESDAY	Alpha representation of the day of the week
DM	9999	MAY	Month
DY	9999	1995	Year
D4Y	9999	1995	Year: 4-digits
DD	9999	17	Day of month

OE Time Conversions

Conversion Code	Input Value	Output Value	Comments
MT	86399	11:59	Generic time format
MTS	86398	11:59:58	Time w/ seconds
MTH	86397	11:59PM	Time w/ AM/PM indicator
MTHS	86396	11:59:58PM	Time w/ AM/PM indicator and seconds

OE Alpha Conversions

Conversion Code	Input Value	Output Value	Comments
MCU	test info	TEST INFO	Upper case
MCT	testing, testing	Testing, Testing	Mixed (text) case
MCL	INFORMATION	information	Lower case
MCX	500	353030	Hexidecimal
MCA	123ABC	ABC	Return only alphabetic characters
MC/A	123$ABC	123$	Return anything but alphabetic characters
MCN	123ABC	123	Return only numeric characters
MC/N	123$ABC	$ABC	Return anything but numeric characters
G1*1	123*ABC	ABC	Group extract: Skip one part as delimited by the asterisk and return one part
G0*2	123*ABC*DEF	123*ABC	Group extract: Skip no parts as delimited by the asterisk and return two parts
T3,2	123*ABC	3*	Text extract: Extract 2 characters starting at character 3

OE Numeric Conversions

Conversion Code	Input Value	Output Value	Comments
MR2	100	1.00	Mask right; 2 decimal places
MD2	100	1.00	Mask decimal; 2 decimal places. MR2 and MR2 typically differ only in how null is handled. MR2 handles null as zero. MD2 handles null as null.
MR2#8	100	*bbbb*1.00	Mask right; 2 decimal places, in a field of 8 spaces. (*bbbb* represents 4 spaces)
ML4	10	.0010	Mask left; 4 decimal places
MR%5	15	00015	Mask right justified in a field of 5 zeros
MR2%5	150	01.50	Mask right justified in a field of 5 zeros; 2 decimal places
ML2#9	100	1.00*bbbbb*	Mask left in a field of 9 spaces; 2 decimal places

As stated before, there are many other conversion codes supported on various systems. Rather than list them all here, suffice it to say you can find this information in the manuals for your own operating environment.

Using Derived Conversions

A derived conversion is an SB+ **expression** that is entered in parentheses in the Conversion prompt on a field definition, screen definition, or report definition. Any expression used in this way can get the input value to be converted from the **common variable** @VALUE, and is responsible for returning the formatted value in the same place.

Anything that an expression can do can be done for the conversion. The following sections describe some of the more common techniques.

Translating a Value to Another File
Totaling a Multivalued List
Translating From a Table
Calling a Process to Convert a Field
Calling a Process to Hide a Field Conditionally

Input Conversions

Normally, when a numeric value is entered it is updated into the record exactly as it was entered. Therefore, if 1234 was entered into a numeric field with a conversion code of MR2, the value will be output as 12.34. This happens because the literal 1234 is stored in the record, and the conversion code implies two decimal places. In many cases, if we enter 1234, we really want 123400 to be inserted into the record, so that 1234.00 will be displayed. This is accomplished using an *input conversion*.

The role of an input conversion is to convert the value as entered prior to it being updated into the record. Typically, this input conversion will be one of a handful of valid OE conversions, as follows:

Conversion	Purpose
MCU	Convert the value to upper case before storing it in the record.
MR2 or MD2	Scale the value by 2 decimal places (to the right) before storing it in the record.
MT	Convert the entry to a valid time format before storing it in the record.

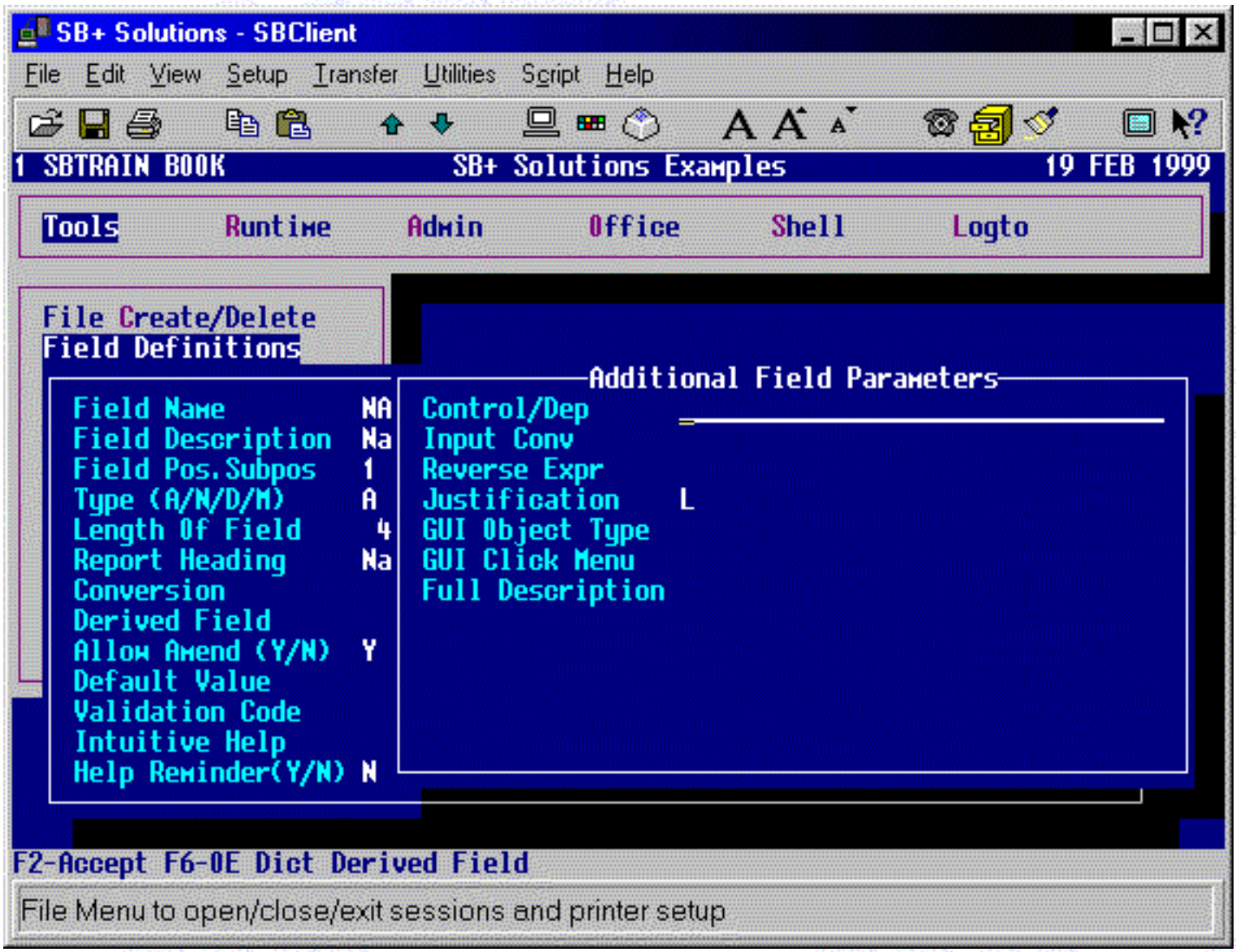
Other conversion codes can be used for the input version, of course, but these are the codes that are most widely used.

It is interesting to note that an input conversion is required for a time field, but is ignored on a date field. Date fields automatically convert the input, so putting an input conversion on a date field is unnecessary.

Installing an Input Conversion

An input conversion can be installed via one of two means, both available in the Field Definitions tool. (For purposes of clarity in this discussion, the Conversion prompt in this tool will be referred to as the output conversion.)

If the input conversion is used alone or is different from the output conversion, press F6-Addit on the Field Definitions tool when a field is being added or edited. This will bring up a window of additional information that appears as follows:



The field labeled Input Conv is where you can enter the input conversion for these types of fields. For example, if you want a name to be stored in upper case, but displayed in mixed case, use an input conversion of MCU with an output conversion of MCT. This will cause an entry of "john" to be stored as "JOHN" but output as "John".

If the input conversion and output conversion are the same code, however, there is a shortcut you can use to save the hassle of having to call up this extra F6-Addit screen. Simply add the literal "[I" (quotes shown for clarity) to the end of the output conversion and SB+ will recognize this as a command to make the input and output conversion identical. Therefore, if a monetary field with two decimal places is to scale the number by 2 decimals on the input and display the value with 2 decimals on the output, the following code can be used:

MR2[I

Another popular combo-conversion like this uses the upper case conversion MCU. This conversion is very useful for code-type entries that are validated against a table or against another file, where the key values or table entries will always be in upper case. If the user enters the right value but in the wrong case, this code will ensure that the value is in the right case before being validated and updated into the record:

MCU[I

This illustrates another big difference between input conversions and **output conversions**. Output conversions are solely aesthetic devices -- they change the way we see the information. Input conversions, on the other hand, are functional devices, changing the way the information is *stored*.

Reverse Conversions

When writing software for multiple countries and currencies, there is often a discrepancy between countries regarding the number of decimal places for monetary amounts. America, Canada, the UK, and a number of other countries use 2 decimal places in their currency figures. Korea, Japan, and a number of other countries, however, use no decimal point -- monetary value is expressed in whole numbers only.

Up until SB+ version 3.x, there was no way to write an application which used a different monetary conversion for each country. Instead, to make software use different conversions in different countries, mountains of modifications were required to change all conversions from one country's standard to another throughout the application. Fortunately, we now have a better way.

On a field definition, the (Output) **Conversion** and **Input Conversion** work together; The Input Conversion converts the value entered by the user into the proper format for storage, and the output conversion converts the stored value to a value suitable for display/reporting. Therefore, if I have an input conversion of MD2 (Mask Decimal at 2 places) and an output conversion of MD2, when the user enters the value "5", the input conversion converts it to "500" and updates the record. The output conversion then takes the 500 from the record, reformats it using the output conversion, and the user sees "5.00". Or at least that's what it appears to be doing...

The reality of the situation is actually quite different. To understand why this is so, we first need to understand all of the different times when a value is displayed:

- First, when the user enters a key value, the record is read, fields are extracted for display, and the output conversion for each field is used to display the field in its proper format.
- When the user enters a value at a field with the conversion, the input conversion is used to scale the entered value before the value is updated into the record, as illustrated above.
- When the user moves the cursor to a field with the conversion, the input conversion is used to "reverse" the input converted value to display a reasonable value for editing.

This last step is where it gets weird. Why does SB+ convert a value for output using the input conversion?

When the cursor moves to a field, SB+ looks at the input conversion to get an idea of how the value was converted when it was updated into the record. By applying the input conversion to convert the field for output, SB+ is attempting to reverse the input conversion, thus showing the user the value they originally entered. At least that's the theory...

This becomes increasingly complicated when the conversion itself is an expression. When creating software for international use that must consider local currency formats, you cannot simply use an MR2 or MD2 conversion for monetary amounts. Instead, the program must extract the appropriate number of decimal places from somewhere in memory (or from disk), and use this in the conversion, as follows:

```
(OCONV(@VALUE,"MR":@PARMS(1)<12>))
```

In this example, the contents of @PARMS(1) attribute 12 are concatenated to the conversion prefix "MR". Therefore, if @PARMS(1)<12> = 2, the conversion is MR2. If @PARMS(1)<12> = 3, the conversion is MR3. The calling process, therefore, needs only to update @PARMS(1)<12> with the number of decimal places appropriate to the country (or transaction).

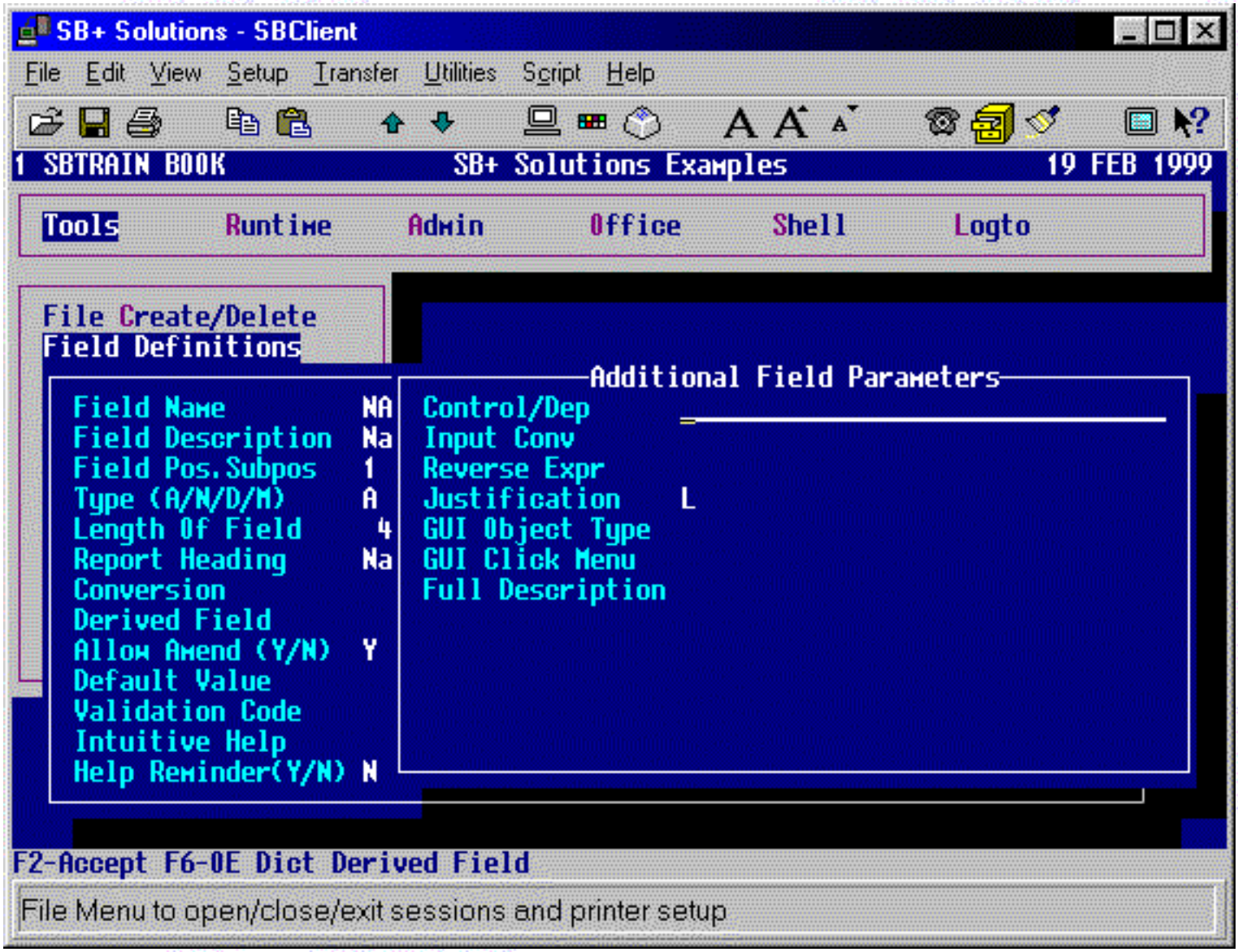
But we're not done yet. We need to put a comparable input conversion on the field so that when I enter 5 in the USA, Canada, UK, or other 2-decimal country, 500 is updated in the record and 5.00 is displayed, and when I enter 5 in Japan, 5 (without

decimals) is displayed. This can be done with an input conversion of:

(ICONV(@VALUE,"MR":@PARMS(1)<12>))

But we're still not done. We need to define a separate *reverse conversion* which is used to convert the value for display when the cursor moves to the field. As stated earlier, this conversion will be applied as an output conversion to *reverse* the input conversion. Fortunately, for most situations, repeating the output conversion will suffice.

Reverse Conversions are entered on the F6-Addit screen on a field definition, as follows:



For this example, we can simply repeat the output conversion on the Reverse Expr field. Therefore, in the end we have the following conversions:

- Input Conversion: (ICONV(@VALUE,'MR':@PARMS(1)<12>))
- Output Conversion: (OCONV(@VALUE,'MR':@PARMS(1)<12>))
- Reverse Conversion: (OCONV(@VALUE,'MR':@PARMS(1)<12>))

Make special note of the enclosing parentheses in each of these expressions. If we forget these parentheses, SB+ will execute the

conversions as OE conversions, and all sorts of strangeness will result.

Note that the calculation of all three conversions can be much more complicated than what is shown here. Processes could be called, records could be read from a number of different files, conditions could be tested, etc. Everything you may need to calculate literally any conversion is available using **paragraphs** and/or **BASIC**.

Reading A Description From Elsewhere

One of the most widely used derived value **expressions** in SB+ uses the F(...) function to translate a value from another file. For example, if we have an order screen that prompts for a customer ID, we can create a derived value field called CUSTOMER.NAME that reads the CUSTOMERS file and retrieves the name from attribute 1 as follows:

```
(F("CUSTOMERS",CUSTOMER.ID)<1>)
```

This derived value basically says "read a customer record using the CUSTOMER.ID field as a key, and return attribute 1." This CUSTOMER.NAME prompt can then be placed on the screen immediately after the CUSTOMER.ID prompt. With these prompts on the screen in the right sequence, once a valid customer ID has been entered, the customer's name will be displayed.

This technique can also be used to read information from SB+ security files, such as DMSECURITY. If a record in a file has a field called ENTERED.BY, which has the ID of a valid SB+ user, the following derived value can be used to get the short name of the user from DMSECURITY:

```
(F('DMSECURITY','~':ENTERED.BY)<21>)
```

This particular type of derived value can also be used to create a global equate name called USER.NAME. To do this, enter /EXP.EQU to start the **Global Equates** tool, and enter the following:

Equate Name	Expression
USER.NAME	F('DMSECURITY','~':@USER.ID)<21>

Once this has been defined, you can reference the name USER.NAME throughout the software, and SB+ will use the derived value as entered here to read a record from DMSECURITY, returning the short name from attribute 21.

In summary, to read a value from another file, create a **derived value field** using the F(...) function.

Reading From Multiple Elsewheres

Using the F(...) function in a derived value **expression**, it is easy to read a record from another file and extract a value to be displayed. But what can you do when the record may exist in one of several files? For example, a record may exist in a current file, or any number of historical files. What can we do when this happens?

To solve this particular dilemma, consider the following paragraph, called FIND.RECORD. It is called with the following derived value:

(P('FIND.RECORD,':*fieldname*:',*file1*,*file2*,*file3*...')<*amc*>)

In this syntax, *fieldname* is the name of the field containing the key to the record to be found. The parameters *file1*, *file2*, and following are a list of files to search. Lastly, *amc* is the attribute number to be extracted. (The ellipses indicate that any practical number of files can be specified.)

To create this process, enter /PD.P at a menu or entry screen prompt, enter FIND.RECORD as the process name, and enter the following paragraph lines:

```
LOCAL FIND.KEY,FILES,NDX,MAX
*
FIND.KEY = FIELD(@PARAM,',',1)
FILES = @PARAM[COL2()+1,32200]
*
NDX = 1
MAX = DCOUNT(FILES,',')
*
@RTN.FLAG = 1
WHILE (NDX <= MAX) AND @RTN.FLAG DO
  @RTN.FLAG = 0
  READ @VALUE FROM FIELD(FILES,',',NDX),FIND.KEY
  NDX = NDX + 1
REPEAT
*
IF @RTN.FLAG THEN
  @VALUE = ''
  @PARMS(2)<1> = ''
END ELSE
  @PARMS(2)<1> = FILES<1,NDX-1>
END
EXIT 0
```

Let's take a closer look at this paragraph...

First, because all the parameters are passed into the paragraph in the **common variable** @PARAM, we first move these values into **local variables** so we have our own copy to work with. This allows us to split the parameters into two separate variables (FIND.KEY and FILES), which makes the values much easier to work with inside of the paragraph. See **Chapter 8** for more information about the FIELD(...) and COL2() **functions** used here.

Next, we initialize a local variable named NDX to 1 and set a variable called MAX to the number of files that were passed. These counters will then keep track of which file we're reading inside of the loop.

The @RTN.FLAG =1 immediately outside the loop is used to prevent the loop from exiting on the first iteration. As long as a list of files has been specified (i.e. MAX is greater than 0), the loop will then attempt to read a record from each file in sequence.

Inside the loop, we first initialize @RTN.FLAG to 0. This may seem a bit odd, considering we just set the flag to 1 outside of the loop. This is necessary, however, because of the way that the **READ** statement updates this variable. If the read attempt is unsuccessful, @RTN.FLAG is set to 1. However, if the read is successful, the value of @RTN.FLAG is unchanged. Therefore, if we didn't initialize this variable prior to the read and the record was found, we'd never know.

The loop will continue until either we exhaust the list of files or the record is found in one of the files. In the event that the record is found, we return the record in @VALUE, because that is where the derived value expression expects the return value from the process call. As a side benefit, I'm also stuffing the name of the file where the record was found in @PARMS(2)<1>, just in case I might need it later. On the other hand, if the record is not found, we clear @VALUE and @PARMS(2)<1>.

Finally, the **EXIT** 0 at the bottom of the paragraph simply says to exit the paragraph and set @RTN.FLAG to 0. This allows the paragraph to be called in a variety of different contexts without regard to whether a non-zero @RTN.FLAG is significant. With this one little line, the paragraph can be called as a derived value, a conversion, a default value, or even as part of an E: type **validation code**!

Creating a Table Derived Value

The TABLE(...) **function** is very handy for translating a code into a description. It is unfortunate, then, this is the only function in the **expression language** that cannot be used directly in an SB+ **derived value**.

To use the TABLE(...) function, the variable @VALUE must be set to the code to be translated. This is acceptable for **conversions**, where @VALUE has a value prior to the evaluation of the expression, but the derived value doesn't have such a starting value to work from. In fact, the derived value must assume @VALUE is undefined, and is also powerless to change that fact. (There are no features of the expression language that allow the contents of a **common variable** to be changed.)

However, the derived value does have the ability to call a process, and that process can do pretty much anything you want. Therefore, we can create a **paragraph** called TABLE which uses the TABLE(...) function, translating whatever value is passed as a parameter. This then gives us the ability to write derived values that use the TABLE(...) function -- thereby rolling over another perceived roadblock in SB+.

The code for the TABLE paragraph appears as follows:

```
@VALUE = FIELD(@PARAM,',',1)
@VALUE = TABLE(@PARAM[COL2()+1,32200])
```

To call the TABLE process, one of the following derived values could be used:

(P('TABLE,*findvalue*,*tablename*'))
or
(P('TABLE,':*findexprn*:',*tablename*'))

For both derived values, the name of the table is passed in the place of the tablename token. The two expressions differ, however, when it comes to the value to be found.

The first derived value passes a fixed value in *findvalue* (entered inside of the quotes) to the process. This would cause the derived value to return the same value every time. In contrast, the second derived value references an expression in *findexprn*. This allows the contents of a given field (or more!) to be searched for in the table, which is far and away a more practical application of the technique.

It is also possible to vary the name of the table in the expression, though this is not very practical for most applications. One should also note that this particular paragraph can read tables that are stored in literally any available file, using one of the following syntax variations:

(P('TABLE,*findvalue*,*tablefile*,*tablename*'))
(P('TABLE,':*findexp*rn:',*tablefile*,*tablename*'))
(P('TABLE,':*findexprn*:',':*tablefileexprn*:',':*tablenameexprn*))

Calculating Extended Totals

On a typical sales order screen, you may have prompts such as the following:

```
Stock
Number Description..... ..Price Quantity Extended
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
```

In this context, the Extended field should be calculated as Price multiplied times Quantity. If the fields are actually named PRICE and QUANTITY, for example, we can build a **derived value** **expression** for the Extended field as follows:

```
(PRICE * QUANTITY)
```

What makes this particular expression so powerful is that it automatically supports multivalued prices and quantities. No matter how many prices and quantities you enter into the screen, the Extended field will always calculate each and every one separately.

To understand this better, we need to take a close look at how the expression language views field names. If an expression contains a field name such as NAME, and NAME is a field definition for attribute 4 of the current record, the expression language interprets NAME to be @RECORD<4>. For a multivalued field, however, the rules are different. For example, if an expression contains a field called PRICE which points to a multivalued list in attribute 10, the expression language interprets this field name to be @RECORD<10,@CNT>. Therefore, if PRICE is attribute 10 and QUANTITY is attribute 11, the above derived value will actually be interpreted as:

```
(@RECORD<10,@CNT> * @RECORD<11,@CNT>)
```

Feature or Bug?

The implied @CNT feature with multivalued fields in the **expression language** is very handy when faced with the calculation of extended totals, such as in the previous example. There are times, however, when this particular feature can be a real pain.

Note that the expression language serves as the basis for the **paragraph language**. Therefore, if @CNT is implied when referencing multivalued fields in an expression, it is also implied when multivalued fields are referenced in a paragraph. Herein lies the (potential) problem.

Let's assume we're creating a sales order entry screen, and on this screen somewhere we have a Stock number prompt, such as the example on the previous page. When F2 is pressed to save an order, we want to check the stock number prompt to ensure that some products have been ordered. To do this, we'll need to create a Process After Screen Accept which verifies that the stock number field has a value.

One might be led to believe that the following paragraph will work:

```
IF (STOCK.IDS = '') THEN EXIT 1
```

Oddly enough, this paragraph does work, but not all of the time. If F2 is pressed while the cursor is on any single valued prompt on the screen, this will work as expected. However, if the cursor is on the third stock number prompt after two stock numbers have already been entered, this paragraph will fail. To see the reason, let's look at what SB+ sees when this paragraph is executed:

```
IF (@RECORD<10,@CNT> = '') THEN EXIT 1
```

If we press F2 when the cursor is on the third stock number prompt, @CNT is equal to 3, and therefore this expression checks to see if the @RECORD<10,3> has a value, not @RECORD<10>, as we would expect. However, if we press F2 on a single valued prompt, @CNT is zero, and @RECORD<10,0> is equivalent to @RECORD<10> -- therefore the process works as expected.

Always remember: When referencing multivalued fields by name in a paragraph, you must always be aware of the value of @CNT, or you must reference the field a different way.

Most people don't like having to keep track of @CNT at all times, so the SB+ folks provided a function called POS(...) which is incredibly useful for overcoming this feature. The syntax for POS(...) is as follows:

```
POS(fieldname)
```

In this syntax, *fieldname* is the name of a field in this file. POS(...) will simply look up the attribute number in the field definition and will return that value. Therefore, to reference the STOCK.IDS field in its entirety, the paragraph could be rewritten as follows:

```
IF (<POS(STOCK.IDS)> = '') THEN EXIT 1
```

In this syntax, the <...> is a reference to an attribute in @RECORD. Using the POS(...) function, SB+ looks up the field position and therefore makes the expression evaluator see the expression as if it had been written as follows:

```
IF (@RECORD<10> = '') THEN EXIT 1
```

As a rule, if you want to reference a particular value in a multivalued field, simply use the field name itself. If you want to reference the multivalued attribute as a whole, use the POS(...) function.

Summing Extended Totals

Let's assume we've created a sales order entry screen with a section that looks like the following:

```
Stock
Number Description..... ..Price Quantity Extended
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
```

Below the Extended column, we should probably include a sum of all of the extended totals, so we can keep a running total of the value of the order. This is a fine idea, but the Extended field is a derived value. How can SB+ add values that don't really exist anywhere?

However it accomplishes the feat, SB+ **does** support derived values that are based on other derived values. With this example in particular, if the Extended column is a field called EXTENDED, the following **derived value** **expression** can be entered for a field called TOT.EXTENDED to total all of the values in the field:

```
(S(EXTENDED))
```

To calculate this field, SB+ counts the number of prices and quantities, recalculates each extended total, and then accumulates this total into the sum. This may seem like a bit of extra work, but it all happens so fast it's nearly invisible. Once this field has been created in the Field Definitions tool, it can be placed below the Extended column on the screen.

Simply placing the field on the screen this way is not enough to make the screen work as expected, however. Because of the way SB+ traverses through a prompt sequence, the Total field will not be encountered until after all of the stock numbers have been entered. Additionally troublesome is the fact that the Extended column will not be recalculated if the quantity changes and the user uses the up arrow to go back to the stock number.

To correct these problems, we need to tell SB+ that the Extended and Total fields are Recalc fields. See "**Creating an Automatic Recalculation Prompt**" later in this chapter for more information about recalc fields.

Calling a Process

Like any SB+ **expression**, a process may be called as a part of the derived value expression, or as the entire **derived value** expression. This allows you to do just about anything to calculate a value.

For example, if we have an order entry screen where a process is called to calculate the order net, another process is called to calculate the tax amount, and still another process is called to calculate the freight amount, an order total derived value could be created as follows:

```
(P('CALC.ORD.NET') + P('CALC.TAX,USA') + P('CALC.FRT'))
```

Or, if one process is used to calculate all of the fields together:

```
(P('CALC.ORD.TOTAL'))
```

It's important to note that any type of process can be called this way. If the calculation of a field requires a number of different types of processes, the P(...) function could call a paragraph which calls a screen which calls a periodic update which calls other paragraphs, and so on, and so on... The only thing the called process needs to be concerned with is the contents of the **common variable** @VALUE when the process is complete. Whatever is in @VALUE when the process terminates is the value that will be returned to the expression.

Calling a Subroutine

Like any SB+ **expression**, a BASIC subroutine may be called as a part of the derived value expression, or as the entire derived value expression. This allows you to do just about anything to calculate the value for a field.

For example, if we have an order entry screen where a subroutine is called to calculate the order net, another subroutine is called to calculate the tax amount, and still another subroutine is called to calculate the freight amount, an order total derived value could be created as follows:

```
(B('CALC.ORD.NET') + B('CALC.TAX,USA') + B('CALC.FRT'))
```

Or, if one subroutine is used to calculate all of the fields together:

```
(B('CALC.ORD.TOTAL'))
```

Note that in order for the B(...) function to work, the subroutine must be compiled and cataloged under the name as entered between the quotes.

Whatever is in @VALUE when the subroutine returns is the value that will be returned to the expression. See "**Using BASIC in an SB+ Application**" for more information.

Limiting the Length of a Field

Depending on the way your SB+ user security record is set up, all input prompts will work in one of two ways:

- The prompt will only allow up to a certain number of characters, or
- The prompt will scroll horizontally to allow you to enter any number of characters.

On a field by field basis, you can actually change this behavior -- but only slightly.

If you want a particular field to be restricted to 10 characters in length, you can either **validate** the entry (**as discussed earlier**) or you can tell SB+ to advance automatically to the next field after the tenth character has been entered. To tell SB+ to advance automatically, simply enter the field length in the field definition as a negative number. Therefore, if the field is to be restricted to no more than 10 characters, the field length should be entered as -10 in the field definition.

Creating An Automatic Recalculation Prompt

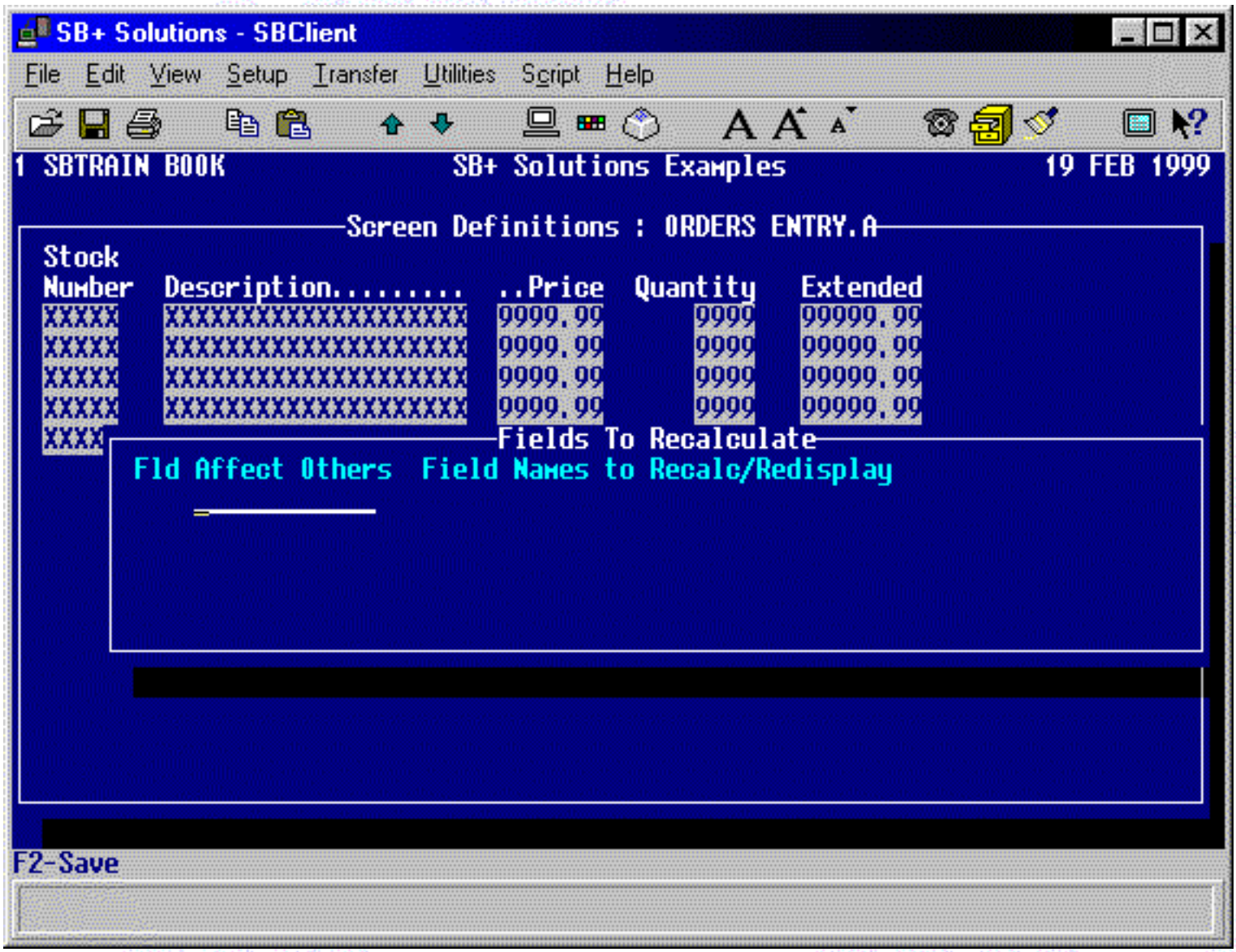
Let's assume we've created a sales order entry screen with a section that looks like the following:

```
Stock
Number Description..... ..Price Quantity Extended
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
                          Total xxxxxx.xx
```

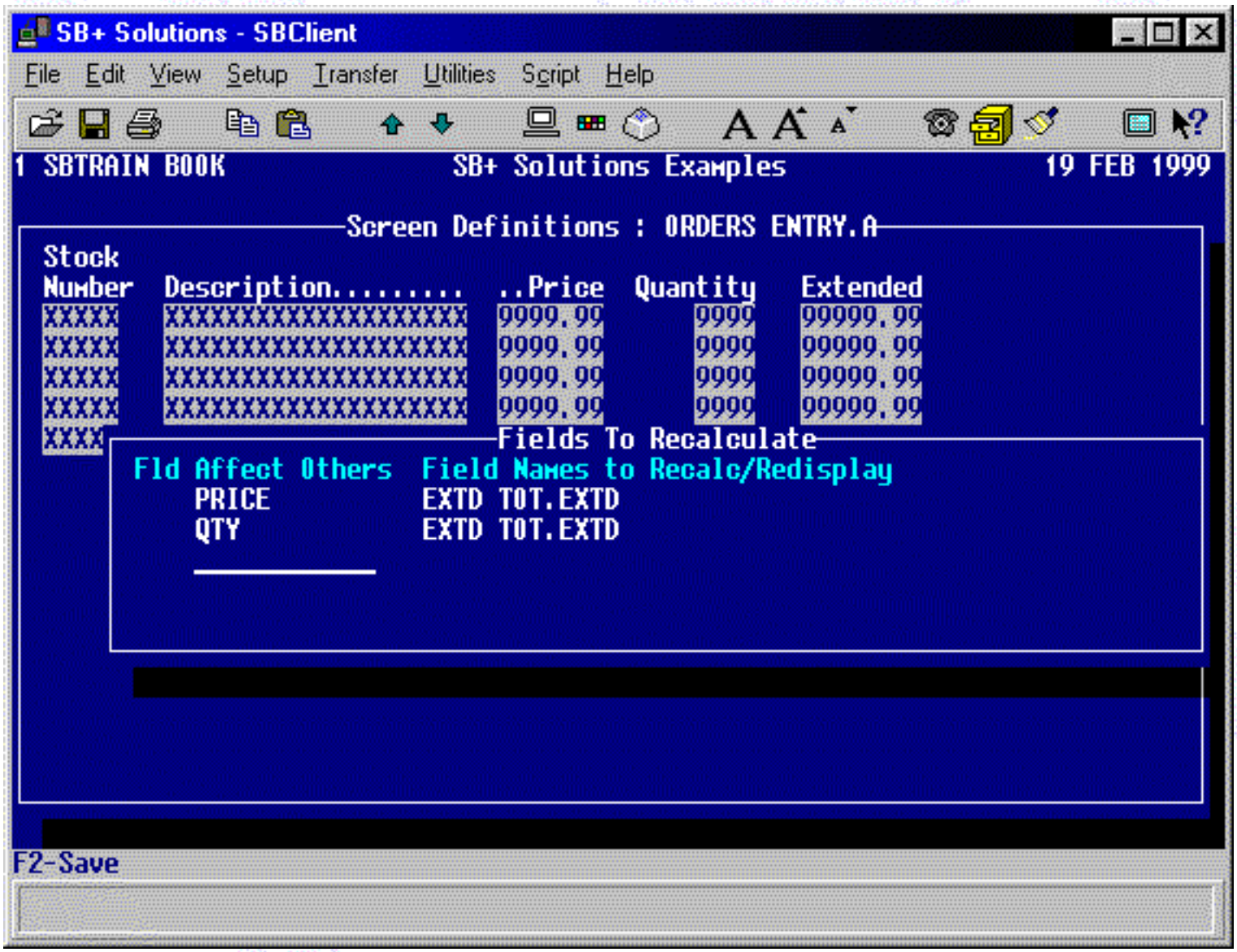
On this screen, Stock Number, Price, and Quantity are input fields, and using the price and quantity the Extended field is calculated for each line using a **derived value field**. Under the extended column is a field labeled Total, which totals the individual extended prices into one value. This is also done using a derived value field.

Simply placing the fields on the screen this way is not enough to make the screen work as expected. Because of the way SB+ traverses through a prompt sequence, the Total field will not be encountered until after all of the stock numbers have been entered. Additionally troublesome is the fact that the Extended column will not be recalculated if the quantity changes and the user uses the up arrow to go back to the stock number.

To correct these problems, we need to tell SB+ that the Extended and Total fields are *Recalc* fields. By definition, a recalc field is a prompt on a screen that is recalculated and redisplayed when some other value changes -- outside of the normal prompting sequence. To set up these fields this way, start the Screen Definitions tool, recall a screen to edit, and then press F10-Action to recall the action bar. From the action bar, select RecaLc by pressing the letter L. Once this has been done, the following screen will be displayed:



To use this screen, enter the name of one of the fields on the screen in the first column. The field must be an input field, and should also be directly related to the values being recalculated. For each value in the first column, enter one or more fields to be recalculated in the second column. Following our previous example, if the price, quantity, extended total, and total fields are named PRICE, QUANTITY, EXTENDED, and TOT.EXTENDED, respectively, the following setup could be used to recalculate the extended amount and total extended values when the price or quantity changes:



Always remember that the fields as listed in the first column must be input fields -- otherwise the recalculation will not work as expected.

Skipping Multivalued Fields

When setting up a **skip for a single valued field**, we can use a process called from the Process Before slot on the F5-Field screen in a screen definition. This process is responsible for determining if a prompt is to be skipped, and if so, setting the **common variable** @RTN.FLAG to either "S" or "1" to skip to the next or previous prompt, respectively. However, when skipping a multivalued prompt, this technique must be varied slightly.

If you set @RTN.FLAG to "S" when skipping a multivalued field, the entire set of multivalues are skipped, instead of skipping just a single prompt. Instead, setting the variable to "S0" ("S" zero) properly skips only one prompt and stays inside of the multivalued set. Therefore, the SKIP process can be enhanced to support both single and multivalued fields with the following code:

```
CASE(@OTHER(18) = 3) OR (@OTHER(18) = 25)
  EXIT 1
CASE @CNT
  EXIT S0
CASE 1
  EXIT S
END CASE
```

Note that the logic to skip to the previous field is essentially unchanged. Whether the field is single valued or multivalued, setting @RTN.FLAG to a 1 will skip back to the previous field.

The common variable @CNT, referenced on line 3 of this paragraph, contains the multivalue number for the current prompt. If @CNT equals zero, the prompt is single valued. If the prompt is multivalued, @CNT will be greater than zero. You could also check @LINE<1,2> -- if this variable is negative, the field is multivalued.

Limiting the Number of Multivalues

In certain applications, you may want to limit the number of multivalues that can be entered into a given prompt. For example, if a customer entry screen has a multivalued address prompt, you may want to restrict the address to no more than three lines. Or, if you're in the mood to be exceptionally clever, you could check to see what country the address is for, and then limit the number of multivalues based on a defined maximum for that country.

There are a couple of techniques available to accomplish this.

One way to limit the number of entries into a multivalued field is by calling a process prior the entry of each value in the prompt. This process checks to see if too many values have been entered, and corrects the situation and exits the prompt if necessary.

One might be led to believe that such processing before the prompt should be called from the Process Before on the F5-Field screen in the screen definition. This, however, is not the right process slot. On multivalued fields, the Process Before is executed once before the first value in the entire multivalued field, not once for each value. Therefore, any process entered into this slot is ignored for the second and subsequent values in the set.

Oddly enough, only one process slot is executed prior to the entry of each value in a multivalued prompt. Then again, it's not really a process slot at all.

The **default expression** for a field (as entered on the field definition) is the only place where a process can be called prior to the entry of each value in a prompt. Using the P(...) function, a process can be called to limit the number of values in the prompt. Such a process would then be called from the default expression as follows:

```
P('LIMIT.MV,3')[M]
```

This syntax simply says to call a process called LIMIT.MV, passing a parameter of 3. This parameter then determines the maximum number of multivalued lines to be allowed.

Also note the [M] on the end of the expression. This suffix tells SB+ that this is a **mandatory default**, and should be executed for each value in the field, regardless of whether the field has a value.

The LIMIT.MV process is a quick little paragraph that appears as follows:

```
IF (@CNT > @PARAM) THEN DATA "\"
```

This basically says "If the current multivalue counter is greater than the maximum number of multivalues allowed for this entry, data stack a backslash (to clear the entry) so that the extra value(s) will be deleted." Once all of the extra values have been deleted, the backslash key will tell SB+ to exit the prompt and move to the next one.

As long as new values are added to the end of the list, this technique works as expected. However, if the Insert Line key is used to insert a new value at the top of the list, this particular process does nothing to prevent the new value from being added. Only if the cursor moves past the last prompt will this particular process do anything to clean up the extra values. Therefore, a second process must be created and called as a validation process on the field to prevent the field from growing past the defined maximum. This process could be called as follows from the validation code:

C:TRIM.MV,3

Like the default process, this process also needs a parameter which defines the maximum number of lines. This allows the one process to be used over and over for validation on any number of fields. The actual text of the paragraph TRIM.MV appears as follows:

```
LOCAL AMC
*
AMC = @LINE<1,1>
IF (AMC GT 0) THEN
  WHILE <AMC,@PARAM + 1> # '' DO
    @RECORD = DEL(@RECORD,AMC,@PARAM + 1)
  REPEAT
END ELSE
  WHILE @WORK<0-AMC,@PARAM + 1> # '' DO
    @WORK = DEL(@WORK,0-AMC,@PARAM + 1)
  REPEAT
END
```

This particular paragraph must account for a variety of different factors. First, the field being limited may be in either @RECORD or @WORK. Simply by checking the attribute number we can tell which it is. Positive attribute numbers reference attributes in @RECORD. Negative attribute numbers reference attributes in @WORK. Second, the incoming record as it was read from disk may have several attributes past the defined maximum for this screen. This is actually pretty typical when a new application is created to manipulate old data. To accommodate that contingency, the process loops, deleting the value immediately following the defined maximum until there is no longer a value in that position.

While this does limit the number of multivalues that can be entered into a prompt, it may seem a bit barbarous to quietly delete the extras, if any, that are at the end of the list. If you simply want to skip out of the list and clean up the extras later, perhaps the following syntax in place of LIMIT.MV would be a little more palatable:

```
IF (@CNT > @PARAM) THEN EXEC 'SKIP'
```

This particular variation will execute the SKIP process which will skip to the next field without deleting the extra values. Some other process, such as a process called from the Process After Screen Accept, will then need to remove the extraneous values. Such a process might look like this:

```
LOCAL AMC,LIMIT
*
AMC = FIELD(@PARAM,',',1)
LIMIT = FIELD(@PARAM,',',2)
*
IF (AMC GT 0) THEN
  WHILE <AMC,LIMIT + 1> # '' DO
    @RECORD = DEL(@RECORD,AMC,LIMIT + 1)
  REPEAT
END ELSE
  WHILE @WORK<0-AMC,LIMIT + 1> # '' DO
    @WORK = DEL(@WORK,0-AMC,LIMIT + 1)
  REPEAT
END
```

While similar to the validation process discussed earlier, this particular process takes both the attribute number and multivalue limit in the parameter line. Such a process could be called from the Process After Screen Accept as follows:

```
LIMIT.MV,21,3
```

Or, this process could be called from a separate paragraph called from this same slot, using:

```
EXEC 'LIMIT.MV,':POS(ADDRESS):',3'
```

This latter syntax is preferable, as it does not require the attribute position to be hard coded. One must be aware, however, that POS(...) returns a positive value for both @RECORD and @WORK fields, so if an @WORK field is being referenced, the following variation must be used:

```
EXEC 'LIMIT.MV,':(0-POS(W.ADDRESS)):',3'
```

Of course, the ADDRESS and W.ADDRESS fields are simply field names for the example. In your own programming, you would substitute real names of the fields to be limited for these example field names.

Hopefully, this illustrates that there are almost always several sides to any given solution to a problem in SB+. The solution you choose will always be a product of your own development standards, methodologies, and style.

Updating Hidden Multivalues

With certain applications, you may need to update multivalued fields in the record without actually having the prompts displayed on the screen. These fields are essentially hidden from the user, but the application must keep them updated nonetheless.

For example, consider we're creating a sales order screen with a multivalued set of prompts that appears as follows:

```
Stock
Number Description..... ..Price Quantity Extended
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
```

For each stock number that is entered, let's assume we want to extract a commission percentage from each inventory record and store the percentages in the order record in tandem with the stock numbers. This information will then be used by some later program to calculate the amount of commissions to be paid to the person who made the sale. (We'll assume the salesperson will be identified elsewhere on the order). Note that we don't want this information to be displayed to the user of this program, as this type of information is of no value to someone entering the order. However, this information is still needed by some later program, so it must be updated into the record as each stock number is being entered.

To do this, we first need to create a field where the hidden values will be stored. In this particular example, let's assume we've created a field called COMMISSION.PCTS which is defined as multivalued attribute 14 in the order record.

Once the field has been defined, we need a process to load the commission percentage from the inventory record. The best place to call such a process would be from a validation code on the stock number prompt, because this is probably going to be where the inventory record will be first read.

(For this particular field, we need to validate the record exists in the inventory file. Therefore, why not write a process which reads a record from the inventory file and extracts the commission percentage if the record is valid?)

Such a **validation process** could be a paragraph that appears as follows:

```
READ @OTHER.REC FROM 'INVENTORY',@VALUE
IF @RTN.FLAG THEN
  ERROR "This is not a valid stock number"
  EXIT 1
END
                *
COMMISSION.PCTS = O14
```

In this paragraph, we first attempt to read a record from the INVENTORY file. If the read is unsuccessful, we display an error message (with **ERROR** ...) and tell SB+ to reprompt for the entry (**EXIT** 1). However, if the record is found, we extract attribute 14 of @OTHER.REC and update it into the current multivalue position of the COMMISSION.PCTS field.

Note that we don't reference a specific multivalue number on COMMISSION.PCTS. Because this is a multivalued field, the value number is implied, based on the value of the **common variable** @CNT. For more information, see "**Feature or Bug?**"

elsewhere in this chapter.

In contrast to this approach, we could also call this process from the Process After slot on the F5-Field screen on the screen definition. This would then allow us to use the F: validation code to verify that the record exists on the file. Unfortunately, this would cause the inventory record to be read twice for the prompt, which is unnecessary I/O. (The F: code reads the inventory record once, and the Process After field would read the record again.) To compensate for this problem we could call a validation process on the field which contains the first 5 lines of the paragraph shown above, and have a separate process called from the Process After which uses the last line of this paragraph. This, too, is one possibility, but then two separate processes are created when the entire job can be done in one.

For this particular technique it is most effective to use the validation process both for validation and to load values from the record. In fact, any time you are faced with a validation code where an F: code seems appropriate, ask yourself if you may need values extracted from the record. If the answer is "yes", or even a "maybe", it is probably best to forego the F: and create a process to validate the field instead.

Once you've set up the validation process to update the field, the next thing we need to do is tell SB+ that if the user inserts a blank line or deletes a line from the multivalued set, the hidden value must be updated as well.

On the F5-Field screen for the controlling multivalue in the set, the Control/Dep/Read field is normally the letter "C". To tell SB+ to update a field in the record that is not on the screen, you will need to enter the attribute number of the hidden field following the "C", separated by a comma. Multiple attribute numbers can also be entered, each separated by a comma. Back to our commission percentage problem, the Control/Dep/Read field on the stock number prompt on this screen should read:

```
C,14
```

This says that the field is a controlling multivalue, and attribute 14 (which we defined as COMMISSION.PCTS) is to be updated when a value is inserted into or deleted from the multivalued stock number prompt.

In summary, by creating a process to update the hidden value and also telling SB+ to keep the hidden values in sync with the controlling multivalue, we can effectively maintain a group of multivalued fields -- whether we see them on a screen or not.

Updating Humongous Multivalue Sets

In certain applications, the set of multivalues is so large that all of the fields will not fit onto the screen at one time. This is inconvenient, no doubt, but does not necessarily impose a limit or restriction on entering values into the fields.

For example, assume we're creating a sales order entry application with a multivalued set of prompts that appears as follows:

```
Stock
Number Description..... ..Price Quantity Extended
xxxxxx xxxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
```

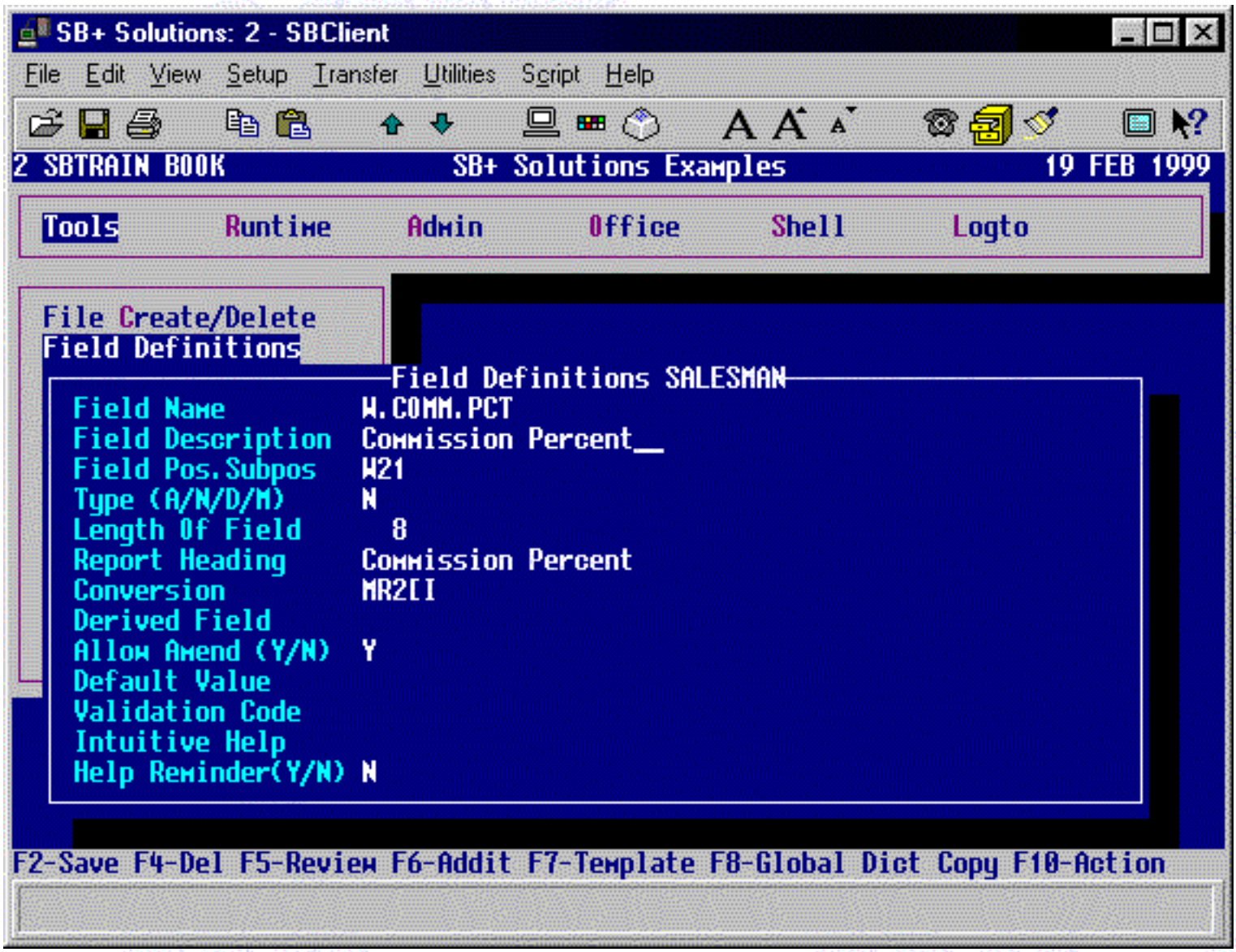
For each stock number, we need to store a price, quantity, and commission percentage, but there just isn't enough room to prompt for all of these. We could possibly shorten the description, but it's undoubtedly too short already.

Even though we can't fit the field on the screen, we can still prompt for the commission percentage for each multivalued line while maintaining the right prompting sequence. In other words, if the screen were wide enough to accommodate all of the fields on the screen, the screen should:

- Prompt for a stock number,
- Display the description,
- Prompt for a price,
- Prompt for a quantity, and
- Prompt for the commission percentage.

The fact that one of these fields does not fit should not affect this sequence of events -- it's just a little more challenging this way.

In order to use this particular technique, we first need to create a temporary field definition which we'll call W.COMM.PCT. This field definition is a single valued work field, meaning that the Field Pos.Sub Pos prompt in the field definition will be the letter W followed by some number, such as in the following screen:



This field definition says "instead of updating an attribute in the **common variable** @RECORD, update attribute 21 of the common variable @WORK with the value entered into this field".

Once the work field has been created, the next step is to create an input screen that prompts for the work field. This screen must be created as a typical entry **subscreen**.

Next, a **paragraph** must be created to call the subscreen, such as the following:

```
LOCAL HOLD.CNT
*
HOLD.CNT = @CNT ;* Save the multivalue position
W.COMMISSION.PCT = COMMISSION.PCTS
EXEC 'I*ORDERS*ENTRY.B'
@CNT = HOLD.CNT ;* Restore the multivalue position
COMMISSION.PCTS = W.COMMISSION.PCT
```

This paragraph first saves the current value of the @CNT variable, because the input process I*ORDERS*ENTRY.B will reset the value and we need the original value later in the paragraph.

Next, the paragraph moves the current value (pointed to by @CNT) from COMMISSION.PCTS to the single valued W.COMMISSION.PCT. Once this is done the commission percentage is in a variable that we can change with the entry screen.

Such is the purpose of executing the input process named I*ORDERS*ENTRY.B. (For the sake of this example, assume that this is the name of the process that runs the screen which prompts for W.COMMISSION.PCT.) Because we created this screen as a regular SB+ subscreen, it can edit W.COMMISSION.PCT and the changed value will be returned to the paragraph that called the screen.

We must always assume that the entry screen has changed the value of @CNT, and therefore we must reinstate its original value, as stated on line 6 of the paragraph. Once this has been done, we can move the contents of the work field into the appropriate multivalue of COMMISSION.PCTS, and the process is complete.

To put this all together, we simply need to link this paragraph into the Process After slot on the F5-Field screen for the Quantity prompt. With this in place, after the user has entered a quantity our subscreen will pop up, allowing the user to enter a commission percentage for the line. Once the commission percentage has been entered and accepted (by pressing F2), the cursor will advance to the next multivalued line and the user can continue adding or editing new values as necessary.

But there's more. As the application now stands, the user must enter a value into the subscreen and then press F2 to accept the value and advance the cursor to the next prompt on the main screen. For most applications, this is unacceptable. Instead, the cursor should advance to the next prompt on the main screen after we've entered the last value on the subscreen. To do this, we need to tell SB+ to press F2 automatically after the last field on the subscreen.

To tell SB+ to press F2 automatically after a prompt, simply enter **G:U** in the Process After slot on the field in the subscreen. Using our example, by entering G:U in the Process After slot for the W.COMMISSION.PCT field, when a value is entered into the prompt the subscreen will go away automatically, thereby eliminating the need for an F2 key on this screen.

Finally, we need to tell SB+ that the COMMISSION.PCTS field is to be updated, though it is not on the main screen. On the F5-Field screen for the controlling multivalue in the set (Stock Number, in this example), the Control/Dep/Read field is normally the letter "C". To tell SB+ to update the field in the record that is not on the screen, enter the attribute number following the "C", separated by a comma. In our commission percentage problem, the Control/Dep/Read field on the stock number prompt on this screen should read:

```
C,14
```

This says that the field is a controlling multivalue, and attribute 14 (a.k.a. COMMISSION.PCTS) is to be updated when a value is inserted into or deleted from the multivalued stock number prompt.

Finally, after all this is done, we have a screen that will prompt for additional multivalued fields while maintaining a proper prompting sequence. Of course, this is a very limited example -- using this technique, any number of extra multivalued fields can be updated on the subscreen.

Interleaving Multivalued Prompts

Another way to update large quantities of related multivalues is with a feature that SB+ calls *interleaved multivalues*. This particular feature allows the fields in a multivalued set to be placed on multiple rows on the screen, instead of a single row. This makes the screen display a little harder to follow, but this is quite a bit simpler to implement than many other options.

Let's consider how this could be used with the example from the previous section. To recap this example, we're creating a sales order entry screen which basically appears as follows:

```
Stock
Number Description..... ..Price Quantity Extended
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
```

For each stock number, we need to store a price, quantity, and commission percentage, but there just isn't enough room to prompt for them all. We could shorten the description or remove the extended total to make room for the extra field, but these fields are very important to the screen and would be missed if they were removed. Instead, let's open up a row between each of these rows, and put the commission percentage on the new row, such as:

```
Stock
Number Description..... ..Price Quantity Extended
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
Commission Percent xxxxx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
Commission Percent xxxxx
xxxxxx xxxxxxxxxxxxxxxxxxx xxxx.xx    xxxx xxxxx.xx
Commission Percent xxxxx
```

Regardless of what it looks like, the commission percent field is just a typical dependent multivalue. Like all dependent multivalues, such as Description and Price, the window size of the controlling multivalue carries forward to all of the dependents. It doesn't matter that the dependents may be on later rows -- as long as the prompt sequence is in the right order. See "**Resequencing Prompts on a Screen Definition**" for more information about prompt sequencing.

Implementing this type of a screen is remarkably simple, particular considering the other alternatives. To do so, we need to call up the F5-Field screen for the controlling multivalue in the screen definition and change the window size.

Normally, for a multivalued window, you simply enter the number of rows to be displayed for the set in the Window Size prompt. With interleaving multivalues, however, we need to add a suffix to the window size to tell SB+ how many extra rows to leave between rows. For example, our sample screen would have a window size of "3.1", which tells SB+ that three rows of values will be displayed at a time, and there is one extra row between each.

Finally, the words "Commission Percent" will need to be typed onto the screen definition for the three occurrences of this field. This is admittedly unattractive, but we can rest easy knowing that this is just one of many ways that the field can be labeled. The important thing is that the field is actually labeled, so the user knows what to enter at the prompt.

There are two ways to evaluate the effectiveness of this technique. On one hand, we can fit only half as many rows on the screen at any given time, and the repeated heading for the commission percent field isn't all that attractive. However, with this technique all of the information is on the screen simultaneously, and there are no hidden screens or fancy logic required to update the additional field. Whether this is a beneficial trade-off or not depends on your application and your own personal taste in screen cosmetics.

There are variations on this theme. For example, we could change the screen display to appear as follows:

```
Stock
Number Description..... ..Price Quantity Extended
xxxxxx xxxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
Commission Percent xxxxx
-------------------------------------------------------
xxxxxx xxxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
Commission Percent xxxxx
-------------------------------------------------------
xxxxxx xxxxxxxxxxxxxxxxxxxx xxxx.xx     xxxx xxxxx.xx
Commission Percent xxxxx
```

This particular sample screen uses a window size of 3.2, which skips two rows between each data row. With the extra row, we can add a dash line or graphic line to visually separate the individual values. While this makes the screen slightly more attractive, it requires a couple of extra rows. In short, unless the screen has lots of blank space, this will probably not be a workable option.

Restricting Multivalued Prompts Using Multivalue Restrictions

In certain applications, you may want to restrict a multivalued field from being changed in some way. By SB+'s definition, a multivalued field can be updated (and hence restricted) in one of four ways:

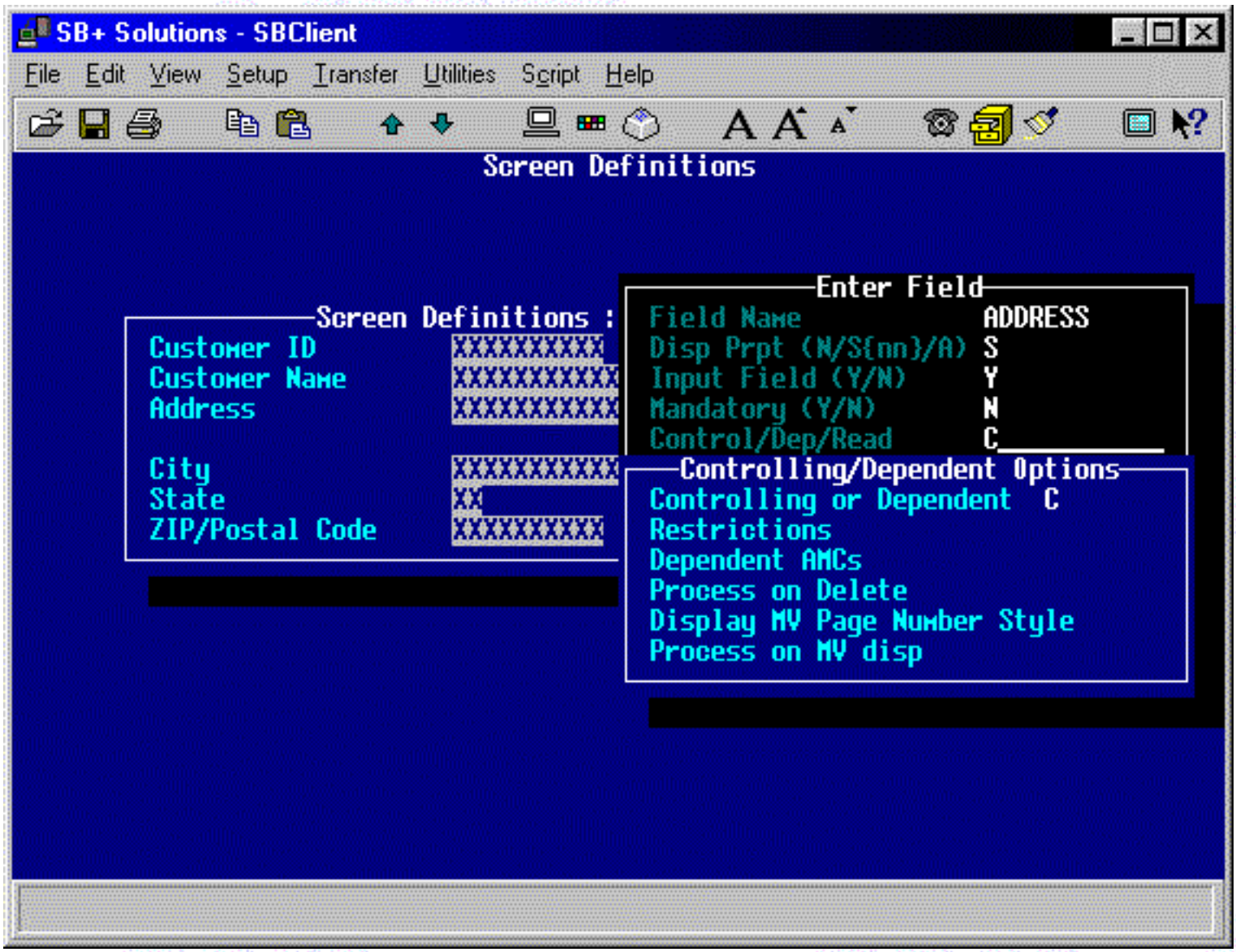
- A value can be inserted between two other values or at the top of a list,
- A value can be deleted from the list,
- A value can be changed in the list,
- A value can be added to the bottom of the list.

Using combinations of SB+ *multivalue restriction codes*, you can apply these restrictions to any controlling multivalued field on a screen.

On the F5-Field screen in the screen definition, the controlling multivalue generally has the letter "C" in the Control/Dep/Read prompt. To add restrictions to the controlling multivalue, one or more of the following codes can be entered after the "C":

I	Restrict insertion: If the user presses the Insert Line key when the cursor is on the controlling multivalue, SB+ will display an error message.
D	Restrict deletion: If the user presses the Delete Line key or enters a backslash when the cursor is on the controlling multivalue, SB+ will display an error message.
A	Restrict amendment: SB+ will not allow the controlling multivalue to be changed.
E	Restrict append: SB+ will not allow an entry past the last entry in the field.

If you press F3 on the "C" in this field, the following screen will be displayed:



If a particular multivalued prompt must have a constant number of values, the insertion, deletion, and append features should be restricted. To do this, simply enter CIDE in the Control/Dep/Read prompt. Or, to allow entries to be added to the bottom of an otherwise fixed list, CIDA could be used.

Various combinations of these restrictions are available, depending on the needs of your application. Also, the restrictions can appear in any order following the "C". Therefore, CIDE, CDEI, and CIED are all equivalent.

With SB+, all of the inserting, deleting, and appending of values must be done from the controlling multivalue. Therefore, these restriction codes are entered only on the controlling multivalue prompt. If you want to restrict amendment on a dependent multivalue, this can be handled in a **validation** process, such as the following paragraph:

```
IF fieldname # @VALUE THEN
  ERROR "message"
  EXIT 1
END
```

In this paragraph, *fieldname* is the name of the field that is being validated. (When we reference the field by name we are looking at the value stored in the record or work variable. In contrast, @VALUE contains the new value entered by the user.) While this will work, it is so specialized that one process will be required for each dependent field validated this way. If this technique is

widely used, you could have dozens of little processes like this, which some people may view as unnecessary clutter.

Alternatively, we can create one generalized process to validate any non-amendable dependent multivalue prompt, as follows:

```
LOCAL AMC,TEMP
*
AMC = @LINE<1,1>
IF AMC < 0 THEN
  TEMP = @WORK
  AMC = (0 - AMC)
END ELSE
  TEMP = @RECORD
END
*
IF TEMP<AMC,CNT> # @VALUE THEN
  ERROR "message"
  EXIT 1
END
```

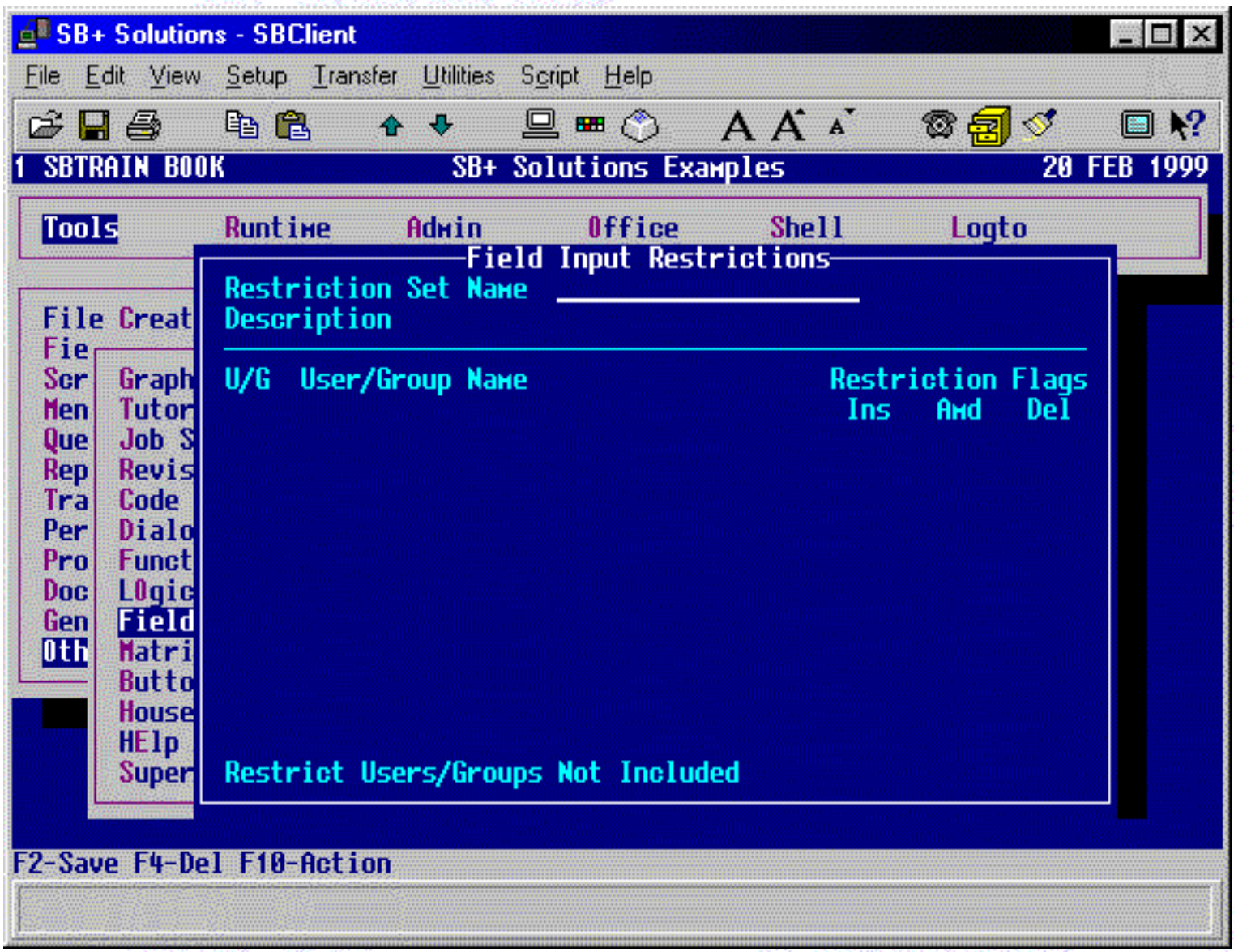
This paragraph does the same thing as the first paragraph, except that it calculates the location of the original value for the field, instead of hard-coding the value to be extracted from a specific attribute in either @WORK or @RECORD.

Using the restriction codes on the controlling multivalue, and validation processes on the dependent(s), SB+ provides a wealth of control over how these types of fields can be updated.

Restricting Multivalued Prompts Using Field Input Restrictions

Field Input Restrictions are codes which define which users and groups can insert, amend, or delete values from a particular field. For multivalued fields, field input restrictions can be used to control which users and groups can remove multivalues from a set. These restrictions cannot, however, be used to control which users can insert, amend, or append new values into an existing list.

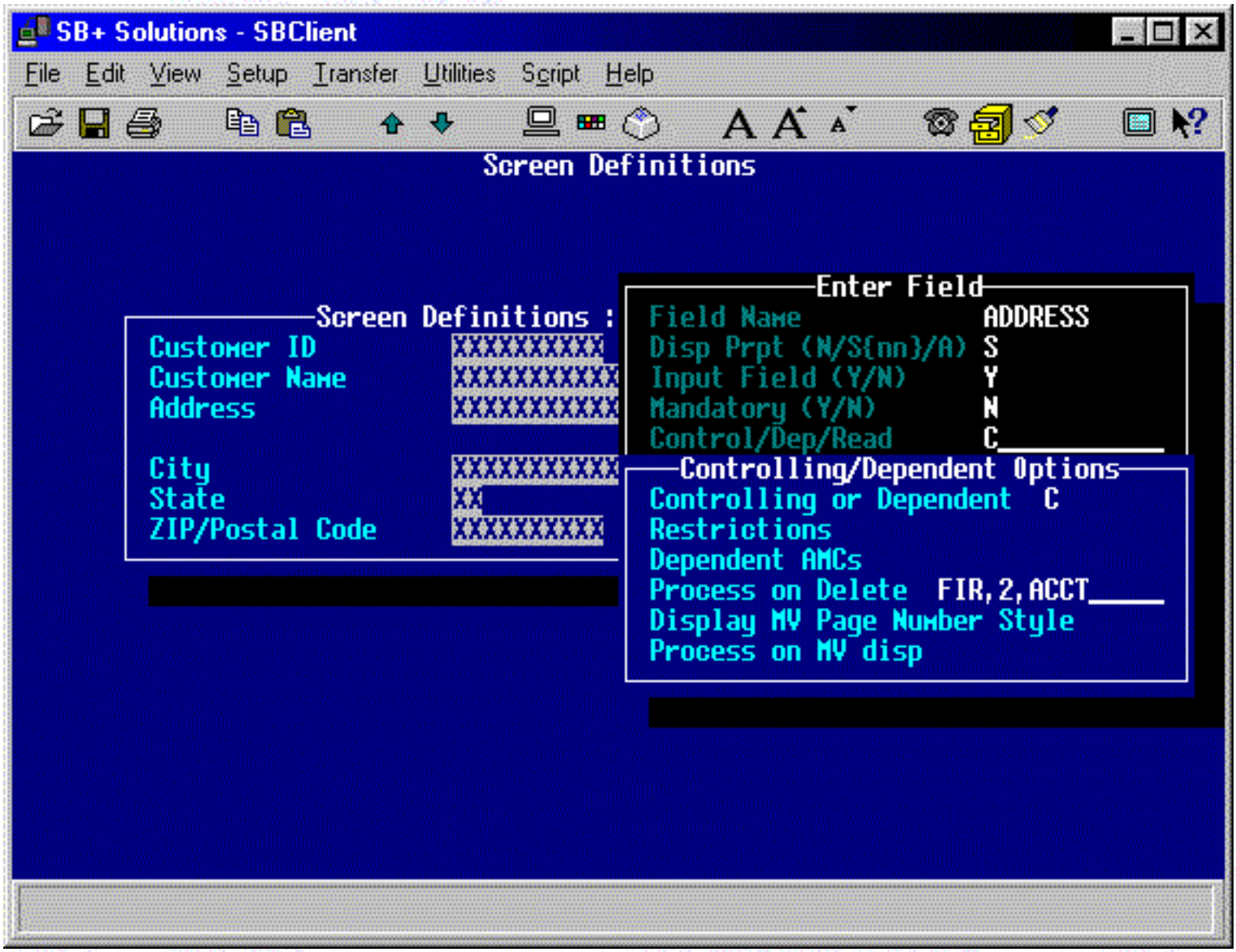
These restrictions are defined using the Field Input Restrict Defs tool. This tool can be started from the menus using Tools, Other Tools/Utilities, Field Input Restrict Defs. Or, for the direct approach, the tool may be started by entering /FIR.DEFN at any input prompt or menu. When started, this tool appears as follows:



On this screen, the "Del" restriction flag is the only one used for multivalue restrictions. If a particular user or group is disallowed from deleting (i.e. the Del flag is "D"), they will not be allowed to remove values on the field where the restriction is applied.

To apply the restriction to a controlling multivalued field on a screen, move the cursor to the multivalued field and press F5 to display the F5-Field information. Use the arrow keys to move the cursor to the Control/Dep/Read prompt and press F3. The

following screen will appear:



Note the process shown in the "Process On Delete". This says that the FIR process will be called when someone attempts to delete one of the rows in the multivalued set. If the person is disallowed (in the field input restriction set named "ACCT") from removing the row, an appropriate error will be displayed and the row will not be removed.

Calling a Process on Multivalue Delete

Under normal circumstances, users can freely add, edit, delete, or append rows to a multivalued set of fields on the screen. Unless we tell SB+ to do otherwise, (using **multivalue restriction codes** or **field input restrictions**) there are no restrictions preventing users from making such changes.

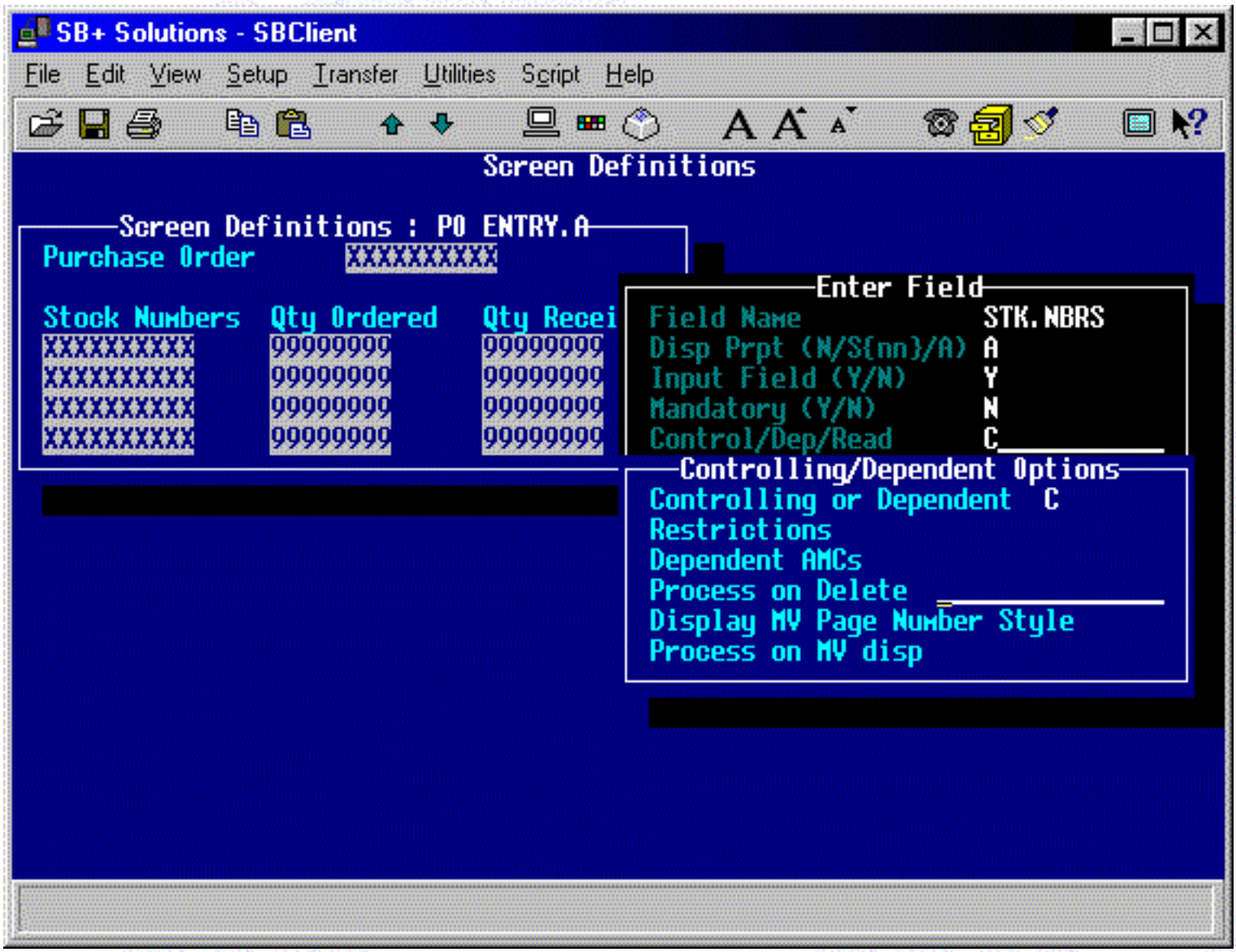
At times, you may need more control over when it is appropriate to allow a particular multivalue row to be removed. For example, let's say we have a purchase order screen that looks something like this:

Purchase Order xxxxxxxxxx

Stock Number	Qty Ordered	Qty Received
xxxxxxxxxx	xxxxxxxx	xxxxxxxx
xxxxxxxxxx	xxxxxxxx	xxxxxxxx
xxxxxxxxxx	xxxxxxxx	xxxxxxxx
xxxxxxxxxx	xxxxxxxx	xxxxxxxx

On this screen, if we've received any quantity for a particular line, we don't want that line to be removed. If no quantity has been received for that particular line, the user will be allowed to remove the row.

To implement this, we'll use the a process slot called "Process On Delete". To find this prompt, press F5 on a controlling multivalue, use the arrow keys to move the cursor to the "C" in the Control/Dep/Read slot, and press F3. The following screen will appear:



In the Process on Delete slot, we can add a paragraph to be called immediately after the user presses the line delete key, but before the row is actually deleted. In this process, if we set @RTN.FLAG to 1, the row will not be removed. We can also display whatever error message is appropriate, as follows:

```
IF QTYS.RCVD > 0 THEN
  ERROR "You cannot remove a line with received quantities"
  EXIT 1
END
```

Note that we don't need to specify which of the multivalues we want when referencing QTYS.RCVD. Because we referenced this field by name, SB+ knows which multivalue number we're talking about (i.e. the current multivalue). For more information about the implied multivalue position, see "**Feature or Bug?**" elsewhere in this chapter.

Displaying a Multivalue Page Indicator

When creating a screen that will update a large number of values in a multivalued prompt (or set of prompts), it can be helpful to have a multivalue page indicator to help the user keep track of roughly where they are in the list. To facilitate this, SB+ allows you to select one of three ways to display this information:

Types of Page Indicators

Previous/Next Page Indicator

If the cursor is on the second or subsequent pages of a multivalued prompt, a character will be displayed next to the top multivalue to denote previous information exists. If there are pages beyond the page that is displayed, a character will be displayed next to the bottom multivalue to denote that more information exists past this page. These indicators give the user some idea that something is out there, but doesn't give any information about how much.

The actual characters that are displayed are configured in the terminal definition. Use the /TERM.DEFN process to view/update the terminal definition. Consult the SB+ system administration manual for more information about defining terminals.

Page x/y Indicator

Below the bottom multivalue on the screen, SB+ will display a page indicator such as 1/3. This particular example states that the user is on page 1, and there are a total of 3 pages of information currently in this multivalued prompt. This particular method takes up a few more characters on the screen, but gives a more complete picture of how much information is in the multivalued field. Incidentally, you have no control over where this is placed. It will always be in the same column as the controlling multivalue field, one row below the last displayed value.

No Indicator

This is the default option when SB+ is installed "out of the box". Basically, there is no indicator of additional information either prior to, or following, the current page of information.

Selecting a Page Indicator

These options are identified by the numbers 1, 2, and 0 (or null), respectively, and are entered following the "C" code on the Control/Dep/Read prompt on the F5-Field screen on a screen definition. To have a multivalued prompt with a previous/next page indicator, the following could be entered at this prompt:

C;1

Or, to have a controlling multivalue which restricts insertions and deletions and uses a "page x/y" multivalue page indicator, the following could be used:

CID;2

For a controlling multivalue which restricts insertions and deletions, updates hidden attributes 14, 15, and 16, and uses a "page x/

y" multivalue page indicator, the following could be used:

CID,14,15,16;2

Though these examples don't specifically illustrate it, there are two important issues to keep in mind when using this technique:

- The semicolon is important, as this separates the multivalue page indicator from the other options on this prompt.
- The semicolon and multivalue page indicator, if used, must be the last thing entered in the Control/Dep/Read prompt on the F5-Field screen.

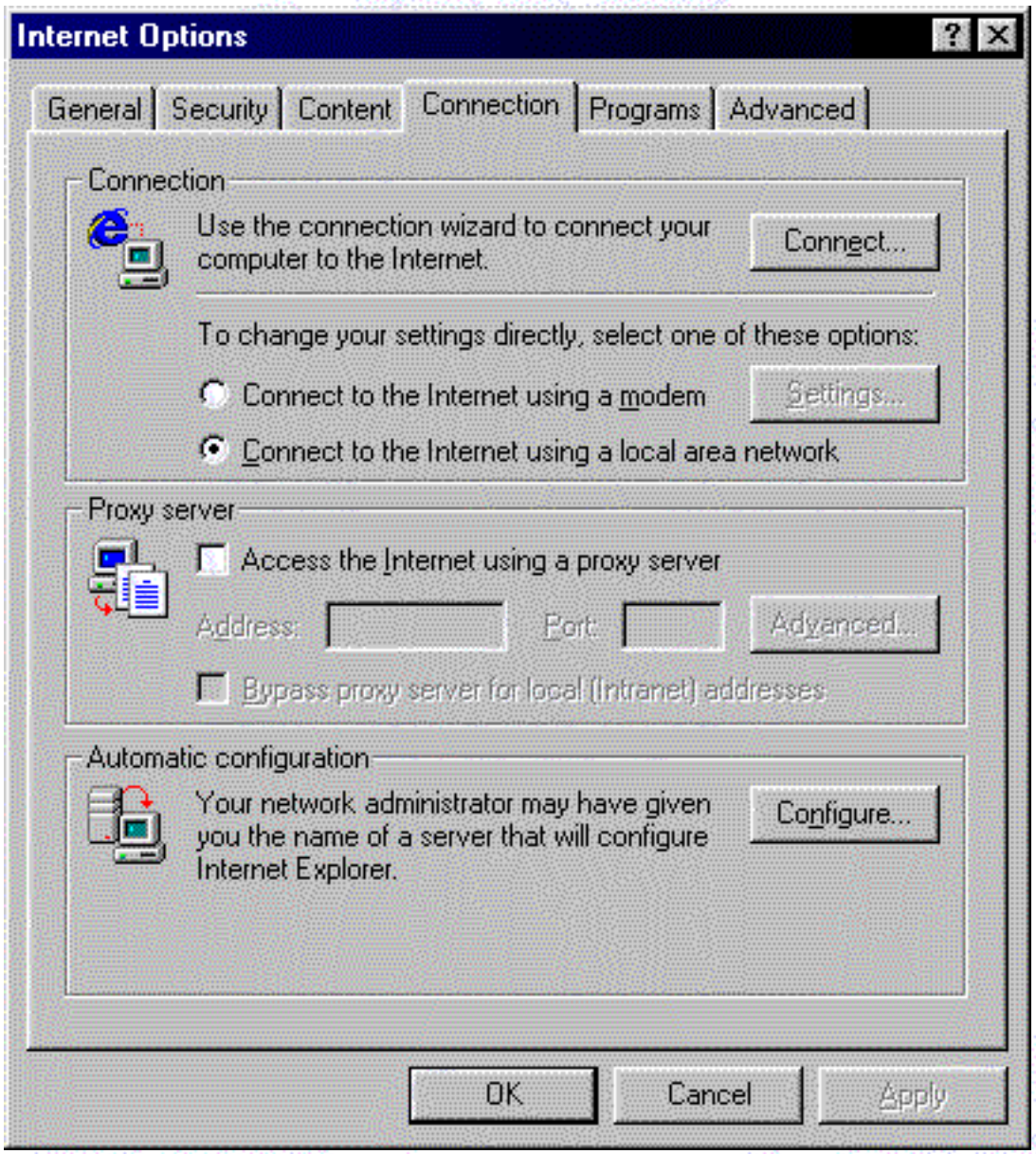
Which technique you use is, of course, a matter of the needs of the application and your personal style. However, it is recommended that you consider making one of these codes a standard for your entire application. Otherwise, different people could use different codes and compromise the standard look-and-feel of your application.

Implementing Character-Based Check Boxes and Radio Buttons in SB+

In the Microsoft Windows world, application builders have a variety of different types of prompts to choose from, rather than simply input and/or display prompts. Two of the more unique prompt types are called "check boxes" and "radio buttons".

Unlike a typical entry prompt, where a user enters a value into a field, a check box allows the user to toggle one or several options by placing check marks in boxes that correspond to the option(s) desired. Radio buttons work the same way, except only one option may be selected at a time (the way a station selector on a car radio works, hence the name).

To see what these look like in a typical Windows application, let's look at one of the setup screens for a Windows application:



On this screen, we see a series of buttons which are used for defining connection settings. Only one of the round buttons can be selected. These are radio buttons.

Note the square box for "Access the Internet using a proxy server". This is a check box. It is either selected or not selected. Unlike

the round buttons, there are no multiple options here -- just an "on" or "off" state.

On one hand, the character mode of SB+ does not "naturally" support radio buttons and check boxes like this. However, just because SB+ doesn't have a particular feature doesn't mean that you, the application developer, can't use the existing tools to expand the basic functionality of the product. This particular technique is a perfect example of where existing features of SB+ can be used to create a completely new feature. Of course, if you're writing your application for SB+'s GUI feature, you will not need this section. Nonetheless, this illustrates how easy it is to expand the basic functionality of the product through a few well-placed processes.

Before we get too far, let's clarify the similarities and differences between typical check boxes and radio buttons. First, similarities: Both allow selections to be made from a list of valid options. Each of the options can be "on" or "off", so each flag is essentially a binary switch. An option can be selected by setting a flag to "on", and deselected by setting a particular flag to "off". As to the differences, there is only one: Radio buttons are mutually exclusive options, while any number of check boxes in a series can be selected.

Next, we need to define the purpose for the field. In other words, what will be the effect of an option being set or reset?

For the purpose of this discussion, we're going to assume that when a particular option is selected, a code will be updated into some field in the record. When an option is deselected, the appropriate code will be removed from the field in the record. This allows the user to update codes in the record simply by selecting options from a list on the screen, rather than having to remember a list of potentially cryptic codes.

To embellish this example, let's assume we're creating a customer entry screen. On this screen, we're entering information that will be used to determine a customer's credit rating. Therefore, we need to know whether the customer owns a home, owns a car, owns stocks or bonds, or owns other appreciable assets. To implement this, we'll create a series of check boxes on the screen, one for each option, something like the following:

[ ] Home
[ ] Car
[ ] Stocks/Bonds
[ ] Other Assets

Because the customer can own one or all of the items on the list, we'll allow one, several, or all options to be selected.

Finally, if the user selects the first option, we want to insert the code HOME into a field called ASSETS in the customer record. If the user selects the second option, CAR will be inserted. If the user selects the third option, STOCKS will be inserted. Finally, if the last option is selected, OTHER will be inserted into the record. In the end, we should have a multivalued list of codes that corresponds to the selected options.

To implement this, the following programming needs to be completed:

- We need to create a multivalue field where the buttons can be edited.
- We need a process to be called from the Process After Read which will translate the values from the incoming record to button values for editing, or set a default button or buttons if the record is new.

Creating a Button Field Definition
Creating a Process to Setup the Buttons
The Validation Process
Calling the Process After Read

Putting It All Together
So, Do You Like a Challenge?

Implementing a Word-Wrapped Comment Field

To implement a comment field in SB+, you can simply create a multivalued **field definition** and put it on a screen. This then allows you to enter anything into the field that you want. It does not, however, restrict the entry in any way -- which can be a problem.

Consider for a moment creating a screen to enter comments that will appear at the bottom of an invoice form when a customer buys something. The invoice form has a window that is only 50 characters wide, and 10 lines long. Therefore, we don't want to allow the user to enter lines longer or wider than these dimensions.

On one hand we could create a **validation** process to ensure that the user doesn't enter too much, but then again, if you entered 50 characters of text and the system told you it was too long and rejected the entry, wouldn't you be a little upset? The fact of the matter is, rather than giving the user a loaded gun and then complaining when they use it, let's take the ammunition away and prevent the problem.

We can do this by adding word wrap to the comment field. This feature, oddly enough, can be enabled for all controlling multivalues that have no dependents. With word wrap enabled, the user cannot enter a line that is longer than the defined field width. If they try to do so, the characters exceeding the line width will wrap around to the next line.

To implement this feature, go to the F5-Field screen for the multivalued field that will be used to enter comments and enter the (SB + standard) process name INVOKE.TEXT.ED in the Process Before prompt. Once this is done, you will need to add a process to the screen definition in the Process After Screen Accept slot to verify that the number of values entered does not exceed the maximum. That's all there is to it. Fortunately, INVOKE.TEXT.ED is already built into SB+ so there's nothing additional needed to make it work.

Behind the veil, the INVOKE.TEXT.ED process simply presses the key to invoke the word processing mode when the cursor is moved to the field. In an SB+ paragraph, this process is as follows:

```
DATA "@23"
```

"So what value is this?" you may wonder. "The process already exists and works, so why should I care what the process is really doing?"

Note the @23 inside quotes following the DATA statement. This particular syntax tells SB+ to data stack key number 23, instead of a literal value "@23". Key number 23 just happens to be the key to enable word wrap in the terminal definition. Other keys do other things, so the purpose of this is simply to demonstrate that editing keys can be data stacked just like any other value, as is evidenced by INVOKE.TEXT.ED.

But There's a Problem...

When you use the word processing mode to edit a field, either the F2 key must be pressed, or the down arrow must be pressed beyond the bottom of the field to exit the word processing mode and move to the next prompt. If the multivalued comment field is the last prompt on a screen, users must press F2 twice in order to finish entering the values on a screen. The first F2 exits the multivalued comment window, while the second F2 actually saves the record. Some people find this annoying.

To remedy this, we can create a new process called CASCADE.F2 which will check to see if F2 was pressed to exit the word

processing mode, and if so, data stacks the F2 key to save the record. With this in place, the user can still arrow out of the multivalued field without saving the record, but if they press F2, the record will be saved without a second F2.

CASCADE.F2 appears as follows:

```
IF (@OTHER(18) = 32) THEN EXIT U
```

This process simply states: "If the last keypress was F2, go to the update step and save the record. The question, though, is this: Where should this process be called?

There are only two process slots which could be used to call CASCADE.F2. The first is the Process After on the F5-Field screen for the multivalued comment window. The second is the Process Before on the F5-Field screen for the first prompt after the key field.

In certain releases of SB+, the Process After on the comment field will be ignored if the word processing mode was used. Whether this bug will appear in later releases again is unknown, so this particular process slot is not recommended.

The Process Before on the first field following the key field, though, will work despite the bug. Normally, the last edit key number will be zero, which signifies that the <cr> key was pressed following the entry of the key. Only when the user presses F2 on the last field on the screen will this variable have the proper value to tell the process to cascade the F2 and go to the update step and save the record.

Structuring a Process After Read

There are two main reasons why a process would be called after the record has been read:

- Verify that the record can be loaded
- Load values from other records.

Knowing this, the Process After Read can be structured in such a way as to make debugging and tracing through the code easier. In contrast, all of this logic can be coded into one process, but one humongous process is often more difficult to comprehend than a series of smaller processes that are judiciously called.

Constructing a Verification Process Only

If a Process After Read is responsible only for verifying whether the record can be loaded, one process can be used. However, if the verification is complicated or involves several different auxiliary files, it may be better to create several verification processes, with one process per file or verification step (whichever is appropriate). With several processes constructed this way, they can be called with a single paragraph as follows:

```
EXEC 'verify1'
IF @RTN.FLAG = 0 THEN EXEC 'verify2'
IF @RTN.FLAG = 0 THEN EXEC 'verify3'
...
IF @RTN.FLAG = 0 THEN EXEC 'verifyn'
*
IF @RTN.FLAG THEN
  RELEASE 0,@KEY
  EXIT 1
END
```

Each verification process called from this paragraph is responsible only for setting @RTN.FLAG to "1" if the record should not be loaded. If this flag is set by any of the called processes, this process then releases the record lock set by SB+ and exits back to reprompt for the key field. Also note that if any of the processes sets @RTN.FLAG this way, none of the subsequent processes will be called.

Constructing a Read Process Only

Similar to the verification process, if the read process is simple and involves only one auxiliary file, it can be written with one process. However, if the read process uses several auxiliary files, or has several different steps in order to load the values, it may be more appropriate to build a structure such as the following:

```
EXEC 'read1' ;* Load info from the first file
EXEC 'read2' ;* Load info from the second file
EXEC 'read3' ;* Load info from the third file
...
EXEC 'readn' ;* Load info from file #n
```

Each of the processes called from this process is responsible for loading information from a single file or a single step. Note that there is no mention of @RTN.FLAG here, as there was with the verification process. Because this process is for reading only,

there should never be any verification code in any of these processes, and therefore @RTN.FLAG should never be changed.

Constructing a Process to Verify and Read

When the Process After Read must verify that the record can be loaded and also load other values from other files, the rules as described above apply. However, one additional process must be constructed, which is the actual process to be called from the Process After Read. This process is a paragraph with the following lines:

```
EXEC 'verify_process' ;* Verify record can be loaded
IF @RTN.FLAG = 0 THEN
  EXEC 'read_process' ;* Load other values
END
```

This little process encapsulates both the verification process and the read process, and calls the read process only if the verification process allows the record to be loaded.

Handling Conditions

There are many times when it can be helpful to call a process, but only when a certain condition is true. As an example, we may want to restrict access to certain prompts on a screen if a person is not in a particular security group. This can be done by wrapping a condition around the SKIP process and then calling this wraparound process as the Process Before on each of the fields to be skipped.

There are three ways to handle a condition using SB+:

- The Process Definition - Conditional tool can be used to enter the process.
- A paragraph can be created with the Process Definition - Paragraph tool.
- A **BASIC** subroutine can be written.

While it is certainly an option, using a BASIC subroutine for this purpose is a bit drastic. Therefore, this section will focus on only the first two methods.

Using a Conditional Process
Using a Paragraph

Pre-Delete Verifications

When a screen definition is originally created, the F4 key is defined as:

G:DE

Translating this code into English, this says to set the **common variable** **@RTN.FLAG** to the literal "DE". This has two results:

- The user will be prompted to confirm that the record should be deleted, and
- If the user responds affirmatively, the process will jump to the delete step to physically remove the record from the file.

To skip the confirmation step, change the DE to only D. Doing so can be dangerous, though, as the user will have no last minute warning to prevent the delete from happening.

Often, there are other conditions that can be verified to see if the record should be deleted. For example, if a customer record has no outstanding balance, the record could be deleted. However, if there is an outstanding balance, the record should not be deleted. In this case we don't want to prompt the user for verification, but rather use the known condition to determine if the record should be removed.

Fortunately, any process can be called from the F4 process slot in a screen definition's function key set, not just G:DE. Therefore, you can put any logic you want in a process called from this slot to verify any condition as needed. If the process terminates with @RTN.FLAG = 'D', the record will be deleted. If the process terminates with @RTN.FLAG = 0 or 1, the record will not be deleted.

Following our customer example, we could create a paragraph to be called from the F4 process slot with the following line:

```
IF (BALANCE = 0) THEN EXIT D
```

Note that there is no logic to set @RTN.FLAG to zero if the condition is false. SB+ automatically sets @RTN.FLAG to zero before the process is called, so this becomes an issue only when the process has logic which may otherwise change the value of @RTN.FLAG.

Updating Hidden Fields in the Record

There are times when an application calls for certain attributes in the record to be updated, even though they don't appear on the entry screen. As an example, I recommend that any records that are updated by a screen should be stamped with a create date, time, and user ID as well as the date, time, and user ID who last changed the record. This type of information can then be used to document when a change was made to a record, rather than relying upon the memory of a user as to the last time he or she updated the record.

In a screen definition, the Process After Screen Accept prompt can be used to update these hidden fields. This process slot could have also been called Process Before Write, because it is the last thing that is done prior to the record being written. Anything that needs to be done behind the scenes can be done using this process slot.

Officially, this process slot has two purposes:

- Verify that the record can be written, and
- Update hidden attributes in the record.

Knowing this, if you have any fields to be updated in the record after the user presses F2, but before the record is written, the Process After Screen Accept can be used. To see an example of how this is done, turn back a few pages to the "**Handling Conditions with a Paragraph**" section to find my time/date/user ID example.

Updating Several Records Simultaneously

The framework of a typical input process supports the update of one record at a time. There are times, however, when this is simply not enough. Particularly when dealing with large, complex data structures, the ability to update several records or files at a time can be a very important requirement.

To illustrate how this can be valuable, assume we're creating an application to enter timesheets for our employees. Each employee will have one timesheet per day, and every day each employee can work on any number of different projects, each project listed only once on the timesheet. To put this visually, we want to create a screen which appears something like the following:

Employee ID xxxxx
Date Worked xx/xx/xx

Project Time Description
xxxxxxx xx:xx:xx xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxx xx:xx:xx xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxx xx:xx:xx xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxx xx:xx:xx xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxx xx:xx:xx xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxx xx:xx:xx xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

These timesheets, once entered, will be posted to our labor tracking system. However, the entire timesheet may not need to be posted all at the same time. Therefore, we need to keep track of a posting date for each row of the timesheet.

It is also possible for timesheets to have several dozen projects per employee per day. With this in mind, if we store the entire timesheet in one record the records in the timesheet file could be very large and cumbersome to work with.

Based on these two considerations, this application will store the timesheet in two different files. In one file, called TSHEADER, we'll store the list of projects that an employee worked on for a given day. In another file, called TSDETAIL, we'll store the individual project details for each timesheet.

The file structure for these two files could appear something like the following:

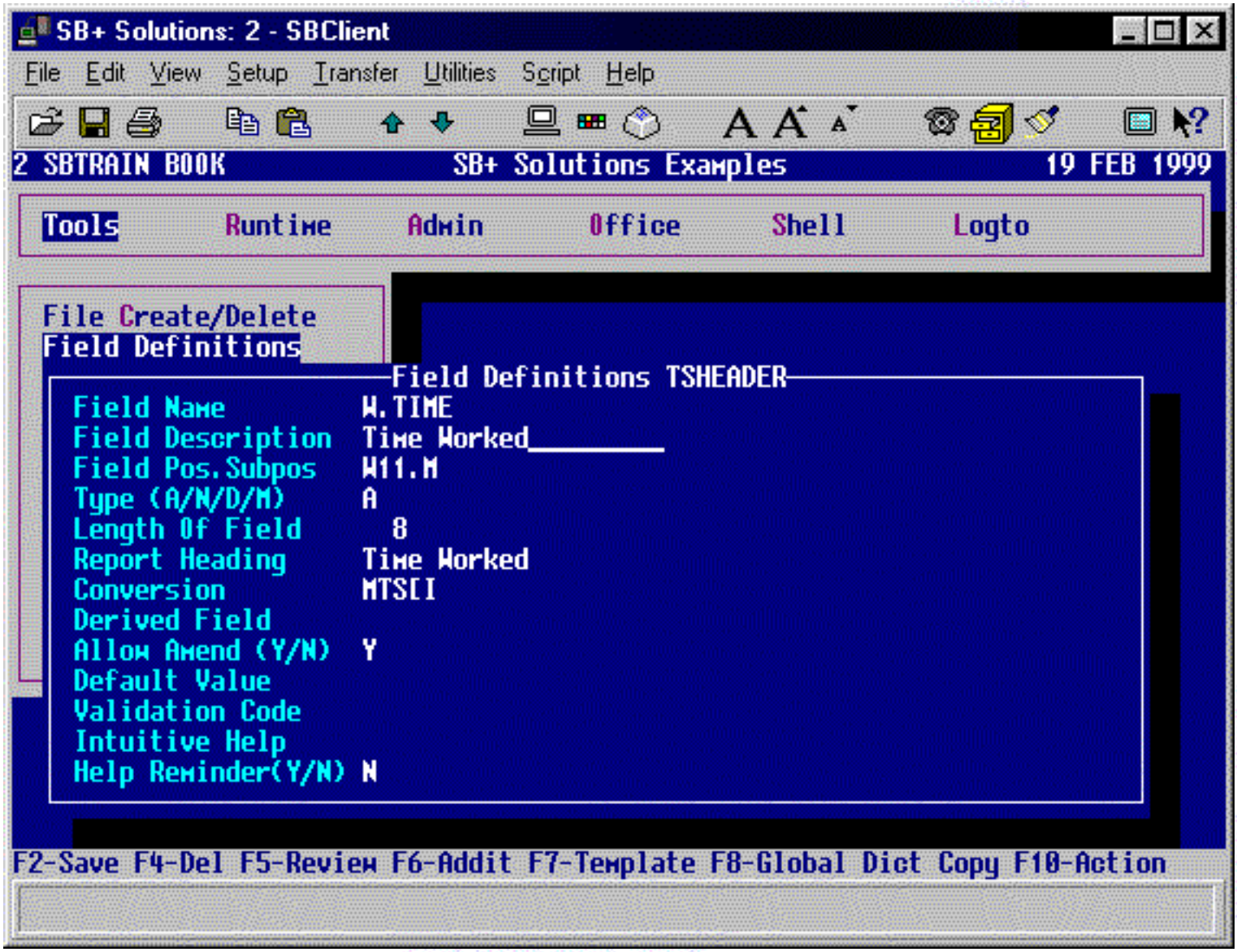
TSHEADER
key: Employee ID : "*" : Current Date
001 Multivalued list of projects for this employee and date

TSDETAIL
key: Employee ID : "*" : Current Date : "*" : Project
001 Time Worked
002 Description of Time Worked
003 Post Date

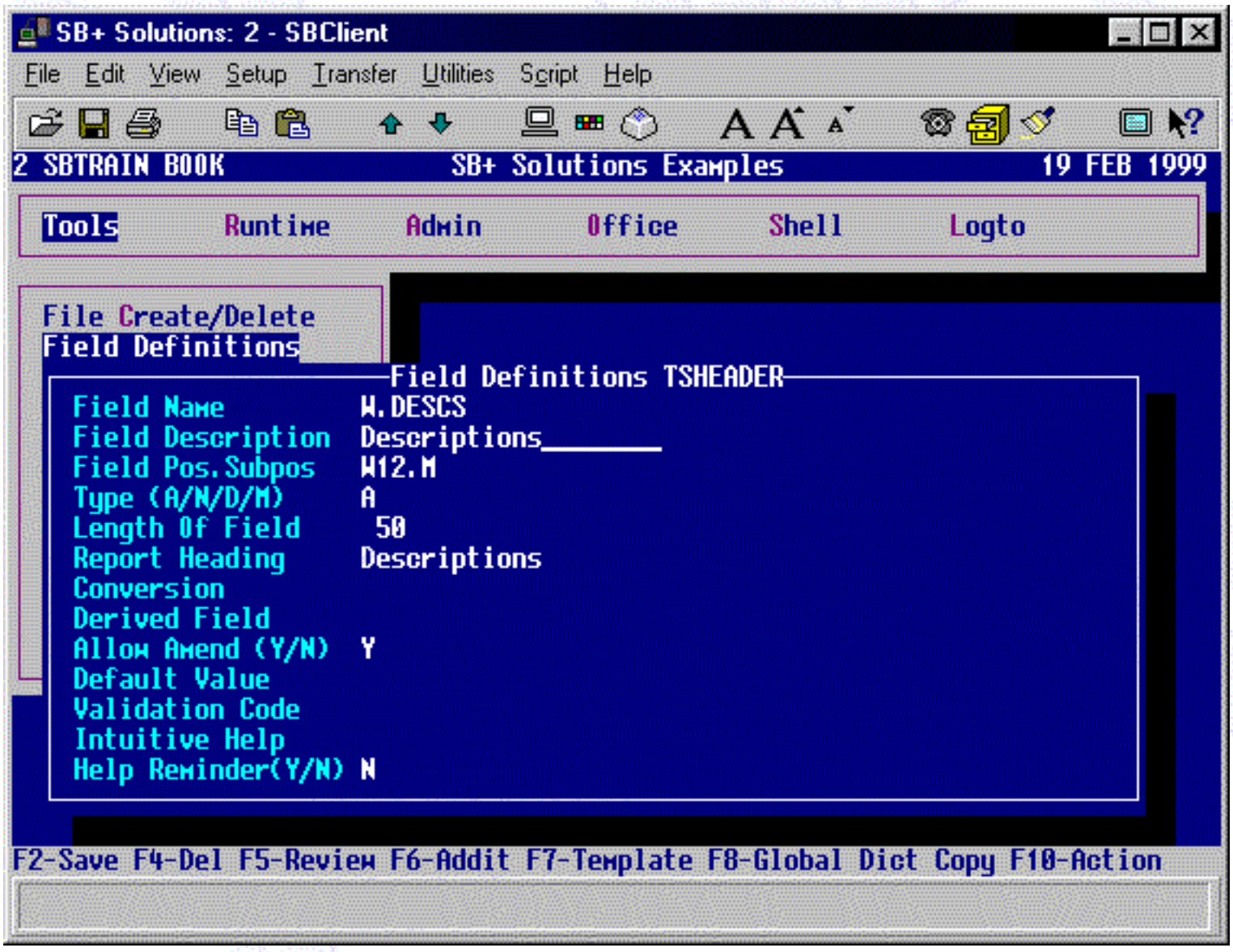
The goal of this application, then, is to update one TSHEADER record and several TSDETAIL records per timesheet. To do this, we must first **create the files**. Next, create a screen to enter the details. Because the TSHEADER record will be saving one record per timesheet, this is the file that the screen should be created for. In essence, the TSHEADER file is the primary or main file, and we'll update the TSDETAIL file as a secondary or auxiliary file.

With this in mind, let's assume we're creating a screen called ENTRY.A in the TSHEADER file. On this screen, we need to create two work fields for the entry of the key. Once these fields have been placed, the next step is to place the field definition for the Project column on the screen.

However, now we have a problem: There are no field definitions in TSHEADER for the time worked or the descriptions. To overcome this, we can create two work fields in the TSHEADER file where this information can be stored, as follows:



And for the description field:



Note that both of these fields are multivalued, and both reference the **common variable** @WORK (as is evidenced by the Wx.M field position). With field definitions like this, we can load information from the TSDETAIL file after the main record has been read, and then save information into the TSDETAIL file when the main record is saved.

To do this, let's first focus on the process to be called in the Process After Read slot in the screen definition. The goal of this process is to read all of the TSDETAIL records for the appropriate TSHEADER record, and load information into the work fields. This paragraph can be named anything you like, and should have the following paragraph lines:

```
@CNT = DCOUNT(<POS(PROJECT.XREF)>,@VM)
WHILE @CNT DO
  READ @OTHER.REC FROM 'TSDETAIL',@KEY:"*":PROJECT.XREF
  W.TM.WORKED = O1
  W.DESC = O2
  W.POST.DT = O3
  *
  @CNT = @CNT - 1
REPEAT
```

First, we need to determine how many detail records to be read. Knowing there should be one detail record for each project as referenced on the header record, we can use the DCOUNT(...) function to count the number of projects in the PROJECT.XREF

field (which points to attribute 1 in the TSHEADER file) and read one detail record per project.

Note the syntax of the DCOUNT(...) function. Because we don't know what the common variable @CNT could be when this process is called, we cannot reference the PROJECT.XREF field directly in the first parameter. Instead, we tell the expression to extract a full attribute from @RECORD, using the attribute position that corresponds to the field name PROJECT.XREF. The following alternative yields the identical results:

```
@CNT = 0
@CNT = DCOUNT(PROJECT.XREF,@VM)
```

With this particular variation, by initializing @CNT to zero prior to the DCOUNT, the entire field is extracted from @RECORD.

Next, knowing the total number of records to be read, we can then read one record per project using the **WHILE** loop in the paragraph. In this particular example, we're counting from back to front, thus eliminating the need for a second variable .

For each iteration of the loop, we read a record from the TSDETAIL file. Because the first two parts of the TSDETAIL key are the TSHEADER key, we can build each TSDETAIL key simply by concatenating the TSHEADER key to the project code for each iteration.

Once we have the record, the next step is to extract the values from the record and stuff them into the work variables that we will be using for this screen. The following three lines are used to do this:

```
W.TM.WORKED = O1
W.DESC = O2
W.POST.DT = O3
```

While deceptively simple, these statements do a great deal. Basically, we're moving values from @OTHER.REC to @WORK, but where? How does the process know which multivalued position is to be updated on each of these work fields?

Because we're referencing the work fields by name, the multivalue position is implied based on the value of the @CNT variable. With @CNT being our loop index, the process updates the right multivalue for each iteration. Therefore, if W.TM.WORKED references @WORK<11>, W.DESC references @WORK<12>, and W.POST.DT references @WORK<13>, the lines as shown above actually mean:

```
@WORK<11,@CNT> = @OTHER.REC<1>
@WORK<12,@CNT> = @OTHER.REC<2>
@WORK<13,@CNT> = @OTHER.REC<3>
```

Also note that the references to O1, O2, and O3 are actually references to attributes 1, 2, and 3 of the common variable @OTHER.REC. This particular syntax variation is one of the more confusing aspects of the expression language, but is a useful time saver nonetheless.

At the end of the WHILE loop, we need to decrement @CNT with the following line:

```
@CNT = @CNT - 1
```

Once @CNT has been decremented, the REPEAT closes the WHILE loop, and the process is complete.

After this process has been created and linked into the Process After Read slot in the screen definition, we have completed half of

the programming required to implement this header/detail file update screen. We can now load values from both the header and detail files, but have no logic as of yet to update both files.

On the surface, one might think that the read logic could simply be reversed in order to update the detail file. While this provides a good starting point for our process, it's not that simple. Because values can be deleted from the multivalued project field, we need additional logic to delete the appropriate detail records from the TSDETAIL file.

The paragraph to update the TSDETAIL file can be named anything you like, and should have the following paragraph lines:

```
LOCAL NDX
*
* Step 1: Update TSDETAIL
*
IF @ACTION < 3 THEN
  @CNT = DCOUNT(<POS(PROJECT.XREF)>,@VM)
  WHILE @CNT DO
    IF W.POST.DT = '' THEN
      READU @OTHER.REC FROM 'TSDETAIL',@KEY:"*":PROJECT.XREF
      O1 = W.TM.WORKED
      O2 = W.DESC
      WRITE @OTHER.REC ON 'TSDETAIL',@KEY:"*":PROJECT.XREF
    END
    *
    @CNT = @CNT - 1
  REPEAT
END
*
* Step 2: Clean up any deleted projects
*
NDX = DCOUNT(@ORIG.REC<POS(PROJECT.XREF)>,@VM)
WHILE NDX DO
  IF @ACTION = 3 OR LOC(@ORIG.REC<POS(PROJECT.XREF),NDX>,PROJECT.XREF,@VM) = 0 THEN
    DELETE 'TSDETAIL',@KEY : "*" : @ORIG.REC<POS(PROJECT.XREF),NDX>
  END
  *
  NDX = NDX - 1
REPEAT
```

The code labeled Step 1 is essentially the original Process After Read code, reversed with a few enhancements. First, we update the detail records only if the header record is not being deleted. Next, we need to update only those records that are posted, though for the sake of this example we're not going to get into posting in any detail.

In Step 2, we need to cycle through the original list of projects, checking to see if any of the original projects no longer exist in the new list. If any of the original projects have been removed, we need to delete the corresponding TSDETAIL record.

Note that the loop index for this second loop is not @CNT, but is a **local variable** instead. This is done because in this second loop, the only reference we have to a field in @RECORD is inside of the LOC(...) function, and we want to look at the attribute as a whole. There is no place in this section where we want to look at a specific multivalue in PROJECT.XREF. Therefore, because @CNT is zero as a direct result of the prior loop, we can look at PROJECT.XREF as a whole simply by referencing the field name. If we used @CNT as the loop index, the LOC(...) function would need to appear as:

```
LOC(@ORIG.REC<POS(PROJECT.XREF),@CNT>,<POS(PROJECT.XREF),@CNT>,@VM)
```

instead of:

LOC(@ORIG.REC<POS(PROJECT.XREF),@CNT>,PROJECT.XREF,@VM)

The bottom line here is simplicity. By using a local variable as the loop index, we simplify the LOC(...) function syntax, thereby making the code easier to read.

Once this process has been created and linked to the Process After Update on the screen definition, we've completed the second half of the logic to update multiple files from a single screen. No additional logic is needed to update the TSHEADER file, as this will happen automatically when the record is filed.

Structuring a Process After Screen Accept

The Process After Screen Accept slot has two purposes:

- Verify that the record should be written, and
- Update hidden fields in the record.

With this in mind, a process called from this slot can be structured in such a way as to make debugging and tracing through the code much easier.

Constructing a Verification Process Only

If the process called in the Process After Screen Accept is responsible only for verifying that the record can be written, one process can be used. However, if the verification is complicated or involves several different auxiliary files, it may be better to create several verification processes, with one process per file or verification step (whichever is appropriate). With several processes constructed this way, they can be called with a single paragraph as follows:

```
EXEC 'verify1'
IF @RTN.FLAG = 0 THEN EXEC 'verify2'
IF @RTN.FLAG = 0 THEN EXEC 'verify3'
...
IF @RTN.FLAG = 0 THEN EXEC 'verifyn'
*
IF @RTN.FLAG THEN
  EXIT 1
END
```

Each verification process called from this supervisory process is responsible only for setting @RTN.FLAG to "1" if the record should not be saved. If this flag is set by any of the called processes, this process exits back to the screen without saving the record. Also note that if any of the processes set @RTN.FLAG, none of the subsequent processes will be called.

Constructing an Update Process Only

If the processing to be done when a record is saved is simple, it can all be written with one process. However, if there are several other files involved, the updates are different based on different update actions (a.k.a. @ACTION values), or there are several steps involved in updating the record, the following structure is more appropriate:

```
CASE @ACTION = 1
  EXEC 'new_process'
CASE @ACTION = 2
  EXEC 'amend_process'
CASE 1
  EXEC 'del_process'
END CASE
```

This process, then, will call different processes depending on the update action on the record. More commonly, however, the updates for a new and amended record are the same, in which case the following process will suffice:

```
CASE @ACTION < 3
  EXEC 'ins/amend_process'
CASE 1
  EXEC 'del_process'
END CASE
```

The processes being called from either of these variations, then, should be structured as follows:

```
EXEC 'upd1' ;* Update first set of values
EXEC 'upd2' ;* Update second set of values
EXEC 'upd3' ;* Update third set of values
...
EXEC 'updn' ;* Update n-th set of values
```

Each of the processes called from here, then, actually updates a particular set of fields (grouped functionally) in the record for the appropriate update action.

Constructing a Process to Verify and Read

When the Process After Screen Accept must verify that the record can be saved and also update hidden values in the record, the rules as described above apply. However, one additional process must be constructed, which is the actual process to be called from the Process After Screen Accept. This process is a paragraph with the following lines:

```
EXEC 'verify_process' ;* Verify record can be saved
IF @RTN.FLAG = 0 THEN
  EXEC 'upd_process' ;* Update hidden values
END
```

This little process encapsulates both the verification process and the update process, and calls the update process only if the verification process allows the record to be saved.

Structuring a Process After Update

In many cases, the file updates that must be done from a single screen are significantly more complicated than the header/detail structure discussed earlier. When this is the case, there is a particular structure that can be used which makes debugging and tracing through the code incredibly easy.

While all of the file updates from a single screen can be handled inside one humongous paragraph, this makes debugging and tracing through the code very difficult. Instead, the updates should be broken up into several processes, with one process per file per update.

If there are several files to be updated from a single screen, and the updates are different depending on whether the record is new, being amended, or being deleted, the process after update should be structured as follows:

```
CASE @ACTION = 1
  EXEC 'new_process'
CASE @ACTION = 2
  EXEC 'amend_process'
CASE 1
  EXEC 'del_process'
END CASE
```

This process, then, will call different processes depending on the update action on the record. Often, the updates for a new and amended record are the same, in which case the following process will suffice:

```
CASE @ACTION < 3
  EXEC 'ins/amend_process'
CASE 1
  EXEC 'del_process'
END CASE
```

The processes being called from either of these variations, then, should be structured as follows:

```
EXEC 'file1upd' ;* Update file1
EXEC 'file2upd' ;* Update file2
EXEC 'file3upd' ;* Update file3
...
EXEC 'filenupd' ;* Update filen
```

Each of the processes called from here, then, actually updates a particular file for the appropriate update action. For example, if a screen is to update two secondary files, called FILE.X and FILE.Y in the process after update, we can create the following processes:

UPD.FILES - Update FILE.X and FILE.Y

```
CASE @ACTION < 3
  EXEC 'UPD.X.AND.Y' ;* Update for new/amend
CASE 1
  EXEC 'DEL.X.AND.Y' ;* Update for delete
END CASE
```

...which calls...

UPD.X.AND.Y - Update FILE.X and FILE.Y on new/amend

```
EXEC 'UPD.FILE.X' ;* Update FILE.X on new/amend
EXEC 'UPD.FILE.Y' ;* Update FILE.Y on new.amend
```

...and...

DEL.X.AND.Y - Update FILE.X and FILE.Y on delete

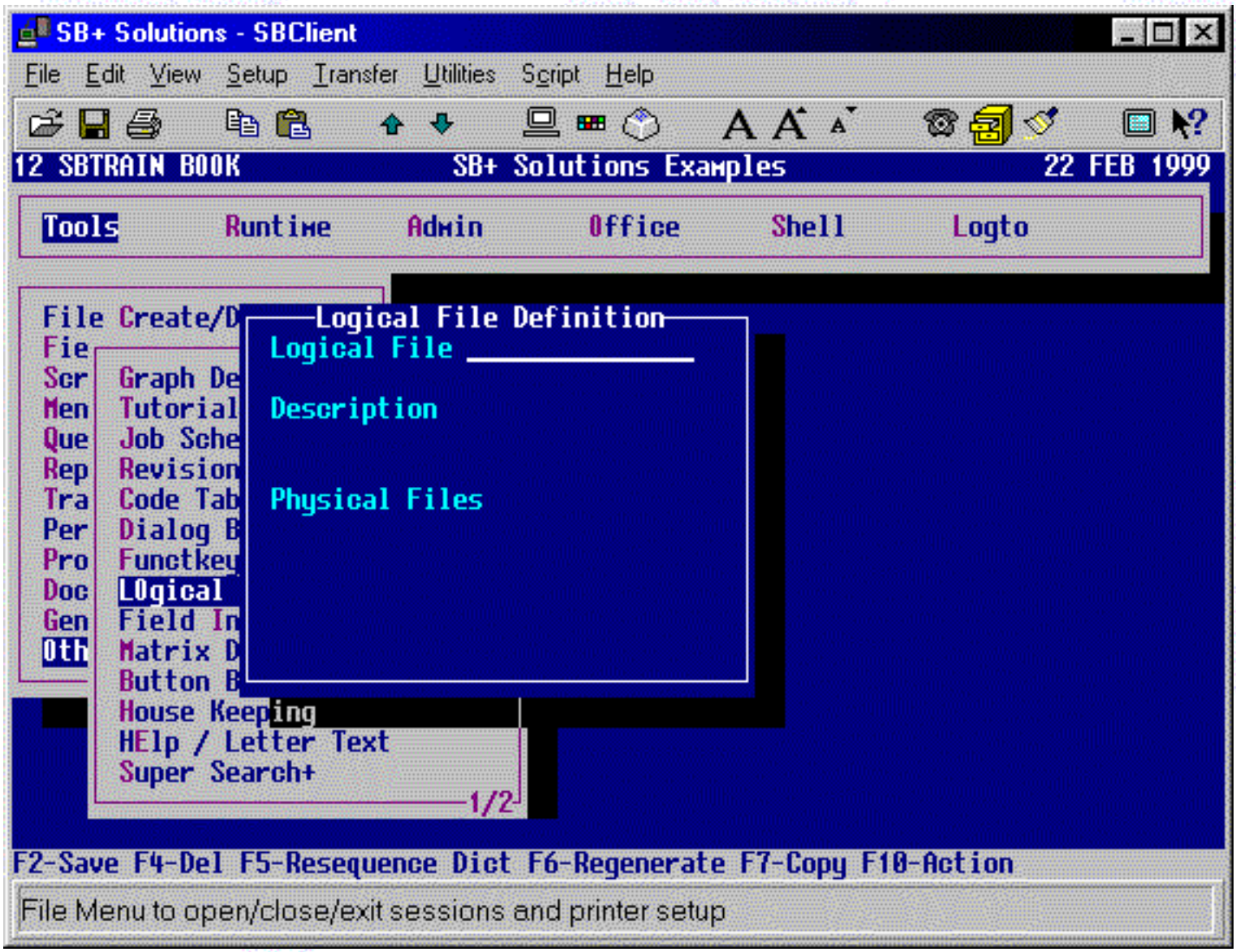
```
EXEC 'DEL.FILE.X' ;* Update FILE.X on delete
EXEC 'DEL.FILE.Y' ;* Update FILE.Y on delete
```

...which call four other processes that actually do the updates. This structure makes it very easy to find a particular update knowing only the type of update (new/amend/delete) and the name of the file being updated.

Sometimes, however, two or more files are so interrelated that it is impractical to separate the updates into two separate processes. In this case, one process will suffice, but it should be well commented so that someone reviewing the code later has no problem finding the section of code where the individual files are being updated.

Creating a Logical File

To create a logical file, use the Logical File Definition screen. This tool can be started from the menus by selecting Tools, Other Tools/Utilities, then LOgical File Definition screen. Alternatively, /LF may be entered at any input prompt or menu to start the tool. When the tool is started, the following screen is displayed:



A logical file consists of two components:

- An actual data file, and
- A logical file definition.

The tool makes no differentiation between the two -- it creates both simultaneously.

At the Logical File prompt, enter the name of the file you want to create. The file name should be entered in all uppercase, and should be named using whatever naming standards apply.

The Description prompt allows you to enter a description for the logical file. Be clear and concise and use words that will be helpful in locating the logical file in the future.

Finally, at the Physical Files prompt, enter a list of files that will be read from and written to by screens created in this logical file. Note that no details are entered here; SB+ will prompt you for details regarding each file when you press F2 to save the contents of this screen.

The first file listed here is considered the primary file to be updated, and all others are considered secondary files for update, much like a **header/detail file relationship**. Note that a few restrictions apply to the primary file. First, the key to the primary file is assumed to be whatever is in @KEY, which for all intents and purposes is the transaction key. (This can be overridden.) Next, the primary file can be updated only once for each transaction entered, though all other files can have multiple records updated for each transaction.

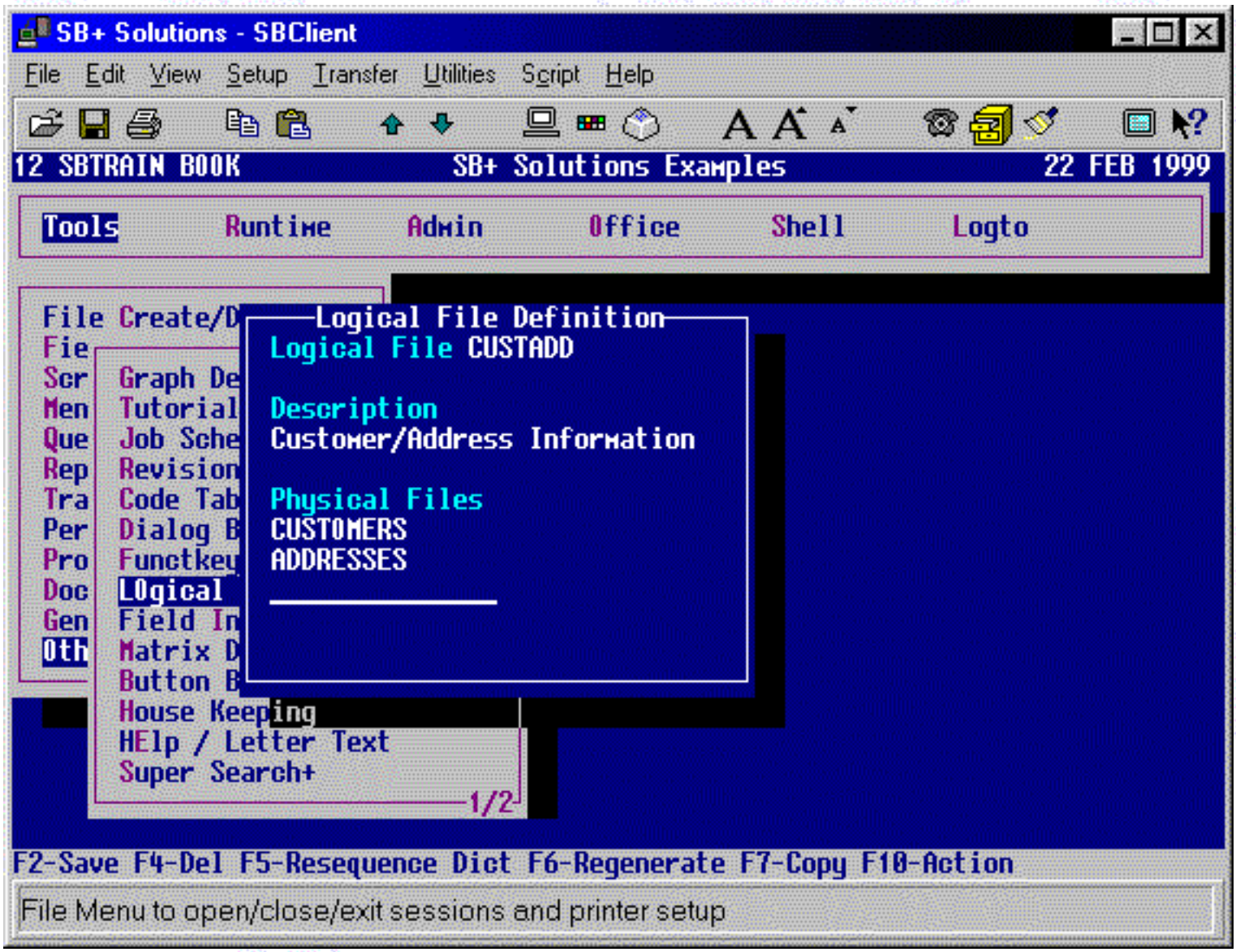
For example, let's create a logical file for entering customer and address information. In the customer file, we have the following fields:

CUST.ID	0	The customer ID (key to the record)
NAME	1	The customer name
ADDRESS.TYPES	2.M	A multivalued list of codes which define the types of addresses available for this customer, such as H=Home, O=Office, etc.

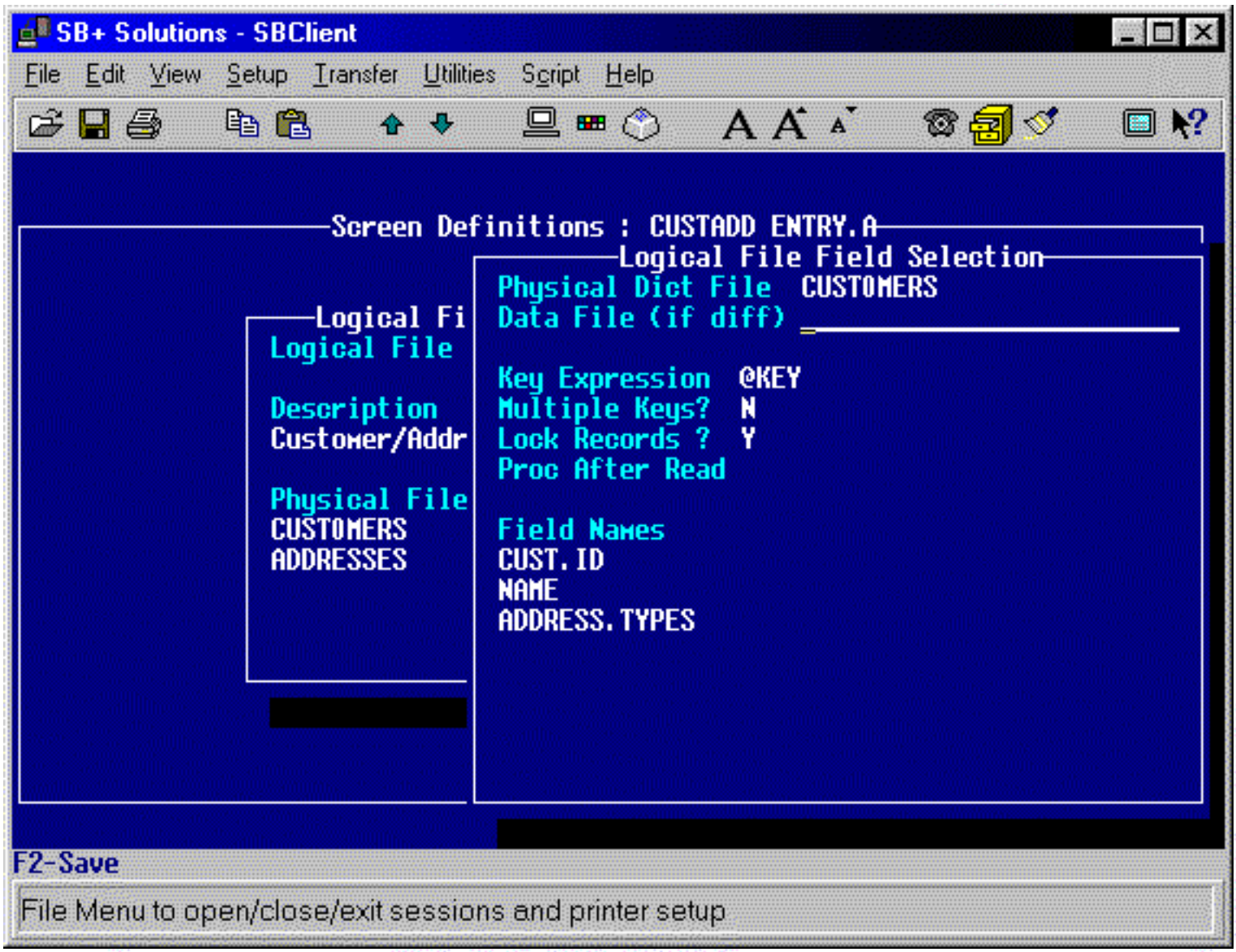
In the address file, we have the following fields:

ADDRESS.ID	0	The address key: constructed of customer ID*address type (from customer record)
ADDRESS	1	Address line for this customer and address type
CITY	2	City for this customer and address type
ST	3	State for this customer and address type
ZIP	4	ZIP/Postal Code for this customer and address type

The goal, then, is to create one logical file and screen which can be used to automatically update both files. Such an update could be called "CUSTADD", and would look something like this:



This logical file definition says that we're going to update both our files, CUSTOMERS and ADDRESSES, with any transactions entered into any screens in this logical file. When F2 is pressed, the following screen appears:



This screen allows us to enter details about which CUSTOMER fields are to be updated by screens in the logical file. A similar screen will appear for the ADDRESSES file.

The Physical Dict File is a display-only prompt, showing the file that we're working with. Following this, the Data File (if diff) prompt allows us to nominate a data file to be updated if it is different from the dictionary. (See "**Using Split Dictionary/Data Files**" for more information about this topic.)

At the Key Expression prompt, enter any standard SB+ **expression** to be used for calculating a key to each record in the CUSTOMERS file. Note that because this is the primary file to be updated, SB+ automatically loads the expression "@KEY" into the prompt.

The next prompt, Multiple Keys, allows you to define whether a single record or multiple records will be updated in this file for each transaction entered into this logical file. Because this is the primary file, and the primary file cannot have multiple records updated for each transaction, this will be "N".

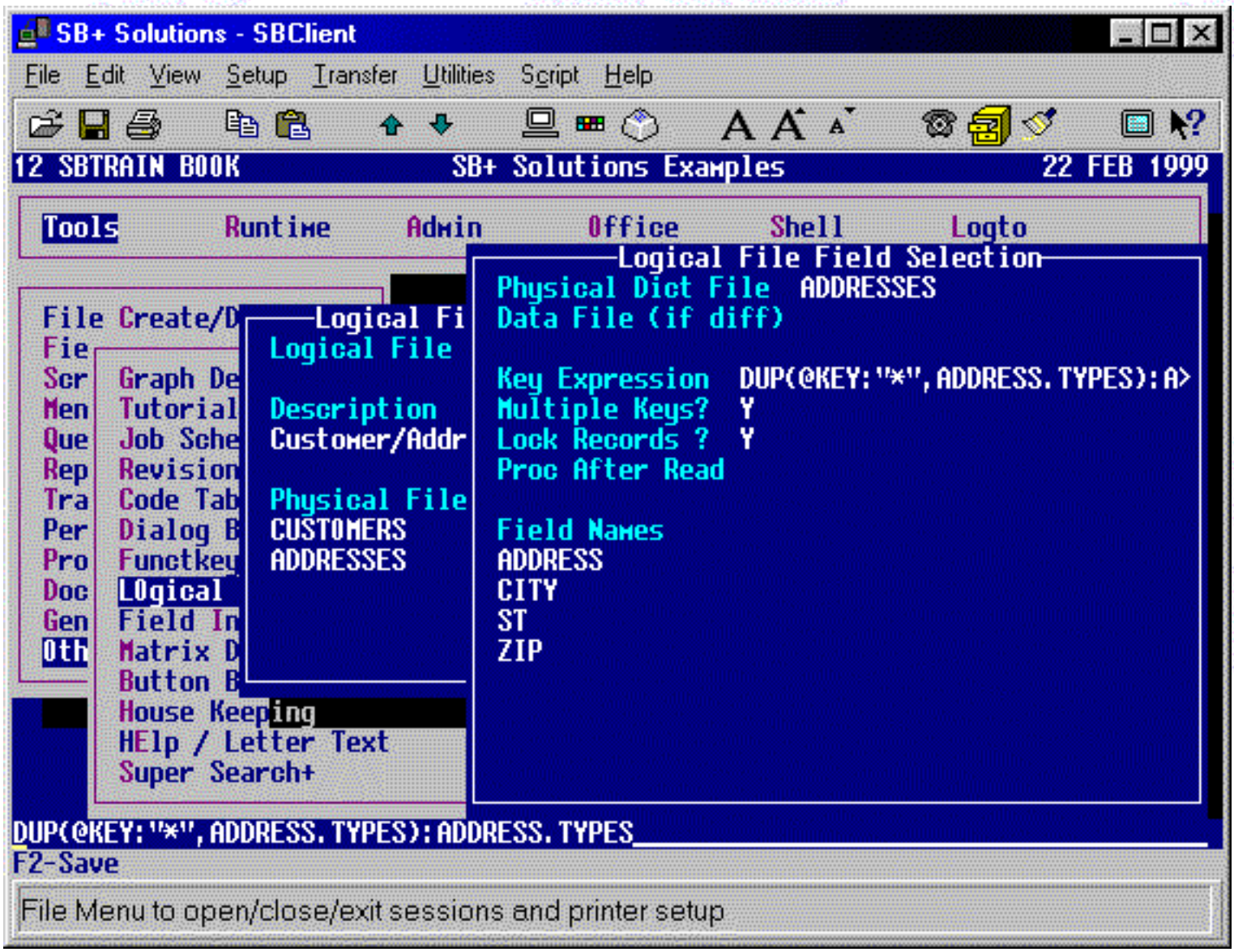
At the Lock Records prompt, we've entered a "Y" (yes) to tell SB+ that we want customer records to be locked during entry and editing. If in your projects the records don't need to be locked (which is dangerous at best), you can enter "N" here.

The Proc After Read is a process slot which allows you to call a process immediately after the customer record has been read. Note that this is different from the **Proc After Read Rec** on the screen definition -- this process is executed once for each physical file in the logical file definition. The Proc After Read Rec on the screen, however, is executed once after all physical files in the logical file definition have been read.

In the process called in the Proc After Read slot, if @RTN.FLAG is set to 1, the record (and all associated records) will not be loaded. This is the same as **rejecting a read** in the standard Proc After Read Rec slot in a screen definition.

Following the Field Names prompt is a list of fields in this file that will be updated by the logical file. For our CUSTOMERS file, we're only going to be updating the name and a list of address types, though notably there could be numerous other fields in this file.

When we press F2 on this screen, the definition screen for the ADDRESSES file appears, as follows:



Note the key expression used here. (I've brought it down to the bottom of the screen so you can see it all.) The key to the address file is the customer ID (which is in @KEY) concatenated to the address type. However, because the address type is multivalued, we need to duplicate the customer ID and delimiting asterisk (using the DUP **expression** **function**) so we have a customer ID and asterisk to be added to each address type. Without DUP, if the customer ID is XYZ and the address types are H and O, the keys

will be:

```
XYZ*H
O
```

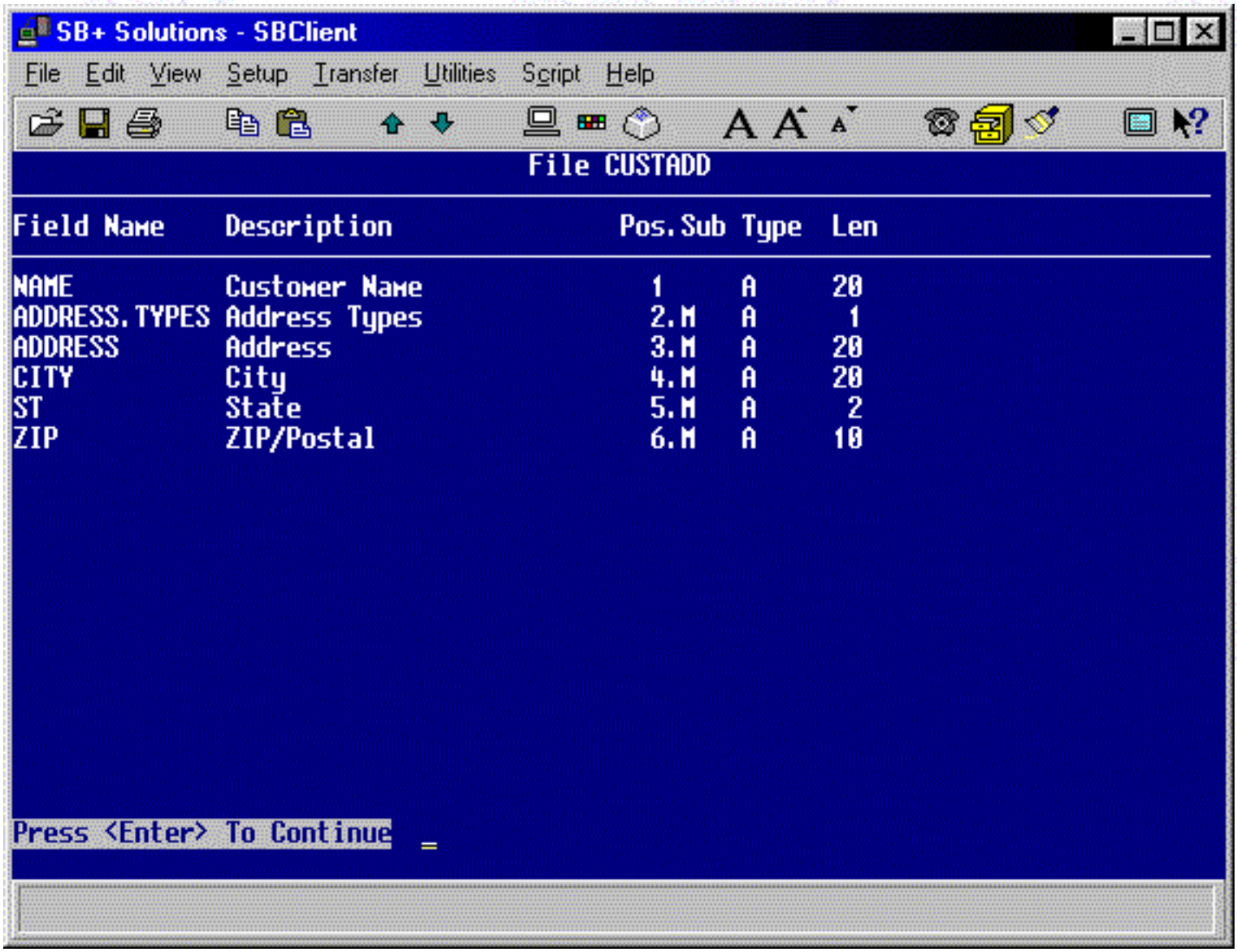
Note that the customer ID and asterisk aren't added to the second key -- that's because there's only one customer ID and asterisk to be added to two address types. Using the DUP syntax above, the address keys will correctly be generated as:

```
XYZ*H
XYZ*O
```

When we press F2 after entering the details on this screen, SB+ will display a brief warning message, and then the logical file has been created. We can then **create the screens** we want to use with this file.

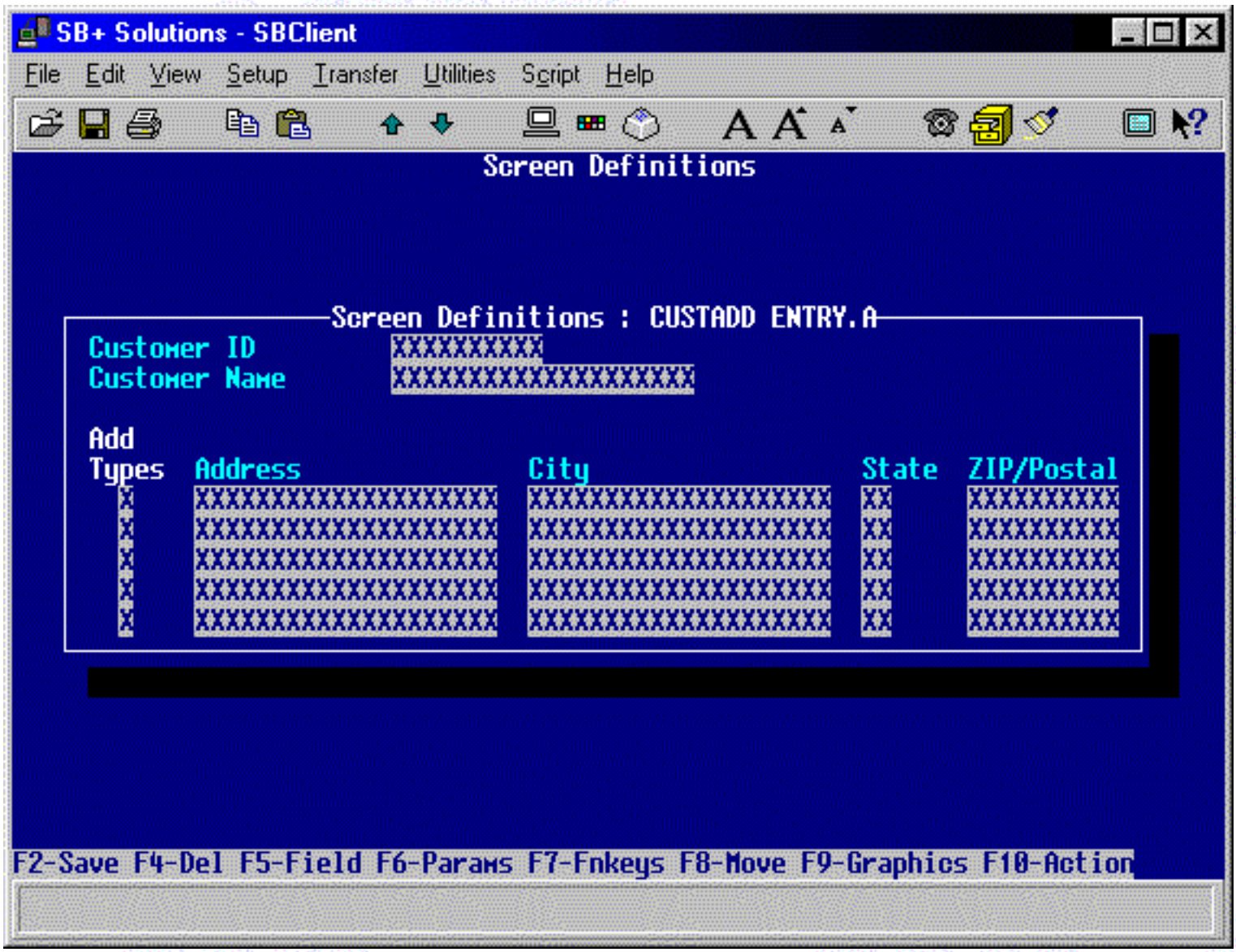
Creating a Screen in a Logical File

Creating a screen for a logical file is no different than creating a screen for a physical file, with one exception. When the logical file is created, SB+ creates dictionaries in the logical file for all the fields in other files. This usually causes some concern, because the attribute positions for fields in the logical file seem to be related in no way to the attributes in their original files. Using our earlier example, let's see what fields SB+ created in our CUSTADD file:



While it does appear that the attributes are in the wrong place, SB+ maintains the connection behind the scenes in the logical file definition (stored in the dictionary of the logical file). In short, there's nothing to be concerned about.

Next, using the **Screen Definitions** tool, create a screen which appears as follows:



Note that I typed the "Add Types" prompt label directly onto the screen so we could have a stacked column heading. See "**Painting Text on a Screen Definition**" for more information about this topic.

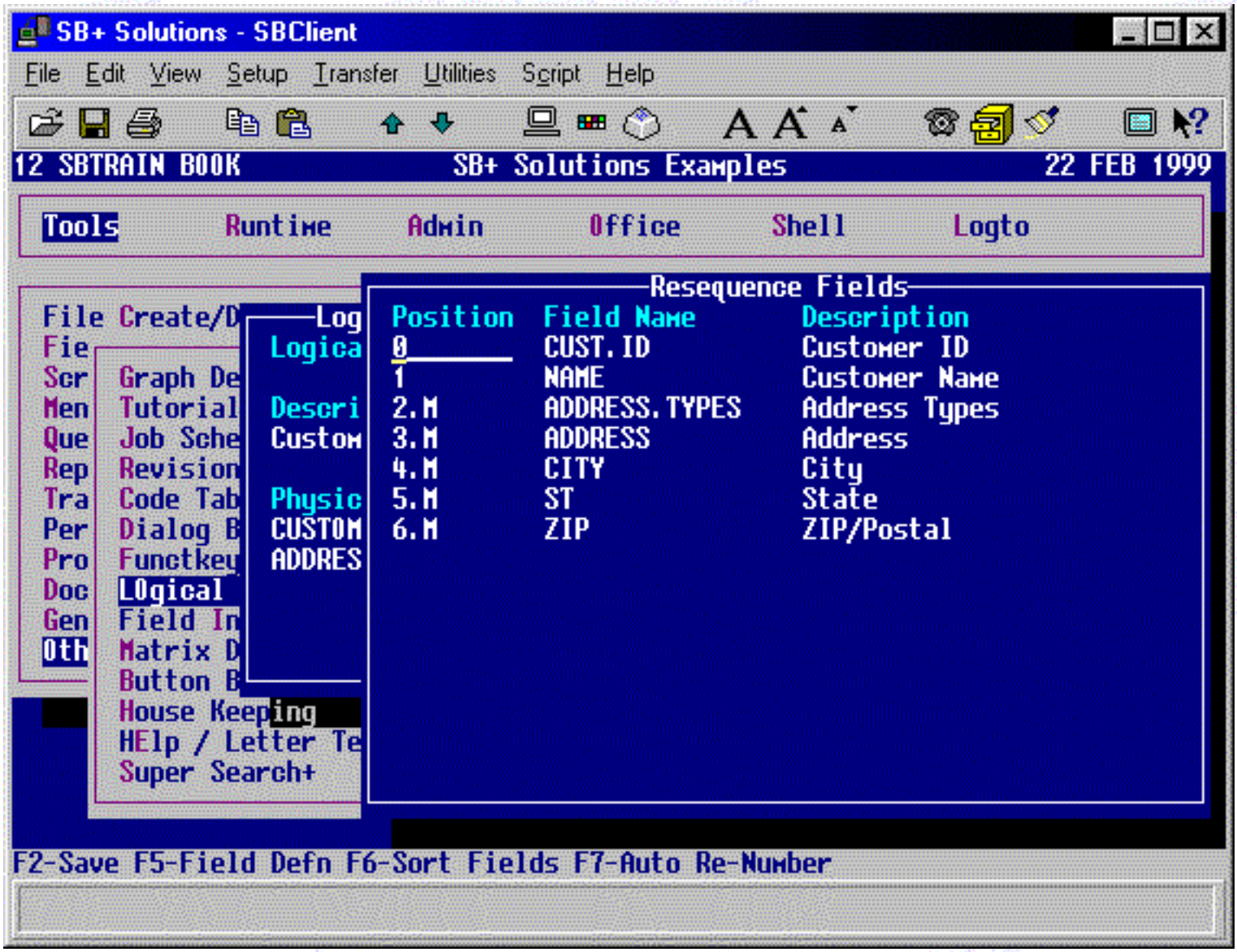
Like a normal screen definition, you can add **default values**, **validations**, **intuitive help**, **F1 help**, **conversions**, **derived values**, **function keys**, and special processing to this screen.

When this screen is running, we're allowed to enter a customer ID and any number of address types and corresponding addresses. When F2 is pressed, a record is written to the CUSTOMERS file with the name we enter, and separate record(s) are written to the ADDRESSES file for each address type.

Resequencing Fields in a Logical File

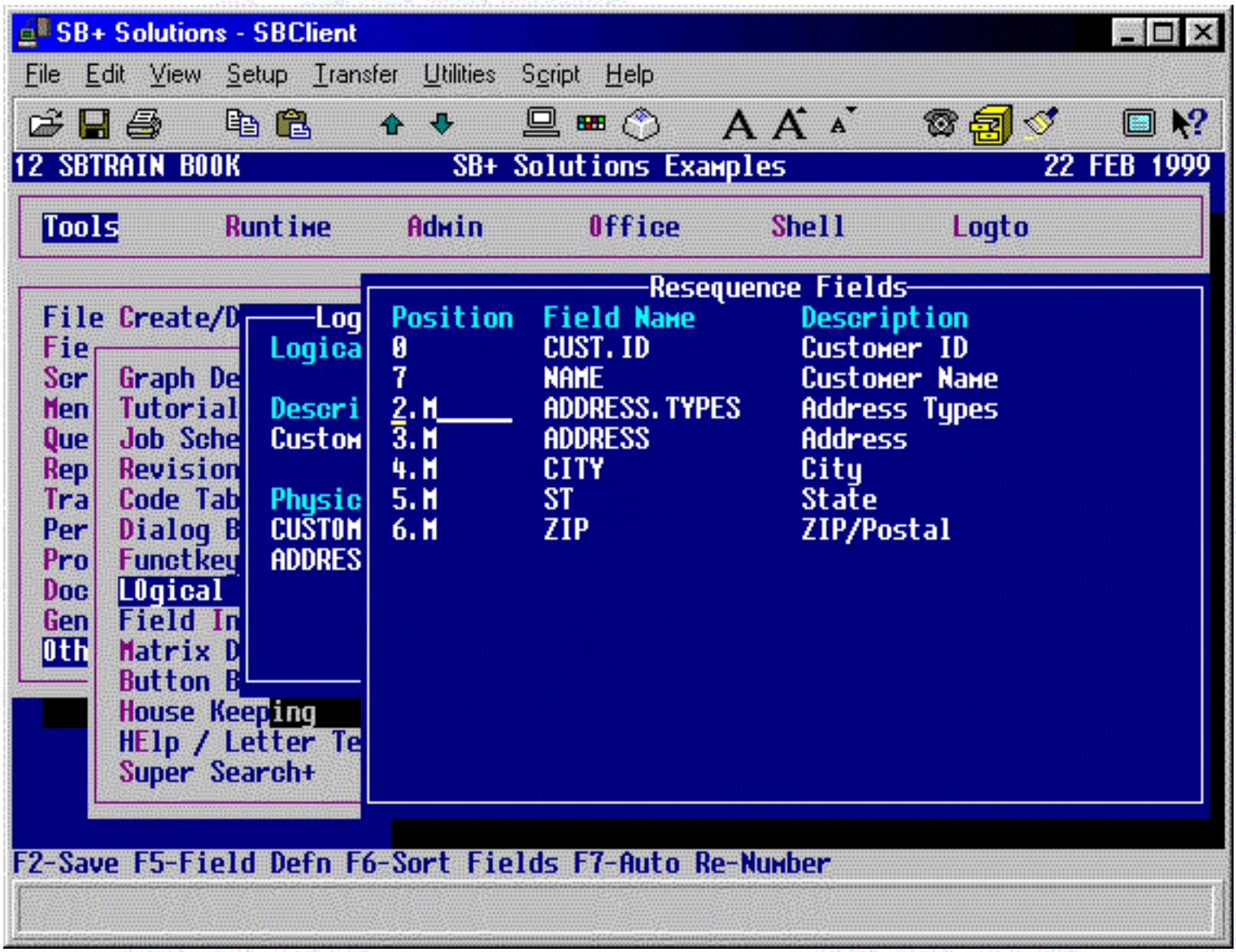
When you create or edit the logical file definition, SB+ creates **field definitions** in the logical file. You may not care for the order that SB+ uses when creating the fields. Fortunately, the field order is easily changed.

In the Logical File Definition tool, load a logical file definition and press F5-Resequence Dict. The following screen will be displayed:

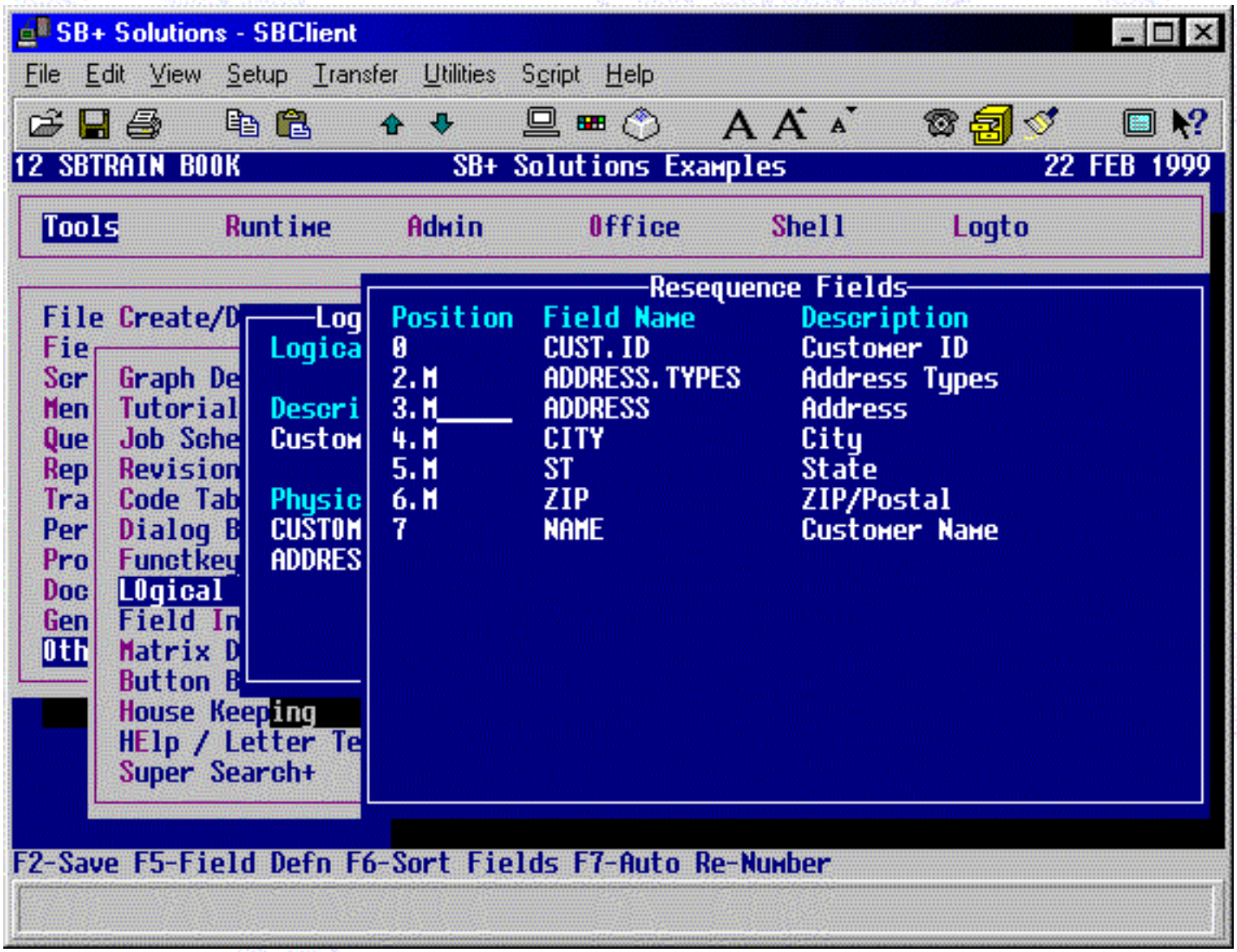


On this screen, you can rearrange the field order in the logical file. Note that this will have no effect on the prompting order of these fields on any screens you may have created.

To move a field to another position in the file, enter a new value in the Position column. For example, to move the NAME field to be position 7, the following could be entered:

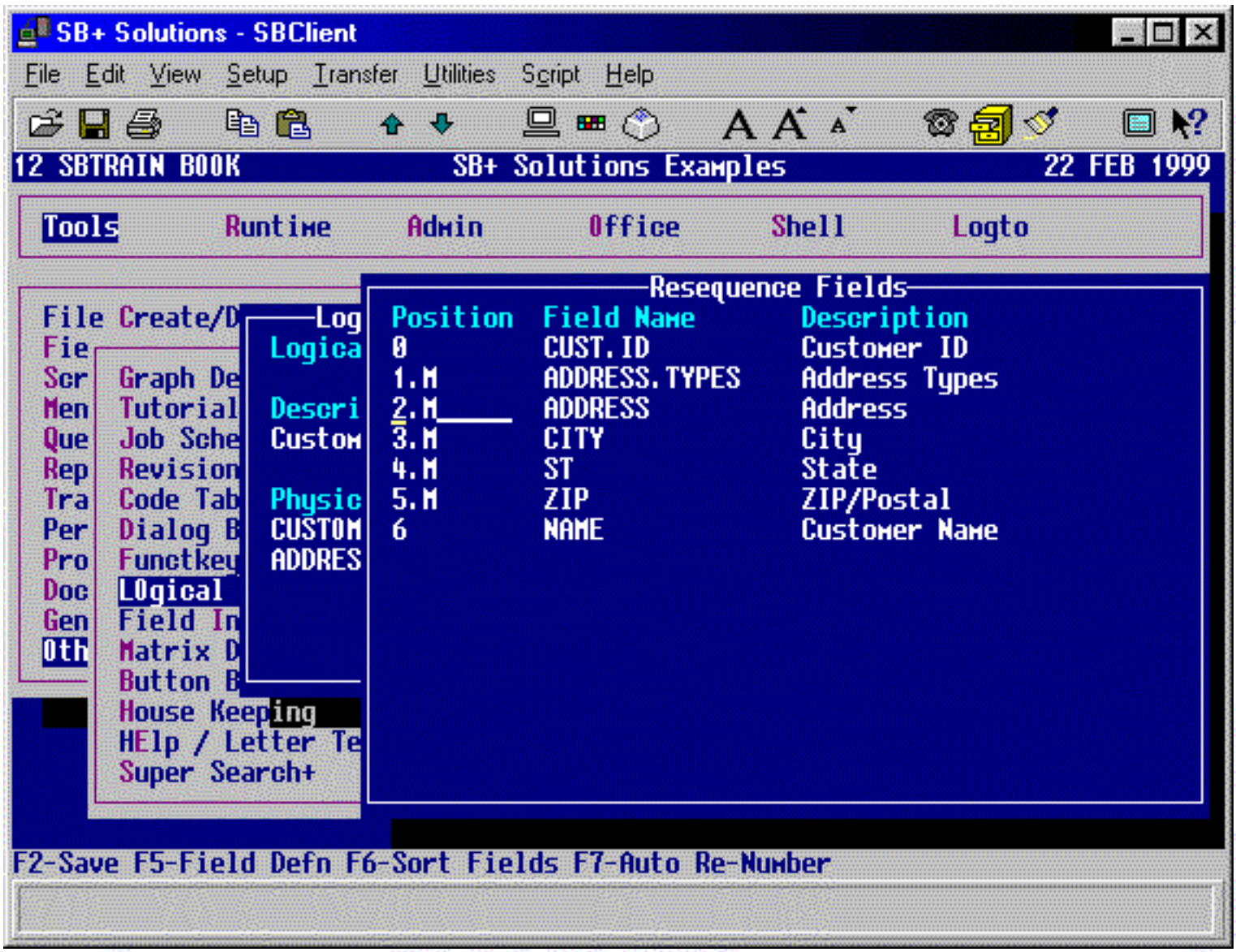


Once you've moved a few fields, you can refresh the display to show the field in its proper place. To do this, press F6-Sort Fields. Once pressed, this key will sort the fields in attribute position order and will redisplay them as follows:



To edit any field in the logical file, simply move the cursor to the row of the field you want to edit and press F5. The field definition screen will then be displayed where you can make any changes you need.

Finally, to renumber the prompts in the logical file (from 1 to *n*), press the F7-Auto Re-Number key. Once pressed, this key will resequence the fields in the file one-by-one, closing any gaps you may have created by changing the field positions, as follows:



When renumbering, always remember that any fields that have the same field position before renumbering will also have the same field position after renumbering!

Regenerating Fields for a Logical File

When **creating** or **resequencing** fields in a logical file, it's important to keep track of any derived values you may have which reference fields in the logical file. When the field positions are changed in a logical file, the calculations based on those fields aren't automatically updated, and as a result will no longer function after the resequence. Therefore, you may need to regenerate the derived fields (or better stated, regenerate the **expressions** for the derived value fields) after resequencing the prompts in the logical file.

On the logical file definition screen, there is a function key named F6-Regenerate. When you press this key, SB+ will regenerate the expressions for any calculated values referenced in the logical file. If there are any errors with the regeneration, such as fields which are no longer in the logical file, or expressions that are incorrect, appropriate messages will display. If no messages are displayed, the regeneration of the derived field expressions is complete and the calculated values should work as expected.

Considerations for Logical Files

While logical files provide a convenient method for updating multiple files, they are by no means a perfect solution for every kind of multi-file update. Consider, for example, these limitations:

- There must be one physical file created for each logical file. These files hold no information, but they do add an entry to the MD or VOC of the account. Therefore, if a great number of logical files are used in a system, there will be a good deal of MD/VOC clutter as a result. (Incidentally, an undersized or cluttered MD/VOC can cause everything to run at less than peak efficiency.)
- When entering transactions, such as invoices, accounts receivable, payables, etc., it can be beneficial to have the original transaction intact for an effective audit. Granted, logical files are intended to not update a transaction file with the record as a whole, so for situations where you need to be able to reconstruct a transaction in its entirety, the logical file may not be the best choice. See "**Updating Several Records Simultaneously**" for another approach.
- If there is a 1:1 relationship between the main record and a secondary record, multivalued fields work just fine. However, if there is a 1:*n* relationship between the main record and secondary record, only single-valued fields can be updated in the secondary file.
- (Personal opinion) It's too easy to resequence fields in the logical file. When writing software on a development system and deploying it on a customer system, if someone resequences the fields in the logical file, all of the field definitions will need to be sent to the customer's system or the logical file won't work properly.

Implementing a Hidden Key

A *hidden key* is simply a key value that is not displayed on the screen. Instead, the screen reads the record based on the hidden key and displays the contents of the record with no visible reference to the actual key value.

This technique is useful for a variety of different applications. For example, in security-aware applications you may want to keep the record ID private. Or, if a screen is updating a control record over and over and the user doesn't care or need to know the actual record key, this technique is particularly valuable.

On a screen definition, F6-Params, there is a slot called "Key Value (If Not Inp)". If an expression is entered into this slot, the screen will process the expression to ascertain the key and therefore expects no key field(s) to be present in the screen definition.

To see this at work, let's assume we have an application that takes parameters from a record in the PARAM file called CONFIG. The screen that updates this record shouldn't prompt for the key, because the key will always be the same and the user doesn't necessarily know (or care) what the key value is. Therefore, by entering "CONFIG" (with the quotes) into the Key Value slot in the screen definition, SB+ will automatically load the appropriate record when the screen is invoked, without prompting for the key.

This technique has an interesting side effect as well. When F2 or F4 is pressed from the screen, the screen will save or delete the record and then exit the screen completely. There's no reason to prompt for more information when the screen can only edit one record.

It's also interesting to note that this particular technique can call a process to calculate the key value, using the P(...) function in the expression. Knowing this, there is little that you cannot do in calculating a hidden key.

Implementing a Virtual Key

Normally, the value entered for a key is the actual value that is used to read a record from the file. However, in certain applications it may be necessary for the screen to translate the value entered by the user to the actual key value behind the scenes. This type of a key is called a *virtual key*.

For example, assume we're creating a customer entry screen. We want the user to be able to enter a customer's social security number, but we don't want to use the social security number as the key to the record. To complicate this further, we may want to retrieve customer records based on both the customer's and spouse's social security number. In summary, we want to create a customer entry application that allows us to enter any number of different alternate keys for a single customer and have the system find the right record.

To implement this technique, we'll let the system assign sequential numbers for the customers behind the scenes, and then cross-reference this internal customer number by all of the different alternate keys. Therefore:

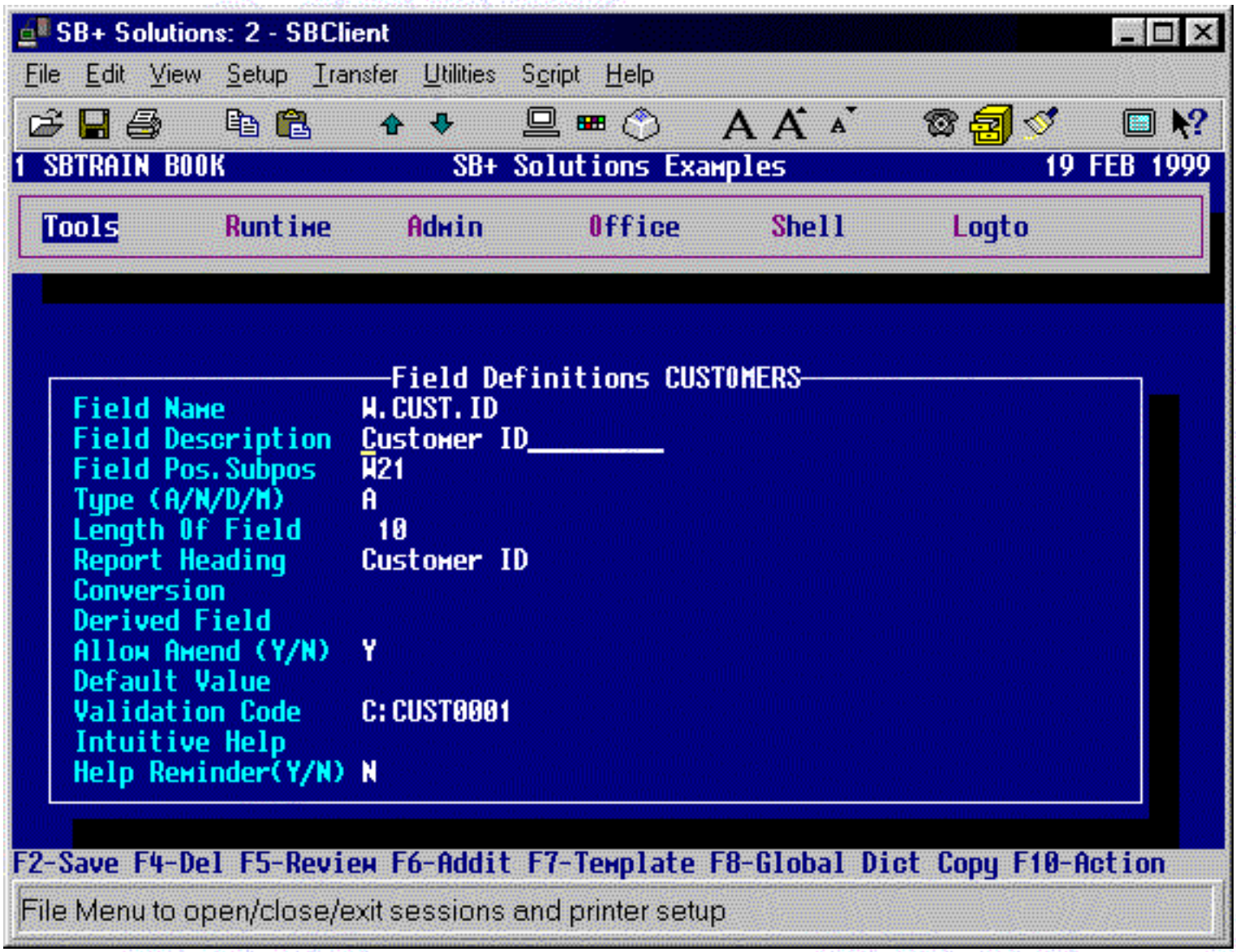
- We need a file where the alternate keys will be stored.
- We need a work field where the alternate key can be entered.
- We need a process to translate the alternate key as entered into a real key that can be used to read a record.
- We need a process to assign real keys to new records and update the alternate key file.

To create a file for the alternate keys, simply use either the SB+ File Create (/FC) process. Once the file has been created, we can create simple field definitions for the file using the Field Definitions tool (/FD) as follows:

Field Name	Pos
ALT.KEY	0
REAL.KEY	1

(In this case, creating field definitions for this file won't serve much of a practical purpose, other than documenting the fields in the file. We won't use the actual field names in any **expressions** or processes.)

Next, we need to create a work field where the user can enter a value to be used to find the actual key. This work field can be entered into the main file's dictionary using the Field Definitions tool. For the sake of this example, let's create a field called W.CUST.ID as follows:



This field definition defines a work field which will store input in the 21st attribute of the **common variable** @WORK. On the actual entry screen, we'll place this field for the entry of the key, rather than placing an actual key field on the screen.

Note that this field calls a process (named CUST0001) for validation. This process is where the value entered is translated into the actual key value, and is a paragraph that appears as follows:

```
@RTN.FLAG = 0
READ @OTHER.REC FROM 'altkeyfile',@VALUE
IF @RTN.FLAG THEN
  @KEY = 'NEW' : @PORT
END ELSE
  @KEY = O1
END
EXIT 0
```

(Of course, in this syntax *altkeyfile* should be replaced with the name of the file where the alternate IDs are being stored.)

This process simply takes the value that has been entered and attempts to read a record from the alternate key file. If the record is found, we return the actual key value from attribute 1 of the record. If no record is found, however, the process sets the key value

to "NEW" concatenated to the current port number. Like a typical screen with a sequentially-assigned key, this token will then be checked in the Process After Screen Accept and a real key can be assigned at that time.

But why call this process as a **validation** process? Couldn't it just as well be referenced in the Process After slot on the same field?

In short, while this process could certainly be called from either of these locations, the validation process slot is the preferred option. This process slot allows you to translate the value entered not only into a key value, but also into a primary display key as well. To illustrate this, assume our customer records have a primary key, which is the customer's social security number, and any number of alternate keys (federal ID numbers, spouse's social security number, etc.) When the user enters any of the alternate keys, we want to get the actual key value and also translate the value entered into the primary key for display. Assuming the primary key is in attribute 1 of the customer record, we could adapt the previous process as follows:

```
@RTN.FLAG = 0
READ @OTHER.REC FROM 'altkeyfile',@VALUE
IF @RTN.FLAG THEN
  @KEY = 'NEW' : @PORT
END ELSE
  @KEY = O1
  @VALUE = F('mainfile',O1)<1>
  @REFRESH = 7
END
EXIT 0
```

While this type of translation can also be done from the Process After slot, using a validation process is simpler and is a better use of SB+.

Finally, we need a series of processes to update our alternate key file. We'll start with a Process After Update which is a paragraph with the following lines:

```
CASE @ACTION = 1
  EXEC 'ins_list'
CASE @ACTION = 2
  EXEC 'upd_list'
CASE 1
  EXEC 'del_list'
END CASE
```

The *ins_list* process appears as follows:

```
LOCAL LIST,NDX
*
IF <POS(ALT.IDS)> # '' THEN
  LIST = INS(W.CUST.ID,1,-1,<POS(ALT.IDS)>)
END ELSE
  LIST = W.CUST.ID
END
*
NDX = DCOUNT(LIST,@VM)
WHILE NDX DO
  WRITE @KEY ON 'altkeyfile',LIST<1,NDX>
  NDX = NDX - 1
REPEAT
```

Note that this process simply makes a list of the primary ID (W.CUST.ID) and any list of alternates (ALT.IDS) and then writes one record in the alternate key file for each.

The *upd_list* process appears as follows:

```
LOCAL ALT.ID,INS.LIST,DEL.LIST,NDX
*
INS.LIST = ''
DEL.LIST = ''
*
NDX = DCOUNT(<POS(ALT.IDS)>,@VM)
WHILE NDX DO
  ALT.ID = <POS(ALT.IDS),NDX>
  IF LOC(ALT.ID,@ORIG.REC<POS(ALT.IDS)>,@VM) = 0 THEN
    INS.LIST = INS(INS.LIST,1,-1,ALT.ID)
  END
  NDX = NDX - 1
REPEAT
*
NDX = DCOUNT(@ORIG.REC<POS(ALT.IDS)>,@VM)
WHILE NDX DO
  ALT.ID = @ORIG.REC<POS(ALT.IDS),NDX>
  IF LOC(ALT.ID,<POS(ALT.IDS)>,@VM) = 0 THEN
    DEL.LIST = INS(DEL.LIST,1,-1,ALT.ID)
  END
  NDX = NDX - 1
REPEAT
*
NDX = DCOUNT(INS.LIST,@VM)
WHILE NDX DO
  WRITE @KEY ON 'altkeyfile',INS.LIST<1,NDX>
  NDX = NDX - 1
REPEAT
*
NDX = DCOUNT(DEL.LIST,@VM)
WHILE NDX DO
  DELETE 'altkeyfile',DEL.LIST<1,NDX>
  NDX = NDX - 1
REPEAT
```

This process rationalizes the differences between the original list of alternate keys and the current list of alternate keys, and inserts and deletes records from the alternate key file as appropriate. Notice we're not doing anything with the primary key (W.CUST.ID); For the sake of this example we'll assume that this field cannot be changed on the screen.

The *del_list* process appears as follows:

```
LOCAL LIST,NDX
*
IF @ORIG.REC<POS(ALT.IDS)> # '' THEN
  LIST = INS(W.CUST.ID,1,-1,@ORIG.REC<POS(ALT.IDS)>)
END ELSE
  LIST = W.CUST.ID
END
*
NDX = DCOUNT(LIST,@VM)
WHILE NDX DO
  DELETE 'altkeyfile',LIST<1,NDX>
```

```
  NDX = NDX - 1
REPEAT
```

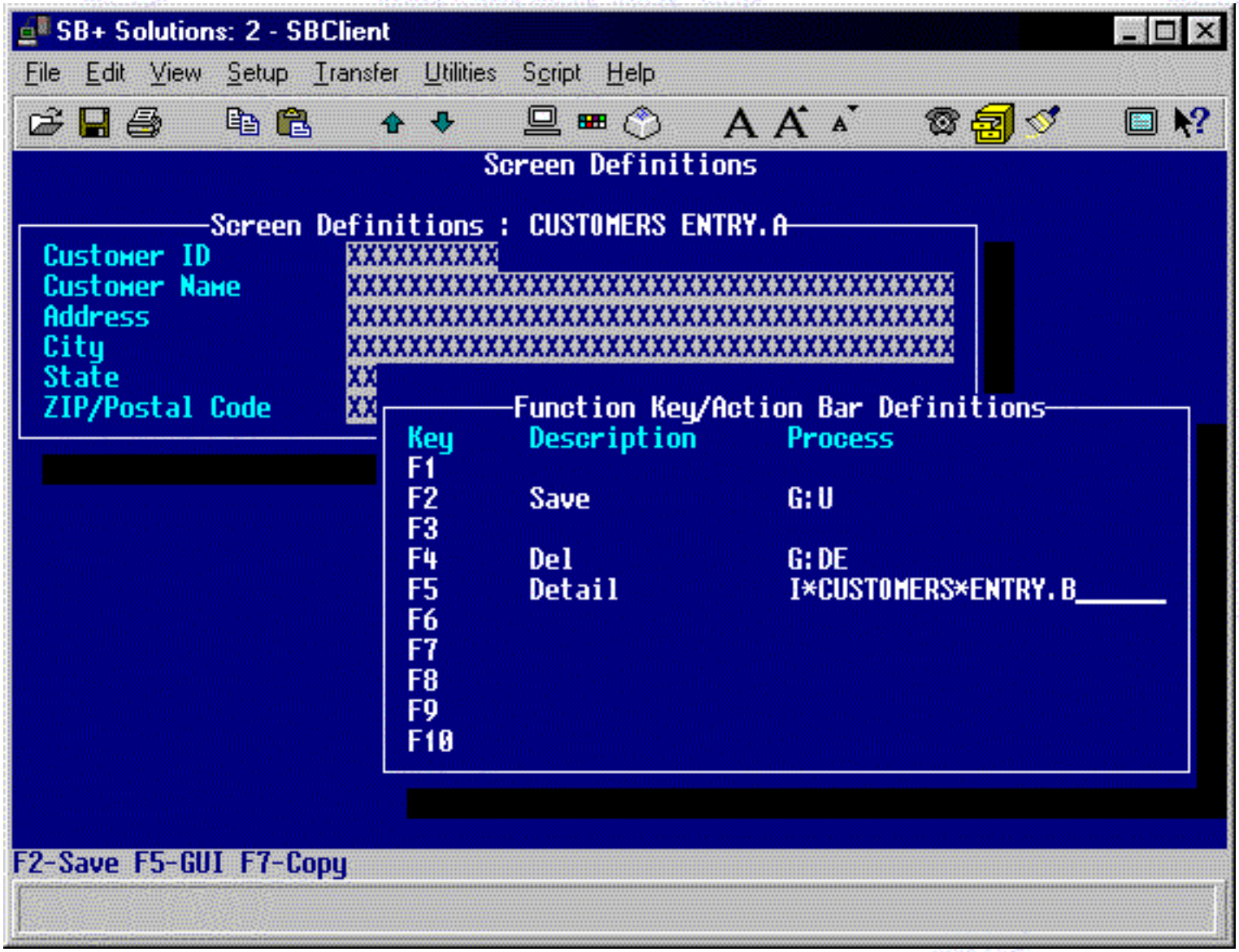
This process is almost exactly the reverse of the *ins_list* process. However, note that while we insert new records into our alternate key file based on the current record, we delete records from the alternate key file based on the original record as of the time it was loaded into this screen. In other words, we don't care what editing may have been done to the record prior to the delete, as we will be deleting the alternate key records based on the record as it was prior to the edits.

Finally, it's very important to set the Control/Dep/Read flag to "R" on the work field used for the key entry. This tells SB+ to read the record after the entry is made in the work field. Without it, SB+ will never read the record!

Implementing a Manual Subscreen

By definition, a manual subscreen is a subscreen that is invoked via the press of a function key. It is called *manual* because the user must perform some manual task in order to invoke it.

To link a subscreen to a main screen this way, simply press F7-Fnkeys on the screen definition for the main screen, and add the process call for the subscreen as one of the function keys, as follows:



Under certain circumstances, it may be best not to call the process directly. To illustrate this point, allow me to tell a true story -- though the names have been changed to protect the innocent.

One of my first SB+ projects was to create a sales order entry application with a main screen for entering products and a subscreen (called manually from function key F5) for entering payments. After the design was finished, we completed development and testing and installed the screen in the production environment.

Not long thereafter, the client began complaining that the screen was losing all of the payment information. The users of the

screen would enter the product information, press F5, enter the payment information, and then save the record. However, none of the payment information was being saved.

Oddly enough, when we (the developers) tested the application, it worked flawlessly. No matter what we did in testing, we simply couldn't replicate the problem. Not to worry, though, the users were able to replicate the problem well enough without us, albeit 2000 miles away.

Had we been able to watch someone enter an order, it would have become instantly obvious as to the nature of the problem. You see, the users weren't entering product information first; They were entering payment information first. Once the payment information had been entered, then they entered an order ID (which was usually "NEW") and then the product information. (Once they entered the order ID, the order was loaded from the file and the payment information they entered on the subscreen was lost.)

Therefore, if blame were to be cast, it was a combination of both parties. We allowed them the ability to enter subscreen information prior to entering a key for the record, and they simply used it, not knowing better.

The main point of this story is to illustrate the need for a process that checks if a key has been entered prior to calling another process. In my toolbox this process is called SUBSCREEN, and can be created as a paragraph as follows:

```
IF (@KEY # '') THEN
  EXEC @PARAM
END ELSE
  ERROR 'Please enter a key before using this feature.'
END
```

With SUBSCREEN, you pass the name of the process you want to call in the parameter. SUBSCREEN then checks to see if a key has been entered, and if so, calls the process as named. If no key has been entered, an appropriate error message is displayed. Therefore, if you want to call:

```
I*CUSTOMER*ENTRY.B
```

...but only want it called if the key has a value, enter the following instead:

```
SUBSCREEN,I*CUSTOMER*ENTRY.B
```

Implementing a Popup Subscreen

By definition, a *popup subscreen* is a subscreen that "pops up" simply by moving the cursor to or from a prompt. This type of subscreen is typically used to maintain a proper prompt flow when all the prompts will not fit onto a single screen.

To implement a popup subscreen, simply create a regular subscreen, but instead of linking the subscreen process to a function key, link the subscreen process to the Process Before or the Process After a field on the F5-Field screen.

When two screens are linked together this way, after the user has entered a value into the prompt, the subscreen will pop up (hence the name). Once the user has pressed F2 from the subscreen, the main screen will be redisplayed, and the cursor will advance to the next prompt.

Process Slot Overview

The following sections describe all of the process slots available on an input process, their purpose, and some considerations as to their use.

Process Before Process: (Process Definition - Input [/PD.I])

When an input process is invoked, the process as listed in this slot is executed first. This process will be called only once for the entire execution of the input process.

This process is typically used either to validate that the process can be run (for additional security), to load configuration information into @PARMS(...), or both.

Process Before Screen Display: (Screen Definition [/SD], F6-Params)

Prior to the screen being displayed, the process as listed in this slot is executed. This process will be called each time the screen is redisplayed (except when "/R" is used). This process slot is useful for clearing work variables and adding additional information to the @SCR.IMAGE **common variable** to augment the screen display.

Process After Screen Display: (Screen Definition [/SD], F6-Params)

Immediately after the screen has been displayed, the process as listed in this slot is executed. This process will be called each time the screen is redisplayed (except when "/R" is used). This process is useful for clearing work variables.

Process Before - Key Field: (Screen Definition [/SD],F5-Field)

Prior to prompting for the key field, or a part of the key field (as in the case of multi-part keys), the process name listed in this slot is executed. This process will be called each time the cursor moves to this field.

Typically, this slot is used to clear work fields that are used in the construction of a multi-part key or to set a default of "NEW" for screens where the key will be assigned at file time.

Default Expression - Key Field: (Field Definition [/SD],F5-Field, F5-Field Defn)

If a default expression includes a process call (i.e. P(...)) and the field has no value, or the default for a field is a mandatory default, the process named in parentheses will be executed. When a process is called this way, the process should be used to return the calculated default in @VALUE. No other fields in @RECORD or @WORK should be changed by the default process, however, @OTHER.REC and @PARMS(...) may be updated for later use.

Validation Code - Key Field: (Field Definition [/SD],F5-Field, F5-Field Defn)

If the "C:process" validation code is used, each time a person enters a value into the field the process specified after the "C:" will be executed. This process takes @VALUE as the value entered (after input conversion), and is responsible for verifying the entry. If the entry is acceptable, @RTN.FLAG is expected to be returned as "0". If the entry is to be rejected, @RTN.FLAG should be

set to a "1".

If part of the validation is a test to see if the record exists, do not make this a part of the field validation on the key field in this process slot. If you validate that the record exists as part of the validation in this slot, the input process will read the record again, which is unnecessary. Instead, use a process call in the **Process After Read** which tests @ACTION, and if @ACTION is "1", you know the record was not found.

Process After - Key Field: (Screen Definition [/SD],F5-Field)

This process is called after the validation for the key field has verified a successful entry. This process slot is useful for constructing the key for a multi-part key screen, or concatenating the port number onto "NEW" for a key to be assigned at file time. Also, like any process before or after a field on the screen, this process call may be used to conditionally call a **subscreen**. This is uncommon, however, on the key field of a screen.

Process After Read Record: (Screen Definition [/SD], F6-Params)

This process is called immediately after the record has been read from the file. At this point, @RECORD has the record, @KEY has the key value, @ORIG.REC has a copy of the record, and @ACTION is either "1" or "2" to denote a new or existing record, respectively.

This process is used primarily for two different tasks: 1) Verify that the record can be loaded, and 2) read additional information from other sources as necessary. Note that at this point, the record is locked. If this process exits with @RTN.FLAG = 1 the cursor will return to the key prompt (or last prompt of the multi-part key) again without releasing the lock. In this case, you must release the lock before the process finishes, or the lock will remain until you (or your user) log off.

When both validation and loading information must be done in this process slot, it is not recommended to do the two together inside of one process. Instead, use a paragraph like the one shown following, which calls other processes to do the validation and loading of additional information.

```
EXEC 'validation_process'
IF @RTN.FLAG = 0 THEN
  EXEC 'load_process'
END
```

Process Before - Data Fields: (Screen Definition [/SD],F5-Field)

This process is executed once per each single valued prompt, immediately before the user is requested to enter values into that prompt. Multivalued fields, however, function differently. For multivalued fields, this process is executed once prior to the first multivalue in the set, and is ignored for all subsequent values.

Like any process before a field, this process is useful for conditionally calling a **subscreen** or other similar type of process. This process is more useful, though, for calling a process to skip the field conditionally.

Default Expression - Data Fields: (Field Definition [/SD],F5-Field, F5-Field Defn)

If a default expression includes a process call (i.e. P(...)), and the field has no value, or the default for the field is a mandatory default, the process named in parentheses will be executed. When a process is called this way, the process should be used to return

the calculated default in @VALUE, leaving all other values in @RECORD and @WORK intact. However, @OTHER.REC or @PARMS(...) values may be updated for later use.

Unlike the Process Before on multivalued fields, when a process is called from the default expression of a multivalued field, it is called once for each multivalue, not only once for the entire multivalue set. For this reason, this process slot can be used to call a process to limit the number of multivalues that can be entered into a set.

Validation Code - Data Fields: (Field Definition [/SD],F5-Field, F5-Field Defn)

If the "C:process" validation code is used, each time a person enters a value into the field the process specified after the "C:" will be executed. This process takes @VALUE as the value entered (after input conversion), and is responsible for verifying the entry. If the entry is acceptable, @RTN.FLAG is returned as "0". If the entry is to be rejected, @RTN.FLAG should be set to "1".

Process After - Data Fields: (Screen Definition [/SD],F5-Field)

This process is called immediately after a field has been entered and validated successfully. It is typically used to call a **subscreen** conditionally based on the value entered into the field.

Under no circumstances should this process be used to augment the validation on the field. (By the time this process is called, the value as entered has been updated into the record or work field. Rejecting the value in a validation called from this process will not reset the original value, and so this technique should never be used.)
Process After Screen Accept: (Screen Definition [/SD], F6-Params)

This process is called when @RTN.FLAG is set to "U", "D", or "DE", prior to the record being written. It is used to verify that the record can be written, and also to update other values in @RECORD prior to the write. Note that even if the "Write Record" flag is "N" or null, this process will still be executed when @RTN.FLAG is set as described above.

Though @RTN.FLAG is set prior to calling this process, you cannot check the value of @RTN.FLAG in the process to determine which value caused the process to be called. Instead, check @ACTION. If @RTN.FLAG was "U", @ACTION will be 1 or 2. Otherwise, @ACTION will be 3.

The following paragraph can be used as generic template for a Process After Screen Accept (and also Process After Update):

```
CASE @ACTION < 3
  EXEC 'ins/upd process'
CASE 1
  EXEC 'del process'
END CASE
```

In this process, if @RTN.FLAG was "U" when the process was called, the "ins/upd process" will be called to insert or update fields as necessary. If @RTN.FLAG was "D" or "DE" when the process was called, the "del process" will be called to delete anything as necessary.

Process After Update: (Screen Definition [/SD], F6-Params)

This process is called when @RTN.FLAG is set to "U", "D", or "DE", after the record has been written. It is used to update other records in other files when a record is saved.

This process should never terminate with @RTN.FLAG set to anything other than "0". If this happens, the screen will behave very strangely.

Process At Escape: (Screen Definition [/SD], F6-Params)

This process is called when changes have been made to a record and then Escape is used to exit the record. It is used to reverse any changes that have been made during the execution of the screen. For example, when an order processing system encumbers inventory
as stock numbers are entered onto a screen, if the order is not saved the inventory must be unencumbered to restore the values as they were prior to the order information being entered.

Process After Process: (Process Definition - Input [/PD.I])

This process is called after the user has exited the screen, and is typically used to reset any values that may have been set (such as security parameters) by the Process Before Process. This process will be called only once for the input process, and will be the last thing that happens before the process terminates.

Input Screen Prompt Cycle

Every time the cursor moves to a prompt on an input screen, several things happen:

- If the prompt has a Process Before, it is called.
- Next, the value is extracted from either @WORK, @RECORD, or @KEY, depending on the field position defined on the field definition. If the field is multivalued, the value pointed to by the variable @CNT is extracted.
- If the extracted value is null, or the prompt has a mandatory default, the default expression (if present) is executed.
- If the field has an output conversion, the extracted value or default value is converted and displayed for input.
- The user is then prompted to enter or edit the value. If the user enters anything other than a function key, the remaining steps will be taken. (Function keys interrupt this flow and cause the remaining steps to be skipped.)
- If the prompt has an input conversion, the value entered is input converted with the code. Incidentally, date fields are automatically input converted, whether an input conversion exists on the field or not.
- The field is then validated using the Validation Code from the field definition. If the validation code is a C: code, the specified process is called. If the validation fails, control returns to step 3.
- The value is replaced back into @RECORD, @WORK, or @KEY based on the field position defined on the field definition. If the field is multivalued, the value is replaced based on the multivalue position pointed to by the **common variable** @CNT.
- If the field has an output conversion, the value is converted for output and redisplayed.
- If the field has a Process After, it is called.

Understanding Input Process Drivers

When an input process is started (running interpretively), SB+ checks the current drivers file to see if drivers exist for the process. If not, SB+ creates the necessary driver records and then runs a subroutine to interpret them.

Typically, the name of the drivers file is *XXX*DRIVERS (where the *XXX* is the system ID), though the true name of the driver file and driver ID is found on the Process Definition - Input record, which can be viewed and edited using /PD.I.

Using drivers, SB+ encapsulates nearly all of the information required for running a screen. This is largely for efficiency, as this allows SB+ to read the field definitions for all the prompts on a screen once for the entire execution of the screen, instead of once each time the cursor is moved to each prompt. Because much of the information for a prompt is built into this separate driver record, if something about a field changes it may or may not be readily apparent. Instead, you may need to *rebuild the drivers* to see the net effect of the change.

The following aspects of a screen can be changed while the screen is running and will have an immediate effect:

- Processes called for default or validation
- Intuitive Help processes
- Processes called from function keys or the action bar
- Processes called Before or After a prompt, if they are large enough...
- Processes called from any of the F6-Params slots, if they are large enough...

No doubt you've noticed the ambiguity with a few items in this list. SB+ uses something called the 80-byte rule to determine whether a process is included in the driver or not.

The 80 Byte Rule?

When a paragraph or some other type of process is saved, SB+ parses the source code that was entered and creates a form of pseudo-object code (p-code) inside of the process record. When the process is executed, SB+ doesn't look at the source, but instead looks at the p-code to know what to do.

The 80-byte rule states that if the p-code for a given process is less than or equal to 80 bytes, the p-code itself will be copied into the driver. If the p-code for a given process exceeds 80 bytes, only a reference to the process will be included in the driver, and the process will then be read each time it is needed.

Other than the code that is linked to these process slots, everything else about an input screen is included in the driver. With this in mind, always remember that if the conversion, field position, attribute name, or any other facet of a screen changes, you will need to rebuild the drivers in order to see the change.

Fortunately, rebuilding the drivers file is easy: Simply clear the drivers file and SB+ will create new drivers as needed. Of course, this will add a bit of overhead to your application as SB+ regenerates the drivers, so you'll want to avoid doing this on production environments. (Drivers are also cleared when you exit a tool after changing any part of the software.)

The None - Execute Now feature can also be used to regenerate a set of drivers for an input screen without clearing the entire drivers file. This provides a suitable option when you need to regenerate only one set of drivers.

When are Drivers Automatically Cleared?

Driver records are automatically cleared upon exiting a tool after a process or definition record has been changed.

Interestingly, when drivers are cleared, all drivers are cleared, not just the drivers for the screen being worked on. Therefore, if your application has several thousand drivers on file and someone makes the most minimal change to a field definition, all of the drivers will be erased, and SB+ will be forced to regenerate them as each of the screens are used!

Normally, regenerating drivers happens so quickly that it doesn't leave a noticeable impact on system efficiency. However, depending on how often drivers are cleared, you may notice some appreciable slowdowns in your application.

The best protection against these slowdowns is to be sure your production environment and development environments are totally separate, and the drivers file in the production environment is cleared only when new software is installed in that environment. By minimizing the number of times your production drivers are cleared, you can keep your application running at peak efficiency.

Incidentally...

For the most part, there is no reason to become intimately familiar with the SB+ drivers. They do their job properly and there's rarely a reason to know anything about them.

However, for the academic set...

Releases 2.3 and later of SB+ have a built-in process called DRIVER.DOC which allows you to review the driver records that SB + uses, with annotations throughout to help you understand the format. This process is very useful for getting an understanding of what SB+ "thinks" it's doing, particularly if you suspect SB+ to be mentally impaired.

Output Process Options

There are a few options available on the Process Definition - Output (/PD.O) screen to modify the behavior of your output process (es). In fact, the list of options for output processes is exactly the same as input processes, though many of them don't apply.

There are six options available at the Options prompt on this screen. Each can be used independently, or combinations of codes may be used. These codes are:

A	Restrict amendment: Not applicable to output processes.
D	Restrict deletion: Not applicable to output processes.
I	Restrict insertion: Not applicable to output processes.
O	Non-Amendable: Not applicable to output processes.
N	No refresh: Normally, SB+ refreshes the calling screen when a screen is exited. With this option set, this refreshing is not done.
S	**Subscreen**: This process will share common memory with the calling process. Oddly enough, however, while this may appear to be a very useful option, it is ignored when used with output processes.

"Cruising"

Because the fundamental structure of an SB+ application is just processes calling processes, one query screen can easily call another query screen with the following:

- The calling process must set up the key to be passed to the query to be called.
- If the query to be called extracts its key from common memory, the caller must move the key value or values into common memory.
- If the query to be called prompts for the key value, the caller must data stack the key value for it.
- The process name for the query to be called must be executed.

To illustrate this, let's assume we have two query screens that we want to link together, one for customers and one for sales orders. Customers can have multiple sales orders, but the order can have only one customer.

From the customer query screen, if the user presses the F5 function key, the order query screen should be called to view all of the customer's orders. From the order screen, if the user presses F5, the customer query screen should be displayed. (If this seems cyclic, it is. For the purpose of this discussion, however, this isn't a problem.)

Also for the purpose of this discussion, we'll assume that both of these query screens extract their keys from the variable @VALUE in common memory. This is a very important assumption, as the implementation varies depending on whether the key is prompted or not.

From the customer query screen, the following paragraph can be linked to the F5 key to view the customer's orders:

```
@VALUE = <POS(SO.XREF)>
IF @VALUE # '' THEN
  EXEC 'O*ORDERS*INQ.A'
END
```

For illustrative purposes, these paragraph lines assume the name of the field where the order keys are stored is called SO.XREF, the name of the query process to be called is O*ORDERS*INQ.A, and the process takes a key value (or value(s)) from @VALUE.

On the flip side of this example, we can link the following paragraph to the F5 key for the order screen to invoke the customer query:

```
IF CUST.ID # '' THEN
  @VALUE = CUST.ID
  EXEC 'O*CUSTOMER*INQ.A'
END
```

This paragraph, like the previous paragraph, assumes the name of the field where the customer key is stored is called CUST.ID, and also that the name of the query process to be called is O*CUSTOMER*INQ.A.

With these two little paragraphs, we've quickly linked two query screens to each other. From a developer's point-of-view it doesn't seem like much, but from the user's perspective this can be a very valuable enhancement to the application. As long as there are relationships between files, we can easily tie queries together this way, allowing the user to "cruise" where ever they like.

"Cruising"

Of course, this technique is specifically for screens where the key values are not prompted. For screens where the key values are prompted, slightly different logic must be used. In such a situation, moving from the sales query to the customer query can be done with a small variation on the original process:

```
IF CUST.ID # '' THEN
  @VALUE = ''
  DATA CUST.ID
  EXEC 'O*CUSTOMER*INQ.A'
END
```

With a prompted key, we cannot pass the key value between screens in @VALUE. Instead, the key is fed into the input buffer using the DATA statement. When the query is invoked (with the **EXEC** as shown), the first prompt will receive its input from the input buffer, rather than prompting for the key. This way, it's as if the user is entering the key for the customer query screen.

Moving from the customer query to the order query, however, is not so straight-forward. Because there could be multiple orders for a single customer, we must process each order key individually, with the following paragraph:

```
LOCAL LIST,NDX,MAX
*
LIST = <POS(SO.XREF)>
NDX = 1
MAX = DCOUNT(LIST,@VM)
*
WHILE (NDX <= MAX) AND (@OTHER(18) # 14) DO
  @VALUE = ''
  DATA LIST<1,NDX>
  EXEC 'O*ORDERS*INQ.A'
  NDX = NDX + 1
REPEAT
```

Basically, this paragraph emulates the list processing that SB+ does automatically when a multivalued list of keys is passed in the **common variable** @VALUE. With an incoming list of order keys, this paragraph loops from 1 to the end of the list, data stacking each key into the query individually. As with anything, there are pros and cons to this technique:

Pro: With the check of the common variable @OTHER(18) in the WHILE loop, we can exit the list midway by pressing Escape from the query screen. Passing a list of keys in @VALUE does not support this.

Con: The query screen will be completely repainted for each record that is displayed. This will cause the application to run slower and appear less attractive. (You can, however, add functionality to eliminate this problem, by telling the screen not to refresh between records and then handling the screen refreshing yourself.)

Of course, you must evaluate these issues before determining which technique is most appropriate for your application.

"Zooming" Between Queries

"Zooming" is much like "cruising", except several queries can be called from a single query all with a single function key. To implement this, we need a process called from a function key that will "sense" the query to be called and act accordingly.

To illustrate this, let's assume we have a sales order query screen. Each sales order has a customer code, product codes, a shipment carrier code, and a payment type code, just to name a few. We want to tie a customer query, product query, carrier query, and payment type code query to the order query, but are limited in the number of function keys available. Therefore, we can tie all of the queries to function key F5, and let SB+ figure out which query should be called.

This particular technique can be implemented only with a non-amendable input screen, as the cursor movement plays an important role in the selection of the query. In short:

- If the cursor is on the customer code, the customer query should be called.
- If the cursor is on a product code, the product query should be called.
- If the cursor is on the carrier code, the carrier query should be called.
- If the cursor is on the payment type code, the payment type query should be called.

To implement this, we can create a paragraph as follows and link it to function key F5 for the query screen definition:

```
LOCAL FIELD.ID
*
FIELD.ID = @LINE<1,18>
CASE (FIELD.ID = 'CUST.ID')
  EXEC 'customer_query'
CASE (FIELD.ID = 'PRODUCT.ID')
  EXEC 'product_query'
CASE (FIELD.ID = 'CARRIER.ID')
  EXEC 'carrier_query'
CASE (FIELD.ID = 'PMT.TYPE')
  EXEC 'paymenttype_query'
END CASE
```

On the third line of this process, note the reference to the **common variable** @LINE<1,18>. As the cursor moves through the screen, SB+ uses this field to hold the name of the current field. Therefore, we can use this information in our process to determine what field the cursor is on when the function key was pressed and act accordingly.

The question becomes, what do we name this function key? With so many uses, it seems impractical to try to name it:

F5-Customer/Product/Carrier/Payment Type Query

Besides, it's unlikely that there will be enough room on the function key line for a description this long. Therefore:

F5-Zoom

...*almost* seems like a workable alternative. Who knows, if we get enough people using this label, Dick's dream may actually catch on!

"Double-Clutching"?

This is the one Pick term I really don't have a good definition for. However, for the purpose of defining the full complement of Dick's terminology, this section will describe something that we'll call double-clutching or better yet, selective cruising.

The point of this technique is simple: We want the user to be able to cruise from one query to another, with the ability to select the information that is to be passed from query to query.

For example, let's assume we have a customer query and a sales order query. Each customer record has an multivalued list of order keys in attribute 31 that we want to use to invoke the order query screen. However, because there could be hundreds of order keys in the list, we want to give the user the option to select one or several from the list and then pass this smaller list into the query.

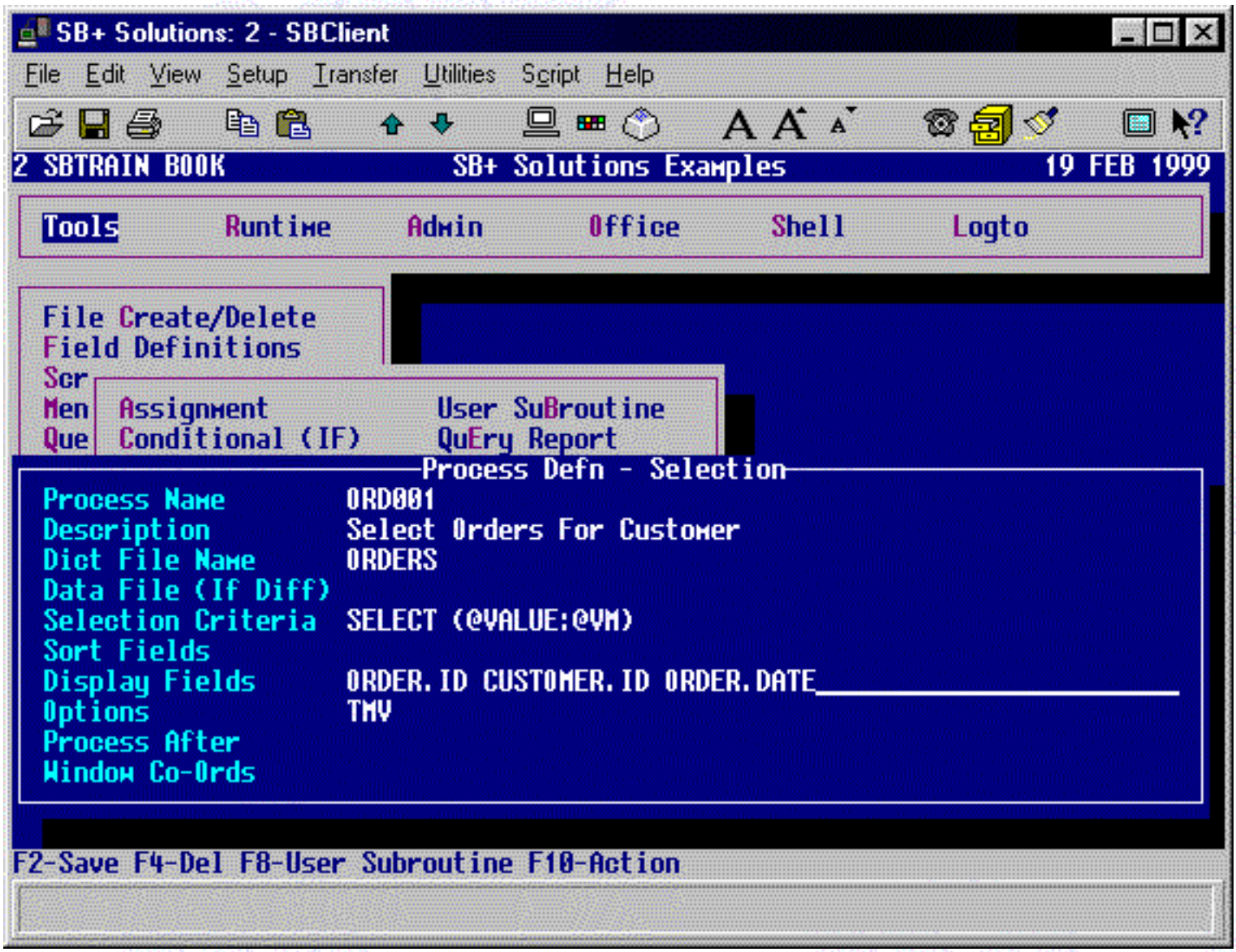
Following this example, on the customer screen we want the user to be able to press F5 to recall a list of orders to select from. Once the user has selected the orders to view, we then call the order query for them. This can be implemented using a paragraph as follows:

```
@VALUE = <POS(SO.XREF)>
EXEC 'select_process'
*
IF NO.ESC THEN
  EXEC 'O*CUSTOMER*INQ.A'
END
```

(Note that we're assuming the key values will be passed into the customer query in the **common variable** @VALUE.)

This process extracts the entire list of order keys from the SO.XREF attribute on the customer record and calls a selection process which could be defined as follows:

"Double-Clutching"?



This selection process will take an incoming list of order keys from @VALUE, display the fields as shown, and allow the user to select several. The options "TMV" tell the process to display titles for the individual display fields in the selection window, and return the selected keys in a multivalued list in the common variable @VALUE.

Isn't that convenient? These options setup the return value in the exact place and format that we need!

Referring back to the paragraph, notice the line:

IF NO.ESC THEN

This checks to see if the user pressed the Escape key from the selection process. If Escape was pressed (@OTHER(18) equals 14), we don't want to invoke the query. However, if the last key pressed was not Escape, it must have been either F2 or <cr>, which means that @VALUE will have at least one order key. In this case, we simply pass this value into the order query and the rest, as they say, is history. (See "**Global Equates**" for a description of NO.ESC).

Paging Through a List of Keys

When multiple keys are passed from one query to another query, SB+ will loop from 1 to the number of keys in the list, reading and displaying each record in sequence. While this powerful feature is incredibly easy to implement, it is also restrictive; If you exit a record, there is no way to go back to view it again without exiting the query and starting over.

Using a paragraph between the calling process and called query, we can control the paging through the list, allowing the user to page forward or backward through the list at will.

For example, let's assume we have a customer query screen and a sales order query screen. For each customer record, there may be multiple order keys. Therefore, when we invoke the order query from the customer query, we want to allow the user to page forward to the next key with the F6 key, and backward to the previous key with the F5 key.

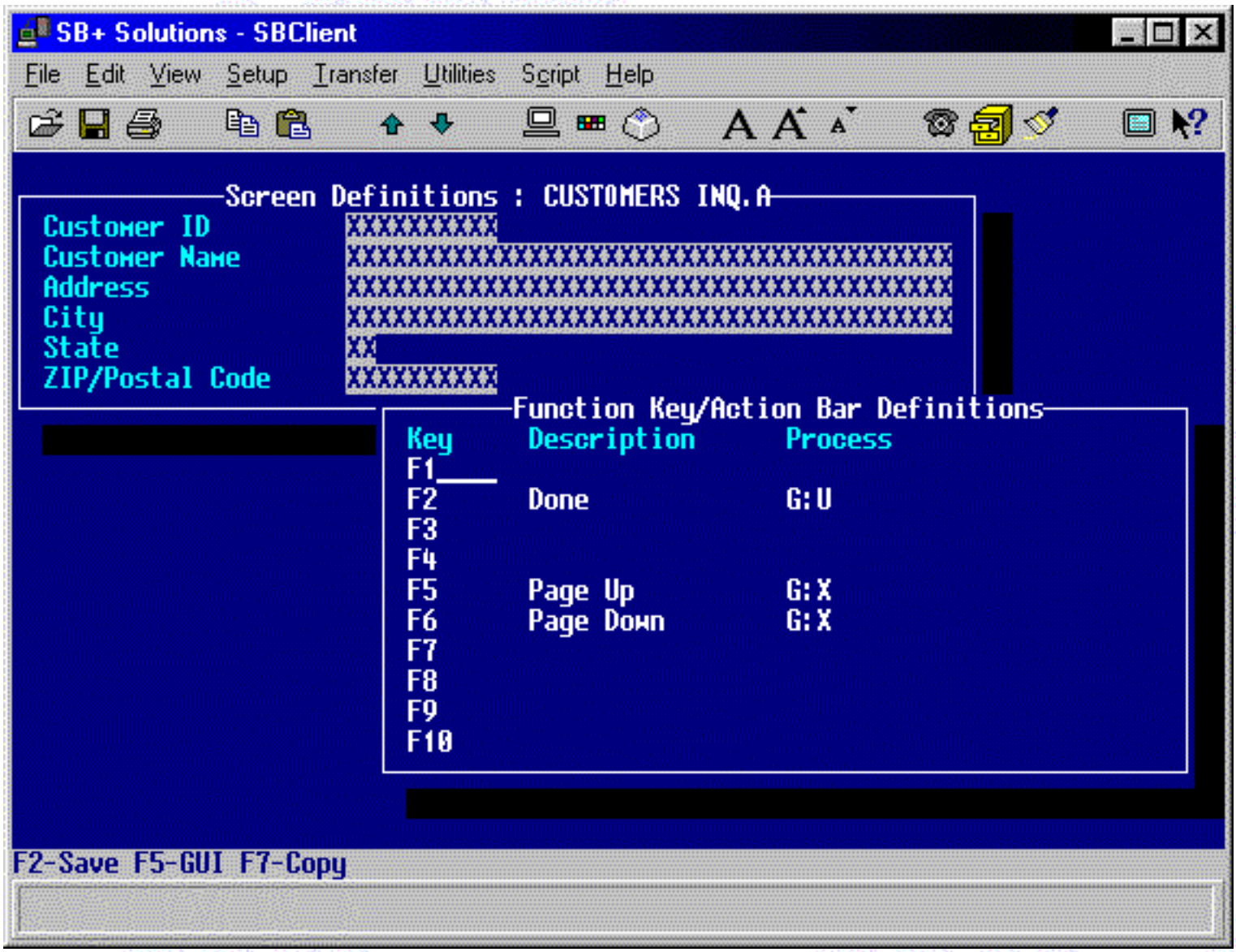
Assuming that the order query screen gets its key from the **common variable** @VALUE, the following paragraph can be used:

```
LOCAL LIST,NDX,MAX
*
LIST = <POS(SO.XREF)>
NDX = 1
MAX = DCOUNT(LIST,@VM)
*
WHILE (NDX <= MAX) AND (@OTHER(18) # 14) DO
  @VALUE = LIST<1,NDX>
  EXEC 'O*ORDERS*INQ.A'
  *
  CASE @OTHER(18) = 35 ;* F5 key
    IF NDX > 1 THEN NDX = NDX - 1
  CASE 1
    NDX = NDX + 1
  END CASE
REPEAT
```

At the beginning of this paragraph, we extract the list of orders from the SO.XREF attribute and move it into a **local variable** called LIST. Next, we set a loop index (NDX) to the starting value and calculate the number of items in the list (in MAX).

Next, the WHILE loop cycles through the list until either the user presses Escape or all of the records have been processed. After each call to the query screen (O*ORDERS*INQ.A), the process checks to see how the user exited the query. If F5 was the key used to exit the order query, the user wants to see the previous record, so we simply decrement the loop index. If any other key is used to exit the order query, we increment the loop index to display the next record in the list.

The only thing to be defined is the actual F5 and F6 keys for the query. These can be defined as follows:



Note that both keys use the G: (*inline Goto process*) to set @RTN.FLAG to the literal "X". This tells SB+ to exit the query when either of these keys are pressed. The calling process will then determine which key was used, and act accordingly.

If you're looking for a challenge, there's a couple of enhancements to this technique that you may find useful. First, how would you enhance this technique so that we could show a record indicator (i.e. "x/y" or "Record 1 of 14") on the inquiry? Secondly, if a user presses the page up key from the first record, or page down from the last, how would you modify this process to loop around to the other end of the list?

Loading Additional Information for the Query

There are occasions when the structure of a file is not conducive to a useful query. For example, there may be a file in the system that contains only references to other information in other files. To a user, this reference information should be invisible -- it serves an internal need, but the casual users don't need to see it. Therefore, rather than display a query of information that the user doesn't need, it's better to use the reference information to load everything that the user does need.

For example, let's assume we have a sales order query application. Each sales order has a field called SALESMAN which contains a three-digit code for the person who made the sale. On the entry screen, the code was important -- it was how the entry clerk identified the salesman. On the query, however, there's a pretty good chance the customer service manager doesn't care much about the salesman's code; they probably want to see the salesman's name instead.

There are two ways to load additional information for a query such as this:

- **Derived Values**
- Using a Process After Read.

Specific details about creating derived values and a Process After Read can be found throughout **Chapter 3**. Therefore, for the sake of this discussion, we'll focus on the advantages and disadvantages of using each of these techniques on a query screen.

The most obvious advantage to using derived values on a query screen is the fact that they are easy to create. A simple field definition can be created to read information from any file in the system in a matter of seconds. Also, derived values are processed only when that particular field is encountered on a screen. If a query screen has a **subscreen** that uses derived values to extract information from other files, the other files will not be read until the subscreen is invoked.

This last feature is also the biggest drawback; If a particular subscreen is invoked multiple times, records will be read from the other files multiple times. Depending on the number of times a particular subscreen could be called, system efficiency could be compromised.

Also in the negative column: If two derived values on the same screen reference the same file and record, the one record will be read once per prompt, instead of once for the screen. Like the previous problem, this too has the potential of compromising system efficiency.

The Process After Read, however, can overcome these deficiencies. In the case of our salesman example, we could create a paragraph to load the salesman's name into a work field and then display the work field on the query. Or, if multiple fields were to be extracted from one record, the Process After Read could read the record and move the appropriate fields into work fields for display.

While this prevents records from being read multiple times, it also has the potential of reading information that may not be necessary. For example, if information for a subscreen is loaded in the Process After Read, the user may never view the subscreen, in which case the extra processing is a wasted effort.

Generally speaking, a mix of these techniques is best. You must evaluate the need each time you want to reference other information, and then use whichever technique is most appropriate. It all depends on the application, standards, and of course, your own personal style.

Updating a Viewed Record

Typically, an inquiry is not responsible for performing any updates to the record(s) being viewed. To that end, SB+ does not perform any record locking or updating when a record is read for an output process or non-amendable input process.

There are situations, however, when you may want to update a record to denote that it has been viewed. As an example, consider an electronic mail application. If a person views a message, you may want to denote that the message has been viewed or even send a confirmation back to the sender. In either of these cases, something has to be updated when the message is viewed.

This is accomplished with a process called on the screen definition in the Process After Read slot. At this point in the execution of the screen you know a record has been read and will be displayed, so this is the best place to call a process to do the updating.

Let's assume we're creating the E-mail application mentioned earlier. Each message record will be stored in a file called MESSAGE and will have an attribute called VIEW.FLG which determines if the message has been viewed. Let's also assume that until the message is viewed, VIEW.FLG is set to "N". We must change it to "Y" when the message is viewed.

The following paragraph, called from the Process After Read slot, will do the trick:

```
READU @RECORD FROM 0,@KEY
VIEW.FLG = 'Y'
WRITE @RECORD ON 0,@KEY
```

Oddly enough, this paragraph is called immediately after the record has been read, and the first thing it does is read the record again! We need to do this because the query process doesn't lock the record, and if we're going to update the record we should lock it. There's no way we can tell SB+ to lock the record for a query, so what we really need is the ability to tell SB+ to simply not read the record, leaving the task to our process.

To accomplish this, we need to add some options to the F5-Field window for the key field, assuming that the key is prompted and not extracted from common memory. (If the key is extracted from common memory there is no way to suppress the main read.) On the key field, add the following options to the Control/Dep/Read field:

```
RIA
```

This will tell SB+ "while this is the read step, don't read the record -- we'll do it.". Our process after read, then, can read the record, update it, and leave the record in common memory for the query to display.

After the read/lock, the paragraph sets the view flag to the desired value and then writes the record back to the file.

Note the file reference of zero in the READU and WRITE. This shortcut notation tells SB+ that we are reading from and writing to the main file that was opened for the query, (a.k.a the **common variable** @F.FILE). Though a minor syntax variation, this prevents the main file from being opened over and over and over again. Also note that we're using the common variable @KEY as the key for the record, rather than trying to refer to the field by some field name. This may appear to be a minor issue, but it allows us to have one technique that will work for screens with either a single or multi-part key.

There are any number of reasons why a person would want to update a record for a query. Using this as a baseline, you can see how with a little imagination you could do some pretty remarkable updates on a query -- even though the process isn't intended to update at all!

Process Slot Overview

The following sections describe all of the process slots available on an output process, their purpose, and some considerations as to their use.

Process Before Process: (Process Definition - Output [/PD.O])

When an output process is invoked, the process listed in this slot is executed first. This process will be called only once for the entire execution of the process.

This process is typically used either to validate that the process can be run (for additional security), to load configuration information into @PARMS(...) elements, or both.

Process Before Screen Display: (Screen Definition [/SD], F6-Params)

Prior to the screen being displayed, the process listed in this slot is executed. This process will be called each time the screen is redisplayed (except when "/R" is used to do so). This process slot is useful for clearing work variables and adding additional information to the @SCR.IMAGE **common variable** to augment the screen display.

Process After Screen Display: (Screen Definition [/SD], F6-Params)

Immediately after the screen has been displayed, the process listed in this slot is executed. This process will be called each time the screen is redisplayed (except when /R is used). This process is useful for clearing work variables.

Process Before - Key Field: (Screen Definition [/SD],F5-Field)

Prior to the prompting for the key field (assuming the key field is prompted), or a part of the key field (as in the case of multi-part keys), the process name listed in this slot is executed. This process will be called each time the cursor moves to this field. Typically, this slot is used to clear work fields that are used in the construction of a multi-part key.

Default Expression - Key Field: (Field Definition [/SD],F5-Field, F5-Field Defn)

If a default expression includes a process call (i.e. P(...)) and the field has no value, or the default for a field is a mandatory default, the process named in parentheses will be executed. When a process is called this way, the process should be used to return the calculated default in @VALUE. No other fields in @RECORD or @WORK should be changed by the default process, however, @OTHER.REC and @PARMS(...) may be updated for later use.

Validation Code - Key Field: (Field Definition [/SD],F5-Field, F5-Field Defn)

If the "C:process" validation code is used, each time a person enters value into the field, the process named following the "C:" will be executed. This process takes @VALUE as the value entered (after input conversion), and is responsible for verifying the entry. If the entry is acceptable, @RTN.FLAG is expected to be returned as "0". If the entry is to be rejected, @RTN.FLAG should be set to a "1".

Process After - Key Field: (Screen Definition [/SD],F5-Field)

This process is called after the validation for the key field has verified a successful entry. This process slot is useful for constructing the key for a multi-part key screen. Also, like any process before or after a field on the screen, this process call may be used to call a **subscreen** conditionally. This is uncommon, however, on the key field of a screen.

Process After Read Record: (Screen Definition [/SD], F6-Params)

This process is called immediately after the record has been read from the file. At this point, @RECORD has the record, @KEY has the key value, @ORIG.REC has a copy of the record, and @ACTION is "2" to denote an existing record has been loaded. It is fundamentally impossible for an output process to set @ACTION to "1", though a non-amendable input process could do so.

This process is primarily used for two different tasks:

- Verify that the record can be loaded, and;
- Read additional information from other sources as necessary.

When both validation and loading information must be done in this process slot, it is not recommended to do the two inside of one process. Instead, use a paragraph like the one shown following, which calls other processes to do the validation and loading of additional information.

```
EXEC 'validation_process'
IF @RTN.FLAG = 0 THEN
  EXEC 'load_process'
END
```

Process Before - Data Fields: (Screen Definition [/SD],F5-Field)

This process is executed once per each prompt, immediately before the user is requested to enter values into that prompt.

On an output process, the Process Before Field should not be used. Because there is no cursor control to allow movement from field to field, after the record has been read, all of the processes before and after fields are done all at the same time.

Process After - Data Fields: (Screen Definition [/SD],F5-Field)

On an output process, the Process After Field should not be used. Because there is no cursor control to allow movement from field to field, after the record has been read, all of the processes before and after fields are done all at the same time.

Process After Process: (Process Definition - Input [/PD.O])

This process is called after the user has exited the screen, and is typically used to reset any values that may have been set (such as security parameters) by the Process Before Process. This process will be called only once for the entire process, and will be the last thing that happens before the process terminates.

Understanding Output Process Drivers

When an output process is started (running interpretively), SB+ checks the current drivers file to see if there are drivers on file for the process. If not, SB+ creates the necessary driver records and then runs a subroutine to interpret the drivers.

Typically, the name of the drivers file is *XXX*DRIVERS (where the *XXX* is the system ID), though the true driver file and driver ID is found on the Process Definition - Output record, which can be viewed using /PD.O.

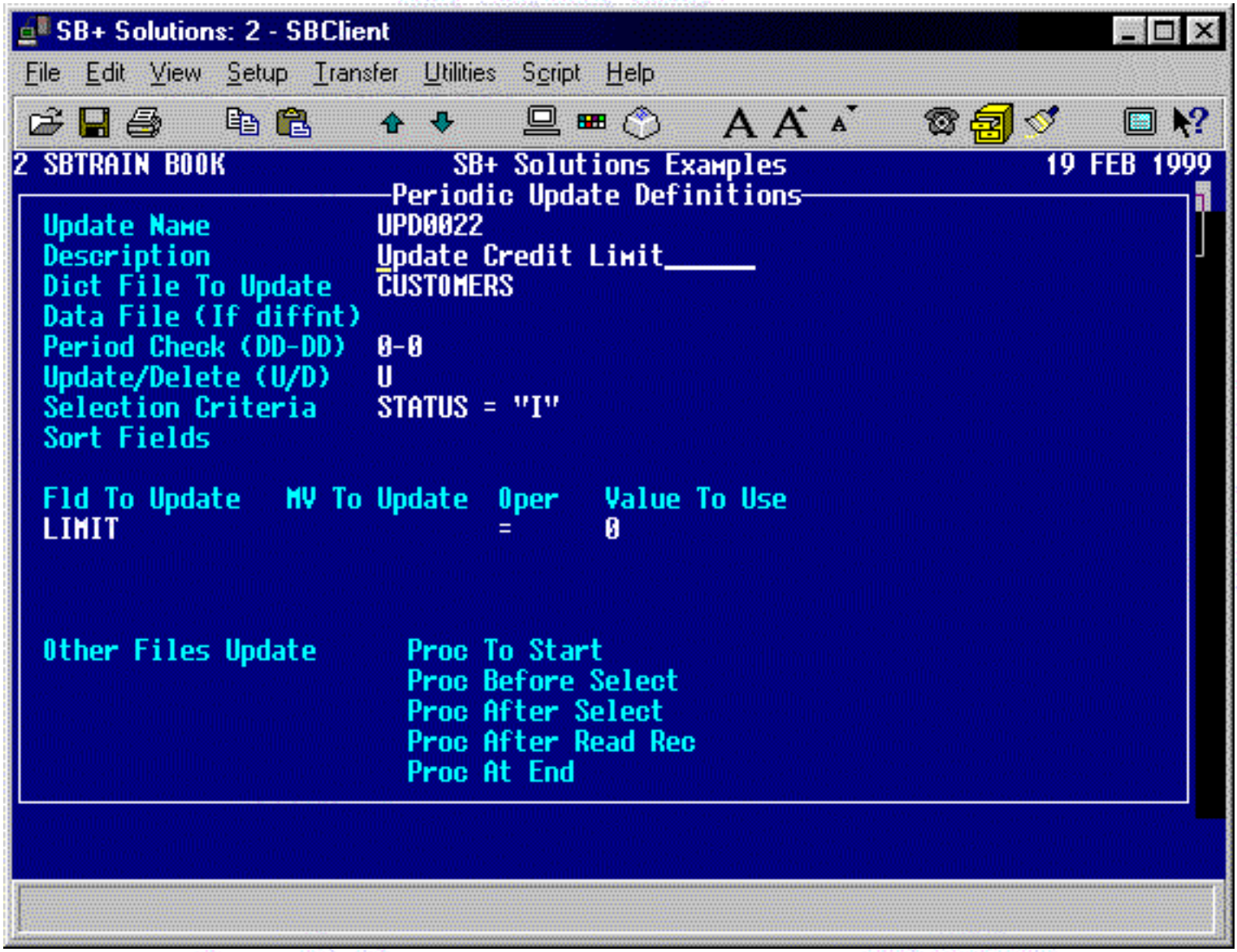
Using drivers, SB+ encapsulates nearly all of the information required for running a screen. This is largely for efficiency, as this allows SB+ to read the field definitions for all of the prompts on a screen once for the entire execution of the screen, instead of once each time the cursor is moved to each prompt. Because much of the information for a prompt is built into this separate driver record, if something about a field changes it may or may not be readily apparent. Instead, you may need to *rebuild the drivers* to see the net effect of the change.

A **more detailed discussion of SB+ driver records** can be found near the end of **Chapter 3**.

Updating One File

The simplest form of a periodic update is one that selects records in a file and then processes each record individually. This is accomplished either by defining the fields to be updated in the Fld To Update section of the screen, or by calling a Process After Read to do the updates. Remember, though, if the Process After Read is used to update the main record and the Fld To Update prompt is blank, the process must save the record -- the periodic update will not.

The following example selects a list of customers with a status of "I" (inactive) and sets their credit limit to zero.



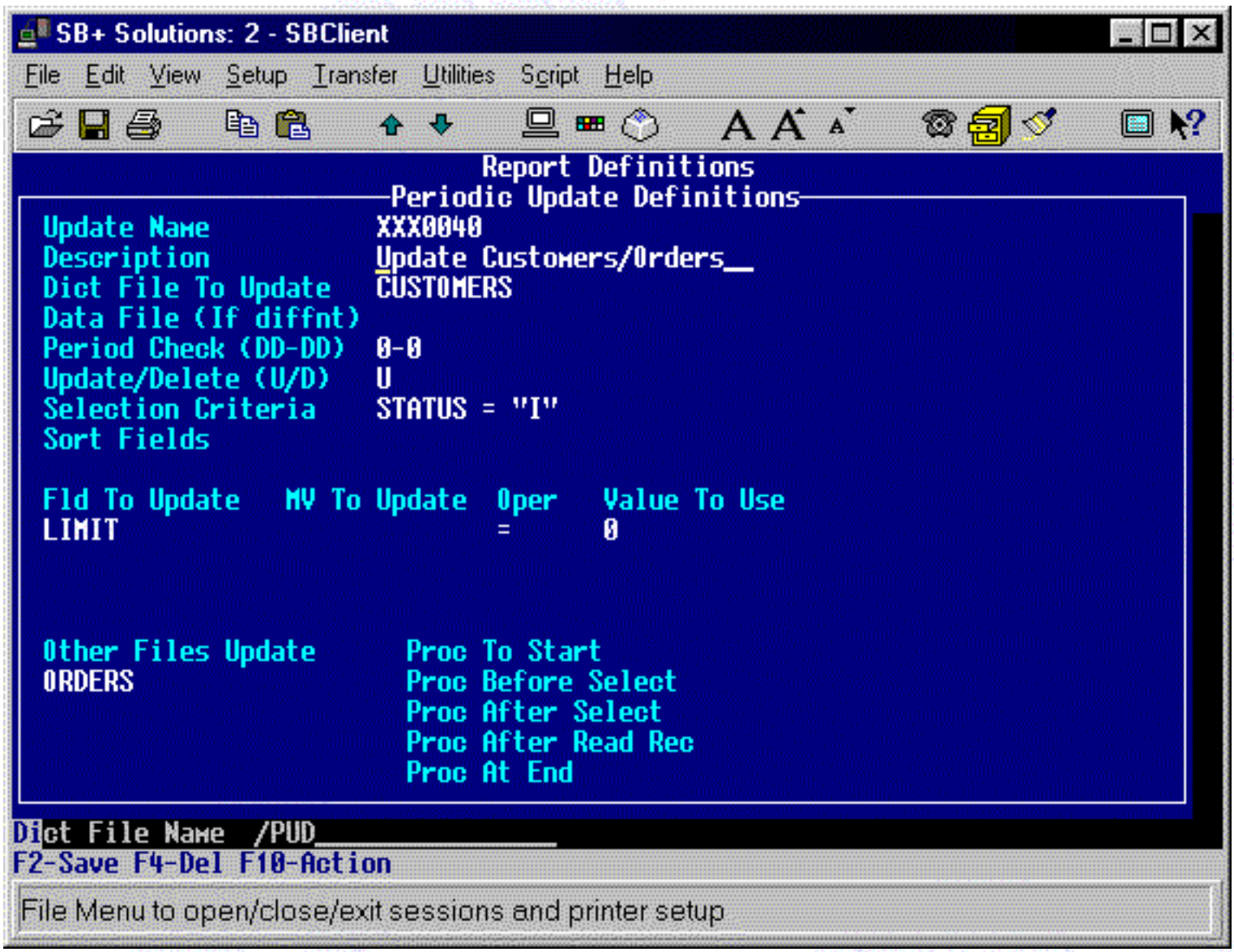
Note that there are no other files to be updated (in the "Other Files Update" field), nor is there a process being called in the "Process After Read Rec" slot. Therefore, this update will function as follows:

- All customers with a status of "I" will be selected to an active select list. See "**Selection Criteria**" for more information about selecting records to be updated.
- For each customer selected, the record will be read, the credit limit field will be updated, and the record will be written back to its original location.

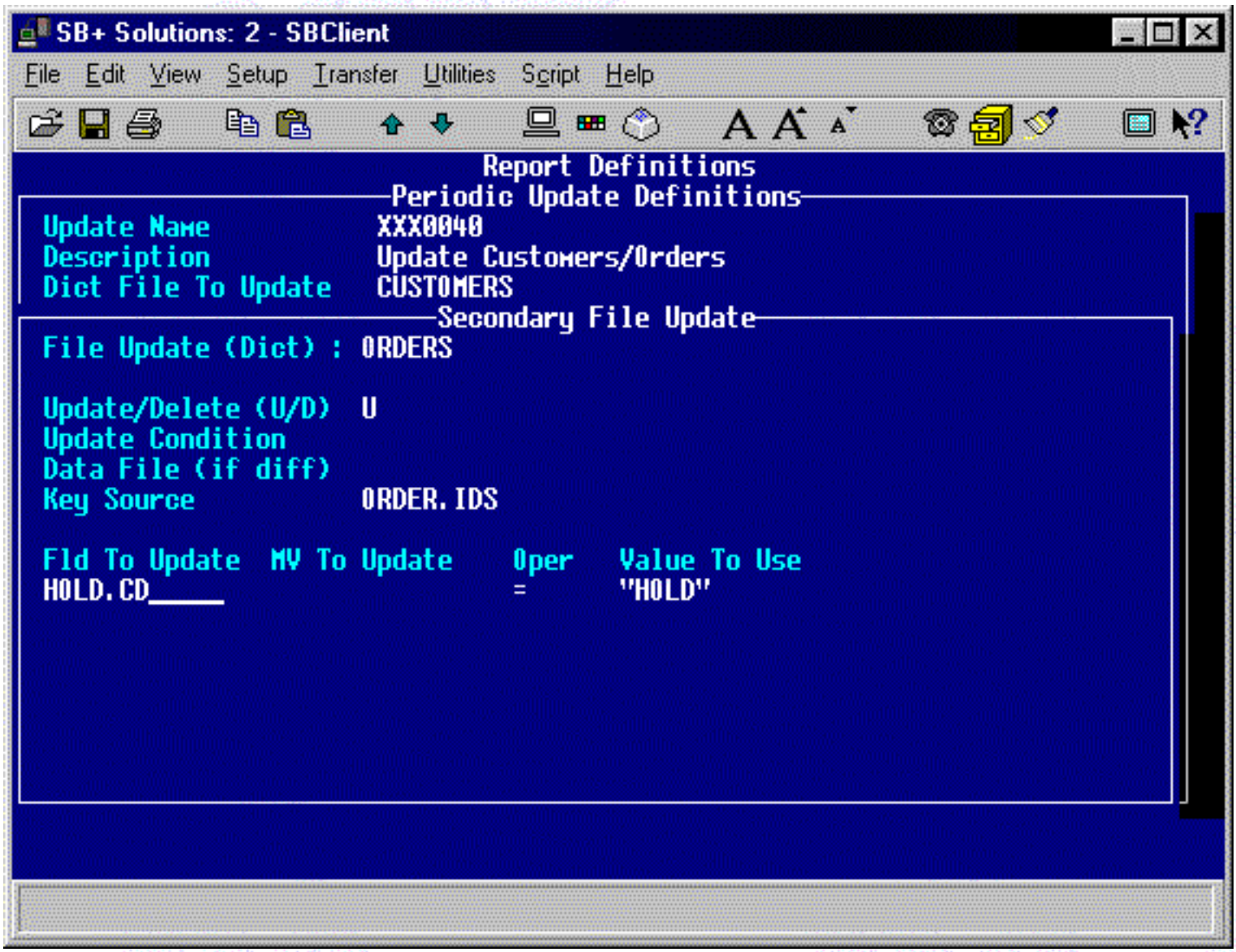
Updating the Main File and Other Files

If a periodic update is going to update the main file and other files, the main file updates can be defined either in the Fld to Update section, or as a Process After Read. The other files to be updated can be defined either in Other Files Update section, or as a part of the Process After Read. If the updates to either the main record or other records are of any complexity or require conditional logic, the Process After Read is the preferred alternative.

The following example illustrates an update that will select all of the customers with a status of "I" and will not only set their credit limit to zero, but will also update a hold code on all of their outstanding orders:

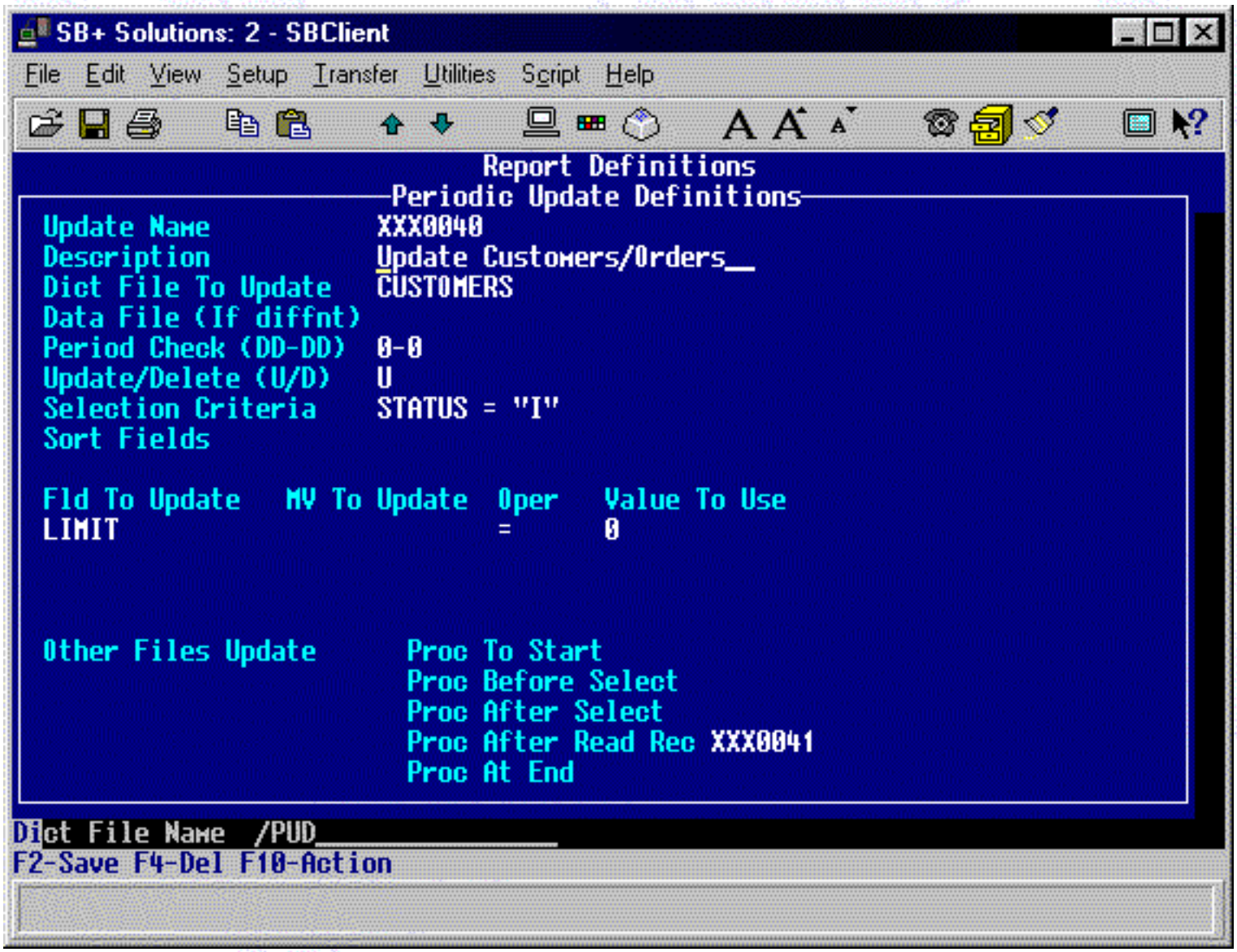


The secondary update for this periodic update appears as follows:



Note that the list of orders can be a multivalued field. This is one of the strengths of the secondary update feature on periodic updates -- if the list of secondary keys to be updated is multivalued, the update as defined in the lower half of the screen will be done to each record individually with no extra programming!

We could also code this using a process in the Process After Read Rec slot. In this case, the main periodic update definition would appear as follows:



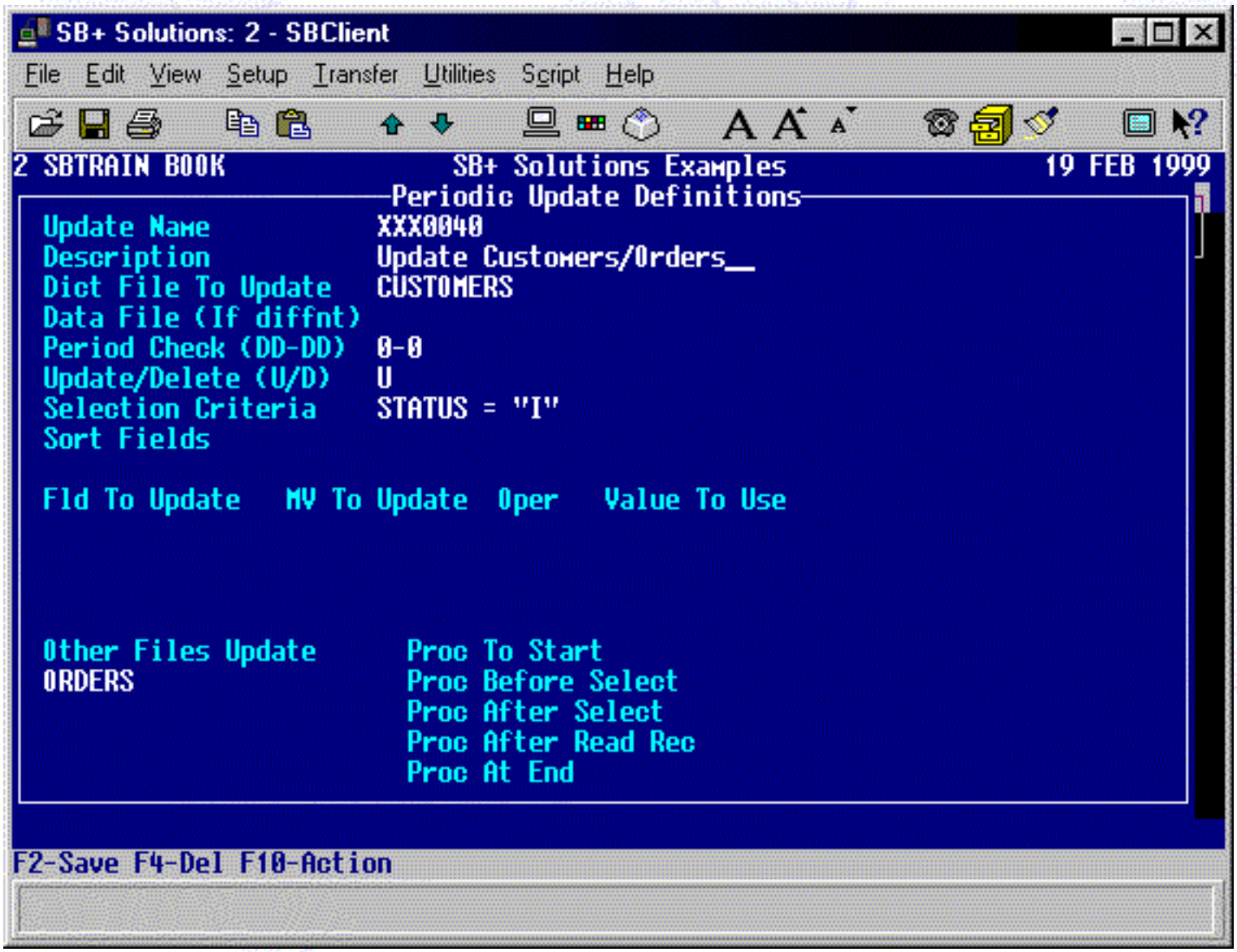
Note that in this example, there are no secondary updates defined in the "Other Files Update" prompt. Also note that there are no fields being updated in the record! These details have been taken out of the main periodic update definition and moved into the process called from the Process After Read Rec, which appears as follows:

```
*
* Update the current record
*
LIMIT = 0
WRITE @RECORD ON 0,@KEY ;* Write the record on the current open file using the current key
*
@CNT = DCOUNT(<POS(ORDER.IDS)>,@VM)
WHILE @CNT DO
  READU @OTHER.REC FROM 'ORDERS',ORDER.IDS
  @RTN.FLAG = 0
  *
  @OTHER.REC<POS(ORDERS,HOLD.CD)> = "HOLD"
  WRITE @OTHER.REC ON 'ORDERS',ORDER.IDS
  *
  @CNT = @CNT - 1
REPEAT
```

Important Note: When there are no fields being updated in the periodic update definition and all updates are happening in the Process After Read Rec, the record must be written in the Process After Read Rec. SB+ will not write the record if there are no fields to update in the periodic update definition.

Updating Only Other Files

A periodic update doesn't need to update the main record if the main record is being used to update something else. In this case, the Fld To Update is left blank, and either the Other Files Update section or a Process After Read is used to update the other files. If the updates to the records are of any complexity or require conditional logic, the Process After Read is the preferred alternative.



Note that in this example, there are no fields listed in the "Fld To Update" column. There is also no process being called in the "Proc After Read Rec" slot. Therefore, it's pretty likely that this process updates the secondary file (ORDERS), but doesn't update the main record.

Please take notice of the word "likely" in the above sentence. With what we see here, there's no guarantee that the record will be unchanged throughout this process. But how is that possible? What possible slot could be used to update the record?

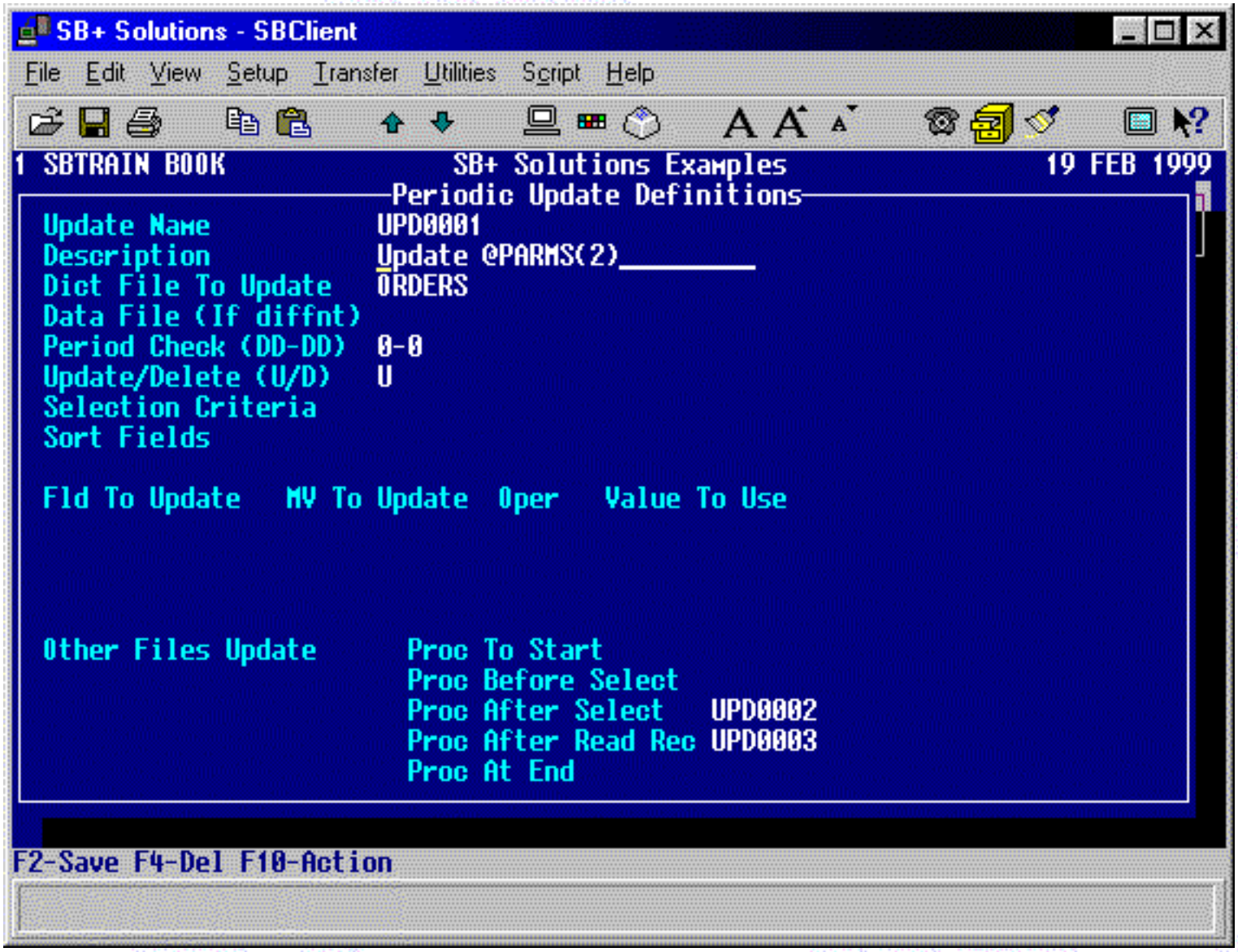
On the secondary file update screen, you can enter an **expression** to be used to update each field. Each expression can call a process using the P(...) **expression function** and therefore each process could *possibly* change fields in the record. This isn't the best structure for a periodic update (and highly NOT recommended), but it can be done. Therefore, always be aware that what

you see on a screen such as this one is only a part of the story. To be absolutely certain of what the process is doing, you'll need keep looking.

Updating Nothing

Oddly enough, a periodic update doesn't actually need to update anything! Instead, the update can be used to cycle through a list of records, building information into one or several **common variables**. To implement this, the Fld to Update and the Other Files Update can both be left blank, and a Process After Read can be created to use each record to update some area of common memory as required.

To illustrate this, consider a periodic update which will cycle through a list of sales orders and will update the common variable @PARMS(2) with a list of order types and total order amounts per type.



First, note the Process After Select. This process is necessary to clear whatever might currently be in @PARMS(2), and appears as follows:

@PARMS(2) = ''

The Process After Read, then, is a paragraph which takes the order record and accumulates the order total into buckets in

@PARMS(2) depending on the type of order:

```
LOCAL NDX
*
NDX = LOC(ORDER.TYPE,@PARMS(2)<1>,@VM)
IF NDX = 0 THEN
  NDX = DCOUNT(@PARMS(2)<1>,@VM) + 1
  @PARMS(2)<1,NDX> = ORDER.TYPE
END
*
@PARMS(2)<2,NDX> = @PARMS(2)<2,NDX> + ORDER.TOTAL
```

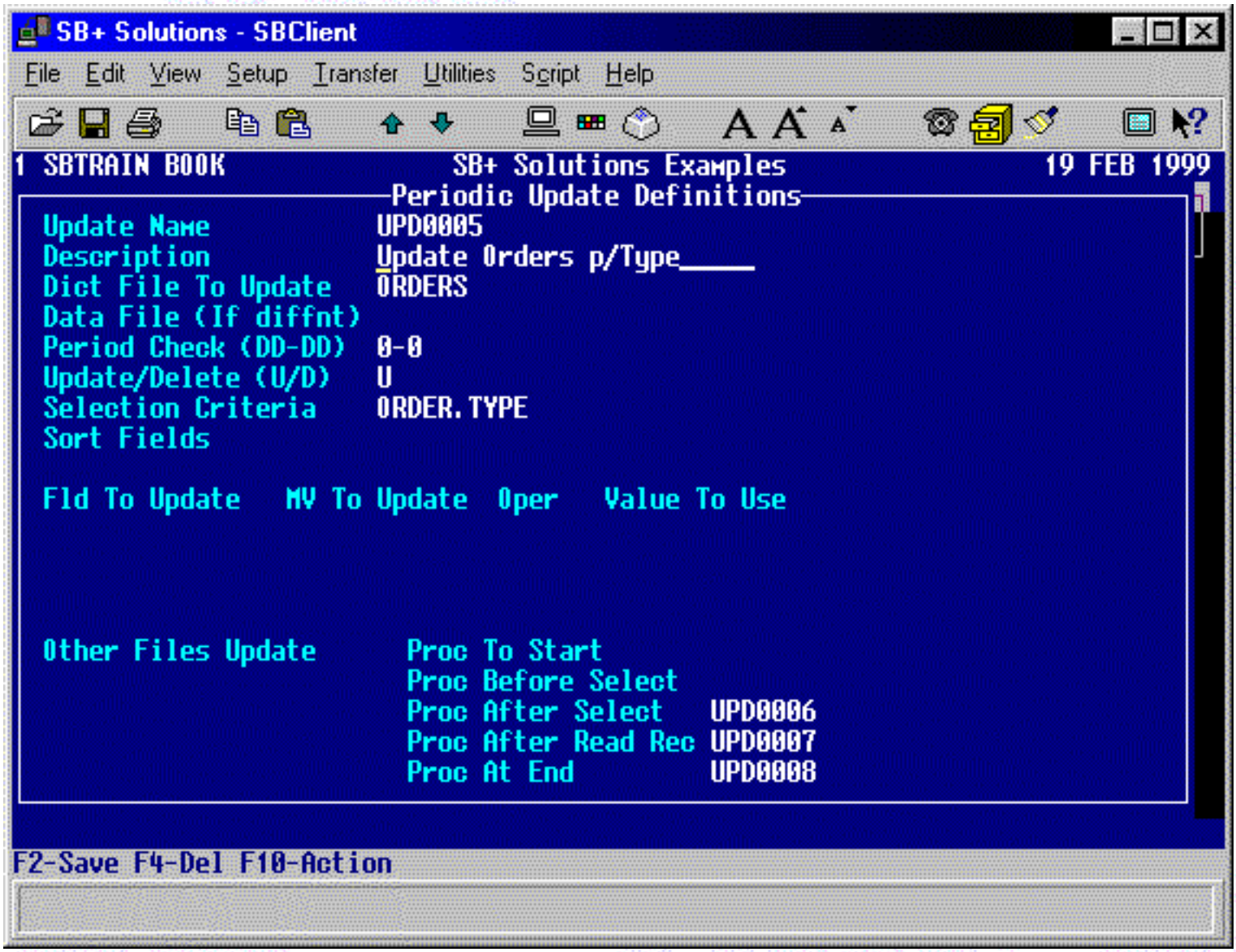
In the end, we have two attributes in @PARMS(2), one with a list of order types, and a second with a corresponding list of cumulative order values for each type. We can then use this information in other areas of the application.

Break Updates

A typical periodic update has a 1:1 read/write ratio. Sometimes this is not appropriate -- particularly if the goal of the periodic update is to summarize information into another file.

If the incoming list of records is in sorted order, a Process After Read can be constructed to evaluate the relationship of the current record to the previous record, and update other file(s) when a break occurs. This then allows a 1:n read/write ratio, which greatly enhances the flexibility of the periodic update feature.

As an example, let's assume we want to accumulate a total per order type into a file called TOT (Totals by Order Type). The periodic update would appear as follows:



The Process After Select is used to clear values, much in the same way it was used to clear values in the previous example. However, in this example, we need to clear not only @PARMS(2) for holding our cumulative total, but also @PARMS(3) which we will use to hold the last order type processed. This paragraph could appear something like the following:

```
@PARMS(2) = ''
@PARMS(3) = ''
```

Our Process After Read, then, must compare the current order type to the previous order type and if they are different, write the totals for the previous order type. This could be written in a paragraph as follows:

```
IF (ORDER.TYPE NE @PARMS(3)) THEN
  WRITE @PARMS(2) ON 'TOT',@PARMS(3)
  @PARMS(2) = ''
END
*
@PARMS(2) = @PARMS(2) + ORDER.TOTAL
@PARMS(3) = ORDER.TYPE
```

Finally, we must have a Process At End to write out the information for the last order type. Because we're deciding when a break occurs as a part of the process after read, we'll never know when the end of the list is encountered and will therefore leave information in @PARMS(2) that should be written. This process will do what we need:

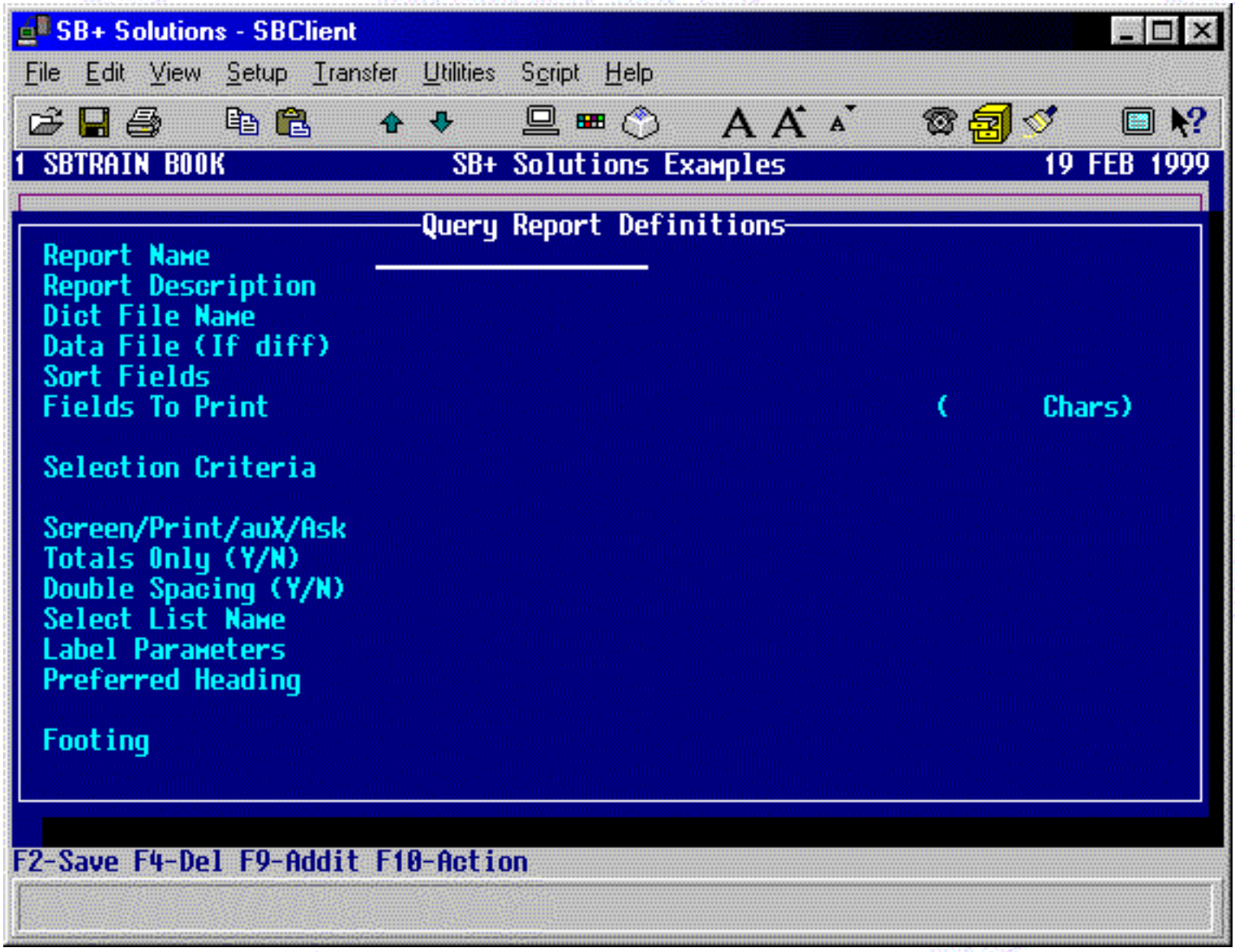
```
WRITE @PARMS(2) ON 'TOT',@PARMS(3)
```

In the end, the TOT file will have one record written for each order type, with a total in each record of all of the orders for that type code.

Creating a Query Report Definition

To start the Query Report Definition tool, select Tools from the main menu. Once the tools menu is displayed, select Query Report Definition. Or, if you want to call the tool directly instead of going through the menu navigation, you can simply enter /ARD (or /QRD) at any input prompt or menu.

The Query Report Definitions tool appears as follows:



Like any definition, a query report must have a unique name. If you are creating a new definition, you may use your process naming standards to help decide on an acceptable name. If you want to recall an existing definition, you may press F3 at this prompt and select from the list, or simply enter the name of the definition you want to recall.

Next, you must enter a description for the report. This description is displayed when F3 is used to recall a report, so be clear and concise.

At the Dict File Name prompt, enter the name of the file where field definitions are stored. Once you name a file here, all of the

following prompts that ask for field definitions will be validated from field definitions in this dictionary. Additionally, when you press F3 on these fields, SB+ will select field definitions from the appropriate dictionary.

If the name of the file where the actual data is stored is the same as the name of the dictionary, the Data File (If diff) can be left blank. However, if the dictionary file and data file are stored separately (such as in a multi-lingual environment), the name of the data file must be entered at this prompt. If the name of the file is to be calculated when the report is run, enter the name of the data file as an SB+ expression in parentheses. As an example, the following entry would open a file as named by the contents of attribute 5 of the **common variable** @PARMS(2):

(@PARMS(2)<5>)

When prompted to enter sort fields, enter the field names of the sort fields, each delimited by a space. The first field name entered here will be the major sort field, the last field name entered will be the minor sort, and all field names in between will be the intermediate sort fields. Of course, F3 is available here to help you select the fields to sort on. Be sure to select the fields in the proper order!

At the Fields To Print prompt, enter a list of fields to be displayed on the report, with a space between each field name. Again, if you have any questions as to the fields that are available, F3-Intuitive Help is available to assist.

In the **Selection Criteria** prompt, enter nearly any valid SB+ selection criteria. (In an odd twist of standards, you cannot use the select list feature of selection criteria at this prompt. Instead, if you want to run the selection from a list, enter the name of the list at the Select List Name prompt.)

The Screen/Print/Aux/Ask prompt allows you to define how the report will be output. The valid values that may be entered here include:

S	Output to screen only
P	Output to printer only
X	Output to aux printer only, if one is connected
D	Output in DIF format to file named *XXX*WORK (where *XXX* is the current system ID) or to the file as named in the report definition (F6-Params, F8-More)
F	Output in text format (one page per item) to file named *XXX*WORK (where *XXX* is the current system ID) or to the file as named in the report definition (F6-Params, F8-More).
A	Ask at runtime

The Totals Only prompt allows you to suppress all of the details from being output. This is equivalent to the modifiers ID-SUPP and DET-SUPP in the query language and will show only breaks and totals (if such fields exist) on the report.

If you enter "Y" at the Double Spacing prompt, SB+ will add the DBL-SPC modifier to the end of the query statement, resulting in a double spaced report.

The Label Parameters prompt is used to enter parameters to be passed to the LIST-LABEL or SORT-LABEL verb. The format of the information entered here will vary depending on the implementation of these verbs in each operating environment. Therefore, consult F1 help for assistance in entering information into this prompt.

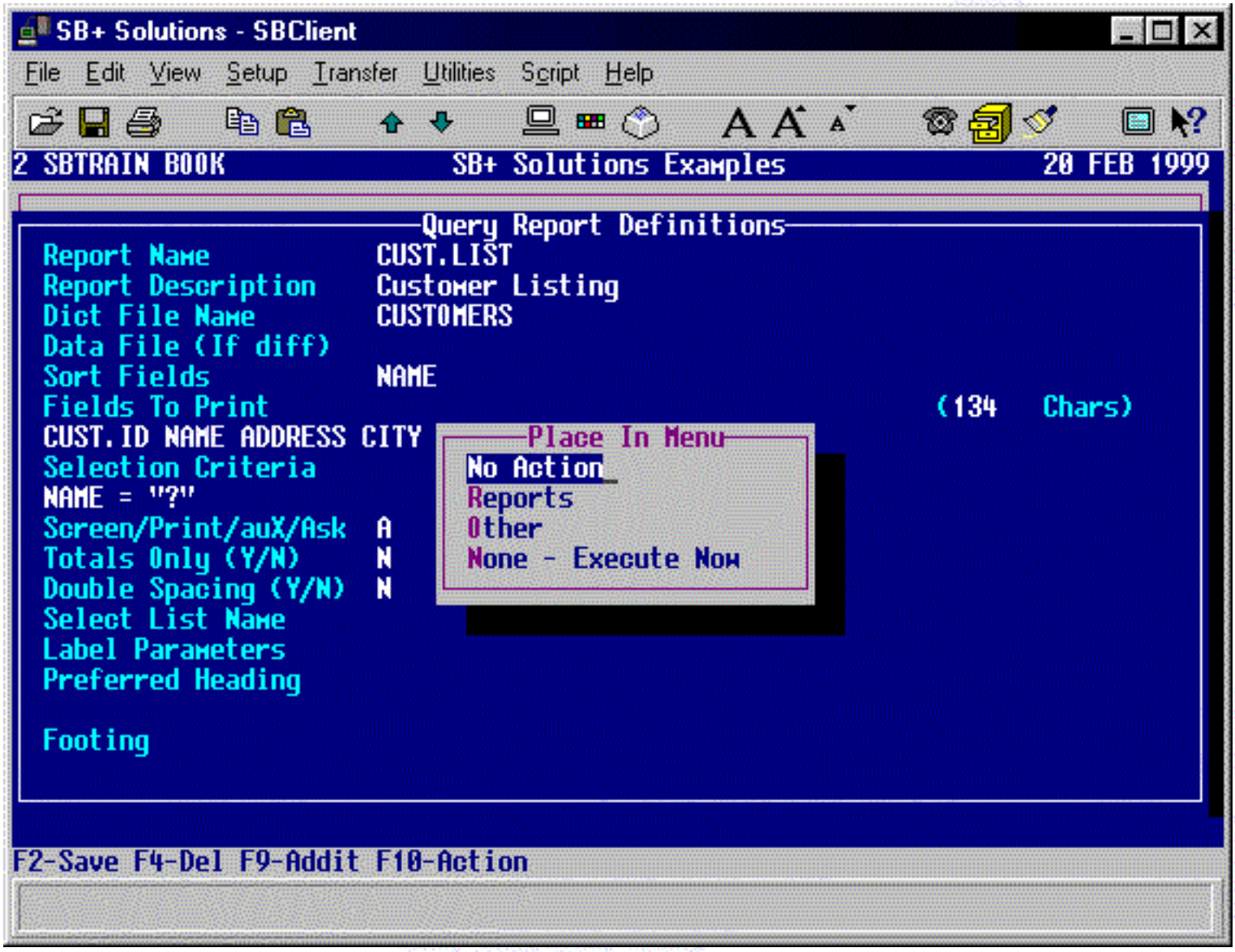
The Preferred Heading and Footing prompts allow you to enter any standard Access heading or footing that you would like to

display. SB+ supports all of the Access heading and footing modifiers including, but not limited to:

- 'P' -- Put a page number in the heading/footing
- 'T' -- Put the time and/or date in the heading/footing
- 'C' -- Center this line of the heading/footing
- 'L' -- Generate a line feed at this point in the heading/footing

Of course, the ultimate authority on which codes are supported will be the manual for the query language for your operating environment.

To save and run an query report definition, press the F2-Save key. A menu will be displayed in the center of the screen with 4 options, as follows:



If you want to start a new report or recall another report, select No Action.

To create an query report process and put this on the SB+ Reports **menu** (which is one of the options on the Runtime menu) select Reports. (Remember, the report definition must have a corresponding process to run it!) SB+ will ask you to enter an ID for the query report process. At this prompt you may enter literally anything, but as a standard you may want to make the process and

definition have the same name.

Once this has been entered SB+ will ask you to enter a description for the process. At this prompt, enter any description you want to appear at the top of the display screen when the report is started. After the description has been entered, SB+ will create the process and update the menu accordingly, and will then redisplay the menu (with no fanfare to let you know anything happened -- but it did!)

If you want to create a process and put it on a menu other than the Reports menu, select Other. When this is selected, SB+ will ask you enter the name of the menu where the process should be added. At this prompt, enter the name of any valid menu, or press F3 for a list. Once you've selected a menu, SB+ will prompt you for a process name and description, similar to when the previous option is selected. Also similar to the previous option, once you've answered the prompts, the appropriate menu will be updated and the menu (as shown earlier) will be redisplayed (again with no fanfare).

Personally, once I've created a report the first thing I want to do is test it. For this mentality, the None-Execute Now option is available. When this option is selected, SB+ will prompt you to enter the process name and description (not unlike the other menu-updating selections), but once these have been entered and the process has been created the report will be executed for review.

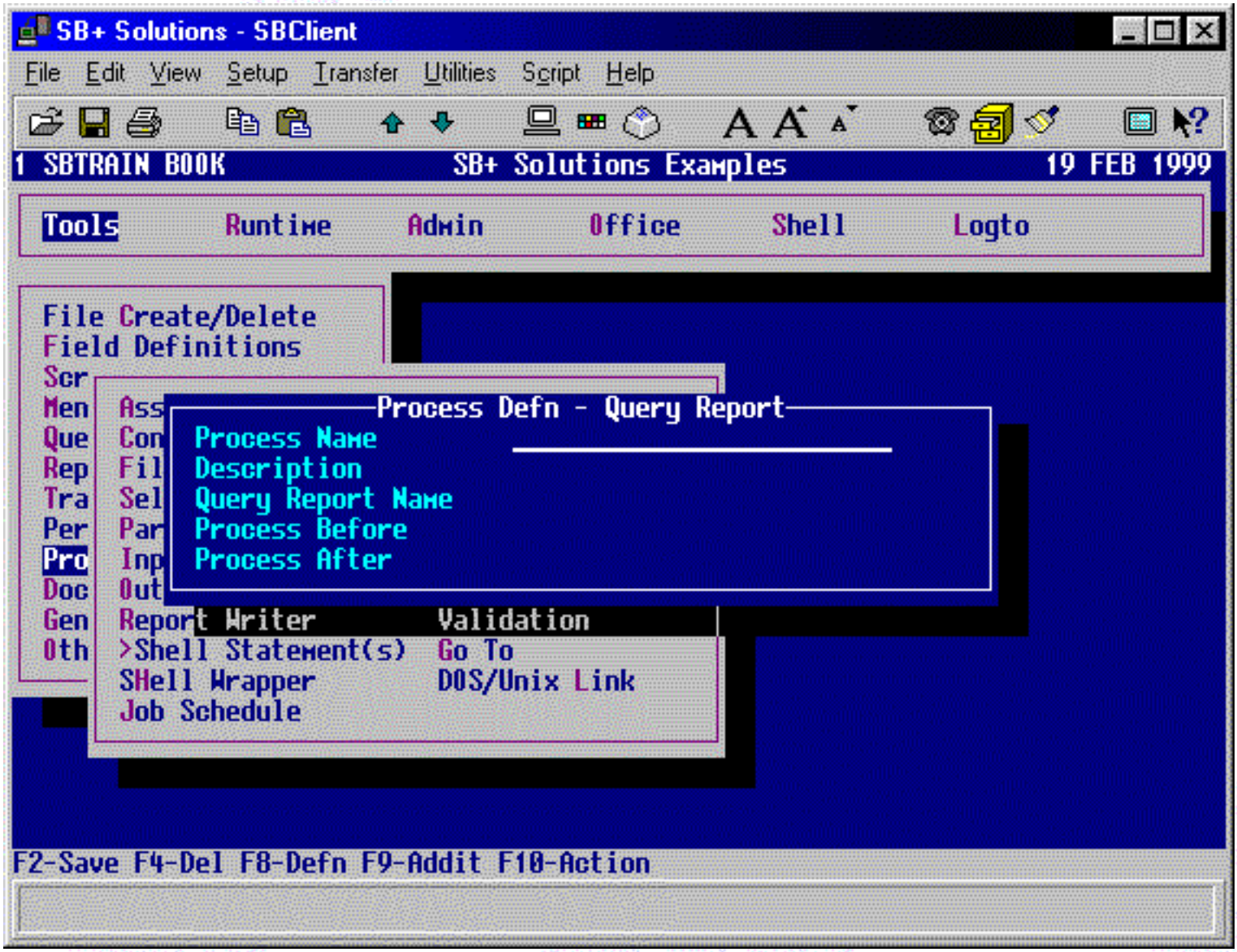
Remember, all of the options on this menu except the first one will create the process for you, so for the most part you don't need to worry about what goes into the creation of a query report process.

Creating a Query Report Process

Though the actual query report process can be created by SB+, you may need to review or change it. For example, though a query report definition tells SB+ *what* to print, it doesn't define *where* the report should be printed. Instead, you must edit the actual query report process to define the output destination for a report. To create a query report process, run the query report from the Query Report Definition screen. To edit the query report process, edit the Process Definition - Query Report process via one of two ways:

- Enter /PD.E at any menu or input prompt, or;
- Select Tools, Process Definitions, then Query Report. (Note: "Query Report" may be named slightly differently, depending on your operating environment.)

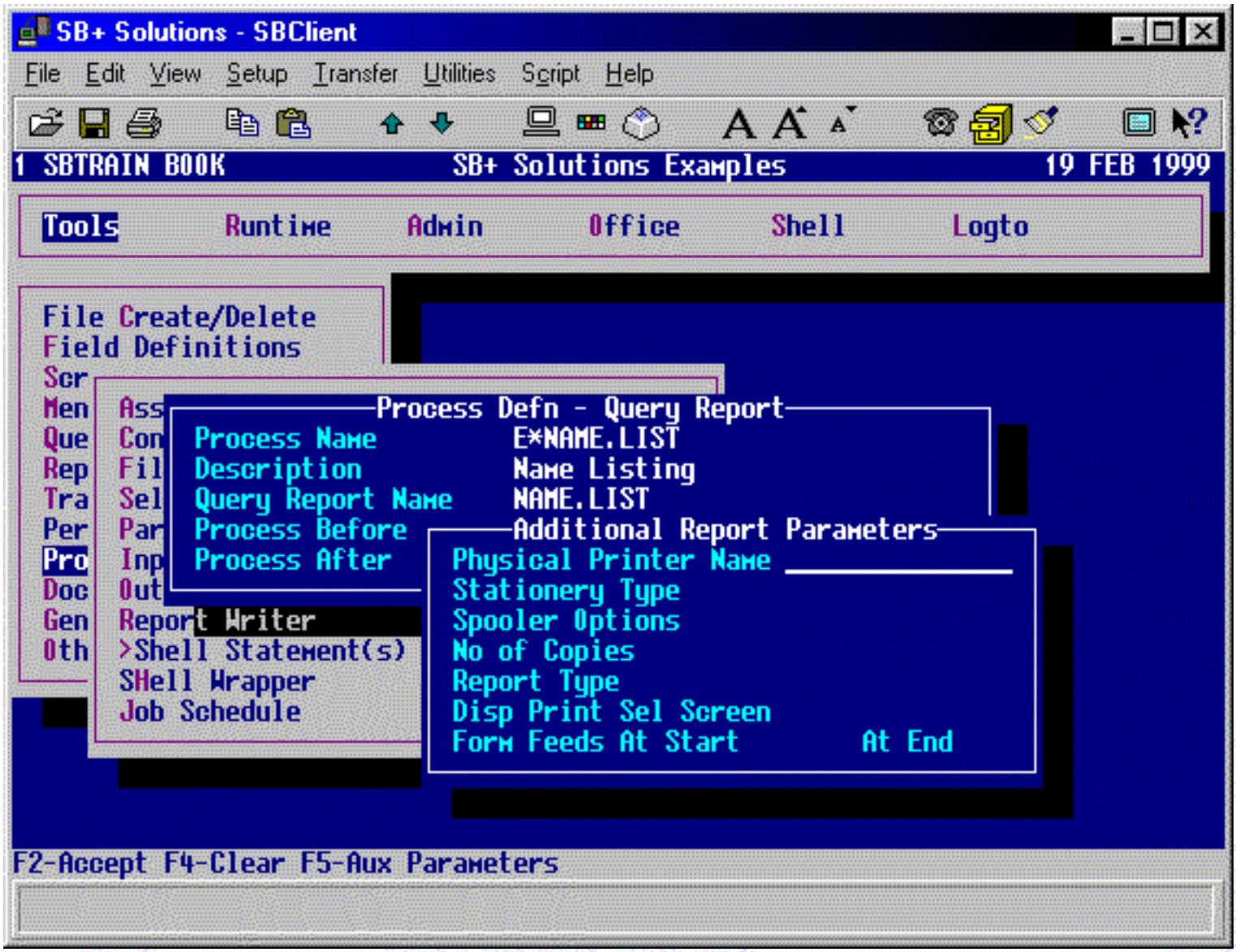
Once this screen has been started, simply enter the name of the query report process to be viewed, or use F3 to select one for display. The screen will appear as follows:



As you can see, there isn't much to this screen. At its most basic level, a query report process is simply a pointer to the query

report definition. However, there is actually more here than meets the eye.

On the function key line, notice the F9-Addit key. Using this key, you can define how and where the report will be output. Without this information, the report will be printed to the current printer on the current stationery, whatever it may be at the time. This additional information screen appears as follows:



Both Query and Report Writer processes use this information to define how the report will be printed. Therefore, rather than duplicate the explanation of this screen for each type of report, see "**Defining Stationery, Location, and Other Printer Specifics for a Report**" later in this chapter.

Defining Break Fields on Query Reports

In standard data processing terminology (an oxymoron?), a report "break" happens when the value of a field has changed from one record to another, and as a result there should be some recognition of the event, such as the printing of totals or a summary line. For example, if we were generating a report of sales by salesman and we wanted to show totals for each salesman, we can define the salesman ID to be a break field. With this defined, totals will be generated when a series of records for a given salesman has ended, as follows:

```
Salesman Order Total
001        1020.00
001        4500.00

***        5520.00

002          45.00
002          32.00
002          10.00

***          87.00
```

To define a field as a break field, simply append the literal "(B" to the end of the field name to break on in the Display Fields prompt. In other words, if the Display fields for this report are:

```
SALESMAN.ID ORDER.TOTAL
```

...changing the fields to:

```
SALESMAN.ID(B ORDER.TOTAL(T
```

...will break on the salesman ID and total the order total. (Totals are discussed in the following section.)

A report is not limited to a single break field, however. Multiple breaks can be defined for a report, such as the following salesman listing by state:

```
Salesman State Order Total
001     WA        800.00
001     WA         20.00

        ***       820.00

001     OR       4500.00

        ***      4500.00

***              5520.00

002     WA         45.00
```

```
002     WA        32.00
002     WA        10.00

        ***       87.00

***               87.00
```

In the case of this report, if the Display Fields would normally be:

SALESMAN.ID STATE ORDER.TOTAL

However, changing the Display fields to:

SALESMAN.ID(B STATE(B ORDER.TOTAL(T

...will generate the report as shown above.

Note that the break fields are shown in the order of significance: The state break will happen inside of the salesman break, resulting in a summary of sales by salesman and by state for each salesman. Reversing these two fields will result in a report that shows a summary of sales by state first, then by salesman within each state. Which report is better is, of course, a matter of your own personal style.

Following the (B you can specify one or several break modifiers, which change the appearance or function of the break. Some of the more common break modifiers include:

- P -- Generate a new page following the break
- U -- Underline all total fields
- V -- Output the value which caused the break instead of the asterisks.
- D -- Suppress break field if only one detail
- B -- Print break value in heading or footing where 'B' is found
- N -- Reset page counter to 1 following the break

Like most prompts in SB+, there is extensive F1 and F3 help available to help you enter this information. Also, because break modifiers are a function of the query language, consult the "official" query language documentation for a comprehensive list of all of the codes available.

Defining Total Fields on Query Reports

If a field on a query report is to be totaled, simply append "(T" to the end of the field name in the fields to display. As an example, to show a total of a field called ORDER.TOTAL on an order report::

ORDER.TOTAL

...would instead be entered as:

ORDER.TOTAL(T

When a field is totaled, totals will be generated at the end of the report as well as for any breaks that may be defined for the report. (If no breaks have been defined for the report, only grand totals will be printed.)

Columns or Forms?

The SB+ Report Writer tool can be used to create two different types of reports:

Columnar reports show information in columns, with any number of records shown on each page. The following sample is an example of what a columnar report may appear like:

```
 State Salesman Order ID Order Amount
xx    xxxxxxxx xxxxxxxx xxxxxxxxx.xx
      xxxxxxxx xxxxxxxx xxxxxxxxx.xx
      xxxxxxxx xxxxxxxx xxxxxxxxx.xx


xx    xxxxxxxx xxxxxxxx xxxxxxxxx.xx
      xxxxxxxx xxxxxxxx xxxxxxxxx.xx
```

In contrast, a form report shows one record per page, with any number of columns and rows on the page. Such a report could appear something like the following:

```
Sold To: xxxxxxxxxxx
         xxxxxxxxxxx
         xxxxxxxxxxx


Ship To: xxxxxxxxxxx
         xxxxxxxxxxx
         xxxxxxxxxxx


Stock Description..... Quantity
xxxxx xxxxxxxxxxxxxxxxxxxxxx xxxxxxxx
xxxxx xxxxxxxxxxxxxxxxxxxxxx xxxxxxxx
xxxxx xxxxxxxxxxxxxxxxxxxxxx xxxxxxxx
                      -------
                      xxxxxxxx
```

Because both columnar reports and form reports can show columns and rows of information, when all is said and done the only difference between the two comes down to one question:

- Should multiple records be printed on a page, or will the report print only one record per page?

When you look at the F6-Params screen for the report, this question is asked by the first prompt. If you want to create a columnar report, enter "Y" when asked "Multiple Records/Page (Y/N)". To create a form report, enter "N".

Another difference between a columnar report and a form report is in the sections that are used. Typically, a columnar report will use all of the report sections as described earlier. In contrast, a form report is typically composed of all detail lines, and has no heading, column heading, break, grand total, or footing section.

The last significant change between a columnar report and a form report involves how multivalues are handled. On a columnar report, any number of values can be shown for a given field in the record. This allows the detail for a record to shrink or grow depending on the number of values to be printed. However, on a form report, all of the columns and rows are of a fixed size and

position, and therefore you must tell SB+ how many values of a particular field are to be displayed.

Other than these differences, creating a columnar report and creating a form report involve basically the same steps:

- Place fields on the report and design the "look" of the report.
- Define the sorting and selection for the report.
- Create any special processes which may be needed for the report.

Using Report Writer Derived Values

Fundamentally, there are two types of fields that can be printed on a report. Real fields are those fields that point to an attribute in @RECORD or @KEY, or fields that have a **derived value** in the field definition. In contrast, a report definition can also contain *derived values*, which are fields that calculate a value but have no actual field definition.

Derived values exist only for the report where they are defined, and can be used to output literally anything that can be calculated or retrieved from the common map. Because there is no actual field definition, derived values can be created and used in any number of reports with no risk of conflicts. On the down side, each field must be created from scratch each time it is needed.

For example, let's assume we want to output the current user ID in the heading of a report. To do this, we need to create a field on the report, but it just doesn't make sense to create an actual field definition. Hence, this is a perfect example of how a derived value can be used.

To create a derived value field on the report definition, press F5-Field to add a field to the report. When asked to enter a field name, enter anything other than a valid field definition name. SB+ will respond by printing a message at the bottom of the screen, as follows:

Not Found: <Enter> Create / Type Len (,Lines) / Esc

If you press <cr> at this prompt, the field definition tool will be invoked so that the field can be added to the field definition. This is not what we want to do. Instead, we'll take the second (and notably cryptic) option, which allows us to enter the type of field to create followed by the length of the field. For example, to create an alphanumeric field that is 8 characters long, "A8" may be entered at this prompt. For a monetary field that is 10 characters long, "M10" can be entered. Any of the four data types (A, D, N, M) can be used, and the length can range from 1 character to the ridiculous.

Once you've entered the type and length of the field, the only thing left to do is enter a derived value (i.e. SB+ expression) in the Derived Value field on the F5-Field window. Once a derived value has been entered, press F2 and the field will be added to the report definition.

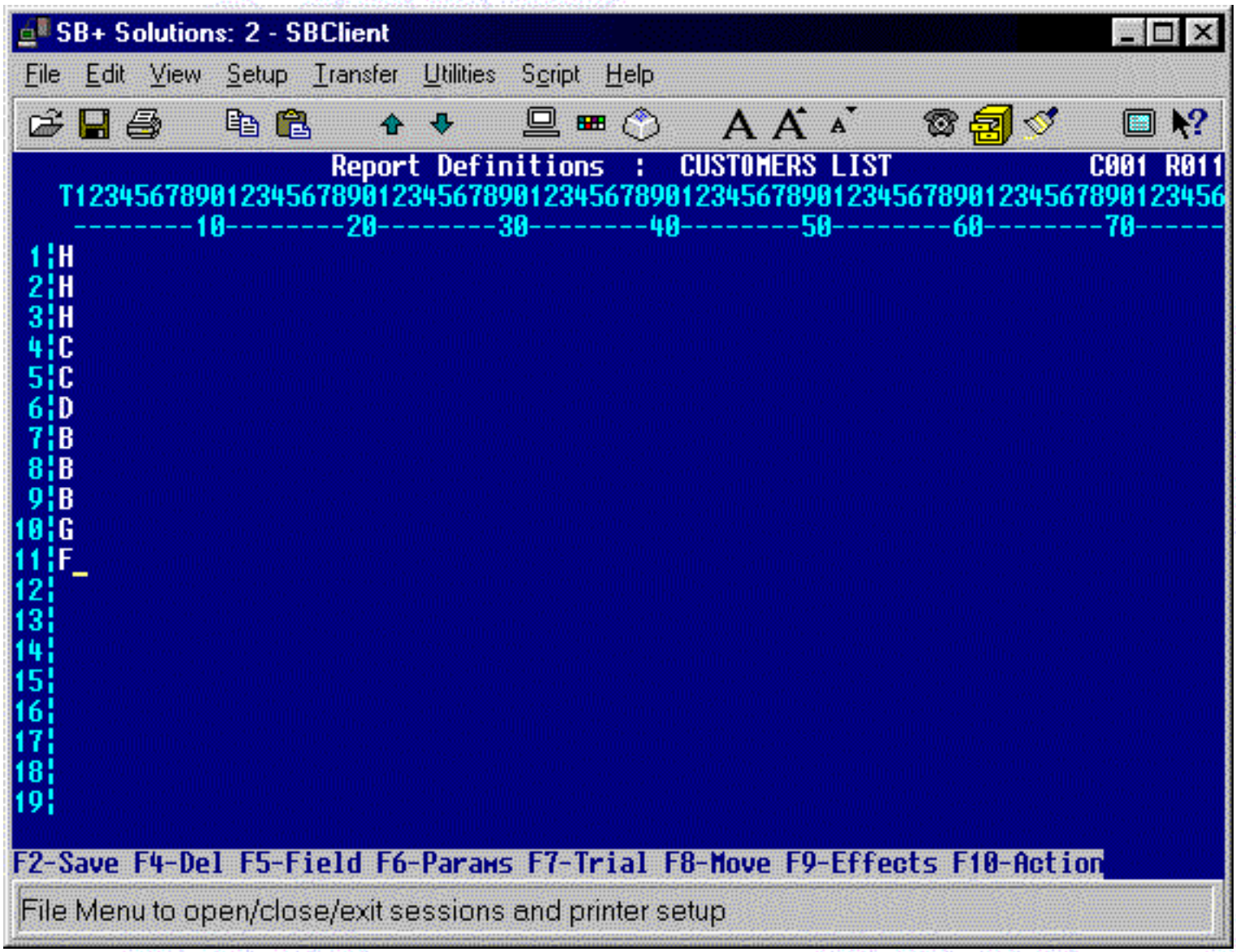
There are a hundred and one (or more) uses for derived fields, many of which will be explained in the following sections. Therefore, it is imperative that you become familiar with these types of fields so that you can create new derived fields easily and quickly.

Creating Columnar Reports

When you create a columnar report with the SB+ Report Definitions tool, the report is created as a series of sections, each with its own characteristics and purpose. There are six different sections that can appear in a report:

Heading	The heading section defines the lines that will be printed at the top of each page. Therefore, this section is executed once for each page that is printed. Each heading line is identified by an "H" in the leftmost column.
Column Heading	The column heading is an extension to the heading lines, and will be output once for each page, following the heading lines. Each column heading line is identified by a "C" in the leftmost column.
Detail	The detail section defines the fields to be output for each selected record. This section will be output once for each record selected for the report. Each detail line is identified by a "D" in the leftmost column.
Break	If the report has break fields defined, each time the value in one of the break fields changes from one record to the next, the break section will be executed. If a report has, say, two breaks and the value in the major break field changes, the break section will be processed once for the minor break first, then again for the major break. The break section will be executed once for each break field at the end of the report. If the report has no grand total section, the break section will be processed one additional time for the major break. Therefore, if you have a report with any breaks, it is important to include your own grand total section, instead of allowing SB+ to process the break section one additional time. Each break line is identified by a "B" in the leftmost column.
Grand Total	The grand total section defines the final totals for the report. It is executed once after all of the selected records have been processed. Each grand total line is identified by a "G" in the leftmost column.
Footing	The footing section defines the lines to appear at the bottom of each page and is executed once for each printed page. Each footing line is identified by an "F" in the leftmost column.

When more than one of these sections are used in the same report, all of the lines for each section must appear together and the sections must appear in the order as listed above, as illustrated in the following example:



Any section of a report definition may be defined with any number of lines, as long as the total of all lines in the report definition does not exceed 100. Also, each line of a report definition may be up to 450 characters in width. If you don't have a printer which can print 450 columns, not to worry: SB+ has an automatic page-splitting feature which will print the full 450 columns of the report, split horizontally over several pages that can be pasted together. While pasting pages together doesn't necessarily produce the best looking report, the fact still remains: it is possible.

Reports printed to the screen can also be scrolled up, down, left, or right. Therefore, even if your display device only supports 80 columns (or maybe even fewer), with horizontal paging you can see a report that is much wider.

Of course, the maximum rows and columns for a report are mere statistics without the consideration of the paper that a particular report will print on. (If the report will only be printed on the screen, ignore the following...) In order to determine the practical size for a report definition, you must first know the size of the paper (as measured in columns and rows) for the report. Regardless of what SB+ *can* do, the size of the paper will determine what you *should* do.

Placing Fields on a Report Writer Report
Templating Reports
Creating Break Totals/Grand Totals
Creating a Record Counter

Creating Conditional Totals
Creating Detail Section Totals
Creating Breaks on Report Writer Reports
Creating Summary Reports
Conditionally Suppressing Detail Lines
Calling a Process After A Line Has Been Printed

Creating Form Reports

Form reports allow you to create a report definition that will print a complete page at a time. For example, some typical form reports might include sales order review forms, invoices, and/or customer statements.

Building a form report in SB+ is just as easy as building a **columnar report**, and many of the same techniques apply. However, there are a few of differences to note:

- Form reports do not generally use heading, column heading, break, grand total, or footing lines. Instead, a form report is comprised of all detail lines.
- On the F6-Params screen, when asked "Multiple Records Per Page", enter "N" for form reports. This flag tells SB+ that for each record processed, a full page will be output.
- When placing multivalued fields on a form report, you must tell SB how many rows of information will be printed for each field. Unlike a columnar report, it simply will not suffice to assume a multivalued field will fill up a page and automatically roll over to the next.

Other than these issues, creating a form report is essentially the same as creating a columnar report. The same **selection options** apply. The same function keys are used to add fields to the report and change the report parameters. The same type of "after-read" processing can be used to construct the information to be output. In short, nearly everything in this chapter can apply to both types of reports, with the exceptions as illustrated in the next few pages.

Creating a Form Report
Fields on a Form Report
Numbering Pages on a Form Report

Report Writer Conversion Magic

When you're building complex reports with loads and loads of information, sometimes you need to make sacrifices in the format. Most times, these sacrifices come in the form of showing more codes than descriptions. This can be acceptable for the detail section of the report, but can come across a little terse when done in the summary.

Fortunately, it's very easy to reformat the value of any field in a report definition using the Conversion prompt in the F5-Field window. At this prompt, you may enter any standard OE conversion, or better yet, any valid SB+ expression (enclosed in parentheses). In the case of the example shown in the **previous section**, if the name of the salesman is shown in attribute 4 of a file called SELLERS, we can translate any salesman ID code to a name with the following conversion:

(F('SELLERS',@VALUE)<4>)

In this syntax, SB+ has the unconverted value in the **common variable** @VALUE. You can use this information in any standard SB+ expression (or process call, using the P(...) syntax) to change the value that is displayed. Moreover, if the field is multivalued -- no problem -- SB+ will convert and display each value separately with no additional work on your part!

Stripping Data From A Record Prior to Output

As illustrated prior, the process after read can be used to build information to be output. It can also be used to strip information from the current record prior to the record being formatted for output. This is most useful when you want to pre-process the record and remove values depending on certain conditions.

To see how this can be useful, let's create a report of sales information. Each sales record has multivalued information in fields called STOCK.NBRS, QTYS, and PRICES. When this report is invoked, we only want to show the detail for certain stock items (such as promotional items), as described in a multivalued list in @PARMS(2)<1>. All of the information for other products should be stripped from the record.

As long as you don't explicitly write the record in a process, there is no danger in changing any of the **common variables** used for the report (i.e. @RECORD or @WORK). Report processes do not update any records, so once a detail has been printed the common variables are abandoned and loaded with next record's information.

The process to strip out certain stock items from an order can be a paragraph which looks something like the following:

```
@CNT = DCOUNT(<POS(STOCK.NBRS)>,@VM)
*
WHILE @CNT DO
  IF LOC(STOCK.NBRS,@PARMS(2)<1>,@VM) = 0 THEN
    @RECORD = DEL(@RECORD,POS(STOCK.NBRS),@CNT)
    @RECORD = DEL(@RECORD,POS(QTYS),@CNT)
    @RECORD = DEL(@RECORD,POS(PRICES),@CNT)
  END
  *
  @CNT = @CNT - 1
REPEAT
```

Counting from the last element in the STOCK.NBRS array to the first, we simply remove the values from the STOCK.NBRS, QTYS, and corresponding PRICES arrays for the items we don't want to show on the report. When the record is output, only the details we want to see will be shown.

Unfortunately, this particular process has overlooked the possibility that we may have a record with no eligible stock numbers on it. In this case, rather than showing a line with no stock detail, we may be better served to skip the output of the detail completely, using this variation:

```
@CNT = DCOUNT(<POS(STOCK.NBRS)>,@VM)
*
WHILE @CNT DO
  IF LOC(STOCK.NBRS,@PARMS(2)<1>,@VM) = 0 THEN
    @RECORD = DEL(@RECORD,POS(STOCK.NBRS),@CNT)
    @RECORD = DEL(@RECORD,POS(QTYS),@CNT)
    @RECORD = DEL(@RECORD,POS(PRICES),@CNT)
  END
  *
  @CNT = @CNT - 1
REPEAT
*
IF STOCK.NBRS = '' THEN EXIT 1
```

After the loop is done, if we have no stock numbers we can set @RTN.FLAG to 1 (with the EXIT 1) which tells the Report Writer we don't want to show this detail. Note that because @CNT will be zero when the loop terminates, we can reference the field by name, and it will represent the field as a whole, not a specific value inside of the field.

Building Information To Be Output

Similar to stripping information from a record prior to output, you may also use a Process After Read to build information to be output on the report. Information can be built into @RECORD, @OTHER.REC, @WORK, @PARMS(...), and @USERDATA (...) and referenced in derived fields in the detail section of the report.

To see how this can be useful, assume we have customer records in a file called CUST. Inside of each customer record is a name and a multivalued list of "address type codes", which are used to read records from a file called ADDRESS. In short, if our customer ID is 1000 and he has two address codes (P=Personal, B-Business), there will be two ADDRESS records called 1000*P (for the personal address) and 1000*B (for the business address).

For the purpose of this demonstration, we want to create a report that outputs all of our customers and the corresponding business address for each. If a customer doesn't have a business address, we don't want to show the customer on the report.

To implement this, we must first create a report in the CUST file. On this report, we'll show the customer ID and the name from the CUST record, and the mailing address from the ADDRESS record. Remember, we're only interested in the 'B'usiness addresses.

One way to approach this problem would be to create **derived value fields** in the CUST file to read each part of the address (Address, City, State, Zip) from the ADDRESS file. Similarly, we could create derived fields which use the F(file,item)<fieldpos> function to read each field from the ADDRESS record. But this wouldn't be a good use of system resources. Always remember that any time the F(...) function is used, a record is read. We certainly wouldn't want to read the same ADDRESS record four times for each customer record being output, right?

Therefore, we can create a Process After Read to load this information from the ADDRESS file and store it in a **common variable**, such as @PARMS(2), as shown in the following paragraph:

```
@PARMS(2) = F('ADDRESS',@KEY : '*B')
IF @PARMS(2) = "" THEN EXIT 1
```

Though the code itself is brief, it accomplishes everything we want to do: Read a business address record from the ADDRESS file for the current customer ID (as defined by @KEY), and store the record in @PARMS(2). If no address exists, skip the record (via **EXIT** 1). With this process in place, we can reference the individual fields in @PARMS(2) in derived fields in the report, and pull anything we want from this copy of the address record.

Anything you can calculate based on the current record can be done inside of either a process after read or a derived field. For simple calculations or one-time reads, a derived field will work fine. However, when you need several pieces of information from the same record, or a calculation is of substantial complexity, use the Process After Read to build the information to be printed.

Creating a Report for Multiple Parallel Files

Imagine if you will: Over the years and years that your company has been doing business, general ledger information has been accumulated in a number of different files, one per fiscal year. Today, 17 years into this process, someone has asked for a report which shows all of the information from all 17 files. As if this weren't complicated enough, the details on the report need to be sorted by general ledger code and then in ascending date order for each code.

Without SB+, such a report might evoke feelings of fear, perhaps even panic. However, using SB+ this kind of a report is hardly worth a sweat.

A group of files that share a common dictionary are commonly referred to as *parallel files*. Each file contains its own information, but the layout of that information (as defined by field definitions) is common to all of the files.

To implement such a report, we'll use a **paragraph** to start the report, and the paragraph will then call a **periodic update** to load pertinent information from all of our files into a central repository. Once we've combined all of the information we need into the central file, we can run the report from the data that's been collected into the central file.

The starting paragraph appears as follows:

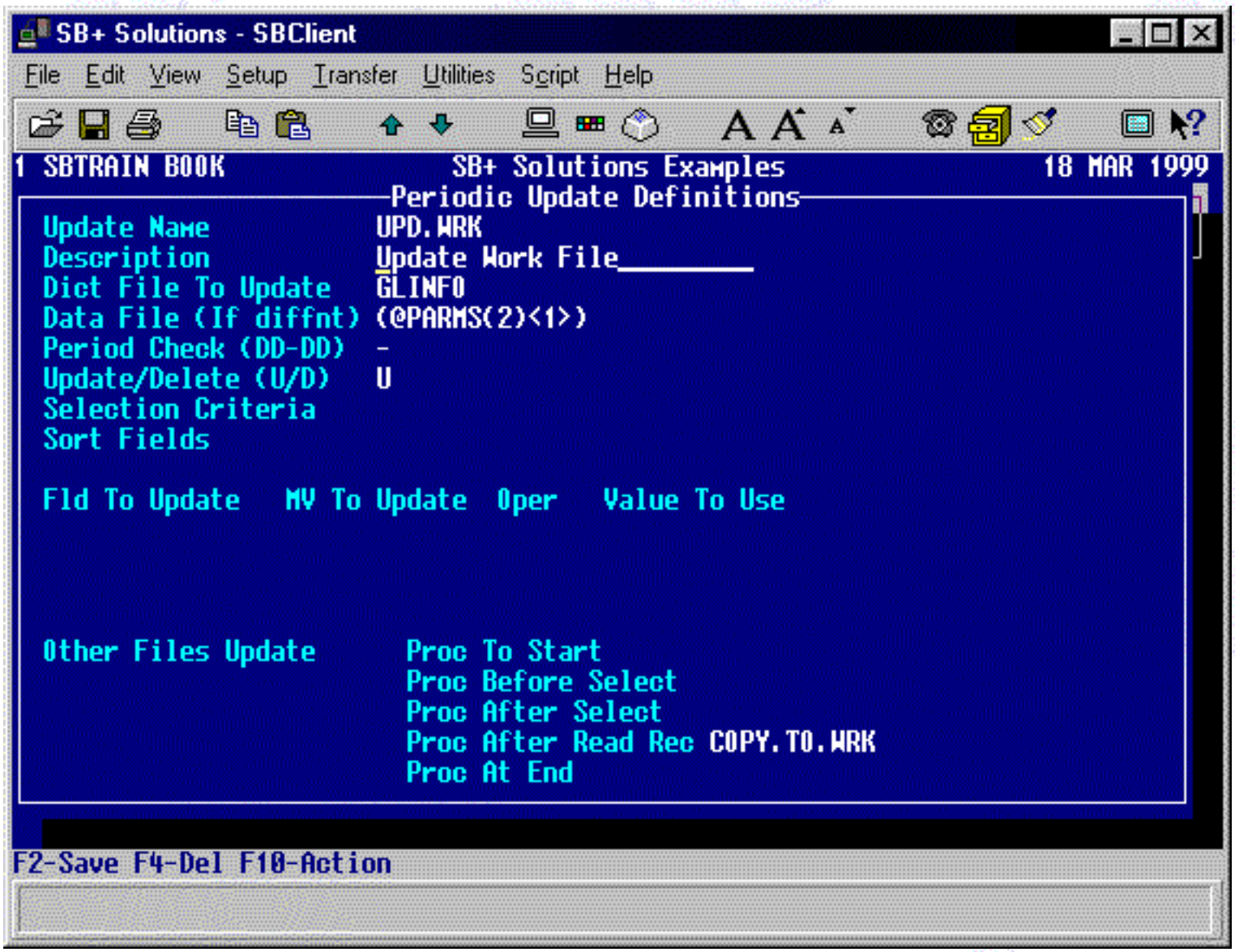
```
LOCAL PROMPT,ANS,NDX
*
* First, prompt for screen/print/aux/etc.
*
PROMPT = F('DMCONT','PROMPT')<17>"G\1"
*
ANS = 'S' ;* Set the default to 'S'creen
INPUT ANS,1,PROMPT
*
IF NO.ESC THEN
  @OTHER(8)<15> = ANS
  *
  * Next, let's build the information for the report
  *
  EXEC '>:CLEAR-FILE DATA GLINFO.TEMP'
  NDX = OCONV(@DATE,'D4Y') ;* D4Y = Return year w/ 4 digits
  WHILE NDX >= 1982 DO
    @PARMS(2) = "GLINFO.":NDX
    EXEC "UPD.WRK" ;* The periodic update
    NDX = NDX - 1
  REPEAT
  *
  * Now we can execute the report
  *
  EXEC 'R*GLINFO*LIST'
END
```

This paragraph first prompts for the output of the report. By prompting for the report destination first, putting the user's response in @OTHER(8)<15>, when the report is finally run, SB+ knows where we want to send the report and therefore won't ask us. This allows us to put all prompting at the front of the report, and after the user has entered a value, the report runs to completion without any additional user intervention. (Note that in production programming, I have a process to do this prompting and validation of the user's entry. For the sake of brevity, validation is omitted in this example.)

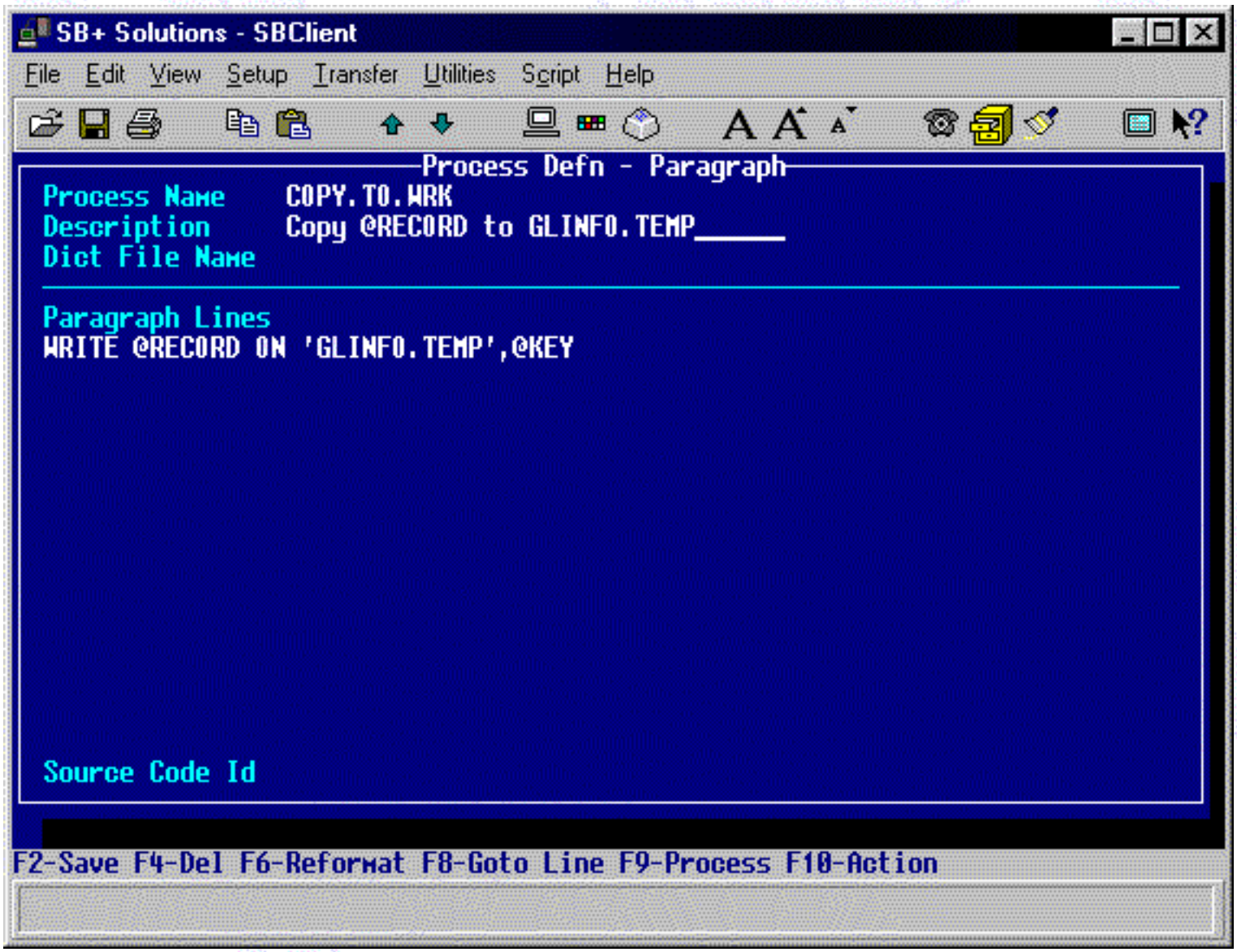
The global equate NO.ESC will be true if the user presses the Escape key from the opening prompt. If they do, we simply skip over all the remaining lines of the paragraph and the process terminates. If they don't press Escape, they must have entered something into ANS, and therefore we tuck the entry away in @OTHER(8)<15> to be used later by the report.

In the next section, we call a periodic update named UPD.WRK. This update will read records from each of the GL files and will copy them to the GLINFO.TEMP file. Before we go populating the GLINFO.TEMP file, however, we need to clear it to remove any unwanted information. This is done with the *inline TCL statement*, identified by the literal >: at the beginning of the command. (Note that we're assuming the GLINFO.TEMP file has been created and is sized properly.)

In our loop, we need to run the periodic update for a bunch of files -- namely, all files from 1982 to the current date. Note that instead of hard-coding 1999 as the ending date, I'm using the current year so the program will continue to run in the year 2000 (and beyond) without modification. For each iteration of the loop, we construct the name of the GL info file into @PARMS(2), and then call the update. The update then copies all of the records from the file as named into the GLINFO.TEMP file. This update appears as follows:



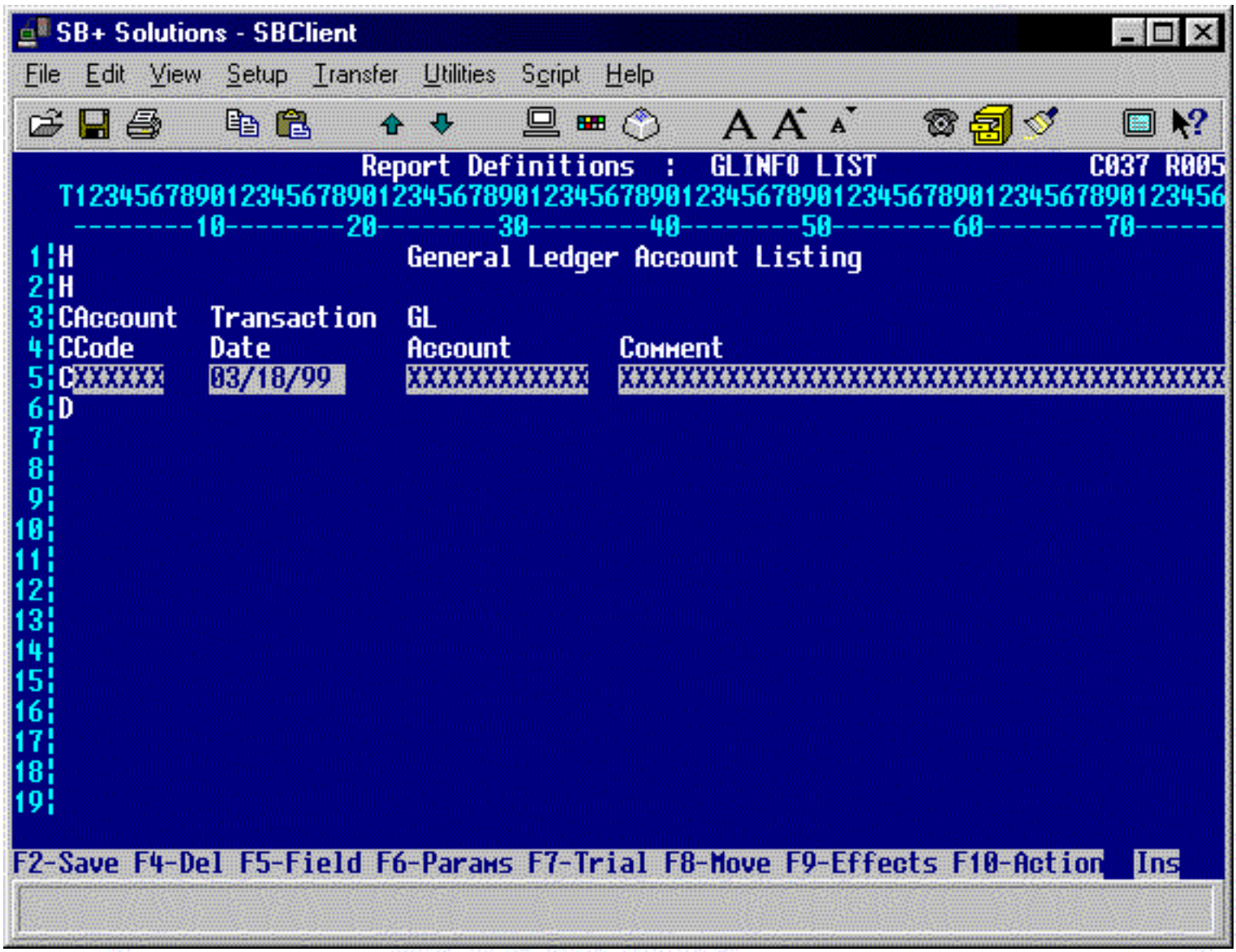
And the paragraph called from the Proc After Read Rec, which causes the GLINFO.TEMP file to be updated, appears as follows:



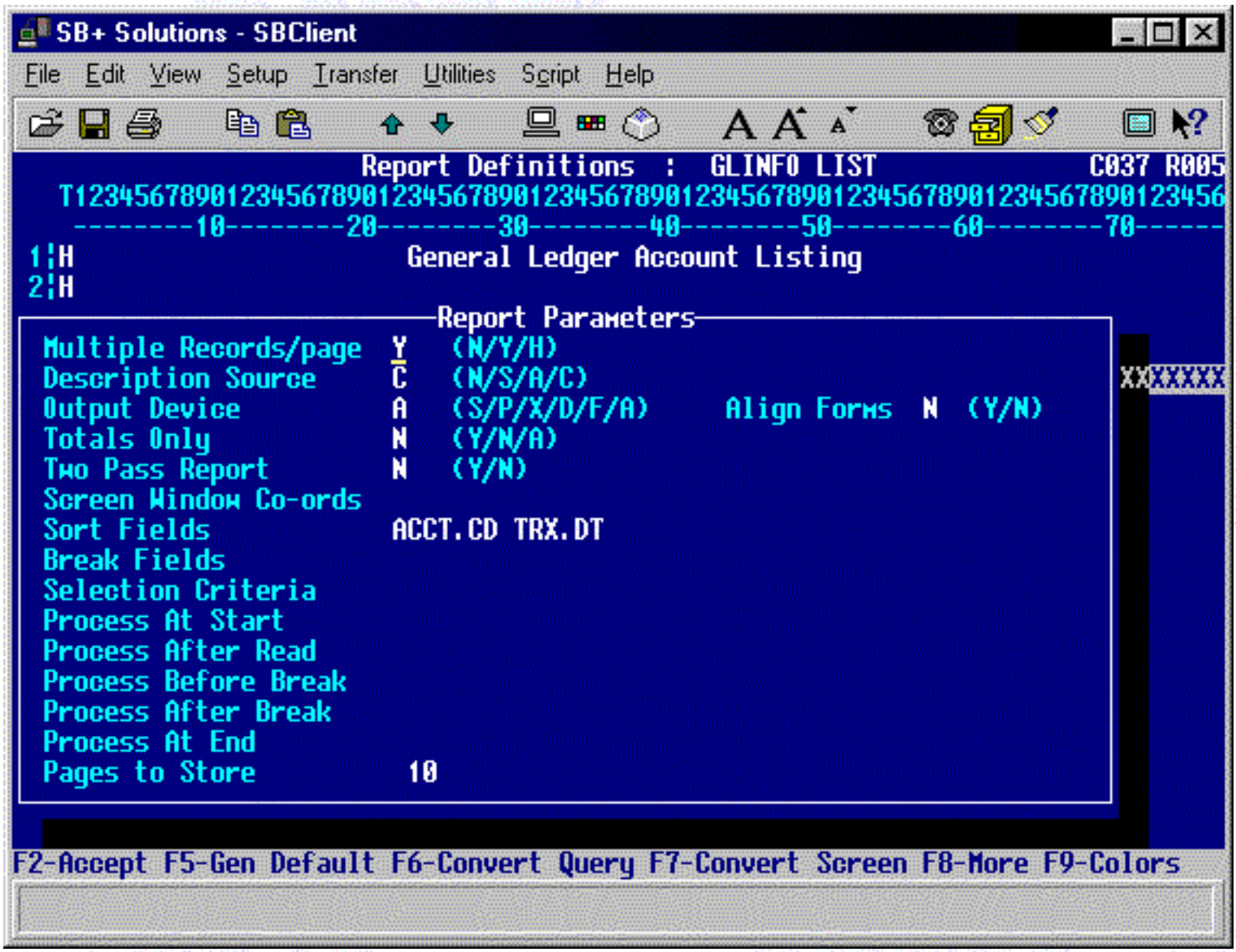
Note that we don't need to move the record from the source to the destination file on a field-by-field basis. This paragraph will copy the entire record from the main file to the temporary file.

Back to the earlier paragraph... Once the periodic update has been executed for each of the incoming files, we can run our report. Note that the report lives in the GLINFO file (the common dictionary), instead of any of the incoming files or the temporary file. This allows us to use any of the fields in the GLINFO file in our report, while all the time pulling data for the report out of the GLINFO.TEMP file.

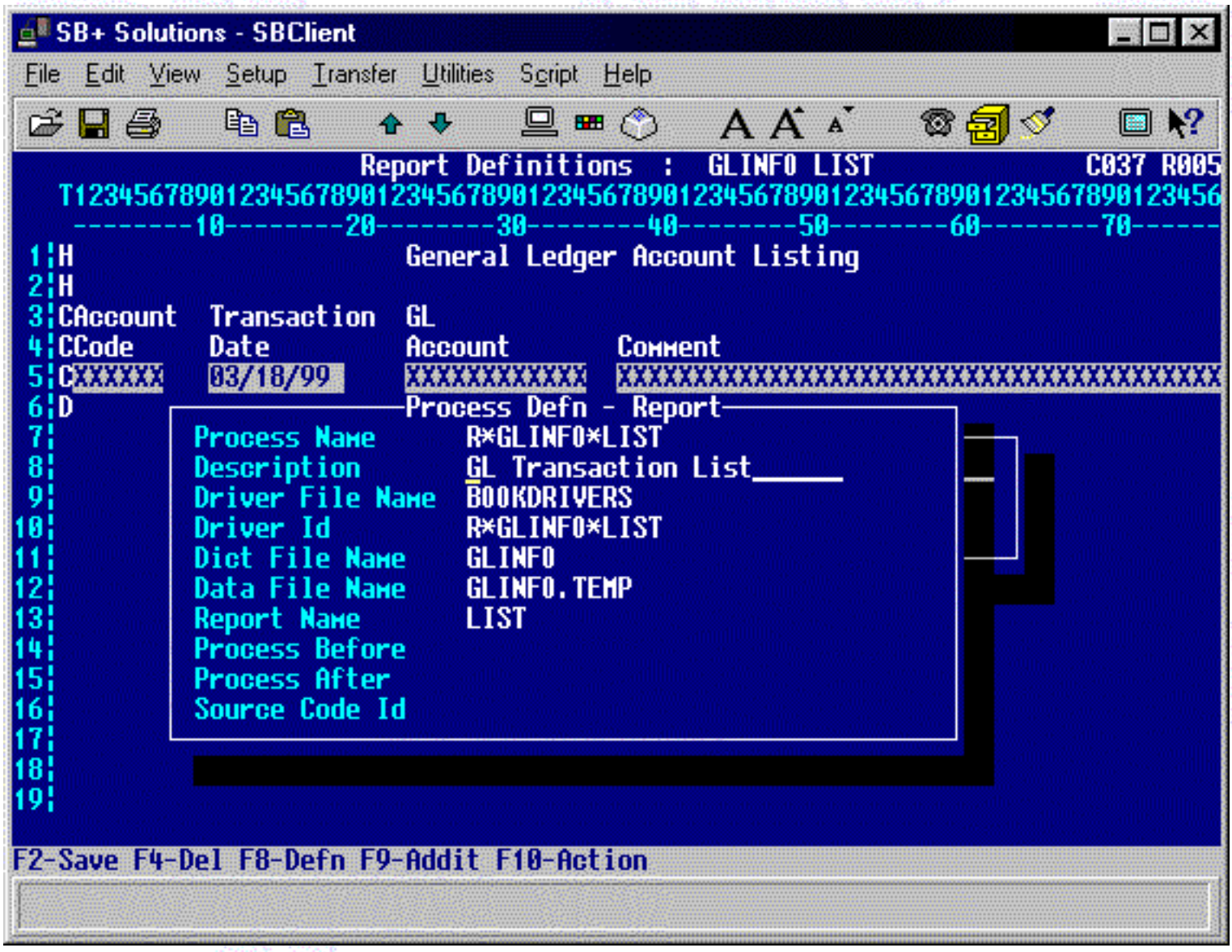
The layout of the report appears as follows:



We can see the sort fields on the F6-Params screen as follows:



And finally, we can see the process used to run the report, where the dictionary file and data file are different, as follows:



There are a number of variations on this technique. For example, you may not have enough disk space to build a file that contains the contents of 17 or more other files. In this case, you need only move the fields needed for sorting and selection to the central file, plus a reference so you can find the original record when the report is running. The report, then, would run from this temporary file, and would use the Process After Read to read the actual data record from one of the multitudes of files.

Running the Report from a Window

Any report that can be run to the screen can be run either on the whole screen or inside of a window. The latter method is very useful, particularly when you want to run a report without disturbing some information that may be displayed on a calling entry or query screen.

Running the report inside of a window will not limit the abilities of the Report Writer tool, with one exception. While you can still page forward, back, and side to side to view all of the text, a report running in a window will not use the compressed font (132-mode) of the display device.

To change a report to run inside of a window, simply go to the F6-Params screen in the Report Definitions tool, arrow down to the field labeled "Window Coords", and enter the window coordinates in the following format:

starting column,starting row,width,depth

Of course, like any window coordinates prompt in SB+, you may press F3 and use the arrow keys to define the size of the box you want.

Selecting Records to Output

Up to this point, we've looked at many different ways to *build* information to be output on an SB+ Report Writer report. Equally important, we need to look at the different ways we can *select* records to be included on the report.

At any prompt labeled "Selection Criteria" you may enter a phrase to select:

- One record;
- A group of records, based on a condition;
- All records;
- All records in a select list, or;
- A group of records as defined by keys in a multivalued list in common memory;

The following sections describe these selection options in more detail.

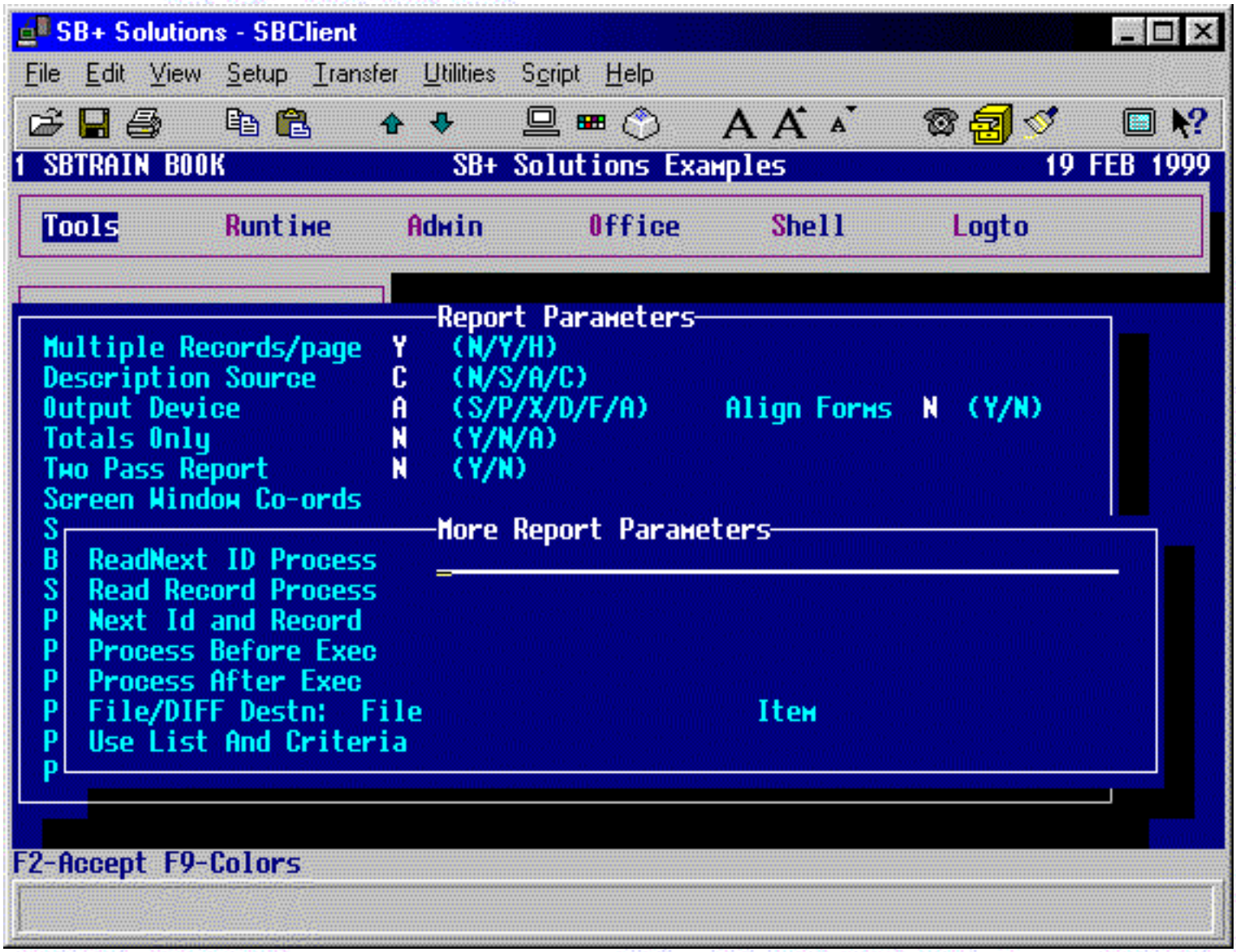
Selecting One Record
Selecting a Group of Records Based on a Condition
Selecting All Records
Selecting All Records in a Saved List
Selecting Keys From a Multivalued List in Common Memory
QSELECTing A Multi-attribute List
Implementing Impossible Selection Criteria

Calculating Keys On-The-Fly

The selection options that SB+ provides covers most of what you'll need under normal circumstances. However, which us has never faced the abnormal?

SB+ allows you to calculate the key value for the report, thus eliminating the need to select anything. For example, if you have a linked-list structure where record A points to record B, record B points to record C, and so on, you may not want to select the records in the file. Instead, tell the Report Writer to traverse the linked list to get the keys.

On the F6-Params screen in the Report Writer there is an F8-More key available. When you press this key a small window appears as follows:



On this screen, the ReadNext ID Process and the Next Id and Record slots can be used in place of **Selection Criteria** back on the F6-Params screen. These process slots allow you to move from record to record on a report, instead of having to select all of the records up front using standard selection criteria options. The Read Record process, on the other hand, allows you to substitute whatever logic you require in place of the standard read step of the report.

The ReadNext ID Process
The Read Record Process
The Next Id and Record Process

Creating a Report with Multiple Selection Options

Though the SB+ Report Writer tool has only one **Selection Criteria** prompt, you are not limited to only one selection option per report. In fact, any single report in the Report Writer can be fronted by any number of selection options. Perhaps you want the same report to be shown with different information. Or maybe you'd like one report to be shown sorted several different ways. Fortunately, all of these options (and more) are very easy to implement.

Any report definition can get a list of keys from a saved list. How this list is built, including what is in it and in what order, is completely up to you.

For example, let's assume we have a list of records in a file called NAMES. This file has attributes for NAME, ADDRESS, CITY, STATE, and ZIP. Assume also that we have a report showing all of these fields.

We can create a menu to run this report which appears something like the following:

Sort by Name
Sort by City
Sort by State

If the user selects the first option, we can call a process to SSELECT the file in name order, saving a list. Similarly, if the user selects the option to sort by city or state, we can call processes to SSELECT the file by city or state, respectively. Regardless of which process is called, the end result must be the creation (or deletion) of a select list.

Once a list has been created, the report process can be called, which will then use the select list and output the records in the proper order, depending on the option that was selected.

Now, there are a couple of interesting problems with this technique. First, if there are many records to be selected, SB+ will select all of the records to be output (which could be a time-consuming task) prior to asking the user where they want the report to be output. This can be irritating, as it introduces what could be a significant pause between the start of the report and a very important intermediate prompt. Secondly, there is really no good out-of-the-box tool in SB+ for building select lists. Fortunately, we can build a couple of standard processes to correct both problems.

For starters, let's look at the problem with the delay between the start of the report and the prompt asking for the output destination. This problem happens because the sequence of events is such that the report process prompts for the output destination, and this process is not invoked until after the select list has been constructed. To correct this problem, we merely need to remove this feature from the report process, and move it up in the sequence of events so that it happens prior to the selection of records for the report.

On the surface, this sounds difficult. The prompting for the output destination is such an integral part of the report process, the two may appear inseparable. However, this is not the case.

First, however, a little history...

When prompted for "Output to Screen/Print/Aux (S/P/X)", whatever you enter is stored in common in @OTHER(8) <15> -- or in Report Writer terms, @RV.OUTPUT. This is done for a variety of reasons. Most importantly it provides a way for you to know what the user entered when doing other processing for the report. Oddly enough, however, this value can be set prior to calling the report process, and the Report Writer will assume you've already answered the question and therefore will not ask it again.

Therefore, if we create a process to set @OTHER(8)<15>, we can call this process at any point in the processing for the report, and therefore suppress SB+'s prompting for the same information at a later time.

The following process will accomplish exactly that:

```
LOCAL PROMPT,ANS
*
PROMPT = F('DMCONT','PROMPT')<17>"G\1"
*
ANS = 'S' ;* Set the default to 'S'creen
INPUT ANS,1,PROMPT
@OTHER(8)<15> = ANS
```

This is a very simplistic process, no doubt, and could stand to be enhanced with some input validation, but for the most part, it illustrates how easy it is to input a value into a **common variable**, such as @OTHER(8)<15>. .

Now, on to the next problem; SB+ does not have a really good tool for building a select list. With the following paragraph, however, this isn't much of a problem either.

```
LOCAL PROC,NDX,MAX
*
PROC = 'PQ'
*
* Delete the list if it previously exists
*
PROC<-1> = 'HDELETE-LIST WRK.':@PORT
PROC<-1> = 'P'
*
* Build the selection information into the proc.
*
PROC<-1> = 'H' : @VALUE<1,1>
PROC<-1> = 'STON'
*
MAX = DCOUNT(@VALUE,@VM)
IF MAX > 1 THEN
  NDX = 2
  WHILE (NDX <= MAX) DO
    PROC<-1> = 'H' : @VALUE<1,NDX> : '<'
    NDX = NDX + 1
  REPEAT
END
*
* Now save the list
*
PROC<-1> = 'HSAVE-LIST WRK.':@PORT
PROC<-1> = 'P'
*
WRITE PROC ON 'MD','PROC.':@PORT
EXEC '>:PROC.':@PORT
DELETE 'MD','PROC.':@PORT
```

This paragraph will take a series of selection statements in @VALUE (separated by value marks) and will create a proc which executes the statements in succession. In the end, if there are any keys left in the active select list, the list will be saved under the name WRK.port. This name can then be used in the selection criteria of the report. Also note that this proc deletes the list before it selects anything. This way, the process doesn't have to check to see if anything was selected, which can be problematic on certain

operating environments.

To use this process, use a paragraph to construct a series of selection statements in @VALUE, then call this process. For example, if this process were named SELECT.TO.LIST, we can invoke it as follows:

```
@VALUE = 'SSELECT NAMES BY NAME'
EXEC 'SELECT.TO.LIST'
```

With this process, the NAMES file will be selected in name order, and a list will be saved with the keys. Or, if nothing is in the NAMES file, the list will simply be deleted, which will prevent the report from printing erroneous information.

Best of all, these two processes can be used over and over, giving you the option to have multiple selection options on nearly any report in your system quickly and easily!

Of Course, There Are Exceptions...

While you can dynamically change the sort order and selection criteria for any report using processes to build select lists to be passed to the report, you cannot dynamically change the **break fields** for a report. Or better stated, you cannot easily change the break fields for a report.

Like anything in SB+, if you want something bad enough there will be a way to accomplish it. As a general rule, however, break fields are assumed to be fixed for each report definition, and therefore a reporting program needing multiple break options will need to invoke multiple report processes to accomplish its desired result.

Version 2.x Common Map

Section 1

RECORD - In an input or output process, this variable contains the record that is being manipulated or displayed by the screen. For periodic update processes, this variable contains the actual record being processed in a given cycle.

For selection processes, this variable is used a couple of different ways. First, if the selection process uses the "?" to prompt for input, RECORD is where the values are entered. The first prompt is input into RECORD<1>, the second into RECORD<2>, and so on, regardless of the field positions of the fields as named in the selection criteria. Also, as the actual selection window is being displayed, RECORD contains each record as it is being displayed.

KEY - Every place where RECORD is set, KEY contains the record key used to read the record.

WORK - This variable is not set or changed by SB+, but is a single dynamic array that can be used by a developer for entry or display of information. This variable is used for constructing multi-part keys, hidden keys, or for building information for later use in a screen, report, or update application.

OTHER.REC - This variable is a place where secondary records can be read by either your application or SB+. Typically, this variable is used during the update of secondary files in the Process After Update, in lieu of using **local variables** (i.e. when you can't use any more local variables). However, SB+ uses this variable in a couple of different ways which aren't too obvious.

For example, if you have the following paragraph:

```
READ @OTHER.REC FROM 'MD','XYZ'
@VALUE = F('CUSTOMER','ABC')
```

The first line reads @OTHER.REC from the MD file, using 'XYZ' as a key. The second line reads a record from the CUSTOMER file using 'ABC' as a key, but where does it store it after it has been read? First glance would make one think that the record was read into @VALUE, right? However, this is not the case. Whenever a record is read using the F(...) function (without the <n> on the end) the record is read into @OTHER.REC, and then copied to the variable on the left side of the expression. Therefore, in this little example, the record is read from the CUSTOMER file into @OTHER.REC, than copied over to @VALUE. In short, the product of the preceding READ is lost when the F(...) function is used this way.

RESVD - This variable is reserved for future use.

CNT - In an entry or query screen, this variable tracks the current multivalue position. If a field is not multivalued, @CNT will be zero. When constructing a standard process library, this is one way you can tell whether you're working in a multivalued or single valued field at the time the process is called. If @CNT is zero, the field is single valued -- any other value says the field is multivalued.

ACTION - In an input process, this field defines whether a new record is being added, an existing record has been loaded for modification, or if a record is being deleted. If this variable is "1", the record is new. If "2", the record was read from disk and is being modified (amended). If "3", the user has pressed the F4 key or done something else to tell SB+ to delete the record.

Prior to the entry of a key, the value of this variable could be anything. Generally, it is either "0" or "1". Values "1" and "2" are set

after the record has been read. Value "3" is set when @RTN.FLAG is set to "D" or "DE". Therefore, in the Process After Read, you don't need to worry about @ACTION = 3, because it would be impossible at that point. In the Process After Screen Accept or Process After Update, however, all three settings should be accounted for.

LINE - On any input screen (including a regular input process, SB+ tool, and the "?" prompting of a selection process) this variable contains information about the current prompt. LINE is a single attribute, with numerous multivalues defining various characteristics of the prompt. These values are:

1	The attribute number in @RECORD to be updated.
2	The value number of the attribute in @RECORD that this prompt will be updating. Typically, this is zero for single valued fields, negative for multivalued fields (-1:control multivalue, -2,-3...:dependents), or positive for actual multivalue positions in an attribute. Note that when the value is negative, it will not tell you which multivalue is being updated, but rather the sequence of multivalued fields as they appear on the screen. (@CNT will tell you the actual multivalue number.)
3	This value tells SB+ the column number of the first character of the input prompt on the screen.
4	This value tells SB+ the row number of the input prompt on the screen.
5	If the prompt has an **output conversion** or **derived value**, it will be in this value
6	This value tracks the sequential number of the field on the screen. All fields on the screen are numbered sequentially from 1..n so that SB+ knows which value in the OUTVAL variable to reference for a given field.
7	On a controlling multivalue, this field tells the number of rows in the multivalued window (also known as Window Size)
8	This value tells SB+ the length of the input area on the screen
9	This value tells SB+ what type of input this field will accept. It can be one of three values: • 0 -- Alphabetic • 1 -- Numeric • 2 -- Date Note that there is no monetary type here. Monetary fields are treated as numeric fields.
10	If a field has an input conversion, it will be stored in this value
11	This field has up to three subvalues, as follows: • SV 1: If the **Allow Amend** flag on the field definition is a "Y", this value will be "1". If the Allow Amend flag is "N", this field will be "0". • SV 2: On a controlling multivalue, if there is a **process to be called when a value is deleted**, it will be defined here. • SV 3: If a **derived value field** is the controlling multivalue, this field will tell SB+ the attribute number of a field in @RECORD which determines the number of values to be displayed
12	If there is a **default expression** for this field, it will be stored here. Note that when BASIC code is generated for an input process, this will always be blank.
13	If this field has **validation codes** defined, they will be stored here. Note that when BASIC code is generated for an input process, this will always be blank
14	If this field is a required entry (a.k.a. mandatory) this value will be "1". Otherwise, if the entry is optional, this value will be "0". Note that this value is based on the "Mandatory (Y/N)" setting on the F5-Field screen and is not related in any way to any validation that may apply for the field.

15	If there are recalculated fields based on this field, this value keeps a subvalued list of the step numbers of the fields to be recalculated when the value in this field changes
16	On a controlling multivalue, this field tracks the attribute numbers of the fields that are dependent on this controlling field. SB+ uses this to determine which attributes in @RECORD to update when a row of a multivalued set is deleted or a new row is inserted. For dependent multivalues, this value tracks the step number of the controlling multivalue field.
17	This value contains both the **F1 help** message and the **intuitive help** process, if any, in the form: help message[intuitive help Note that the "[" is an open bracket, not a control character
18	This value contains the name of the field as it is defined in the **field definition**.

MAINFILE - In an input, output, report, or update process, this variable keeps track of the name of the main file being used.

F.FILE - This variable is the file buffer that MAINFILE was opened to. In BASIC, you can read, write, delete, and release using this variable without having to open MAINFILE to some other file buffer variable.

ORIG.REC - In an input process, immediately after RECORD has been read it is copied to this variable. This allows a developer to compare the original record to the current record at file time to determine any differences.

LOCK.KEY - The key value at the time the record was read and locked. Perhaps it could have been better named ORIG.KEY? (Everyone's a critic...)

HEAD - This variable contains the current screen heading with embedded video characteristics.

SCR.IMAGE - This variable has the screen image for the current input or output screen, complete with embedded cursor positioning, video attributes, etc.

STATUS.LINE - This variable contains the current status line display string.

BOX.CORDS - This variable contains the current window box coordinates, in the form:

column]row]length]depth

Note that the brackets between values are actually value marks (ASCII character 253).

OUTVAL(36) - This variable contains the data values of each field in screen (with embedded cursor positioning and video attributes).

Section 2

VALUE - This is probably the most dynamic variable in the common map. In an input process, it contains each default, each value entered, and is also used in the calculation of each conversion and derived value. In paragraphs, it can be used for passing

parameters, calculating values, or pretty much anything else you can dream about.

RTN.FLAG - This variable is used to tell SB+ to do something, like jump to a different field on the screen, go to the update step, reject an entry in validation, reject the updating of the record, etc.

PARAM - This dynamic array is used to pass parameters into processes. In a paragraph:

```
EXEC 'process,parameter'
or
CALL subroutine,parameter
```

PARAM contains everything following the comma. Any number of parameters may be passed this way, but the called process or subroutine is responsible for parsing the individual parameters from the single variable.

REFRESH - This variable is set by the developer to tell SB+ to refresh the screen when it otherwise wouldn't. The valid values for this variable include:

1	Refresh current window/screen
2	Refresh all data values only in current screen
-2	Recalculate values and window sizes and redisplay data values in current screen
3	Refresh everything (all level)
4	Clear the F1-Help area of the current screen
7	Refresh current field (used primarily in C: validations)

Of these values, 2 (refresh all), -2 (recalculate and refresh all), and 7 (refresh single field) are the most commonly used options.

MENU.OPT - This field contains a number which defines the most recent menu option selected. In menus with Post Action = 3 or 4, this variable contains the name of the process that was selected.

PROC.NAME - This variable is a LIFO attribute-delimited stack of the processes that have been called. As one process calls another, the new process being called is pushed onto the beginning of the stack, and is popped off when that process terminates.

LEVEL.NO - As the section 1 variables are pushed (when one input process calls another) this variable is incremented. When the called process terminates and returns to its caller, this value is decremented. SB+ uses this variable to track when to display the Max Levels warning.

FILES.OPENED - As files are opened using OPEN in paragraphs, this variable tracks the names of open files. For example, if the following paragraph were executed:

```
OPEN 'CUSTOMERS' TO 20
```

...attribute 20 of this variable will hold the word CUSTOMERS. Up to 30 attributes can be in this variable, one for each position of FILEVAR.

FILEVAR(...) - As files are opened using OPEN in paragraphs, this variable holds the open file buffers. For example, if the above

paragraph were executed, FILEVAR(20) would hold the open file buffer for the CUSTOMERS file, assuming, of course, that this file could be opened.

PARMS(...) - This 40-position dimensioned array is a work area for developers. SB+ never touches this array, so you don't have to worry about it getting inadvertently changed by SB+ (as you do with OTHER.REC). However, you must be very careful to avoid PARMS(...) conflicts, where one application uses this array one way, and it calls another application which uses the array in a different way.

SBPARM(...) - This 20-position array is used by SB+ to pass information where required. It should never be changed by developers, but there are a couple of places that you can inquire on:

- SBPARM(6) -- SB+ Report Writer processes use this location to store the actual SELECT or SSELECT statement that was executed.
- SBPARM(18) -- In a menu, this location keeps track of the current menu name, the process that will be executed if the current option is selected, the window coordinates of the menu, and a couple other interesting tidbits of information regarding the menu.

OTHER(...) - This is another array that is used by SB+. Like SBPARM(...), there are a couple of places you can get some useful information:
Note that in version 2.x there is no open file buffer for the XXXDEFN file.

- OTHER(2) -- In a file update process, this variable holds the secondary record being updated.
- OTHER(3) -- When an input or output process is being executed, this location contains the name of the main driver record.
- OTHER(4) -- The ID of the current transaction (for Transaction processes)
- OTHER(5) -- Batch key/number (in transaction entry)
- OTHER(6) -- Batch record (in transaction entry)
- OTHER(8) -- Report variables (Use "/GENHELP", select "Technical Reference", then "Report Writer Variables" for a comprehensive list of all of the report variables stored in @OTHER(8).)
- OTHER(9) -- In a report, this variable contains break totals.
- OTHER(10) -- In a report, this variable contains grand totals
- OTHER(18) -- Last edit key number pressed (0 = <enter>)

Section 3

SB.CONT - SB+ control record. See the SB+ documentation for a more detailed description of the information stored in this variable.

CONTROL - This variable holds the current system control record (XXXCONTROL PARAMS). Consult the SB+ documentation for a more detailed description of the information stored in this variable. This record is updated using the process /HK. CONTROL.

PORT - The current port number, set at login.

SYSID - The current system ID, set at login.

ACNT.NAME - The current account name, set at login.

TERM.DEFN - The current terminal definition record, loaded at login when the terminal type is known. Consult the SB+

documentation for a more detailed description of the information stored in this variable.

PRINT.DEFN - The current printer definition record, loaded at login when the printer type is known. Consult the SB+ documentation for a more detailed description of the information stored in this variable.

PASS.DEFN - This variable contains security information for the current user, including the user's group ID, process and menu restrictions, and other group security information. Consult the SB+ documentation for a more detailed description of the information stored in this variable.

USER.ID - This variable contains the ID of the current user, as identified by logon. This may also be changed by the standard process VU (Verify User).

USER.KEYS - This variable contains user macro key strings, as defined by either User Security or the UK (User Keys) process.

PCTERM - If "1", the current terminal emulation uses PC smart refresh ability as defined for SB Client. If "2", the current terminal emulation uses PC smart refresh ability as defined for SBTERMITE. If "0", the terminal is not considered a PC. This parameter is loaded from the terminal definition at login.

F.MD - File variable that MD opened on, set at login.

F.DMCONT - File variable for SB+ DMCONT file, opened at login.

F.DMDRIV - File variable for SB+ DMDRIVERS file, opened at login.

F.PASS - File variable for SB+ DMSECURITY file, opened at login.

F.CONT - File variable for system control file (XXXCONTROL), opened at login.

F.PROC - File variable for system process file (XXXPROCESS), opened at login.

F.MENU - File variable for system menu file (XXXMENUS), opened at login.

F.DMPROC - File variable for SB+ DMPROCESS file, opened at login.

RES3(11) Reserved for future use

Version 3.x and Higher Common Map

SB+ version 3.x and higher includes GUI features for making your applications look more like regular Windows applications. For this reason, and a few others, the common map changed a little. The following section describes these changes.

(To avoid being redundant, this section will only detail the changes between the 2.x and 3.x common map. Variables that are the same between the two configurations will not be listed here.)

Section 1 - Variables Added

LF.INFO - This variable contains a logical file definition when a screen is running from a logical file. Logical files are new features of the 3.x release which assist the developer in creating applications which reference and update multiple files.

OUTVAL.FLAGS - This variable contains a variety of information stored in conjunction with OUTVAL for the display of information on a screen.

F.INDX - When the file as named by @MAINFILE has an index definition, this variable contains the open file buffer for the index file when the screen is running.

F.INDX.DEFN - When the file as named by @MAINFILE has an index definition, this variable contains the index definition for the file when the screen is running.

GUI - This variable contains GUI information for the current terminal definition.

GUIRES - This area is reserved for use by SB+.

USERDATA(...) - This variable is a 10-position dimensioned array which can be used for any purpose that the developer deems appropriate. This variable is similar to @PARMS(...) in that SB+ never changes the value of any of the array elements. However, unlike @PARMS(...), SB+ will save and restore @USERDATA as processes call one another and return.

FREE.TEMP(...) - This variable is reserved for use by SB+.

Section 1 - Variables Changed

OUTVAL - In the 2.x release of SB+, this variable was a 36 position dimensioned array. In the 3.x release, this variable has been changed to a dynamic array, and the OUTVAL.FLAGS variable has been added to store additional related information. This was done to remove the 36 field limit on screens which exists in the 2.x release.
Section 2 Changes

There have been no section 2 changes to the common map between SB+ release 2.x and 3.x.

Section 3 - Variables Added

F.DEFN - This variable contains the open file buffer for the current XXXDEFN file.

F.G.PROC - This variable contains the global process file variable. In order for this to be used, a global process file must be defined.

F.G.MENU - This variable contains the global menu file variable. In order for this to be used, a global menu file must be defined.

F.G.DEFN - This variable contains the global definition file variable. In order for this to be used, a global XXXDEFN file must be defined.

F.DMSYSDEFN - This variable contains the file buffer for the DMSYSDEFN file, a new SB+ file added in the 3.x release. (Terminal and printer definition records (and others), which used to be stored in DMSECURITY, are now in DMSYSDEFN.)

BT.NODE(...) - This 7 position array contains node details for a given level in the current index.

BT.LEV.NO - This variable contains the current level number of the index being referenced.

BT.ID - This variable contains the list of IDs for a given node of an index.

BT.POS - This variable contains a positional reference within a node of an index.

GUIDATA - This variable contains default settings for GUI terminals when in GUI mode.

RES1 - This variable is reserved for use by SB+.

Expression Basics

Literals

The most fundamental form of expression is a literal, which can be either a number or alphabetic string. If alphabetic string, the literal must be enclosed in either single or double quotes (as long as the beginning and ending quote match). For example, the following are legal literals:

```
56
'Hello, World'
"SB+"
```

Field Names

In place of a literal, a field name may be referenced. If we have a CUSTOMER file with fields named NAME, ADDRESS, and CITY, we can reference any of these fields by name, such as:

```
NAME
ADDRESS
CITY
```

Of course, the case of the field name must match the case of the field definition name. Therefore, if the above are valid field names, the following are invalid references to the fields:

```
Name
Address
City
```

Any time a multivalued field is referenced by name, the value as pointed to by the **common variable** @CNT is used. To reference the multivalued field as a whole you must either set @CNT to 0 (which can be very dangerous), or by using the POS(...) function, as follows:

```
@RECORD<POS(NAME)>
```

Common Variables

Another fundamental component of the expression language is references to common variables. Each of the common variables are referenced with a preceding "@" sign, which makes common variables visually distinct from other types of variables. For example, the following are valid common variable references:

```
@RECORD
@KEY
@ORIG.REC
@ACTION
```

Shortcuts for Common Variables

Any attribute in @WORK can be referenced by a shortcut in the form Wn,m (where n is an attribute number, and m is an optional value reference). Therefore, to reference @WORK<5>, W5 could be used. To reference value 2 of attribute 4 of @WORK, W4,2 could be used.

The same is true for @OTHER.REC, except the shortcut letter is "O" (not zero) instead of "W". To reference @OTHER.REC <2>, O2 could be used. To reference the third value of attribute 7, O7,3 could be used.

Lastly, when referencing specific attributes of @RECORD, the variable name can be omitted. Therefore, attribute 5 of @RECORD can be referenced as:

@RECORD<5>

...or more simply:

<5>

Similarly, value 2 of attribute 5 can be referenced as <5,2>, instead of @RECORD<5,2>.

Next Generated Number

Another fundamental component of the expression language is the "G" command which assigns next sequential numbers. The syntax of this expression element is as follows:

Gn,m,sysid

In this syntax, the "n" parameter determines the accumulator to generate the number from. The "m" parameter is an optional length parameter, and if supplied it tells SB+ to right justify the assigned number in a field of zeros, "m" characters in length.

If the accumulator is to be read from another system record, the other system ID can be supplied in the "sysid" parameter.

System Variables

There are a few variables that are referenced the same way as common variables (i.e. they have a preceding "@" character), but aren't really variables at all. These include:

- @AM or @FM - The attribute mark character
- @VM - The value mark character
- @DATE - The current date (input converted format)
- @TIME - The current time (input converted format)

Attribute and Value References

A literal or common variable may be followed by an attribute and/or value reference, in the form:

<attribute{,value}>

For example, to reference attribute 5 of @OTHER.REC, the following could be used:

@OTHER.REC<5>

To reference value 2 of attribute 4 of @RECORD, the following could be used:

@RECORD<4,2>

If the value reference is included, but is zero, it is assumed that the value reference has been omitted. Therefore, the following are equivalent:

@PARMS(2)<4,0>
@PARMS(2)<4>

Substring Extraction

Following a common variable, literal, or field name (and the optional attribute and value reference), substring extraction may be defined, in the form:

[starting character,length]

In this syntax, the first parameter defines the first character to be extracted and the second parameter defines the number of characters to be extracted. For example, to extract the second character of the third value of attribute 5 of @RECORD, the following could be used:

@RECORD<5,3>[2,1]

...or, using shortcuts the following is equivalent:

<5,3>[2,1]

Conversions

Following an expression, an input or output conversion may be specified (in double quotes). This allows you to do conversions on a field without using OCONV or ICONV. Honestly, these are carryovers from earlier versions of SB+ from the time when ICONV and OCONV were not supported. Still, they are a valid option in the expression language.

For example, to output-convert @VALUE using the "MR2" conversion, the following could be used:

@VALUE"MR2"

Or, to input-convert @VALUE using the same conversion:

@VALUE"MR2[I"

Note the "[I" on the end of the conversion. This is how SB+ determines whether to input-convert or output-convert the value being

converted. (The "[" is simply an open bracket, not a control character.)

Expression Operators

Simple expressions can be combined with a variety of different operators. These variables are divided into five levels of precedence, as follows:

Level 1 (highest precedence, strongest binding)

*	Multiplication
/	Division (with rounding)
|	Division (without rounding)

Level 2

+	Addition
-	Subtraction

Level 3

:	Concatenation

Level 4

=	Equal To
#	Not Equal To
<	Less Than
<=	Less Than or Equal To
>	Greater Than
>=	Greater Than or Equal To

Level 5

AND	Logical AND: If both operands are true, the result is true (generally "1"). If one or the other operand is not true (i.e. zero), the result is zero.
OR	Logical OR: If one operand or the other is true, the result is true. If both operands are zero, the result is zero.

(The lower the level, the tighter the binding between operators and operands.)

Based on this list of operators and their corresponding precedence levels, the following are valid expressions:

```
PRICE * 1.5
3 + 2 * 5
```

Precedence is important particularly in this last example. In this example, the correct result is 13, not 25. Of course, parentheses can always be used to change the way that a particular expression is processed, as in the following example:

```
(3 + 2) * 5
```

...in which case the result of 25 is correct.

When running interpretively, SB+ will "sense" multivalued fields, and apply the operators on a value-by-value level when adding, subtracting, or multiplying multivalued fields. In generated code, however, SB+ is unable to sense this. Therefore, in the 2.x release, always do everything at a single-value level. In the 3.x release, SB+ has added the MV(...) and SV(...) functions which can help you get the proper results.

Expression Functions

(In the following examples, the keyword "exprn" is used to signify any valid SB+ expression of any complexity.)

Mathematical Functions

ABS(exprn)	This function will return the absolute value of the expression enclosed in quotes. ABS(@VALUE) - If @VALUE = -3, the function will return 3.
INT(exprn)	This function will return the integer part of the expression enclosed in quotes. Any remainder is discarded. INT(@RECORD<5>) - If @RECORD<5> = 4.6, the function will return 4.
MATH(expression 1, operator, expression 2)	This function will call a subroutine called STRING.MATH as cataloged in the current account, passing the expressions and operator as parameters. For more information about how this is used, see the comments in the program named STRING.MATH in DMSKELCODE.
MOD(value, divisor)	This function will return the remainder of the first parameter divided by the second. MOD(PRICE,PCT) - If price is 10 and PCT is 3, the function will return 1.
NOT(exprn)	This function will return the logical opposite of the expression enclosed in quotes. For example, if the expression is true, NOT(exprn) will return false, and vice versa. IF NOT(PRESENT) THEN - If PRESENT is true, NOT(PRESENT) will return false. If PRESENT is false, NOT(PRESENT) will return true.
S(exprn)	This function will take a multivalued expression and will return the sum of the multivalues. If referencing a multivalued field name in the expression, use caution. Under certain circumstances, this function doesn't work properly because of the implied multivalue position that is inherent in referencing multivalued fields by name. However, if you reference a multivalued field using the POS(...) function, this problem can be avoided. @VALUE = S(AMTS) - If AMTS is 21]5]6, @VALUE will be assigned the value 32.

String Functions

AT(column, row)	This function will return the cursor positioning codes for placing the cursor at the column and row as listed in the parentheses. This is equivalent to the BASIC function @(column,row) PRINT AT(20,10) - Cursor will be moved to column 20, row 10 on the screen. Note that this is not applicable to a GUI screen.

CHAR(exprn)	This function will return the ASCII character for the number or expression in parentheses. CHAR(65) - The character "A"
COL1()	Following a FIELD(...) function, this function will return the character position of the character immediately preceding the substring returned by the FIELD(...) function. Note that there are no parameters to this function.
COL2()	Following a FIELD(...) function, this function will return the character position of the character immediately following the substring returned by the FIELD(...) function. Note that there are no parameters to this function.
DCOUNT(exprn, delimiter)	This function will return the number of values in the expression as delimited by the delimiter. NDX = DCOUNT(@VALUE,@VM) - If @VALUE = 21]5]6, the DCOUNT(...) function will return 3, because there are 3 values in the string.
DEL(exprn, attribute, value)	This function will return the expression with the attribute and/or value removed. If the value parameter is zero, an attribute will be removed. If the value parameter is greater than zero, a value will be removed. If the value parameter is less than zero, any amount of weirdness could occur, depending on your operating environment. XYZ = "21]6]5" @VALUE = DEL(XYZ,1,2) @VALUE will equal "21]5" after this code.
DUP(string1, string2)	This function will count the number of values in the second parameter, and will duplicate the first parameter to have as many values. For example, to add "5" to all of the values in a multivalued field called LIMIT, the following could be used: DUP(5,LIMIT) + LIMIT The DUP(...) function will return a multivalued list of 5's with as many values as there are values in LIMIT. The addition will then be done on a value-by-value level.
FIELD(exprn, delimiter, instance)	This function will return a substring of the first parameter using the second parameter as a delimiter. The third parameter determines which substring to extract. For example, if @VALUE contains: 24*13*2*12 ...the following will return the number "2": FIELD(@VALUE,'*',3)

INDEX(exprn to search, exprn to find, instance)	This function will search the first parameter for a specific instance of the second parameter, and will return the character position where the search string was found, or "0" if not found. For example, if @VALUE contains: 24*13*2*12 ...the following will return the value "9": INDEX(@VALUE,'1',2)
INS(exprn,attribute,value,value to insert)	This function will return the expression with the value to insert inserted at the attribute and value position. If the value position is "0", an attribute will be inserted. If the value position is greater than "0", a value will be inserted into the attribute before the position as named. The "value" parameter may also be one of the following alphabetic literals, enclosed in quotes: AR Insert the value into the string in ascending right justified order. DR Insert the value into the string in descending right justified order. AL Insert the value into the string in ascending left justified order. DL Insert the value into the string in descending left justified order. @VALUE = 21 : @VM : 3 @VALUE = INS(@VALUE,1,-1,"X") ;* @VALUE will equal 21]3]X @VALUE = 21 : @VM : 3 @VALUE = INS(@VALUE,1,1,"X") ;* @VALUE will equal X]21]3 @VALUE = "ABC" : @VM: "DEF" @VALUE = INS(@VALUE,1,"AL","BIG") ;* @VALUE will equal ABC]BIG]DEF
LEN(exprn)	This function will return the number of characters in the expression. @VALUE = 21 NDX = LEN(@VALUE) ;* NDX will equal 2
LOC(find string, search string, delimiter)	This function will search through the second parameter for the find string in the first parameter, based on the delimiter in the third parameter to separate values. An exact match must be found, otherwise this function will return zero. @VALUE = 21 : @VM : 4 : @VM : 5 NDX = LOC("4",@VALUE,@VM) ;* NDX will equal 2 (4 is in the second position in the multivalued list) @VALUE = '21,4,5' NDX = LOC(5,@VALUE,',') ;* NDX will equal 3 (5 is the third position in the comma-delimited list)
MATCHES(exprn, pattern)	This function will determine if the expression matches the pattern. If it does, the function returns "1". If not, the function returns "0". MATCHES(@VALUE,"3N1X") - Will return true (1) if @VALUE is three numeric digits followed by one character of any type.

SEQ(exprn)	This function will return the ASCII numeric code for the first character of the expression. For example: SEQ('A') ...will return the value 65.
REPL(find string, replacement string, search string)	This function will return the search string with the find string replaced by the replacement string. REPL("G","X","GEORGE") - Will return XEORXE.
TABLE(exprn)	This function will look up the value of @VALUE in the table as named by the expression, and will return the description of the code (or null if the code is not found). The expression may return either a single table name, or the name of a table file and table name separated by a comma, as follows: • TABLE("tablename") -Lookup @VALUE in the table named "tablename" in the current XXXDEFN file and return the description. • TABLE("file,tablename") - Lookup @VALUE in the table named "tablename" in the file as named and return the description.
TRIM(exprn)	This function will return the expression with extraneous spaces removed. Leading and trailing spaces are trimmed off, as are duplicate spaces between words. @VALUE = TRIM(" THIS IS PADDED ") ;* @VALUE will = "THIS IS PADDED"

Logical Functions

ALPHA(exprn)	If the expression in parentheses is strictly alphabetic, this function will return "1". Otherwise, this function returns "0". ALPHA(21) - Will return false (0). ALPHA("XYZ") - Will return true (1). ALPHA("X@") - Will return false. ALPHA("X2") - Will return false.
CASE(condition 1, value 1;condition 2, value 2;...)	This function will evaluate the first condition and if it is true, return the first value. If this first condition is false, the second condition is evaluated, etc., until a condition is true, or all conditions have been evaluated. It is very important to always include an exception case so the CASE(...) function will always return some value.
IF(condition, true value, false value)	This function will evaluate the condition in the first parameter. If the condition is true, the true value is returned. If the condition is false, the false value is returned. IF(@VALUE=4,21,6) - Will return 21 if @VALUE is 4. Otherwise, this will return 6.

NUM(expression)	If the expression in parentheses is strictly numeric, this function will return "1". Otherwise, this function returns "0". Note that this function typically sees a preceding minus and the first period as numeric characters. A minus anywhere else in the string, or more than one period will cause this test to return "0". (This can vary depending on your operating environment.)

I/O Functions

Truth be told, there is only one I/O function. However, there are enough variations on this function to make it seem like several different functions. The following illustrates these variations:

F(file, record ID)<attribute>	This function will perform a READV of the file, record, and attribute as listed and will return the value as read. F("CUSTOMERS",@VALUE)<4> - Will use @VALUE as a key to a record in the CUSTOMERS file, and will return attribute 4 of that record. The record is discarded after the value has been extracted.
F(file, record ID)	This function will read the record from the file as listed into @OTHER.REC and will return the entire record to the expression. F("CUSTOMERS",@VALUE)- Will use @VALUE as a key to a record in the CUSTOMERS file, and will return the entire record to the expression, leaving the record in @OTHER.REC.
(F(file, record ID))<attribute>	This variation on the F(...) function is a combination of the previous two forms with all of the benefit of both. When this syntax is used, the entire record is read into @OTHER.REC, however, only the attribute as listed is returned to the expression. (F("CUSTOMERS",@VALUE))<4> - Will use @VALUE as a key to a record in the CUSTOMERS file, and will read the entire record into @OTHER.REC. Then, attribute 4 is extracted and returned. @OTHER.REC remains intact with the record after the expression has completed.

Miscellaneous Functions

B(exprn)	This function will call a subroutine as named by the expression, returning @VALUE back to the expression. If the expression has a comma in it, everything up to the comma is the subroutine name and everything following the comma will be moved into the **common variable** @PARAM for passing the parameters to the subroutine. @PARMS(2) = B("SUBNAME") - Will call a subroutine named "SUBNAME". The value returned from the subroutine (in @VALUE) will then be copied to @PARMS(2).
DECRYPT(exprn)	This function works with the ENCRYPT(...) function. An expression encrypted using ENCRYPT(...) can be decrypted and returned to its normal state using this function. In this syntax, exprn is the expression to be decrypted. The decrypted value will be returned to the left side of the equation, as in: @RECORD<1> = DECRYPT(@RECORD<1>)

ENCRYPT(exprn)	This function works with the DECRYPT(...) function. An expression is passed as the parameter, and the result is the value in an encrypted format. This is a great feature for storing values (like credit card information) in a format that cannot be easily viewed using the editor or other unprotected system tools. @RECORD<1> = ENCRYPT(@RECORD<1>)
ICONV(exprn, conversion)	This function will perform an input conversion on the expression using the conversion as listed. For example, this function can be used to convert a date from human-readable format into its internal form, such as: ICONV("12.31.96","D")
MV(exprn)	This function does not calculate any value. Instead, it tells SB+ that the expression should be treated as a multivalued expression, and each individual multivalue should be extracted and handled separately. For example, if the following lines are in a paragraph: W4 = 1 : @VM : 2 W5 = 3 : @VM : 4 @VALUE = MV(W4 + W5) ...the result stored in @VALUE will be 4]6, because instead of adding the complete W4 to the complete W5, the MV(...) function told SB+ to handle and calculate the addition for each multivalue separately. This function is only required when SB+ could otherwise not figure out whether a field was multivalued -- such as a work field referenced by W*n*, a @PARMS(...) or @USERDATA(...) element, or a **local variable**. When referencing fields by name (based on field definitions), SB+ has enough information to know whether the field is multivalued or single valued. When running paragraphs interpretively, SB+ can usually figure out whether an expression element is a single or multivalued expression, and acts accordingly. However, when code is generated with /GC, SB+ occasionally needs you to use this function (or the SV(...) function) to explicitly declare your intent.
OCONV(exprn, conversion)	This function will perform an output conversion on the expression using the conversion as listed. For example, this function can be used to convert a date from internal form to a human-readable format, such as: OCONV(10000,'D2') ...will return the value "18 MAY 95".
P(process name)	This function will call a process as named by the expression, returning @VALUE back to the expression. If the expression has a comma in it, everything up to the comma is the process name and everything following the comma will be moved into the common variable @PARAM for passing the parameters to the process. @PARMS(2) = P('PROCNAME') - Will call a process named "PROCNAME". The value returned from the process (in @VALUE) will then be copied to @PARMS(2).

POS(field name)	This function will return the field position (attribute number) of the field as named in the parameter. This is not a real-time function. When the expression is evaluated (which is usually when the derived value, conversion, or default expression is entered, or the paragraph is saved) the POS(...) function returns a value and this value is placed into the expression. In other words, if the field position changes after it has been referenced in a POS(...) function, the expression will still look at the old position until it is regenerated. This can be done either by pressing <cr> through and resaving the field definition, or by using the Regenerate Expressions tool in SB+.
SV(exprn)	This function does not calculate any value. Instead, it tells SB+ that the expression should be treated as a single value expression, and each individual multivalue (if present) should NOT be extracted and handled separately. For example, if the following lines are in a paragraph: W4 = 1 : @VM : 2 W5 = 3 : @VM : 4 @VALUE = SV(W4 + W5) When the paragraph is running interpretively, the result stored in @VALUE will be 4]6 (same as the MV(...) function would return). When code is generated for the paragraph, the generated code will produce a non-numeric error message, because it's trying to add two values which are not purely numeric. Note that like the MV(...) function, this function is only required when SB+ could otherwise not figure out whether a field was single valued -- such as a work field referenced by W*n*, a @PARMS(...) or @USERDATA(...) element, or a local variable. When referencing fields by name (based on field definitions), SB+ has enough information to know whether the field is multivalued or single valued. When running paragraphs interpretively, SB+ can usually figure out whether an expression element is a single or multivalued expression, and acts accordingly. However, when code is generated with /GC, SB+ occasionally needs you to use this function (or the MV(...) function) to explicitly declare your intent.

* - Comment Lines

Any line that starts with an asterisk is a comment. The text following the asterisk is ignored. This is useful for documenting what's going on in your paragraph.

Example:

```
*
* This is a comment line.
* Use it to document your code.
*
```

Please use comments often to document your paragraphs so others who have to maintain your work will think you're a genius. If they can't figure out what you were thinking, how would they know? Remember, the way you write code today will determine the choice of words used to describe you six months later!

n - Line number

Any line starting with a number is a normal paragraph line, but the number may be used as a reference for a GOTO statement in the paragraph.

Example:

```
10 * Process this information
*
...
*
GOTO 10
```

Under normal circumstances, **GOTO** statements can be avoided, and therefore line numbers are unnecessary. However, should you truly *need* to use a GOTO, it's nice to be able to assign a line number to any line. Note that line numbers do not need to be assigned to every line, only to the lines that are the target of a GOTO.

ASSIGN - Assign a Value to a Variable

This statement is optional, and is used for assigning a value to a variable. The format of the command is:

ASSIGN *variable* = *exprn*

...where *variable* is a variable to be assigned, and *exprn* is a standard SB+ **expression**. In this syntax, the variable can be anything that can be assigned, such as a field name, **common variable**, or **local variable**. The expression can be of any complexity.

For example:

```
ASSIGN @PARMS(2) = "XYZ"
ASSIGN @PARMS(35)<2,5> = F("CUSTOMERS",@VALUE)<POS(NAME)>
```

...or the more common form, without the ASSIGN verb:

```
@PARMS(2) = "XYZ"
@PARMS(35)<2,5> = F("CUSTOMERS",@VALUE)<POS(NAME)>
```

CALL - Call a BASIC Subroutine

This statement is used to call a BASIC subroutine from the paragraph. The subroutine name follows the CALL, and a list of parameters can optionally follow the subroutine name, in one of two formats as follows:

Passing Parameters Using @PARAM

This format of the CALL statement is compatible across all versions of SB+, and appears as follows:

CALL *subroutine,parameter(s)*

Anything following the comma following the subroutine name will be passed into the subroutine in the **common variable** @PARAM. While there is only one @PARAM variable, you can pass any number of parameters into the subroutine. However, once inside the called process, you must parse @PARAM into separate variables, similar to the following paragraph lines:

```
COUNTRY = FIELD(@PARAM,',',1)
STATE   = FIELD(@PARAM,',',2)
```

Note that parameters passed using this syntax must be fixed literals -- this format does not allow you to pass the contents of variables as parameters.

Passing Explicit Parameters

This format of the CALL statement is compatible with SB+ version 3.x and higher, and appears as follows:

CALL *subroutine*(*p1*,*p2*,...,*p9*)

In this syntax, up to 9 parameters can be explicitly declared to be passed into the subroutine. Unlike the @PARAM method of passing parameters, with this syntax each parameter can be any expression of any complexity. Unlike BASIC to BASIC calls, however, matrix variables (MAT ARRAY) cannot be passed.

CASE - Evaluate Mutually Exclusive Condition(s)

The CASE statement, combined with BEGIN CASE and END CASE, are used to construct a CASE block in a paragraph. This construct can be used to evaluate a series of mutually exclusive conditions, executing a series of statements for whichever condition is true.

The case construct appears in a paragraph as follows:

```
BEGIN CASE
  CASE condition
    statements
    ...
  CASE condition
    statements
    ...
END CASE
```

There are several issues of note with this construct. First, the BEGIN CASE statement is optional, and is only required if one case block is nested inside of another case block. The END CASE, however, is required. Second, the condition can be any valid expression. If the condition results in a non-zero (i.e. true) value, the statements in that case will be executed.

Only one condition in the case block can be true. Once a true condition has been found, and the statements have been executed, the remaining cases are skipped.

For example:

```
CASE @VALUE = 1
  LIMIT = 4000
CASE @VALUE = 2
  LIMIT = 2000
CASE 1
  LIMIT = 0
END CASE
```

Note the CASE 1 at the end of this block. CASE 1 is the "everything else" case. If all other cases are false, the CASE 1 will occur. This is generally a good practice to follow to ensure that the CASE does *something* under all circumstances.

CRT - Display a Message

This statement is valid in release 3.x and following, and is used to output a value to the screen only. Regardless of whether a printer file is open (using PRINTER ON in **BASIC** or a Report Writer report spooling to the printer), this statement will output a value to the screen.

CRT exprn

Any valid SB+ expression can follow the CRT statement, such as:

CRT AT(5,10):@VALUE

Typically, outputting a value to a specific screen location like this is unnecessary. By placing fields at the appropriate places on the screen or report, there is rarely a need for this type of functionality. However, for debugging and that once-in-a-while necessity, it's nice to have the feature available.

DATA - Data Stack an Expression

This statement will "stack" the expression to be fed into the next input prompt or prompts. For example, if the CUST.ID in the current record is the key to the CUSTOMERS file, you can force the customer ID into the first prompt on the customer entry screen with the following paragraph code:

```
DATA CUST.ID
EXEC "I*CUSTOMERS*ENTRY.A"
```

With this code in place, the customer entry screen will appear, the key will be preloaded (from the *data stack*), and the record will automatically appear. Users love this kind of functionality as it allows them to fluidly move between programs without having to re-enter information in each.

To stack multiple items, the following could be used:

```
DATA "XYZ", "ABC"
```

This will feed the values XYZ and ABC into the next two input prompts. Note that these input prompts can be any inputs in SB+ -- a prompt, a paragraph INPUT, or perhaps even keyboard input while a report is running to the screen.

Occasionally, you may have the need to data stack a function key. This is easily done. Simply add 30 to the function key number (F2=32, F5=35, etc.) and data stack as follows:

```
DATA "@35"
```

...to stack function key F5!

DELETE - Delete a Record

This statement will delete a record from a file. The syntax is as follows:

DELETE *filename,key*

In this syntax, *filename* is the quoted name of a file, an expression, or a file number (as referenced in an OPEN). The *key* parameter defines the key of the record to be deleted. Note that this statement will delete the record from the file without any confirmation or fanfare, and once a record has been deleted it is truly gone!

If the current open file is called CUSTOMERS, the following are equivalent:

```
DELETE "CUSTOMERS",@KEY
DELETE @MAINFILE,@KEY
DELETE 0,@KEY
```

Note this last option: The number zero can be used in place of the current open file name or @MAINFILE. This is true in all I/O functions.

DELETEI - Delete a Record and Update Indices

This syntax was introduced in the 3.x release and will delete a record from a file and update all indices for the file. The syntax of this statement appears as follows:

DELETEI *filename,key*

In this syntax, *filename* is the quoted name of a file, an expression, or a file number (as referenced in an OPEN). The *key* parameter defines the key of the record to be deleted. Note that this statement will delete the record from the file without any confirmation or fanfare, and once a record has been deleted it is truly gone!

If the current open file is called CUSTOMERS, the following are equivalent:

```
DELETEI "CUSTOMERS",@KEY
DELETEI @MAINFILE,@KEY
DELETEI 0,@KEY
```

Note this last option: The number zero can be used in place of the current open file name or @MAINFILE. This is true in all I/O functions.

DISP - Display a Message

This statement will display a message on the screen. It is followed by two parameters (separated by a comma) which define the display type and the actual message itself, as follows:

DISP *type*,*message*

The *type* parameter may be several things, but the more valuable options include:

3	Beep and display *message* centered on the bottom line of the screen. The user must press a key to make this type of message go away. DISP 3,"This is a message"
4	Display *message* in a dialog box and set @RTN.FLAG to a value based on an option selected by the user. When this option is used, the message may be expressed in two different ways: If message is a single word (no spaces), SB+ will look in the *XXX*DEFN file (where *XXX* is the current system ID) for a dialog box named accordingly. For example: DISP 4,"DBOX" ...will display a dialog box from the *XXX*DEFN file named "DBOX". If no dialog box is found, an error message will be displayed telling you the dialog box cannot be found. This is a particularly difficult message to diagnose, as the user is expecting one message and a completely different message is displayed. If the message is not a single word, SB+ will expect the message to be formatted as follows: message\buttons\flags In this syntax, buttons is a comma-delimited list of words that SB+ will build as buttons on the bottom of the dialog box. For each button, flags represents the value to be assigned to @RTN.FLAG when that button is selected. For example, to have a box which displays "Are you sure?" with "Yes" and "No" buttons (where "Yes" returns "0" and "No" returns 1), the following can be used: DISP 4,"Are you sure?\Yes,No\0,1" Like a type 3 message, the user must press a key to make this type of message go away. However, the escape key is not enabled. In other words, the user must select something from the dialog box in order to continue.
8	Display a processing message. This type of message will print a graphic line near the bottom of the screen with the message shown below it. This type of message expects no user response. DISP 8,"Processing Checks"
9	Display an informational message. This type of message will appear on the bottom line of the display. This type of message expects no user response. DISP 9,"Use the arrow keys to move around the screen"

EQU - Create Paragraph Equate

This statement is used to create an *equate*, or symbolic name for an expression. This is most commonly used to associate a name to a **common variable** to make a paragraph more readable, as in:

```
EQU HEADING TO @PARMS(20)
```

Once this has been defined, anywhere HEADING is referenced in the paragraph, @PARMS(20) will actually be referenced/ updated.

Equates are non-executing statements. Instead, they are provided as a way to make paragraphs more readable. Despite their intended use, equates can be used to make a paragraph either more or less readable, depending on the frequency and complexity of the equates. More equates tend to make the paragraph more difficult to follow, as it requires the reader to understand both the logic and the applicability of the equates to the logic. Fewer equates, strategically placed, can make a paragraph easier to read and understand.

Paragraphs can also use **global equates**, which are defined elsewhere in this chapter.

ERROR - Display a Message

This statement is equivalent to using **DISP** 3 to display a message, as follows:

ERROR *message*

In this syntax, *message* may be any valid SB+ expression, such as:

```
ERROR "This is not a valid entry"
ERROR @PARMS(2)<5>
ERROR @VALUE : " is not a valid customer"
```

In the first example, the error message is a literal string. In the second example, SB+ will get the error message from @PARMS (2) attribute 5. (This assumes, of course, that something has put a message in that location!) Finally, in the third example, the error message is a combination of the contents of @VALUE and a literal string.

SB+ error messages and user messages can also be displayed with the ERROR command, as shown in these examples:

```
ERROR @VALUE : " [E1]"
ERROR "[U26]"
ERROR "[UERRORS,21<1>]"
```

In this first example, @VALUE will be concatenated to the SB+ error message #1 ("NOT FOUND"). The second example will read user message number 26 and display it. Finally, the last example will read a record in the ERRORS file with a key of 26, and will display attribute 1 as the error.

EXEC - Execute Another Process

This statement is used to execute another process from the paragraph. Any expression can be used to nominate the process to be called. For example, to call a process using a literal for the process name, the following could be used:

```
EXEC "XYZ0021" ;* Execute a process named XYZ0021
```

If the process name is in a **common variable**, such as @PARMS(2) attribute 5, the following could also be used:

```
EXEC @PARMS(2)<5> ;* Execute the process named by @PARMS(2)<5>
```

Parameters may be passed (in the common variable @PARAM) when a process is called this way, using the format:

EXEC "*process,parameters*"

When parameters are passed this way, the parameters must be literal values -- not the result of an expression. When the results of an expression are to be passed as a parameter, the following could be used:

EXEC "*process,*":*exprn*

Finally, both the process and parameters can be the result of expressions:

EXEC *process_exprn*:',':*parameter_exprn*

Parameters passed to a process are always passed in @PARAM. If multiple parameters are passed, the developer is responsible for parsing them from @PARAM. If no parameters are passed, and there is no comma following the process name, @PARAM will be passed into the called process with whatever value it has at the time. If you want to ensure that a parameter is passed as a null, use the following:

EXEC "*process,*"

The trailing comma tells SB+ there is a parameter to follow, and that parameter is null.

EXIT - Exit the Paragraph and Set @RTN.FLAG

This statement is used to exit the paragraph and optionally set the **@RTN.FLAG** **common variable**. To simply leave the paragraph and leave @RTN.FLAG unchanged:

```
EXIT
```

To exit the paragraph and set @RTN.FLAG to a value, such as the literal "1":

```
EXIT 1
```

Note that the "1" is not quoted. This is a very important distinction -- you can only set @RTN.FLAG to a literal value this way. If you want to set @RTN.FLAG to the result of an expression, use:

```
@RTN.FLAG = exprn
EXIT
```

Return flag settings vary depending on the slot where the flag is being set. For example, to skip over a prompt in the Process Before field slot, @RTN.FLAG can be set to "S" for single valued fields and "S0" for multivalued fields. To skip backward over a prompt, @RTN.FLAG can be set to "1" in the Process Before field. To reject an entry in a validation process, @RTN.FLAG can also be set to a "1". There are a number of additional @RTN.FLAG settings -- simply be aware that the value of @RTN.FLAG is no more important than the slot where the flag is being set.

FOR/NEXT - Loop

This statement was introduced in version 3.x, and is used to loop a certain number of times. The syntax for this statement matches the comparable **BASIC** statement and appears as follows:

FOR *variable* = *startvalue* TO *endvalue*
  statements
  ...
NEXT *variable*

In this syntax, *variable* defines a variable which will be incremented for each iteration of the loop. The *startvalue* parameter defines the starting value of the loop. The *endvalue* parameter defines the ending iteration of the loop. The *statements* will be executed for each iteration of the loop.

Normally, the loop increments by one each time. However, if you want to change the increment, or step value, you can use the STEP modifier on the end of the FOR statement, as follows:

FOR *variable* = *startvalue* TO *endvalue* STEP *increment*
  statements
  ...
NEXT *variable*

This syntax can be used to loop from a higher value to a lower value, using a negative increment, as follows:

FOR *variable* = *startvalue* TO *endvalue* STEP -*increment*
  statements
  ...
NEXT *variable*

An optional WHILE or UNTIL clause can also be added to the end of the FOR statement, as follows:

FOR *variable* = *startvalue* TO *endvalue* WHILE *condition*
  statements
  ...
NEXT *variable*

...or...

FOR *variable* = *startvalue* to *endvalue* UNTIL *condition*
  statements
  ...
NEXT *variable*

If the WHILE clause is added, the loop will continue for either the number of iterations as defined, or as long as the condition following the WHILE is true. If the UNTIL clause is added, however, the loop will continue for either the number of iterations as defined, or until the condition following the UNTIL is true.

Both STEP and WHILE or STEP and UNTIL can be combined in one statement. UNTIL and WHILE, however, cannot be used together.

FOR/NEXT - Loop

GOTO - Jump to a Label

This statement is used to jump to a **label** in the paragraph. For example, if there is a label number 10, the following statement will jump to that label:

```
GOTO 10
```

This is commonly used to go back to a spot in a paragraph without having to formulate complex looping logic with FOR/NEXT for WHILE, as illustrated in this example:

```
10 @RTN.FLAG = 0
READ @OTHER.REC FROM 'CUSTOMERS',@VALUE
IF @RTN.FLAG = 1 THEN
  DISP 4,"Add New Customer?\Yes,No\0,1"
  IF @RTN.FLAG = 0 THEN
    EXEC 'I*CUSTOMERS*ENTRY.A'
    GOTO 10 ;* Check again -- they may have pressed Escape!
  END
END
```

IF..THEN..ELSE - Evaluate A Condition

This statement will evaluate a condition and execute some statements based on the truth of the condition. The following code illustrates a simple IF block:

IF *condition* THEN *statement*

In this syntax, if *condition* is true, the *statement* following the THEN will be executed. Sometimes, however, several statements need to be executed. For this situation, a slightly different syntax variation can be used:

```
IF condition THEN
  statement
  statement
  ...
END
```

With this variation, any number of statements can appear between the THEN and END.

Sometimes different statements need to be executed when a condition is true vs. when the condition is false. For this, we can add the ELSE clause to the IF block, as follows:

IF *condition* THEN *statement* ELSE *statement*

Or, if multiple statements are to be executed for the true or false condition:

```
IF condition THEN
  statement
  statement
  ...
END ELSE
  statement
  statement
  ...
END
```

In all of these examples, *condition* can be any expression which results in a true or false value. For example, the IF statement could be as simple as:

```
IF LIMIT > 400 THEN EXEC "XYZ0021"
```

...or as complex as something like this:

```
IF ((LIMIT > 400) AND (MATCHES(CUST.TYPE,"1N") OR MATCHES(CUST.TYPE,"1A"))) THEN EXEC "XYZ0021"
```

INPUT - Request User Input

This statement will display a prompt at the bottom of the screen and will request the user to enter a value. The syntax for this statement is as follows:

```
INPUT variable,length,prompt
```

In this syntax, *variable* is the name of a variable where the value as entered will be stored. Whatever value this variable has at the time of the input will be the default for the prompt. The *length* parameter defines the number of characters to be shown for input. It doesn't necessarily restrict the input to a certain number of characters -- it only restricts how much will be shown at any given time. Finally, the *prompt* parameter defines the prompt the user will see when the input is requested. It can be a simple message such as in:

```
INPUT @VALUE,10,'Enter a value'
```

Or, to add F1 help to the prompt, add a backslash followed by the F1 help message to the prompt text.

```
INPUT @VALUE,10,'Enter a value\Numbers only'
```

To give the prompt intuitive help, add a backslash-open parenthesis combination to the end of the prompt text, followed by the process to be called:

```
INPUT @VALUE,10,'Enter a value\[F3.CALC'
```

For both F1 and F3 help, a combination of these two can be used:

```
INPUT @VALUE,10,'Enter a value\Numbers only[F3.CALC'
```

Note that this statement allows no validation of the value entered. If you want the value to be validated, you'll have to code the validation yourself in the paragraph.

OPEN - Open a File

This statement is used to open a file to a numeric file buffer. The syntax for using this statement is as follows:

OPEN *file* TO number

In this syntax, *file* may be a quoted literal filename, or an expression, as in the following examples:

```
OPEN "XYZ" TO 1
OPEN @PARMS(2)<4> TO 29
```

The *number* parameter, then, defines the file number that this file is being opened to. This value may be from 0 to 30, though it is worth mentioning that the OPEN works differently depending on the number you use.

If you open a file to number 0, the file name is copied into @MAINFILE and the file buffer is opened to @F.FILE. On the other hand, if you open a file to number 1, 2, ...30, the file name will be stored in @FILES.OPENED<*n*> and the file buffer will be stored in @FILEVAR(*n*) (where *n* is the number the file was opened to.)

For most applications there is no need to open a file this way. Instead, refer to files by quoted name in the paragraph instead of by number.

In certain cases, opening a file using OPEN can cause some serious problems. To illustrate how this could happen, let's look at a few nuances of the OPEN statement.

- When OPEN is used in a paragraph, SB+ does not check to see if a particular file number is already open before opening the file you name. Therefore, if one process opens a file to number 5, and this process calls another process which opens a different file to number 5, when the called process returns back to the caller, file number 5 is not what the original process had intended. This can cause all sorts of weirdness.
- SB+ is constantly using this variable to open files for itself. Therefore, you could write code to open a particular file, then SB+ can come in behind you and change the actual file that is open in a particular position without your knowledge or consent. This, too, can result in all sorts of weirdness, not the least of which could include severe data corruption.

If a file cannot be opened with OPEN (i.e. it doesn't exist or the permissions won't allow the file to be opened), @RTN.FLAG will be set to "1". Be sure to account for this possibility when you use OPEN!

PRINT - Print a Message

This statement is one that works differently in version 2.x vs. later versions. In version 2.x, this statement is used to output a value to the screen, regardless of whether a printer file is open (using PRINTER ON in BASIC or from inside a running report). In version 3.x and following, PRINT has been changed to respect the status of the printer. In other words, if a paragraph is called from a report or BASIC subroutine after the printer has been enabled, using PRINT in the paragraph will output text to the report spooler file, instead of to the screen. The CRT statement, also added as of version 3.x, assumes the role of the 2.x PRINT functionality.

The PRINT statement is followed by one parameter, an expression, which can be a literal as follows:

```
PRINT "this is a line of text"
```

Or, if the information to print is in common somewhere (such as @PARMS(2)<5>):

```
PRINT @PARMS(2)<5>
```

Or, if the information to be displayed is a combination of the two:

```
PRINT "The value is " : @VALUE
```

READ - Read a Record

This statement is used to read a record from a file. It appears in the paragraph as follows:

READ *variable* FROM *file*,*key*

In this syntax, *variable* is the name of the variable where the record will be stored in memory. The *file* parameter is used to define the name or number of the file where the record is to be read from, and *key* is an SB+ expression that defines the key of the record to be read, as in the following examples:

```
READ @OTHER.REC FROM "XYZ",@VALUE
```

...or...

```
READ @OTHER.REC FROM 5,@VALUE
```

(Of course, the file "XYZ" must have previously been opened to file buffer 5 for this second syntax to work properly.)

The name of the file can also be an expression, as in:

```
READ @OTHER.REC FROM @PARMS(2)<5>,@VALUE
```

If there is no record with this key, *variable* will be set to null and @RTN.FLAG will be set to 1. If the file cannot be opened, @RTN.FLAG will be set to 1, though in version 2.x no error will be displayed. (This is a particularly difficult problem to diagnose!) On the other hand, if the record was found, @RTN.FLAG is left unchanged. Therefore, if you're going to be checking @RTN.FLAG after the READ, be sure to set it to zero *before* the read!

There are two generally accepted forms of resetting @RTN.FLAG. The first form resets @RTN.FLAG before the read, as follows:

```
@RTN.FLAG = 0
READ @OTHER.REC FROM "CUSTOMERS",@VALUE
```

This syntax allows you to check @RTN.FLAG after the READ and do something depending on whether the record is found or not. Sometimes, however, you simply won't care whether the record was found, and therefore the following syntax is an option:

```
READ @OTHER.REC FROM "CUSTOMERS",@VALUE
@RTN.FLAG = 0
```

This syntax reads the record and then immediately resets @RTN.FLAG, because hey, we don't care if the record is found or not, right?

READL - Read/Lock a Record With Exception

This is a variation of READU, but instead of waiting for a record to be locked, this statement will set @RTN.FLAG to -1 when the record is locked by someone else. Logic following a READL can then ascertain whether or not the read was successful by checking the value of @RTN.FLAG. The following values can be returned:

unchanged	The read is successful
1	The record was not found or the file cannot be opened
-1	The record is locked by someone else

The format of the READL is the same as READ, except for the obvious addition of the "L" to the "READ", as follows:

READL *variable* FROM *file,key*

In this syntax, *variable* is the name of the variable where the record will be stored in memory. The *file* parameter is used to define the name or number of the file where the record is to be read from, and *key* is an SB+ expression that defines the key of the record to be read, as in the following examples:

```
READL @OTHER.REC FROM "XYZ",@VALUE
```

...or...

```
READL @OTHER.REC FROM 5,@VALUE
```

(Of course, the file "XYZ" must have previously been opened to file buffer 5 for this second syntax to work properly.)

The name of the file can also be an expression, as in:

```
READL @OTHER.REC FROM @PARMS(2)<5>,@VALUE
```

If there is no record with this key, *variable* will be set to null and @RTN.FLAG will be set to 1 (and a record lock will be set). If the file cannot be opened, @RTN.FLAG will be set to 1, though in version 2.x no error will be displayed. (This is a particularly difficult problem to diagnose!) If the record is locked by someone else, @RTN.FLAG will be set to -1. Finally, if the record was found, @RTN.FLAG is left unchanged. Therefore, if you're going to be checking @RTN.FLAG after the READL, be sure to set it to zero *before* the read, as follows:

```
@RTN.FLAG = 0
READL @OTHER.REC FROM "CUSTOMERS",@VALUE
```

READNEXT - Read Key From Active Select List

This statement was added in version 3.x and performs the same function as READNEXT in **BASIC**. This statement appears as follows:

READNEXT variable

When this statement is executed, a value will be taken from the currently active select list and assigned to "variable". This value can then be used in later processing. When there are no more items in the list, @RTN.FLAG is set to "1". For example, the following paragraph will process through a list of selected items:

```
@RTN.FLAG = 0
READNEXT @VALUE
WHILE @RTN.FLAG = 0 DO
  statements
  ...
  READNEXT @VALUE
REPEAT
```

READU - Read Record With Lock

This is a variation of READ which sets a record lock when the record is read. If the record cannot be read, the paragraph will wait until the record is available, then it will read the record and set the lock. Logic following a READU can then ascertain whether or not the read was successful by checking the value of @RTN.FLAG. The following values can be returned:

unchanged	The read is successful
1	The record was not found or the file cannot be opened

Note that both settings of @RTN.FLAG will lock the record. If this isn't what you want, you'll need to use the RELEASE statement after the READU to release the record lock.

The format of the READU is the same as READ, except for the obvious addition of the "U" following the "READ".

READU *variable* FROM *file,key*

In this syntax, *variable* is the name of the variable where the record will be stored in memory. The *file* parameter is used to define the name or number of the file where the record is to be read from, and *key* is an SB+ expression that defines the key of the record to be read, as in the following examples:

```
READU @OTHER.REC FROM "XYZ",@VALUE
```

...or...

```
READU @OTHER.REC FROM 5,@VALUE
```

(Of course, the file "XYZ" must have previously been opened to file buffer 5 for this second syntax to work properly.)

The name of the file can also be an expression, as in:

```
READU @OTHER.REC FROM @PARMS(2)<5>,@VALUE
```

If there is no record with this key, *variable* will be set to null and @RTN.FLAG will be set to 1 (and a record lock will be set). If the file cannot be opened, @RTN.FLAG will be set to 1, though in version 2.x no error will be displayed. (This is a particularly difficult problem to diagnose!) Finally, if the record was found, @RTN.FLAG is left unchanged. Therefore, if you're going to be checking @RTN.FLAG after the READU, be sure to set it to zero *before* the read, as follows:

```
@RTN.FLAG = 0
READU @OTHER.REC FROM "CUSTOMERS",@VALUE
```

This syntax allows you to check @RTN.FLAG after the READ and do something depending on whether the record is found or not. Sometimes, however, you simply won't care whether the record was found, and therefore the following syntax is an option:

```
READU @OTHER.REC FROM "CUSTOMERS",@VALUE
@RTN.FLAG = 0
```

READV - Read an Attribute From a File

This statement was added in version 3.x, and is used to read one attribute from a record in a file. The syntax of the READV appears as follows:

READV *variable* FROM *file,key,attribute*

In this syntax, *variable* is the name of a common or **local variable** where the information will be stored after it has been read. The *file* parameter can be a quoted file name, expression, or file number, similar to the way the file is expressed with the READ statement. The *key* parameter, also like READ, can be any valid SB+ expression used to define the key to the record to be read. Unlike the READ, however, is the addition of the *attribute* parameter, which is used to define the number of the attribute to be extracted from the record.

If the READV cannot read the record, @RTN.FLAG will be returned as "1". If the READV successfully reads the record, @RTN.FLAG will remain unchanged. It is imperative, therefore, that @RTN.FLAG is cleared prior to the READV (or any forms of READ for that matter), as follows:

```
@RTN.FLAG = 0
READV @OTHER.REC FROM "CUSTOMERS",@VALUE,5
```

Prior to version 3.x, a READV could only be done using the F(*file,id*)<*amc*> expression function. This functionality remains in the product. Therefore, the following are roughly equivalent:

```
READV @VALUE FROM "CUSTOMERS",@KEY,5
...vs...
@VALUE = F("CUSTOMERS",@KEY)<5>
```

Note the phrase "roughly equivalent". While these will both return the value in attribute 5 of a customer record, they differ when the record is not found. Using READV, if the record is not found, @RTN.FLAG will be set to "1". With the F(...) function, if the record is not found @VALUE will be set to null. Note that null will be returned using this syntax both when the record is not found, and also when the attribute is truly null. There is no way to distinguish which has occurred using the F(...) syntax.

RELEASE - Release a Record Lock

This statement is used to release a record lock that has been set by either READL or READU. The syntax for this statement is as follows:

```
RELEASE file,key
```

In this syntax, the file parameter can be a quoted file name, expression, or file number, similar to the way the file is expressed with the READ statement. The key parameter, also like READ, can be any valid SB+ expression used to define the key to the record lock to be released. For example, if CUSTOMERS is the current open file, the following are all equivalent:

```
RELEASE "CUSTOMERS",@KEY
RELEASE @MAINFILE,@KEY
RELEASE 0,@KEY
```

A record lock needs to be released if a READU or READL is used and the record is not updated with a corresponding WRITE, WRITEI, DELETE, or DELETEI. A release is also required when a Process After Read is used on a screen to **reject the read** and reprompt for the key such as in the following paragraph:

```
IF @ACTION = 1 AND @KEY # "NEW" : @PORT THEN
  RELEASE 0,@KEY
  ERROR "Please use NEW to add new records"
  EXIT 1
END
```

SLEEP - Pause the Paragraph

This statement is used to pause the paragraph for a certain number of seconds or until a certain time of day. The syntax for this statement is as follows:

SLEEP *time*

In this syntax, *time* can be one of the following:

- *hh:mm* -- If this format is used, the paragraph will sleep until the hour and minute as defined by "hh" and "mm". For example, SLEEP 21:00 will sleep until 9:00pm. If the statement is executed at 9:00:01pm, the paragraph will sleep until 9:00pm the following day.
- *seconds* -- If this format is used, the paragraph will sleep the number of seconds as defined. For example, SLEEP 45 will sleep 45 seconds.
- *(exprn)* - When this format is used, SB+ will evaluate the expression in parentheses and will sleep either the number of seconds that the expression returns, or to the time that the expression returns. Of course, the result of the expression must be either a time or number of seconds in one of the formats as shown above. (This option applies to versions 3.x and later only.). For example, if @VALUE is "20", SLEEP (@VALUE+1) will sleep 21 seconds.

WHILE..DO/REPEAT - Loop

These statements, like FOR/NEXT construct, allow looping in a paragraph. Unlike the FOR/NEXT, this is the only looping construct available prior to SB+ version 3.x. The syntax of the WHILE loop is as follows:

```
WHILE condition DO
  statements
  ...
REPEAT
```

In this syntax, the *condition* is an **expression** that is evaluated each iteration of the loop. As long as the condition is true, the loop will continue, executing *statements* for each iteration. When the condition is no longer true, the loop will terminate and the paragraph will continue execution with the statement following the REPEAT.

For example, the following paragraph fragment sums the values in @PARMS(2)<1> and is completely unnecessary given the S(...) **function** in the **expression language**.

```
LOCAL NDX
*
@VALUE = 0
*
NDX = DCOUNT(@PARMS(2)<1>,@VM)
WHILE NDX DO
  @VALUE = @VALUE + @PARMS(2)<1,NDX>
  NDX = NDX - 1
REPEAT
```

Note that the loop counts from the back of the list to the front, instead of the other way around. This allows you to simplify coding for operations like this (because otherwise 2 locals would be required). This technique is also useful when manipulating a dynamic array, such as in the following example.

```
LOCAL NDX
*
NDX = DCOUNT(@VALUE,@VM)
WHILE NDX DO
  IF LOC(@VALUE<1,NDX>,@VALUE,@VM) # NDX THEN
    @VALUE = DEL(@VALUE,1,NDX)
  END
  NDX = NDX - 1
REPEAT
```

In this example, which removes duplicates from a multivalued @VALUE, the loop pointer NDX does not need to be reset as a value is removed from the array. Had we counted from front to back, the code would have been much more complicated.

WRITE - Write a Record

This statement is used to write the contents of a **common variable** or **local variable** to a file. The syntax of this statement is as follows:

WRITE *expression* ON *file*,*key*

In this syntax, the *expression* parameter defines the variable or **expression** to be written. The *file* parameter can be a quoted file name, expression, or file number, similar to the way the file is expressed with the READ statement. (The file does not have to be previously opened.) The *key* parameter, also like READ, can be any valid SB+ expression used to define the key to the record to be written.

If the current file is the CUSTOMERS file, the record is in @RECORD, and the key is in @KEY, the following are equivalent:

```
WRITE @RECORD ON 'CUSTOMERS',@KEY
WRITE @RECORD ON @MAINFILE,@KEY
WRITE @RECORD ON 0,@KEY
```

Note that file number zero can be used in place of the file name to reference the current open file. This is true for all I/O functions in the paragraph language.

Any expression can be written to any file using any expression to calculate the key. Also, typically @RECORD is written by the screen, and therefore does not need to be written in a paragraph as is shown above. More commonly, WRITE is used to update secondary files using either @OTHER.REC or a local variable, as follows:

```
READU @OTHER.REC FROM "CUSTOMERS",CUST.ID
@RTN.FLAG = 0
*
O1 = O1 + THIS.ORDER.TOTAL
*
WRITE @OTHER.REC ON "CUSTOMERS",CUST.ID
```

WRITEI - Write and Update Indices

This statement is an enhanced version of the WRITE statement (added in version 3.x) which tells SB+ to write a record and update any indices that may be defined for the file as named. The syntax for WRITEI is exactly the same as WRITE, except for the obvious "I" suffix on the statement, as follows:

WRITEI *expression* ON *file,key*

In this syntax, the *expression* parameter defines the variable or **expression** to be written. The *file* parameter can be a quoted file name, expression, or file number, similar to the way the file is expressed with the READ statement. The *key* parameter, also like READ, can be any valid SB+ expression used to define the key to the record to be written.

If the current file is the CUSTOMERS file, the record is in @RECORD, and the key is in @KEY, the following are equivalent:

```
WRITEI @RECORD ON 'CUSTOMERS',@KEY
WRITEI @RECORD ON @MAINFILE,@KEY
WRITEI @RECORD ON 0,@KEY
```

Note that file number zero can be used in place of the file name to reference the current open file. This is true for all I/O functions in the paragraph language.

Any expression can be written to any file using any expression to calculate the key. Also, typically @RECORD is written by the screen, and therefore does not need to be written in a paragraph as is shown above. More commonly, WRITE is used to update secondary files using either @OTHER.REC or a **local variable**, as follows:

```
READU @OTHER.REC FROM "CUSTOMERS",CUST.ID
@RTN.FLAG = 0
*
@OTHER.REC = INS(@OTHER.REC,2,-1,ORDER.ID)
*
WRITEI @OTHER.REC ON "CUSTOMERS",CUST.ID
```

In this example, the order ID will be added to the end of a multivalued list in attribute 2 of the customer record and any indices defined for the customer file will be updated.

WRITEV - Write Attribute

This statement was added in version 3.x and will write an expression or variable to a single attribute on a record in a file. The syntax for this statement is as follows:

WRITEV *exprn* ON *file,key,attribute*

Like the WRITE, the *exprn* parameter defines a variable or **expression** to be written. This expression can be any value, but should not be more than a single attribute. The *file* parameter, like WRITE, can be a quoted file name, expression, or file number (as opened using the OPEN statement) and defines the file where the record will be updated. The *key* parameter, like WRITE, can be any valid SB+ expression and defines the key to the record to be updated. The *attribute* parameter defines the specific attribute to be updated in the record.

For example, the following will write the contents of @VALUE<1> to the a record in the CUSTOMERS file:

```
WRITEV @VALUE<1> ON 'CUSTOMERS',@KEY
```

Or, if the customer file is the current open file, either of the following could be used:

```
WRITEV @VALUE<1> ON @MAINFILE,@KEY
WRITEV @VALUE<1> ON 0,@KEY
```

Note that file number zero can be used in place of the file name to reference the current open file. This is true for all I/O functions in the paragraph language.

Creating the BASIC Subroutine

To create an SB+ BASIC process, first create the subroutine. The subroutine must have the following characteristics:

- The first line of the subroutine must be a SUBROUTINE line, with no parameters.
- The subroutine must $INCLUDE DMSKELCODE COMMON
- The subroutine must RETURN to the caller. If it doesn't, your user will find themselves inadvertently at TCL -- at midnight -- and will be calling you for support.

With this in mind, a subroutine to do nothing would appear as follows:

```
SUBROUTINE XXXXXXX
                *
$INCLUDE DMSKELCODE COMMON
    *
RETURN
```

Of course, you would probably want to have some heading on the subroutine which identifies who the author is, the date the subroutine was created, a brief description of the subroutine, and perhaps modification history. For example, the following is a version of my standard header.

```
SUBROUTINE XXXXXXX
*
* Written By:  (your name)
* Date:       (the current date)
* Description: (a description of the process, perhaps
*              spanning multiple lines if necessary. If
*              space and time allow, it is always best to
*              define all of the inputs and outputs of a
*               subroutine, and the context that the
*              subroutine will be used in.)
*
************************************************************
* Modification History
************************************************************
* Date..... Changed By..... Description of Change.....
*
************************************************************
* Include(s)
************************************************************
                *
$INCLUDE DMSKELCODE COMMON
*
* (Do something useful in this section)
    *
RETURN
```

Between the $INCLUDE line and the RETURN, you can do anything you need. Keep in mind, however, that the rules of common still apply. For example, if your BASIC routine is for validation, you'll take input from VALUE and return RTN.FLAG to tell the caller whether the input is valid or not. If the BASIC routine is called from a Process Before Field and is intended to skip the field under certain conditions, you'll need to take input from wherever you need and set RTN.FLAG on the return.

Finally, be generous in your comments. Inevitably someone else will be reading your code at some time in the future, and the readability of your code will determine their choice of adjectives preceding your name.

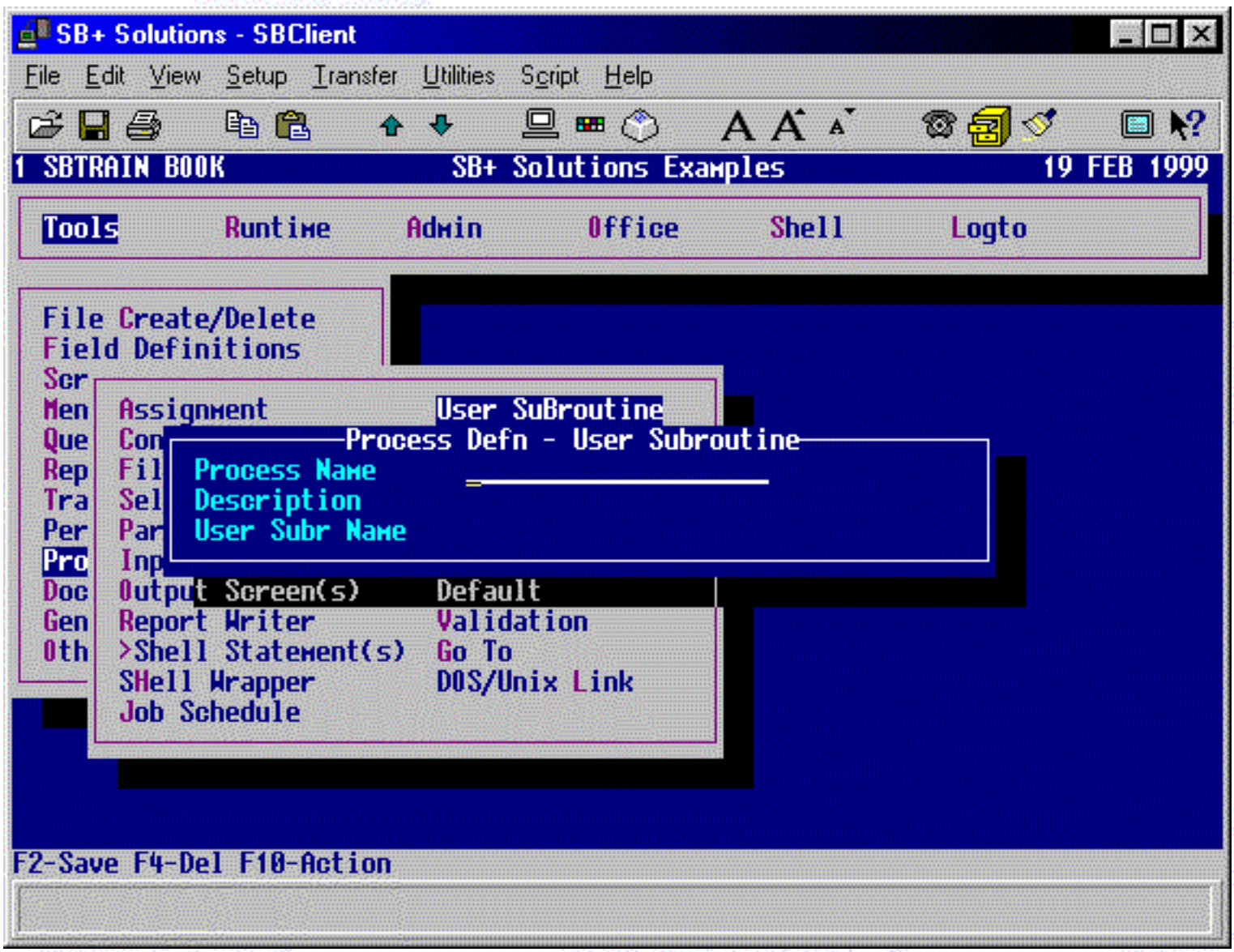
Creating the BASIC Process

BASIC subroutines can be called directly in a process slot with the following syntax:

B:subroutine,parameters

While this avoids the need for a BASIC process, it isn't something that I would recommend. Calling a subroutine directly avoids any security checks that could be done, and also makes it difficult to track how much BASIC you've used in your application. By religiously creating BASIC process records, you can run a simple analysis on your process file to determine how effectively you've used paragraphs!

To create the BASIC process, start the Process Definition - BASIC tool. This can be done either by selecting Tools, Process Definitions, BASIC, or by entering /PD.B at any input prompt or menu. This tool appears as follows:



At the "Process Name" prompt, enter the name of a process, using your process naming standards. Typically, the name of the process and the name of the subroutine are the same, except that the subroutine name has the system ID appended to the front.

At the "Description" prompt, enter a brief description of the process. Try to use words that will be helpful in finding the process later (particularly if you're using sequentially assigned process names).

At the "User Subr Name" prompt, enter the name of the subroutine you've created. As stated earlier, the subroutine name is typically the same as the process with the system ID appended to the front. Therefore, if your system is ABC, and the process name is XXX0023, the subroutine name would be ABC.XXX0023. Granted, this is merely a convention, but it has proven itself to be an effective standard in my own development.

If this process should pass a parameter to the subroutine, simply enter a comma and the parameter following the subroutine name in this prompt. Of course, the parameter is optional.

Once this information has been entered, press F2 to save the process record. Once saved, the subroutine can be called by its process name, instead of calling the subroutine directly using the B:xxx syntax.

Selecting One Record

To select one record in a Selection Criteria, simply enter the record key value in single quotes, as in:

'101'

Any literal or SB+ **expression** can be inside of the single quotes, so if you want to pass the key value into the report in @PARMS(2)<5>, the selection criteria would appear as follows:

'(@PARMS(2)<5>)'

Note that when an SB+ expression is used, it must be enclosed in parentheses. This is how SB+ knows the difference between a literal key and a calculated one.

Selecting a Group of Records Based on a Condition

To select a group of records, you can enter a selection condition *triad* in the form:

fieldname operator value

In this syntax, *fieldname* is the name of any valid field in the file. This field must reference either a key value, attribute in the record, or valid **derived value field**. The *operator* value may be one of several different operators, including:

=	Equal To
>	Greater Than
<	Less Than
>=	Greater Than or Equal To
<=	Less Than or Equal To
#	Not Equal To

Lastly, *value* is a quoted literal string (using only normal (") quotes). For example, to select all records where the STATE field is not equal to null, the triad:

STATE # ""

...could be entered. To select all customers with a credit limit over $5000, use:

LIMIT > "5000"

The literal value may also have a leading bracket, trailing bracket, or both. These are the wildcard characters, and are used as follows:

NAME = "AB]"	Select records with names starting with AB
NAME = "[SMITH"	Select records with names ending in SMITH
NAME = "[X]"	Select records with an "X" anywhere in the name field

The literal value can also be used to prompt the user to enter a value when the selection criteria is evaluated. To do this, replace the literal with a question mark. Therefore, to prompt the user to enter a credit limit and then select all customers with a limit over the value entered, use:

LIMIT > "?"

When this is executed, a small window will be opened in the lower right corner of the screen where the user can enter whatever credit limit they prefer in running the report.

There are several options that can follow the question mark inside of the quotes to modify the behavior of the prompting box.

These are:

V	Use the **validation** from the field definition
D	Use the **default** from the field definition
I	Use the **input conversion** from the field definition
O	Suppress the **output conversion** on the field definition
L	If two fieldname operator value triads are connected with AND or OR, and the prompt with this option is left null, this field and its connected field will both be dropped from the selection expression.
A	If two fieldname operator value triads are connected with AND or OR, and the prompt with this option is left null, this field will not be dropped from the selection expression. (Without this option, the triad would be dropped.)
M	The prompt is **mandatory** (i.e. a required entry)

These options can be combined and listed in any order, so if you want to prompt for a credit limit, show a default, and validate the entry, use:

```
LIMIT > "?VID"
```

Note that the "I" is included here, though there appears to be no good reason to do so. This is a good habit to get into, particularly when prompting for fields which may have an input conversion. Unlike a typical input screen, SB+ does not automatically use the input conversion, but it does use the output conversion. Therefore, if the LIMIT field has both input and output conversions of MR2, the prompting box (without the "I" option) will take an entry of 5000 and will select records with a credit limit higher than 50.00. With the "I" option, however, when you enter 5000, SB+ will select records with a credit limit higher than 5000.

These fieldname operator value triads may be combined with AND or OR to create more complex selections. For example, to select all customers with an "X" in their name living in a state that starts with the letter "A", the following could be used:

```
NAME = "[X]" AND STATE = "A]"
```

Of course, the wildcards, question mark, and the question mark modifiers may be used in any or all of the subexpressions. Also, it is important to note that there is no precedence order between AND and OR. In situations where there could be a conflict based on precedence, the results will be unpredictable.

The Double Question Mark (??)

If a user enters something on a line, then presses F3 to invoke a selection box which prompts for information, the value entered up to the point of pressing F3 will be placed into the first prompt on the window. Sometimes, however, the information as entered "belongs" to a prompt other than the first one in the selection box. When this happens, you can use a double question mark to tell SB+ which prompt should get the input.

To illustrate this, assume we want the user to be able to enter a state code and then press F3. When F3 is pressed, we want to display a window prompting for name and state. Because state is not the first prompt, we can tell SB+ to load the information typed before the F3 into the state prompt as follows:

```
NAME = "?" AND STATE = "??"
```

Sometimes it can be beneficial to know whether the user typed anything before pressing F3 or not. To figure this out, call a paragraph for the intuitive help process. If @VALUE is null, nothing was entered. Otherwise, @VALUE contains the characters that were typed up to the point of pressing F3. Using this information you can call separate intuitive help processes based on the different conditions and eliminate what may be unnecessary prompting.

Incidentally, the options that apply to the "?" also apply to the double question mark.

Selecting All Records

When the Selection Criteria is null, all records will be selected. It just doesn't get much easier than that.

Selecting All Records in a Saved List

To create a report that uses a select list, enter the name of the select list in the Selection Criteria prompt. This list may be created by a pre-report process, or by literally any other process. As long as the list exists when the report is executed, the records as defined by the list will be selected for the report.

In certain situations, it can be useful to run the report from different lists, depending on the port where the report is run. For those types of reports, append the following to the end of the list name:

:PORT

For example, if the name of the list should be "WRK.xxx" (where xxx is the current port number), enter the following in the selection criteria prompt:

WRK.:PORT

Selecting Keys From a Multivalued List in Common Memory

To select records for the report based on a list of keys in memory somewhere, enter the following syntax in the Selection Criteria prompt:

```
SELECT (exprn:@VM)
```

In this syntax, exprn is any valid SB+ **expression**, which can include references to common memory, file translates, process calls, or any other possible expression variation. For example, if the list of keys to run the report for is in the **common variable** @VALUE, use:

```
SELECT (@VALUE:@VM)
```

If the list of keys to run the report for is in a file named XYZ, field 5, and @PARMS(2)<4> has the key value for the XYZ record, use:

```
SELECT (F("XYZ",@PARMS(2)<4>)<5>:@VM)
```

To call a process which returns the list of keys in @VALUE, use:

```
SELECT (P("processname"):@VM)
```

Keep in mind that for this syntax to work you need the preceding SELECT, and also the trailing ":@VM". If you forget one or the other, it just isn't going to return the intended results.

QSELECTing A Multi-Attribute List

In the early days of SB+, one of the more popular selection options was to QSELECT a multi-attribute list. This option is still available, though not as widely used as other selection options.

In the Selection Criteria prompt, the following syntax will tell SB+ to QSELECT a multi-attributed record, returning each attribute as a separate key:

QSELECT *filename* '*itemID*'

In this syntax, *filename* is the name of a file where a record exists, and *itemID* is the key to that record. When this is executed, each attribute becomes a separate key in the selected list. These keys are then used to build the information to be displayed in the selection box. Note that when this technique is used, the "W" (return first word) option is also commonly used.

Implementing Impossible Selection Criteria

Despite the multitude of options that SB+ provides for selecting records for the report, there will be times when this will be insufficient. For example, what selection criteria could be used to select all orders which have a total amount (minus the discount) in excess of 135% of the credit limit for the state where the order was shipped?

While this is indeed impossible using the various **Selection Criteria** techniques, such a selection is still possible. (And it's a darn good thing -- you won't make many friends if you tell folks "SB+ can't do that"! VOE)

Instead of using Selection Criteria, we can use the Process After Read to filter the records for the report. In this process slot, if we want SB+ to skip the current record, we simply set the **common variable** @RTN.FLAG to "1". For example, this dinky little **paragraph**, called from the Process After Read, could be used for the above "impossible" selection:

```
CR.LIMIT = F('STATES',SHIP.TO.STATE)<6>
IF ORD.TOT - DISC.AMT < CR.LIMIT * 1.35 THEN EXIT 1
```

(Of course, field names and attribute positions are assumed.)

Any verification of any complexity can be done in this process slot. As long as @RTN.FLAG is set to 1 (as is done using **EXIT** 1 in the above example), the record will be skipped.

This technique can also be combined with other Selection Criteria options. When using the two together, use Selection Criteria to filter out certain records, then use this technique to filter records at a more detailed level.

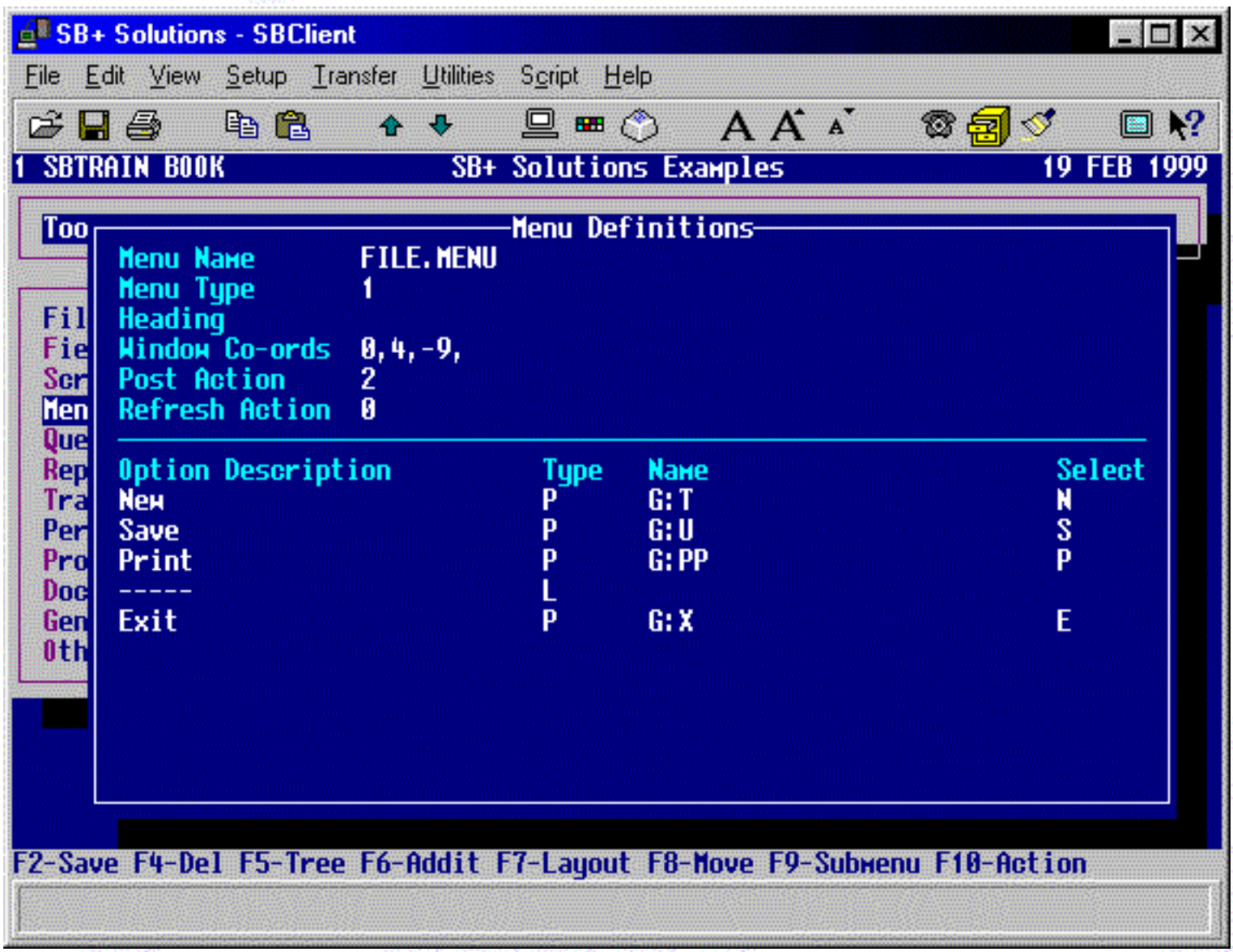
Implementing an Action Bar "File" Option

Applications written under Microsoft Windows almost always have a menu with the first option being the word "File". SB+ applications can be built with this same feature, by creating a standard menu which is called from an action bar option called "File".

Following the Windows convention, this menu should have (at least) the following options:

New	Abandon the current record and start a new one.
Save	Save the current record.
Print	Print the current screen image.
Exit	Exit -- Exit the screen

The menu definition for this file menu, called FILE.MENU, is as follows:



The goal of this menu is really very simple: When an option is selected, the menu should return a value in the **common variable** @RTN.FLAG. Note that the "G:*x*" syntax (called an *inline Goto process*) is used for this purpose. If "New" is selected, @RTN.FLAG should be set to "T", which tells SB+ to go to the top of the screen. If "Save" is selected, @RTN.FLAG is set to "U" to go to the update step. If "Exit" is selected, @RTN.FLAG is set to "X" to exit the screen.

The Print option, however, is not as straightforward as the other options. We want to call the SB+ "P" process to print the screen, but we shouldn't call the process until the file menu is gone. Otherwise, the file menu will appear on the text that is printed, and possibly cover up other important information that we want to see.

If the first character of @RTN.FLAG is set to "P" by the action bar option, SB+ will extract everything following the first character and execute it as a process after the menu goes away! (Note the Post Action = 2 makes the menu go away as soon as the selected option has completed.) Therefore, if we wanted to call the DIARY process this way, we could set @RTN.FLAG to "PDIARY". In this case, we want to call the "P" process, so @RTN.FLAG is simply set to "PP".

Once this file menu is created in the menu definitions tool, it can be called from a standard action bar simply by entering the first action bar option as "File", and then entering "M:FILE.MENU" as the process to call. Or, if you'd like, you can create a menu process to be invoked using the Process Definition - Menu (/PD.M) process.

Implementing an Action Bar "Toolbox" Option

One useful option for a standard action bar is called a "toolbox". This option allows the developer to setup a standard menu of additional options (such as TCL, the SB+ diary, etc.) that the user can call from any screen.

To implement a toolbox, simply nominate a menu name for the toolbox menu, such as TOOLBOX.MENU or USER.MENU. Once this menu has been named, you may reference it on any action bars simply by entering an action bar option of "Toolbox" and a process to call of "M:TOOLBOX.MENU".

With this menu in place, users can add options to the menu at any time, and the options will automatically be available on all screens that have this toolbox option on the action bar.

Implementing an Action Bar "Query" Option

Like a toolbox, the "Query" option on an action bar is nothing more than a menu that invokes the various inquiry programs in your application. With such an option on the action bar, users can look at anything in the system without having to exit their current program, navigate menus, and then select the appropriate option. Instead, they select Query from the action bar and choose the inquiry to use from the menu, returning to the current screen when complete.

To implement a query menu, first create a menu of the query programs you want to see. On the standard action bar, enter "Query" as the action bar label, and "M:QUERY.MENU" (or whatever name you choose) as the corresponding process. Or, you may also create a **menu process** instead of referencing the menu via the M: syntax.

Implementing an Action Bar "Office" Option

To implement an "Office" option on a standard action bar, simply create an action bar option called "Office", and call the SB+ standard OFFICEMENU using the process "M:OFFICEMENU". With this on the standard action bar, users are able to use the SB + office automation tools inside any screen in the application. You may alternatively create a **menu process** to invoke the office menu and link the actual process to your action bar.

Depending on how fussy you are about screen positioning, the standard office automation menu may not be in the right place on the screen. Therefore, you may want to copy the menu and change the screen coordinates so that the menu appears in the right place in relation to the option on the action bar. Considering SB+ will overwrite the office automation menu each time you upgrade to a new release, this isn't a bad idea anyway.

In each of the standard SB+ tools, there is a Copy (or Rename) feature. Regardless of the name, this feature will allow you to copy one record to another name. In this case, you could copy the menu called OFFICEMENU (the standard SB+ office automation menu) to a new name, thus eliminating the possibility of losing your office automation menu when you upgrade to a new version of SB+.

For more information about copying records in the SB+ tools, consult the SB+ documentation.

Implementing an Action Bar "Help" Option

The Help option on an action bar is intended to provide the user a page or more of documentation to help them know more about the screen or application being used. This is a most useful addition to any screen, but requires a significant investment in time following the original development to document each screen effectively.

Unlike the other action bar options, this is one option that we want to perform differently depending on where it is called from. Fortunately, we can still create a single process to be called from the action bar that figures out where it is being called from and acts accordingly.

SB+ has a standard process named CALL.HELP which is used to display help messages on the screen. This process has a single parameter which defines the ID of the text to be displayed (from the XXXHELP file). The trick, then, is to figure out the name of an item in this file.

To do this, we can use the **common variable** @PROC.NAME. This variable keeps an attribute-delimited list of processes that have been called to this point, with the most current process listed first. Therefore, from the action bar we can get the name of the calling process simply by extracting attribute 2 of @PROC.NAME

With these two issues in mind, we can create a standard help paragraph called XXX.HELP (where XXX is the system ID) with the following lines:

```
IF F('XXXHELP',@PROC.NAME<2>) # '' THEN
  EXEC 'CALL.HELP,':@PROC.NAME<2>
END ELSE
  ERROR "There is no help message available"
END
```

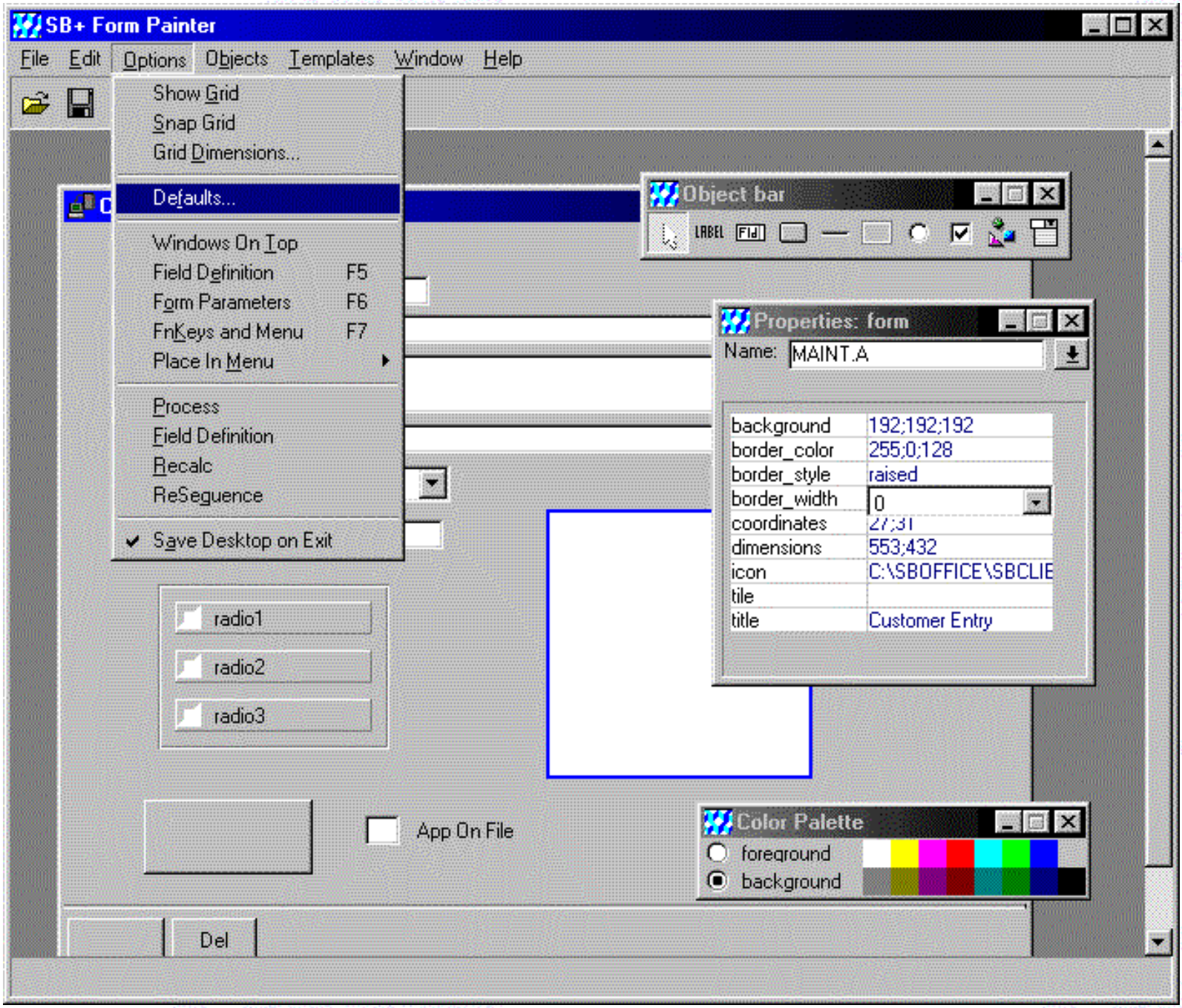
This process checks first to see if there is a help message on file, and if so the CALL.HELP process is called to display the message. If there is no message on file, a message to that effect is displayed.

Of course, this is a very brief example of what can be done with this particular process. There are any number of variations on this theme, depending on your project, standards, and imagination.

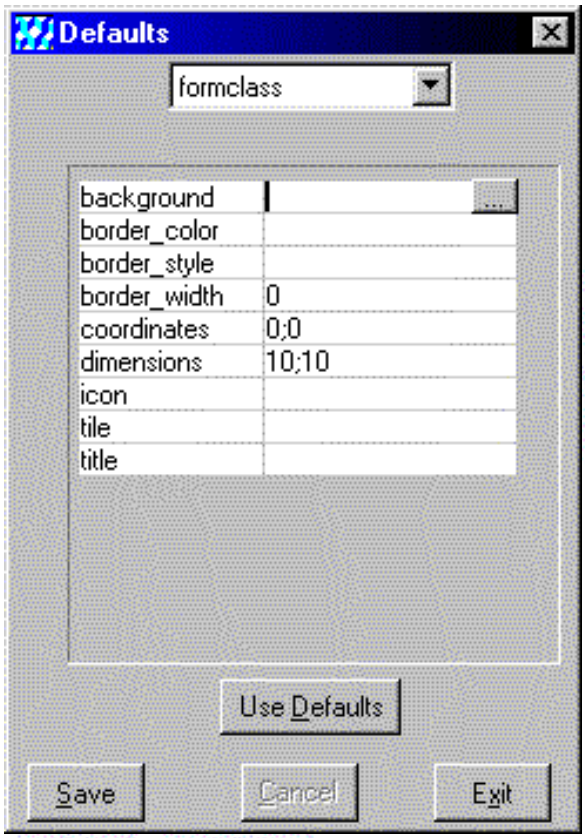
Controlling the GUI Look and Feel

Fresh out of the box, the GUI form painter creates screens with objects which all look similar. Form backgrounds are always grey, as are button faces; text is always black and in a certain font; images are shown a particular way; the list goes on and on. Consistency like this is good -- it helps promote a standard look and feel across all your GUI forms.

But what if you want a *different* standard? What if you want your buttons to have a yellow background? Or perhaps you'd like your textclass objects to have a cyan or navy blue background with white text in a different font? These changes, and many more, are all available by changing the GUI *defaults*. These defaults can be changed by selecting Options | Defaults in the GUI form painter, as follows:



When this option is selected, the following screen is displayed:



On this window, you can change the default properties for each of the different objects supported by the GUI. (Click on the combo box at the top to select the various GUI objects.) Once the default has been changed, all new objects will use these defaults. Existing objects, however, will not be automatically updated to use the new defaults, and therefore you should set these properties before doing any significant GUI development.

See "**GUI Object Properties**" for more information about the available properties for each object.

Textclass Properties

The following properties can be defined for textclass objects on a GUI form:

background	This property defines the background color for the text box using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color.
border_color	This property defines the color of the text box border and is effective only when the border_style property is "flat" and the border_width is greater than zero.
border_style	This property defines the appearance of the text box border, and is one of three values: • flat - The border is on the same plane as the rest of the form, not raised or lowered. • raised - The border appears to make the object be "raised" off of the underlying form. • lowered - The border appears to make the object be "lowered" into the underlying form,
border_width	This property is a number of 0 to 5, representing the number of pixels used in the construction of the border. The greater the number, the wider the border. Note that the border_color and border_style properties are ignored if the border_width property is set to 0.
coordinates	This property defines the row and column (in the format *row*;*column*) where the object is placed in the GUI form (in pixels, in relation to the upper left corner of the GUI form).
cursor	This property defines the cursor image to use when the mouse pointer is hovering over the object. Cursor files are images with a CUR extension, commonly found in the \WINDOWS\CURSORS and \SBOffice\SBClient \sbopen\APPSBMP directories.
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the object.
font	This property defines the font used for the text in the box. When changing this property, use the ellipsis button to select an appropriate font. Note that the font used must be installed on all of the user systems, otherwise SB+ will revert to the default prompt when necessary.
foreground	This property defines the foreground (text) color for the text in the box using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color.

Toggle Properties

The following properties can be set for toggle objects on a GUI form:

background	This property defines the background color for the toggle object using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color. Note that this property is ignored when the tile property is used.
border_color	This property defines the color of the toggle border and is effective only when the border_style property is "flat" and the border_width is greater than zero.
border_style	This property defines the appearance of the border, and is one of three values: • flat - The border is on the same plane as the rest of the form, not raised or lowered. • raised - The border appears to make the object be "raised" off of the underlying form. • lowered - The border appears to make the object be "lowered" into the underlying form,
border_width	This property is a number from 0 to 5, representing the number of pixels used in the construction of the border. The greater the number, the wider the border. Note that the border_color and border_style properties are ignored if the border_width property is set to 0.
coordinates	This property defines the row and column (in the format *row*;*column*) where the object is placed in the GUI form (in pixels, in relation to the upper left corner of the GUI form).
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the object.
false_fill	This property is used when the style property is set to "diamond" and defines the background color for the toggle when it is unselected.
false_graphic	This property defines a graphic image to be displayed when the toggle is unselected.
font	This property defines the font used for the toggle label. When changing this property, use the ellipsis button to select an appropriate font. Note that the font used must be installed on all of the user systems, otherwise SB+ will revert to the default prompt when necessary.
foreground	This property defines the foreground (text) color for the toggle label using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color.

style	This property controls the presentation of the toggle, and can be one of the following options: • bitmap - When the object is selected (true), the graphic as nominated in the true_graphic property is displayed for the object. When the object is not selected (false), the graphic as nominated in the false_graphic property is displayed for the object. • button - When the object is selected (true), the object appears as a depressed button. When the object is unselected, the object appears as an unpressed button. The color and image of the button is controlled by the true_fill, true_graphic, false_fill, and/or false_graphic properties. • check - When the object is selected (true), a check mark appears in the box. When the option is not selected (false), no check mark appears in the box. • circle - When the object is selected (true), the circle is filled with the color as defined under the true_fill property. When the option is not selected (false), the circle is filled with the color as defined under the false_fill property. If no colors are defined under the true_fill and false_fill properties, a black dot will be displayed in the circle when selected, and nothing will be displayed in the circle when the object is unselected. • cross - When the object is selected (true), the box is filled in with an X. The color of the X is determined by the true_fill property. When the object is not selected (false), the box is empty. • diamond - When the object is selected (true), the box is filled in with a small diamond. The color of the diamond is determined by the true_fill property. When the object is not selected (false), the diamond is empty. • mscross - When the object is selected (true), the box is filled with a check mark, in the color as defined by the true_fill property. When the object is not selected (false), the box is empty. • mscircle - When the object is selected (true), a black dot is displayed in the circle. When the object is not selected, the circle is empty. (The color of the dot is not configurable.)
tile	This property is used to define a background image for the label. There is no option currently available for scaling the image to the size of the object. The text as shown in the toggle_label property will print on top of this background image.
toggle_border_width	This property defines the width of the border around each of the check boxes.
toggle_label	This property defines the label for the object, which is shown to the right of the toggle box.
toggle_size	This property can be used to change the size of the check box used for the toggle.
true_fill	This property defines the color of the marker inside the object (the dot/diamond/check/etc.) when the toggle is selected. (Note that the color of the marker cannot be changed when the style property is set to "mscircle".)
true_graphic	This property defines a graphic image to be displayed when the toggle is selected.

Validating Against a Table

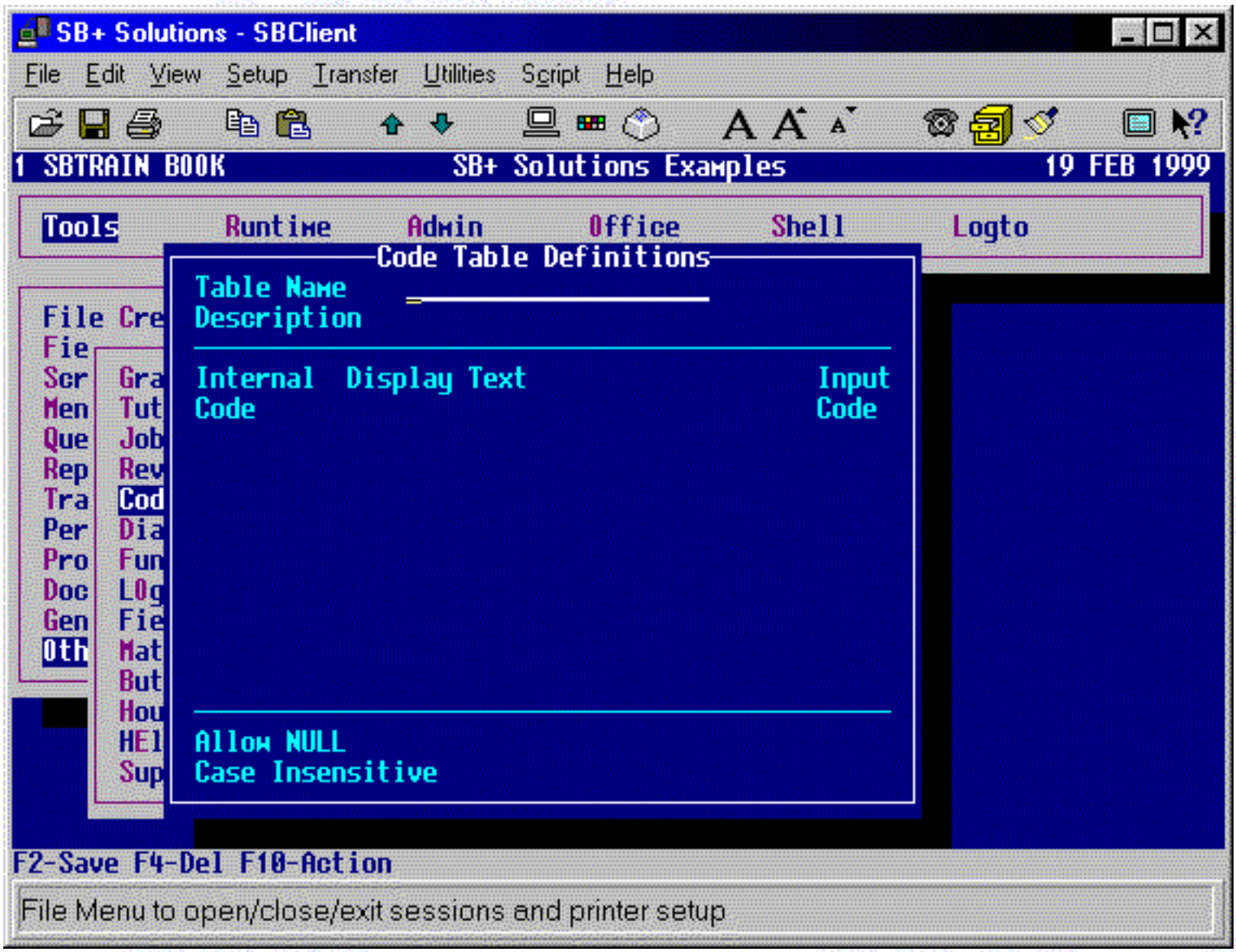
Validating against a table is similar to the V: validation code, except that the codes are actually stored outside of the software. The table can be easily changed at will, and most important, there is no software modification required.

To verify against a table, two things need to be done: 1) a table needs to be constructed, and 2) the validation code needs to be entered.

Building a table is very, very easy using the Code Table Definition tool in SB+. To start this tool, you may either select Tools, Other Tools/Utilities, then Code Table Definition, or simply enter /TABLE.DEFN at any menu or input prompt. The tool will then ask you where you want the table to be stored, with the default being your current XXXDEFN file.

For most applications, all tables can be stored in the XXXDEFN file without incident. However, if you want to differentiate between code tables that are user modifiable vs. system tables that are developer modifiable, you may want to store the developer tables in XXXDEFN and the user tables in another file elsewhere (perhaps named XXXTABLES?) in the account.

Once you've entered the name of a file where the tables are to be stored, the Code Table Definition screen is displayed, as follows:



To create a table, simply enter a name at the first prompt (using your naming standards). Next, enter a description. Below the middle line, simply enter a list of codes in the first column and a description for the codes in the second column. If you want the user to enter one code but store a different code in the record that is saved, enter the code they should enter in the Input Code column, and the code that is to be stored in the first column.

At the Allow Null prompt, enter 'Y' or 'N'. If 'N' is entered here and the user enters a null value in a prompt validated against this table, an error will result. If 'Y' is entered here and the user enters null, the entry will be considered valid. This can also be accomplished using the following validation code:

"" OR E:TABLE('*tablename*') > "{*message*}

The Case Insensitive flag is intended to allow the user to enter their entry in any case and SB+ will find it in the table. As of version 4.x, this does not work properly. Therefore, always enter your codes in upper case and use the MCU input conversion to convert your entry to the proper case prior to validation.

Once you've entered all of the information for your table, simply press F2 to save the table, then press Escape to exit the Code Table Definition tool.

Once the table has been built, the next step is to build a validation code which uses the table. This is a simple E: validation which appears as follows:

E:(TABLE('*tablename*')>"){message}

In this example, *tablename* is the name of the table with the valid entries for the particular prompt. If the table is in a file other than XXXDEFN, this parameter may be entered in the format file,tablename.

It's important to note that this particular syntax variation is not one you need to remember. The F3 help on the Conversion prompt in the Field Definition tool will build this code for you, and will also build a conversion and intuitive help for the prompt. Whether you want the conversion is simply a matter of standards and personal taste, but the intuitive help that is built is of great value.

Whoa! Go back a second: Did I say press F3 on the "Conversion" prompt? You read it right, the F3 help to build a table validation is on the conversion prompt in the Field Definition tool. Go figure...

Table validation is a technique that is so valuable, you may want to consider creating a standard process to do all of your table validations for you. With this standard process you can avoid having to remember the E: syntax and the strange benefits of the F3 key on the Conversion field. For example, I use the following process (called VAL.TABLE) which does table validation in my projects:

```
IF (TABLE(@PARAM) = "") THEN
  ERROR "That entry is not valid"
  EXIT 1
END
*
EXEC "UNIQUE"
```

This paragraph basically states "if the last entry made is not in the table as named in @PARAM, then display the error. If it is in the table but has been listed prior (if the field is multivalued), display an error message to that effect."

The validation code used to call VAL.TABLE appears as follows:

C:VAL.TABLE,tablename

In this example, tablename is the name of the table to use for the lookup. This is passed into the paragraph in the **common variable** @PARAM, and can be either the name of a table or a file name and table name separated by a comma.

Using this one syntax, verifying entries from tables is simple and, generally speaking, easy to remember. The goal, of course, is maximum productivity with minimum memorization.

Radio Button Properties

The following properties can be set for radio button objects on a GUI form:

background	This property defines the background color for the radio button object using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color. Note that this property is ignored when the tile property is used. Important Note: The background in a set of radio buttons is usually not very distinct. However, you can adjust the amount of visible background by changing the separation and toggle_offset properties.
border_color	This property defines the color of the button border and is effective only when the border_style property is "flat" and the border_width is greater than zero.
border_style	This property defines the appearance of the border, and is one of four values: • flat - The border is on the same plane as the rest of the form, not raised or lowered. • raised - The border appears to make the object be "raised" off of the underlying form. • lowered - The border appears to make the object be "lowered" into the underlying form, • msstyle - The border appears as it would in a typical Microsoft Windows application.
border_width	This property is a number of 0 to 5, representing the number of pixels used in the construction of the border. The greater the number, the wider the border. Note that the border_color and border_style properties are ignored if the border_width property is set to 0.
coordinates	This property defines the row and column (in the format *row*;*column*) where the object is placed in the GUI form (in pixels, in relation to the upper left corner of the GUI form).
cursor	This property defines the cursor image to use when the mouse pointer is hovering over the object. Cursor files are images with a CUR extension, commonly found in the \WINDOWS\CURSORS and \SBOffice \SBClient\sbopen\APPSBMP directories.
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the object.
direction	This property controls the presentation of the buttons, and has the following options: • horizontal - The buttons will be presented all on the same row, side-by-side. • vertical - The buttons will be presented vertically, one on top of the other.
false_fill	This property is intended to be used when the style property is "circle" or "mscross" and defines the fill color of the unselected radio buttons using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. (This property appears to be ignored.)

false_graphic	This property is used when the style property is "bitmap", and defines an image to use in filling the radio button (the whole button and label area, not just the button polygon). The image is shown only once, aligned to the left.
font	This property defines the font used for the radio button labels. When changing this property, use the ellipsis button to select an appropriate font. Note that the font used must be installed on all of the user systems, otherwise SB+ will revert to the default prompt when necessary.
label_border_color	This property defines the color of the border around the labels shown next to the radio buttons and is effective only when the label_border_style property is "flat" and the label_border_width is greater than zero.
label_border_style	This property defines the appearance of the label border, and is one of three values: • flat - The border is on the same plane as the rest of the form, not raised or lowered. • raised - The border appears to make the object be "raised" off of the underlying form. • lowered - The border appears to make the object be "lowered" into the underlying form,
label_border_width	This property is a number of 0 to 5, representing the number of pixels used in the construction of the label border. The greater the number, the wider the border. Note that the label_border_color and label_border_style properties are ignored if this property is set to 0.
radio_labels	This property allows you to override the labels used for the radio buttons.
scale_bmp	This property is used with the tile property, and defines whether the graphic is shown in its original size, or it it scaled to the size of the object area. Valid options include: • TRUE - The graphic is scaled to fill the radio button dimensions. • FALSE - The graphic is shown once in the button area.
separation	This property defines the space (in pixels) between each toggle in the radio button object, in the format *row;column.*
style	This property defines the style, or presentation, of the buttons themselves. The following options are available: • circle - Graphic circles are drawn for the radio buttons, and the selected circle is filled with the color defined in true_fill. • bitmap - The selected radio button will be displayed using the graphic file defined in the true_graphic property, and the unselected radio buttons will be displayed using the graphic defined in the false_graphic property. • mscross - Each radio button appears like a toggle (check box), though only one can be checked. The color of the check mark in the box is defined by the true_fill property. • mscircle - Radio buttons will appear as they would in a standard Microsoft Windows application. The fill color is not configurable with this option.
tile	This property defines the background image to be displayed for the radio buttons. This is most visual when used with the separation property to separate the buttons from one another and from the outside border.

toggle_background	This property defines the background color for each toggle (button and label set) in the radio button object (in contrast to the background property, which defines the background color for the object itself.) using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color. Note that this property is ignored when the tile property is used.
toggle_border_width	This property defines the width of the border around each radio button and is used when the style property is "bitmap","mscircle", or "mscross".
toggle_cursor	This property defines the cursor image to use when the mouse pointer is hovering over the object. Cursor files are images with a CUR extension, commonly found in the \WINDOWS\CURSORS and \SBOffice \SBClient\sbopen\APPSBMP directories.
toggle_dimensions	This property defines the width and depth of each toggle in the radio button object in *width*;*depth* format.
toggle_foreground	This property defines the color of the foreground text in the labels using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors.
toggle_offset	This property defines the amount of *gutter space* between the outside radio button object box and the toggles themselves. The format of this value is *row*;*column*.
toggle_scale_bmp	This property is used with the toggle_tile property, and defines whether the graphic is shown in its original size, or it it scaled to the size of the toggle area. Valid options include: • TRUE - The graphic is scaled to fill the radio button dimensions. • FALSE - The graphic is shown once in the button area.
toggle_size	This property controls the size of the actual buttons in each toggle.
toggle_tile	This property defines a background image to use for each of the toggles. If the toggle_scale_bmp property is TRUE, the graphic will be scaled to the size of each toggle. If the toggle_scale_bmp property is FALSE, the graphic will be tiled throughout the toggle area.
toggles	This property defines the number of radio buttons to be included in the set.
true_fill	This property is used when the style property is "circle" or "mscross" and defines the fill color of the selected radio button using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors.
true_graphic	This property is used when the style property is "bitmap", and defines an image to use in filling the selected radio button (the whole button and label area, not just the button polygon). The image is shown only once, aligned to the left.

Combo Box Properties

The following properties can be set for combo box objects on a GUI form:

background	This property defines the background color for the combo box using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color.
border_color	This property defines the color of the combo box border and is effective only when the border_style property is "flat" and the border_width is greater than zero.
border_style	This property defines the appearance of the combo box border, and is one of three values: • flat - The border is on the same plane as the rest of the form, not raised or lowered. • raised - The border appears to make the object be "raised" off of the underlying form. • lowered - The border appears to make the object be "lowered" into the underlying form,
border_width	This property is a number of 0 to 5, representing the number of pixels used in the construction of the border. The greater the number, the wider the border. Note that the border_color and border_style properties are ignored if the border_width property is set to 0.
coordinates	This property defines the row and column (in the format *row*;*column*) where the object is placed in the GUI form (in pixels, in relation to the upper left corner of the GUI form).
cursor	This property defines the cursor image to use when the mouse pointer is hovering over the object. Cursor files are images with a CUR extension, commonly found in the \WINDOWS\CURSORS and \SBOffice\SBClient \sbopen\APPSBMP directories.
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the object.
font	This property defines the font used in the combo box. When changing this property, use the ellipsis button to select an appropriate font. Note that the font used must be installed on all of the user systems, otherwise SB+ will revert to the default prompt when necessary.
foreground	This property defines the foreground (text) color for the text in the combo box using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*; *bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color.
num_lines	This property defines the number of lines (1-20) that will be shown in the combo box when the user clicks on the down arrow to see the list of available options.

Image Properties

The following properties can be set for image objects on a GUI form:

background	This property defines the background color for the image block using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color. Note that this property is ignored when the tile property is used.
border_color	This property defines the color of the label border and is effective only when the border_style property is "flat" and the border_width is greater than zero.
border_style	This property defines the appearance of the label border, and is one of three values: • flat - The border is on the same plane as the rest of the form, not raised or lowered. • raised - The border appears to make the object be "raised" off of the underlying form. • lowered - The border appears to make the object be "lowered" into the underlying form,
border_width	This property is a number of 0 to 5, representing the number of pixels used in the construction of the border. The greater the number, the wider the border. Note that the border_color and border_style properties are ignored if the border_width property is set to 0.
coordinates	This property defines the row and column (in the format *row*;*column*) where the object is placed in the GUI form (in pixels, in relation to the upper left corner of the GUI form).
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the object.
scale_bmp	This property is used with the tile property, and defines whether the background image is shown in its original size, or it it scaled to the size of the defined area. Valid options include: • TRUE - The graphic is scaled to fill the defined dimensions. • FALSE - The graphic is tiled (repeated) throughout the image area.
tile	This property can be used to define a background image for the object. If the scale_bmp property is FALSE, the image will be tiled (repeated) to fit the dimensions of the label. If the scale_bmp property is TRUE, the image will be scaled to fit the label dimensions.

Adding an Image Object to a GUI Form

Image objects are arguably the most exciting component of a GUI form. While the **toggles**, **combo boxes**, and **radio buttons** provide an alternative to the character mode methods for setting a value in the record, image objects provide a functionality that is simply impossible under character mode.

There are effectively two different types of images that can be added to a form:

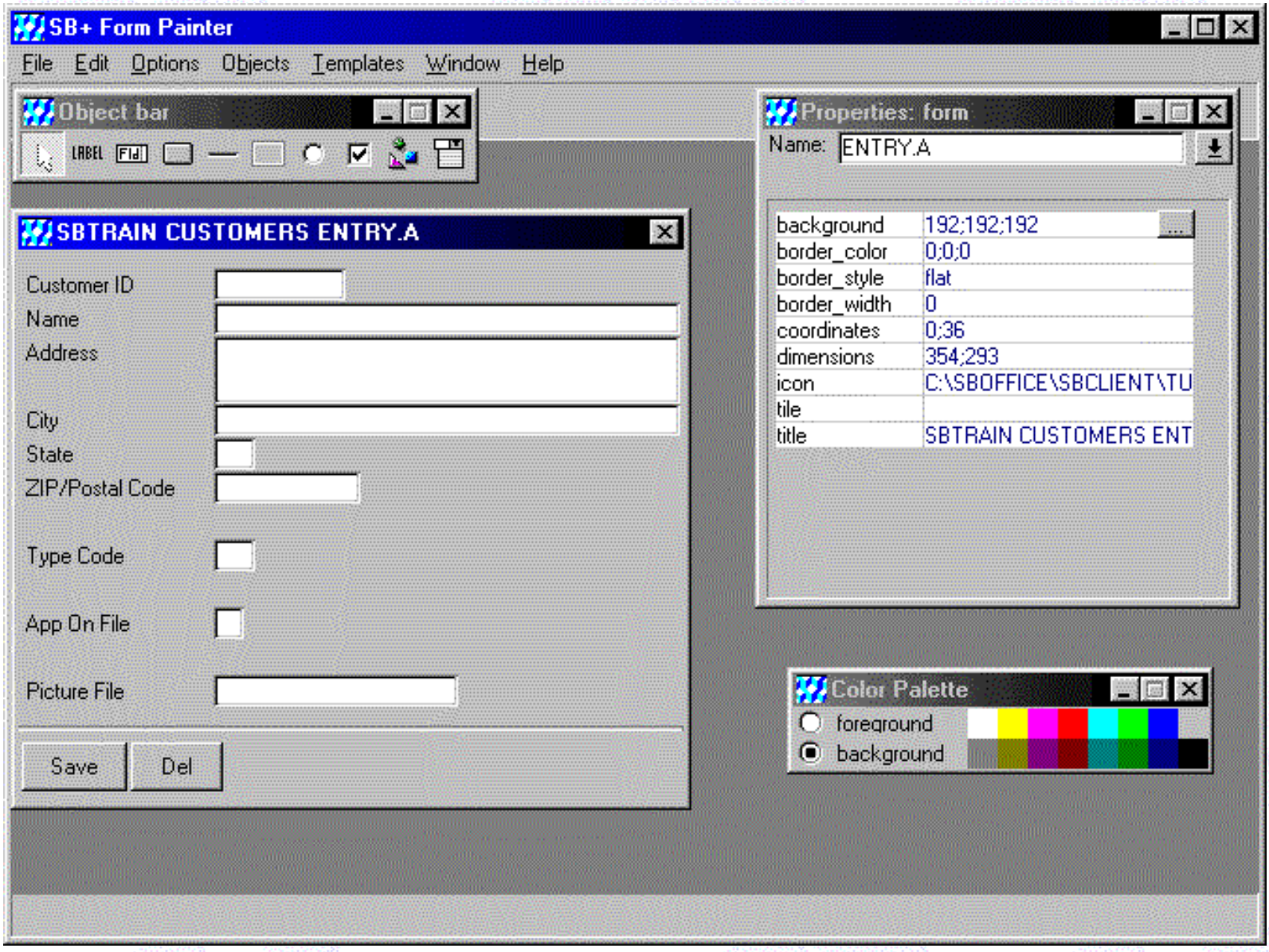
- S*tatic images* are those images which will always appear the same regardless of the record being shown. These images are used for adding a corporate logo or related image to a GUI form, and are shown continually while the form is displayed.
- *Dynamic images* are those images which change as each record is loaded. These include customer or product photos or other images that provide visual information to the user.

The following sections outline steps for adding each type of image to a GUI form.

Adding a Static Image Object to a GUI Form
Adding a Dynamic Image Object to a GUI Form

Converting a Character Screen to a GUI Form

Once the file and screen name have been entered, the GUI form painter displays the existing form for you to make the appropriate changes, as shown in the following example:



For this example, I've created a typical character mode screen with a bunch of input prompts and have loaded the screen in the form painter. Note that I've made no attempt to pre-define each prompt for specific GUI objects. As a result, all of the prompts are textual.

A number of obvious GUI enhancements can be made to this screen, such as:

- The State field should have a combo box with a list of valid states.
- The Type Code field should have radio buttons allowing the user to select one of the valid type codes for this customer.
- The App On File field, a Y/N field, should be converted to a toggle.
- The Picture File prompt should actually display a photograph, instead of the name of the image file.

Converting a Text Field to a Combo Box
Converting a Text Field to Radio Buttons
Converting a Text Field to a Toggle

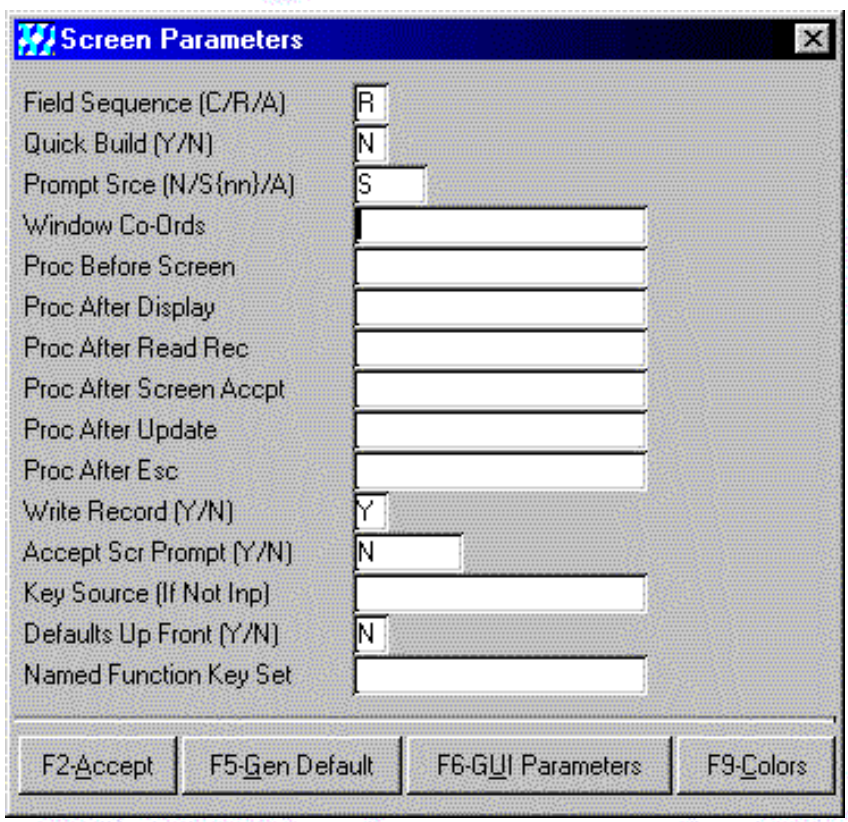
Converting a Text Field to an Image

Creating a New GUI Form

Creating a new GUI form is oddly similar to creating a screen definition with several significant differences. As to commonality, both tools are used for entering screen parameters and prompts for the screen. Both tools use the same keystrokes for adding fields (F5), reviewing/editing the screen parameters (F6), viewing the field definition for a field (F5/F5), reviewing/editing the additional parameters for a field (F5/F6), etc.

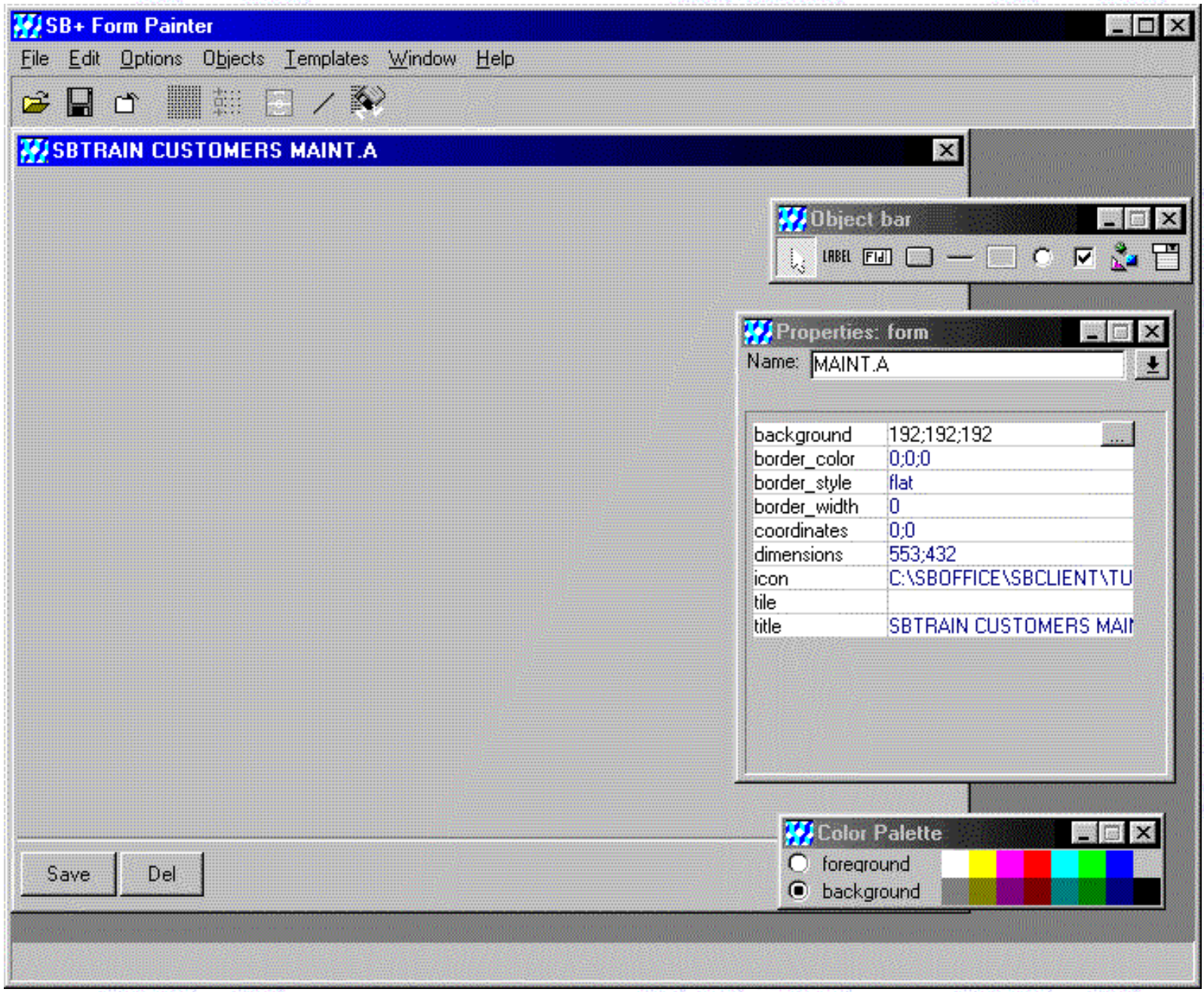
Despite the similarities, however, there are a number of significant differences. First, and most obviously, the GUI form painter runs in GUI mode and therefore has control over all of the GUI parameters for a screen. Therefore, you can change fonts, colors, and other settings that are not available in character mode. Second, instead of placing regular input and display fields and then converting them to GUI objects (as done in character mode; See "**Converting a Character Screen to a GUI Form**" for more information), the GUI form painter allows you to place a particular GUI object directly on the form. Finally, and perhaps most significant, is that when you create a form in GUI, the character mode screen is usually (though not always) just about useless. Therefore, if you need to run both character mode and GUI, start in character mode and convert to GUI, instead of the other way around.

Once the **GUI form painter has been started**, enter the name of a file where you want the screen to be created. When asked to enter a screen name, enter the name of a screen which does not exist in that file. Once these values have been entered, the screen parameters window will appear (as it would in the Screen Definitions tool) and will appear as follows:



(Note that the example given here assumes that no comparable character mode screen is intended.)

In a character mode screen, we would first focus our attention on the Window Co-Ords. In GUI, however, we don't need to worry about the window size; we can always resize it in the GUI form painter. Therefore, we can simply press F2 to accept what is shown here. After a couple of seconds, the default form is shown in the form painter, as follows:



SB+ automatically adds buttons for our F2-Save and F4-Del, as it would in the Screen Definition tool. At this point, we can begin adding objects to the form. Note the "Object Bar" currently shown in the upper right corner (the position of this will vary). This bar contains a series of buttons, each button corresponding to a GUI object or tool, as follows:

	Pointer tool: When this tool is highlighted (as shown here and above), the mouse pointer can be used for moving and resizing prompts.
	Prompt Label: This button allows you to place a textual label at a specific location.
	Textclass: Place a particular text field (prompt and input area together) at a specific location.
	Command Button: Place a button at a specific location. Buttons can be used like function keys to perform some action, like invoking **popup subscreens** or for calling other processes to perform miscellaneous tasks.
	Graphic Line: This tool allows you to draw separator lines on the form.
	Rectangle: This tool allows you to draw a graphic box around a collection of prompts, or show a corporate logo or other image.

	Radio Button: This button allows you to place a radio button field at a specific location.
	Toggle: This button allows you to place a toggle field at a specific location.
	Image: This button allows you to place an image field at a specific location.
	Combo Box: This button allows you to place a combo box field at a specific location.

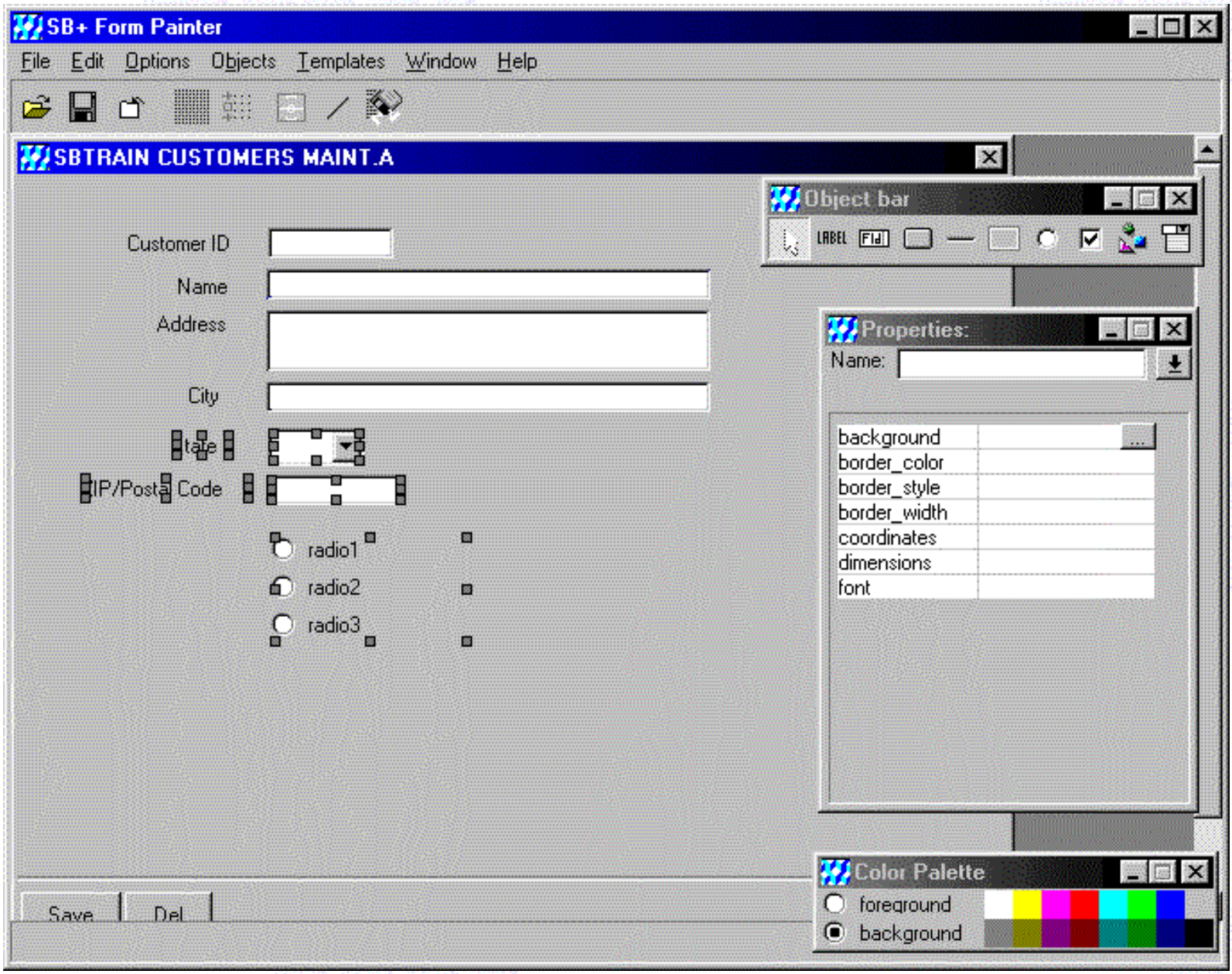
Adding a Textclass Object to a GUI Form
Adding a Label to a GUI Form
Adding a Radio Button Object to a GUI Form
Adding a Toggle Object to a GUI Form
Adding a Combo Box Object to a GUI Form
Adding an Image Object to a GUI Form
Adding a Command Button to a GUI Form
Adding Graphics to a GUI Form

Moving/Resizing Objects on the GUI Form

Moving Objects

Once an object has been placed on the GUI form, it can easily be moved. Simply click on the object and drag it to the new location. For radio buttons and toggle objects, the field descriptions will move along with the object itself. For combo box, textclass, and image objects, the label can be moved separately.

Unlike character mode, multiple GUI objects can be moved together. To move a series of prompts, hold the Ctrl key while clicking on a number of objects. As each object is clicked, *handles* will appear to confirm the object is selected, such as:



In this example, the **label objects** for the State and ZIP/Postal fields are selected, as is the **combo box object** for the state, the **textclass object** for the ZIP/Postal code, and the **radio button object** for the type code.

Once all of the items have been selected, continue to hold the Ctrl key and drag one of the objects to a new location. (You will see the one object move with the cursor, though all of the selected objects are being moved.) When you release the mouse button, all of the objects will be in their new

location.

You can also use the "lasso" to select a group of prompts. With the mouse pointer somewhere on the form (not pointing to a specific object), click and drag the mouse pointer over a group of prompts. Those prompts that are **completely** inside of the lasso when the mouse button is released will be selected.

Nudging Objects

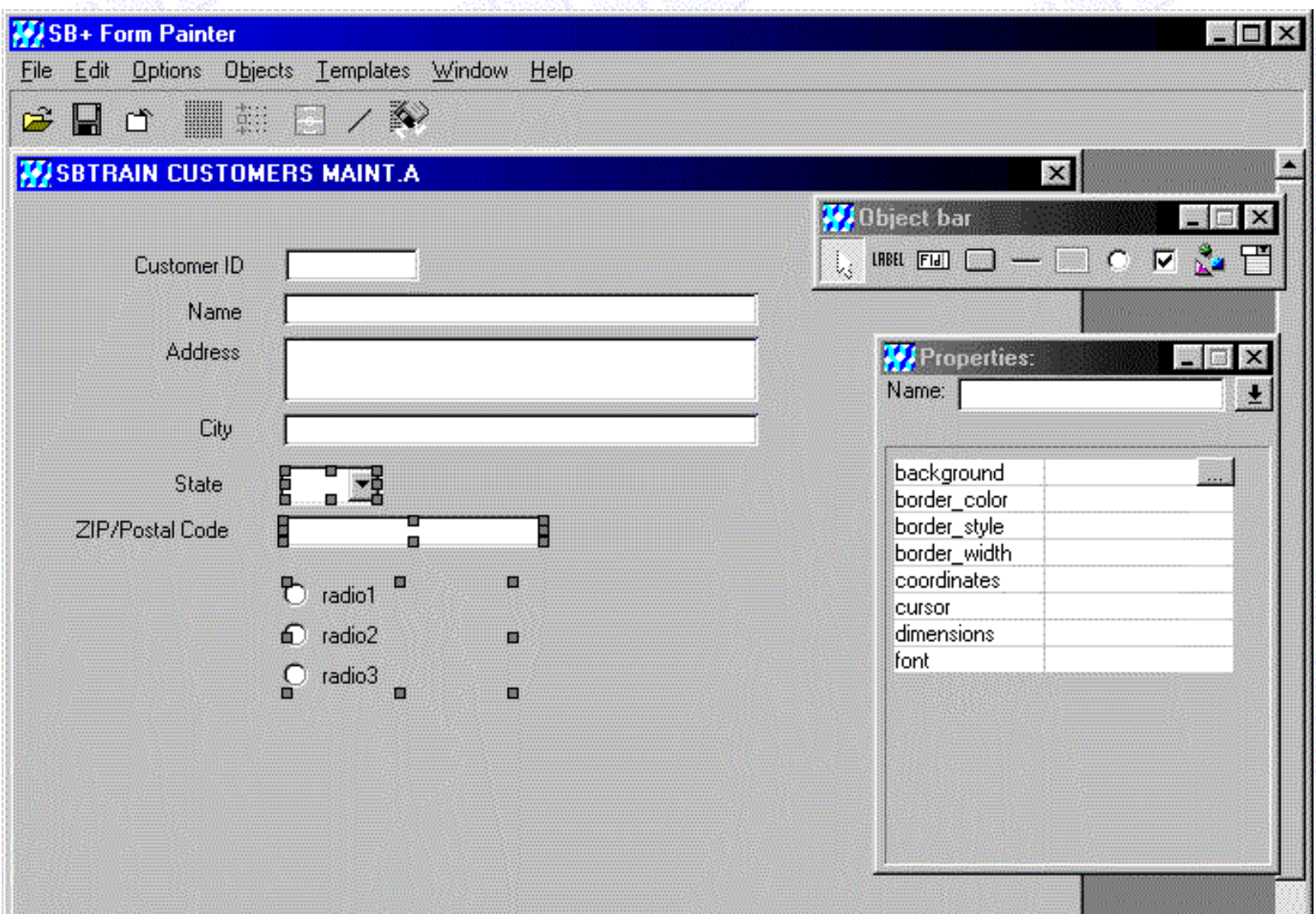
A selected object or objects can also be *nudged* (moved a pixel at a time in any direction) using the arrow keys after selecting the field(s). This can be very helpful for making very fine adjustments to objects for perfecting alignment anomalies. Of course, the "coordinates" property can also be manually adjusted to force all of the prompts to a specific row or column. (The value entered for this property is entered in the format *row*;*column*.)

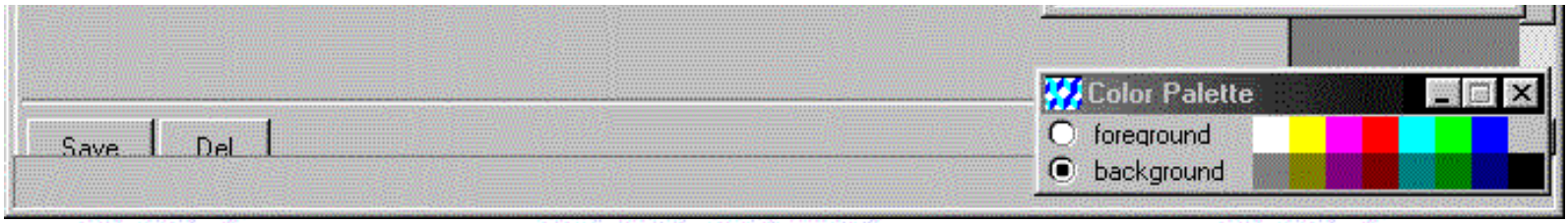
When nudging prompts or editing properties for objects, you can move quickly from prompt to prompt without the mouse. Simply press the Tab or Shift-Tab keys to move forward or backward through the objects.

Resizing Objects

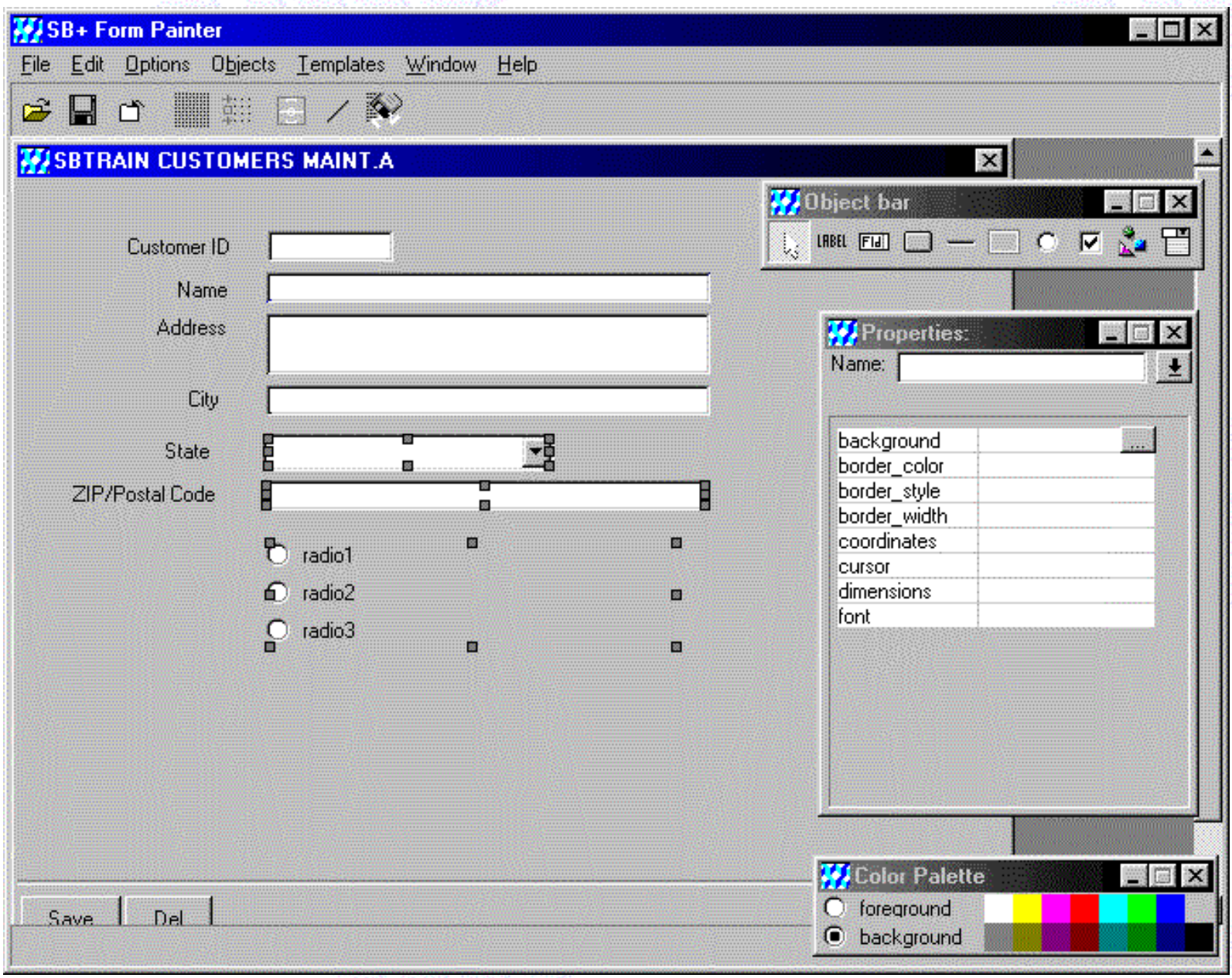
Any time you click on an object, *handles* appear in all four corners and on all four sides. As you float the mouse pointer over the handles, various arrows will appear which identify the handle currently under the cursor. Diagonal arrows mean you're over a corner, vertical arrows mean you're over the top or bottom border, and horizontal arrows mean you're on the left or right border. To resize the object, click and drag any of the handles until the object is the size you wish.

If multiple items are selected, and the Ctrl key is held while resizing an object, all of the selected items will be resized in proportion, such as in this example:





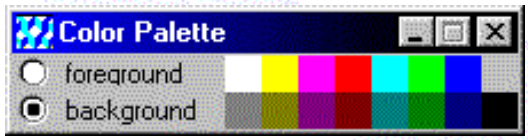
In this example, we've selected three objects to be resized together. Now, when we click on the right side handle of the ZIP/Postal Code field and move the cursor to the right, the objects all grow to the new length:



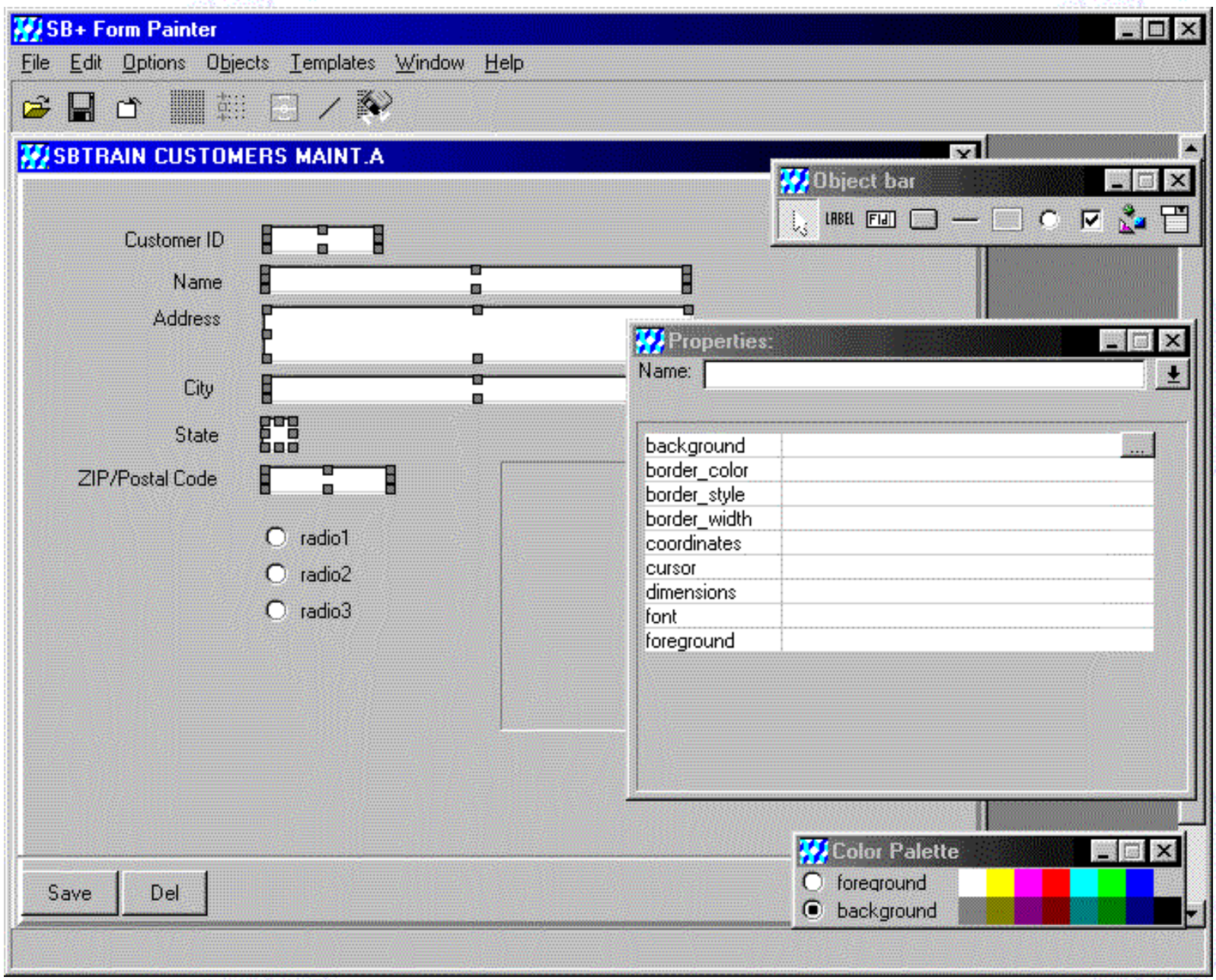
Any object can be resized this way. However, keep in mind that if you're resizing an image object, and that object has the "scale_bmp" property set to "true", changing the width and/or depth of the object may cause the image to be skewed.

Using the Color Palette

The Color Palette is a small, almost insignificant, window inside of the GUI form painter which allows you to quickly change the foreground or background color for one or several prompts. Using it, you can easily make color changes to your form without manually editing the "background" and "foreground" properties for each object to be colorized. This window appears as follows:



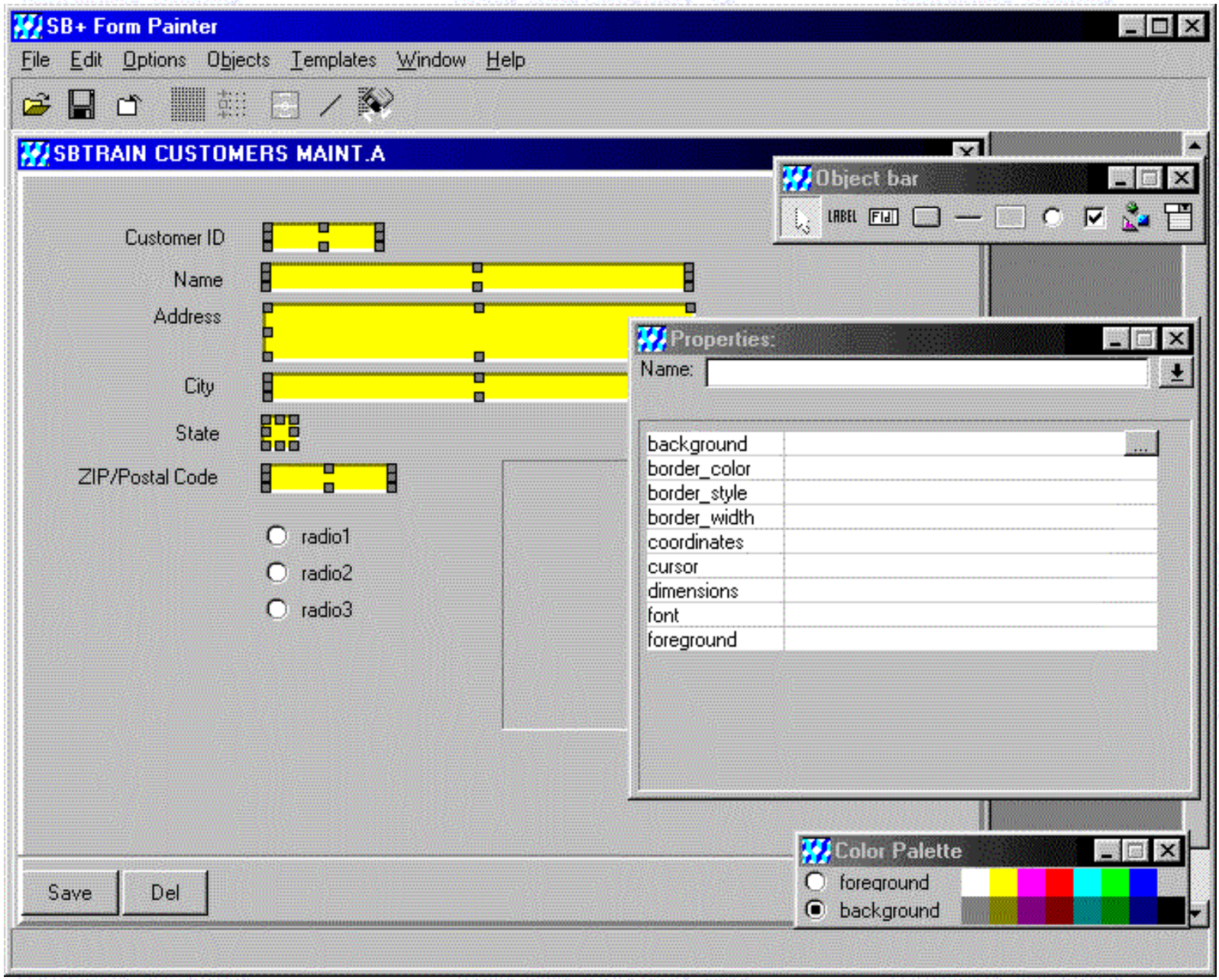
To use the Color Palette, select the object or objects to be colorized. Any single object can be selected simply by clicking on it. Multiple objects can be selected by holding the Ctrl key and then clicking on each object. Or, you can "draw a lasso" around a series of objects to select everything inside the lasso. In this example, all of the text fields have been selected with the lasso:



Once prompts have been selected, click either the "foreground" or "background" radio button shown on the left of the Color Palette, depending on whether you want to add color to the foreground (text) or background of the objects. (If the option you want to use is already selected, you don't need to

click on it.)

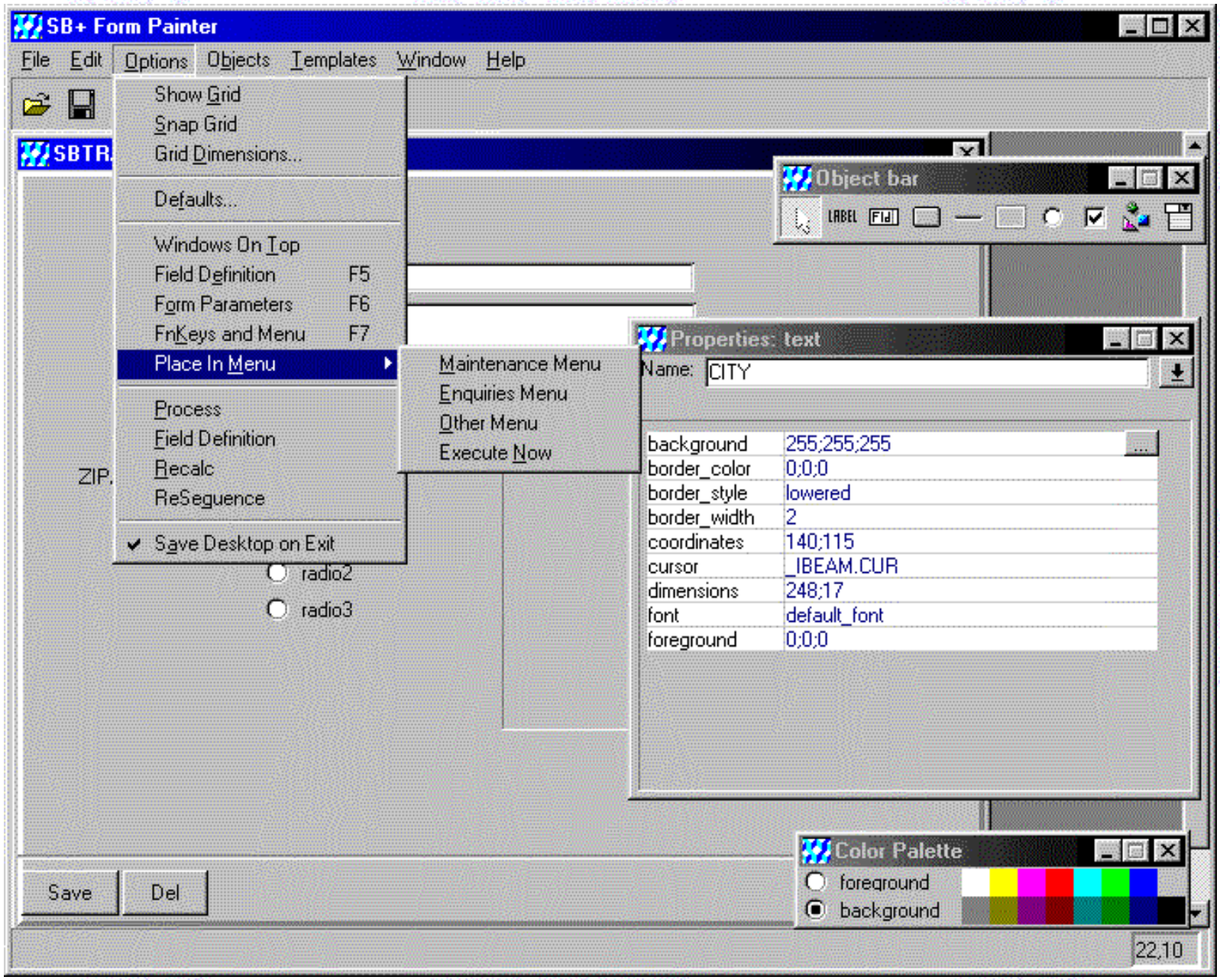
After selecting "foreground" or "background", click on the color you wish for that object or objects' property. The change will be made to all selected options immediately.



If, after making the change, you want to try another color, simply click on another color. To undo the change, click on the original color in the Color Palette -- note that the Edit | Undo option (from the GUI form painter main menu) does not undo color changes.

Running the GUI Form

One of the most widely used and appreciated features of the Screen Definitions tool is the ability to quickly save and run the screen using F2, None - Execute Now. If you try this in the GUI form painter, you'll quickly find it just doesn't work the same way. When you press F2, the GUI form is saved to the host, but where's the "Place in Menu" menu? Sadly, it doesn't exist in the GUI form painter -- at least not in the same way it exists in the Screen Definitions tool. It does in fact exist, however, in a different place. If you look on the Options menu, you'll see the Place In Menu option as follows:



When you want to run the GUI form, you *could* select Options | Place in Menu | Execute Now, but frankly, that's just too many keystrokes. Instead, you'll notice a button bar immediately shown below the GUI form painter's main menu. This button bar has a number of options, as follows:

	Open: When this button is pressed, you can open or create a new form.
	Save: When this button is pressed, the GUI form definition is saved to the host. This can also be done using the F2 key.
	Close: When this button is pressed, the GUI form painter is closed. (The Escape key works just as well.)

	Display Grid: When this button is pressed, a dot grid is displayed on the form. This grid is useful for lining up prompts precisely. Use this with Options | Grid Dimensions to control the width and depth between each dot in the grid.
	Snap-To Grid: When this button is pressed, objects will 'snap' to the dots on the grid. With this option enabled, options will gravitate towards the dots so you don't have to align the objects manually.
	Field Definition: After selecting a single prompt on the screen, you can click this button to go immediately to the field definition for the selected field. If no field is selected when the button is clicked, the field definitions screen will appear for the current file without loading a specific field definition.
	Slash: While this looks like a graphic line, it's actually a slash, and is used for running a process inside of the GUI form painter. This way, if you need to see a paragraph or other process while inside of the GUI form painter, you can click here, enter the appropriate tool name, and you're on your way.
	Run: This will invoke the None - Execute Now option without having to navigate through all the menus. (Incidentally, this works best when the form has the focus. If the Color Palette, Properties window, or Object bar has the focus, click first on the form and then select this option for the best results.)

Form Properties

The following properties can be defined for a GUI form:

background	This property defines the background color for the form using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color. Note that this property is ignored when the tile property is used.
border_color	This property defines the color of the form border and is effective only when the border_style property is "flat" and the border_width is greater than zero.
border_style	This property defines the appearance of the border, and is one of three values: • flat - The border is on the same plane as the rest of the screen, not raised or lowered. • raised - The border appears to make the form be "raised" off of the underlying screen. • lowered - The border appears to make the form be "lowered" into the underlying screen.
border_width	This property is a number of 0 to 5, representing the number of pixels used in the construction of the border. The greater the number, the wider the border. Note that the border_color property is ignored if the border_width property is set to 0.
coordinates	This property defines the row and column (in the format *row*;*column*) where the screen is positioned in the GUI form painter. It has no effect on the screen when it is executed.
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the GUI form.
icon	The purpose of this property is unknown.
tile	This property can be used to define a background image for the form. The image will be tiled to fit the dimensions of the form. There is no way to scale the image named here.
title	This property defines the title of the form. This title will be shown at the top of the form both in the GUI form painter, and also when the screen is running.

Label Properties

The following properties can be defined for a label object on a GUI form:

background	This property defines the background color for the label using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color. Note that this property is ignored when the tile property is used.
border_color	This property defines the color of the label border and is effective only when the border_style property is "flat" and the border_width is greater than zero.
border_style	This property defines the appearance of the label border, and is one of three values: • flat - The border is on the same plane as the rest of the form, not raised or lowered. • raised - The border appears to make the object be "raised" off of the underlying form. • lowered - The border appears to make the object be "lowered" into the underlying form,
border_width	This property is a number of 0 to 5, representing the number of pixels used in the construction of the border. The greater the number, the wider the border. Note that the border_color and border_style properties are ignored if the border_width property is set to 0.
coordinates	This property defines the row and column (in the format *row*;*column*) where the object is placed in the GUI form (in pixels, in relation to the upper left corner of the GUI form).
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the object.
emphasized	This property has two values, which define the appearance of the text: • TRUE - The label text is shown normally. • FALSE - The label text is shown de-emphasized, i.e. the standard visual indicator for display-only prompts.
font	This property defines the font used for the label. When changing this property, use the ellipsis button to select an appropriate font. Note that the font used must be installed on all of the user systems, otherwise SB+ will revert to the default prompt when necessary.
foreground	This property defines the foreground (text) color for the label using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color.
graphic	Either a text string (using the string property) or a graphic image may be used for the label. If the latter is desired, this property defines the graphic image to be used for the label. Note that this graphic image must be stored in a location which is available to the common user, otherwise it will not be displayed in the production environment.

justification	This property defines the justification (alignment) of the string or graphic in the label. Valid options include: • bottom_center • bottom_left • bottom_right • center • left • right • top_center • top_left • top_right
scale_bmp	This property is used with the graphic property, and defines whether the graphic is shown in its original size, or it it scaled to the size of the label area. Valid options include: • TRUE - The graphic is scaled to fill the label dimensions. • FALSE - The graphic is shown once in the label area, aligned according to the justification property.
string	Either a text string or graphic image (using the graphic property) may be used for the label. If the text string is desired, this property defines the actual string to be displayed. Note that this property is ignored when the graphic property is used.
tile	This property can be used to define a background image for the label. This image can be used with the string property to display a string on top of an image. If the scale_bmp property is FALSE, the image will be tiled (repeated) to fit the dimensions of the label. If the scale_bmp property is TRUE, the image will be scaled to fit the label dimensions.

Command Button Properties

The following properties can be defined for a button object on a GUI form:

background	This property defines the background color for the button using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color. Note that this property is ignored when the tile property is used.
border_width	This property is a number of 0 to 5, representing the number of pixels used in the construction of the border. The greater the number, the wider the border. Note that the border_color and border_style properties are ignored if the border_width property is set to 0.
coordinates	This property defines the row and column (in the format *row*;*column*) where the object is placed in the GUI form (in pixels, in relation to the upper left corner of the GUI form).
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the object.
down_graphic	This property defines the graphic to be displayed for the button label when the button is "pushed" by clicking the mouse on it. This property is mutually exclusive of the down_string property.
down_string	This property defines the string to be displayed for the button label when the button is "pushed" by clicking the mouse on it. This property is mutually exclusive of the down_graphic property.
emphasized	This property has two values, which define the appearance of the text: • TRUE - The label text is shown normally. • FALSE - The label text is shown de-emphasized, i.e. the standard visual indicator for display-only prompts.
font	This property defines the font used for the text on the button. When changing this property, use the ellipsis button to select an appropriate font. Note that the font used must be installed on all of the user systems, otherwise SB+ will revert to the default prompt when necessary.
foreground	This property defines the foreground (text) color for the button using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color.
graphic	Either a text string (using the string property) or a graphic image may be used for the button label. If the latter is desired, this property defines the graphic image to be used. Note that this graphic image must be stored in a location which is available to the common user, otherwise it will not be displayed in the production environment.
help_string	This property defines a message that will be displayed when the user "floats" the mouse pointer over the object.

justification	This property defines the justification (alignment) of the string or graphic in the button label. Valid options include: • bottom_center • bottom_left • bottom_right • center • left • right • top_center • top_left • top_right
scale_bmp	This property is used with the graphic property, and defines whether the graphic is shown in its original size, or it it scaled to the size of the button area. Valid options include: • TRUE - The graphic is scaled to fill the button dimensions. • FALSE - The graphic is shown once in the button area, aligned according to the justification property.
string	Either a text string or graphic image (using the graphic property) may be used for the button label. If the text string is desired, this property defines the actual string to be displayed. Note that this property is ignored when the graphic property is used.
tile	This property can be used to define a background image for the button. This image can be used with the string property to display a string on top of an image. If the scale_bmp property is FALSE, the image will be tiled (repeated) to fit the dimensions of the label. If the scale_bmp property is TRUE, the image will be scaled to fit the label dimensions.

Separator Properties

border_style	This property defines the appearance of the separator line, and is one of three values: • flat - The line is on the same plane as the rest of the form, not raised or lowered. • raised - The line appears to make the object be "raised" off of the underlying form. • lowered - The line appears to make the object be "lowered" into the underlying form,
coordinates	This property defines the row and column (in the format *row*;*column*) where the object is placed in the GUI form (in pixels, in relation to the upper left corner of the GUI form).
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the object.
direction	This property controls the shading of the separator line, and can be one of two values: • horizontal • vertical For strictly horizontal and vertical lines, the effect of one setting over the other is slight. For separator lines where the dimensions are not flat (that is, the separator is more of a box than a line), the effect is more distinct.
foreground	This property controls the color of the separator line using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color.

Rectangle Properties

background	This property defines the background color for the rectangle using three numbers representing the red, green, and blue contributions to the overall color. The format of this entry is *redcolor*;*greencolor*;*bluecolor* where each color is a number between 0 (no color) and 255 (full intensity). When changing this property, use the ellipsis button to select the appropriate color from a chart of available colors. Or, use the Color Palette to select an appropriate color. Note that this property is ignored when the tile property is used.
border_color	This property defines the color of the rectangle border and is effective only when the border_style property is "flat" and the border_width is greater than zero.
border_style	This property defines the appearance of the label border, and is one of four values: • flat - The border is on the same plane as the rest of the form, not raised or lowered. • raised - The border appears to make the object be "raised" off of the underlying form. • lowered - The border appears to make the object be "lowered" into the underlying form, • msstyle - The border appears as it would in a typical Microsoft Windows application.
border_width	This property is a number of 0 to 5, representing the number of pixels used in the construction of the border. The greater the number, the wider the border. Note that the border_color and border_style properties are ignored if the border_width property is set to 0.
coordinates	This property defines the row and column (in the format *row*;*column*) where the object is placed in the GUI form (in pixels, in relation to the upper left corner of the GUI form).
dimensions	This property defines the width and depth (in the format *width*;*depth*) of the object.
tile	This property can be used to define a background image for the rectangle. Note that the background image will be tiled (repeated); there is no feature for scaling the background image with a rectangle object.

Using a Non-Amendable Field

In SB+, a *non-amendable field* is a field where the Allow Amend (Y/N) flag in the field definition is a "N" instead of a "Y". This setting tells SB+ that this field can be amended only if the record is new (i.e. the **common variable** @ACTION is "1"). Unfortunately, this setting cannot be overridden when the field is put on a screen, so it is often too restrictive to be of value in many applications.

To see how this can be used, and also how it can be a problem, let's assume we're creating a customer entry screen. On this screen we have a Credit Limit prompt that will allow a credit limit to be entered for new customers, but the field cannot be amended for existing customers. For this screen, the Allow Amend = "N" works exactly as we want. If the user is entering a new record they can enter the credit limit without incident, but the cursor will not move to the field when existing records are recalled for editing.

Following this example, let's assume we want to create a new screen that allows the credit limit to be edited. Because the credit limit field is set up in the field definition to disallow amendment, we cannot use this field on the new screen. Instead, we must create a second field that points to the same attribute position but instead has the Allow Amend flag set to a "Y". This is the only way we will be able to have a screen that can edit this prompt using the Allow Amend flag settings.

Fortunately, there are better ways to skip a prompt. See the other topics in this section for more information.

Using the Process Before Field

The best way to skip a prompt is using the Process Before on the F5-Field screen in Screen Definitions. To do this, you can simply call a paragraph that sets the **common variable** @RTN.FLAG to one of two values. If you want to skip to the previous prompt, set @RTN.FLAG to 1. To skip to the next prompt, set @RTN.FLAG to "S". Question is, how do you know which way to skip?

The common variable @OTHER(18) contains a number which describes the last edit key pressed. By checking this value, you can tell whether the cursor was moving through the screen moving forward, or whether the cursor was moving through the screen in reverse, using either the up arrow or backtab. If @OTHER(18) is a 3, the up arrow was the last edit key pressed. If @OTHER(18) = 25, the backtab was the last key pressed. In both cases, we'll want to skip back to the previous prompt. With all other edit key values, we'll skip to the next prompt. This can be put into the paragraph language as follows:

```
IF (@OTHER(18) = 3) OR (@OTHER(18) = 25) THEN
  EXIT 1
END ELSE
  EXIT S
END
```

I call this particular process SKIP. It has become a very important standard process in my own process library. With this one process, I can skip any single prompt on the screen.

Note, however, that SKIP would never be called directly from the Process Before on the F5-Field screen. If we did this, the field would always be skipped, which defeats the purpose of having an input field. Instead, SKIP should be called from a separate process which decides whether the prompt should be skipped, and then calls the SKIP process if necessary, such as the following:

```
IF (CUSTOMER.STATUS # "A") THEN
  EXEC 'SKIP'
END
```

This process, by whatever name, should then be listed as the Process Before on the F5-Field screen in the screen definition.

Skipping and Clearing an Input Prompt

Under certain conditions, you may want to not only skip the prompt moving forward, but also clear any value the field may have. This can be done using a variation of the SKIP process, one I call SKIP.CLEAR:

```
IF (@OTHER(18) = 3) OR (@OTHER(18) = 25) THEN
  EXIT 1
END ELSE
  DATA '\'
END
```

This process is very similar to the SKIP process, except that if the cursor is moving forward through the screen, the process **stacks** a backslash into the keypress buffer, which then clears any value that may be in the field.

Of course, it is important to verify that a null entry is valid for fields that are skipped this way. If the **validation** on the field fails, the cursor will stop on the field and the prompt will not be skipped as expected.

Can Multiple Prompts Be Skipped At Once?

SB+ does support skipping multiple prompts at once. However, you can skip a group of prompts only when moving forward through the screen -- you cannot skip a group of prompts moving in reverse through the screen.

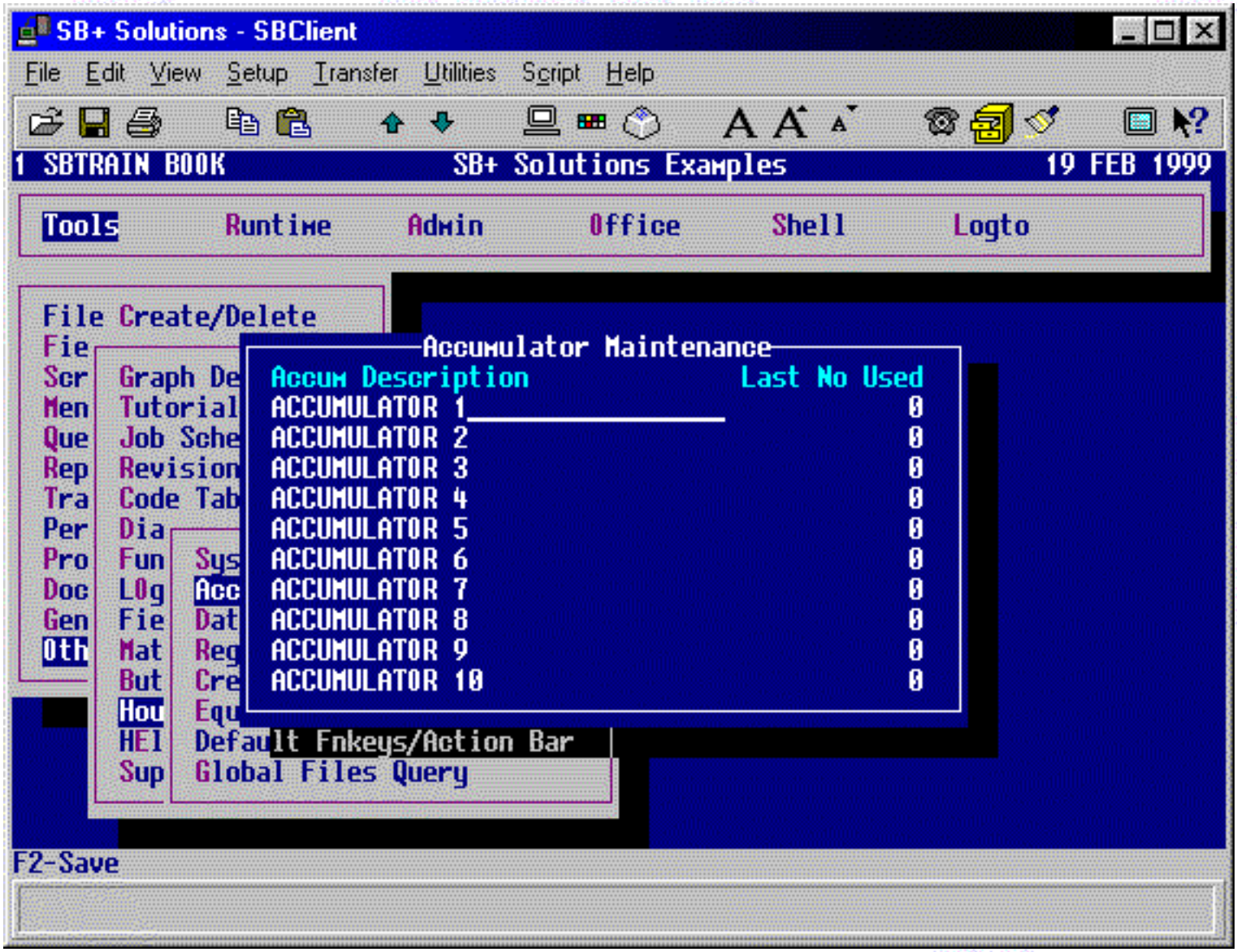
If I wanted to skip 6 prompts moving forward through the screen, I can set @RTN.FLAG to "S6" in a paragraph called from the Process Before on the F5-Field screen on the screen definition. This will certainly skip 6 input fields. However, there is nothing to restrict me from using the up arrow or backtab to access those fields. Therefore, skipping multiple prompt does not provide any level of protection against the field being updated, though it does provide a way of conveniently skipping forward through the screen.

The question therefore becomes this: Why are you skipping the prompt? Should the user be restricted from accessing the prompt? Or is the prompt being skipped simply as a way of moving the cursor to key fields to speed up entry of the main fields on the screen? These are valid considerations when setting up fields to be skipped on the screen, and must be considered together as both complimentary and contrasting techniques.

Using Accumulators

SB+ accumulators are managed using the Accumulator Maintenance tool. This tool may be accessed through the menus by selecting Tools, Other Tools/Utilities, House Keeping, and then Accumulator Maintenance. Or, for a more direct approach, /HK. ACCUM may be entered at any menu or entry prompt to start the tool.

This tool appears as follows:



The SB+ **expression language** generates sequential values from these accumulators using the following syntax:

Gn

In this syntax, n is a number which corresponds to the accumulator number in the Accumulator Maintenance tool. If the value stored for this accumulator was 10 prior to the execution of the expression, the value 11 will be returned and will serve as the basis for the next assigned number.

To look at this another way, the expression:

```
G7
```

...is equivalent to the following BASIC code:

```
READVU VALUE FROM F.CONT,'GENNO',7 ELSE VALUE = 0
VALUE = VALUE + 1
WRITEV VALUE ON F.CONT,'GENNO',7
```

As you can see, the SB+ expression syntax is significantly more compact than its BASIC counterpart!
The accumulator value may also be masked to be a certain number of digits in length, using the following syntax variation:

```
Gn,m
```

In this syntax variation, n still represents an accumulator, but the m parameter has been added to denote the field mask. For example, to mask the sequentially assigned value to five characters:

```
G7,5
```

...returns 00001 from accumulator 7 the first time the expression is used, 00002 the second time, 00003 the third time, and so on. Care must be taken when this variation is used, as eventually the numbers will reach a maximum (based on the number of digits) and will roll over.

Two issues must always be kept in mind when using SB+ accumulators:

- All accumulators are stored in one record for a given system. If your application relies heavily upon accumulators, it may be problematic to have all of the accumulators stored in one record in the system. This record will constantly be locked and unlocked, so there is always the risk of lock contention on a heavily shared system.
- SB+ does not check to see if a value has been used for a key prior to assigning the next sequential value. If a key value is assigned and a record already exists under that key, the old record will be overwritten, unless you specifically add code to verify this.

For these reasons, some people (myself included) choose to create their own version of accumulators based on a different structure.

Sequentially Assigning a Key as a Default

The simplest way to implement a sequentially assigned key value is to put a **default value** on the field definition for the key field. If the key value were to be assigned from accumulator 7, the default expression would be:

G7

With this default value, each time the cursor moves to the key field a new value will be assigned.

While this technique is certainly easy to implement, it does have one particularly interesting side-effect -- it wastes numbers. If a user presses <cr> to accept the sequentially assigned key value and then uses Esc to exit the record without saving, a new key value will be generated and the old value will be lost. The result, then, is a file with gaps in the key sequence. Many times, this is inadequate.

Fortunately, there are **other options**...

Sequentially Assigning a Key Prior to Saving the Record With Record Lock

The only way to sequentially assign a key and not waste numbers is to wait until the last possible moment before generating the key value. This "last possible moment" is in the **Process After Screen Accept** on the screen definition. The problem is, how do you know when a new key value is needed?

The generally accepted standard at this time is to assign a default value of "NEW" to the key value when new records are being added. In the Process After Screen Accept, if the key value is "NEW", we then replace the key value with a sequentially assigned value. The record will then be written using the new key.

The are three tasks that must be done to implement this technique:

- Set a **default value** of "NEW" on the key field
- Call a process after on the key field to make the key value unique
- Call a process after screen accept to assign the sequential key value before write

To set a default of "NEW" on the key field, go to the F5-Field screen for the key field, move the cursor to the Process Before slot, and enter the following:

```
D:("NEW")[M]
```

This syntax tells SB+ that the Process Before slot is being reassigned to be an override to the default on the field definition. Also, the [M] suffix tells SB+ that this default is mandatory, and regardless of the key value, the "NEW" default should always be assigned when the cursor moves to the key field.

So why not simply set the default on the field definition? Well, the key field will probably appear on both an input and output screen, and we don't want the "NEW" default appearing on the output screen. Therefore, by overriding the default on the input screen only, we can have the default for the input screen be different from the default for the output screen.

Now, about making the key value unique...

When the "NEW" default is used, SB+ sets a record lock using "NEW" as the key. If two or more users are trying to add new records to the file, SB+ will wait until the lock from the first user is cleared before letting the second user add a record.

To overcome this problem, we can append the port number to the "NEW" key in the Process Before, using the following syntax:

```
D:("NEW":@PORT)[M]
```

Some people like this technique because it is so easy to implement. Others dislike it because the default appears different for each port and users get confused. For those who dislike the approach, there is another alternative. Instead of changing the code in the Process Before slot, link a process called SET.NEW.LOCK to the Process After slot on the key field. This process is a paragraph that you create with the following line:

```
IF @KEY = 'NEW' THEN @KEY = @KEY : @PORT
```

When this process is called, the key value is changed prior to the record being read, but the new value is not displayed. This makes

the default on the screen appear as "NEW", even though the record has been locked using "NEW":@PORT as a key.

Finally, the last step in this process is to write a process to change the key value from the "NEW" key to an actual value. This process is a paragraph called from the Process After Screen Accept and appears as follows:

```
LOCAL DUMMY
*
IF @KEY[1,3] = 'NEW' THEN
  RELEASE 0,@KEY
  @RTN.FLAG = 0
  WHILE @RTN.FLAG = 0 DO
    @KEY = G7
    READ DUMMY FROM 'file',@KEY
  REPEAT
  EXIT 0
END
```

This paragraph first checks to see if the key value starts with "NEW". If it does, the process first releases the old record lock -- we don't want this staying around after we're done with the record. Next, the WHILE loop says to loop until a record is not found. Inside the loop we assign a key value and check to see if a record exists in file for this key. (In real life, file would be replaced with an actual file name.) If the record exists, @RTN.FLAG remains zero, and the loop continues until we find a key that is not used. This loop, then, prevents records from being inadvertently overwritten.

With these three tasks completed, the sequentially assigned key implementation is complete. However, there is a simple enhancement to this technique that can be most useful.

In certain applications, new records should be added with the "NEW" key only, instead of allowing the user to enter new records with any key. To restrict the screen to allow "NEW" keys for only new records, create this process (called VAL.NEW.KEY) and link it to the Process After Read in the F6-Params window on the screen definition:

```
IF (@ACTION = 1) THEN
  IF (@KEY[1,3] # 'NEW') THEN
    ERROR 'New records must be added using the "NEW" key'
    RELEASE 0,@KEY
    EXIT 1
  END
END
```

This process simply checks to see if the key starts with "NEW" when @ACTION is equal to 1 (i.e. the record is new). If the key starts with anything but "NEW", the error message is displayed, the lock released, and the cursor moves back to the key field (via the EXIT 1).

Sequentially Assigning a Key Prior to Saving the Record Without Record Lock

This technique is very similar to the previous technique, but doesn't lock the record. Or, more accurately, it doesn't lock the record for long. To implement this technique, the following tasks must be done:

- The "NEW" **default** must be set
- A **Process After Read** must be created to unlock records with the "NEW" key
- A **Process After Screen Accept** must be created to change the "NEW" key to an actual key value.

To set a default of "NEW" on the key field, go to the F5-Field screen for the key field, move the cursor to the Process Before slot, and enter the following:

```
D:("NEW")[M]
```

In the Process After Read, we need some code to unlock the record when a new record is being added. This can be accomplished using the following paragraph line:

```
IF @KEY = 'NEW' THEN RELEASE 0,@KEY
```

This can be a paragraph unto itself, or can be integrated into an existing paragraph as needed. SB+ will still lock the record after the key has been entered, but this process, called immediately after, will unlock it.

Note also that we don't append the port number to the key field, as we did in the previous technique. Because we're immediately unlocking the record, this extra step is unnecessary. However, on a slow system with several users adding new records, momentary lock contention is still possible. When this happens, it may be necessary to add the Process After on the F5-Field screen as described for the previous technique.

Finally, we need a process called from the Process After Screen Accept to translate the "NEW" key to an actual key value. Fortunately, the process used for this purpose for the previous technique will work just fine for this technique as well:

```
LOCAL DUMMY
*
IF @KEY[1,3] = 'NEW' THEN
  RELEASE 0,@KEY
  @RTN.FLAG = 0
  WHILE @RTN.FLAG = 0 DO
    @KEY = G7
    READ DUMMY FROM 'file',@KEY
  REPEAT
  EXIT 0
END
```

Validating Against Another File

One of the most popular validation codes is the F: code, which is used to verify whether a record exists in another file with a key equal to the value just entered. With such a powerful relational data management system under SB+, perhaps you can see how valuable such a validation can be.

For example, assume for a moment we're creating a sales order entry screen. One of the prompts on this screen is called Customer ID, which is a reference to a record in a file called CUSTOMERS. Therefore, when an entry is made at this prompt, we should verify that there is a corresponding record in the CUSTOMERS file.

The F: code is exactly the code to do this. Following the F: there are up to three parameters, as follows:

F:*file,field,onfile*

The *file* parameter simply defines the name of the file to read. Whatever is in @VALUE at the time (i.e. the user's last entry) will be used for the key value in attempting to read a record from the file.

The *field* parameter, if supplied, names a field in the record to display at the bottom of the screen following successful validation. Therefore, if you want to show the customer name at the bottom of the screen after the validation, simply place the name of the customer name field in this parameter. This is an optional parameter, and many people choose to leave it blank and use other means (such as a **derived value field** shown on the screen) to show the customer name.

The *onfile* parameter is either a "Y", "N", or "A" which defines whether the record must be on file ("Y" or "A"), or the record cannot be on file ("N"). (When "A" is used, the record must be on file, but a null entry is considered valid.) This parameter is also optional, and is assumed to be "Y" if not provided. (Note that if this parameter is used, but the field parameter is not, there will be 2 commas between the file name and this flag.)

With this in mind, the following F: validation codes are equivalent and could be used for the customer validation:

```
F:CUSTOMERS,,Y
F:CUSTOMERS
```

An E: validation can also be used for this type of verification, but performs differently. Using the previous example, an E: validation could be constructed as follows:

```
E:(F('CUSTOMERS',@VALUE) # ""){message}
```

This validation basically says "read a record from the CUSTOMERS file using @VALUE as the key. If the record is not null, accept it. Otherwise, display the error message. What sets this validation apart from the F: validation is that the E: validation checks to see if the record is null, not necessarily whether the record exists. Fortunately, if the F(...) function does not find a record it will return a null value, but then again it will also return a null value if it reads an existing record that is null. Therefore, if customer records could be null, the E: validation is probably not an acceptable option. On the other hand, if you want to validate that the record exists and is not null, the E: validation is not only acceptable, it's one of your best options.

This particular E: validation code also has a hidden benefit. If the record is on file, it will be in the **common variable** @OTHER.REC, and can be used in later validations on the same screen. To see how this is applicable, assume we're creating a sales order entry screen. Somewhere on this screen we'll need to enter inventory items to be sold to the customer. For each inventory item,

we'll want to display the description of the item and validate the price against a list of acceptable prices for the item. Therefore, on the inventory ID prompt we'll need to validate the inventory item is on file, but on the description field and the price field we can use @OTHER.REC to look up the description and valid prices in the inventory record. This allows us to use the inventory record three times while reading it only once, which will help system performance substantially.

A paragraph may also be written to perform the same validation. As an example, the following paragraph combines the best of the F: code (verifying the record exists, rather than checking for a null record), and the best of the E: code (leaving the record in @OTHER.REC):

```
READ @OTHER.REC FROM 'CUSTOMERS',@VALUE
IF @RTN.FLAG THEN
  ERROR 'message'
END
```

Of course, these paragraph lines can be used as-is, or combined with other lines for a complex validation.

Finally, the E: code can be replicated in a paragraph as well, using something like the following:

```
IF F('CUSTOMERS',@VALUE) = "" THEN
  ERROR "message"
END
```

So, as you can see, there are a myriad of options available to you for validating against another file. Which of these formats works best for you will depend largely on your project, your standards, and your programming style.

Validating the Length of an Entry

To validate that a field is a certain number of characters in length, the L: code is used. The format of the L: is as follows:

```
L:n
```

In this syntax, n is a number which defines the proper length for the field. Therefore, to validate that a state field is 2 characters long, the following code can be used:

```
L:2{message}
```

The E: validation code can also be used to validate the length of the field. Using the above example, an E: validation code can be written as follows:

```
E:(LEN(@VALUE) = 2){message}
```

A paragraph can also be written to validate the length of the entry, as follows:

```
IF (LEN(@VALUE) # 2) THEN
  ERROR "message"
  EXIT 1
END
```

Such a paragraph can then be called with the C: validation code, or integrated into another validation paragraph.

Validating Against a Pattern

A pattern match validation is similar to a length validation, except that the type of entry is checked as well as the length. This is accomplished using the P: validation code, shown as follows:

```
P:pattern{message}
```

In this example, pattern is a standard Pick "match" pattern. These patterns are constructions of numbers and letters which define the format of the entry. Consult any Pick or Pick-like manual for more information on the format of patterns.

Pattern match validations are especially popular for validating phone numbers. For example, if a phone number must be nnn-nnn-nnnn or nnn-nnnn, the following validation may be used:

```
P:3N'-'3N'-'4N OR P:3N'-'4N{message}
```

Of course, this works only for USA and Canadian phone numbers. For other international phone number formats this particular series of validation codes is too restrictive. Therefore, it may be necessary to create one process (either a validation or paragraph process) for validating USA and Canadian phone numbers, and other processes for other international formats.

The following paragraph is equivalent to the pattern match validation shown above:

```
IF MATCHES(@VALUE,"3N'-'3N'-'4N") = 0 THEN
  IF MATCHES(@VALUE,"3N'-'4N") = 0 THEN
    ERROR "message"
    EXIT 1
  END
END
```

You may be wondering why I would choose to create a paragraph instead of a standard validation process for the phone number validation. The reason is really very simple: In a paragraph we can change the value prior to validating it. We can't do this with any other type of validation option. Therefore, if the phone number is not entered in the right format, we can simply reformat it (if possible) before checking the entry.

Following this thought, if we wanted the phone number validation to add the dashes automatically, we could say something like: "If 7 or 10 numbers are entered, add the dashes automatically prior to validation. Otherwise, validate the phone number as entered." This can easily be put into a paragraph as follows:

```
LOCAL LEN.VALUE
*
@VALUE = OCONV(@VALUE,'MCN') ;* Strip all punctuation
LEN.VALUE = LEN(@VALUE)
*
CASE (LEN.VALUE = 10)
  @VALUE = @VALUE[1,3] : '-' : @VALUE[4,3] : '-' : @VALUE[7,4]
  @REFRESH = 7 ;* Redisplay this value
CASE (LEN.VALUE = 7)
  @VALUE = @VALUE[1,3] : '-' : @VALUE[4,4]
  @REFRESH = 7 ;* Redisplay this value
CASE 1
  ERROR "message"
```

```
  EXIT 1
END CASE
```

If you like challenges, how would you change this process to allow both numbers and letters? In other words, if we wanted the process to support phone numbers like 800-ILOVESB (is that a real number?), what changes would be required?

Of course, there are many other places where pattern match validations can be used. With a good knowledge of how to construct patterns, this particular code can be very useful in many areas of your application.

Validating Against a Range

When entering percentages, you may want to verify that the entry is between 0 and 100 percent. This is most easily done with the R: validation code, which is structured as follows:

```
R:lo TO hi{message}
```

In this example, lo is the lowest number in the range. The entry must be greater than or equal to this value to be valid. The hi value, in contrast, is the upper limit of the range. The entry must be less than or equal to this value to be valid. Therefore, to validate percentages between 0 and 100 percent, the following validation could be used (assuming no input conversion):

```
R:0 TO 100{message}
```

From a paragraph's perspective, the following code is equivalent to the above example:

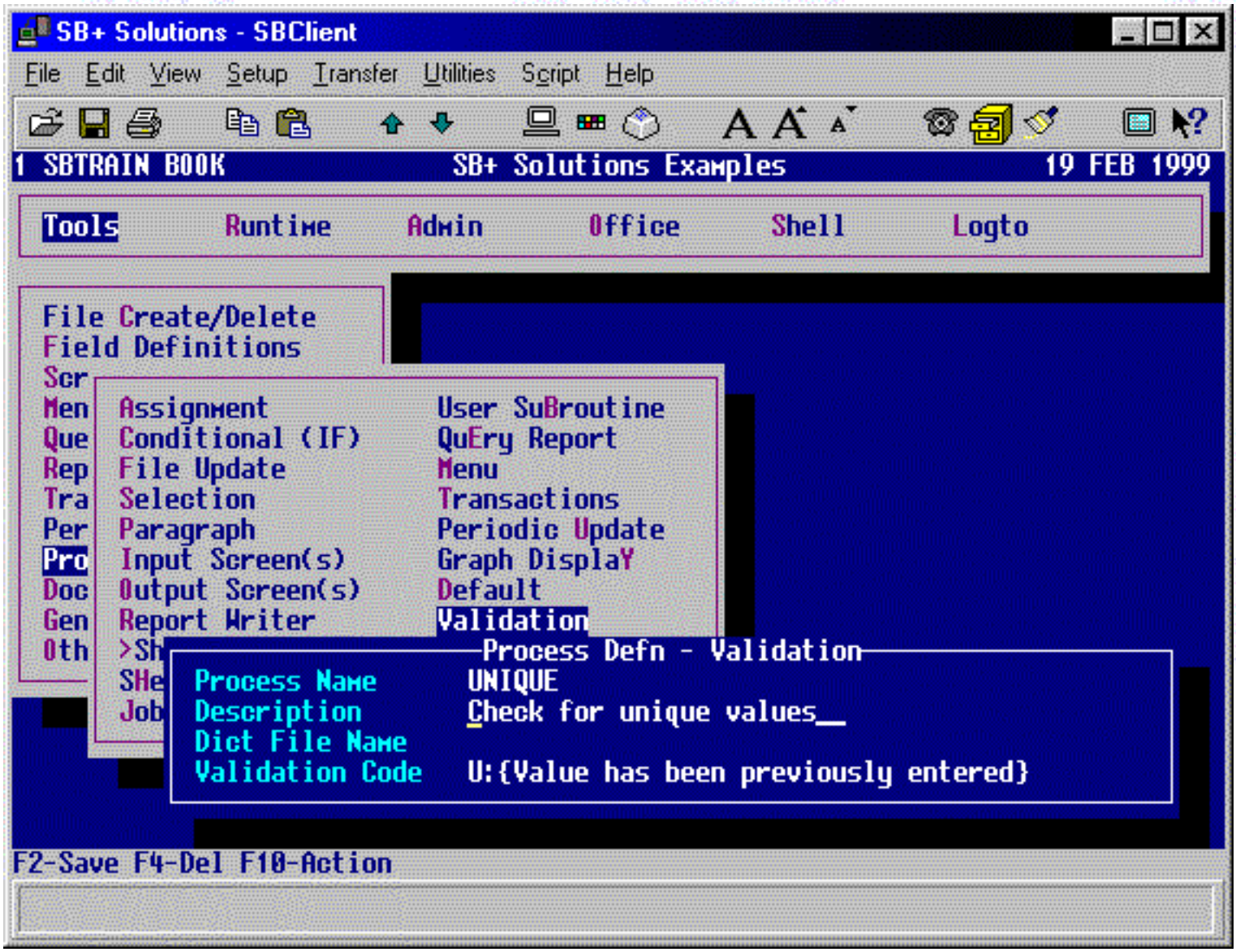
```
IF (@VALUE < 0) OR (@VALUE) > 100) THEN
  ERROR "message"
  EXIT 1
END
```

Incidentally, the range validation (either using the R: or the paragraph) is not limited to numeric values only. Alpha fields and date fields can also utilize this validation code if required.

Validating A Multivalued Entry is Unique

In the above example there is a reference to a process called UNIQUE. This particular process is one of the smallest SB+ standard processes I've ever created, but is undoubtedly one of great value.

UNIQUE is a validation process, entered using the /PD.V tool. Simply stated, it's a U: validation with a message, as follows:



This dinky little validation code takes care of all of the headwork required to verify that an entry into a multivalued list is unduplicated. With this code in place, if I enter a value into a multivalued list more than once, the message is output and, more important, @RTN.FLAG is set to 1 to reprompt for the entry.

A test for uniqueness is not typically a main validation, therefore you will rarely see a validation as follows:

C:UNIQUE

Instead, this validation is typically a second or subsequent test, as shown in the earlier table validation example.

Different people have different opinions on this matter, however. Some feel that the uniqueness test should be first, followed by the test for a proper entry. Others feel the test for the proper entry should be first, and only then should you test for uniqueness. Personally, I feel that if the test for a proper entry is long and complicated, test uniqueness first and proper entry second. This will allow the unique test, which is very fast, to rule out entries before going through what could be a long and complex series of validations for proper entry. If the proper entry validation is short and to the point, do it first and follow up with the test for uniqueness. This simple rule of thumb will maximize the efficiency of your product and minimize the guesswork in structuring complex validations.

Now, it may seem superfluous to have a standard process where the name is actually 300% bigger than the code itself. Couldn't we use something like the following instead?

C:VAL.TABLE,table AND U:{message}

SB+ certainly supports combining validation codes using AND, and this is syntactically correct, but it has one major drawback: **What message will be displayed?**. When validating on several different criteria, it's always best to have several different error messages so the user can tell exactly what is wrong. This is always better than an error message like "Either the entry is incorrect or duplicated", which leaves the users thinking your program is too stupid to figure out which.

Validating Against a List

Code entries, such as a state code or country code, may be easily validated against an acceptable list of options. This can be done using the V: validation code.

The V: validation code is followed by a comma-delimited list of acceptable values. For example, if an entry must be either A, B, or C, the following V: code could be used:

V:A,B,C{message}

This format is very easy to program, but isn't very flexible. If the list needs a new code, or one of the existing codes needs to be removed, a software change is required. Therefore, the V: code should be used only if the list is fixed and will rarely, if ever, require modification.

The Single Message Dilemma

Following the validation code, an error message may be entered. The message must be enclosed in braces, and it must be the last part of a validation code.

For example, if a particular prompt is supposed to validate that an entry is an integer with no spaces, the following validation can be used:

```
I: AND S:{Entry must be an integer with no spaces}
```

When the error message displays, it tells the user there is one of two possible reasons why the validation failed. While this certainly covers all of the bases, it would be better if we could display separate messages for each part of the validation, such as:

```
I:{Entry must be an integer} AND S:{Entry must not have spaces}
```

As of SB+ version 3.x, this is legal. However, due to the length of the messages, adding more than one error message to a validation code makes the code more difficult to read. Instead, if you want SB+ to select one (or more) of several error messages, the best option is to create a paragraph to do the validation, such as the following:

```
IF INT(@VALUE) # @VALUE THEN
  ERROR "Entry must be an integer"
  EXIT 1
END
*
IF INDEX(@VALUE,' ',1) THEN
  ERROR "Entry must not have spaces"
  EXIT 1
END
```

In this paragraph we're essentially recreating the work that the I: and S: validation codes perform, and displaying appropriate messages for each condition. While the extra work of creating a paragraph may seem unnecessary, it illustrates that the smoke and mirrors behind many standard SB+ functions are easily recreated to allow you the ultimate in flexibility in your application development

Errors vs. Warnings

When validating an entry, one of two things could happen. If you want to reject the entry altogether, you'll want to display an error to tell the user why the entry is unacceptable. In contrast, there may be times when you'll want to display a warning, allowing the user to accept the entry despite the message.

When anything but the C: validation code (Call Process) is used, SB+ can only reject the entry and display an error. It can never be used to display a warning. However, when the C: validation code is used, SB+ is not so limited. Processes called for validation can display errors and reject an entry just like any other type of validation code, and can also display warnings, allowing the users to continue past the messages.

As far as the message is concerned, there is nothing in a message that differentiates a warning from an error. Instead, the secret lies in the **common variable** @RTN.FLAG.

When a validation process is called and terminates with @RTN.FLAG = 0, the entry will be accepted. Any number of warning messages may be displayed by the process, but if @RTN.FLAG is zero when the process is done, the entry will be accepted. In contrast, if @RTN.FLAG is 1 when the process terminates, the entry will be rejected. Keep in mind that anything which sets @RTN.FLAG to a 1 will reject the entry, whether the entry should be rejected or not. Therefore, you as the developer must always be cognizant of the value of @RTN.FLAG when developing validation processes.

When reviewing the following sections, keep a close eye on the EXIT 1 statement in the example paragraphs. This statement will exit the paragraph at that point and set @RTN.FLAG to 1, which is the signal to SB+ to reject the entry. This undoubtedly violates the single entrance/single exit paradigm of structured programming, but is the generally accepted technique for the development of processes in SB+ .

Error Message Formats

We've seen how error messages follow the validation code, enclosed in braces. Error messages can appear in several forms inside the braces, however. The following sections describe the most common error message formats:

Literal Error Messages

Literal error messages are by far the easiest to implement, but are the least flexible of all message types. The format of a literal error message is as follows:

{message}

With this type of error message, the message text is stored with the validation code, which means that it is actually a part of the field definition. Because the field definition is actually part of the software, this type of error message is, as well, a part of the software. Therefore, if the error message has to change, it requires a change to the software. If other changes have been made to that particular field definition, the differences will need to be rationalized prior to upgrading a user account with the new field definition. Otherwise, the simple task of installing a new validation message could theoretically damage existing functional code.

User Error Messages

If you use F3 on the Validation Code prompt on the Field Definitions tool, somewhere along the line SB+ will ask you to enter an error message. Following the entry of the error message, SB+ will ask you if you want to update the user message item. If you say "Y" to this prompt, your error message will go away (supposedly), and SB+ will replace it with something like this:

{[U21]}

In this example, the error message is referred to as a user message -- message number 21, to be specific. SB+ assigns new sequential numbers for each message, so the first message for a new system will be message 1, and the numbers will be sequentially assigned from there. In reality, the user messages are stored in a record in your XXXCONTROL file called MESSAGE. Message number 1 will be stored in attribute 1 of this record. Message number 2 will be stored in attribute 2 of this record, and so on. Therefore, as you've probably figured out, all messages for a given system are stored in the same record.

Using this error message format, the message is not a part of the software, per se, because it is not stored inside of the field definition. However, because all error messages are stored in the same record, if one error message changes, the entire suite of messages for a given system must be transferred to your production environment to effect the change.

Another problem with this technique is the access time to display an error message. If a system has 1000 error messages in the user message item, each time any error message is output all 1000 error messages will be read in order to display the one. For systems with more than a couple of users, this can slow the system down immensely.

Reading a Message from an SB+ Control Record

Incidentally, SB+ uses a technique similar to the user message record for displaying its own messages. SB+ error messages are stored in the DMCONT file, in records named ERROR, PROMPT, MESSAGE, and DIALOG. The format of these error message records is exactly the same as a typical user message item: one message per attribute, multiple attributes per record.

If so see...	The message is in...
[E5]	DMCONT ERROR<5>
[P2]	DMCONT PROMPT<2>
[M6]	DMCONT MESSAGE<6>
[D1]	DMCONT DIALOG<1>

There are certainly no restrictions on using the SB+ error messages in your own application. However, there is also no guarantee that the messages will stay in the same places from one release of SB+ to another. Therefore, if there is even the most remote possibility that your application will be run using different versions of SB+, you should not reference SB+ standard error messages in your software.

Reading a Message from an External Source

The most flexible method for displaying error messages is to store them in an external file, one message per record, and use the following format to display the error:

{[Ufile,itemid<attr>]}

This format says "read a record from file, using itemid as the key. Extract attribute number attr and display it. Note that all three parameters are required, even if the record contains only one attribute.

With this format, if one message changes, only that record will need to be transferred to a customer to affect the change. Also, because error messages can be read from any file, they are not directly a part of the software and can therefore be changed at will without software modification. On the minus side, however, you will need to create a screen for entering error messages into your message file. However, this being the only negative to the technique (not counting the esoteric syntax), it is far and away the recommended technique for storing and displaying error messages.

Variable Error Messages

Sometimes it can be helpful to display variable information inside of the error message. For example, if a person enters "XY" at a State prompt, a message stating:

'XY' is not a valid state

can be used instead of:

Invalid state.

To do this, we have three alternatives:

- Embed an **expression** into the text of the message;
- Pass parameters to the message;
- A combination of the two.

Embedding an Expression into the Error Message

Any expression may be referenced in the error message text, simply by putting the expression within parentheses in the message. For example, the error message shown earlier can be entered as:

'(@VALUE)' is not a valid state code

The trick to this error message is knowing that @VALUE has the last value the user entered. The error message processor then extracts the expression (@VALUE) from the message, processes it, and replaces it with the actual value before the message is displayed.

Any expression can be called from within the parentheses. Therefore, if you want to call a process to display a particular error message, the error message text can be entered as:

(P('*processname,errornumber*'))

This particular syntax will call a process named *processname* and will pass *errornumber* as a parameter. The process may then use this error number to read information from another file (or whatever) and then display the message.

Note that this syntax will process anything between parentheses as an expression. Therefore, if you put parentheses as part of the message text, the text between the parentheses will be extracted and treated as an expression, not as text. Therefore, a message such as the following will not display the message you expect:

Customer not on file. (Try F3.)

In fact, the parentheses will not be shown at all!

Passing Parameters into the Error Message

For any error message referenced using the {[whatever]} syntax, literal parameters may be passed to the message by adding a comma followed by a list of parameters (separated by commas) immediately preceding the right bracket. Therefore, if the error message is stored in attribute 1 of the file ERRORS with an ID of "54", and we wish to pass the literal "WA" to the error message, the following syntax may be used:

[UERRORS,54<1>,State]

The error message is then formatted as follows to receive the parameter:

This \1\ is not on file.

Prior to the message being displayed, SB+ will look for any parameters in the text. Any \1\ references will be replaced by the first parameter, \2\ by the second parameter, etc. Any number of parameters may be passed to the error message this way, and parameters may be placed any number of times in the message, as in:

This \1\ is not on file. Press F3 for a list of \1\s.

Passing Parameters to an Error Message with an Embedded Expression

The two techniques of passing parameters to an error message and embedding an expression in the text can be combined in the same error message with the following message format:

"(@VALUE)" is not a valid \1\ code.

The error message can then be referenced in the field definition using the following syntax:

[UERRORS,54<1>,State]

Note that this is the same as in the previous example, where the literal parameter is passed as part of the error message reference. Following this example, if "XY" were an invalid state code, this message would appear as:

"XY" is not a valid State code.

Validating a Date Greater Than Today

If a date entry must be greater than or equal to the current date, the following expression-type validation may be used:

```
E:(@VALUE>=@DATE){message}
```

Like any E: validation, if the result of the expression is non-zero (true), the validation passes. Conversely, if the result of the expression is zero, the validation rejects the entry.

This same technique can be used with a C: validation code (Call Process) as well, where the paragraph process (specified following the C:) does the following:

```
IF @VALUE < @DATE THEN
  ERROR "message"
  EXIT 1
END
```

As stated earlier, the C: technique is the best way to handle **multiple validations**. In this particular example, note that the expression in the paragraph code tests for failure, while the expression in the E: validation tests for success.

To break these paragraph lines down, the first line tests to see if the condition is true. If the date entered is less than the current date, we display an error and set @RTN.FLAG to 1 to reject the entry. The END then closes the IF..THEN statement block.

Validating a Date Range

When validating a date range, there are two values that must be checked -- a beginning date and an ending date. Of course, the beginning date must be prior to or the same as the ending date, and the ending date must be after or equal to the ending date. This is all very straightforward. However, it can become complicated if one or the other is not a required entry.

You can never assume that the beginning date will have a value when validating the ending date. If the user presses the up arrow to cycle through the screen prompts in reverse, the ending date could very well be entered prior to the beginning date. Always remember that even though the prompts on a screen fall into a certain sequence, there is nothing forcing the user to follow that prompt sequence when the screen is running. Therefore, never make any
assumptions about the field prompting order on a screen.

It is also important to remember that both the beginning and ending dates must be validated. Because the dates could be entered in any order, it is important to validate the beginning date if the ending date has been entered first, or validate the ending date if the beginning date has been entered first. Again, never assume anything about the prompt order.

For the beginning date, therefore, the following expression validation could be used:

```
E:IF(END.DT#"",@VALUE<=END.DT,1){message}
```

This expression basically states that if the ending date has been entered, verify that the beginning date value is less than or equal to it. Otherwise, if the ending date has no value, return a true result (a.k.a. the literal "1") to the validation, forcing its success.

For the ending date, we could use the following E: validation:

```
E:IF(BEGIN.DT#",@VALUE>=BEGIN.DT,1){message}
```

This basically states the converse of the previous expression. If the beginning date has been entered, verify that the ending date is greater than or equal to it. If no beginning date has been entered, return a true result to the validation, forcing its success.

Using a paragraph, the beginning date validation would appear as follows:

```
IF END.DT # " THEN
  IF @VALUE > END.DT THEN
    ERROR "message"
    EXIT 1
  END
END
```

... and for the ending date...

```
IF BEGIN.DT # " THEN
  IF @VALUE < BEGIN.DT THEN
    ERROR "message"
    EXIT 1
  END
END
```

Personally, I prefer the paragraph approach because with a few minor tweaks these processes can be generalized to be used anywhere I need date range validation in the software. Also, if the date range validation is to be combined with other validation, the standard processes can easily be called from other validation processes, virtually eliminating the duplication of this code throughout the software.

Validating a Weekday Entry

The standard Pick internal date is a number that represents the number of days that have elapsed since Sunday, December 31, 1967. This red-letter day is often referred to as "day zero".

Because the internal date is merely a sequential number that is incremented each day, the current day of the week can be attained by taking the remainder (or *modulo*) of the internal date divided by 7. If the remainder is 0, the day is a Sunday. If remainder is a 6, the day is a Saturday.

If a particular field were to accept only weekday (Monday through Friday) entries, the following validation code could be used:

```
E:(MOD(@VALUE,7) > 0 AND MOD(@VALUE,7) < 6){message}
```

However, this would probably not be a recommended technique, as it requires the MOD(...) function to be executed twice. A better technique, then, would be to call a paragraph to do the validation, which executes the MOD(...) function only once:

```
LOCAL DAYNO
*
DAYNO = MOD(@VALUE,7)
IF (DAYNO < 1) OR (DAYNO > 5) THEN
  ERROR "message"
  EXIT 1
END
```

Like any paragraph used for validation, these lines could either be used alone or combined with other validation codes to build any level of complex validation required. Also, while it is true that some implementations support a conversion code which calculates the day number, the MOD(...) technique as shown above is the most portable code across all platforms.

Validating an End of Month Entry

Occasionally it can be useful to verify that a date is the last day in a given month. This has long been a problematic issue for programmers, especially when factoring in the various leap year exceptions. With Pick and Pick-like systems, however, this type of validation can be implemented reasonably easily, without worrying about leap years.

To validate that a date is an end of month, we first need to know the correct value for the end of the current month. Rather than trying to figure out all the leap year exceptions, we can simply calculate the internal number for the first day of the next month, then back it off one day. This can be done in a validation process (called using the C: validation code) as follows:

```
LOCAL CURR.DT,THIS.MM,THIS.YY,NEXT.MM,NEXT.YY,EOM
*
CURR.DT = @DATE ;* Get the current date
THIS.MM = CURR.DT"DM"
THIS.YY = CURR.DT"DY"
*
IF THIS.MM = 12 THEN
  NEXT.MM = 1
  NEXT.YY = THIS.YY + 1
END ELSE
  NEXT.MM = MM + 1
  NEXT.YY = THIS.YY
END
*
EOM = ICONV(NEXT.MM : '-01-' : NEXT.YY,'D') - 1
*
* Do the validation
*
IF (@VALUE # EOM) THEN
  ERROR 'This is not the end of the month.'
  EXIT 1
END
```

This particular paragraph works fine with American dates which are formatted "month-day-year". For international dates, formatted "day-month-year", the process would require modification to the calculation of the EOM variable, as follows:

```
EOM = ICONV('01-' NEXT.MM : '-' : NEXT.YY,'D') - 1
```

The question is, how can this be done in one process without having a version for American dates and a separate version for international dates? Read the **next section** to find out.

Calculating Date Format

Using a simple paragraph, we can ascertain whether our system uses American or international date formats. This paragraph appears as follows:

```
IF ICONV('13-12-96','D') = '' THEN
  @VALUE = 'A'
END ELSE
  @VALUE = 'I'
END
```

If this small paragraph were called DATE.FORMAT, we could call this process to determine which format of dates we're using. However, we couldn't easily incorporate this into the validation for end-of-month -- can you spot the problem?

```
LOCAL CURR.DT,THIS.MM,THIS.YY,NEXT.MM,NEXT.YY,EOM
*
CURR.DT = @DATE ;* Get the current date
THIS.MM = CURR.DT"DM"
THIS.YY = CURR.DT"DY"
*
IF THIS.MM = 12 THEN
  NEXT.MM = 1
  NEXT.YY = THIS.YY + 1
END ELSE
  NEXT.MM = MM + 1
  NEXT.YY = THIS.YY
END
*
IF P('DATE.FORMAT') = 'A' THEN
  EOM = ICONV(NEXT.MM : '-01-' : NEXT.YY,'D') - 1
END ELSE
  EOM = ICONV('01-' : NEXT.MM : '-' : NEXT.YY,'D') - 1
END
*
* Do the validation
*
IF (@VALUE # EOM) THEN
  ERROR 'This is not the end of the month.'
  EXIT 1
END
```

When the DATE.FORMAT process is called, @VALUE returns either 'A' or 'I' to this process. Unfortunately, @VALUE previously held the entry being validated. In short, after DATE.FORMAT has been called we no longer have a value to validate!

The following variation on this theme fixes this problem:

```
LOCAL CURR.DT,THIS.MM,THIS.YY,NEXT.MM,NEXT.YY,EOM,DT.FMT
LOCAL HOLD.VALUE
*
HOLD.VALUE = @VALUE
DT.FMT = P('DATE.FORMAT') ;* Get the date format (A or I)
@VALUE = HOLD.VALUE
*
CURR.DT = @DATE ;* Get the current date
```

```
THIS.MM = CURR.DT"DM"
THIS.YY = CURR.DT"DY"
*
IF THIS.MM = 12 THEN
  NEXT.MM = 1
  NEXT.YY = THIS.YY + 1
END ELSE
  NEXT.MM = MM + 1
  NEXT.YY = THIS.YY
END
*
IF DT.FMT = 'A' THEN
  EOM = ICONV(NEXT.MM : '-01-' : NEXT.YY,'D') - 1
END ELSE
  EOM = ICONV('01-' : NEXT.MM : '-' : NEXT.YY,'D') - 1
END
*
* Do the validation
*
IF (@VALUE # EOM) THEN
  ERROR 'This is not the end of the month.'
  EXIT 1
END
```

In this paragraph, note how we save the contents of @VALUE prior to calling DATE.FORMAT, and then restore the contents of @VALUE after we've set the DT.FMT variable. Later then, we can use the DT.FMT variable without worrying about changing @VALUE.

Verifying X Greater Than Y

Assume for a moment we're constructing a customer screen which prompts for monthly customer income and a credit limit for the customer. By definition, the credit limit must not exceed the customer's monthly income.

To set up such a validation, we need to sequence the prompts on the screen so that the customer's income is prompted before the credit limit. After the customer's income has been entered, we then want to validate the credit limit field using the following code:

```
E:(@VALUE <= MONTHLY.INCOME){message}
```

Unfortunately, we cannot be assured that the user will enter the monthly income first, even though the prompts are sequenced that way. The user may use the arrow keys to move through the screen in reverse, or the monthly income field may be optional. If the monthly income field has no value, we should probably skip the validation, using the following validation code:

```
E:(IF(MONTHLY.INCOME # "",@VALUE <= MONTHLY.INCOME,1){message}
```

In this example, if the monthly income has a value, we perform the standard validation from the previous example. If the monthly income has no value, however, we return a 1 (or "true"), which causes the validation to succeed.

This validation can also be expressed in a paragraph process, using the following lines:

```
IF (MONTHLY.INCOME # "") THEN
  IF (@VALUE > MONTHLY.INCOME) THEN
    ERROR "message"
    EXIT 1
  END
END
```

Like all paragraphs called for validation, this process (by whatever name you decide) would be called using the C: validation code.

Note that these aren't exactly equivalent. The paragraph code validates for failure, whereas the E: validation code validates for success. As a result, the E: validation code allows negative numbers to be entered, whereas the paragraph will not.

Validating a Number is Prime

This is one of the esoteric examples in this book, but nonetheless demonstrates a useful technique. In this example, we're going straight to the paragraph to verify that a number is evenly divisible by only itself and 1, using one of the hundreds of prime number calculation routines available today. This paragraph appears as follows:

```
LOCAL NDX
*
@RTN.FLAG = 0
IF MOD(@VALUE,2) = 0 THEN
  @RTN.FLAG = 1
END ELSE
  NDX = 3
  WHILE (NDX < @VALUE) AND @RTN.FLAG = 0 DO
    IF MOD(@VALUE,NDX)= 0 THEN
      @RTN.FLAG = 1
    END
    *
    NDX = NDX + 2
  REPEAT
END
```

Several different issues are explored in this paragraph. First, the loop variable is declared as a **local variable**, rather than using a **common variable** for the loop index. This ensures that this paragraph will not conflict with any common memory used by any other process that may call it. Second, this paragraph initializes @RTN.FLAG to zero at the start to denote the assumption that the number is prime. Only when the MOD(...) function inside the loop returns a zero value will @RTN.FLAG be set to a 1, which terminates the loop and exits the paragraph. Finally, note that there is no error message being output by this paragraph. Because this type of paragraph could be used both as a validation and also for other purposes, the error message is not a part of the paragraph, but instead is a part of the C: validation code on the field definition, as follows:

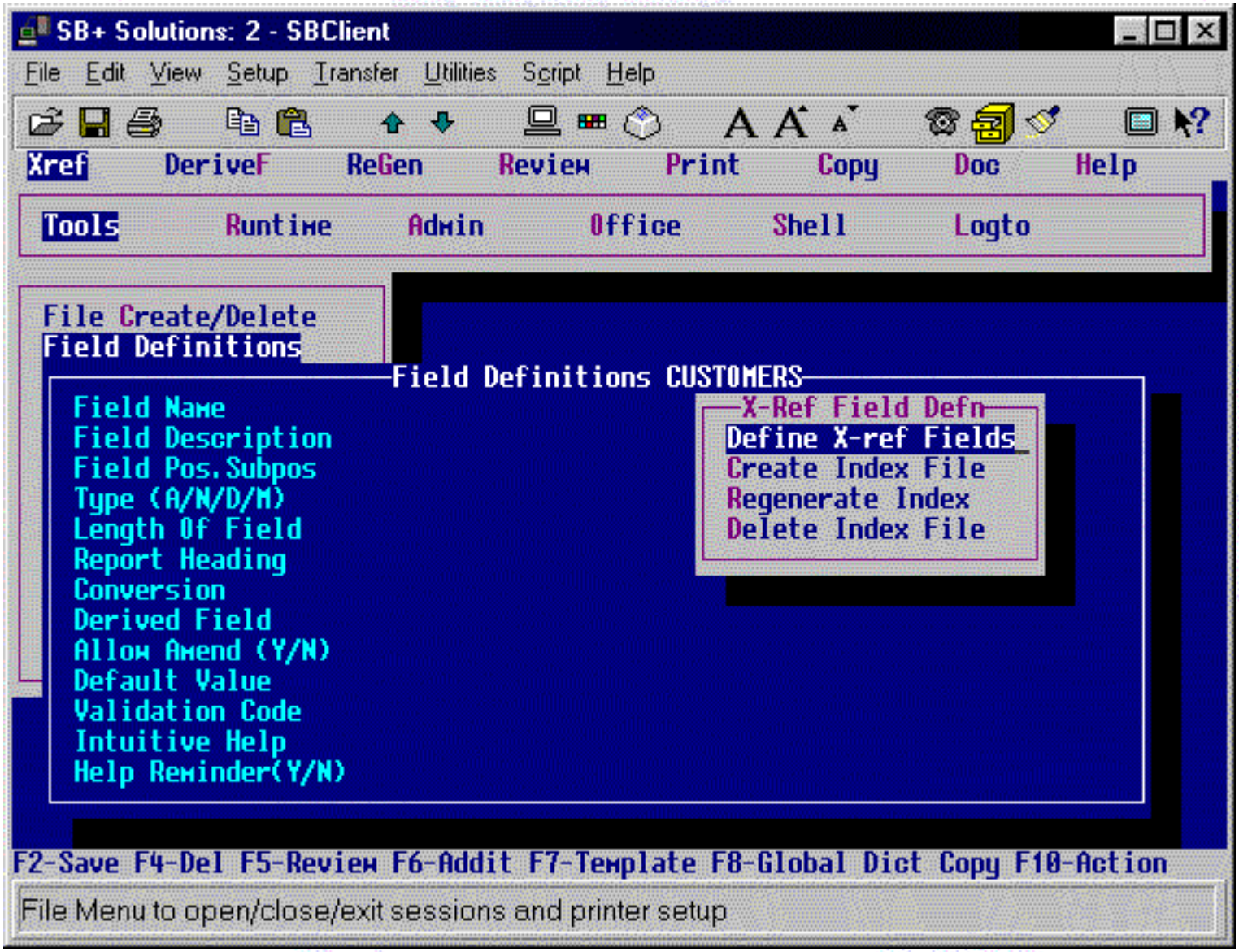
```
C:VAL.PRIME{This number is not prime}
```

This simply illustrates that while paragraphs have great control over the messages that are output, the paragraph itself is not required to display a message. Of course, like many other matters, this issue is completely up to individual style and development standards.

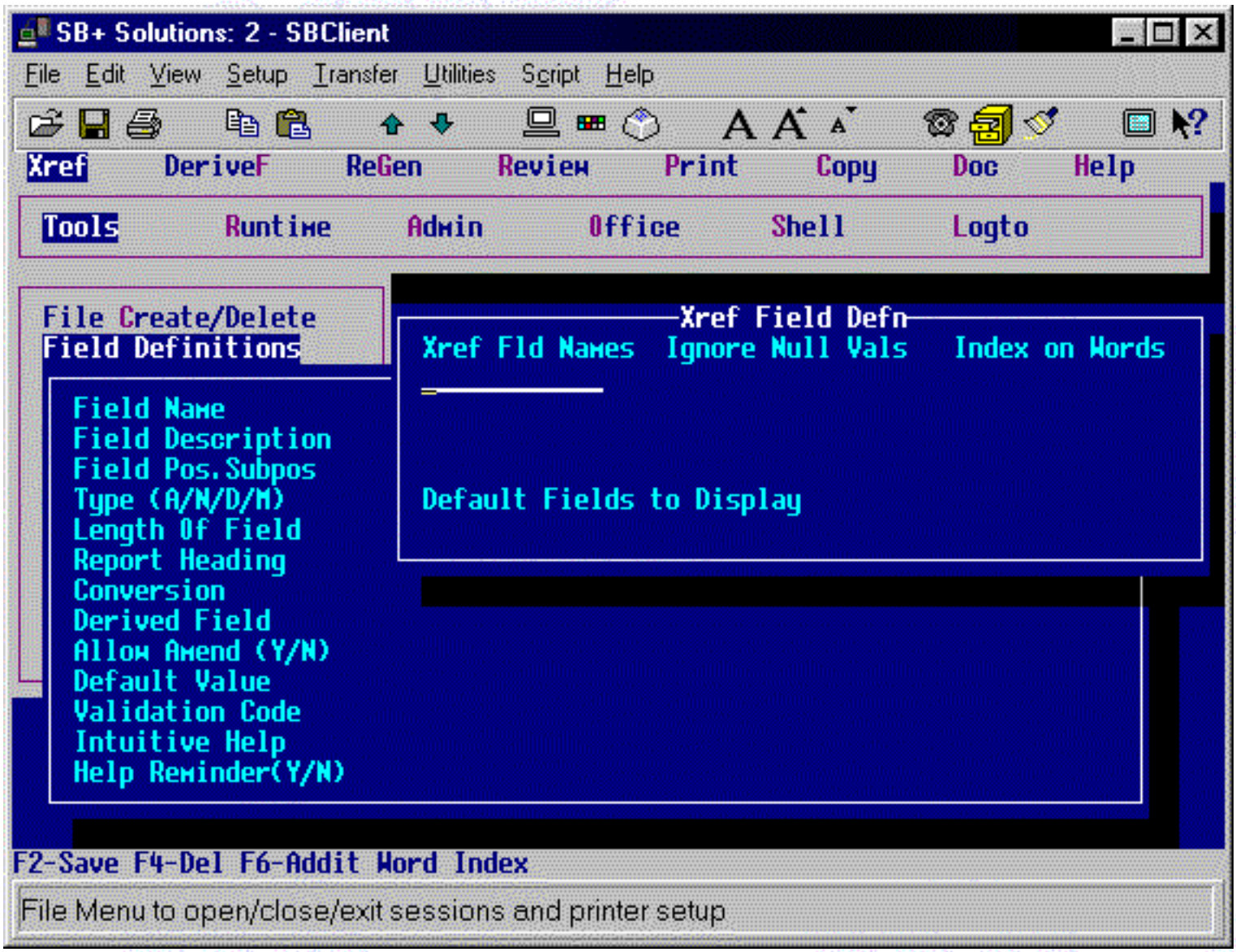
Defining an SB+ Index

To define an index for fields in SB+, the **Field Definitions** tool is used. This tool is available from the Tools menu, or can be started by entering /FD at any menu or input prompt. Once this tool has been started and you have entered the name of a file where data is (or will be) stored, the Field Definitions screen is displayed.

When the cursor is on the Field Name prompt, press F10 to invoke the action bar. The first option on the action bar is called Xref, and can be selected either by pressing either <cr> or the letter X. Once selected, a menu appears as follows:



As you can see, this menu corresponds very well to the tasks that we need to do to set up an index. Therefore, we select the first option, and the following screen is displayed:



In the Xref Fld Names prompt, simply list the name of the fields you want an index created for. For each field, you can specify whether or not to ignore null values, or to index on words.

It is usually best to set the Ignore Null Vals field to a "Y" for all index fields. This tells SB+ that if the field being indexed has no value, no index entry will be made. In contrast, if this is set to an "N", all records will be indexed for this field, regardless of whether the field has a value or not. This can be useful, but only when it is necessary to select records with a null value in a particular field, such as when selecting records with a null posting date (for example).

When Index on Words is "Y", SB+ will break the field up into words (as separated by spaces) and will create a separate entry in the list for each word. Therefore, if you index on words on a Name field, and the name is John Q Doe, three index entries will be made. If Index on Words is "N", only one index entry will be made for the entire name.

This particular feature, while very useful, has an annoying side effect. If someone has a name of John Johnson and the index is used to select all records where the name starts with "John", the record will appear in the selection box twice: once for the first name, and a second time for the last name. This gives the impression that two records were selected, when in reality both entries reference the same record. Therefore, keep this issue in mind when setting up a word-by-word index.

After defining fields, the next step is to **create the index file** where the actual index records will be stored.

Creating an SB+ Index File

The second option on the Xref menu is called Create Index File, and this option is used to -- you guessed it -- create a file for the indices.

In SB+, the name of an index file is the dictionary file name with the suffix .INDEX, and all indices for a file are stored together in one index file. Therefore, if the file being indexed is called CUSTOMERS, the name of the index will be CUSTOMERS.INDEX. Regardless of the number of indexed fields defined for the file, all of the information for all of the indices will be stored in this one index file.

When you select the Create Index File option, SB+ will prompt you to enter the name of the file being indexed. Once this has been entered, SB+ will ask you to estimate the number of records in the data file. Using this information, the index file name will be constructed (as described above) and the actual file will be created.

Once the file has been created, the next step is to **regenerate the indices** using any records that may already be in the file.

Regenerating an SB+ Index

If you define an index field after records have been entered into the file, or if you change the list of indexed fields after information has been saved into the file, you will need to regenerate the indices. This is done using the Regenerate Index option on the Xref menu in the Field Definitions tool.

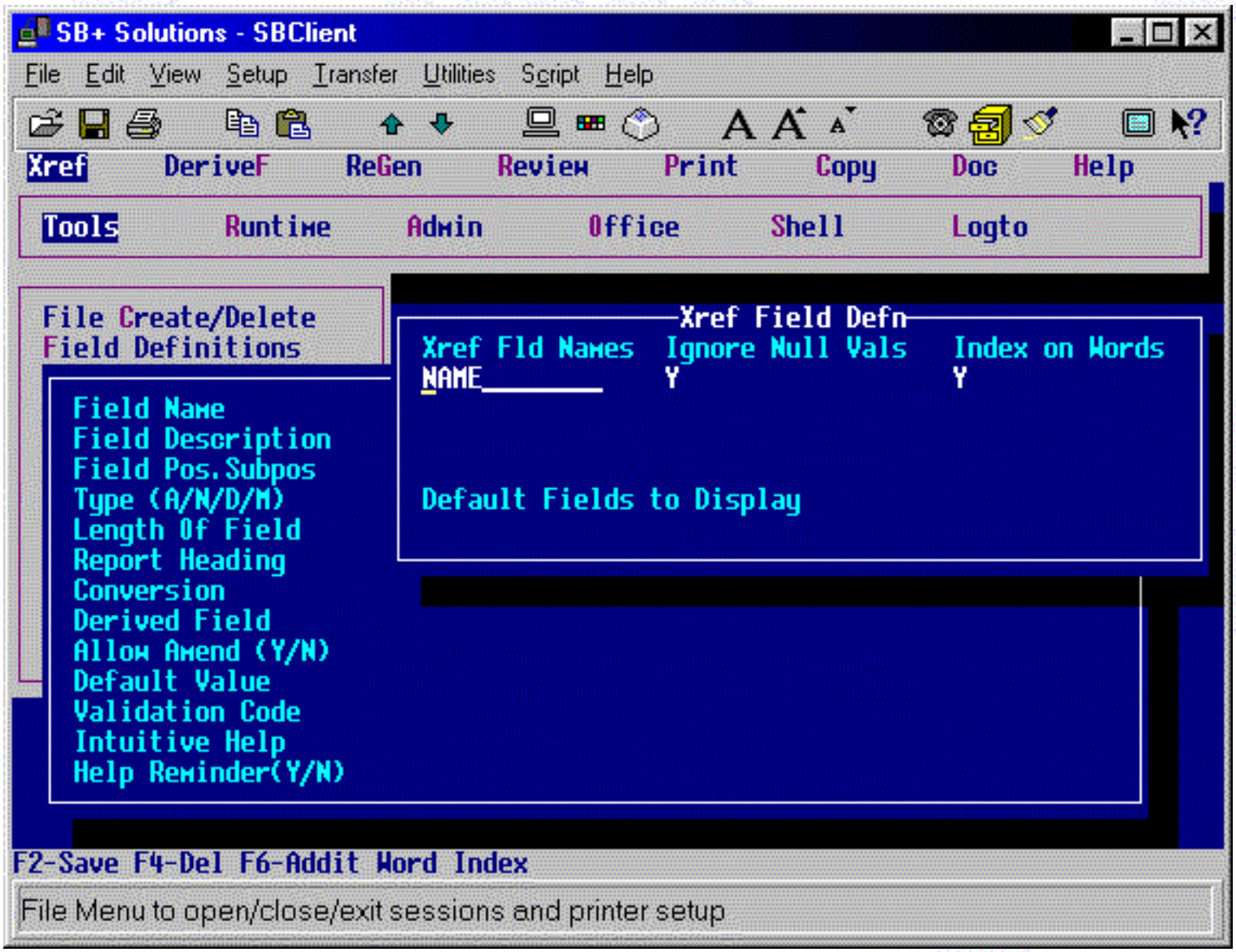
When this option is selected, SB+ will prompt you to enter the name of a file whose indices are to be regenerated. In versions prior to 3.x, once the file name has been entered, all existing indices for the file as named will be cleared and rebuilt using all of the information in the file. In 3.x and later versions, SB+ will prompt you for which index to rebuild, or "ALL", which affords you more control over which indices are rebuilt.

If you're using a version of SB+ prior to 3.x, this is important enough to say again: All indices are cleared and rebuilt using all the information in the file. If the data file is huge and/or there are several indexed fields for each record, it could take a considerable amount of time for the rebuild to be complete. Also, if the rebuild process doesn't finish for any reason, it will need to be restarted from scratch, thus compounding the time problem.

Changing an SB+ Index

Once an index has been defined, you may find the need to change it. For example, if an index has been constructed for a field, and later the field position for that field changes, SB+ will continue to use the old definition, which will result in an incorrect index.

Fortunately, this is easily corrected. To do so, first select Define Xref Fields from the Xref menu in the Field Definitions tool. The screen will appear as follows:



Next, move the cursor to the name of the indexed field that changed. Once the cursor is on the name of the changed field, press <cr>. While SB+ appears to do nothing, it actually reloads the definition for the field behind the scenes. When this is complete, press F2. From that moment on, the index file will be updated with the correct information based on the changed field. Of course, you should **regenerate the indices** after such a change to ensure that the index file is current and correct.

You may also use this to add indices to the file at any point. To add fields, simply add the names of the fields you want indexed on this screen. As with changing an index field, it is always a good idea to regenerate the indices after adding new fields.

Deleting an Indexed Field

Occasionally, after a field has been defined as an indexed field and an index has been created for it, the index is determined to be no longer necessary and can be deleted. To do this, first select Define Xref Fields from the Xref menu in the Field Definitions tool. Next, move the cursor to the name of the indexed field that is to be removed. Enter either a backslash (\) or press the Line Delete key (which is defined differently for each terminal) to remove this line from the list of indexed fields. Once F2 has been pressed, the index will no longer be updated for the removed field.

Deleting the indexed field does not, however, clear any of the existing records in the index file for that field. Therefore, if the goal of the exercise is to regain disk space, you will need to **regenerate all indices** for the file to make the old records disappear. If disk space is not the issue, you need not regenerate the indices; though the index information is not cleared, SB+ won't go looking for it after the index field has been removed from the list.

Deleting an Index File

If you ever need to remove an index file, simply select Delete Index File from the Xref menu on the Field Definitions tool. SB+ will prompt you for the name of the file whose index is to be deleted. Once this has been entered, the index file will be removed. Note, however, that this does not remove the defined indexed fields -- it just removes the index file, which essentially disables all of the indexed fields.

If you've deleted an index file and then later want to use an index again, you'll need to **create a new index file** and **regenerate the index** for the information in the file. Better yet, if you think you're going to need an index, don't delete the index file!

Reading From an Index

There is nothing special to be done to tell SB+ to use the index. If certain rules are followed, SB+ will automatically use an index when it is available.

In **Selection Criteria**, any of the following phrases will select from an index on the CUSTOMER.TYPE field (if such an index has been configured):

```
CUSTOMER.TYPE = "1234"
CUSTOMER.TYPE = "?"
CUSTOMER.TYPE = "(@VALUE)"
```

If no index for CUSTOMER.TYPE has been configured, a regular selection will be done instead. All of this happens automatically.

In contrast, if the CUSTOMER.TYPE field and SHIP.DT fields are both indexed, the following will not work:

```
CUSTOMER.TYPE = "1234" AND SHIP.DT = "12 NOV 94"
```

Even though both fields are indexed, SB+ will not use the indices for intersecting selections like this. The developers at SB+ have made a decision that it is more system efficient to select the entire file rather than selecting from two indices and then normalizing the results. Perhaps they are right -- I've never tried any conclusive tests to prove anything to the contrary.

Therefore, rule #1 of creating a selection process that uses an index is as follows: When constructing selection criteria on an indexed field, never use AND or OR to combine selection phrases. Select one field compared to one literal, or the index will be ignored.

Rule #2: Never include sorting criteria in a selection process that is reading from an index. The presence of sorting criteria causes SB+ to ignore the index and select the entire file based on the selection criteria.

Keep these rules in mind when building selections based on indexed fields, and you'll be amazed at how nice a few well-placed indexed fields can be.

Incidentally, the above rules apply only when SB+ indices are used, though the technique can be used for both SB+ and OE indices. If OE indices are used, a different set of rules will apply. Most importantly, when using OE indices don't tell SB+ about them! Consult the manuals for your OE for more information about creating and using OE-level indices.

Using SB+ Indices With Split DICT/DATA Files

In certain applications, such as those that are translated into several different languages, the field definitions and screen definitions can be stored in a file that is completely separate from the data. As an example, a customer file may have a dictionary named CUSTOMERS, but the data is actually stored in a file named CUSTOMERS.DATA.

SB+ handles this split dictionary and data file scenario very well for most things, but can be particularly frustrating when faced with updating and retrieving information from indices. In short, there appears to be some inconsistency in the index file name between different tools.

When a record is filed in an input process, SB+ attempts to update the indices for the record in a file which is named using the data file name suffixed by .INDEX. Using the example stated earlier, if a customer record is written to CUSTOMERS.DATA, SB + will attempt to update an index file called CUSTOMERS.DATA.INDEX.

But when a selection process attempts to retrieve information from the index file, it assumes the name of the file is the dictionary name suffixed with .INDEX. This then causes the selection process to try to open a file called CUSTOMERS.INDEX, which is different from the name of the file that the input process updated.

To compensate for this problem, be sure there are two pointers to the same index file for each file that is split this way. Whether you name the index file CUSTOMERS. INDEX or CUSTOMERS.DATA.INDEX, be sure there is a valid pointer under both names to the same file, and you're sure to be protected from this little annoyance.

Using Derived Indices

Normally, an index is created based upon some field in the record. Using a sales order file as an example, we may want to **define an index** on the customer ID field so that we can quickly recall a list of orders for each customer. However, indices are not limited to fields inside of the record. Any derived value field can be indexed just like a regular field, which allows you to create derived (calculated) indices which do more than just simple indexing.

Assume for a moment we have a sales order file that has a ship date in a field called SHIP.DT. If this field is null, the order hasn't shipped. Conversely, if the field does have a value, the order has been shipped. Knowing this, we can create an index of unshipped orders by customer by indexing on a field with this derived value:

```
(IF(SHIP.DT=",CUSTOMER.ID,"))
```

This **expression** states that if the shipping date is null, return the customer ID. Otherwise, return null. Next, we tell SB+ to index on this field, ignoring any null values. With this in place, SB+ will update an index by customer ID when the shipping date is null, and will properly update the list when an entry screen changes the shipping date to any non-null value. In the end, SB+ takes over all of the headwork of inserting and deleting entries into the list, which makes this a very simple building block for adding very complex features to an application.

This technique can also be used to create a combination index, which allows you to construct one index based on multiple fields. For example, if we construct an index based on customer ID and a separate index based on shipping date, there is no way we can construct **Selection Criteria** to use these two indices together. However, if we construct a **derived value field** that concatenates the two fields together, we can construct an index using this field and accomplish the same result. Such a derived value field might be called X.1 and could appear as follows:

```
(CUSTOMER.ID:"*":SHIP.DT)
```

(I include the asterisk so there is no confusion between the end of the customer ID and beginning of the shipping date. It does occupy a character position in the index, but the benefit of having the delimiter outweighs the sacrifice.)

Once an index has been defined on this field, we can use a paragraph which calls a selection process to use the index. The paragraph in this example is responsible for constructing a search key in the **common variable** @VALUE and passing this value to the selection process.

The text of this paragraph appears as follows:

```
@VALUE = CUSTOMER.ID : "*" : SHIP.DT
EXEC 'selectprocess'
```

Of course, *selectprocess* will actually reference a valid selection process, which has Selection Criteria as follows:

```
X.1 = "(@VALUE)"
```

As stated earlier, X.1 is the name of the field with the derived value. The "(@VALUE)" simply says to take the expression inside of the parentheses, evaluate it, and replace everything inside of the quotes with the result. In this case, the contents of the common variable @VALUE is put inside the quotes, and the result is used in the selection of the index. The end result, then, is an indexed selection using both a customer ID and shipping date.

Officially, SB+ does not support selecting from two indices at the same time. However, there is no restriction on selecting from one index that is based on two (or more) values!

Translating a Value to Another File

If a screen has a prompt called Customer ID and the entry at that prompt points to a record in the CUSTOMERS file, a **conversion** can be created which will translate the customer ID to the customer name. For the sake of this example, let's assume the customer name is in attribute 1 of the CUSTOMERS record. The conversion to do this translation appears as follows:

```
(F("CUSTOMERS",@VALUE)<1>)
```

Essentially, this conversion reads a record from the CUSTOMERS file using @VALUE (the raw data value) as a key, returning attribute 1 from the record. A simple conversion, no doubt, but a powerful one nonetheless.

Totaling a Multivalued List

To display a total of the values in a multivalued field, the S(...) **expression** **function** can be used for the derived conversion. For example, if we want to total a list of values in the **common variable** @VALUE, the following conversion could be used:

(S(@VALUE))

Any multivalued expression can be placed inside of the inner parentheses. Therefore, the following are all valid expressions of varying complexity:

(S(EXTENDED))
(S(P("*process*")))
(S(F("CUSTOMERS",P("GET.CUST.KEY,":@PARMS(2)<4>)))<21>)

Translating From a Table

The TABLE(...) function is very popular in derived conversions. This function searches through a specified **table** looking for a code matching @VALUE and returns the description of the code when found (or null if not found). If we want to search a table called PMT.TYPES, for example, the following conversion could be used:

(TABLE('PMT.TYPES'))

For this to work properly, @VALUE must contain a code which is in the table. Otherwise, this **expression** will return null. As a result, this technique is most commonly used on fields which reference the attribute where the code is stored.

Calling a Process to Convert a Field

The most powerful and flexible **expression** **function** you can use in a derived conversion is the P(...) function, which calls a process. This can be any type of process, but generally will be either a paragraph or **BASIC** subroutine.

What makes this particular feature so powerful is that you have complete control over how the value is formatted. The **common variable** @VALUE is passed into the process, and from there the process is free to do whatever it pleases in order to break apart, rearrange, look up, and/or otherwise change the value to be displayed.

The syntax for this type of conversion appears as follows:

(P('*processname*'))

In this syntax, *processname* is the name of a process which will do the formatting. To see how this can be useful, let's assume we have a field which stores Canadian postal codes. Canadian Postal codes are three characters followed by a space followed by three more characters. However, due to disk limitations, let's assume our programming team long ago decided to store the postal codes without the space in the middle, though the space should be displayed every time a postal code is printed.

This simple paragraph can be used to reformat the postal codes to be displayed properly:

```
@VALUE = @VALUE[1,3] : ' ' : @VALUE[4,3]
```

This simple, one-line process extracts the first three characters of @VALUE, concatenates a space, and then concatenates the last three characters of @VALUE to arrive at a result. This result is then returned to the caller in the same common variable: @VALUE. Once returned, the changed value will be displayed, and the users are never aware that the postal code isn't stored with the space.

Note that with the above example, a simple conversion expression would suffice, and a process would not need to be called. However, when things get more complicated, and there's no other way to change the value, it's nice to have such a powerful feature available. For example, what if names were stored in last-first-middle initial format, and you need to display the value on the screen in first-middle-last order? Also, to add to the complexity, what if the middle name were optional? The following conversion process could be a good starting point:

```
LOCAL PART.CNT
*
PART.CNT = DCOUNT(@VALUE,' ') ;* How many parts are there? (2=last-first, 3=last-first-middle)
CASE (PART.CNT = 2)
  @VALUE = FIELD(@VALUE,' ',2) : " " : FIELD(@VALUE,' ',1) ;* Rearrange the parts
CASE (PART.CNT = 3)
  @VALUE = @VALUE"G1 1":" ":@VALUE"G2 1":" ":@VALUE"G0 1"
CASE 1
  * Any other format leaves the value unchanged
END CASE
```

For sake of illustration (and not simply to be confusing!) I have used the FIELD(...) function in the first CASE and a group extract mask in the second. For normal day-to-day programming, I would probably use one or the other, but not both in the same paragraph. Nonetheless, this illustrates that there is generally more than one way to construct an expression. Consult your OE system manuals or the **list of standard conversion codes** for more information about the group extraction code.

Calling a Process to Hide a Value Conditionally

With certain applications, you may want to hide a value from display if a user does not have the right permissions to access a particular piece of information. A derived conversion calling a process is perfect for this.

Let's assume that at login our application loads up a security clearance value between 0 and 100 into @PARMS(1)<4>. If this value is less than 50, the user should be restricted from viewing the credit limit for a customer in our Accounts Receivable setup screen. To implement this feature, we can create a paragraph such as the following:

```
IF (@PARMS(1)<4> < 50) THEN
  @VALUE = ''
END
```

This paragraph checks the security clearance and then clears the display value if the clearance isn't high enough. If the security clearance is over 50, however, the value passes through the paragraph unchanged, and is output as if nothing has happened,

To call this paragraph, the following **conversion** could be used:

```
(P("processname"))
```

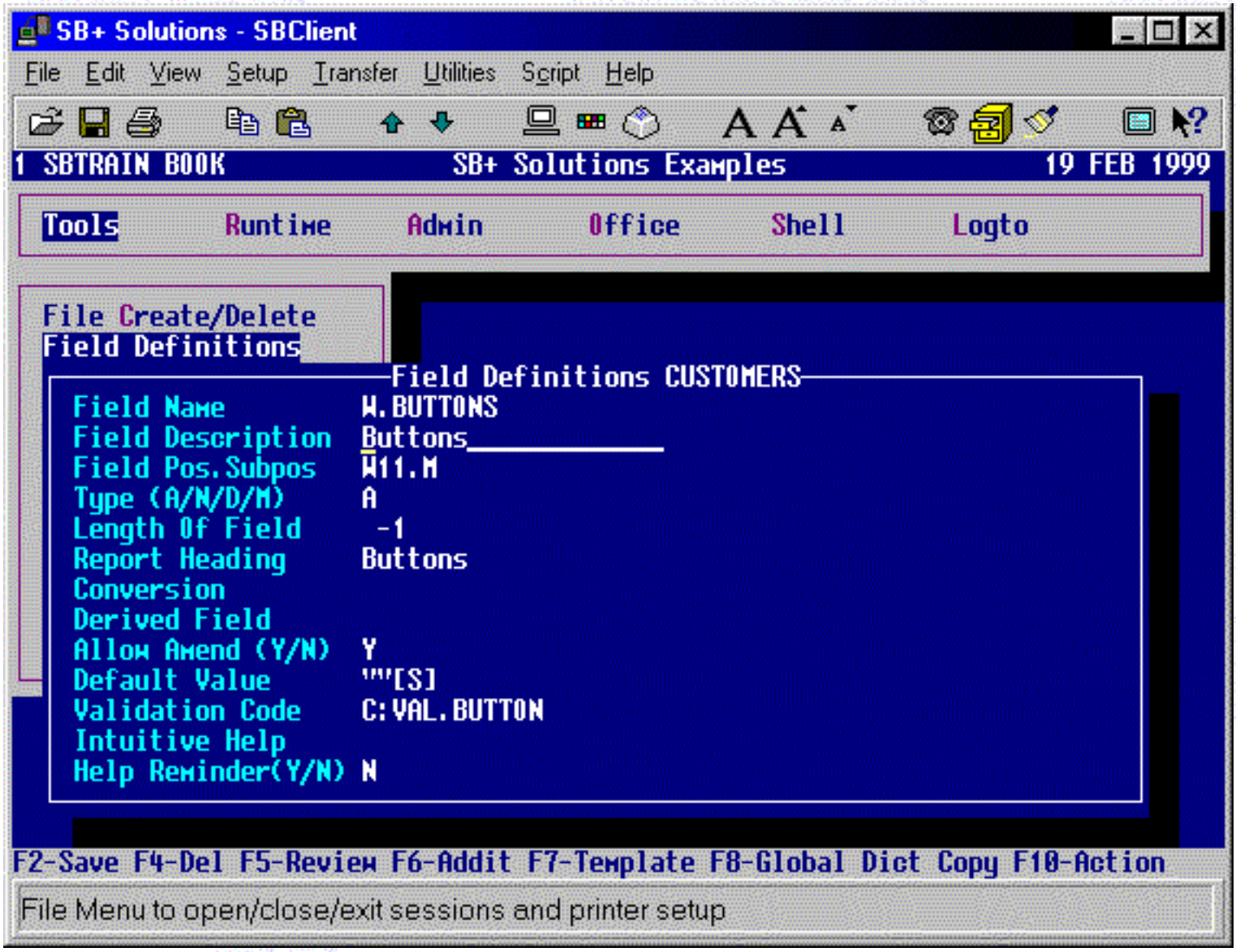
(Of course, *processname* would be replaced by the actual name of the paragraph.)

To take this to another level, our paragraph could read a prompt information record from a permissions file and decide whether to suppress the display of a field. With such a process, we could easily protect any number of fields using one single process called from the conversion prompts on a variety of different fields. (Incidentally, this is the theory behind the SB+ field input restrictions feature.)

Note that when a value is converted this way, the original value is unchanged. Instead, all we're changing is the value to be displayed. Always remember: Conversions can be used to change the display value without any impact on the original value. Or to say it another way, the conversion allows us to define our own *view* of a value without having to actually change the underlying information.

Creating a Button Field Definition

First, let's create the field for the buttons. This field could be called W.BUTTONS, and appears in the **Field Definitions** tool as follows:



There are four important things to note about this field definition:

- The Field Pos.Sub Pos says that this field will be stored in the @WORK **common variable**, attribute 11. Button fields should always be treated as temporary fields, and therefore should never be saved as a part of the main record. Only through the act of setting or resetting an option should the main record be updated. In other words, instead of updating the record directly (like most entry fields), the record will be updated as a consequence of an option being set or reset. Also, the attribute number (11) is not all that significant -- it is simply the next available attribute in the @WORK common variable.
- Notice that the **length is "-1"**. This value says that SB+ will press <cr> automatically once a single character is entered into any of the values.
- The **default expression** says that when the cursor advances to the multivalue past the last option, default the value to null

and skip out of the prompt.

- The **validation process**, VAL.BUTTON, is used to update the buttons as one or more are selected. We'll look more at this process later.

Creating a Process to Setup the Buttons

Next, let's create the process to translate the codes from the incoming record to the button values. For the sake of this example, an "on" button is represented by an asterisk, while an "off" button is represented by the lower case letter "o". Also, let's call the process SETUP.BUTTONS and create it as a paragraph as follows:

```
LOCAL AMC,OPTS,MIN,MAX,CODES,TEMP,NDX,L.POS
LOCAL TEMP.MAX
*
* Parse Incoming Parameters
*
AMC   = FIELD(@PARAM,',',1) ;* Attribute to update
OPTS  = FIELD(@PARAM,',',2) ;* Total number of options
MIN   = FIELD(@PARAM,',',3) ;* Min # options
MAX   = FIELD(@PARAM,',',4) ;* Max # options
CODES = @PARAM[COL2()+1,32200] ;* Valid Codes
*
* Initialize the return values (everything is off)
*
@VALUE = DEL(STR(@VM:'o',OPTS),1,1)
*
* Extract the codes from the record
*
TEMP = @RECORD<AMC>
TEMP.MAX = DCOUNT(TEMP,@VM)
*
* Remove any invalid codes...
*
NDX = TEMP.MAX
WHILE NDX DO
  IF LOC(TEMP<1,NDX>,CODES,',') = 0 THEN
    TEMP = DEL(TEMP,1,NDX)
    TEMP.MAX = TEMP.MAX - 1
  END
  *
  NDX = NDX - 1
REPEAT
*
* Check to see if there are too many values, and remove
* extras from the back of the list if necessary...
*
IF MAX THEN
  WHILE (TEMP.MAX > MAX) DO
    TEMP = DEL(TEMP,1,MAX+1)
    TEMP.MAX = TEMP.MAX - 1
  REPEAT
END
*
* Check to be sure enough buttons are set. If not, add
* a few from the front of the list.
*
IF (TEMP.MAX < MIN) THEN
  NDX = 1
  WHILE (TEMP.MAX < MIN) AND (NDX <= OPTS) DO
    IF LOC(FIELD(CODES,',',NDX),TEMP,@VM) = 0 THEN
      TEMP = INS(TEMP,1,1,FIELD(CODES,',',NDX))
      TEMP.MAX = TEMP.MAX + 1
```

```
END
*
NDX = NDX + 1
REPEAT
END
*
* Set the appropriate buttons to the "on" state
*
NDX = TEMP.MAX
WHILE NDX DO
L.POS = LOC(TEMP<1,NDX>,CODES,',')
@VALUE<1,L.POS> = '*'
*
NDX = NDX - 1
REPEAT
*
* Update the record with the updated code list
                *
@RECORD<AMC> = TEMP
*
* Install parameters into a working table
*
NDX = LOC(AMC,@PARMS(39)<1>,@VM)
IF NDX = 0 THEN
NDX = DCOUNT(@PARMS(39)<1>,@VM) + 1
@PARMS(39)<1,NDX> = AMC
END
*
@PARMS(39)<2,NDX> = OPTS
@PARMS(39)<3,NDX> = MIN
@PARMS(39)<4,NDX> = MAX
@PARMS(39)<5,NDX> = CODES
          *
 @REFRESH = 2
```

(Personally, I'd prefer to have much more in the way of comments for a paragraph of this size or larger, but I limited them here for space considerations.)

Let's take a look at this paragraph...

First, note that there are five incoming parameters. We'll save much of this information in a table to be used later. These variables represent the following values:

AMC	The attribute number where the incoming values are coming from. For the sake of this example, AMC will always be positive and will always reference an attribute in the **common variable** @RECORD.
OPTS	The total number of options in the CODES list.
MIN	The minimum number of options that can be selected at any given time.
MAX	The maximum number of options that can be selected at any given time.
CODES	The master list of codes, in the order that the buttons will appear on the screen

Once we've parsed out these codes from the parameter line, the next thing we need to do is create a list of button values and set all of the buttons to the "off" value. This is done all with the one line:

```
@VALUE = DEL(STR(@VM:'o',OPTS),1,1)
```

This illustrates a couple of different points. First, this line indirectly states that the value to be returned from this process will be passed in the common variable @VALUE. Second, notice the STR(...) function inside of the DEL(...) function. The STR(...) will create a string of value marks and lower case O's for the number of options we need. However, this technique results in an extra value mark at the beginning. The DEL(...) function, then, trims this extra value, leaving the string with the correct number of lower case O's for the number of options as specified. See "**Expression Functions**" for more information about these functions.

Next, we need to figure out where the incoming codes are stored. Again, for the sake of this example all of the codes are assumed to be in @RECORD. This could easily be expanded to account for @WORK as well by setting and testing for a negative attribute number.

Once the list of codes has been extracted from the record, we need to check that each of the codes is valid, based on the list of valid codes as passed in the CODES parameter. If a particular code is not found in the CODES parameter, we need to remove it from the list. The result of this exercise must be a list of codes and buttons which match exactly, so we cannot allow rogue codes to remain in the list.

After the invalid codes have been removed from the list, we must then compare the number of codes in the list to the defined minimum and maximum. If the list has too many values, we need to remove the extras. If the list has too few values, we need to insert new ones.

It's important to note that inserting codes this way is roughly equivalent to setting a default on a field, though the default is set immediately after the record has been read, rather than when the prompt is encountered during the input cycle. Therefore, if a particular prompt must have a minimum number of buttons set, the order of the codes that correspond to the buttons is very significant.

Also, blindly removing codes from a list like this can be incredibly destructive, so we hope this is not something that will be happening on a regular basis. If this is a regular occurrence, though, you may want to display a message to tell the user that the codes are being removed.

Next, the process will cycle through the list of codes, setting the appropriate buttons to the "on" position. Again, note that we're updating the common variable @VALUE, as this is where we want the result of this process to be returned ultimately. Also note this particular process does not account for the possibility that a code in the record may not have a corresponding button, because this contingency has been accounted for earlier in the paragraph.

Once the buttons have been set, we need to update the record with the (potentially) changed list of codes. If the list of codes has changed, this is absolutely necessary to ensure that the number of codes matches the number of buttons selected. If the list of codes has not changed, this certainly doesn't hurt anything, and is simpler than setting a flag to determine whether the list has changed.

Finally, the process updates information into @PARMS(39) so we can use this information later. One particular aspect of button programming that I personally find irritating is when you have to pass parameters to a routine to setup the button field, then pass the same parameters to another routine to validate the field, and again pass the same parameters to another routine to update the record with the new codes. Instead, I prefer to pass parameters to the setup routine only and let the other processes use the parameters stored in common memory (@PARMS(39) in this case). This eliminates the potential problems that can be caused by passing the parameters incorrectly to one of the processes.

The Validation Process

Now that we've talked a little about how the parameters are stored in memory, let's take a look at the paragraph used to **validate** and update the buttons. Before we get too far into this process, however, we need to define some rules for how this prompt will work. For example, what will set a particular button to the "on" value? Conversely, what will set a particular button to the "off" value? Finally, if one button changes, what effect will this have on the other buttons?

The following rules answer these questions:

- If the user enters an asterisk, the button will be set to the "on" value, and the code which corresponds to the button will be inserted into the proper attribute in the @RECORD or @WORK **common variable**.
- If the user enters a lower-case "o", the button will be set to the off value, and the code which corresponds to the button will be removed from the proper attribute in the @RECORD or @WORK common variable.
- If the user enters a value which is different from the current value of the button, we will toggle the button to the alternate value. Therefore, if the button is "on" and we type 'X' for the button, this will set the button to "off". Typing 'X' for the button again will return the button to the "off" state.

When a button is turned on or off, we need to verify that the correct number of buttons are set, based on the minimum and maximum number of buttons specified for the field. The following paragraph will accomplish this:

```
LOCAL AMC,TEMP,NDX,OPTS,MIN,MAX,CODES,BUTTON.ON
LOCAL TEMP.MAX,ORIG.TEMP,L.POS
*
AMC  = @PARAM
TEMP = @RECORD<AMC>
*
* Get the configuration for this button set
*
NDX   = LOC(AMC,@PARMS(39)<1>,@VM)
OPTS  = @PARMS(39)<2,NDX>
MIN   = @PARMS(39)<3,NDX>
MAX   = @PARMS(39)<4,NDX>
CODES = @PARMS(39)<5,NDX>
*
* Change the state of the button
*
CASE @VALUE = '*'
  BUTTON.ON = 1
CASE @VALUE = 'o'
  BUTTON.ON = 0
CASE 1
  BUTTON.ON = ABS((@WORK<0-@LINE<1,1>,@CNT> = '*') - 1)
  IF BUTTON.ON THEN
    @VALUE = '*'
  END ELSE
    @VALUE = 'o'
  END
  @REFRESH = 7
END CASE
*
* Update the record or working variable accordingly
*
IF BUTTON.ON THEN
```

```
  IF LOC(FIELD(CODES,',',@CNT),TEMP,@VM) = 0 THEN
    TEMP = INS(TEMP,1,-1,FIELD(CODES,',',@CNT))
  END
END ELSE
  NDX = LOC(FIELD(CODES,',',@CNT),TEMP,@VM)
  IF NDX THEN
    TEMP = DEL(TEMP,1,NDX)
  END
END
ORIG.TEMP = TEMP ;* Save the codes before min/max tests
*
* Check to see if there are too many values, and remove
* a few from the front of the list if necessary...
*
TEMP.MAX = DCOUNT(TEMP,@VM)
IF MAX THEN
  WHILE (TEMP.MAX > MAX) DO
    L.POS = LOC(TEMP<1,1>,CODES,',')
    @WORK<0-@LINE<1,1>,L.POS> = 'o'
    TEMP = DEL(TEMP,1,1)
    TEMP.MAX = TEMP.MAX - 1
  REPEAT
END
*
* Check to be sure enough buttons are set. If not, add
* a few to the back of the list.
*
IF (TEMP.MAX < MIN) THEN
  NDX = 1
  WHILE (TEMP.MAX < MIN) AND (NDX <= OPTS) DO
    IF LOC(FIELD(CODES,',',NDX),TEMP,@VM) = 0 THEN
      TEMP = INS(TEMP,1,1,FIELD(CODES,',',NDX))
      @WORK<0-@LINE<1,1>,NDX> = '*'
      TEMP.MAX = TEMP.MAX + 1
    END
    *
    NDX = NDX + 1
  REPEAT
END
*
* Update the record with the updated code list
*
@RECORD<AMC> = TEMP
*
* Refresh all values on the screen
*
IF ORIG.TEMP # TEMP THEN
  @REFRESH = -2 ;* Refresh all values
END
```

At the start of this paragraph, the field number where the values are stored is passed into the process in the @PARAM variable. Using this value, the process first extracts the field where the codes are stored, and then gets all of the configuration information for this button field from @PARMS(39).

Based on the three different valid entries to the prompt, the CASE block decides whether a particular button should be on or off. Notice that the process actually changes the input value to an asterisk or lower case 'o' if something else has been entered.

Once we know whether the button is being set or reset, the next thing to do is update the list of codes in the record. If a button is

set, we add a code to the list. Otherwise, we remove the appropriate code from the list.

Next, we need to check to be sure the right number of buttons are on, based on the minimum and maximum values as defined for the button set. If too many buttons are set, we'll remove the extras from the front of the list. If too few are set, we'll insert some new codes into the back of the list. This provides somewhat of a first-in/first-out mechanism for the prompt.

Most important, we then need to update the record with the changed list of codes. In the case of this paragraph, we can accomplish this simply by moving our temporary working list of codes (in the TEMP variable) to the proper attribute in the common variable @RECORD.

The last thing we need to do is display the changed values. For normal input prompts this would be unnecessary. However, because the process can change the value (as is done in the CASE 1) and also can change values other than the current one (with the minimum and maximum testing), we may need to redisplay some peripheral part of the button field.

If the process changed the value (see the CASE 1 section), we set the common variable @REFRESH to 7. This tells SB+ to refresh this one multivalue when this process is done. Otherwise, the user entered an asterisk or lower case 'o', so the right value is already displayed and no refreshing is necessary.

Next, note the line of code which assigns the value of TEMP to a **local variable** called ORIG.TEMP prior to the minimum and maximum checks. At the end, if TEMP does not equal ORIG.TEMP, the minimum and maximum checks have changed the list of valid codes, and therefore all of the button values need to be refreshed. Such is the purpose of assigning -2 to the common variable @REFRESH. Unlike the value of 2, which displays all data values, -2 tells SB+ to refresh all of the values in this multivalued prompt set, which will cycle through each of the displayed multivalues one-by-one and redisplay the (potentially) changed value.

Calling the Validation Process

Calling the validation process is much simpler, as it requires only a single parameter. To call this process, place the following code in the Validation Code prompt on the button field's field definition:

C:VAL.BUTTON,*amc*

In this syntax, *amc* is the attribute number of the field that is to be updated when a button is set or reset. This is the only tie between the SETUP.BUTTON process and the VAL.BUTTON process, so it is unquestionably an important one. If these two processes incorrectly reference different attribute numbers, all sorts of weirdness will result.

Calling the Process After Read

Because the process after read and **validation** processes use parameters, it is very important to call these properly. Failure to do so will inevitably result in a screen that works less than perfectly, which is unacceptable.

The process being called after read is the most complicated process call, because it has so many different parameters. If the process were called from a paragraph in the Proc After Read slot on the screen definition tool, it would be called as follows:

W.BUTTONS = P('SETUP.BUTTONS,*amc*,*opts*,*min*,*max*,*codes*...')

In this syntax, *amc* is the attribute number of the field in @RECORD where the codes are stored. The *opts* parameter defines the total number of buttons in the set. The *min* and *max* parameters define the minimum and maximum number of buttons that must be set at any given time. Lastly, *codes* is a comma-delimited list of codes that correspond to the buttons. Note that the order of the codes as listed in this parameter must agree with the order of the buttons on the display screen.

This process may also pass parameters, using the following syntax:

W.BUTTONS = P('SETUP.BUTTONS,':POS(*field*):',*opts*,*min*,*max*,*codes*')

In this syntax, instead of hard-coding the field number, as in the previous example, this process tells SB+ to look up the attribute number for *field* and use it instead. This makes programming a little easier, as it does not require the attribute number for the field to be known prior to setting up the button field. In short, this technique lets SB+ figure out the proper attribute number, instead of putting that responsibility on you, the developer.

Putting It All Together

Once you have the field definition, the process after read, and the validation process completed, the next thing to do is place the button field on a screen definition. Fortunately, placing a button field is no different from placing a regular multivalued field on the screen, except you may not want a heading to be displayed.

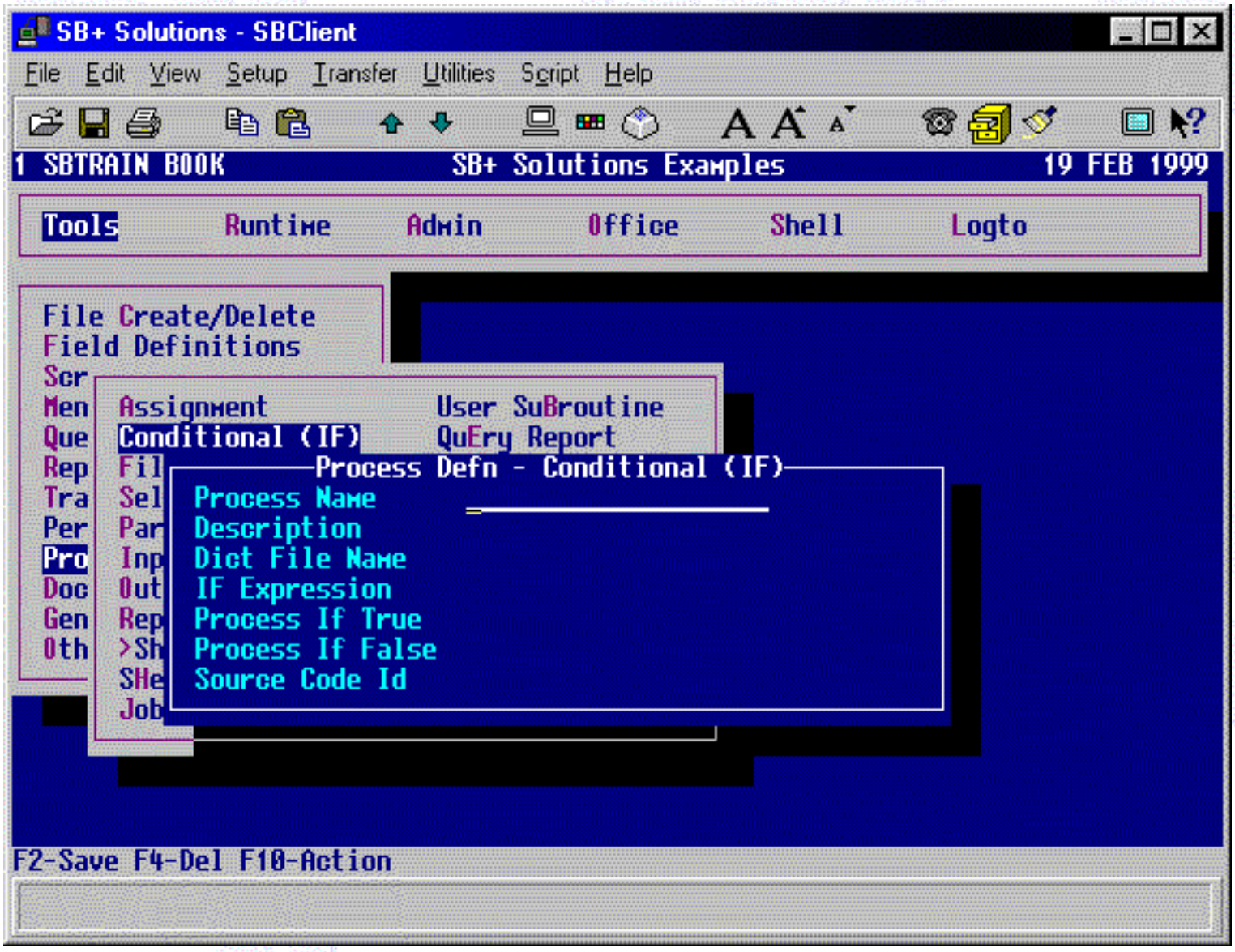
Placing the button, however, does nothing to actually put some text beside the buttons describing what each button represents. This must be done separately, either by placing a dependent, display-only multivalue next to the button field on the screen (which is loaded separately), or by **typing the descriptions** for each button directly onto the screen definition.

So, Do You Like a Challenge?

For the ultimate in flexibility, can you think of a generic way to implement a button field where both the codes and descriptions are read from an SB+ table? It's possible with a little tweaking on what we've done here. The question is: How would *you* do it?

Using a Conditional Process

The Process Definition - Conditional tool can be accessed by entering /PD.C at a menu or input prompt, or it can be selected from the menu structure by selecting Tools, Process Definition, and then Conditional. This tool appears as follows:



Like any process, a conditional process must have a name. Of course, naming standards are recommended to make name selection a routine task.

Once a name has been entered, you may optionally enter a description for the process. This description will appear in the F3 when you are looking for your process in this tool, so it is in your best interest to include a description which is specific and detailed.

Following the description you must enter an expression which will be used to test the condition. This can be any expression, such as:

@OTHER(18) = 32

...or any other expression that would normally fall between the keywords IF and THEN in a paragraph or BASIC subroutine. When the process is used, if the condition results in a non-zero value, the Process If True will be executed. On the other hand, if the condition results in a zero value, the Process If False, if specified, will be executed.

In both the Process If True and Process If False slots, specify the name of a process to be called. (The Process If True is a required entry; However, the Process If False is optional.) This is both a strength and weakness of this tool. It is a strength because this process can only be used to call other processes. This focuses the process on the condition itself, not on the work to be done based on the state of the condition. It is also a weakness, however, because this process cannot perform simple tasks (like setting a value) based on a condition. Instead, a process must be called to set the value. For complex processing based on a condition, this is acceptable. However, when the purpose of the condition is simply to set a **common variable** to a particular value, it doesn't make sense to have two dinky processes when one small one will do. Also, if the condition is more than can be written in a single expression (such as when part of the expression is calculated based on another series of conditions), the Conditional process falls dramatically short.

Using a Paragraph

In contrast to a Conditional Process, a **paragraph** can be used to evaluate a condition and then do something based on the condition. The advantage to using a paragraph is two-fold:

- The condition to be tested can be prefaced by code to calculate values used in the condition.
- The paragraph can call a process based on the condition, or it can simply set values without calling a separate process.

Paragraphs are entered using the Process Definition - Paragraph tool, which can be accessed through the menu structure by selecting Tools, Process Definitions, and then Paragraph. Or, for a quicker approach, you may enter /PD.P at any menu or entry prompt to invoke the tool.

To illustrate the flexibility of a paragraph, let's assume we have six fields in a record, as follows:

ENTRY.DT	The date the record was first created
ENTRY.TM	The time the record was first created.
ENTRY.BY	The ID of the user who created the record.
LST.UPDT.DT	The date the record was last updated.
LST.UPDT.TM	The time the record was last updated.
LST.UPDT.BY	The ID of the user who last updated the record.

The goal of this example is to set these fields when a record is saved. If the record is new, we want to update the ENTRY.DT, ENTRY.TM, and ENTRY.BY fields with the current date, time, and user ID, respectively. Whether the record is new or not, the process needs to update the LST.UPDT.DT, LST.UPDT.TM, and LST.UPDT.BY fields with the current date, time, and user ID, respectively.

To do this, we'll create a process to be called from the Process After Screen Accept slot on the screen definition. This process is a paragraph with the following lines:

```
IF (@ACTION < 3) THEN
  IF (ACTION = 1) THEN
    ENTRY.DT = @DATE
    ENTRY.TM = @TIME
    ENTRY.BY = @USER.ID
  END
  *
  LST.UPDT.DT = @DATE
  LST.UPDT.TM = @TIME
  LST.UPDT.BY = @USER.ID
END
```

The @ACTION **common variable** has up to four different valid values. For the sake of this example, we're checking for only two of these.

On the first line of this paragraph, the process checks to see if the record is being deleted (@ACTION = 3). If so, there is no need

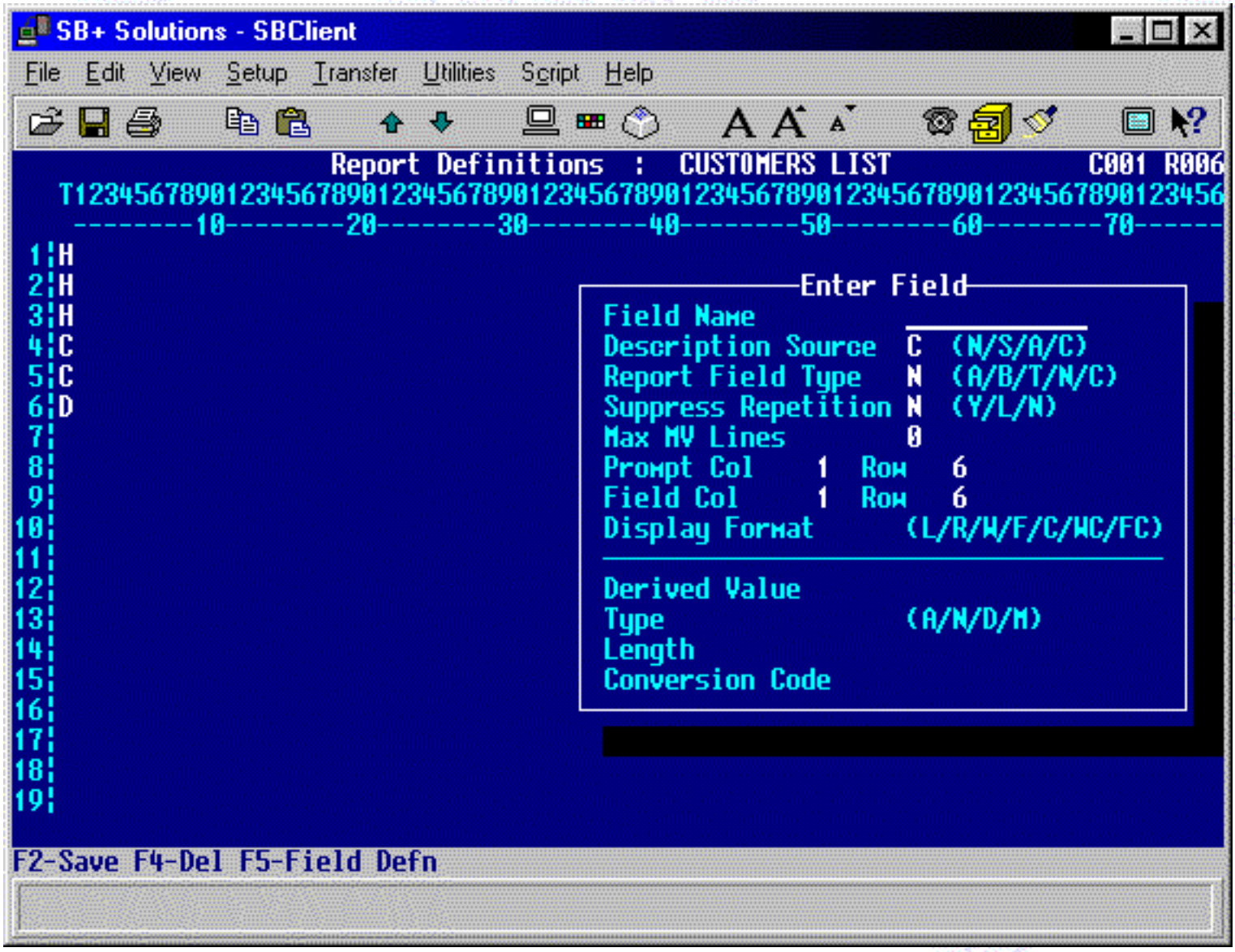
to update the stamp fields, so the remainder of the process is skipped. For all other times, the process checks to see if the record is new (@ACTION = 1). If the record is new, the process sets the CREATE... fields. Finally, whether the record is new or not, the process updates the LST.UPDT... fields.

As you can see, this paragraph is much more compact and straight forward compared to the alternative. If we were to write this same logic using Conditional and Assignment processes, we would need to create up to five different processes and link them together appropriately. Doing so is not only time consuming, but fragments the code so severely that debugging and code readability are significantly compromised.

Placing Fields on a Report Writer Report

Method #1: Placing one field at a time

When the report painting screen is displayed (i.e. the one with the column and row guides), a field may be placed on the report simply by moving the cursor to the position where you want the field and pressing the F5-Field key. When this key is pressed the following screen is displayed:



At the "Field Name" prompt, enter the name of the field to be placed at the current position. Once this name has been entered, default information will be displayed for the placement of the prompt.

When new fields are added to a report, you can pretty much ignore the prompts on the F5-Field screen. Depending on the type and format of the report you are building, you can always return to a particular field and edit it to implement a specific feature.

Though these prompts will be explained in more detail in the following section, the next few paragraphs will explain a little about these prompts to help introduce the techniques that will follow.

At the "Display Prompt" field, enter one of the following values:

- N -- Don't display a label for this field
- S -- Display the label to the left (side) of the field
- A -- Display the label above the field
- C -- Search out the first line in the column heading section and place the label on that line.

"Field Type" may be one of 5 different values:

- N -- Normal field; Simply display the value.
- A -- Accumulated field. Display the value and accumulate the value for a later total field.
- T -- Total field. Display the total of an accumulated field.
- B -- Break field. Display the contents of the field from the prior record, not the current record. If this doesn't make sense now, don't worry about it -- we'll look at break fields in more detail later.
- C -- Conditional field. These fields evaluate a condition and tell SB+ to skip a certain number of lines based on the condition. Again, if this doesn't make sense yet, that's okay -- we'll look at conditional fields later too.

The "Suppress Repetition" prompt is somewhat of a strange prompt, in that it has three settings, only two of which deal with suppressing repetition.

If the report is sorted by a particular field, and you want the value in that field suppressed when it's the same as the previous record, then enter "Y" at the "Suppress Repetition" prompt. If you want to see all details, whether they are repeated values or not, enter "N" at this prompt. To see how this works, assume we have a report with the following information:

```
State Amount
CA    100.00
CA   2200.00
CA      3.40
NE     20.00
NE    504.00
WY    120.00
```

If we tell SB+ to suppress the repetition of the state field, the report will be presented as follows:

```
State Amount
CA    100.00
     2200.00
        3.40
NE     20.00
      504.00
WY    120.00
```

In short, the repeated values for "CA" and "NE" are suppressed.

So what about that third value? Didn't I say that this prompt had three possible values?

For multivalued fields on a columnar report, you may (must?) enter an "L" in this prompt to loop-on multivalues. This flag tells SB + to reprint the detail line over and over and over for each value in the multivalued field, showing each value on a separate line. When this option is used, the "Max MV Lines" (discussed next) must be set to "0.M" (zero-dot-M).

The "Max MV Lines" prompt determines how many values will be displayed for each record of the report. Entries in this prompt may be any of the following values:

- 1 -- Only one value will be shown
- 0.M (zero-dot-M) -- All values will be shown. This option applies to columnar reports only.

If the value is any other positive number, that number of values will be shown. Any values past the cutoff as defined here will not be shown. This is most applicable to form reports, though it can be used on columnar reports as well.

If the value is any other negative number, the absolute value of the number defines the number of values to be shown. However, unlike the previous option, which discards the values past the cutoff, this option will roll additional details onto subsequent pages. This applies only to form reports.

The "Format" prompt determines the justification and formatting of the field on the report. This field may be one of several values:

L	The field will be left justified.
R	The field will be right justified.
C	The field will be centered within its defined width (not necessarily centered to the page)
W	The field will be word-wrapped based on its defined width. Each value will be wrapped based on the position of spaces.
F	The field will be full justified (both right and left aligned) based on the width as defined for the field on the report (not in the field definition).
WC	The field will be word-wrapped based on its defined width and each line will be centered.
FC	The field will be word-wrapped based on its defined width and each line will be full (both left and right) justified.

Below the horizontal line in the F5-Field box, the following fields appear:

At the "Derived Value" prompt, either a field name or other expression may appear. For normal fields, the field name will appear. For derived fields (discussed later), a standard SB+ expression will appear.

The "Type" prompt determines the type of a particular field, and may be one of the following values:

A	Alphanumeric
N	Numeric
M	Monetary
D	Date

Oddly enough, the field does *nothing* when the report is running. Instead, this field is used by the Report Definition tool to know how to present the field during editing. For example, if a field is defined as alphanumeric, the field will show up in the tool with X's. If a field is defined as numeric, it will show up with 9's. If a field is defined as monetary, it will show up as 9's with a decimal point. Finally, if the field is defined as a date, the field will show today's date. The designers of SB+ have done this to help you locate fields when editing the report. Had they shown all fields as just a bunch of X's, it would probably be more difficult to spot where the numeric, money, or date fields were!

Always remember that this value is for reference only. In other words, just because a field is an alphanumeric field doesn't mean the field cannot display a date or monetary value.

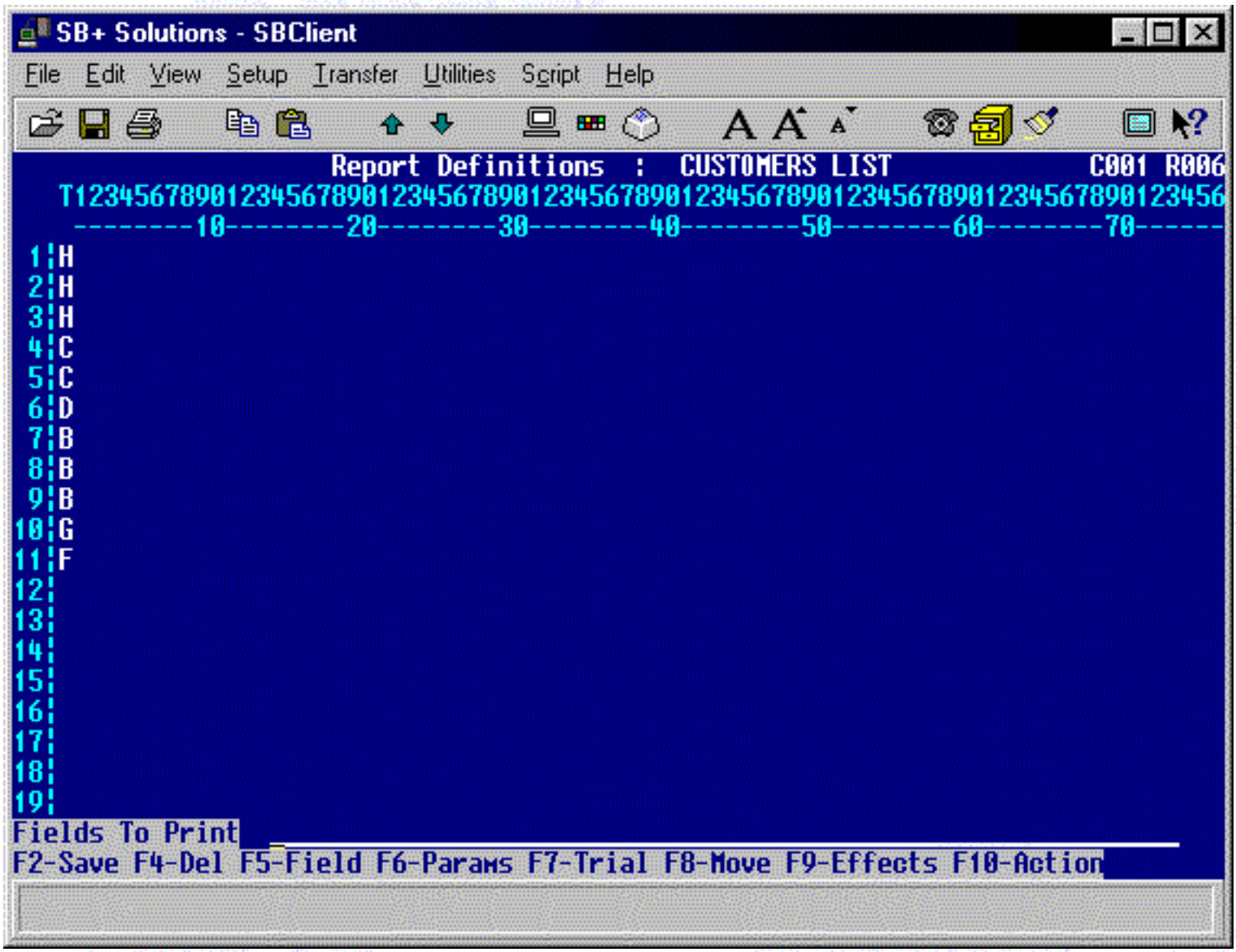
Following this prompt is a "Length" prompt. Here you may enter a number to define the number of characters to be shown on the report for this prompt. For example, if a particular field is defined as 30 columns in the field definition, but the report has room for only 20, you may override the field definition by entering 20 at this prompt. Likewise, if you need to make a field longer, this prompt can be used for that purpose as well. Keep in mind that when you change the width of a field on the report, that change can affect the centering or placement of the field (depending on the justification as defined for the field). It does not, however, change the length of the field as it applies to other screens and reports.

At the "Conversion" prompt, you may enter any standard OE conversion (without parentheses) or an SB+ expression (enclosed in parentheses). The prompt allows you to change the value to be displayed in a field after the field has been extracted from the record and (possibly) accumulated for total. Typically, date fields use a standard OE date conversion like "D4" or "D2/". Similarly, monetary and numeric fields typically use standard OE monetary or numeric conversion codes, such as "MR2", "MR03", "ML4", etc. Alpha fields rarely have a 'standard' conversion, per se. Regardless of these norms, any field can be converted through an SB+ expression by enclosing the expression in parentheses. When this format is used, the value to be converted is passed into the expression in @VALUE, and the result of the expression will become the display value.

Method #2: Building the whole detail line at once

When the report painting screen is displayed for a new report, instead of pressing F5-Field to place a field, move the cursor to the first column following the "D" labeling a detail line. Then press F3. Note: F3 is not shown on the function key line.

When F3 has been pressed, a prompt will be displayed at the bottom of the screen allowing you to enter a list of field names (delimited by spaces) to be included on the detail line, as follows:



At this prompt, either manually enter all of the field names or press F3 and tag the fields for the report (in order).

After all of the field names have been entered, press <cr>, and the entire detail line will be built. This method is considerably faster than placing the fields one-by-one, but should only be used on a detail line which doesn't have any fields. In other words, it's a great way to define the basic layout of a new report, but is practically useless when editing an existing report.

Also, note that while this technique can be used for both columnar and form reports, it lends itself more to columnar reports. Placing fields this way on form reports can make the whole process more confusing than it needs to be.

Templating Reports

One of the advantages to using a product such as SB+ is the common look-and-feel that it gives your applications. Of course, this is all a moot point if all of your reports are formatted differently. Therefore, it's very important to standardize the format of your reports.

One way to have a standard look to your reports is to use a standard heading and footing. For example, perhaps you'd like all of your reports to have a heading which includes the date, time, and the user who ran the report. Following this example a little further, wouldn't it be nice to print the process name, page number, and company name in the footing of the report? And if this isn't enough, wouldn't it be really cool to have the report automatically center the title at the top of the report?

Fortunately, all of this is possible, it's not too difficult, and you only have to do it once for all of your new reports, and as an added bonus it's an excellent exercise for learning the intricacies of creating and manipulating derived values.

Let's assume our heading looks something like the following:

Report Title

Run By: xxxxxxxxxxxxxxxxxxxxxxxx Run Date/Time: xx xxx xxxx xx:xx:xx

To implement this, let's create a new report definition in some file called R132, which basically means "this is the template for a report that is 132 columns wide". (You can create a R80 or R340 later if you like...)

This report will have no real fields, but will have four derived value fields. For the sake of the example, I've taken the liberty of giving names to each of these. If the names given here conflict with existing field names, be sure to change the derived value name accordingly so the Report Writer tool will allow you to create a derived value field.

The names, types, lengths, and derived values for these fields are as follows:

Name	Type/Length	Derived Value	Conversion
R.TITLE	A132	F(@SYSID:"PROCESS",@PROC.NAME<1>)<2>	
R.RUN.DT	D11	@DATE	D4
R.RUN.TM	A8	@RV.START.TIME	MTS
R.RUN.BY	A10	@USER.ID --or-- F('DMSECURITY','~':@USER.ID)<21>	

Let's look at these fields a little more closely.

R.TITLE will read a record from the current XXXPROCESS file using the contents of @PROC.NAME<1> as the key. @PROC.NAME<1> will contain the name of the current report process when the report is running, so this will effectively read the process record for the report that is running and will extract the description from attribute 2. Incidentally, you may want to center this field.

R.RUN.DT will simply output the current date. There is no equivalent report variable (such as @RV.START.TIME vs. @TIME), so there is the possibility of different dates in the report heading if the report is run late at night and goes past midnight. Of course, this field will require a conversion -- which conversion you use is up to you.

R.RUN.TM will extract the start time for the report from an SB+ report variable. We could have used @TIME instead, but this would have resulted in different times on each page, as time is always slipping away... Note that this field will not require a separate conversion, as the conversion has been implemented in the expression.

R.RUN.BY will extract the current user ID from common memory and will display this value. For an added challenge, try to convert this to the name of the user, instead of just the user's ID code! (Hint: The user's short name is stored in a record in DMSECURITY in attribute 21. The key to the DMSECURITY record is a tilde concatenated to the user ID.)

Creating Break Totals/Grand Totals

Any type of field may be totaled on an SB+ Report Writer report. Though this may seem like an obvious statement, there are those who think that only certain fields can be totaled. Now, you probably don't want to total non-numeric fields, but any type of numeric field (including @RECORD, @WORK, and/or any derived field) can be totaled in the report.

The actual totals for fields traditionally appear in a report in one in three different places. These include:

- Break Section
- Grand Total Section
- Detail Section

Before a field can be totaled, however, it must be setup as an "accumulated" field. You cannot total a non-accumulated field.

To make a field an accumulated field, press F5 on a field on your detail line. When the window is displayed, move the cursor to the "Field Type" prompt and change the value from "N" (normal) to "A" (accumulated). Once this change is done, press F2 and you officially have an accumulated field.

To total this field, move to either a break or grand total section and press F5 to add a new field. When asked for a name for the field, enter "T.*field*" (break section) or "GT.*field*" (grand total section). In this name, "T." (or "GT.") is a literal string, and *field* is the name of the field you want to total.) This particular naming convention tells SB+ that this is a "total" field (i.e. the Field Type = "T"), and also defines the field name to be totaled.

Under normal circumstances, there will be no field in the dictionary called "T.field", so SB+ will ask you to create a **derived value**. When the "Not Found..." message appears at the bottom of the screen, enter the type of field and length that you want, and SB+ will create a standard derived value field on the report. By virtue of the name of the field, SB+ will mark it as a total field. Additionally, the derived value, which defines the name of the field being totaled, will be proper.

But what happens if, just by chance, there is a field named this way in the field definition? If this happens, SB+ will not allow you to create a derived value field for the total, and therefore will not total properly. When this happens, do the following:

- Create a derived value field on the report for your total in either the break or grand total sections (or both). The name you use is insignificant.
- Change the field type to "T".
- Enter the name of the accumulated field you want to total in the derived value.

The important part is that the field type is "T" and the derived field is the name of the accumulated field, regardless of whether they're loaded by the "T." or "GT." field name or entered manually.

In Summary...

Any time you want a totaled field on a report, do the following:

- Change the field type of a field to be totaled to an "A" so that SB+ recognizes this as an accumulated field, and;
- Create a new derived field in the break section or grand total section with field type = "T" and the name of the accumulated field in the derived value slot.

Note that this technique can be used for generating totals in the break and grand total section, but not in the detail section. **Detail totals** are handled differently.

SB+ will handle up to 8 **break subsections**, and each break subsection can have the same total field name(s). Said another way, if you have a field called T.LIMIT in one break subsection, you can create another field called T.LIMIT in another break subsection, and SB+ will keep track of the difference between the two when the report is running.

Creating a Record Counter

SB+ keeps track of the record counter as the report is being processed in a **common variable** known as @RV.RECORD.CNT. This field can be printed in the grand total section, but isn't very useful for generating record counts for a report with breaks.

To add record counts to any report, regardless of the number (or absence) of breaks, simply do the following:

- Create a derived field on the detail line called R.CNT. (The name is simply a convention; call it whatever you wish.) The report must have the following settings:

  Field type = A
  Derived value = 1
  Conversion = ("")
- Create a derived field named T.R.CNT (or GT.R.CNT) in the break and/or grand total sections of the report as a standard total field.

The trick to this technique is in the accumulated value field R.CNT. By setting the derived value to "1", we're telling SB+ to add one to the accumulator for each record processed. The conversion is simply a null enclosed in parentheses, which tells the Report Writer that we want to output null for this field, effectively suppressing the display of the literal "1". Other than these tricks, the technique follows standard totaling for break and/or grand total sections.

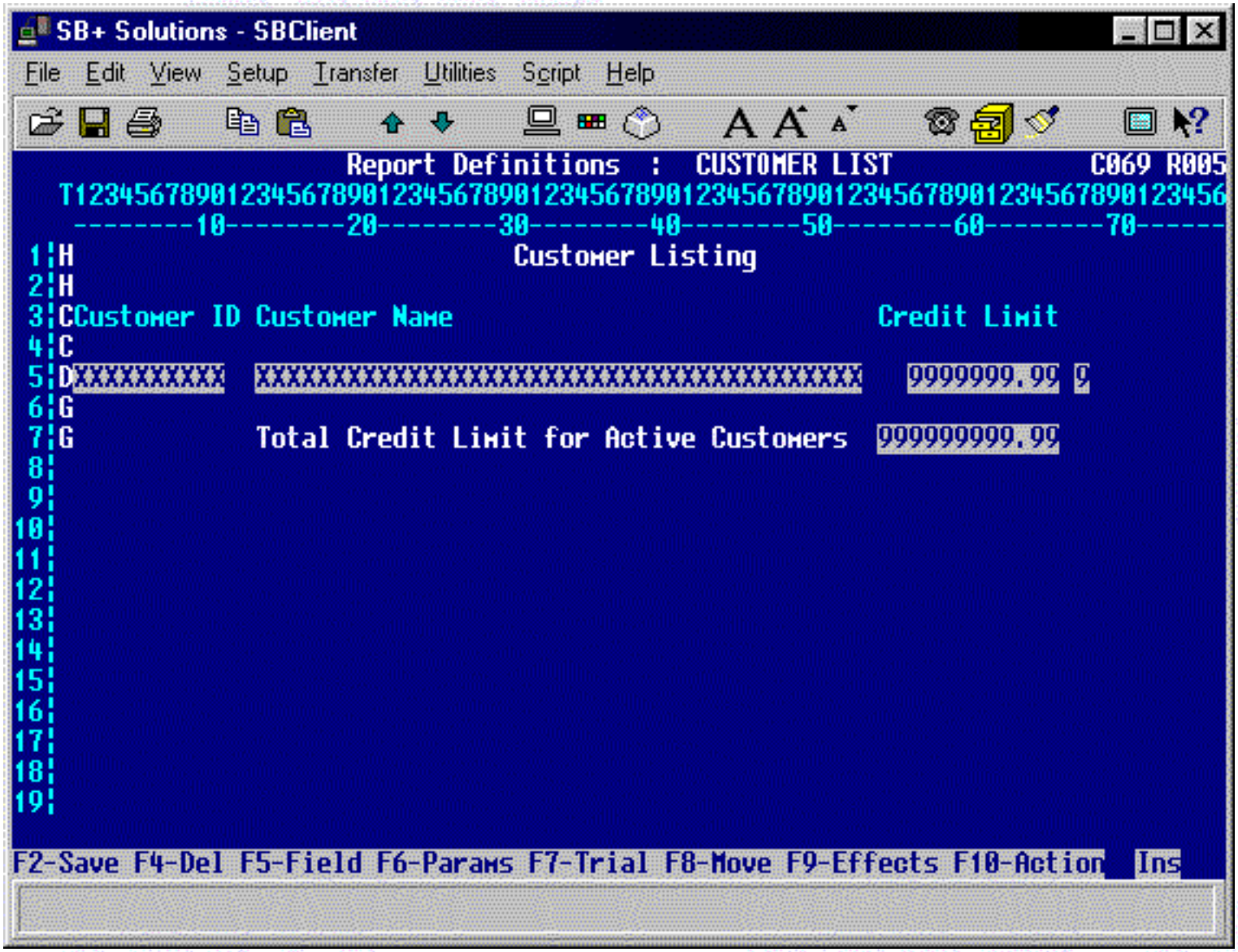
Note that no matter how many different break and grand totals you want for R.CNT, you only need to define the R.CNT field once.

Creating Conditional Totals

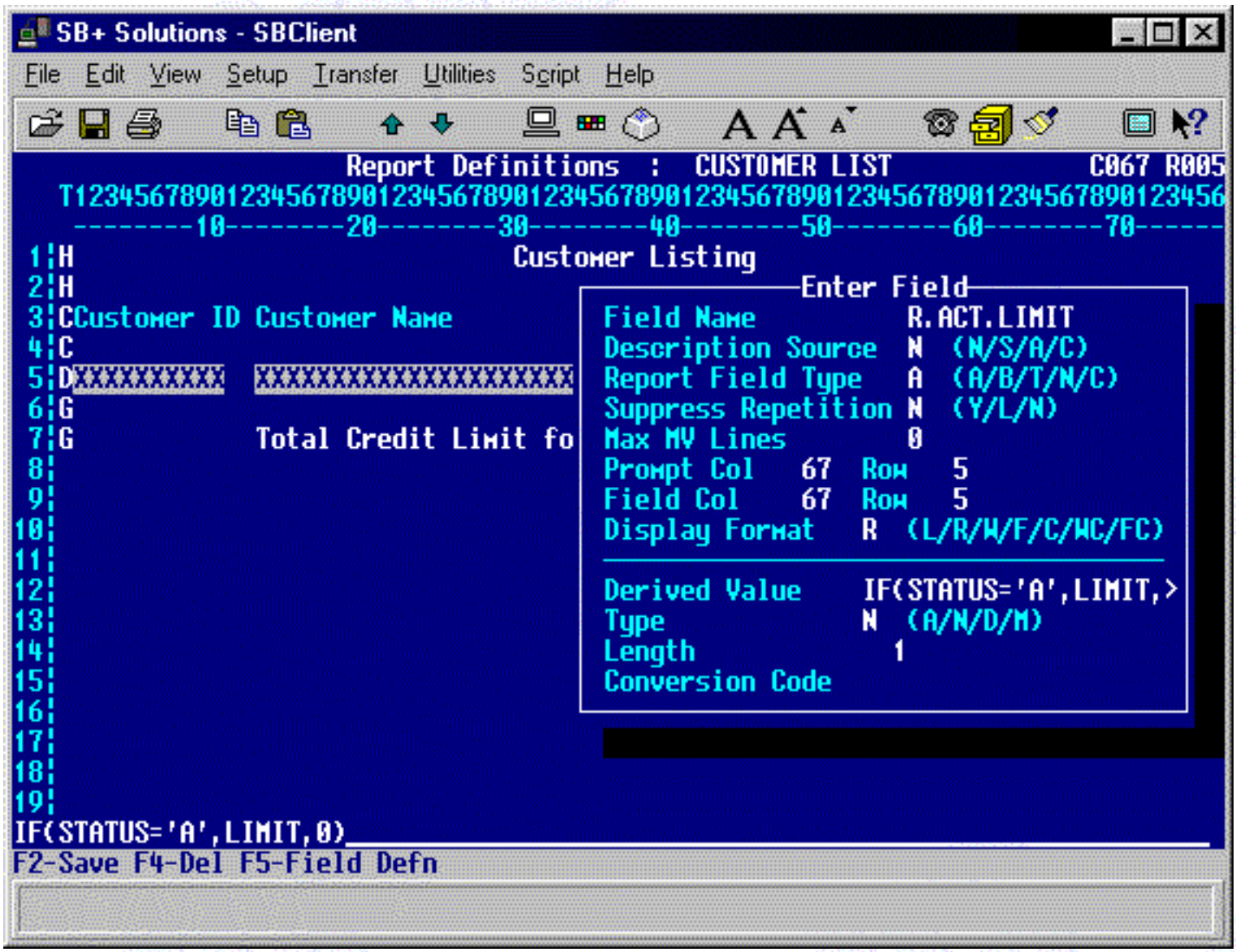
When **creating a total** on a report, you need:

- An accumulated field in the detail section, and
- A total field in either the break section, grand total section, or both.

But what if you need to accumulate a field for total, but only under certain conditions? For example, let's say we have a customer listing with the customer ID, name, and credit limit. At the bottom of the report, we want to display the total credit limit, but only for the active customers. Such a report might appear as follows:



In this report, we don't want to accumulate the credit limit field, because that would cause SB+ to add the credit limit for all customers, inactive and active, to the total. Instead, I've included an extra, single-character **derived value** field at the end of the detail line. This field is our accumulated value, and it appears as follows:

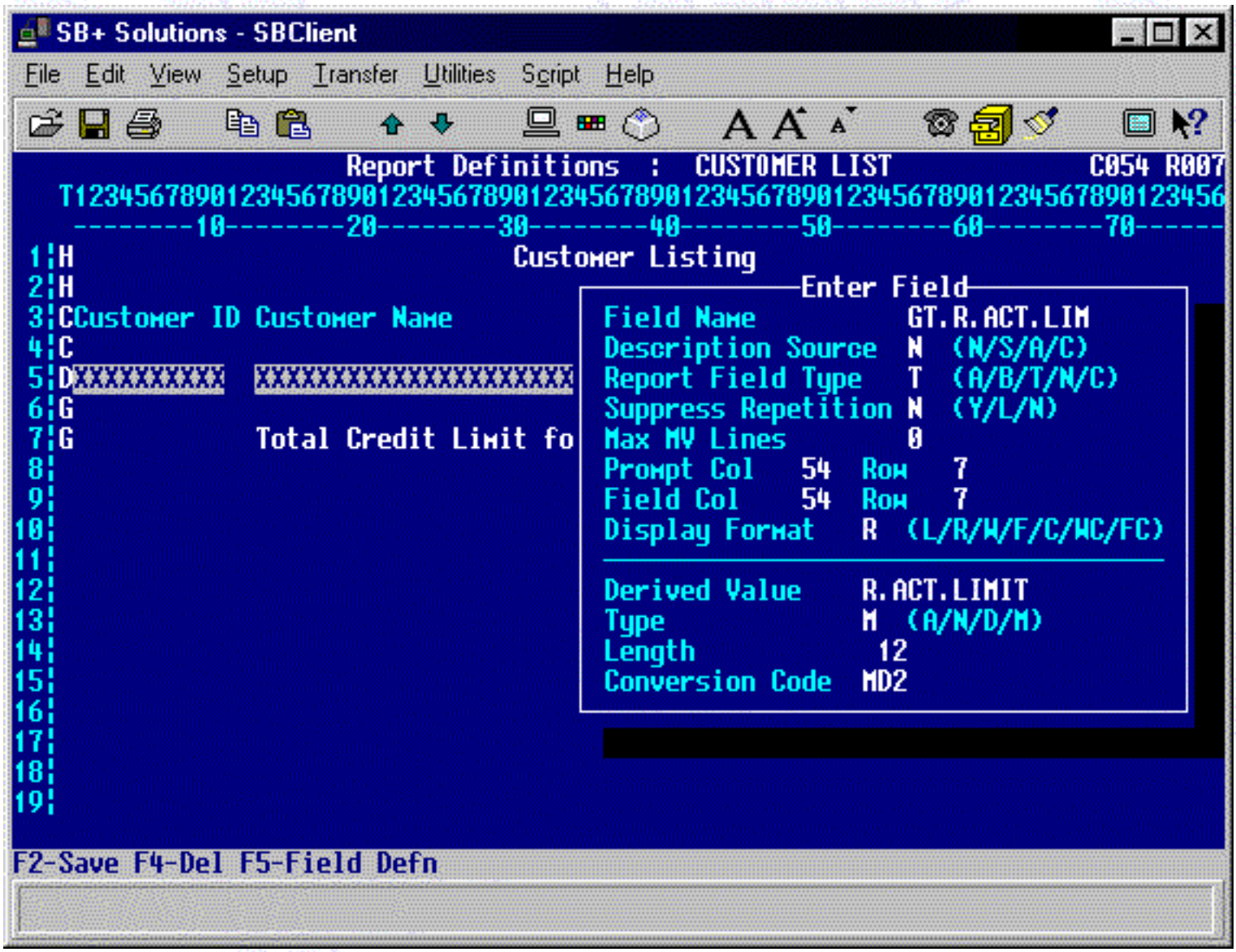


(Note that the expression from the Derived Value prompt has been brought to the bottom of the screen so you can see the entire expression.)

Using the IF(...) **expression** **function**, this derived value checks to see if the STATUS field is "A" (active), and if it is, the value of LIMIT is accumulated. If the condition is false, zero is accumulated. In the end, the credit limit is accumulated only for our active customers.

Finally, note the conversion on this field, which suppresses the display of the field on the report. The outer parentheses tell us the conversion is an SB+ expression, and the expression to output is between the quotes (i.e. nothing). This causes SB+ to change the display value to nothing immediately before adding the field to the print line, and as a result, nothing is displayed for this field.

The total field for this exercise is nothing special; in fact it's no different from any other total field you might create:



Finally, this technique can be used not only for totaling numeric or monetary fields as shown in this example -- it can also be used for conditional **record counting** and other types of analysis you may need to do.

Creating Detail Section Totals

Detail section totals are useful when you want to show a total of a multivalued field as part of the detail section. As a part of the detail section, these totals would be output for each record processed, instead of once per break or at the end (as is the case with the other types of totals).

To create a detail total, simply create a **derived value** field in your detail section with the following **expression**:

S(field)

The S(...) intrinsic will sum all of the multivalues in the field as named and will return a single valued total. As an added bonus, you can sum not only fields in the field definition for the current file, but also any field that is included on the report, including derived fields!

Creating Breaks on Report Writer Reports

A report "break" is a section of information to be output when the value of a particular column changes. For example, on a report sorted by state, we may want the report to show the number of records processed when the state code changes.

Up to eight different breaks may be added to any SB+ Report Writer report. This is accomplished with the following steps:

- On the F6-Params screen in the Report Definitions tool, enter a list of fields that you want to break on, in order of importance.
- In the actual report definition itself, include a break section. This is a section of lines with the line type (i.e. the first character of the line) of "B".

Creating a Report with a Single Break
Creating a Report with Multiple Breaks
Suppressing a Break Field

Creating Summary Reports

When creating a columnar report in SB+, typically all of the details will be shown. However, there are times when you may want to summarize information in a file and show only totals per break and/or grand totals.

To create such a report, define a report as you would normally, but don't give much attention to the detail line -- it will not be printed. Instead, the detail line needs to only include the accumulated fields you wish to total in the break and/or grand total sections. In short, because detail lines aren't printed on summary reports, making them pretty in the report definition is a waste of time.

In contrast, work to create presentable break and grand total sections. A summary report works just like a detail report except that the detail lines aren't printed, so you may have to fuss a bit with the spacing to get it exactly perfect. (The spacing rules for multiple breaks discussed earlier won't cut it.)

To tell SB+ that a report is a summary report, press F6-Params while in a report definition, move the cursor to "Totals Only (Y/N/A)", and change this value to "Y". Once this is done, and you've pressed F2 to save the changes, your detail lines will not be printed when the report is run.

You'll note that this particular prompt has three options, "Y", "N", and "A", which mean:

- Y -- Process, but don't output the detail section
- N -- Process and output the detail section
- A -- Ask the user each time the report is executed

While the first two options are pretty self-explanatory, this third option is worth a little exploring. When "Totals Only" is set to "A", SB+ will prompt the user with "Totals Only (Y/N)" each time the report is executed. If the user enters "Y", only breaks and grand totals are shown. If they enter "N", all details are shown.

This is undoubtedly a neat little option to have in the Report Writer tool. It is also, undoubtedly, very problematic to design a single report that looks good running both with and without detail. For this reason, this is a very nice demonstration tool, but can be more hassle than value when creating reports for a production environment.

Creating User-Selectable Summary/Detail Reports
Creating Combination Summary/Detail Reports
Generating Summary Percentages

Conditionally Suppressing Detail Lines

In previous sections, we've looked at detail reports that use the detail section of the report definition, as well as summary reports that basically skip over the detail section. It may be necessary, though, to skip details based on certain values in each record processed, in which case the previous two options simply aren't enough.

Option #1: Suppressing output of any information for a single record

Say we're building an aging report for a list of open invoices. This report will show the invoice number and the outstanding balance for customers with a credit status code of "Q". To complicate matters, we'll say that the customer record could exist in one of two different files, called CUST and CUST.OLD.

The output will look something like this:

```
Invoice +30 Days +60 Days >60 Days
xxxxxxx xxxxx.xx
xxxxxxx         xxxxx.xx
xxxxxxx                 xxxxx.xx
```

There are several complicating factors to a report like this, not the least of which includes the selection of records. While we can select invoices with a balance, there is simply no good way to select invoices based on a customer status code when the customer record could exist in one of several files.

To overcome this problem, we can use the **Selection Criteria** to select all invoices with an open balance, then install a Process After Read to suppress the individual records we don't want to see.

Such a process could appear something like the following:

```
LOCAL STATUS.CD
*
STATUS.CD = F('CUST',CUST.ID)<5>
IF (STATUS.CD = '') THEN
  STATUS.CD = F('CUST.OLD',CUST.ID)<5>
END
*
IF STATUS.CD # 'Q' THEN EXIT 1
```

In this process, we attempt to read a status code from the CUST file first. If we didn't find the code there, the process checks the CUST.OLD file for the code. Once we have the status code, we can check to see if we want to print the record or not. If the code is not equal to "Q", we set @RTN.FLAG to "1" and exit the paragraph.

Following the Process After Read, the Report Writer checks @RTN.FLAG. If it is "1", the record being processed will be skipped.

This illustrates a very important use of the Process After Read. Not only is this process slot useful for building information to be output on the report, but can also be useful in determining which records are output and which are skipped.

Option #2: Conditionally skipping detail lines

There may come a time when you need a little more control over what is output for a particular detail. Instead of skipping the entire detail section, or skipping the output of a single record, you may need to output a series of different detail lines depending on the record being processed.

In the next example, we're going to create a simple commission report. On this report, we're going to output three different types of salesmen: trainees, apprentices, and executives.

What makes this complicated is that the trainees can be paid a single bonus, called a "level 1". Apprentices can be paid both a "level 1" and "level 2" bonus. Executives are paid "level 1", "level 2", and an additional "level 3" bonus. When the report is output, we want to show only the applicable bonuses for the particular salesman.

To implement this, we can setup three separate detail lines, one for each type of salesman, such as in the following report definition fragment:

```
CType Salesman Level 1 Level 2 Level 3
C
Dxxxx xxxxx C  xxxx.xx
Dxxxx xxxxx C  xxxx.xx xxxx.xx
Dxxxx xxxxx C  xxxx.xx xxxx.xx xxxx.xx
```

Note the "C"s shown on each detail line between the Salesman and Level 1 fields. These are conditional fields which will determine which line is to be printed, not unlike the way multiple break levels are handled.

In this example, let's call the first conditional field, C.1, the second C.2, and the third C.3. We can setup the conditional expression on each of these fields as follows:

```
C.1   IF(TYPE.CD='T',0,1)
C.2   IF(TYPE.CD='A',0,1)
C.3   IF(TYPE.CD='E',0,1)
```

With these fields in place, when the detail section is processed SB+ will find the C.1 field. If the condition is true, the line will be printed; Otherwise the line will be skipped. The same will happen on the second and third detail lines. In the end, the right detail line will be printed for each record processed, and the report will show the information we want.

In this particular example, the position of the conditional fields on the detail line is really not very important. As long as there are no accumulated fields in the detail section, the conditional fields can appear anywhere on the line. However, if there are accumulated values on the detail line, keep these rules in mind:

- All accumulated fields shown before (to the left of) the conditional field will be counted in the totals, even if the line is skipped.
- All accumulated fields shown after (to the right of) the conditional field will not be counted in the totals when a line is skipped.

In summary, conditional fields can be used in all sections of the report, not only the detail and break sections. You can use these types of fields in headings, column headings, footings, and grand total sections too!

Calling a Process After A Line Has Been Printed

In the Report Writer, SB+ has a Process After Read slot that you can use to build information to be output on the print line, or to suppress certain details from being printed. But did you know that there is also a Process After Print?

If you said no, you're partially right. There really isn't such a process slot in a report definition, but it is possible to achieve basically the same results when needed.

You may be wondering when this would be useful. To illustrate this, let's say we're creating a report that uses a variety of derived value fields. If you want to roll these derived value fields into a summary, we must either use the Process After Read to recalculate the values for the summary, or find a way to use the derived fields themselves in the summary. This becomes even more complicated when break fields are involved.

To create a process call after the line has been printed (actually, the process is called when the line is being built, but before it is actually output), simply create a new derived field on the detail line which calls a process, such as the following:

```
P('process')
or
P('process,':field:',':field:',':field)
```

The real advantage to making a call this way (in contrast to the Process After Read) comes when the report has several breaks. Processes called this way will be called after the record has been read and also after any breaks have been processed. Process After Read, in contrast, happens after the read but BEFORE the breaks are processed.

To illustrate the difference between these two process slots, consider a report that breaks on a state code with a salesman summary at each break:

```
State Order Order Total
xx xxxxx xxxxxxxx.xx
  xxxxx xxxxxxxx.xx
  xxxxx xxxxxxxx.xx

Summary for state xx

Saleman Order Total
xxx    xxxxxxx.xx
xxx    xxxxxxx.xx

State Order Order Total
xx   xxxxx xxxxxxxx.xx
    xxxxx xxxxxxxx.xx

Saleman Order Total
xxx    xxxxxxx.xx
xxx    xxxxxxx.xx
```

In this report, each order and order total is output for a given state. However, as the information is being output, we want to create a summary of salesmen and their activity for the state. Note that this is similar to creating a grand total summary, except that we're

creating the summary at each break, instead of a summary at the end of the report.

If we created a process after read to accumulate the break summary information, our first break summary would inadvertently include the first record from the next break. Because the break logic happens after the Process After Read, we get too much information in the summary.

In contrast, however, by calling a process on the detail line, the process will be called after the breaks have been processed. This will result in exactly the right information being rolled into your summary total. As an added bonus, if there are conditional fields on the line prior to your process call, you can control when your process is called, and have even more control over when the totals are updated.

Passing Parameters To A Process After Print

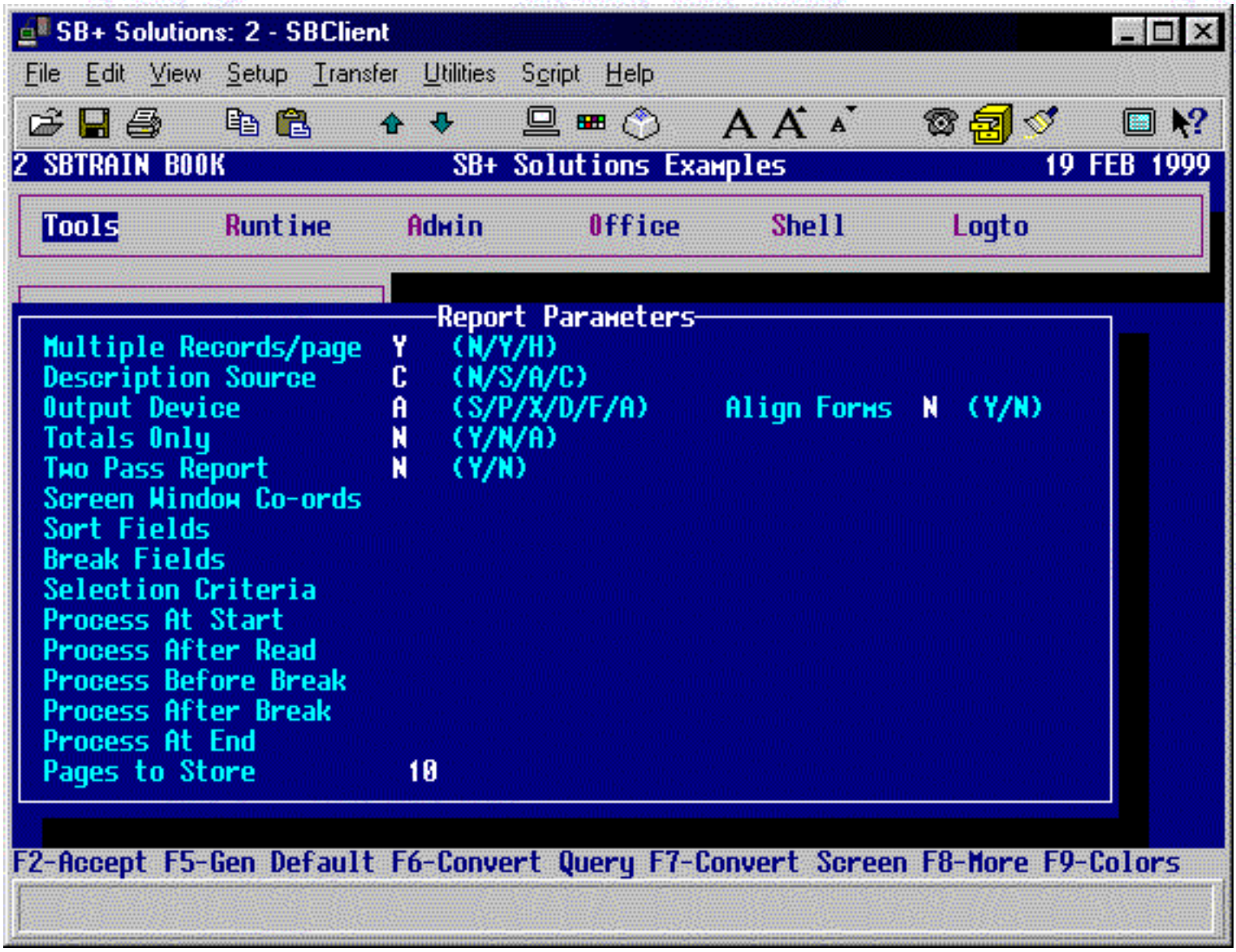
Take a look at the second process call syntax shown earlier. In this particular syntax we're calling a process and passing three fields to the process in @PARAM. This particular method of passing parameters in a Report Writer report has some distinct advantages over parameter passing in the other tools. In the Report Writer, these fields can be not only @RECORD and @WORK fields as defined in the field definition, but also any derived value fields names that exist on the report definition. This allows you to have SB+ pass the right values into the process without having to rebuild all of the logic in a Process After Read.

Note, however, that the expressions themselves will be executed twice -- once for their own field, and once for their use in this process call field. Therefore, you're not actually saving any time in using this technique, but rather it is a simpler way to achieve the desired result.

If time is of the essence, and duplicate processing of complex derived values is out of the question, simply create a Process After Read to do all of the calculation, storing the results in a @PARMS(...) element, and then reference the @PARMS(...) element in your derived value fields. Note, however, this will not always work as expected on reports with break fields, as the Process After Read happens prior to the breaks.

Creating a Form Report

Creating a form report is basically the same as creating a columnar report. Using the Report Definitions tool, enter the name of the file you want to generate the report for. Then, when asked to enter a report definition name, simply enter a name which (briefly) describes the report you want to create. Of course, your naming standards will apply. Once these have been entered, the F6-Params screen will appear as follows:



Once this screen is displayed, enter "N" at the "Multiple Records Per Page (Y/N/H)". This defines the report as a form report.

When you press F2 from the F6-Params screen, the regular report definition screen will appear. Unlike a standard columnar report, however, you don't need to worry about setting up a heading and/or footing. Instead, simply put a "D" in the upper left corner, which will define the first line (and all subsequent lines) as one big detail section. With this accomplished, the only thing left to do is place the fields using F5, maybe place some graphics with F9, and generally put together a beautiful, customer-pleasing report.

Fields on a Form Report

Unlike a columnar report, you must define exactly how many values of each multivalued field are to be shown on a form report. Considering that a form report definition defines both the output for a single record as well as the output for each page of the report, you can see how important it is to define exactly how the page will look when the report is printed.

However, determining the number of values to print on a page can be a troublesome task. If you allocate space for too many values, when fewer values are printed there will be significant amount of white space (unprinted area) on the page. If you allocate space for too many values, some values may be truncated. It's important to find the balance between these two extremes.

When using F5-Field to place a multivalued field on a form report definition, simply enter the number of values to be shown at the "Max MV Lines" prompt. If there are more values than the maximum as defined, the extra values will simply not be shown.

However, of how much value is this? Will YOUR customers be happy to know that they can only see the first 20 items that they ordered? Of course not! For most applications, this behavior is simply not acceptable.

Instead, enter the maximum number of values as a negative number. Therefore, if you want 20 lines to be shown, enter -20 in the Max MV lines prompt. This tells the Report Writer that 20 values of this prompt will fit on this page, and all subsequent values should be shown on a new page. With this option, all values will be shown, though it may take several pages to show them all.

When the second and subsequent pages are printed, all single value fields will be printed on the new page. This will repeat the valuable reference information (such as the order number or customer number) on all pages.

Numbering Pages on a Form Report

Putting a traditional page number on a form report is useless more times than not. Which of your customers care that their statements were pages 101-104? Instead, it would be more useful to print a page number on the report going from 1-4, with the page number resetting itself after every record.

To do this, create a derived value field somewhere in your form report definition. Make the field alphabetic for 20 characters, and enter the following derived value:

```
'Page ' : @RV.MV.PAGE : ' of ' : @RV.MV.PAGE.CNT
```

This expression will output the message "Page 1 of 3" on the first page of three, or "Page 2 of 6" on the second of six pages. In short, it does exactly what we need.

When multivalued fields are output on a form, SB+ determines the number of pages that will be needed to output the values. This is stored in the report variable @RV.MV.PAGE.CNT. As the report is cycling through the data, @RV.MV.PAGE keeps track of the current page number that is being output for the current record. Therefore, when each record is processed, @RV.MV.PAGE is reset to 1, and @RV.MV.PAGE.CNT contains the number of the last page that will be printed for this record.

Of course, this is just one possible way to format these two variables. A shorter form could appear as follows:

```
@RV.MV.PAGE : '/' : @RV.MV.PAGE.CNT
```

This form will print "1/3" for the first of three pages, or "2/6" for the second of six. Naturally, like many issues in SB+, only you and your needs can define what looks best on your report.

Creating Combination Summary/Detail Reports

In certain applications, it can be useful to be able to print three different types of reports: a **detail report**, a **summary report**, and a combination of the two in a single report. To do this, we have a couple of different options:

Option #1: Create Multiple Report Definitions

As shown in the previous section, we can easily create a paragraph with three options, such as the following:

```
LOCAL ANS
*
ANS = 'D'
INPUT ANS,1,'Summary, Detail, or Both (S/D/B)'
*
IF @OTHER(18) # 14 THEN
  BEGIN CASE
    CASE (ANS = 'S')
      EXEC 'summary report process'
    CASE (ANS = 'D')
      EXEC 'detail report process'
    CASE (ANS = 'B')
      EXEC 'summary report process'
      EXEC 'detail report process'
  END CASE
END
```

For some small reports, this may be adequate. However, if there will be numerous details to be printed, and the data is even remotely dynamic, there is a chance that the data could change during the execution of the detail report, and when the summary is printed following the detail (using the 'B' option from this paragraph) the totals will not match. Such problems can be very troublesome to diagnose.

In this case, it would be better to build the summary while the detail is being printed, and then output the summary information at the end. This essentially guarantees that the summary and detail will match, regardless of changes to the data while the report is running.

Option #2: Building Summary Information During Detail Processing

When the report is running, any process listed in the Process After Read (on the F6-Params window) will be executed for each record processed. This is an excellent place to create a process which will summarize information to be printed at the end of the report.

As an example, let's say we want to create a report which shows a list of sales by salesman, with a summary at the end. Such a report might look something like the following:

```
Order Salesman Order Total
xxxxx xxxxxxx   xxxxxxx.xx
xxxxx xxxxxxx   xxxxxxx.xx
xxxxx xxxxxxx   xxxxxxx.xx
xxxxx xxxxxxx   xxxxxxx.xx
xxxxx xxxxxxx   xxxxxxx.xx
```

```
***** Summary *******
Salesman  Order Total
xxxxxxx   xxxxxxx.xx
xxxxxxx   xxxxxxx.xx
xxxxxxx   xxxxxxx.xx
xxxxxxx   xxxxxxx.xx
```

For each order record processed, we want to update a summary of totals by salesman. To do this, we need a couple of processes.

The first process will be used to clear some area of common memory to hold the summary information. For the purpose of this example, let's assume we're going to build this into @PARMS(2). Therefore, we need to create a paragraph called from the Process At Start (in the F6-Params window) which appears as follows:

```
@PARMS(2) = ''
```

With this process in place we can rest assured that our totals will start with a clean slate each time the report is invoked.

Next, we need to create the actual process which will update @PARMS(2) for each record. This process will appear in the Process After Read for the report and could appear something like the following:

```
LOCAL NDX
*
NDX = LOC(SALESMAN.ID,@PARMS(2)<1>,@VM)
IF NDX = 0 THEN
  NDX = DCOUNT(@PARMS(2)<1>,@VM) + 1
  @PARMS(2)<1,NDX> = SALESMAN.ID
END
*
@PARMS(2)<2,NDX> = @PARMS(2)<2,NDX> + ORDER.TOTAL
```

The end result of this process is to build two multivalued arrays in @PARMS(2), the first holding a list of salesman IDs, and the second holding corresponding sales amounts.

Looking at this process a little closer...

For each record that is read, we will use the LOC(...) function to find the salesman ID in the summary list (in @PARMS(2)<1>). If the ID is found, NDX will tell us where. If the ID is not found, we need to add the salesman to the end of the list and update NDX so we know which bucket to put his or her sales into.

The last line of this paragraph simply says to add the total for the individual order record to the bucket as pointed to by the NDX **local variable**.

This particular technique can be adapted in any number of ways. If you need to have several columns of totals, simply allocate an attribute of @PARMS(2) for each column you want. Want the information to be sorted in the summary by salesman? Not a problem -- either restructure the paragraph to insert the codes in order, or write a quick sorting routine that can be called prior to outputting the summary. This technique can also be used to create summaries based on multiple fields, such as salesman and state. Really, the possibilities are mind-boggling.

To output these fields on the report, create multivalued derived value fields in the grand total section of the report. To do this, move the cursor where you want the first column of summary information to be displayed, and use F5 to add a derived field. Be

sure to set "Suppress Repetition" to "L" and "Max MV Lines" to "0.M" to tell SB+ that this is a multivalued derived field. The derived value for the first column will be:

@PARMS(2)<1>

The second derived field, then, will be created the same way and will have a derived value of:

@PARMS(2)<2>

Other than perhaps an optional conversion, this is really all there is to it.

The ReadNext ID Process

At the "ReadNext ID Process" slot, enter the name of a process which will calculate a key for a record to be shown on the report. The process called from this slot has the following responsibilities:

- The key value to use must be returned in @KEY, and
- If there are no more key values, @RTN.FLAG must be set to "1".

Other than these issues, you are free to use whatever means you have to calculate the key. The tricky part is to get the starting key into the picture. (This is most often accomplished by passing the key into the report in a @PARMS(...) or @USERDATA(...) element).

Following our linked list example, let's say we've stored the starting key in @PARMS(2). With this foundation, we can write a process for this slot which appears as follows:

```
@KEY = @PARMS(2)
IF @KEY = '' THEN EXIT 1
```

Basically, this process gets the key from @PARMS(2), then checks to see if there is actually a value. If there is no value, the **EXIT** 1 will stop the report.

But where does the next value come from? To accomplish this, let's assume we have a field in the record called NEXT.KEY. With this in mind, we can create a process after read which sets @PARMS(2) to the next key, as follows:

```
@PARMS(2) = NEXT.KEY
```

With this process installed, after each record has been read the key for the next record will be extracted from the record and installed in @PARMS(2). When SB+ goes to get the next key, it simply looks in @PARMS(2) for the value that was put there by the Process After Read. Finally, when the next key value is null (at the end of the linked list), we know we've arrived at the end of the list, and the report will stop as expected.

The Read Record Process

Normally, when the report is running SB+ reads a record from the current file for each record selected. Using this process slot, you can eliminate this read and install your own read logic instead.

On the surface, this may seem a bit superfluous. SB+ does a good job reading each record, so why would anyone want to change it? Odd as it may sound, there are times when you may want to take control of reading the record.

For example, let's say you need to generate a sales report that brings information from a number of files named SALES.1990 through SALES.2000. To run the report, a periodic update is used to cycle through all of the records in all of the files, writing the key to each record into a temporary file. After the periodic update is complete, you have all of the record keys from all of the 11 files stored in that one temporary file. Now that all of the keys are together, we can select this file, save a list of keys, and then run the report from the list.

SB+ knows to read a record from the current file, but in this case we have 11 files to use in getting the information for the report! How can we tell SB+ to read the record from one of the 11 files?

For this particular example, the following paragraph, called in the Read Record process of the report definition, will work quite nicely:

```
LOCAL YR
*
YR = 2000
WHILE YR >= 1990 DO
  @RTN.FLAG = 0
  READ @RECORD FROM 'SALES.':YR,@KEY
  IF @RTN.FLAG = 0 THEN EXIT
  *
  YR = YR - 1
REPEAT
```

When we use this process slot, SB+ hands us @KEY with the record key. It's up to us to find the file that the record lives in. In this particular example, the paragraph counts from the year 2000 backward to 1990, attempting to read a record from each of the files. If it finds a record (that is, @RTN.FLAG remains 0 after the READ), the paragraph exits and the record as found is used in the report. If, however, the WHILE loop finishes and no record is found, @RTN.FLAG remains set to 1. This, like the same setting in the Process After Read, tells SB+ that there is no record to be output, and therefore that key is skipped.

The Next Id and Record Process

What happens when you need to take control of both the traversing of the records and the reading of the information for the report? When you have such a need, the Next Id and Record process has the perfect solution.

When a process in this slot is called, SB+ hands you nothing -- except, of course, what you may have put into common memory somewhere. You are then responsible for setting @KEY and @RECORD for each record to be output. When you have no more values to be output, your process must set @RTN.FLAG to a 1 to tell SB+ that the report has completed.

This is perfect for traversing a linked list of records, where record A points to B, B points to C, and so on. Assuming we have such a connection between customers, we could create a process for traversing one customer and all of their connections, as follows:

```
@KEY = @PARMS(2)
IF @KEY = "" THEN EXIT 1
*
READ @RECORD FROM "CUSTOMERS",@KEY
*
@PARMS(2) = NEXT.ID
```

This is similar to the logic used in the example shown for the **ReadNext ID process**. However, in this slot we can combine the setting of the current key with the setup for the next key (in @PARMS(2)). Of course, this assumes that some process ahead of the report has set @PARMS(2) with the starting value!

If our customers were in a number of different files, we could still use this process for getting the key and record, as follows:

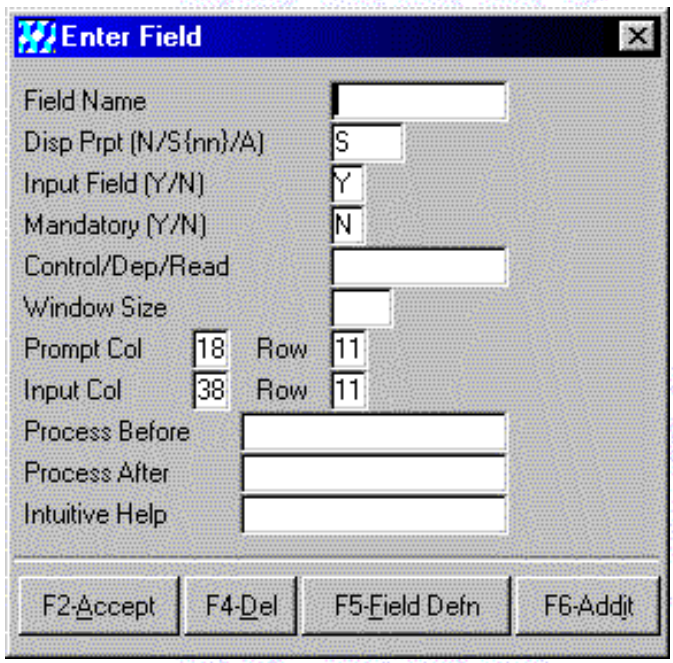
```
@KEY = @PARMS(2)
IF @KEY = "" THEN EXIT 1
*
READ @RECORD FROM "CUSTOMERS",@KEY
IF @RTN.FLAG THEN
  @RTN.FLAG = 0
  READ @RECORD FROM "OLD.CUSTOMERS",@KEY
  IF @RTN.FLAG THEN EXIT 1
END
*
@PARMS(2) = NEXT.ID
```

In this example, the customer record could live in either the CUSTOMERS file or the OLD.CUSTOMERS file. This process will get a key, read a record from one file, and if the record is not found (@RTN.FLAG = 1) the process will look in the other file. If the record cannot be found in either file, the key will be skipped (by setting @RTN.FLAG to 1 using **EXIT** 1). Finally, note that @PARMS(2) is set at the end of the process to the next ID to be processed (which is assumed to be a field in the customer record).

Adding a Toggle Object to a GUI Form

A toggle object (also known as a "check box") is useful when there are only two valid options for a prompt, such as Yes/No or Male/Female. Unlike many of the other GUI objects, a toggle never really has a value; instead, it has a *state*. Essentially, toggle objects are either "on" or "off", and as a result of being set one way or the other, the underlying record is updated.

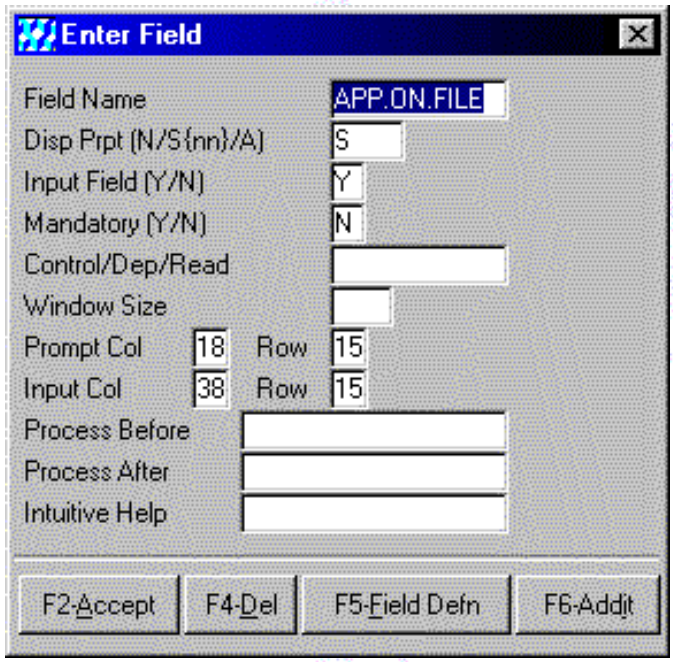
To add a toggle object to a GUI form, click on the  button on the object bar. Once this button has been clicked, the mouse pointer will be changed into a cross, which is your indication to position the object where you want it to appear. To do so, simply move the mouse pointer to the position on the form where the toggle object should be placed, and click the left mouse button. Once the button has been clicked, the standard F5-Field window will appear as follows:



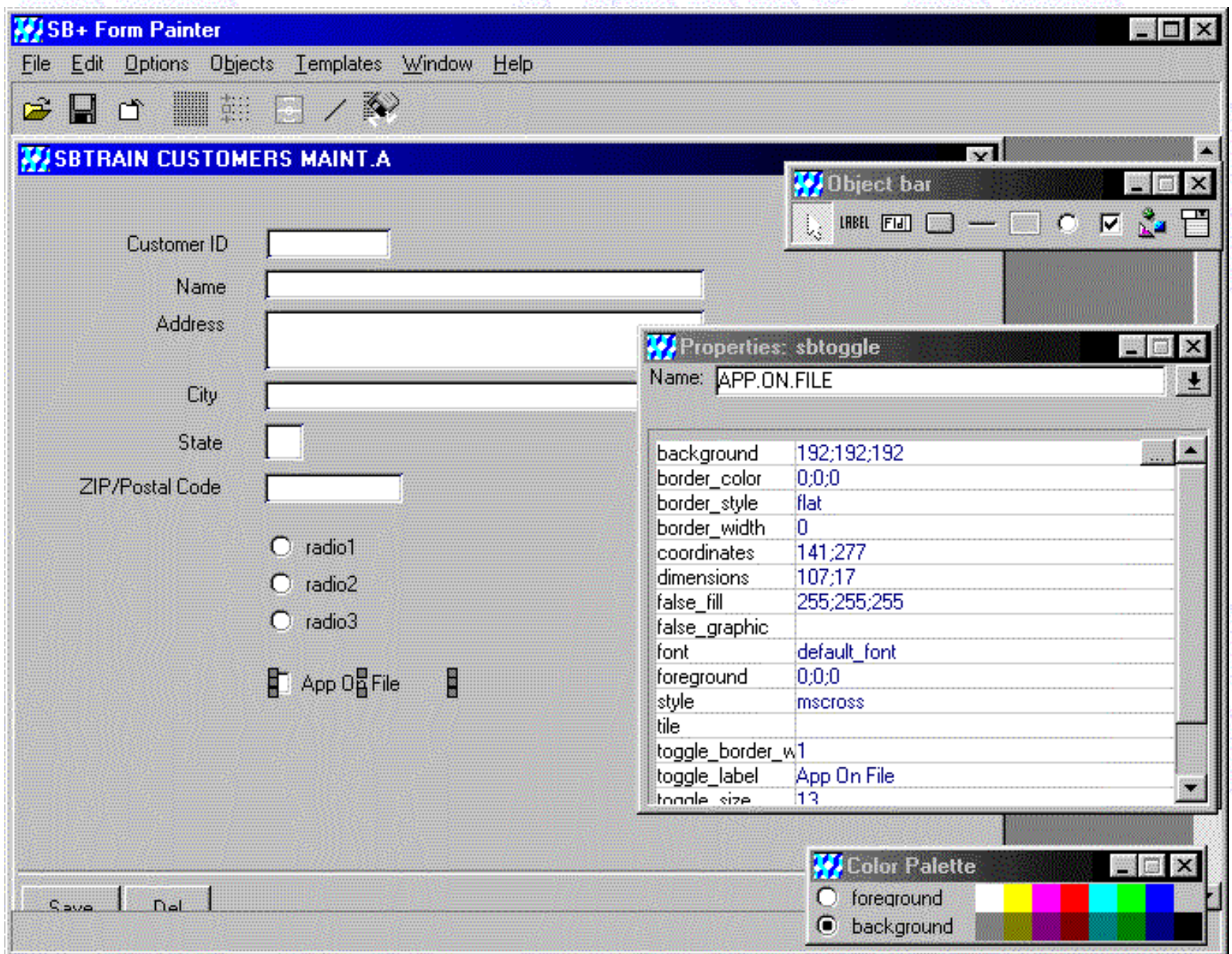
At the Field Name prompt, enter the name of a field which has an **appropriate validation** for a check box, such as a table validation or a list of valid values. For this example, we have a field called APP.ON.FILE which is validated against a table named YES.NO, as follows:

```
E:TABLE('YES.NO')>"{[E42]}
```

Therefore, we can enter this field name into the Field Name prompt, so that the Enter Field window appears as follows:



Once F2 is pressed, the form is updated with the new field, as follows:





At this point, we can then **move the object** anywhere we want. Toggle objects are unlike text fields in that when the toggle moves, the label for the toggle moves as well.

Tables and lists of valid values can obviously have more than two options. However, when a toggle object is used, only the first two options in the table or validation list are valid. The first option in the list is the value to be updated into the record when the toggle is "on" (that is, when there's a check mark in the box), and the second option in the list is the value to be updated into the record when the toggle is "off" (no check mark). Of course, you can always use /EE to verify the effect of checking and unchecking the box while you're testing the screen!

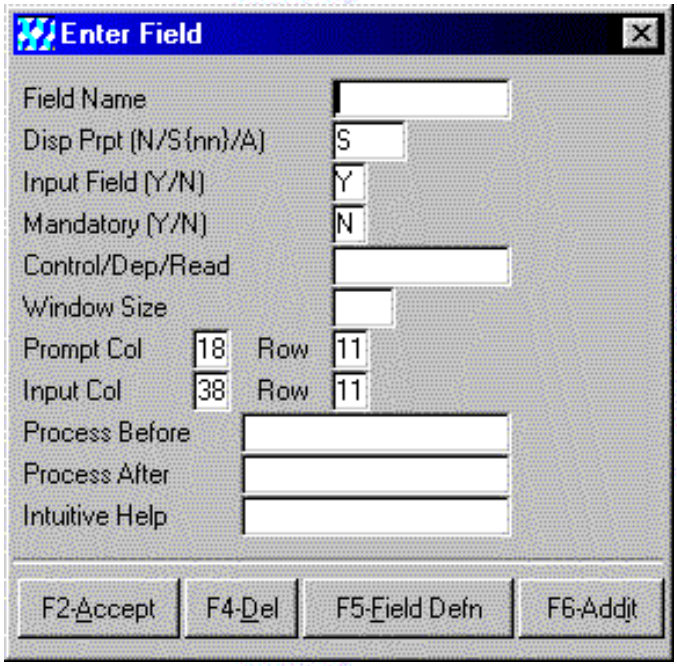
Various additional changes can also be made to the appearance of the object using the properties shown in the Properties window. See "**GUI Object Properties**" for more information about changing properties.

Adding a Combo Box Object to a GUI Form

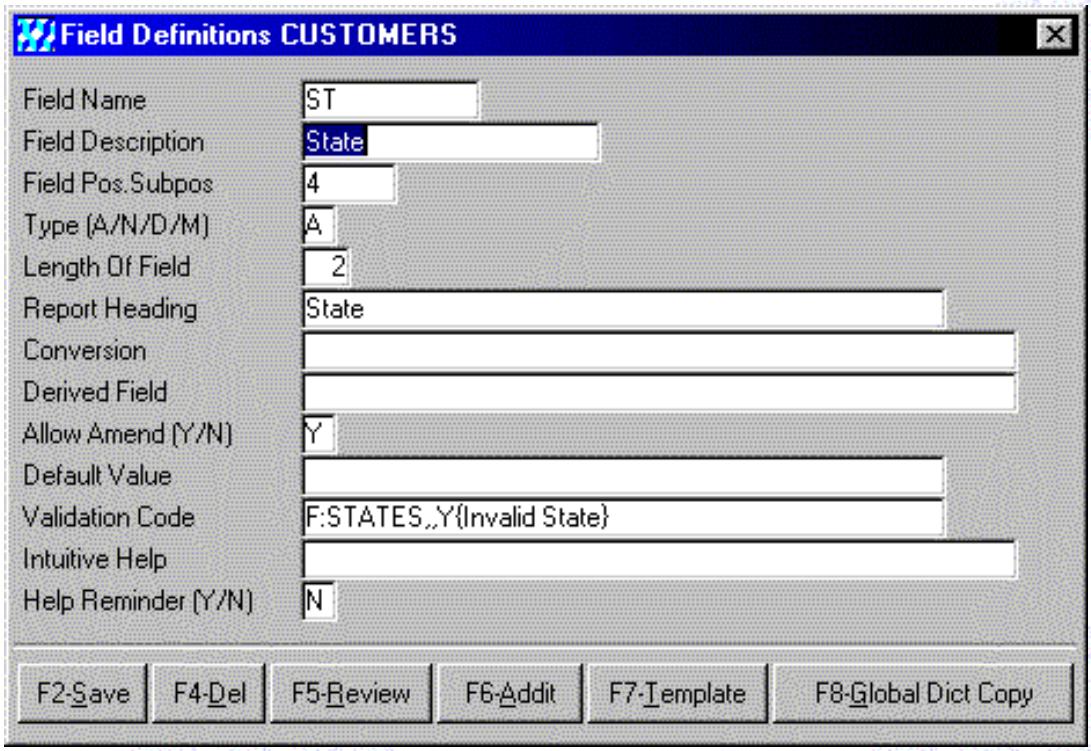
A combo box object (also known as a "drop down list") is an effective object to use when there are a finite number of available options for a prompt, but there are too many for either a toggle (where only two are valid) or radio buttons (where only a small number can fit on a screen at a time). There are two different types of combo boxes available:

- *Non-editable combo boxes* display a list of valid options to select from. The user cannot add to the list or edit any value shown in the list.
- *Editable* combo boxes display a list of valid options to select from, however, the user is allowed to edit the value entered and add new items to the combo box list (and thereby the source of that list).

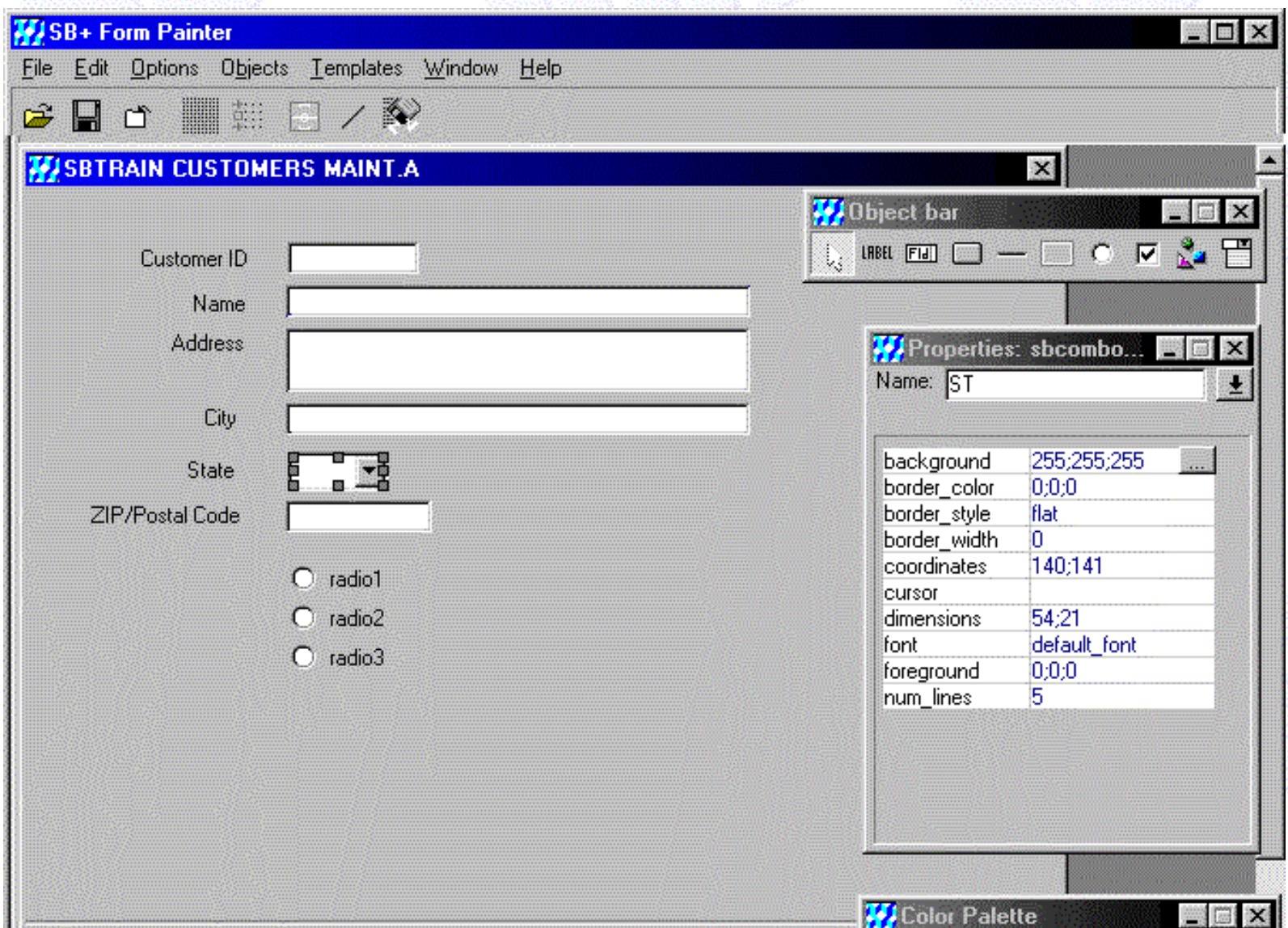
Both types of combo boxes are added to the screen using the same technique. To add a combo box object to a screen, first click on the  button on the Object bar. The mouse pointer will then be changed into a cross, which is your indication to position the pointer where you want the object to appear. To place the object, move the mouse pointer to a position on the form and click the left mouse button. The object will then be created at that location and the following screen will appear:



At the Field Name prompt, we must enter the name of a field which has an **appropriate validation** for a combo box object. For this example, we have a state field (named ST) in our customer screen which will serve well as a combo box:



Once the field name has been entered into the Field Name prompt, we can press F2 to place the field on the form. The form then appears as follows:





Once this prompt has been added to the screen, the combo box object and/or its associated label object can be **moved** to the appropriate position or resized.

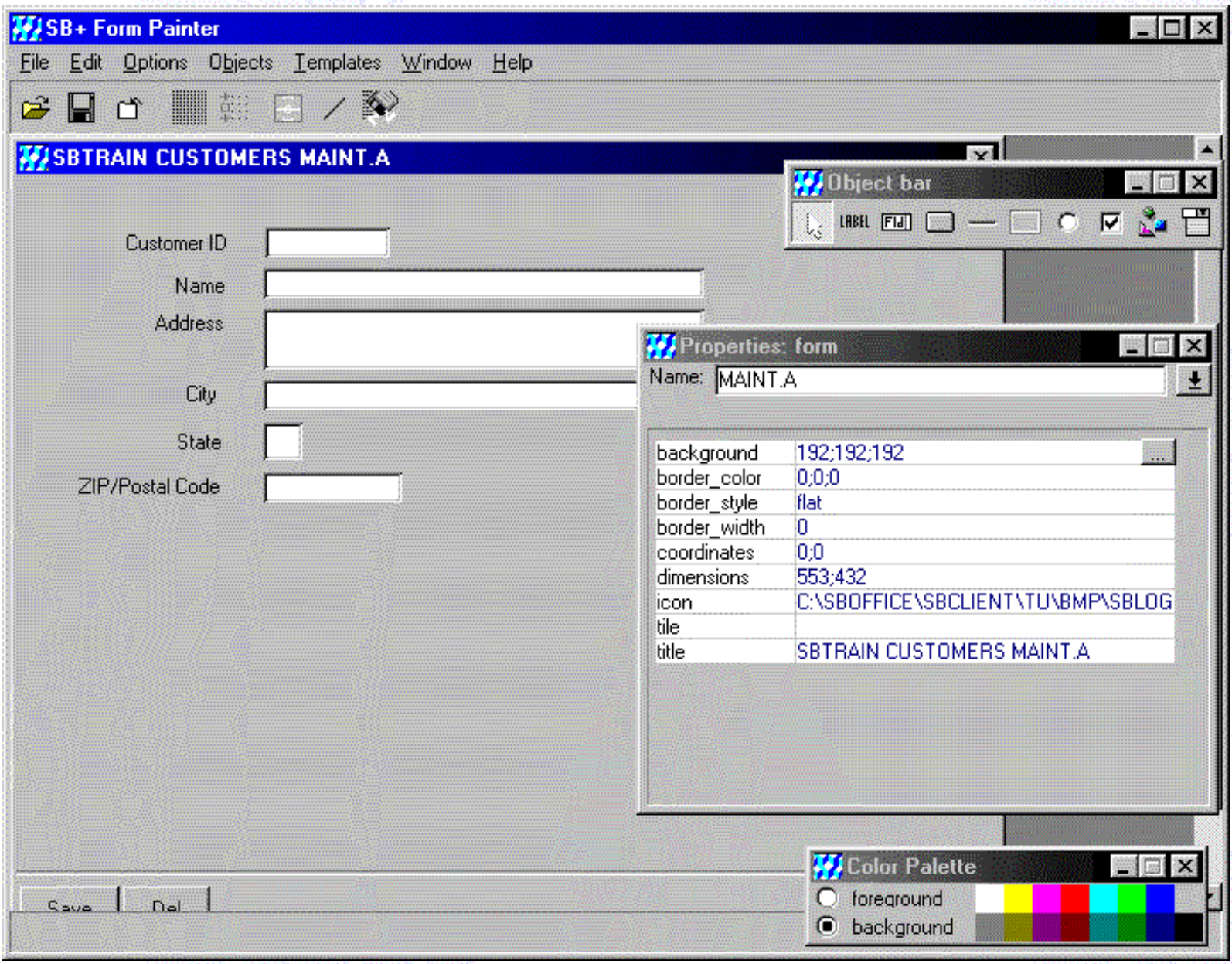
When this screen is running, the user can click on the arrow shown to the right of the input area and a list of states will be displayed. The user can then select one of the items from the list either with the mouse or with the up and down arrow keys.

Adding a Radio Button Object to a GUI Form

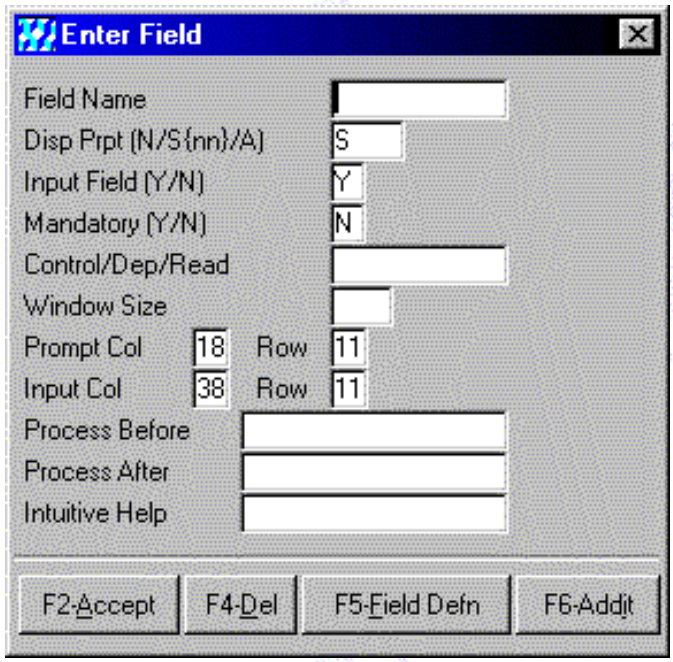
Radio button objects are the perfect solution to a prompt with a small, definitive number of valid options. This object allows you to present all of the available options to the user at the same time, and the user can then click on the appropriate option.

Once a **GUI screen has been created**, adding a radio button object is easy. First, click on the  button on the Object bar. The mouse pointer will then be changed into a cross, which is your indication to position the pointer where you want the object to appear. To place the object, move the mouse pointer to a position on the form and click the left mouse button. The object will then be created at that location. (You can also click and drag to create an object that is bigger than the number of options you need at the present time, thereby planning for the future.)

For example, if we're creating a customer screen, we may have a form that looks something like the following:



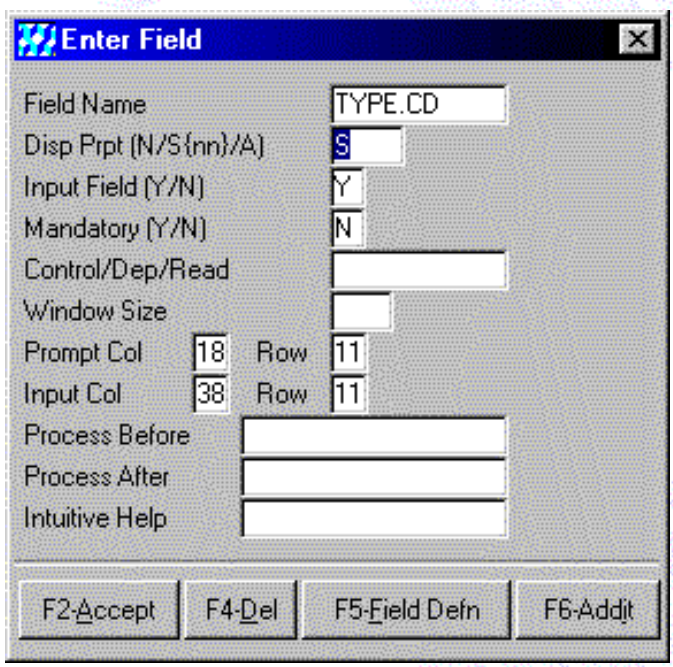
Under the ZIP/Postal Code field, we can add a series of radio buttons which allow the user to select one of a few customer type codes. To do this, we simply click on the radio button in Object bar, move the mouse pointer to the appropriate location, and then click the left mouse button. Once the left button has been clicked, the following window appears:



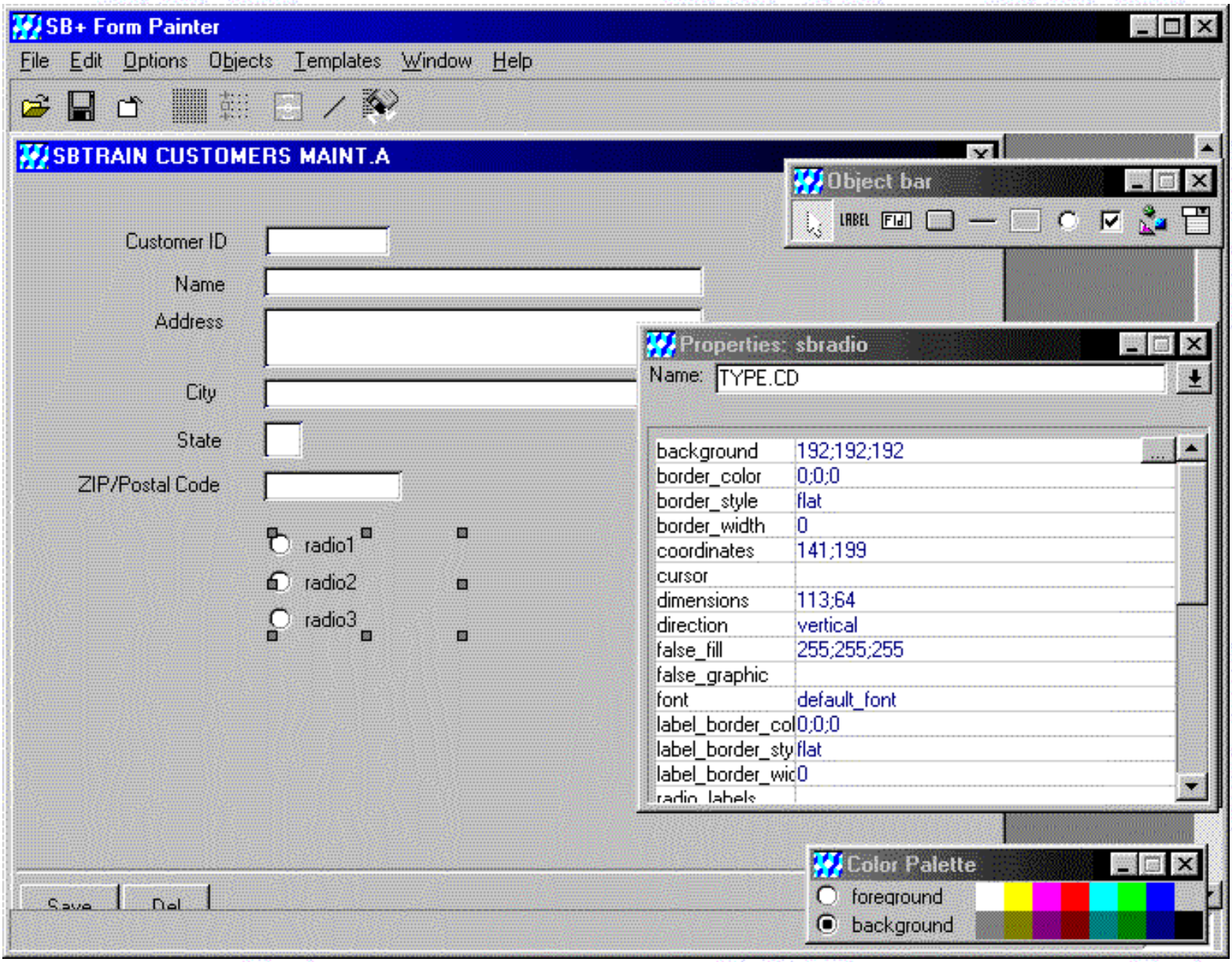
At this prompt, we need to enter the name of a field that can be displayed using radio buttons. For this example we'll use a field called TYPE.CD which is validated against a table of type codes with the following validation code:

E:TABLE('TYPE.CDS')>"{[E42]}

Therefore, I can enter this field name at the first prompt as follows:



Once F2 is pressed from this screen, the radio button field is added to the form, and the form appears as follows:



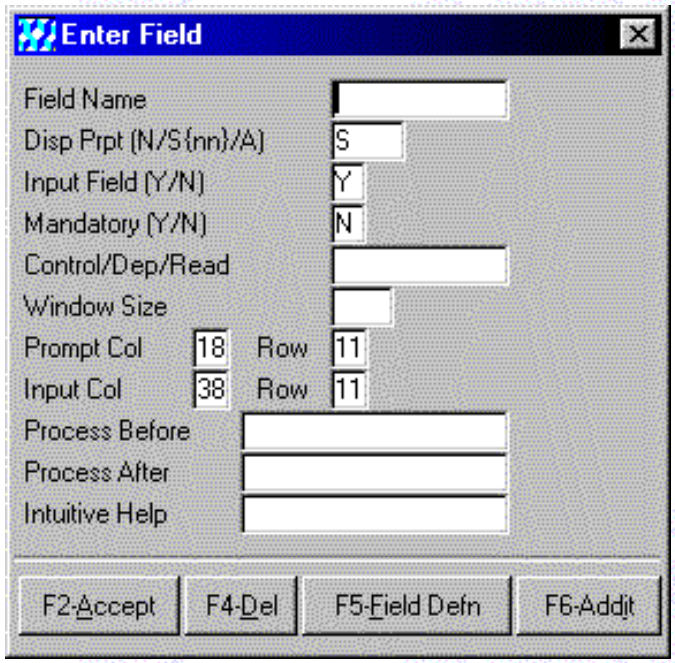
Note the labels for the radio buttons -- radio1, radio2, radio3. These are merely placeholders. When the screen is executed, the descriptions from the table entries will be shown on the running screen instead.

At this point, we can then **move the object** anywhere we want. Radio buttons are unlike text fields in that when the radio buttons move, the labels move as well. Note that the radio button object will adjust its size based on the number of items in the table or validation code. Therefore, be sure to allocate enough space on the screen for the object to grow or shrink (by a couple of options) depending on the dynamics of the information used for validation.

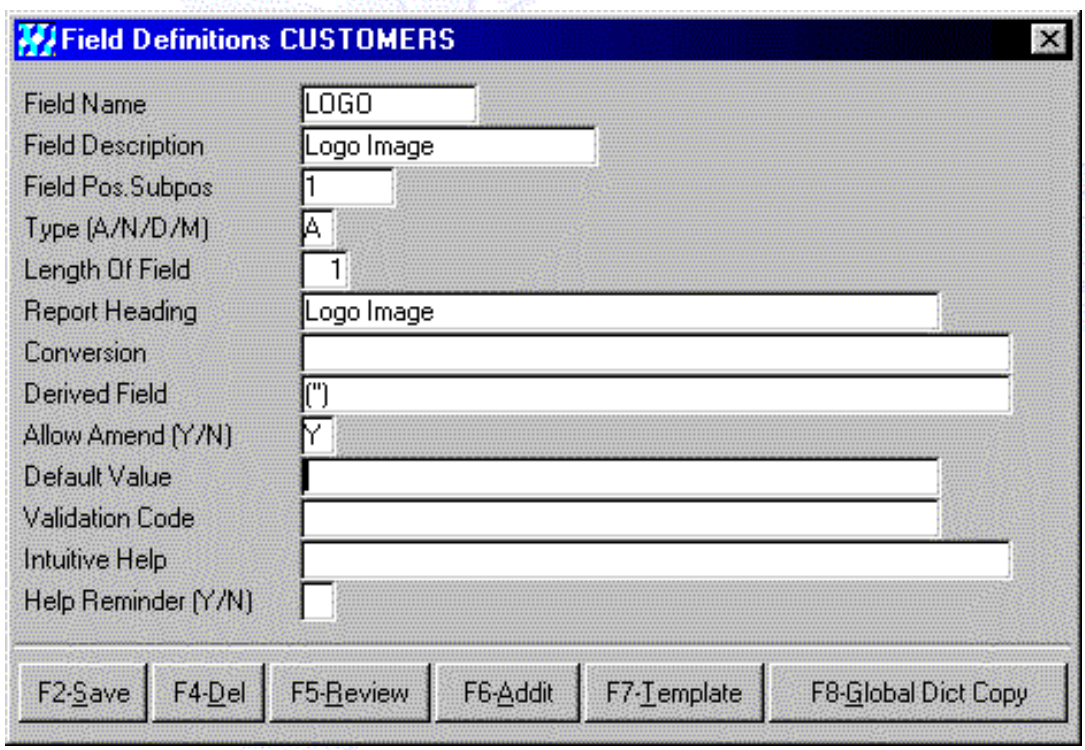
Various additional changes can also be made to the appearance of the object using the properties shown in the Properties window. See "**GUI Object Properties**" for more information about changing properties.

Adding a Static Image Object to a GUI Form

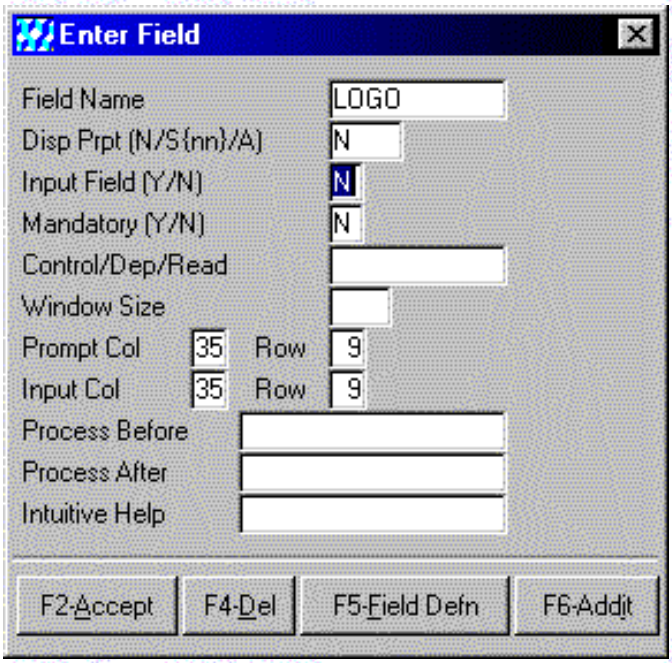
To add a static image object to a GUI form, first click on the  button in the Object bar. Once this has been clicked, the mouse pointer will be converted to a cross, which is your indication to position the pointer where you want the image to be placed. Move the pointer onto the GUI form, and then click and drag the lasso to create a box. When the box is sized properly for the image to be displayed (or reasonably close), release the mouse button. The following screen will be displayed:



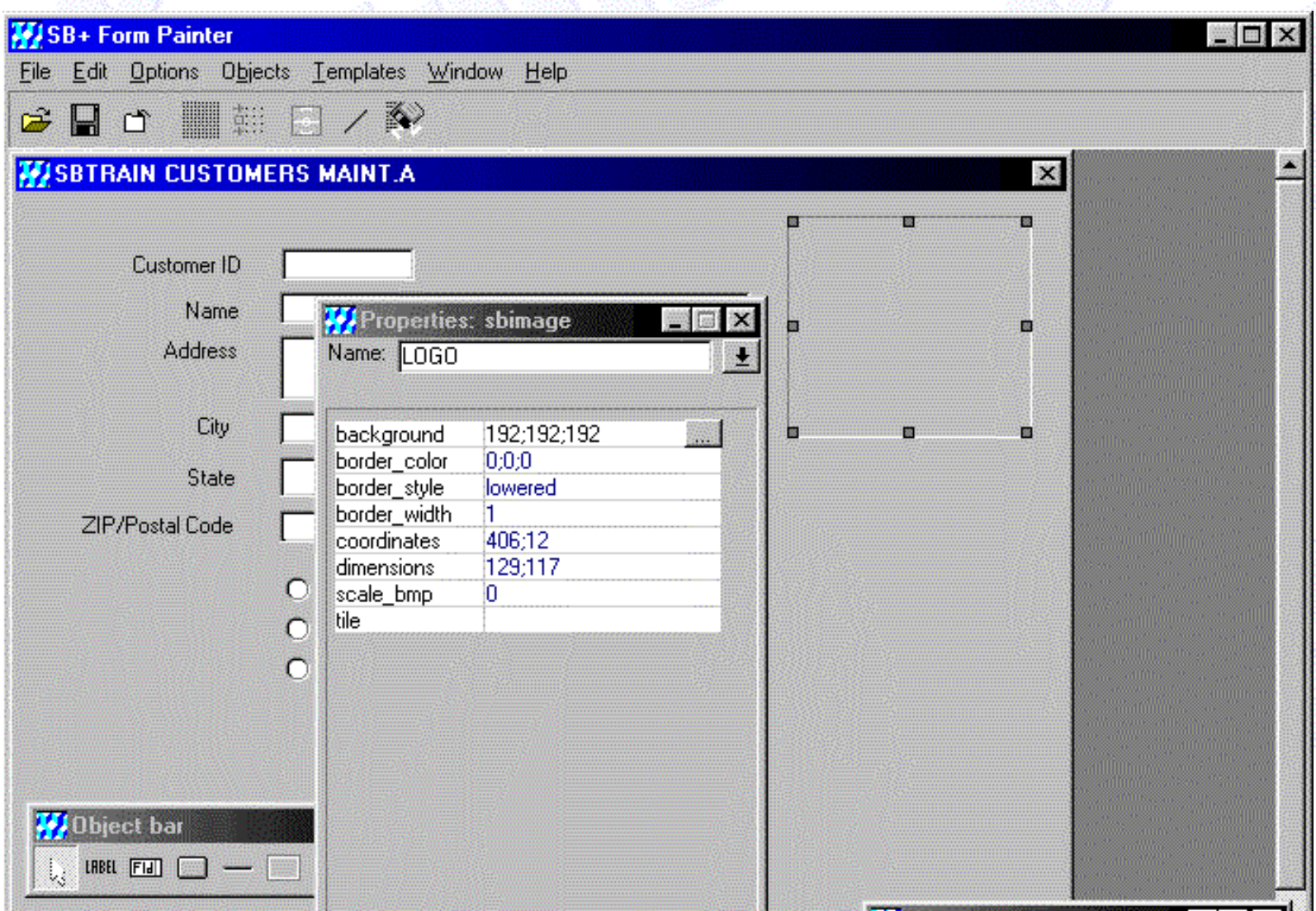
In this window, we have a couple of things to do. First, we need to enter a field name to be displayed. This seems somewhat unnatural, because we don't really want to place a field here -- we just want to show an image. Nonetheless, SB+ mandates that we create a field here, so let's create a new field called LOGO which appears as follows:



This field is a simple derived value, where the derived value is null (quote-quote inside of parentheses). This *nothing field* is perfect for a static image. Back on the screen, we need to be sure this is not an input field. (This will be automatically set properly because we're using a derived value field.) It's also a good idea for images like this to be shown without a display prompt, so let's also set the Disp Prpt field to "N", as follows:



Once F2 is pressed, this field will be added to the form, and the form appears as follows:





As you can see, our image field is the block shown in the upper right corner. (I've moved the Properties window so you can better see the positioning of the image area.)

Now, we need to tell SB+ which image is to be displayed. We do this by changing the "tile" property in the Properties window.

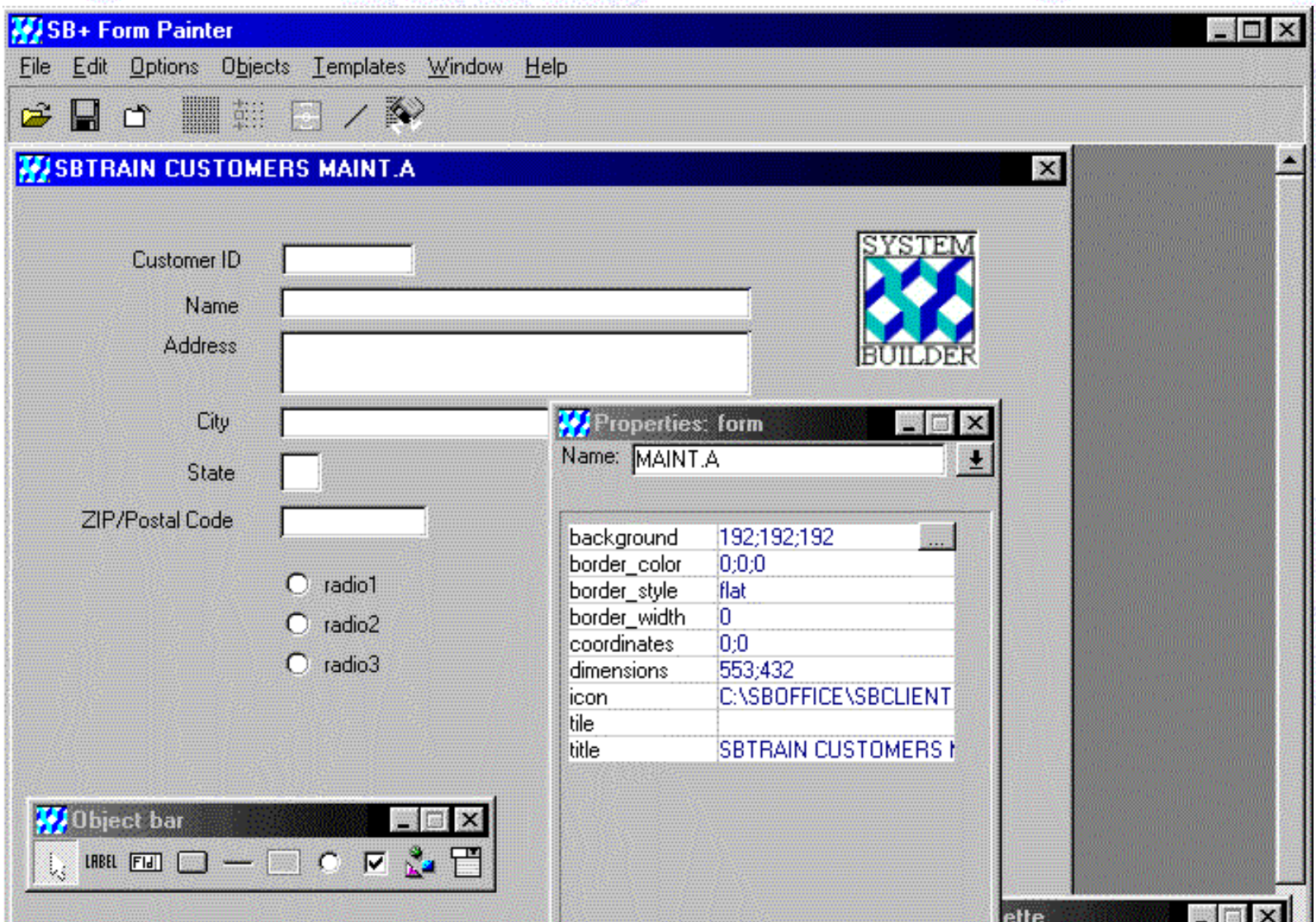
The "tile" property describes the background that is to be displayed in the image box. When this property is changed to nominate a graphic file (GIF, JPG, or BMP), that graphic will be displayed at that location. You can then resize the control to fit the image, or...

...The image can be instructed to resize itself to fit the box! In the Properties window, find the "scale_bmp" property. This property has two states:

- True: The image will be scaled to fit the box
- False: The image will not be scaled, and will instead be tiled (repeated) to fill up the area of the box.

Change this property to the appropriate setting for the results you want. Note that there may be some degradation of the image as it is scaled from a smaller to larger size.

Now that the image has been added to the form, the completed form appears as follows:





One important factor to keep in mind: When displaying images such as this one, all users of the form must have access to the graphic to be displayed. Therefore, take care so you don't include graphics that are stored on protected servers or other machines that are not available to the user public.

Note: Images can also be added to forms using Rectangle objects, though there are restrictions on that approach. (See "**Adding Graphics to a GUI Form**" for more information on using rectangles.)

Adding a Dynamic Image Object to a GUI Form

To add a dynamic image object to a GUI form, first click on the  button in the Object bar. Once this has been clicked, the mouse pointer will be converted to a cross, which is your indication to position the pointer where you want the image to be placed. Move the pointer onto the GUI form, and then click and drag the lasso to create a box. When the size of the box is sized properly for the image to be displayed (or reasonably close), release the mouse button. The following screen will be displayed:

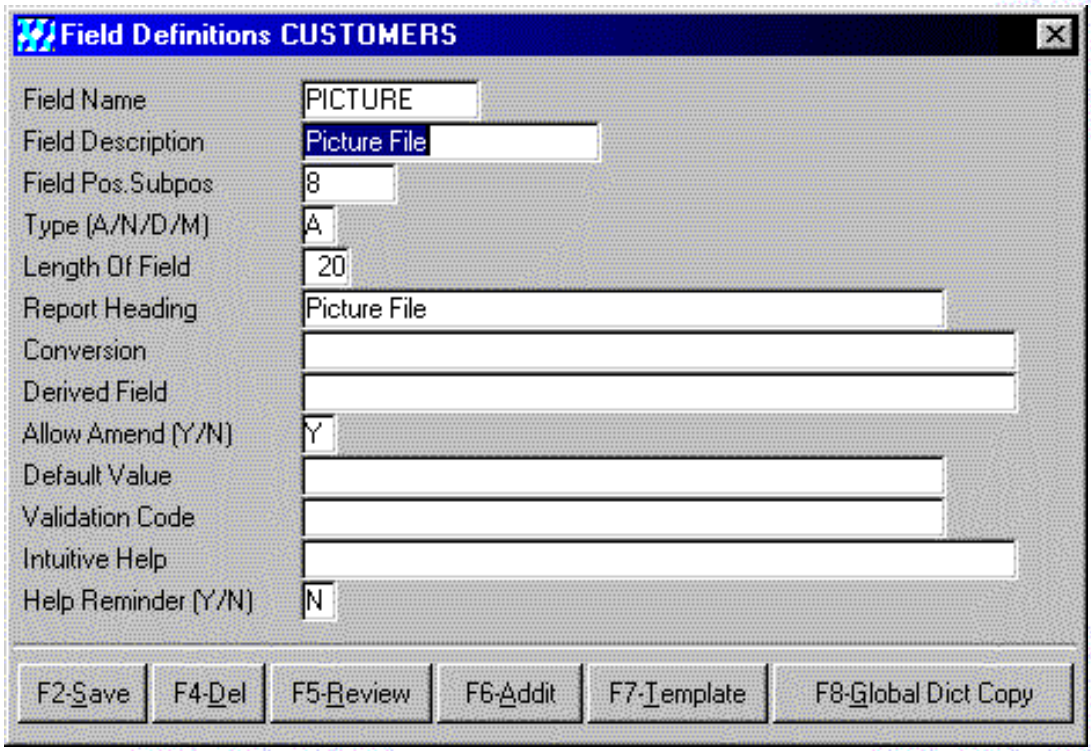


Unlike placing a **static image**, where you place a derived value "nothing field" for the image, the field placed for a dynamic image must be a "real" field (that is, a field with a value). This field then, must contain a string which describes where the image is stored, such as:

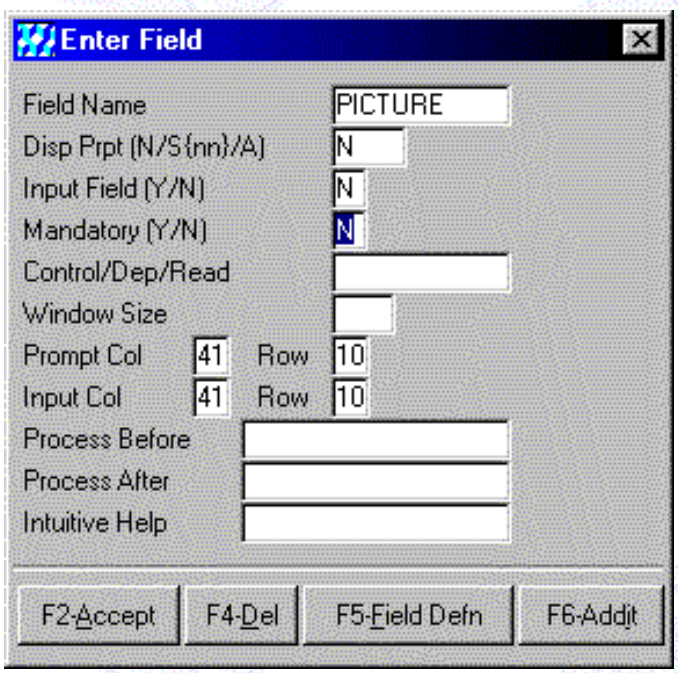
E:\PIX\CUST010.JPG

(For a list of acceptable file types that can be displayed on a GUI form, see the "**GUI Objects Overview**" elsewhere in this chapter.)

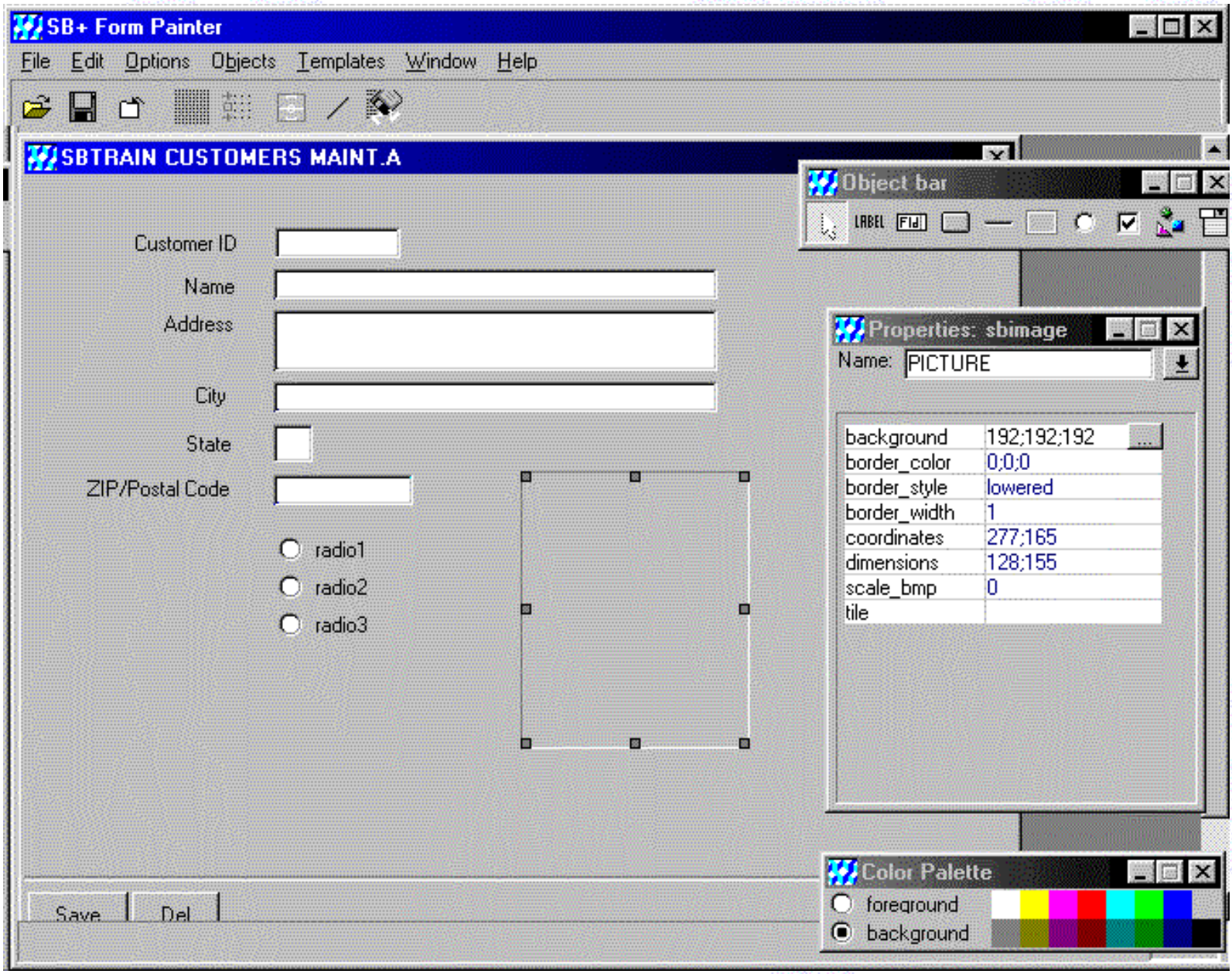
For this example, we have a field in the record called PICTURE which appears as follows:



We also need to be sure the Input Field prompt is "N", as image fields are display-only objects. Finally, for this example, I don't want to see a title for the field, so we'll set the "Disp Prpt" field to "N" as well. The window then appears as follows:



Once F2 has been pressed, the object is added to the form, usually with the wrong size. This is not a problem, however, as we can click and drag any of the handles around the object to make the object smaller or larger. Once this is complete, the GUI form looks like:



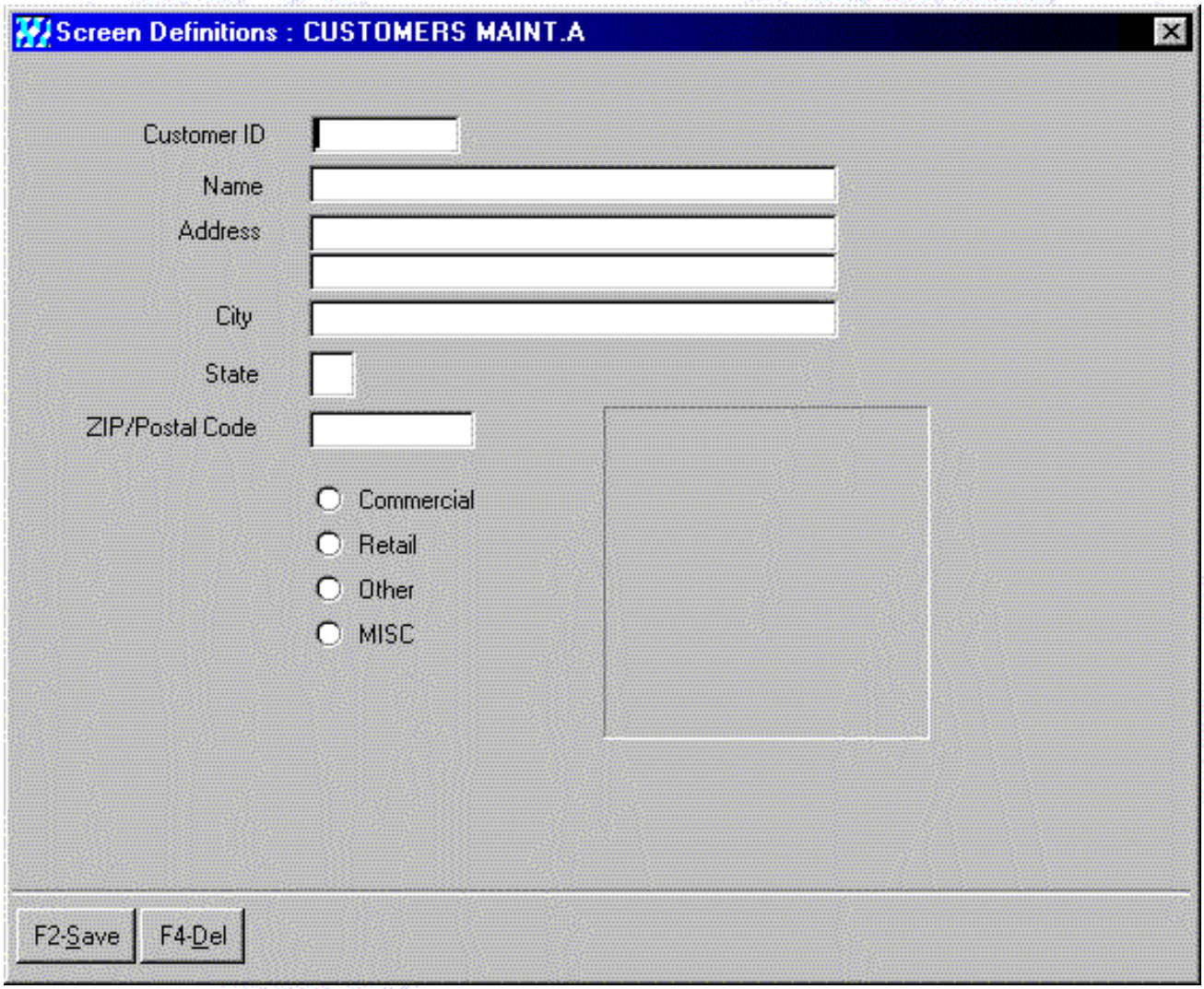
Note that image objects are automatically added to the form as tiled objects, which means the graphic will be tiled (repeated) until the area has been filled. If the image is too large for the area, it will be truncated. In this example, rather than try to create graphics which are all the same size and shape, it may be better to tell SB+ to resize the graphic to fit the box instead. We do this by changing the "scale_bmp" property.

This property has two states:

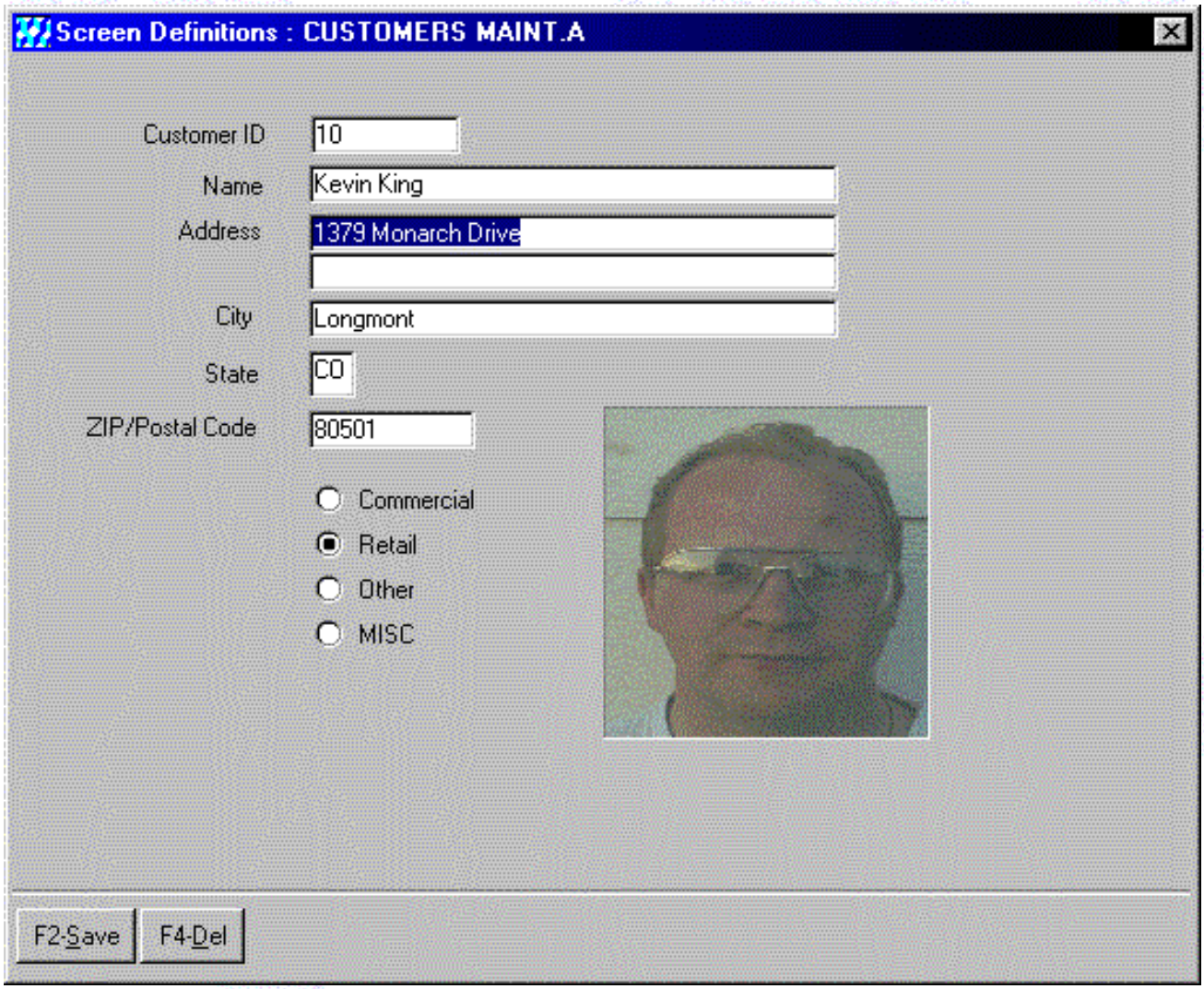
- True: The image will be scaled to fit the box
- False: The image will not be scaled, and will instead be tiled (repeated) to fill up the area of the box.

Change this property to the appropriate setting for the results you want. Note that there may be some degradation of the image as it is scaled from a smaller to larger size.

Once these parameters have been set, our screen appears as follows:



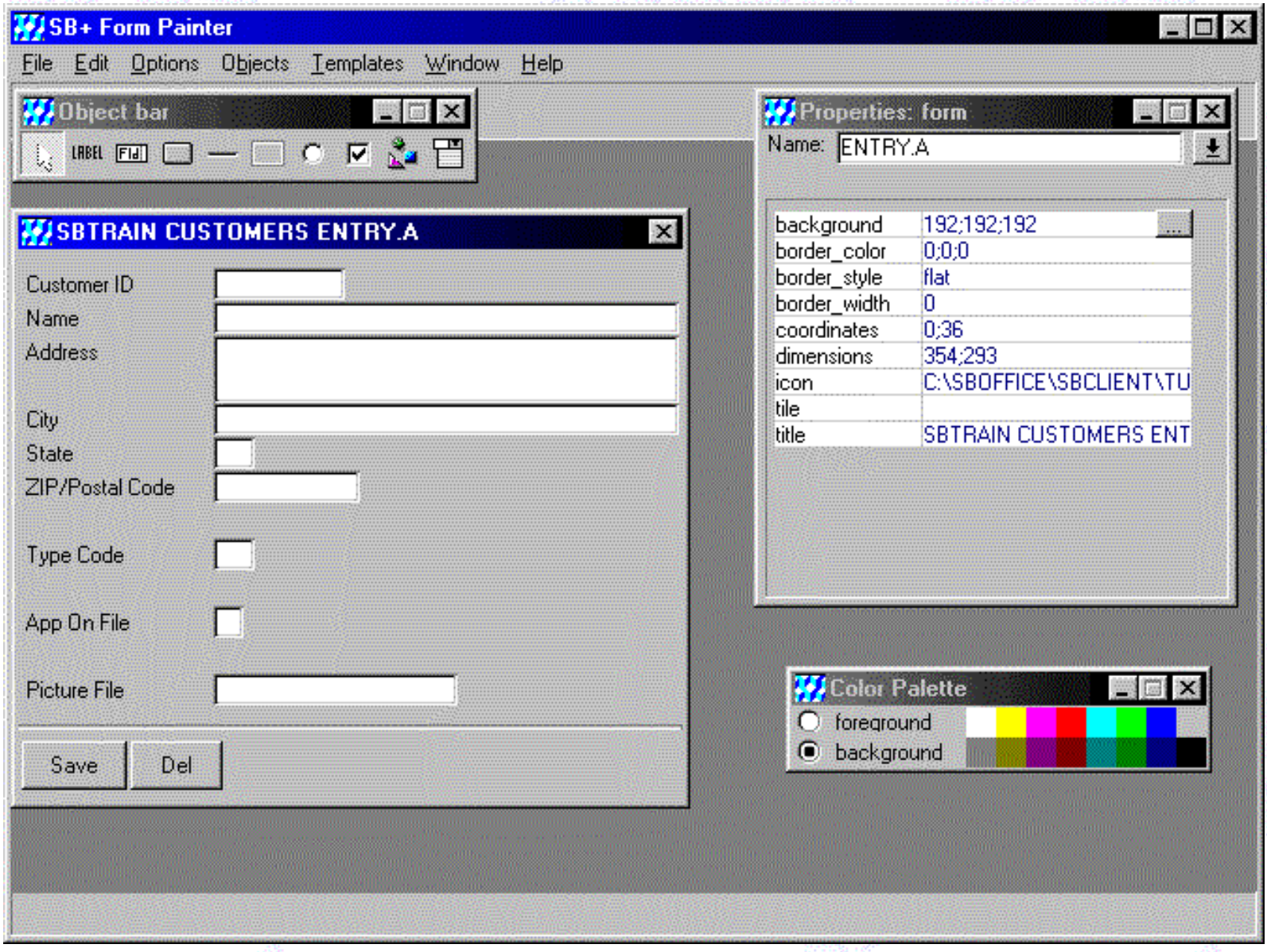
...and when a customer ID is entered:



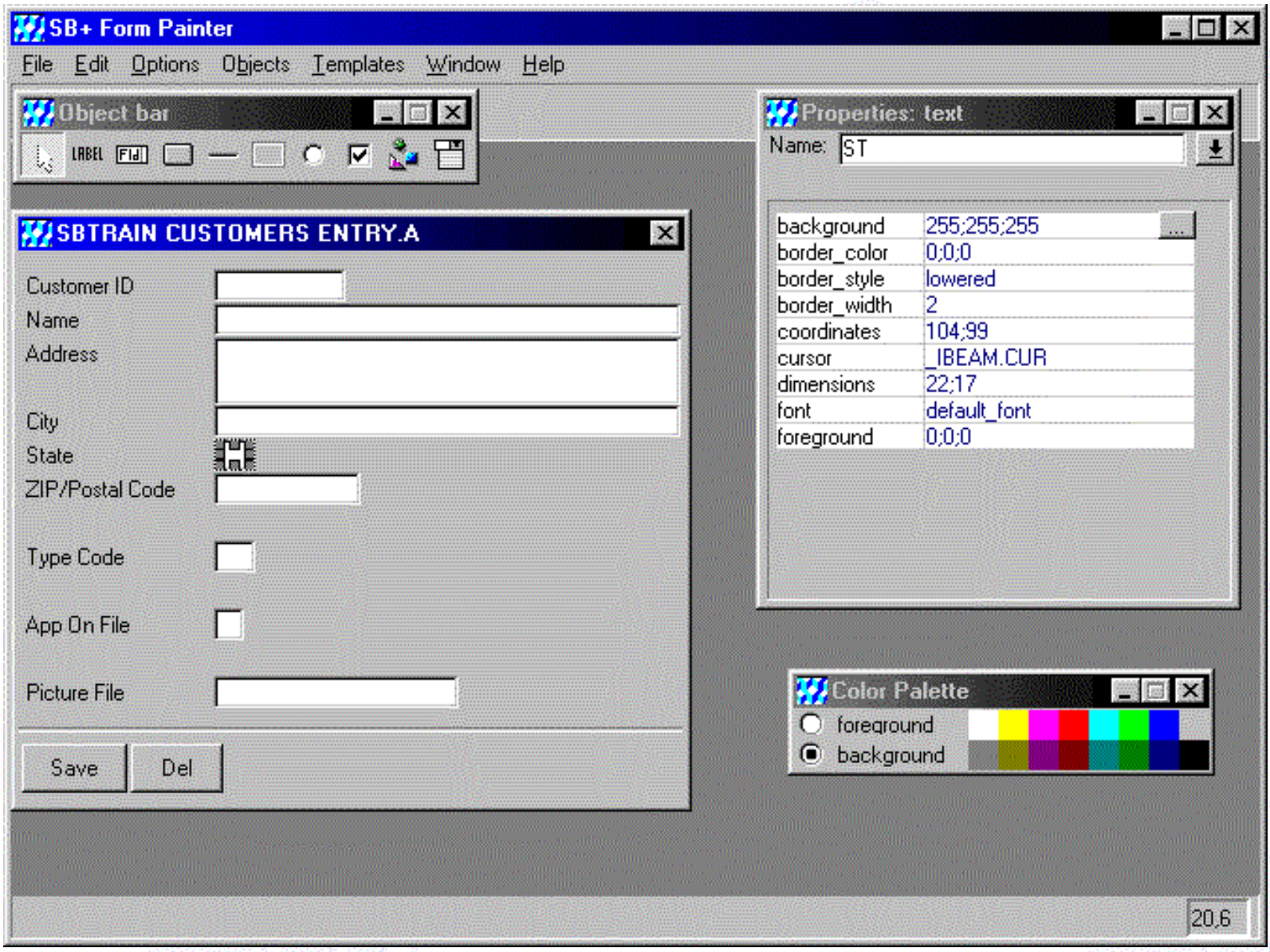
Note that the image quality is slightly degraded due to the capture mechanism used for loading the screens into this text. The actual image used for this example is much clearer than what is shown here. However, the skew (I am not THAT wide) is clearly a feature of the scaling being done by the "scale_bmp" property.

Converting a Text Field to a Combo Box

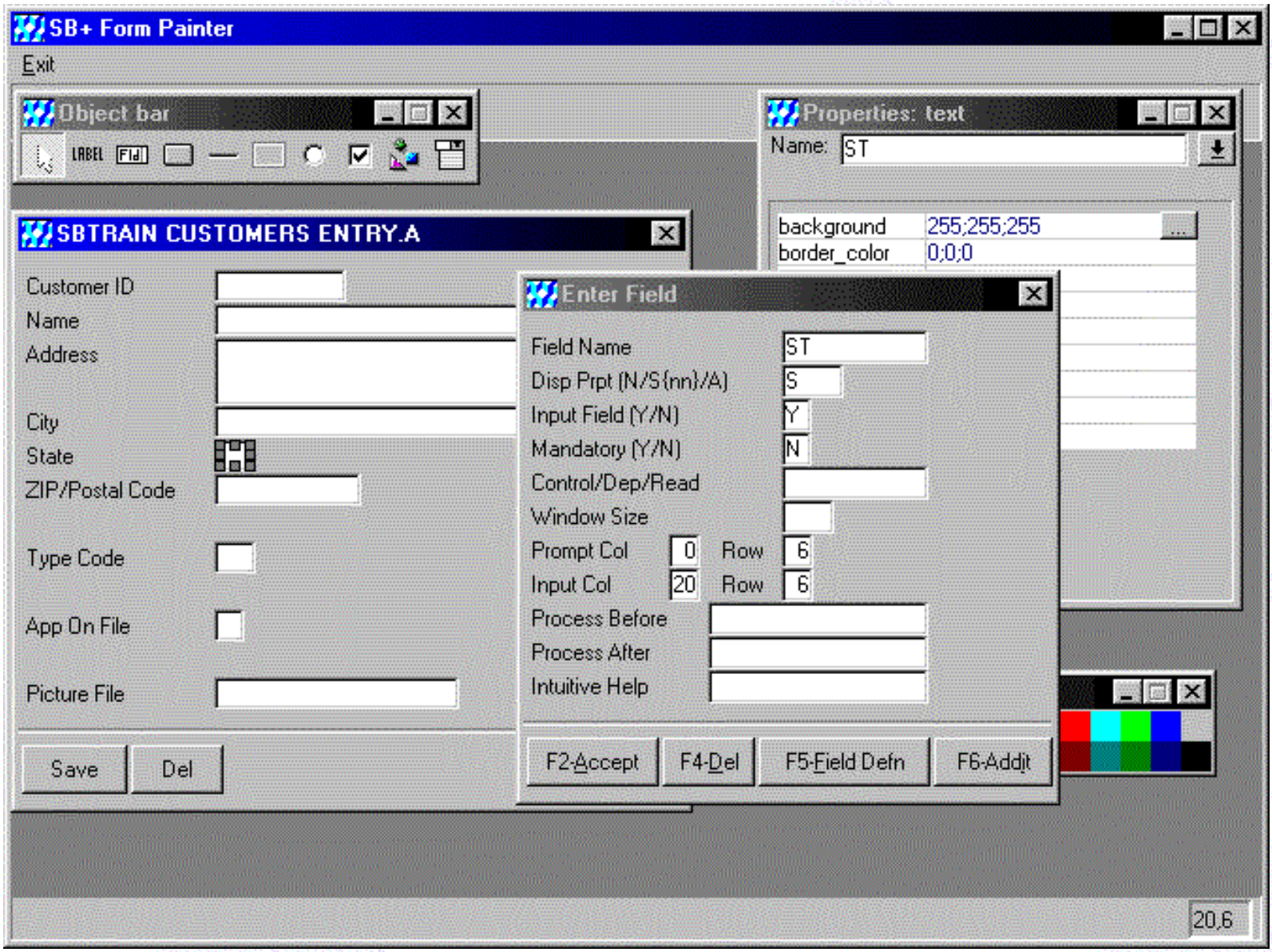
Currently, our customer screen looks like this:



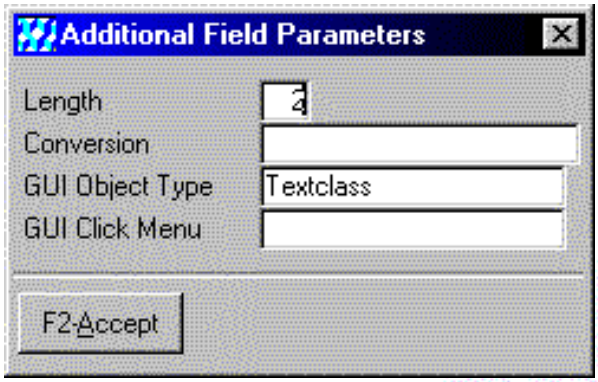
Let's convert the State field to use a combo box object. To do this, we first click on the input area for this field. When we do this, the Properties window changes, showing the properties that can be set for this particular input prompt:



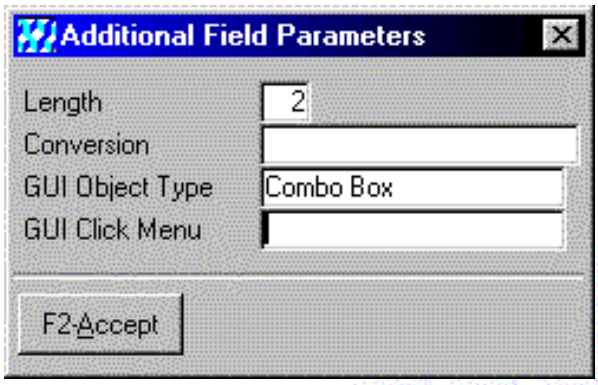
We're going to be changing this prompt from a text box to a combo box, so we won't worry about these properties for now. Instead, we'll simply press F5 to change the field. The following screen is displayed:



Look familiar? It should -- it's the F5-Field window from the Screen Definitions tool. On this window, we can either press F6 (or click the appropriate button) to display this screen:



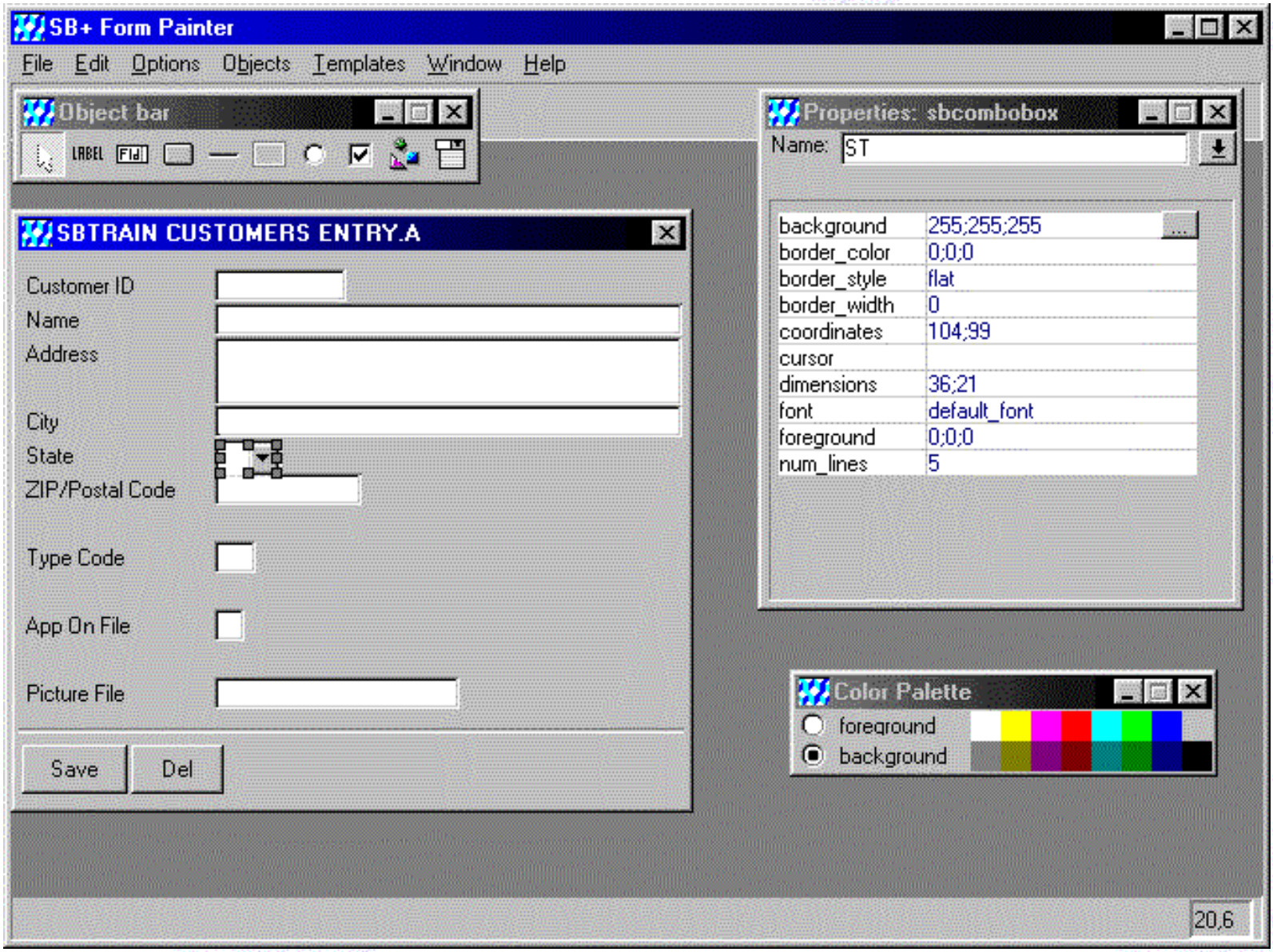
Note the GUI Object Type prompt shown here. At this prompt, we can change this prompt to be any of the valid GUI objects. Therefore, I can arrow down to this field and change the prompt to a combo box, as follows:



Note that SB+ allows this change only when the validation is proper. In the case of this field, the validation is...

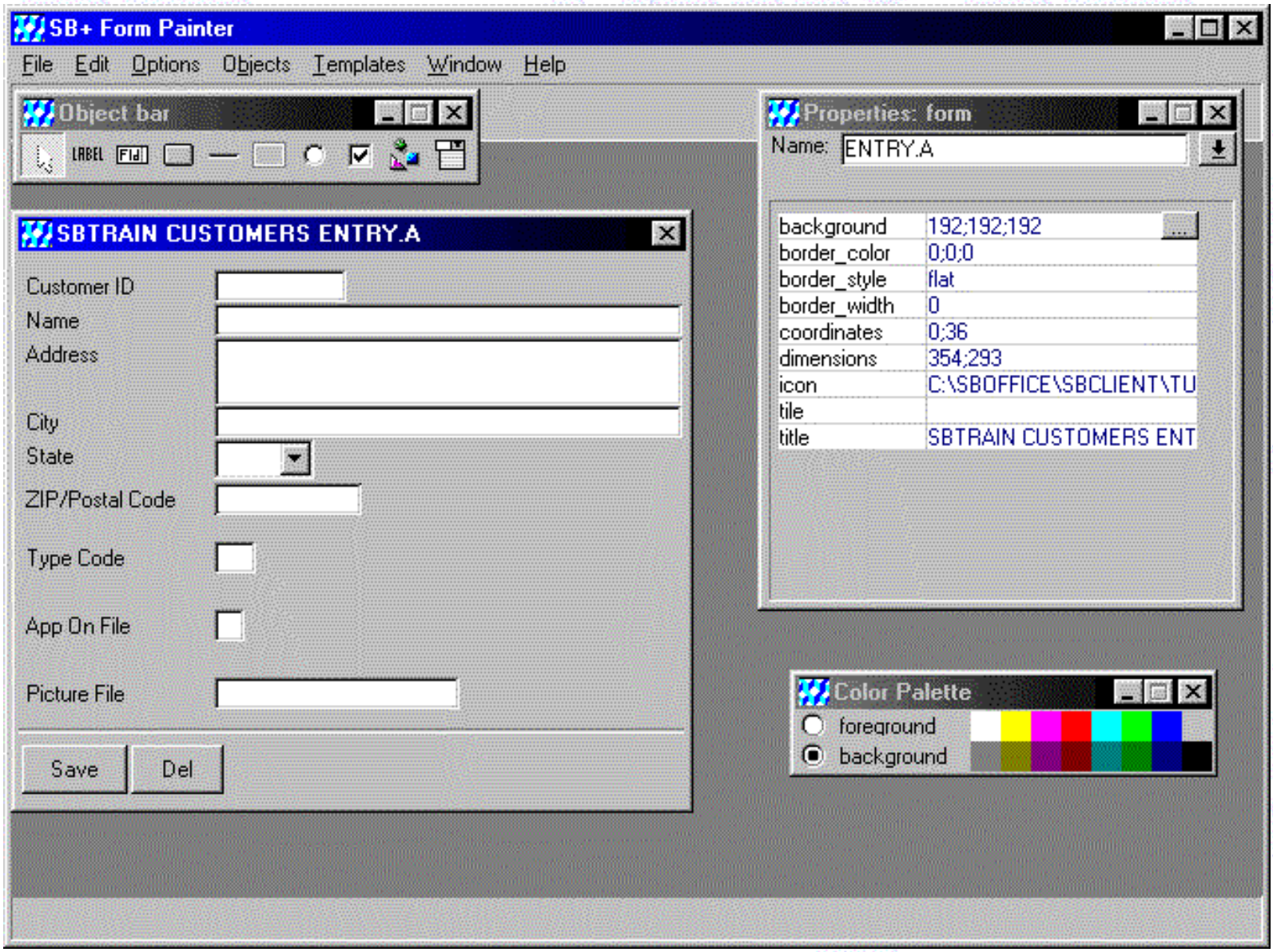
F:STATES,,Y{This is not a valid state}

...which is an allowable validation code for a combo box object. Once this change is made, I can then press F2 twice and see the net effect of my change:



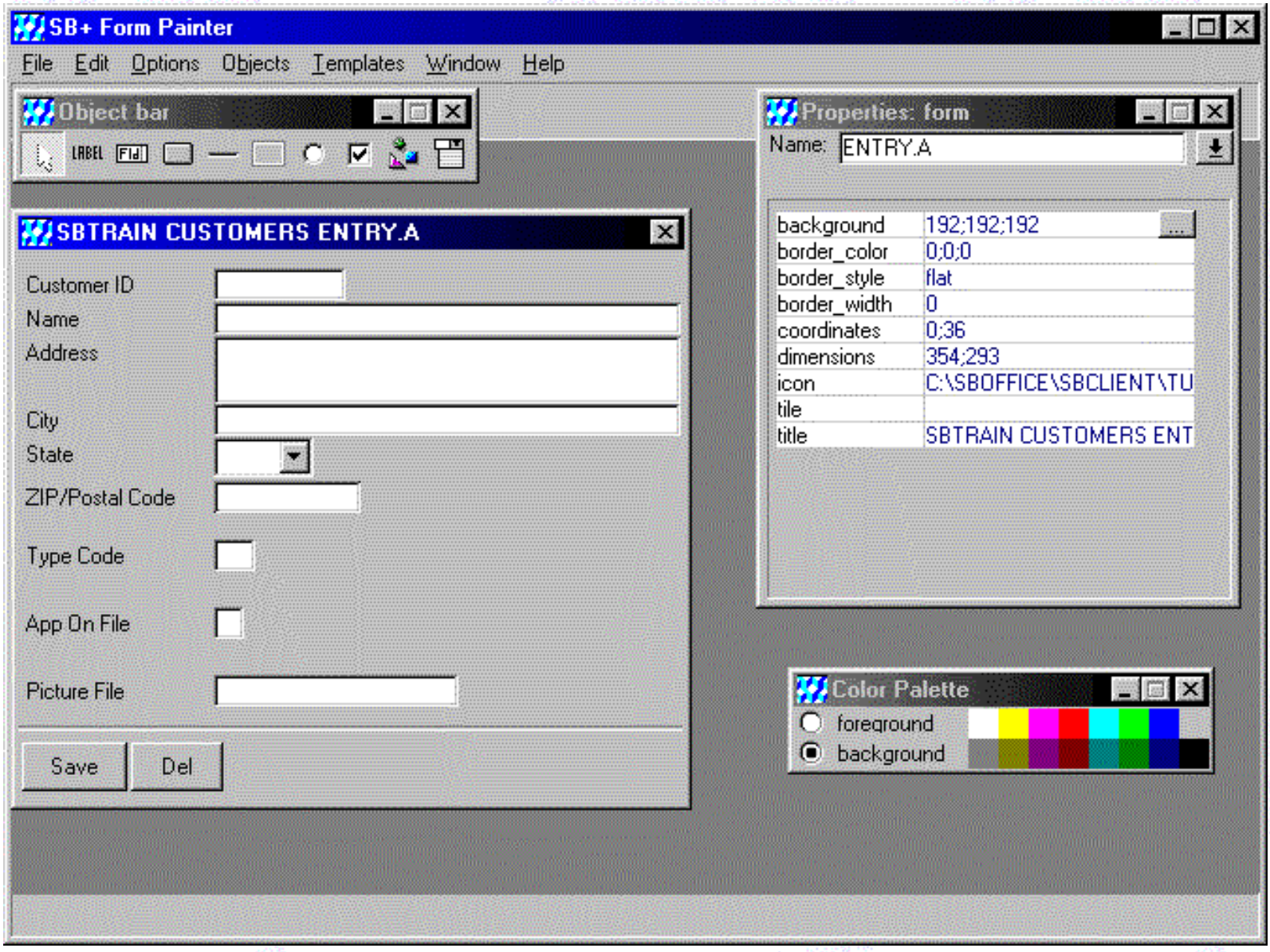
Note the arrow shown to the right of the text box. This tells me that the prompt has successfully been converted to a combo box.

Unfortunately, the spacing is a little off. No problem -- using the mouse I can move the ZIP/Postal Code field (and label) down, and also extend the width of my State prompt by dragging on the handle shown on the right edge. This looks better:

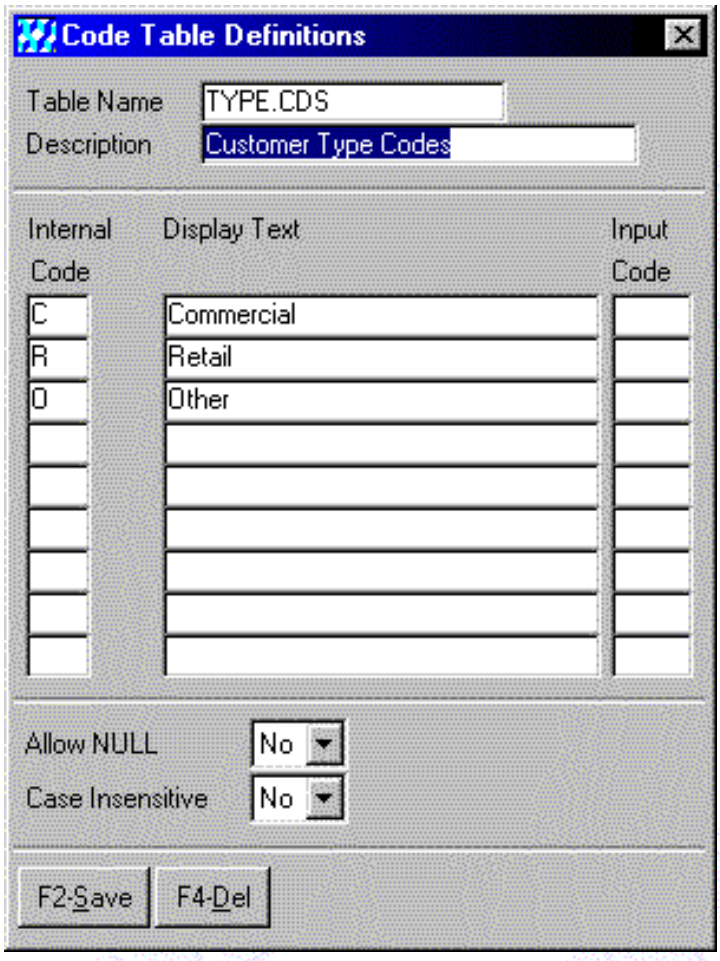


Converting a Text Field to Radio Buttons

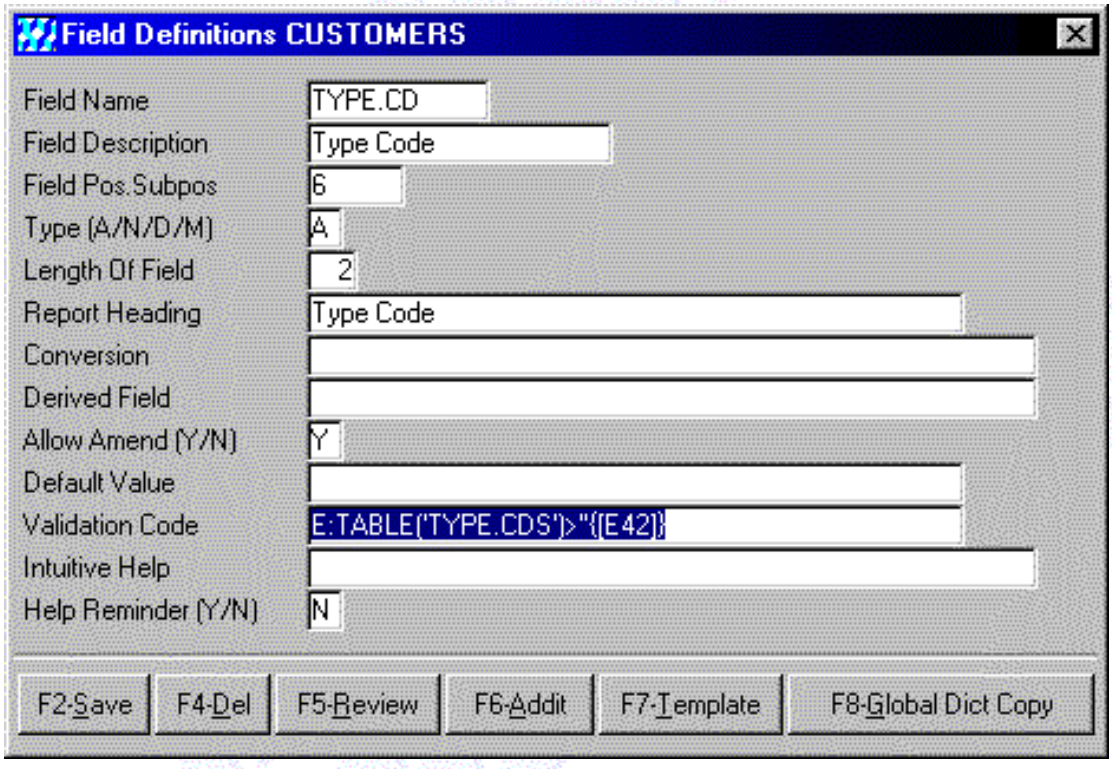
After changing the state field (from the **previous example**), our customer screen appears as follows:



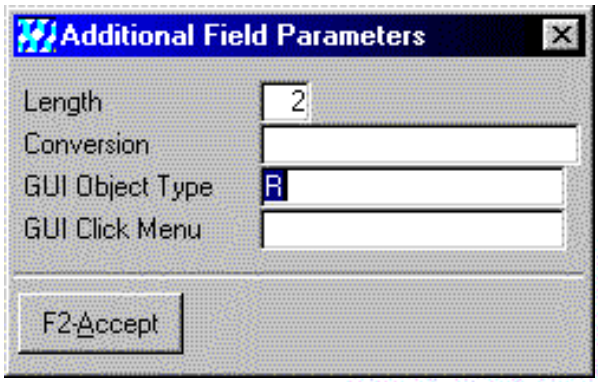
In this exercise, let's convert the type code field to a series of radio buttons. We have a table defined called TYPE.CDS which defines the different types of customers we do business with. This table contains the following entries:



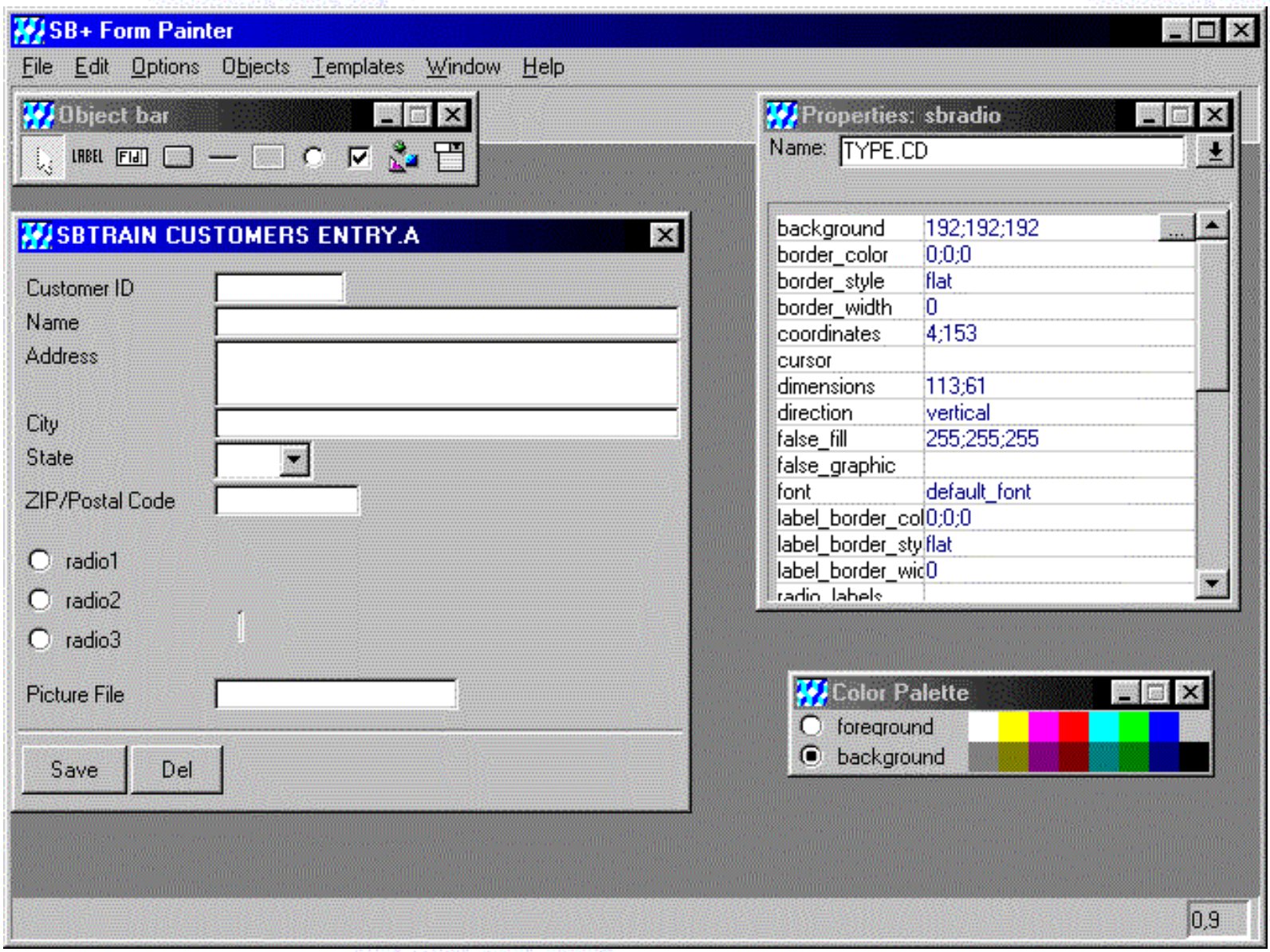
Our field definition is also set up to validate against this table, as follows:



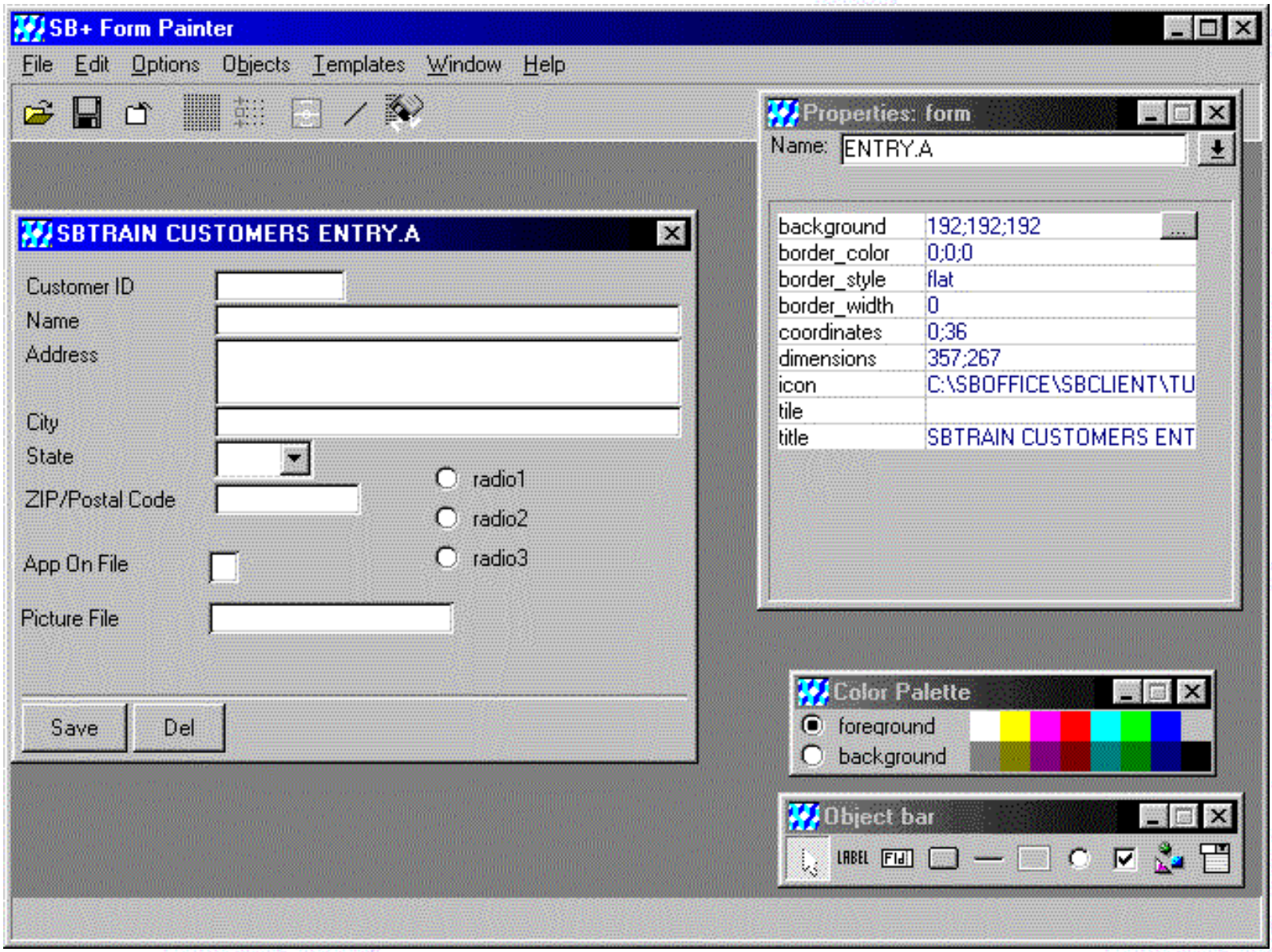
Therefore, the only thing to do is to change the GUI object type to be radio buttons instead of a textclass prompt. To do this, we simply click on the Type Code field (the input area -- not the label), press F5, then F6, and change the GUI object type to 'R':



Press F2 a couple of times, and the screen is redisplayed with our radio buttons:



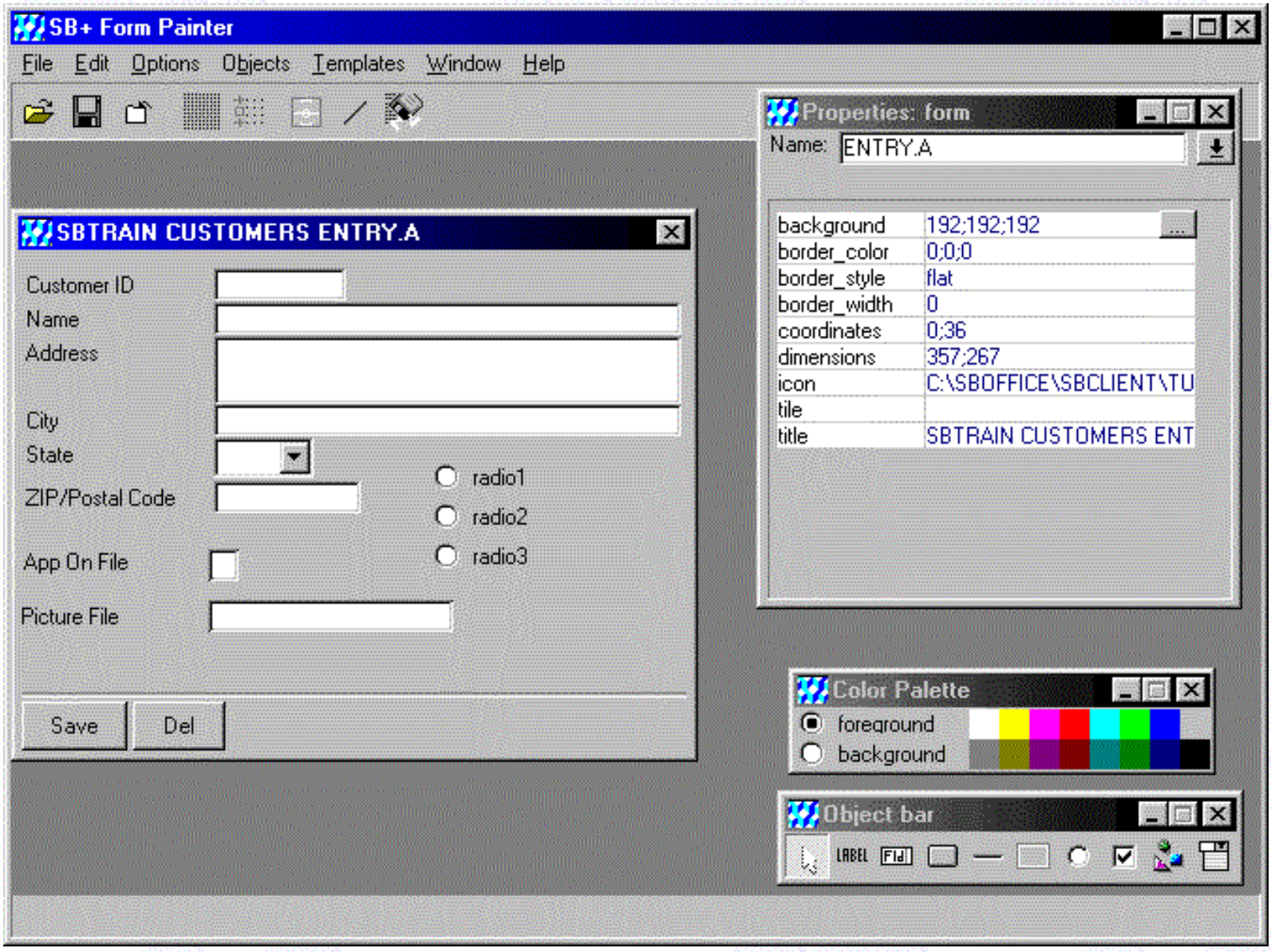
Again, the positioning of the prompt has overwritten some prompts. Using the mouse, however, we can rearrange the prompts as needed:



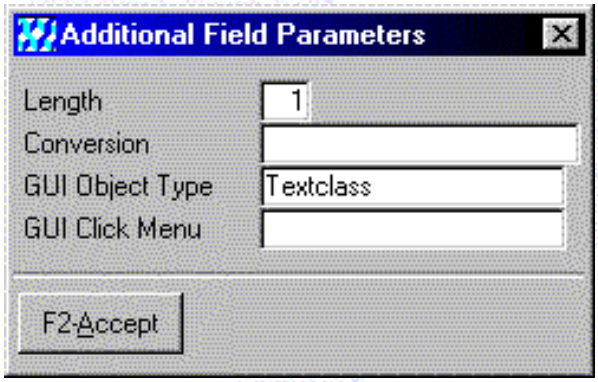
Radio buttons are unlike text fields in that when the radio buttons move, the labels move as well. Also, note the labels for the radio buttons -- radio1, radio2, radio3. These are merely placeholders. When the screen is executed, the descriptions from the table entries will be shown on the running screen instead.

Converting a Text Field to a Toggle

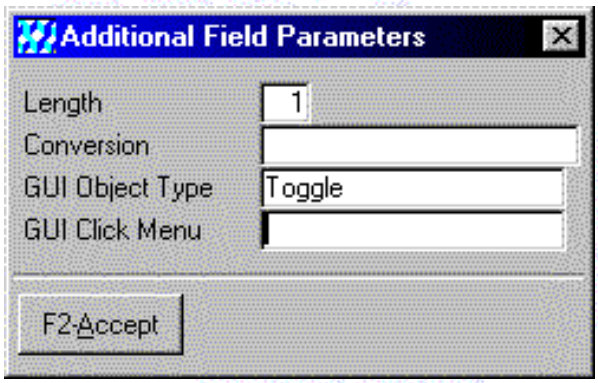
Now that we've **converted the state field to a combo box** and the **customer type code to radio buttons**, let's convert the App On File prompt to be a toggle. First, let's take a look at the screen at is currently exists:



As in the previous examples, we must first click on the App On File input area. Next, we can press F5 to bring up the field information, then F6 to bring up the additional information. The following screen will then be displayed:



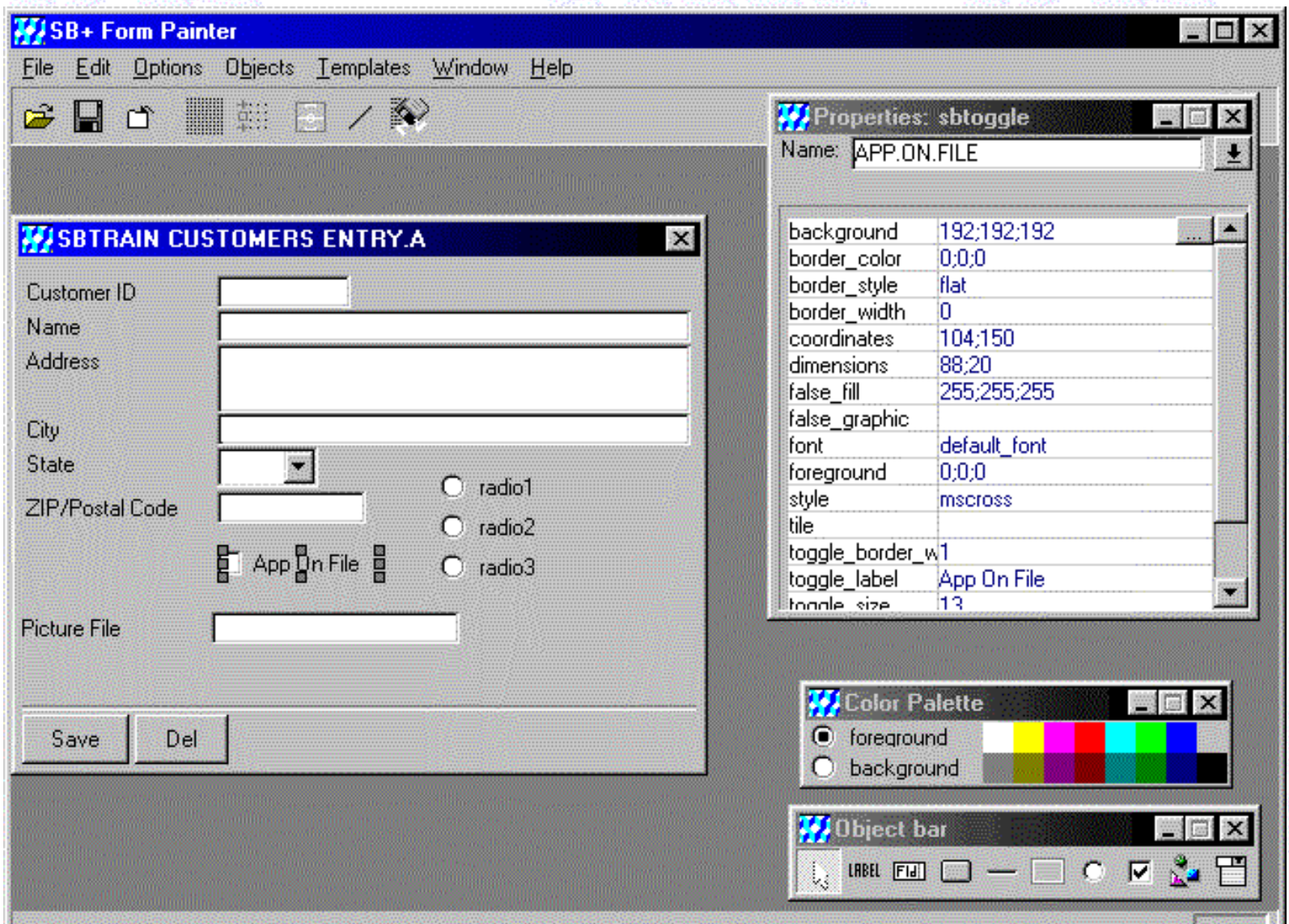
At this point, we can press <cr> a couple of times to move the cursor to the GUI Object Type prompt. At this prompt, enter "T" to tell SB+ that this field is a toggle, as follows:



Of course, to change this to a toggle, we'll have to have previously defined the field with the **proper validation**. In this case, the validation is as follows:

E:TABLE('YES.NO')>"{Entry must be either 'Y' or 'N'}

...so we should encounter no problems in changing this field from a text object to a combo box object. Once this change has been made, and F2 has been pressed a couple of times, we can see the net effect of this change:

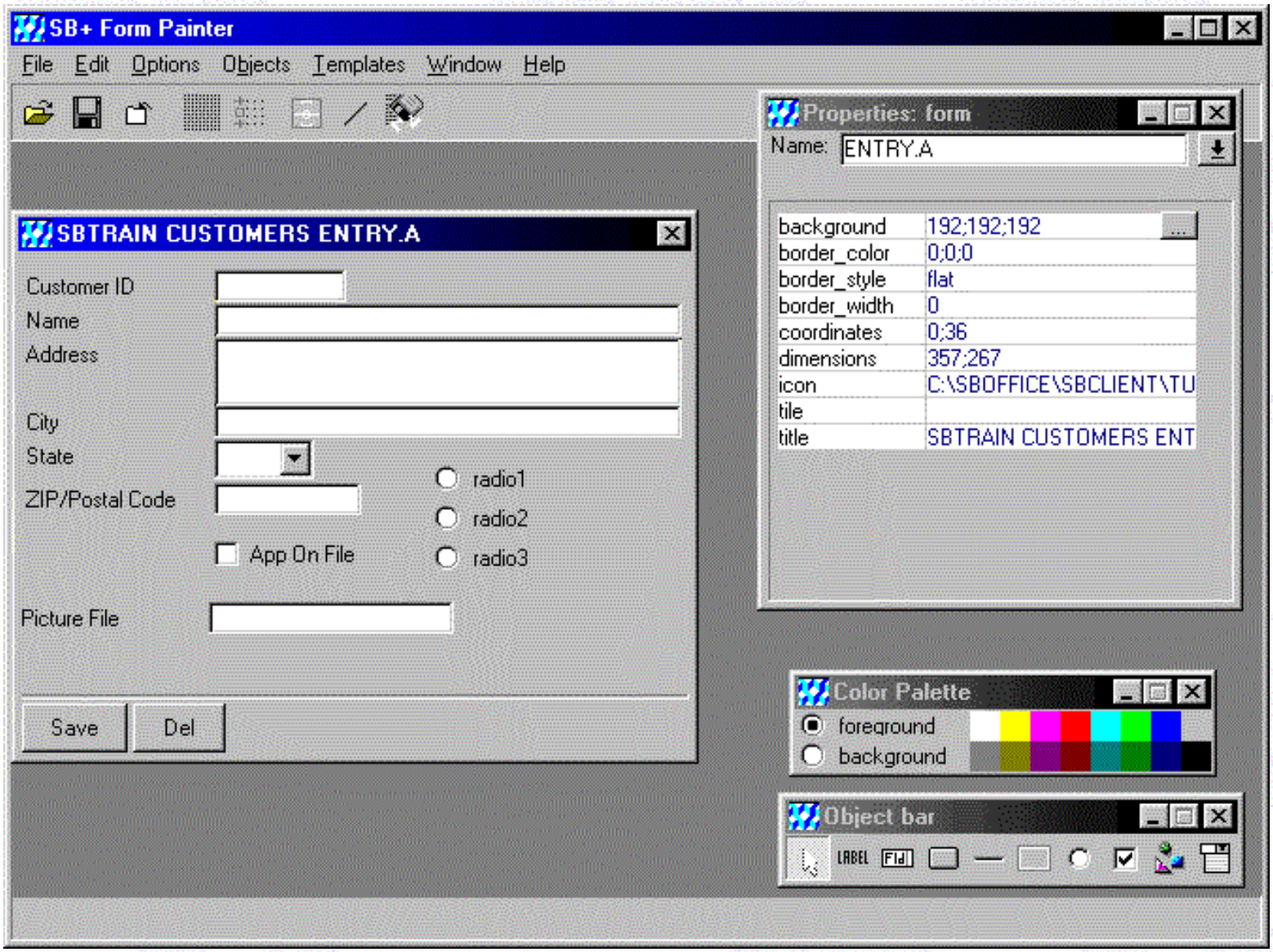




Once the field has been placed, we can then use the mouse to move it anywhere on the screen. Unlike a typical text field, when the prompt moves, the label moves with it.

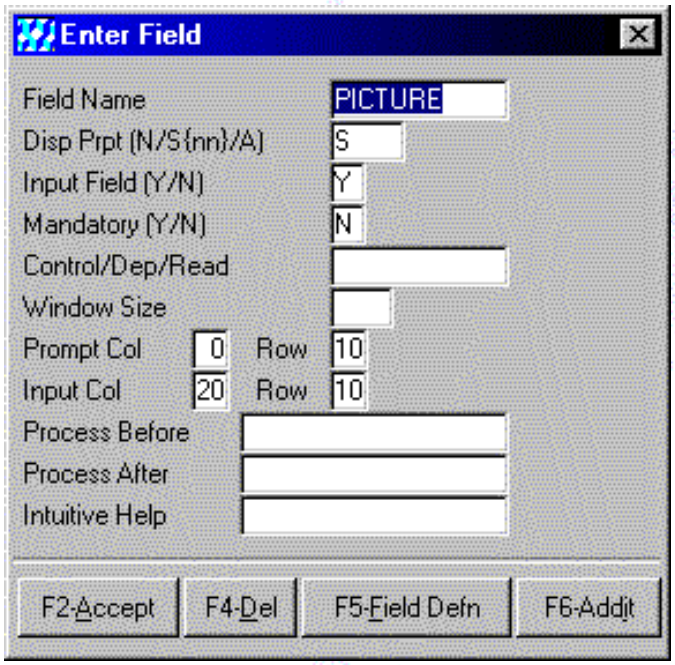
Converting a Text Field to an Image

Continuing our customer entry screen example, we've **converted a text prompt to a combo box**, **converted another prompt to a set of radio buttons**, and have also **converted another text prompt to a toggle**. Our screen currently looks like this:

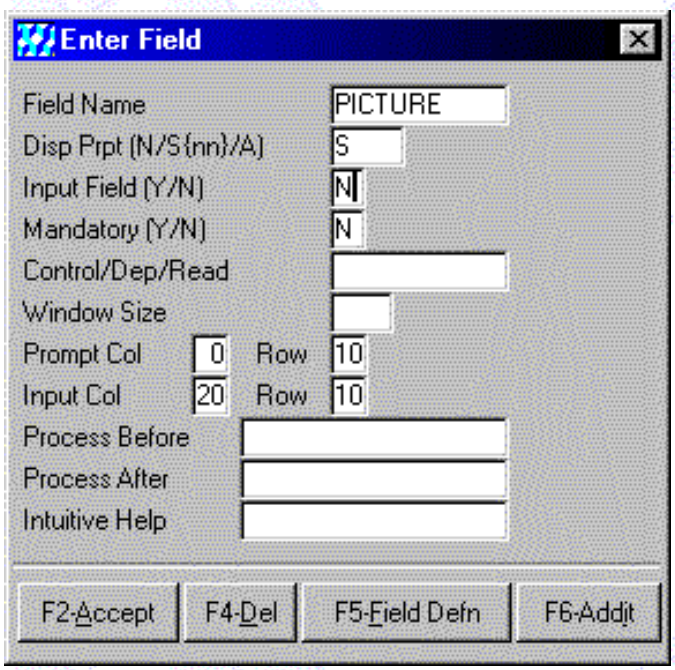


The final task to be completed is to convert our Picture File prompt from a textual entry to an actual image. Using this prompt, we'll show a picture of the customer when the customer's record is recalled.

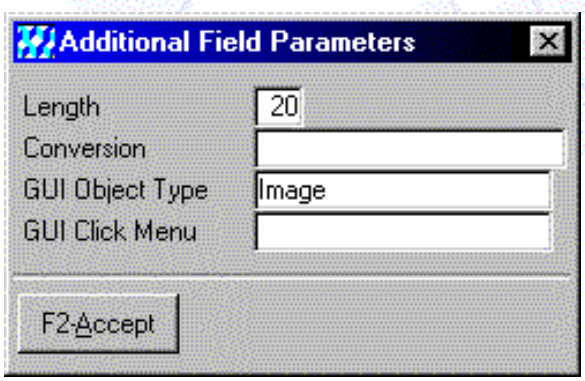
Image objects are display-only -- you cannot enter values into an image object. Therefore, the first thing we need to change is the Input Field prompt on the F5-Field window. First, though, we need to click on the input area associated with the Picture File prompt, and then we can press F5. Once we've done this, we can see the field parameters as follows:



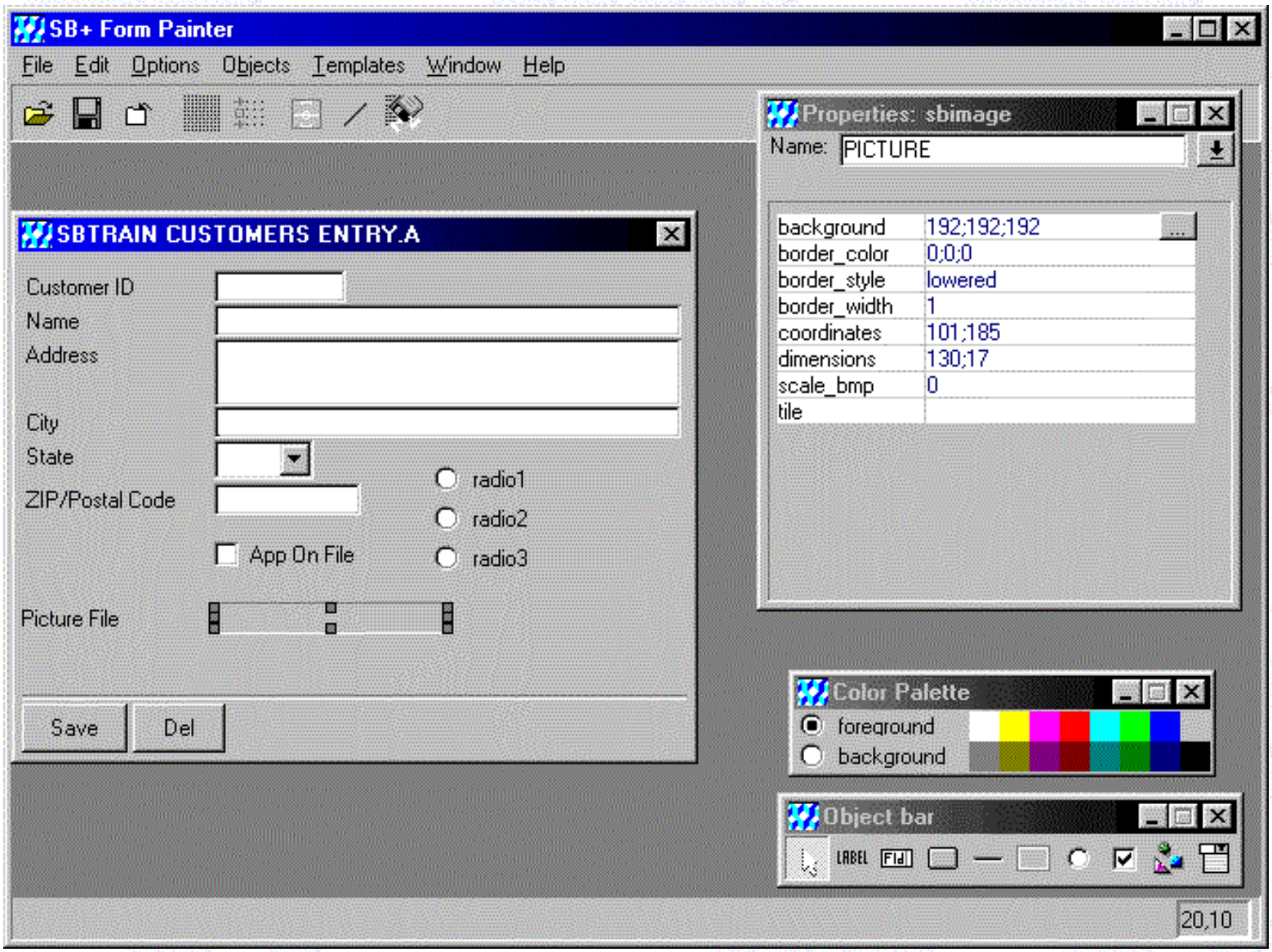
Notice how the field is currently setup as an input field. We need to change the Input Field (Y/N) prompt to reflect that this is a display-only prompt, as follows:



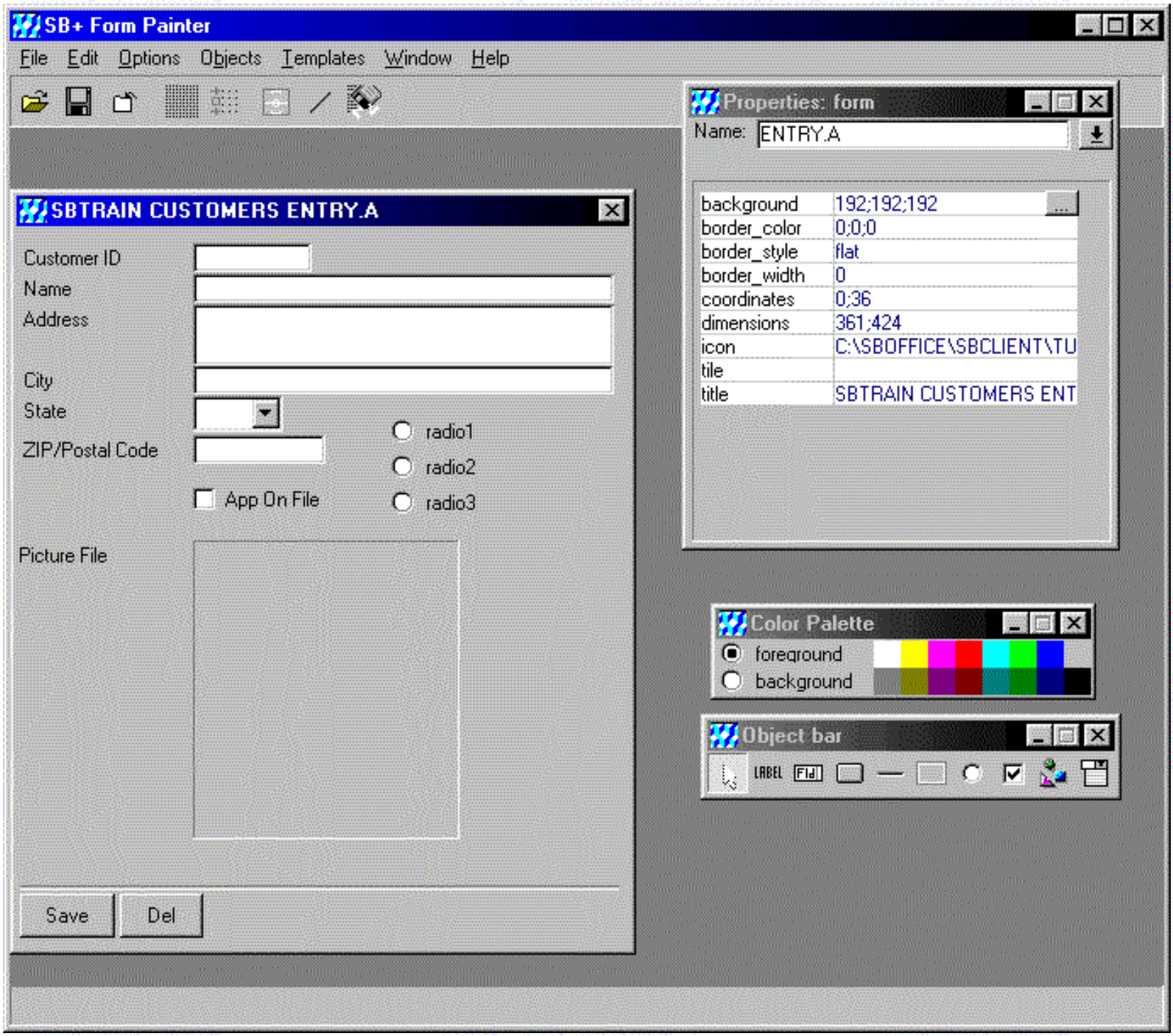
Once this is complete, we can either press F6 (or click on the comparable button) to display the additional parameters for this prompt, as follows:



In this case, I've selected the Image (I) GUI object type. This tells SB+ to display an image corresponding to the value in the Picture File prompt as each record is displayed. Once I've pressed F2 a couple of times, I can see the net effect of this change:

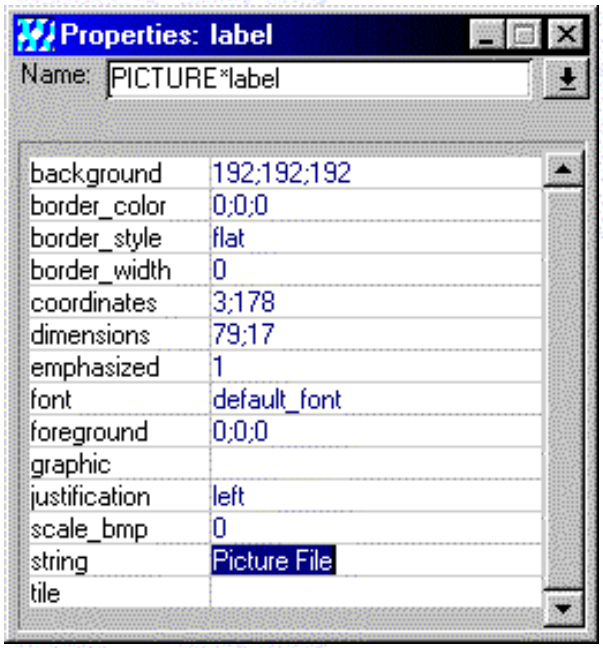


As we've seen in the other examples, the prompt does not automatically get converted to the right width and depth simply by changing its type. Instead, we need to grab one of the handles (such as the lower right handle) and resize the box accordingly. On this screen in particular, there isn't enough room to add a reasonably sized image, so we'll need to make the window itself larger by clicking on the lower right corner of the box and dragging the box to a more appropriate size. Once all this resizing and moving is complete, our screen looks like this:

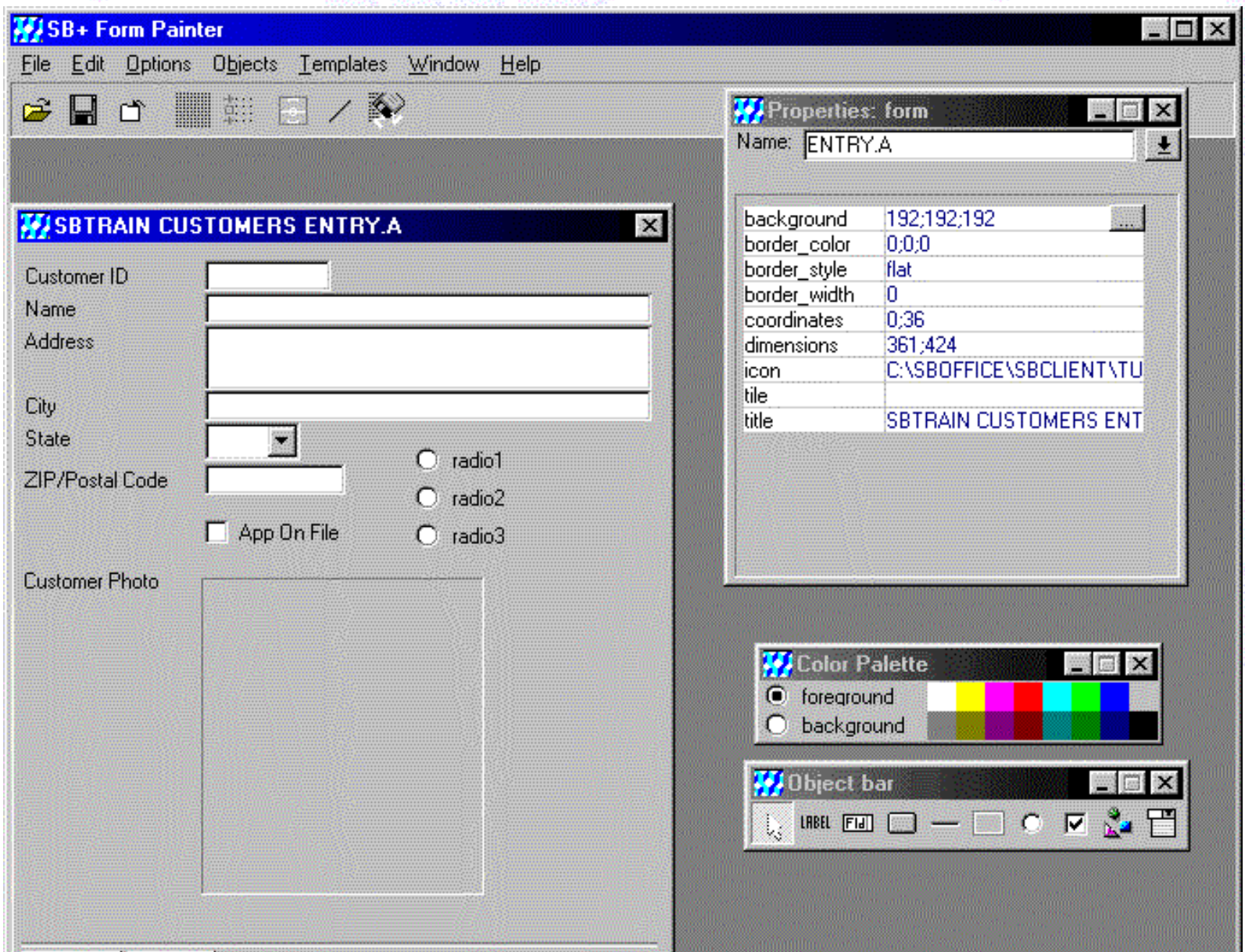


Like a text prompt, the label and input area move separately with an image. Therefore, we can move the input area and/or label independent of one another. In this example, however, our label really doesn't apply anymore; we're not really displaying a picture *file*, we're actually displaying the picture *itself*. Therefore, we should probably either remove or change the label.

Removing the label is simple; we can simply click on the label itself and press the Del key to remove it. But in this case, we won't remove the label entirely -- instead, let's change it to something more appropriate. To do this, click on the label, then look through the Properties list for the "string" property. Once you find this property, click on either the property label or the property value. The property value will be shown in blue to let you know you can change the value:

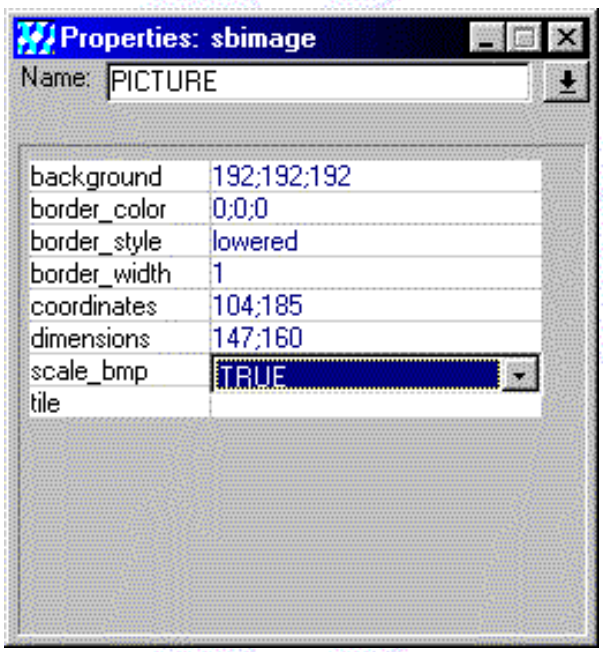


Let's change this property so that the prompt is labeled "Customer Photo". Simply type the words "Customer Photo" into the Properties window and press <cr> when complete. Once <cr> has been pressed, the edit is complete and we can see the results immediately:





One additional property that should be set on the image object is called "scale_bmp". This is a true/false value, and determines whether the image is loaded at its actual size (false), or if the image should be scaled to fit the box (true). Each result has different implications. First, if the image is allowed to be loaded at its actual size, you may not be able to see the image as a whole -- parts may be truncated. On the other hand, if the image is scaled, the perspective of the image may be distorted, making the photograph look like something from a house of mirrors. Generally speaking, however, assuming that most of the photos share consistent dimensions, it's a good idea to scale the graphic to fit the box, as follows:



When running, the screen looks something like this:



Photo quality not withstanding (due to my camera, not to SB+), this demonstration shows how easy it is to add photos and other graphics to your applications without investing a great deal of time or effort!

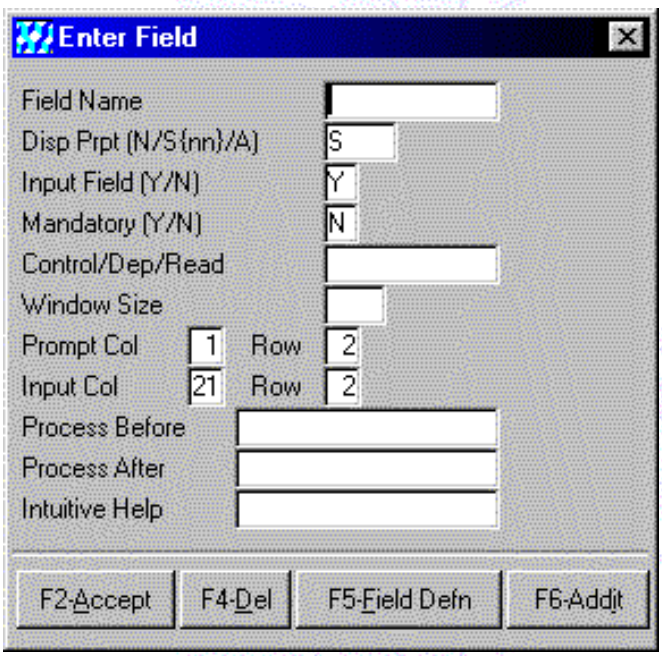
Adding a Textclass Object to a GUI Form

Once the basic GUI form has been created, we can then add objects to the form. One of the most common objects we can use is the Textclass object. This object allows the user to enter any textual value into the prompt.

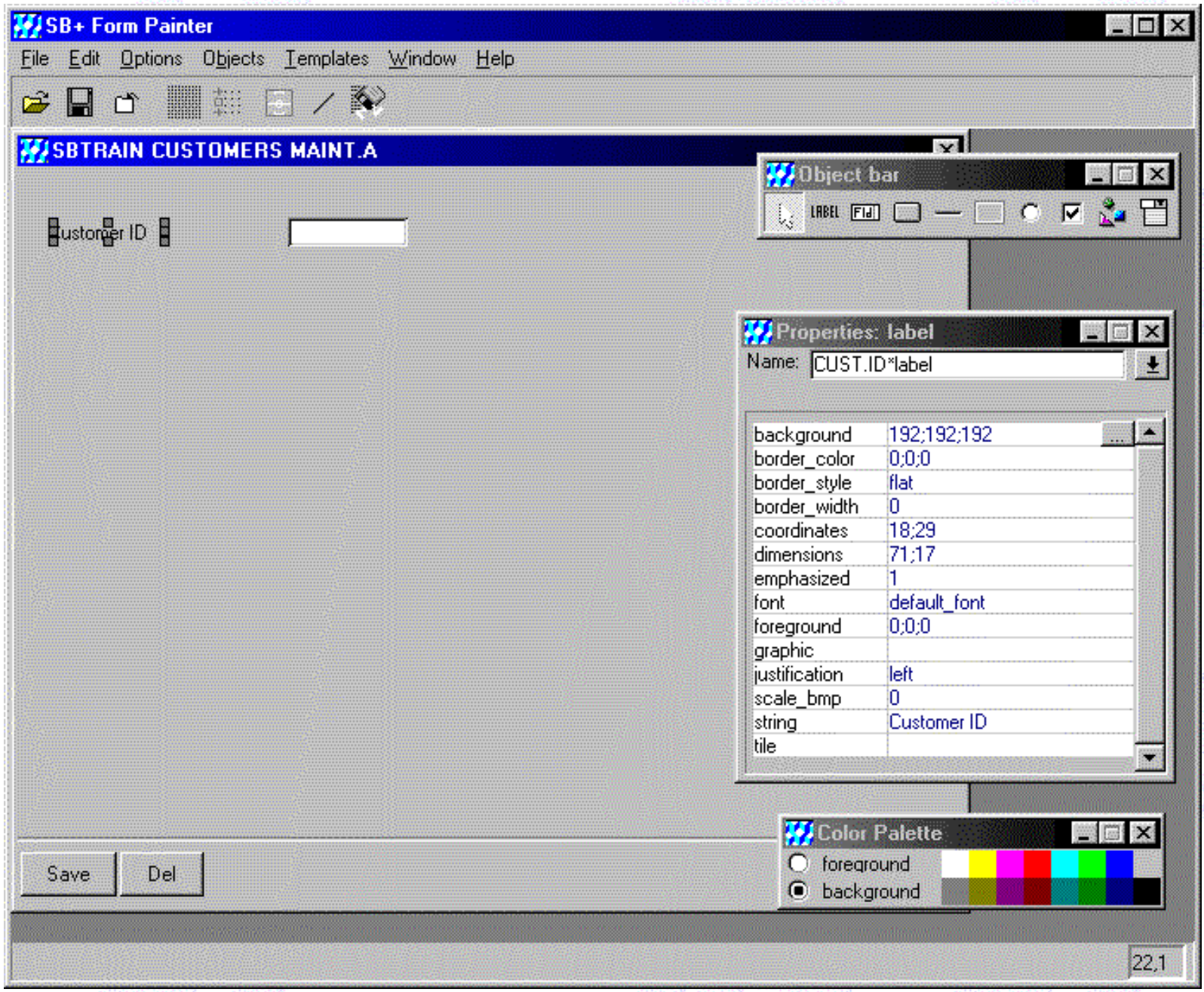
To add a textclass object to a form, click on the 

button (in the Object bar). This is effectively the same as pressing F5 in the Screen Definitions tool, except that we haven't located a position for the prompt yet.

When the button is pressed, the mouse pointer becomes a cross. This is your indication to tell SB+ the position of the field on the screen. Move the pointer to where the label is to appear and click the left mouse button. This opens the F5-Field window (common from the Screen Definitions tool) as follows:



At the Field Name prompt, either enter the name of the field to be placed at this location, or press F3 for a list of available fields. Once the field name has been entered, and F2 has been pressed, the GUI form is updated to reflect the new addition, as follows:



At this point, the prompt is on the form so we can move on to the next prompt. Or, if you prefer, you can move the prompts from place to place simply by clicking and dragging either the input area or the prompt label. (See "**Moving Prompts on the GUI Form**" for more information about moving and resizing objects.)

It's important to note that once a field has been added to the form, two completely separate objects have been added. In the example above, we have a label object (that is, simple text) with the words "Customer ID", and we have a second object (the textclass input area) which is where our customer ID will be entered. Once added to the screen, each of these objects are independent of one another.

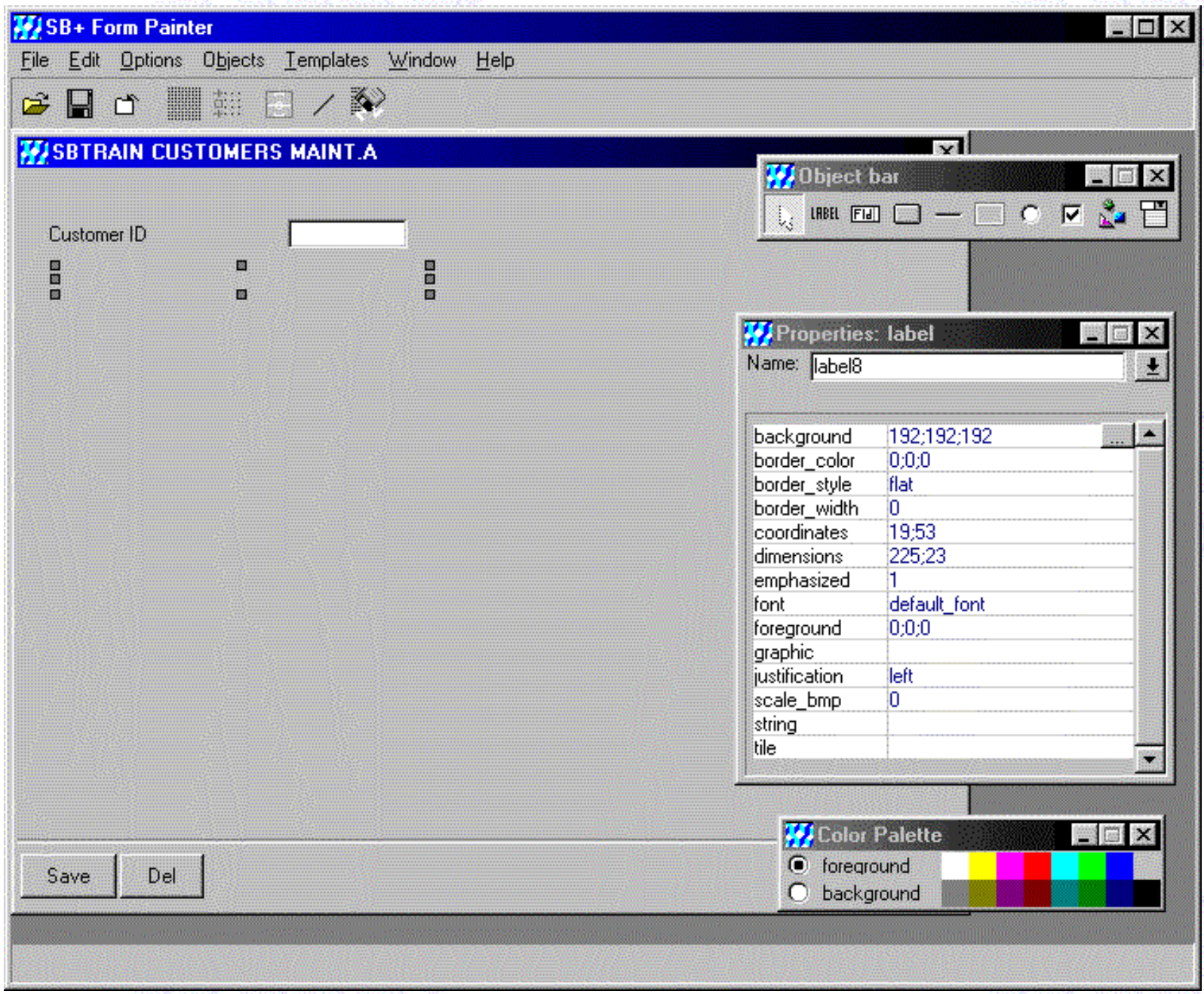
Various changes can be made to each of the objects once they've been added to the form through entries in the Properties window. See "**GUI Object Properties**" for more information about changing properties.

Adding a Label to a GUI Form

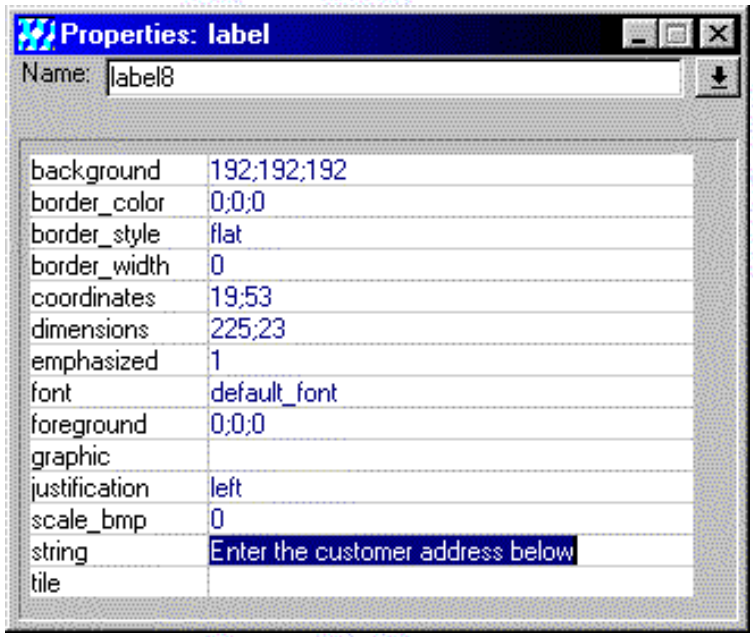
In GUI, a label is a textual word or phrase that appears on the screen either with or without a related textclass object. Labels are added automatically when **textclass fields are added** to a form, and they can also be added independently of any other object.

To add a label to a GUI form, click on the 

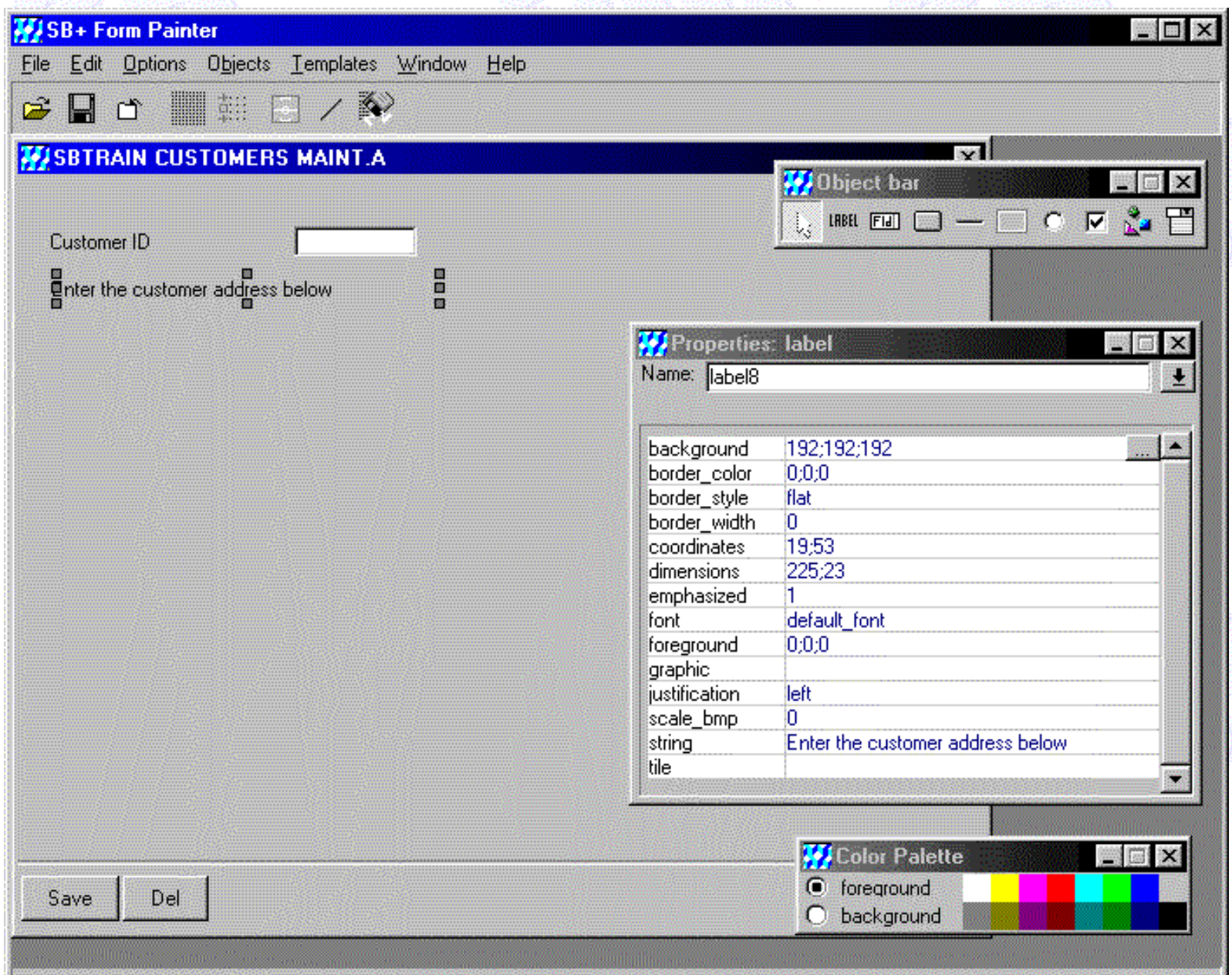
button in the Object bar. The mouse pointer will be changed into a cross, which is your indication to position the label where you want it to appear. To position the label, move the mouse pointer to a position on the form and click and drag the mouse until the graphic "lasso" is approximately the size you want for the label. When the mouse button is released, the new label object will be added to the screen, as follows:



Note that the label itself has no value when created. This value must be entered in the Properties window under the "string" property. For example, to add the message "Enter the customer address below" to this prompt, the following could be entered for the "string" property:



Back on the GUI form, this change appears as follows:



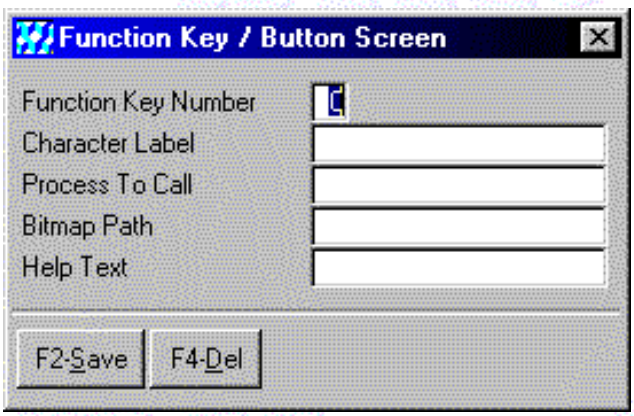


Various additional changes can be made to the appearance of the label using the properties shown in the Properties window. See "**GUI Object Properties**" for more information about changing properties.

Adding a Command Button to a GUI Form

A GUI command button is effectively the same as a function key in character mode. In fact, every function key defined on the **F7-Fnkeys** window is represented by a button on the GUI screen. However, GUI command buttons go beyond the normal function keys. In fact, in addition to the F2-F10 function key buttons, you can also create any number of GUI-only command buttons, each of which invokes a separate process.

To create a GUI-only command button, first click on the  button on the Object bar. The mouse pointer will then be changed into a cross, which is your indication to position the pointer where you want the object to appear. To place the object, move the mouse pointer to a position on the form and then click and drag the mouse pointer to define the size of button you prefer. The object will then be created at that location. Next, the following window will appear:



At the Function Key number, enter 0 to create a GUI only button.

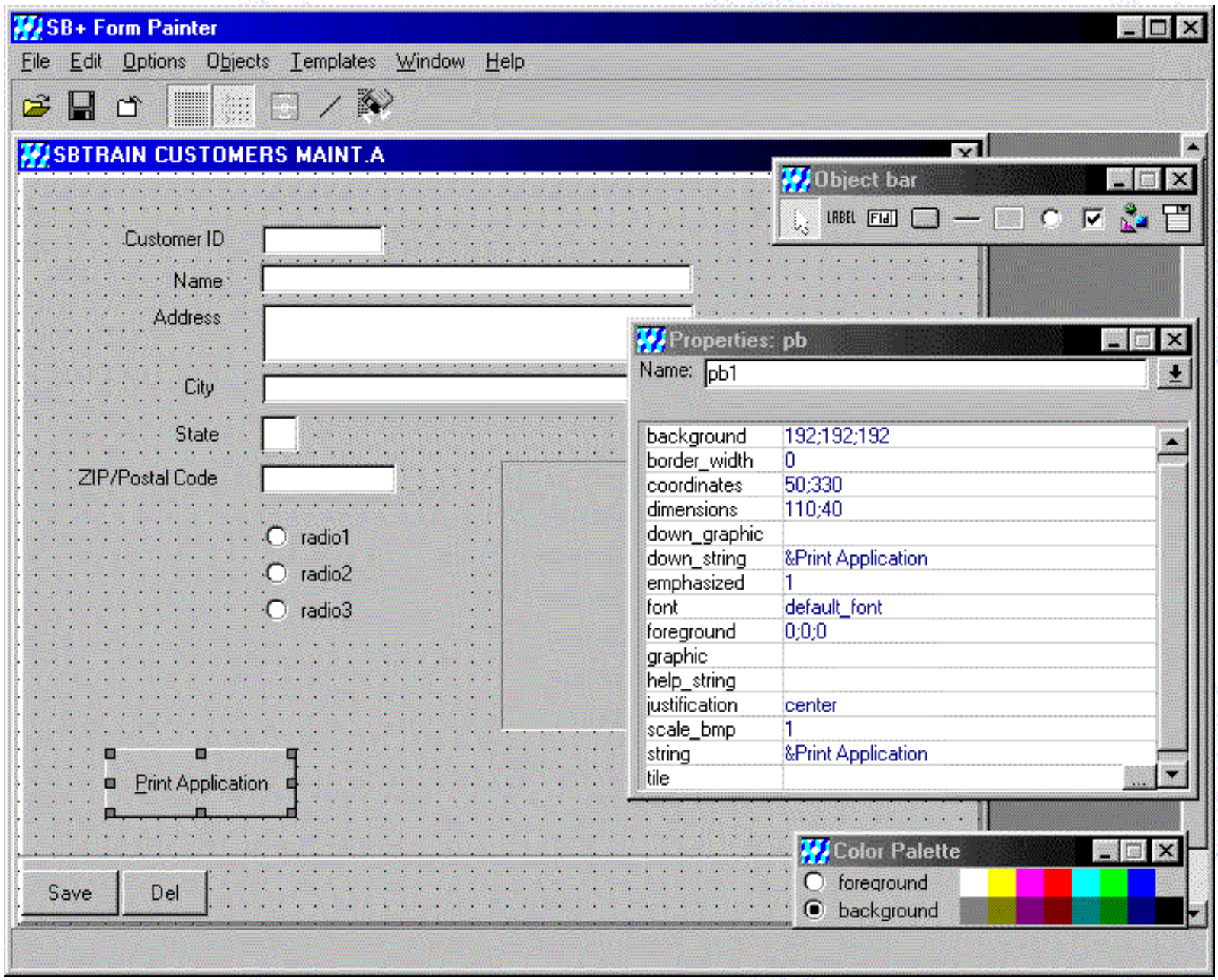
At the Character Label prompt, enter the label you want to use on this button, such as "Print Application". If you want one of the letters in the description to be underlined (indicating an Alt-key), preface that letter with an ampersand. For example, if the button should say Print Application, the character label for the button will be "&Print Application".

The Process to Call nominates the process to be called when this button is pressed. This process can invoke a **subscreen**, **report**, or any other type of process you need. Simply list the name of the process to be invoked in this slot.

At the Bitmap Path prompt, enter the name of a image file which is to be used as the background for the button. If you leave this blank, a typical grey Windows button will be created, much like what you see for the function keys. (But function keys can have images too!)

Finally, at the Help Text prompt, enter a string to be displayed when the user "floats" the mouse pointer over the button.

Once all of this information has been entered, either press F2 or click on the F2-Save button, and the command button will be added to the GUI form, as follows:


SB+ Form Painter
File Edit Options Objects Templates Window Help
SBTRAIN CUSTOMERS MAINT.A
Customer ID
Name
Address
City
State
ZIP/Postal Code
radio1
radio2
radio3
Print Application
Save
Del
Object bar
LABEL
Fld
Properties: pb
Name: pb1
background 192;192;192
border_width 0
coordinates 50;330
dimensions 110;40
down_graphic
down_string &Print Application
emphasized 1
font default_font
foreground 0;0;0
graphic
help_string
justification center
scale_bmp 1
string &Print Application
tile
Color Palette
foreground
background

Adding Graphics to a GUI Form

In the GUI form painter, you can draw graphic lines (called *separator lines*) and boxes (called *rectangles*) to distinctly separate areas of the form.

Drawing Separator Lines

To add a separator line to the form, first click on the  button on the Object bar. The mouse pointer will then be changed into a cross, which is your indication to position the pointer where you want the object to appear. To place the object, move the mouse pointer to a position on the form, click the left mouse button, and drag the mouse pointer to the desired end of the line. The object will then be created at that location.

While there is only one graphic line tool, it can be used for a variety of different effects, depending on the properties of the line. Specifically, three properties control how the line will appear:

- border_style: This property controls whether the line appears to be raised from the form, lowered into the form, or flat on the form.
- dimensions: This property controls the width and depth of the line. The wider or deeper the dimension (depending on the "direction" property), the more significant the emphasis will be on the line.
- direction: The direction property defines whether the line is a horizontal or vertical line. Different shading rules apply depending on the direction of the line.

Drawing Rectangles

A rectangle is a very interesting object: It can be used both as a graphic box and also as image container. To draw a rectangle, first click on the  button on the Object bar. The mouse pointer will then be changed into a cross, which is your indication to position the pointer where you want the object to appear. To place the object, move the mouse pointer to a position on the form, click the left mouse button, then drag the mouse pointer to the desired location for the opposite corner of the rectangle. The object will then be created at that location.

Once a rectangle has been placed on a form, other objects can be placed inside of the rectangle, thereby providing visual separation between the enclosed prompt and the remainder of the GUI form. To do this, first create the rectangle, then use the mouse to either move the rectangle on top of another object, or move the object to be enclosed so that it is inside the rectangle. (You may have to change the layering to properly see the object inside of the rectangle -- see "Layering Objects" later on this page.)

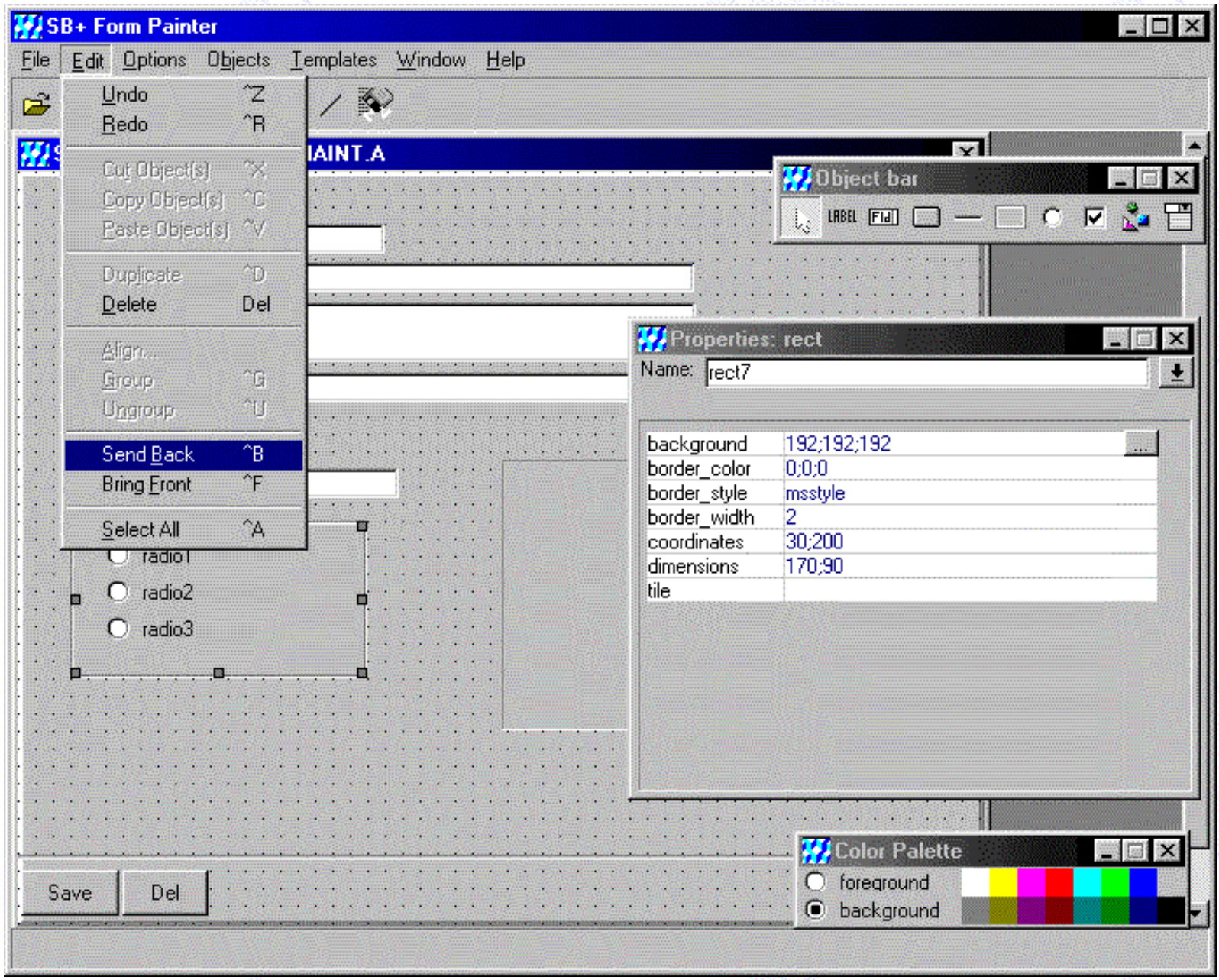
A rectangle can also be used as a graphic object for displaying a corporate logo or other static information. To create such a graphic, change the "tile" property for the rectangle to reflect the name of a graphic (bitmap, JPG, or GIF) that you would like displayed. Unlike an image object, the rectangle object can only display the image in tiled (repeated) form, and has no feature to scale the image to the size of the rectangle.

The appearance of the rectangle can also be modified through changes to a number of other properties:

- border_style: This property controls the appearance of the border. The border can appear raised above the GUI form surface, lowered into the GUI form surface, flat -- right on the GUI surface, or "msstyle", a style very popular with Microsoft products.
- border_width: If using a border_style other than "msstyle", this property controls how far the rectangle is raised from or lowered into the GUI form surface.

Layering Objects

When placing objects on top of other objects, which is most commonly associated with the placement of graphics, you can control the *layering* of the objects; that is, the order that objects lay on top of one another. On the Edit menu, there are two significant options for this, Send Back (which can be abbreviated with Ctrl-B) and Bring Front (which can be abbreviated Ctrl-F):



To use these options, select an object which is on top of another object and press Ctrl-B to send it to the back, thus exposing the underlying object. Or, click on an object which is partially covered by another, and press Ctrl-F to bring the object to the front, thereby moving the other object(s) to the back. In the screen above, I've sent the rectangle object to the back, thus exposing the radio button object which was previously hidden underneath.

Any number of layers can be managed this way, though notably there's rarely a call to use more than a couple of layers on any given form.

Creating a Report With Multiple Breaks

SB+ will allow up to 8 breaks to be defined for a report. This can be somewhat confusing, noting that SB+ only allows one break section to be defined for the whole report. Does this mean that each break must output the same text and fields?

Fortunately, no. To implement multiple breaks, you need only to define the break section as a series of smaller *break subsections*. Each subsection, then, will be used for a different break.

To see how this would appear in a report, let's look at the department report shown earlier, only this time let's sort and break by department, salesman within department, and states for each salesman.

```
Department Salesman State Amount Sold
xxxxx      xxxxxxxx xx     xxxxxxx.xx
                    xxxxxxx.xx

Total for State     xx     xxxxxxx.xx

         xxxxxxxx xx      xxxxxxx.xx
                    xxxxxxx.xx

Total for State     xx     xxxxxxx.xx

Total for Salesman xxxxxx  xxxxxxx.xx

xxxxx      xxxxxxxx xx      xxxxxxx.xx
                    xxxxxxx.xx
                    xxxxxxx.xx
                    xxxxxxx.xx
                    xxxxxxx.xx

Total for State     xx     xxxxxxx.xx

Total for Salesman xxxxxx  xxxxxxx.xx

Total for Department xxxxx xxxxxxx.xx
```

While this makes the report slightly more complicated to read, we have essentially defined only three break subsections: a department break, a salesman break, and a state break. The first salesman had activity in two states. In both states he had two sales. The second salesman had activity in only one state, but five sales in that state.

To implement this in a report definition, we first need to setup the break fields in the F6-Params screen. The break fields are entered in major-to-minor order, so for this example our break fields will be entered as:

```
DEPT SALESMAN.ID STATE
```

This tells the Report Writer that the most major break will be on department, and within department we want a break on different salesmen, and within each salesman we want a break on different states. Once this is defined, we can start building the break

section, one subsection at a time.

The first subsection to be included in the break section should be the most minor break. In this case, it is the state break. The second subsection to be included should be the salesman break. Of course, the third subsection should be the major break, i.e. the break by department.

But how big should each subsection be? We need to account for blank lines above and below each of the break lines, regardless of the break being processed.

It would appear at first glance that each break subsection should be 3 lines, noting how there is a blank line above and below each break line. However, this is not the case. The most minor break subsection (the state subsection) is 3 lines, but all of the other subsections are only 2 lines. Also, the last line of each subsection should be a blank line.

Any time a break is processed, SB+ looks to see if there are any other breaks that are more minor than the current break. Therefore, if we have a break by department, before SB+ processes that break it generates a break by salesman. Before SB+ processes the salesman break it generates a break by state. There are no breaks more minor than the state break so that break is processed normally.

When a state break occurs, the last line output will be a blank line, noting how we've defined the last line of each subsection this way. This will provide just the right amount of space between the break text itself and the next set of detail. If the state break occurs as a result of salesman break, the blank line output as part of the state subsection will provide the necessary space between the state break and the salesman break. Similarly, the blank line at the end of the salesman break will provide the right spacing between it and the department break. For this reason, we don't need a blank line at the beginning of the intermediate and major breaks -- the blank line output from the preceding break is sufficient.

Once all of the break subsections have been defined, the last thing to do is to tell SB+ which subsection to output for which break. We do this by installing *conditional fields* in the report definition.

A conditional field is a special type of Report Writer field. Rather than output some value, the conditional field tells the Report Writer to skip output of a certain number of lines if a particular condition is met.

In the case of the sample report, we'll need three conditional fields. Each conditional field will appear on the first line of a particular subsection, resulting in a conditional field on the first, fourth, and sixth lines of the break section.

Conditional fields may appear anywhere on the line in a break section. Nonetheless, many people prefer to place the conditional fields somewhere near the beginning of the line so that they are immediately evident when viewing the report definition screen.

To create a conditional field, move the cursor where you want the field to be placed and press F5-Field. When asked to enter a field name, enter "C.number", where number is any number you choose. (You may use alphabetic characters as well -- numbers are simply my convention.)

You'll note that even though you're creating a derived value field, SB+ doesn't ask you to define the type and length of the field. Because the field name starts with "C.", SB+ knows that the field is conditional and therefore won't be outputting anything. Therefore, what's the point of a field length and type?

Once you've pressed <cr> on the field name, press <cr> a few more times until a smaller window pops up. On this window you must enter an expression which will determine the number of lines to be skipped. For our sample report, let's assume the conditional for the state subsection is called C.1, the conditional for the salesman subsection is C.2, and the conditional for the department subsection is (you guessed it) C.3. With this in mind, the derived values for the three fields should appear as follows:

```
C.1   IF(@RV.BREAK.FLD=3,0,3)
C.2   IF(@RV.BREAK.FLD=2,0,2)
C.3   IF(@RV.BREAK.FLD=1,0,2)
```

For each of these fields, we need to ask SB+ which field we're generating a break for. This information is available in a report variable called @RV.BREAK.FLD. The number 1, 2, or 3 corresponds to the position of the particular field name in the Break Fields prompt on the F6-Params screen. Because the state field is break #3, we tell SB+ that when that break section is being processed, don't skip any lines in this subsection. If the state break is not being processed, however, we will skip three lines, thereby skipping completely over the state break subsection.

Also note that all of the breaks are (intentionally) mutually exclusive. In other words, when one break is processed, the other breaks are skipped.

It is very important to re-emphasize, however, that when SB+ sees that a break is needed, it processes all of the subordinate breaks first. Therefore, in this sample report if we have a break on the department field, the entire break section will be processed three times before the next detail is output (once for state, once for salesman, and a third time for the department break).

Creating a Report with a Single Break

For example, let's say we have a sales file with fields defined for a department, salesman ID, state, and total sale amount. Furthermore, let's say we want a report which looks something like the following, showing a total amount sold by department:

```
Department Salesman State Amount Sold
xxxxx      xxxxxxxx xx    xxxxxxx.xx
xxxxxxxx xx    xxxxxxx.xx
xxxxxxxx xx    xxxxxxx.xx
xxxxxxxx xx    xxxxxxx.xx

Total for department xxxxx xxxxxxx.xx

xxxxx      xxxxxxxx xx    xxxxxxx.xx
xxxxxxxx xx    xxxxxxx.xx

Total for department xxxxx xxxxxxx.xx
```

To implement this report, we first need to tell SB+ that we want a break on the department field. To do this, press F6-Params while in the Report Definitions tool, arrow down to the Break Fields prompt, and enter the field name for the department field. For the purpose of this discussion, let's say the field is called DEPT. Once this has been entered, press F2-Save to save the F6-Params information.

Next, we need to add a break section to the report. The break section, as mentioned before, is identified by lines starting with a "B". This section must follow the detail section (D lines) and precede the grand total section (G lines)

Looking at the sample report shown earlier, it looks like we're going to need 3 lines in this section. The first and third lines have nothing on them, and as such will output blank lines. The second line, however, is a bit more complicated.

The words "Total for department" are simply typed right onto the report definition. Following this is a field which is intended to show the department we're totaling. To create this field, press F5-Field on the break line where you want the field to go, and enter a field name of "B.*field*". This particular naming convention tells SB+ that this is a special type of field called a *break field*, and it

breaks on the field named *field*.

To illustrate why break fields are important, let's look at what happens when the report runs. For every record that is selected, a detail line is output. In order for SB+ to "see" that a particular break field has changed, it must read a record past the current record and compare the break fields between the two. Therefore, if we place a normal field in the break section we'll output a value from the next record, not the previous record, which is what we want. A break field, on the other hand, outputs a field from the previous record.

OK back to the naming of the field. Break fields are defined by values in two different fields in the F5-Field window. If the field type is "B", the derived value defines the name of the break field. This is how SB+ "knows" which field is to be extracted from the previous record. Fortunately, when we name a field this way, SB+ figures everything out perfectly, and we can simply press F2

Suppressing A Break Field

Sometimes there just isn't enough information on a report to print the whole break section. Particularly when there is only one record processed for a particular break, it can seem useless to output a break information with a total that matches the information on the detail line.

There are two ways of suppressing the breaks in this circumstance, as described following:

Using the Break Modifier "(D"

When defining the break fields, if you append an open parenthesis followed by D to the end of a break field, if only one record has been processed for that break, the break section will be ignored.

This works fine as long as you realize that SB+ will output NOTHING when the break occurs -- not even a courtesy blank line! This can make the report difficult to follow, often giving the illusion that certain records belong to the wrong break. For this reason, this is generally not recommended, especially on reports with multiple break subsections.

Using Conditional Fields to Skip the Break

Instead of the "(D" break modifier, create new conditional fields in the break subsection which skip certain lines if only one detail record has been processed. As an example, if we wanted to skip the state break if only one state had been output, but still output a blank line between the two state details, we could add the following additional field (and call it C.4, perhaps?) to line 2 of the break section:

```
C.4   IF(T.R.CNT = 1,2,0)
```

In other words, if only one detail has been processed in this break section, skip two lines. Otherwise, skip zero lines -- or more simply -- go ahead and print the lines.

It is very important that this field is on line 2 of the break section (line 2 of the state subsection), because it applies only to the state break, and we want the first blank line to be output regardless of the number of detail lines being printed.

Skipping like this can be done based on literally any condition, and with up to 8 break levels, can become very, very complicated. The important thing to keep in mind is always keep individual break subsections separate , if only mentally, and only skip lines that are in the subsection you're working in. In other words, if your subsection is 3 lines, don't skip 10.

Creating User-Selectable Summary/Detail Reports

While SB+ has a built-in "A"sk feature for allowing the user to select whether a report will print detail or totals only, it is of limited value because of the difficulty in creating a report that appears presentable in both summary and detail formats. This can be a problem for many end-user applications which rely on allowing the user this kind of flexibility. Fortunately, this problem can be easily overcome.

Using the SB+ feature as an example, you can easily write a quick paragraph which will prompt for summary or detail, and run one of two different reports based on the answer. Such a process might appear something like the following:

```
LOCAL ANS
*
ANS = 'N' ;* ...the default
INPUT ANS,1,'Totals Only (Y/N)'
*
IF @OTHER(18) # 14 THEN
  IF ANS = 'Y' THEN
    EXEC 'summary report process'
  END ELSE
    EXEC 'detail report process'
  END
END
```

In short, to overcome the limitation of having one report definition for both summary and detail formats, define two separate reports and call them based on some condition, such as in the example shown above.

This particular technique, though simple, demonstrates one of the typical paradigm shifts of SB+ programming: Though the end result of the program is to generate a report, the starting process is not a report process. In fact, the report processes are only pieces of the whole puzzle, not the main process. Similarly, though the user will receive one report from this program, there are actually two reports doing the work.

Incidentally, this paragraph can easily be expanded to include a **combination summary/detail report**, thereby allowing the "Totals Only" prompt to have three valid options: "Y" (summary report), "N" (detail report), and "B" (both)!

Generating Summary Percentages

There are many possibilities available when you use a Process After Read to build summary information in **common variables**. One of these interesting variations involves adding a column of calculated percentages to the summary as shown at the bottom of the report, such as is shown below:

```
********** Summary ***********
Salesman  Order Total  Percent
xxxxxxx   xxxxxxx.xx   xx.xx
xxxxxxx   xxxxxxx.xx   xx.xx
xxxxxxx   xxxxxxx.xx   xx.xx
xxxxxxx   xxxxxxx.xx   xx.xx
```

To add such a column, we first need to create the report from the previous example, with a Process After Read to build the totals into @PARMS(2). Next, we'll need to add a multivalued derived field to the report definition to show the percentages. The derived value of this field will be entered as @PARMS(2)<2> (based on the previous example), defined as numeric, length of 5 characters, and right justified.

At first glance, this probably doesn't make sense. Why would we want to output totals in this field? Shouldn't we be calculating the percent here? Oddly, no.

Instead, we'll use the Conversion prompt to do the calculation of the percentage, using the following **expression**:

```
(ICONV(@VALUE | S(@PARMS(2)<2>),'MR2')"MR2")
```

Remember, the conversion happens once for each value in the field, so this conversion will automatically be applied to each value in @PARMS(2)<2> separately.

This illustrates a very important distinction between the conversion and derived value for any particular field on the report. The derived value is calculated once -- the first time the field is encountered. In contrast, the conversion is applied to each value in the field separately, and therefore may be processed multiple times for a given field.